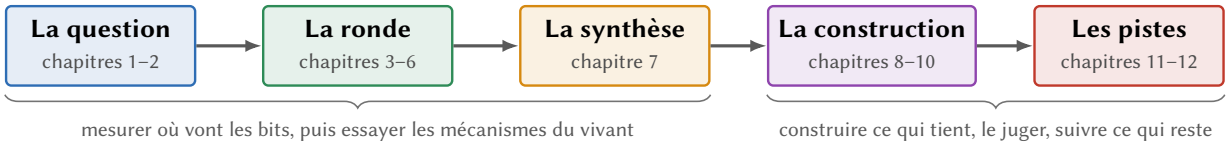
troncons

Tome III : le génome des poids

Où est le bruit, où est la structure ; le brin, la graine et la rotation ; la duplication de gènes, le codage prédictif, le mimétisme ; le codon synonyme dans le format ; la géométrie tropicale du routeur et les spectres des matrices

Mémoire de recherche — la quête, sa ronde et sa construction



Stone

Toronto, le 29 octobre 2026

Fait suite au Tome I, *les fondations mesurées*, et au Tome II, *le système*. Chaque figure se régénère depuis les données de mesure commitées dans le dépôt `troncons` ; chaque nombre cite le fichier dont il vient.

Résumé

Le Tome I de *troncons* avait démontré que, sur un portable ordinaire, servir un modèle bien plus gros que sa mémoire se joue en bits lus sur le disque par poids ; le Tome II avait construit le programme qui le fait, exactement. Il restait une frontière : 3,84 bits par poids d'expert sans perte, 2,64 bits pour la base avec perte du mode rapide. Stone a proposé de s'inspirer du vivant, qui ne stocke pas un organisme mais un génome d'où l'organisme se construit. Ce tome pose la question jugeable : existe-t-il un mécanisme inspiré du vivant qui fasse lire moins de bits par poids que cette frontière, à une fidélité jugée par la KL sur du texte contre la référence exacte, et qui se transcrive plus vite que le disque ne lit ce qu'il remplace ? Le génome y est une analogie qui oriente la recherche, jamais un argument.

Le tome donne la ronde de recherche, la construction et les pistes. La ronde : la question dans les termes de la machine (le bloc MXFP4 au bit près, une borne distorsion-débit dans le domaine des poids, douze familles « génomiques » de la littérature) ; l'éclaireur, qui a trouvé où sont le bruit et la structure ; la recette lue par codons et sa graine ; la duplication de gènes ; le codage prédictif et l'élagage ; six pistes de mimétisme, du cache à deux échelles au recuit ; et la synthèse, qui place tous les mécanismes sur une frontière commune et montre que l'erreur d'un expert ne prédit pas la KL. La construction : le codon synonyme du zéro dans un format `.trc` v1.2 exact, le génome synonyme servi par un mode rapide du moteur, et un juge qui n'a fait ni l'un ni l'autre. Les pistes : la géométrie tropicale du routeur et les spectres des matrices. Chaque chapitre pose ses dérivations en entier et finit par ce qui pourrait être faux.

La première réponse est sévère : une fois l'échelle de leur bloc connue, les poids de gpt-oss se comportent comme du bruit, et la graine (3,06 bits à l'erreur de la base), les motifs partagés (2,90), la recette sans perte (4,05 à 4,37 bits contre 3,84) et la parenté entre experts (0,003 4 bit, dont 0,001 6 au hasard) tombent tous. Ce qui gagne, trois fois, est le synonyme, qui change le code sans changer ce que le calcul lit : le signe d'un zéro (entropie 0,999 999 996 7 bit, jamais lu par le calcul) se fond sans perte, 0,127 91 et 0,125 62 bit par poids d'expert de moins, 1,80 Go de moins à lire sur le 120b, des logits identiques au bit ; le contexte de la colonne rend −0,142 et −0,123 bit à la couche 0 ; la mutation synonyme, qui place l'erreur là où les entrées ne regardent pas, sert les 768 experts du 20b à 2,67 bits au lieu de 3,86, pour une KL de 0,131 8 contre 0,264 0 à la base fixe. Sur ce disque, le format v1.2 fait gagner ×1,44 au jalon J1 du 120b pour 3,3 % d'octets de moins : le gain vient du motif de ses requêtes de lecture, propre à ce disque. Plusieurs chiffres publiés en route sont barrés à leur place : « c'est l'état des fichiers, pas le format » ; deux gains du mode rapide, ×1,24 et ×2,00, qui comparaient un fichier neuf à un ancien et deviennent ×0,94 et ×1,28 contre le v1.2 ; « S^* a 33 % de KL de plus », égal à la base sur dix fenêtres (+0,000 4 ± 0,001 9). La garde tropicale tombe (57,1 % des bascules pour 10 % des cellules, quand 80 % en coûteraient 26,6 %), la loi de puissance des spectres ne s'écrit pour aucune matrice sur 48 au critère principal, et les six pistes du mimétisme restent ouvertes (le préchargement à confiance calibrée plutôt du côté de tombe), ou à la limite pour le routage conditionné par le cache.

Le juge du tome est la divergence de Kullback-Leibler de la loi du jeton suivant contre la référence exacte en double précision, moyennée sur les 2 550 positions des dix fenêtres d'évaluation, chaque comparaison appariée position par position. Un juge indépendant de la construction, avec son propre décodeur écrit depuis la seule description du format, a retrouvé au bit, au signe des zéros près, 96 experts tirés, des logits identiques au bit sur dix fenêtres que personne n'avait vues (0 différence sur

5 120 positions, pour chacun des deux modèles), et la mutation synonyme meilleure que la frontière hors des fenêtres de mise au point ($-0{,}156\,5 \pm 0{,}033\,4$, 4,7 écarts-types) ; la variante retenue, $\lambda = 0$, l'est à 2,09 écarts-types sur les fenêtres d'évaluation et 3,6 hors d'elles. Chaque verdict a son seuil déclaré avant, son témoin qui sait tomber, et un juge qui n'a pas fait le travail.

Restent : faire du v1.2 le format par défaut des fichiers de la machine (décision de Stone) et isoler la variable du motif de lecture qui fait la vitesse de ce disque ; la queue de la KL, faite des bascules du routeur, que rien ne vise encore ; la mutation synonyme du 120b, qui attend sa référence f64 ; et un autre corpus que wiki.test. La réponse à l'intuition de Stone est double, et mesurée : l'ADN comme recette ne marche pas sur ces poids ; l'ADN comme code redondant, où un synonyme change la séquence sans changer le phénotype, est exactement ce qui gagne. Chaque figure se tire de données commitées par un script commité.

Abstract

Volume I of *troncons* showed that, on an ordinary laptop, serving a model much larger than its memory comes down to the bits read from disk per weight; Volume II built the program that does it, exactly. One frontier remained: 3.84 bits per expert weight lossless, 2.64 bits for the lossy base of the fast mode. Stone proposed to learn from life, which does not store an organism but a genome from which the organism is built. This volume asks the question in a form that can be judged: is there a mechanism inspired by life that reads fewer bits per weight than this frontier, at a fidelity judged by the KL on text against the exact reference, and that transcribes faster than the disk reads what it replaces? The genome is an analogy that guides the search here, never an argument.

The volume gives the research round, the construction and the leads. The round: the question in the terms of the machine (the MXFP4 block to the bit, a rate–distortion bound in the domain of the weights, twelve "genomic" families from the literature); the scout, which found where the noise and the structure are; the recipe read through codons and its seed; gene duplication; predictive coding and pruning; six leads of mimicry, from a two-scale cache to annealing; and the synthesis, which places every mechanism on a common frontier and shows that the error of an expert does not predict the KL. The construction: the zero-synonym codon in an exact `.trc` format v1.2, the synonymous genome served by a fast mode of the engine, and a judge that did neither. The leads: the tropical geometry of the router, and the spectra of the matrices. Each chapter sets out its derivations in full and ends with what could be wrong.

The first answer is harsh: once the scale of their block is known, the weights of gpt-oss behave like noise, and the seed (3.06 bits at the error of the base), the shared patterns (2.90), the lossless recipe (4.05 to 4.37 bits against 3.84) and the kinship between experts (0.0034 bit, of which 0.0016 by chance) all fall. What wins, three times, is the synonym, which changes the code without changing what the computation reads: the sign of a zero (entropy 0.9999999967 bit, never read by the computation) is folded losslessly, 0.12791 and 0.12562 bit per expert weight less, 1.80 GB less to read on the 120b, bit-identical logits; the column context gives −0.142 and −0.123 bit at layer 0; the synonymous mutation, which puts the error where the inputs do not look, serves the 768 experts of the 20b at 2.67 bits instead of 3.86, for a KL of 0.1318 against 0.2640 for the fixed base. On this disk, the format v1.2 gives the J1 milestone of the 120b a ×1.44 for 3.3 % fewer bytes: the gain comes from the pattern of its read requests, specific to this disk. Several numbers published along the way are struck out where they stood: "it is the state of the files, not the format"; two gains of the fast mode, ×1.24 and ×2.00, which compared a fresh file with an old one and become ×0.94 and ×1.28 against v1.2; "S^* has 33 % more KL", equal to the base on ten windows (+0.0004 ± 0.0019). The tropical guard falls (57.1 % of the flips for 10 % of the cells, where 80 % would cost 26.6 %), the power law of the spectra holds for no matrix out of 48 under the main criterion, and the six leads of mimicry remain open (the prefetch with calibrated confidence rather on the side of falling), or borderline for the cache-conditioned routing.

The judge of the volume is the Kullback–Leibler divergence of the next-token law against the exact double-precision reference, averaged over the 2550 positions of the ten evaluation windows, each comparison paired position by position. An independent judge of the construction, with its own decoder written from the description of the format alone, recovered 96 drawn experts to the bit, up to the sign of the zeros, bit-identical logits on ten windows nobody had seen (0 differences in 5120

positions, for each of the two models), and the synonymous mutation better than the frontier outside the tuning windows (-0.1565 ± 0.0334, 4.7 standard deviations); the retained variant, $\lambda = 0$, wins by 2.09 standard deviations on the evaluation windows and by 3.6 outside them. Every verdict has its threshold declared beforehand, its control that can fail, and a judge that did not do the work.

What remains: making v1.2 the default format of the machine's files (Stone's decision) and isolating the variable of the read pattern that makes the speed of this disk; the tail of the KL, made of the flips of the router, which nothing targets yet; the synonymous mutation of the 120b, which waits for its f64 reference; and a corpus other than wiki.test. The answer to Stone's intuition is twofold, and measured: DNA as a recipe does not work on these weights; DNA as a redundant code, where a synonym changes the sequence without changing the phenotype, is exactly what wins. Every figure is drawn from committed data by a committed script.

Table des matières

Résumé iii

Abstract v

Table des figures xv

Liste des tableaux xvii

Liste des listings xxi

Notations et conventions xxiii

1 Introduction 1
1.1 La question, posée par Stone et par la machine . . . 2
1.2 Ce que « génome » veut dire ici . . . 2
1.3 Le chemin du tome . . . 3
1.4 Ce que ce tome apporte . . . 4
1.5 Comment lire ce tome . . . 4

Première partie La question et la ronde 7

2 La question du génome : le toit, les bits lus par poids, la frontière 9
2.1 Le toit : ce qu'un jeton doit lire . . . 10
2.2 Le bloc MXFP4 au bit près . . . 10
2.3 La frontière, et la borne distorsion-débit . . . 12
2.3.1 Ce que coûte un poids aujourd'hui . . . 12
2.3.2 La borne d'un codeur sans mémoire, dans le domaine des poids . . . 12
2.4 L'état de l'art, en douze familles . . . 15
2.4.1 Deux filtres . . . 15
2.4.2 Les familles . . . 16
2.5 Ce qui n'a jamais été tenté sur un modèle né en 4 bits . . . 16
2.6 Ce que la lecture prédit . . . 18

3 L'éclaireur : où est le bruit, où est la structure 21
3.1 Les instruments et leurs témoins . . . 22
3.2 Où est le bruit, où est la structure . . . 23
3.2.1 Le signe des zéros : du bruit pur . . . 23
3.2.2 L'énergie est concentrée . . . 23
3.2.3 La borne d'un codeur sans mémoire . . . 23
3.3 Mécanisme 1 : le codon synonyme du zéro . . . 25
3.3.1 Le mécanisme . . . 25

3.3.2 Pourquoi les produits ne changent pas . . . 25
3.3.3 Les mesures . . . 25
3.3.4 Le même codon dans la base . . . 26
3.4 Mécanisme 2 : la mutation synonyme . . . 27
3.4.1 Le vivant, et ce qu'on en transpose . . . 27
3.4.2 La dérivation . . . 27
3.4.3 Le protocole . . . 28
3.4.4 Le premier échec, et son pourquoi : le sur-apprentissage . . . 28
3.4.5 Les résultats . . . 29
3.5 Le jugement de bout en bout : la KL contre la référence f64 . . . 31
3.6 Le verdict . . . 33

4 Le génome des poids : la recette, la graine, S^* 35
4.1 La question et son juge . . . 36
4.2 La graine : la recette littérale, et pourquoi elle ne peut pas suffire . . . 36
4.2.1 Le mécanisme . . . 36
4.2.2 Ce que la théorie permet, avant toute mesure . . . 36
4.2.3 La mesure . . . 37
4.3 La graine qui pousse : le brin lu par codons . . . 38
4.3.1 L'idée . . . 38
4.3.2 Le meilleur brin : Viterbi, et pourquoi il est exact . . . 38
4.3.3 La rotation : répartir l'information de chaque poids . . . 39
4.3.4 La répartition des bits entre rangées . . . 41
4.4 La comptabilité, au bit près . . . 42
4.5 Les résultats sur les poids réels . . . 42
4.6 Où est le bruit, où est la structure . . . 44
4.7 La recette et la découpe base + complément . . . 46
4.8 La sortie des experts : la fidélité, et S^* . . . 46
4.9 La KL contre la référence : S^* est égal à la base . . . 48
4.10 La transcription . . . 49
4.11 Ce qui reste . . . 50

5 La duplication de gènes 53
5.1 La question, posée en bits . . . 53
5.2 La symétrie qui rend la parenté invisible . . . 54
5.3 La parenté brute : écran, appariement, famille . . . 54
5.3.1 Deux lemmes pour lire un cosinus . . . 54
5.3.2 L'écran : une borne haute, sans bijection . . . 55
5.3.3 L'appariement un à un : la vraie parenté d'expert à expert . . . 55
5.3.4 Le meilleur parent de chaque enfant . . . 55
5.3.5 La famille : un sous-espace commun . . . 57
5.4 Le coût exact d'un expert codé comme mutation . . . 57
5.4.1 Les énoncés . . . 57
5.4.2 Les mesures . . . 58
5.5 La mutation tronquée et la sortie de l'expert . . . 60
5.6 L'arbre le plus économe . . . 61
5.7 Les paralogues : servir un expert absent par un autre . . . 63

5.8 Pourquoi l'idée échoue, et où chercher . . . 64

6 Codage prédictif et élagage synaptique 67
6.1 Les codons synonymes du E2M1 . . . 68
6.2 Le codage prédictif sans perte : ce que les voisins disent d'un code . . . 69
6.3 L'élagage et la quantification prédictive bornée : le cadre débit-distorsion 70
6.4 L'élagage dépendant de l'activité . . . 72
6.5 Les entrées réelles, et l'erreur à la sortie des experts . . . 72
6.5.1 L'élagage hebbien : ce qui tient, ce qui sur-ajuste . . . 73
6.6 La KL contre la référence f64 . . . 74
6.7 Ce qui reste . . . 75

7 Le mimétisme : six pistes du monde vers le pipeline 77
7.1 Le tri : quarante-cinq candidats, six pistes . . . 78
7.2 P1 : la mémoire du cache à deux échelles . . . 80
7.2.1 Le score à décroissance exponentielle, estimateur de taux . . . 80
7.2.2 Le protocole et le verdict mécanique . . . 81
7.2.3 Ce que les traces de génération étaient vraiment . . . 82
7.2.4 Les fenêtres froides : la fréquence, pas la demi-vie . . . 84
7.3 P2 : le cache exclusif de la VRAM vers la RAM . . . 85
7.3.1 La pile LRU et le théorème . . . 85
7.3.2 Le coût de la descente . . . 88
7.3.3 La mesure, le verdict et ce que le juge corrige . . . 88
7.4 P3 : le routage conditionné par le cache, borné par la marge . . . 91
7.4.1 La géométrie : une paroi décalée, une bande . . . 91
7.4.2 Le critère de décision en une grandeur . . . 92
7.4.3 L'étage A : la boucle ouverte . . . 93
7.4.4 L'étage B : la KL excédentaire mesurée . . . 94
7.4.5 Le routeur biaisé en ligne, dans le moteur . . . 95
7.5 P4 : le préchargement à confiance calibrée . . . 97
7.5.1 La garantie, posée . . . 97
7.5.2 Le simulateur et le gain . . . 97
7.5.3 La mesure et le verdict . . . 98
7.6 P5 : le choix des symboles synonymes comme verre de spin . . . 99
7.6.1 Le problème posé . . . 99
7.6.2 Les témoins : le verre est réel à petite dimension . . . 100
7.6.3 L'étape 1 : le gain se mesure et ne se transfère pas . . . 102
7.6.4 Le mécanisme annoncé, et ce que le juge en dit . . . 104
7.7 P6 : la fidélité décidée par appel . . . 105
7.7.1 Le modèle et ses grandeurs . . . 105
7.7.2 La mesure, le verdict et ce que le juge corrige . . . 106
7.8 La carte du monde vers le pipeline . . . 108
7.9 Ce qui pourrait être faux . . . 109

8 La synthèse de la ronde : une frontière commune, deux parts de la KL 111
8.1 Un seul juge, et pourquoi des rapports . . . 112
8.2 La frontière commune . . . 113

8.3 Trois propriétés mesurées des poids . . . 116
8.3.1 Une fois l'échelle du bloc connue, les poids sont du bruit . . . 116
8.3.2 Quelques colonnes géantes portent la structure . . . 116
8.3.3 L'erreur de sortie d'un expert ne prédit pas la KL . . . 117
8.4 Deux parts de la KL . . . 120
8.4.1 Ce que disent les quantiles, et le moteur . . . 120
8.4.2 La part lisse : la KL au second ordre . . . 121
8.4.3 La part de bascule . . . 122
8.4.4 Le routeur comme géométrie tropicale . . . 122
8.5 Mécanisme par mécanisme . . . 126
8.5.1 Pourquoi S^* ne gagne pas . . . 126
8.5.2 La leçon, en une phrase par niveau . . . 126
8.6 Le contexte d'un symbole de colonne . . . 126
8.7 Le cahier de construction . . . 129
8.7.1 Lot A : ce qu'il porte, et la preuve qu'il ne change aucun bit . . . 129
8.7.2 Lot B : le génome synonyme, et la mesure qui manquait d'abord . . . 131
8.7.3 Les pistes que les mesures désignent déjà . . . 133
8.8 Ce qui pourrait être faux . . . 133

Deuxième partie La construction 135

9 Un codon synonyme dans le format (lot A, v1.2) 137
9.1 Seize codes pour quinze valeurs . . . 138
9.2 Le gain exact du fondu . . . 138
9.3 Qui lit le code 8, et pourquoi les bits ne changent pas . . . 141
9.4 Le format `.trc` v1.2 au bit près . . . 144
9.4.1 Pourquoi la colonne reste dehors . . . 146
9.5 Ce qui a été écrit, et ses témoins . . . 148
9.6 Les deux fichiers recodés . . . 149
9.6.1 Le codon par le décodeur du juge . . . 150
9.7 Le jalon J1 refait sur le 120b . . . 152
9.7.1 D'où vient le gain : le motif des requêtes de lecture . . . 154
9.8 Ce qui reste . . . 155
9.9 Ce qui pourrait être faux . . . 157

10 Le génome synonyme, mode rapide (lot B) 159
10.1 Ce qui est construit . . . 160
10.2 Le mécanisme . . . 160
10.2.1 L'erreur de sortie et la matrice des entrées . . . 160
10.2.2 Le choix des symboles . . . 161
10.2.3 La transformation dans la base propre de H . . . 162
10.2.4 L'erreur dans les directions des entrées . . . 163
10.3 Le mode rapide ne touche pas le mode exact . . . 167
10.4 Le compagnon `<nom>.rapide.trc`, au bit près . . . 167
10.5 Les mesures de la qualité . . . 168
10.5.1 Le protocole et ses témoins . . . 168
10.5.2 Le repère de l'outil de référence . . . 169

10.5.3 La KL de bout en bout, les 768 experts substitués . . . 169
10.5.4 Le critère B2, et le départage des deux λ . . . 170
10.5.5 La queue de la KL . . . 171
10.5.6 La perplexité ne juge pas la fidélité . . . 174
10.5.7 Le débit des symboles . . . 174
10.5.8 La frontière des variantes . . . 174
10.6 Le mode rapide dans le moteur . . . 176
10.7 La vitesse, et ce que le juge a barré . . . 176
10.7.1 Les séances du lot . . . 176
10.7.2 Une identité, et ce qu'elle sépare . . . 177
10.7.3 Ce que le juge a mesuré . . . 178
10.8 Le 120b . . . 179
10.9 Ce qui reste . . . 179
10.10 Ce qui pourrait être faux . . . 180

11 Le juge de la construction 181
11.1 Ce que le juge a refait, et par quel chemin . . . 182
11.2 Le `.trc` v1.2 par le décodeur du juge . . . 183
11.2.1 Le décodage rANS vectorisé est le décodage normatif . . . 183
11.2.2 J-T0 : les quatre fichiers entiers . . . 183
11.2.3 J-T1 : 96 experts tirés, décodés par le juge . . . 184
11.2.4 Ce que le témoin v1.0 → v1.2 prouve . . . 184
11.3 Les fenêtres neuves, et les experts qu'elles appellent . . . 185
11.4 J-C1 : les logits au bit sur les fenêtres neuves . . . 185
11.5 J-K : la KL du génome synonyme hors de R1 . . . 187
11.5.1 La KL du juge, et son témoin . . . 187
11.5.2 Les nombres . . . 187
11.5.3 L'incertitude d'une KL moyenne : par positions ou par fenêtres . . . 188
11.5.4 Refaire un résultat à deux écarts-types . . . 190
11.6 J-V : la vitesse, et ce qu'elle sépare . . . 190
11.6.1 Le piège . . . 190
11.6.2 La séance à trois fichiers . . . 190
11.6.3 Le 120b . . . 191
11.6.4 Le 20b . . . 192
11.6.5 Ce qui fait l'étage disque : la réserve R1 . . . 192
11.7 La lecture : ce qui est faux, trop beau, ou mal attribué . . . 193
11.8 Ce qui tient selon le juge, et ce qui est barré . . . 193
11.9 Ce que la méthode du juge apprend . . . 194
11.10 Ce qui reste . . . 195
11.11 Ce qui pourrait être faux . . . 195

Troisième partie Les pistes et le bilan 197

12 La géométrie tropicale du routeur 199
12.1 La question et le seuil déclaré . . . 199
12.2 Le routeur comme polynôme tropical . . . 200
12.3 Les données et les témoins . . . 201

12.4 Le verdict : la courbe de la garde . . . 202
12.5 Pourquoi elle tombe . . . 203
12.5.1 La loi de la marge : presque exponentielle . . . 203
12.5.2 La loi de bascule . . . 206
12.5.3 La propagation : une bascule en appelle d'autres . . . 206
12.5.4 Le nombre de garde : une forme close . . . 207
12.5.5 L'extrapolation au mode rapide (modèle) . . . 209
12.6 Après le verdict : deux explorations . . . 209
12.6.1 La garde des racines . . . 209
12.6.2 La mémoire du conflit : une garde à indices multiples . . . 210
12.7 Ce qui pourrait être faux . . . 212

13 Les spectres des matrices et la fractale 213
13.1 La question et la grandeur . . . 214
13.2 Le témoin du hasard : Marchenko-Pastur . . . 214
13.3 L'instrument : Clauset, Shalizi et Newman, pas à pas . . . 214
13.3.1 Le seuil déclaré, et pourquoi il a été amendé avant la lecture . . . 215
13.3.2 Le critère déclaré ne peut jamais dire oui : le théorème du juge . . . 216
13.3.3 Les témoins, la cassure, le second chemin . . . 216
13.4 Le verdict : pas une loi de puissance . . . 217
13.4.1 Les exposants, par rôle et par couche . . . 219
13.5 Le levier et le plafond : ce qui n'est pas jugé . . . 221
13.6 Ce qui pourrait être faux . . . 222

14 Discussion 225
14.1 Ce que le tome établit, ensemble . . . 225
14.2 Les chiffres barrés . . . 226
14.3 La leçon de la quête . . . 226
14.4 Ce qui pourrait être faux, à l'échelle du tome . . . 227
14.5 Travaux futurs . . . 228

A Protocoles : seuils déclarés et témoins 231
A.1 Ce qui est hérité . . . 231
A.2 Les seuils déclarés, étape par étape . . . 231
A.3 Les témoins et leurs cassures . . . 234
A.4 Les amendements . . . 235
A.5 Les protocoles des juges . . . 236
A.6 Ce qui pourrait être faux . . . 236

B Outils 237
B.1 La chaîne du Tome III . . . 237
B.2 Les outils commités . . . 238
B.3 Chaque figure et les fichiers qu'elle lit . . . 255
B.4 Les licences . . . 258

C Journaux 259
C.1 Les dossiers de journaux . . . 259
C.2 Les séances qui portent un chiffre de vitesse . . . 261

C.3 Les passes coupées, jetées, ou non concluantes 261

Bibliographie **263**

Table des figures

2.1 Le bloc MXFP4 au bit près . . . 11
2.2 La base frontière contre la borne distorsion-débit . . . 14

3.1 Le signe des zéros, tenseur par tenseur . . . 24

4.1 Le brin lu par codons, sur une gaussienne vraie . . . 40
4.2 Débit-distorsion du brin, tenseur par tenseur . . . 43

5.1 Le score de parenté de chaque parent candidat . . . 56
5.2 L'arbre le plus économe des experts de la couche 23 . . . 62

7.1 P1 : succès par bloc des deux générations du 120b . . . 83
7.2 P1 : fenêtres froides, succès du LRU contre la politique . . . 85
7.3 P1 : l'espace latent des blocs d'usages d'experts . . . 86
7.4 P2 : les distances de pile et la bande $(V, V + R]$ . . . 87
7.5 P2 : part disque contre la taille de la RAM . . . 90
7.6 P3 : la paroi du routeur décalée de β . . . 92
7.7 P3, étage A : défauts évités contre cellules changées . . . 94
7.8 P4 : précision, couverture et gain simulé . . . 99
7.9 P5 : coupe d'un paysage à 5^{10} états . . . 101
7.10 P5 : gains par instance et amortissement . . . 103
7.11 P5 : où l'erreur est déplacée . . . 104
7.12 P6 : le plan octets–énergie et r selon θ . . . 107

8.1 La frontière commune de la ronde . . . 114
8.2 Une colonne géante aspire l'erreur du brin . . . 118
8.3 L'erreur de sortie d'un expert ne prédit pas la KL . . . 119
8.4 Deux parts de la KL, position par position . . . 123
8.5 L'espace latent des 6 132 états de la couche 23 de gpt-oss-20b . . . 125

9.1 La roue des seize codons E2M1 . . . 139
9.2 Le théorème du gain, tenseur par tenseur . . . 140
9.3 Les 256 octets avant et après le fondu . . . 141
9.4 Pourquoi la GEMV rend les mêmes bits . . . 143
9.5 Le format `.trc` v1.2 au bit près . . . 145
9.6 Pourquoi la colonne reste hors de la v1.2 . . . 147
9.7 Le codon par le décodeur du juge . . . 151
9.8 Ce que le jalon J1 mesure . . . 153
9.9 D'où vient le gain du J1 . . . 156

10.1 La mutation synonyme dans la base propre de H . . . 164
10.2 Une rangée de 32 poids avant et après la mutation synonyme . . . 165

10.3 L'erreur de sortie d'un expert dans les directions de ses entrées, frontière et synonyme 166
10.4 Le critère B2, fenêtre par fenêtre 172
10.5 La queue de la KL 173
10.6 La frontière des variantes, 768 experts 175

11.1 Les experts vus 186
11.2 La difficulté des fenêtres 189

12.1 Les polyèdres réels du routeur de la couche 12 201
12.2 La courbe de la garde tropicale 203
12.3 La garde par couche 204
12.4 La loi de la marge 205
12.5 La loi de bascule 206
12.6 Le théorème du nombre de garde 208
12.7 L'extrapolation de la garde (modèle) 210

13.1 La queue des spectres de la couche 12 218
13.2 Les exposants par matrice 220

Liste des tableaux

1.1 L'analogie du génome, et ce qu'elle désigne ici . . . 3

2.1 La loi des quinze valeurs MXFP4 . . . 12
2.2 Quatre points de la borne distorsion-débit . . . 15
2.3 Les douze familles de l'état de l'art . . . 17
2.4 La case vide . . . 18
2.5 Le pronostic de la lecture . . . 19

3.1 Les instruments de l'éclaireur et leurs témoins . . . 22
3.2 Le signe des zéros . . . 23
3.3 L'énergie des poids par exposant de bloc . . . 24
3.4 Le codon synonyme du zéro, mesuré . . . 26
3.5 Le diagnostic du sur-apprentissage . . . 29
3.6 La mutation synonyme sans contrainte d'entropie . . . 30
3.7 La mutation synonyme sous contrainte d'entropie . . . 30
3.8 La KL de bout en bout, base contre mutation synonyme . . . 32
3.9 Le verdict de l'éclaireur . . . 33

4.1 La graine prédite par le lemme 1 . . . 37
4.2 La graine mesurée contre la théorie . . . 38
4.3 La comptabilité de la recette . . . 42
4.4 Le verdict W, tenseur par tenseur . . . 44
4.5 La structure des matrices : réel contre permuté . . . 45
4.6 La fidélité de sortie du brin . . . 47
4.7 S^* à la sortie des experts . . . 48
4.8 S^* contre la base, la KL sur dix fenêtres . . . 49
4.9 Le coût de transcription du brin . . . 49

5.1 L'appariement hongrois, couche 12 . . . 56
5.2 Le coût exact d'un expert codé comme mutation . . . 59
5.3 Les verdicts de la duplication . . . 59
5.4 La sortie des enfants, sur des entrées réelles . . . 60
5.5 Les paralogues, par profondeur . . . 63

6.1 Le codon synonyme du zéro au codec réel, sur les deux modèles . . . 68
6.2 Ce que les voisins disent d'un code . . . 69
6.3 Le cadre débit-distorsion, en erreur de poids . . . 71
6.4 L'élagage dépendant de l'activité, erreur prédite . . . 72
6.5 L'erreur à la sortie des experts . . . 73
6.6 La KL des plans de quantification et d'élagage . . . 74
6.7 Les comparaisons appariées des plans . . . 75

7.1 Le dédoublonnage des candidats . . . 79
7.2 P1 : le verdict au seuil déclaré . . . 82
7.3 P1 : la réponse réelle contre les jetons d'après la fin . . . 83
7.4 P1 : les fenêtres froides de 256 jetons . . . 84
7.5 P2 : le verdict . . . 89
7.6 P2 : temps de service des défauts par jeton . . . 89
7.7 P3, étage A : défauts évités et cellules changées . . . 93
7.8 P3, étage B à $N = 146$: les désaccords de premier choix . . . 95
7.9 P3 : le routeur biaisé en ligne . . . 96
7.10 P3 en ligne : la dérive par couche . . . 96
7.11 P4 : la piste et son juge . . . 98
7.12 P5 : les témoins . . . 101
7.13 P5 : gains de la descente cyclique, piste et juge . . . 102
7.14 P6 : le décodage du 120b . . . 106
7.15 La carte du monde vers le pipeline . . . 108
7.16 Les six pistes devant leurs juges . . . 109

8.1 Les protocoles de KL de la ronde . . . 112
8.2 La frontière commune, points jugés à la KL . . . 115
8.3 Propriété (a) : le bruit sachant l'échelle . . . 116
8.4 Propriété (b) : les colonnes géantes . . . 117
8.5 Propriété (c) : erreur d'expert contre KL . . . 120
8.6 Les quantiles de la KL par position . . . 120
8.7 Chaque mécanisme et la propriété qui l'a décidé . . . 127
8.8 Ce que le lot A devait rendre . . . 129

9.1 Les lecteurs du code 8 . . . 142
9.2 Ce qui change de la v1.0 à la v1.2 . . . 146
9.3 Les fichiers du lot A . . . 148
9.4 Ce que le fondu rend sur le disque . . . 150
9.5 A-C1 : les logits au bit . . . 152
9.6 Le J1 du 120b, séance s2 . . . 153
9.7 Le motif croisé . . . 154

10.1 Les pièces du lot B et leurs témoins . . . 161
10.2 Le compagnon rapide, champ par champ . . . 168
10.3 La KL de bout en bout, 768 experts substitués . . . 169
10.4 Les différences appariées, fenêtre par fenêtre . . . 171
10.5 Le mode rapide servi par le moteur . . . 176

11.1 Ce que le juge a refait, et par quel chemin . . . 182
11.2 J-T0 : les quatre fichiers, par le lecteur du juge . . . 184
11.3 J-C1 : les logits au bit sur les fenêtres neuves . . . 187
11.4 J-K : la KL sur les fenêtres neuves . . . 187
11.5 J-V : la séance à trois fichiers du 120b . . . 191
11.6 J-V : la séance à trois fichiers du 20b . . . 192
11.7 Ce qui tient selon le juge de la construction . . . 194

12.1 La garde tropicale : les témoins . . . 202
12.2 La courbe de la garde . . . 202
12.3 La garde par couche . . . 204
12.4 La propagation des bascules . . . 207
12.5 L'extrapolation de la garde . . . 209
12.6 La garde à indices multiples . . . 211

13.1 La puissance des deux critères . . . 215
13.2 Les témoins de l'instrument . . . 217
13.3 Le verdict par famille . . . 217
13.4 Les exposants par rôle et par couche . . . 219

14.1 Les verdicts du Tome III . . . 225
14.2 Les chiffres et les phrases barrés du Tome III . . . 227

A.1 Les critères qui décident, et leurs verdicts . . . 232
A.2 Les seuils déclarés du Tome III, datés par git . . . 233
A.3 Les témoins et leurs cassures . . . 234

B.1 Les outils commités du Tome III . . . 238
B.2 Chaque figure et les fichiers qu'elle lit . . . 255
B.3 Les licences des dépendances du Tome III . . . 258

C.1 Les dossiers de journaux du Tome III . . . 259
C.2 Les séances de vitesse du Tome III . . . 261

Liste des listings

9.1 Le fondu $-0 \to +0$ dans le code 148

Notations et conventions

Les conventions sont celles des Tomes I et II. Les unités sont décimales : 1 Go = 10^9 octets ; les multiples binaires s'écrivent Kio, Mio, Gio. Chaque nombre porte sa provenance : [mes.] mesuré ici, sur la machine de Stone, par un outil et un journal commités ; [calc.] calculé à partir de nombres nommés ; [litt.] littérature (chiffre d'éditeur, non reproduit). Une phrase qu'une mesure plus récente a démentie est barrée à sa place (~~ainsi~~), avec ce qui l'a remplacée. Un fichier du dépôt cité comme source d'un nombre s'écrit `mesures/genome/` ; son chemin part de la racine du dépôt `troncons`. Les figures portent dans l'inventaire des figures un numéro III-*x*.*y* (`memoire/tome3/INVENTAIRE-figures.md`) ; une figure tracée hors du style commun le dit dans sa légende.

Les noms du travail

Nom	Signification
troncons	le programme et le projet ; le Tome II décrit le programme
témoin, cassure	le témoin d'une mesure est un cas dont la réponse est connue d'avance ; sa cassure force sa condition à faux, et l'instrument doit alors tomber ; tout témoin neuf est cassé une fois avant qu'on s'y fie (section 3.1)
20b, 120b	gpt-oss-20b (24 couches de 32 experts : 768 experts) et gpt-oss-120b (36 couches de 128 experts : 4 608 experts), les deux modèles du tome, aux poids d'experts en MXFP4
MXFP4, E2M1, E8M0	le format en blocs des poids d'experts : 32 codes E2M1 (4 bits : signe, deux bits d'exposant, un de mantisse ; seize codes pour quinze valeurs, ±0 étant égaux) et une échelle commune E8M0 (un octet, un exposant seul) : 17 octets par bloc (chapitre 2)
KL	le juge du tome : divergence de Kullback-Leibler de la loi du jeton suivant du modèle modifié contre la référence f64, $\mathrm{KL}_t = \sum_i p_{t,i} \log(p_{t,i}/q_{t,i})$ en nats à la position t (seuls les $\log p_{t,i} > -16$ sont sommés), moyennée sur les positions notées (section 3.5, définition 8.1)
référence f64	`trc-reference`, le calcul exact de gpt-oss en double précision (Tome II, lot R) ; ses logits sont la vérité de toute KL
fourche	une copie modifiée de la référence pour un besoin de la ronde (capturer les entrées des experts, leur substituer une base)
R1	les dix fenêtres d'évaluation de wiki.test du Tome II (fenêtres 0 à 9, de 512 jetons chacune) ; la KL y note 2 550 positions, 255 par fenêtre. À ne pas confondre avec *la réserve R1*
calibration	les fenêtres 10 à 21 de wiki.test (6 132 positions), où les mécanismes ont été réglés ; disjointes de R1

Nom	Signification
fenêtres neuves	les dix fenêtres de wiki.test que ni la calibration ni R1 ne touchent, où le juge de la construction a refait les résultats (section 11.3)
P0, P1, P2, P2-10, P3	les protocoles de KL de la ronde (tableau 8.1) : P0 sans perte (logits identiques au bit) ; P1 l'éclaireur, 32 experts sur 768 sur R1 ; P2 le génome, 8 experts sur 3 fenêtres ; P2-10 les mêmes 8 sur les 10 fenêtres de R1 (passe K10) ; P3 le codage prédictif et l'élagage, 128 experts sur les fenêtres 5 à 9. Une KL ne se compare d'un protocole à l'autre qu'en rapport à la base de son propre protocole
K10	la passe qui a refait S^* contre la base sur les 10 fenêtres de R1 (protocole P2-10), déclarée avant ; elle remplace le rapport à 3 fenêtres (réserve R4)
lots R, A, B	R : la référence et le juge du Tome II ; A : le codon synonyme du zéro dans le format v1.2 (chapitre 9) ; B : le génome synonyme et le mode rapide (chapitre 10)
arbitre, F1	l'arbitre est le rôle du Tome II qui a tranché le format ; F1 est sa mesure de la frontière (codes en rANS par voie, échelles en écart à la médiane de ligne), rapportée au Tome II
frontière	les deux points que la ronde devait battre : sans perte, 3,8431 bits par poids (A_ctx) ou 3,8434 (A_flux) ; avec perte, la *base* du mode rapide (signe et quatre classes de magnitude), 2,6441 bits lus à 4,29 % d'erreur relative sur les poids (section 2.3)
A_ctx, A_flux	deux comptabilités du même code sans perte : A_ctx compte les codes en rANS par voie et par contexte d'échelle, les échelles, les tables et les en-têtes ; A_flux compte le flux du codec de l'étude 03, comme l'arbitre le compte. Elles diffèrent de $3 \cdot 10^{-4}$ bit par poids
brin, codon	dans le génome du chapitre 4, le brin est la suite de bits d'une rangée, lue par une fenêtre glissante de l bits (le codon) qu'un mélangeur et une table changent en poids ; ailleurs, *codon* désigne aussi un code E2M1 de 4 bits. Des analogies, pas des arguments
codon synonyme du zéro, fondu	les codes 0 (+0) et 8 (−0) donnent la même valeur ; le fondu $-0 \to +0$ réécrit chaque 8 en 0 avant le codage, sans changer une seule valeur (section 3.3, chapitre 9)
mutation synonyme	le choix, parmi les symboles de la base (même décodeur), de ceux qui minimisent l'erreur de sortie pondérée par H plutôt que l'erreur sur les poids (section 3.4)
H, e_k, λ_k	H la matrice des moments d'ordre deux (amortie) des entrées x d'une matrice d'expert, pondérées par le poids de routeur g ($\sum g^2 x x^\top / \sum g^2$), e_k et λ_k ses vecteurs et valeurs propres
λ	le poids de la contrainte d'entropie de la mutation synonyme ($\lambda = 0$: aucune ; $\lambda = 1$) ; à ne pas confondre avec λ_k
δ, δ^*	l'amortissement ajouté à H ; $\delta^* = 1$ la valeur choisie sans regarder l'évaluation ; au chapitre 8, δ note aussi l'écart d'échelle qui fixe κ

Nom	Signification
κ	deux sens : la classe d'échelle d'un bloc, trois classes selon son E8M0 rapporté à la médiane de la ligne (chapitres 3 à 10) ; au chapitre 12, le nombre de garde $\sigma/\mathbb{E}[m]$ du routeur
S^*	le système du génome : brin, rotation, Viterbi pondéré et trois exceptions exactes par rangée (chapitre 4)
v1.0, v1.2	versions du format `.trc` : v1.0 les fichiers d'origine ; v1.2 y ajoute le fondu du zéro (section 9.4) ; la v1.1 (deux niveaux) est réservée et n'est écrite par aucun codeur
compagnon rapide	le fichier `<nom>.rapide.trc` que lit le mode rapide du moteur, un `.trc` ordinaire qui porte la base synonyme (section 10.4)
J1	le jalon du Tome II, le service exact de gpt-oss-120b dans une fenêtre bornée ; ici, sa vitesse de génération refaite (section 9.7)
ABBA, $r(B/A)$	une série chronométrée qui alterne les deux fichiers (A B B A A B B A), chaque passe dans un processus neuf ; $r(B/A)$ le rapport de leurs vitesses médianes, jugé contre 2σ de la série
réserves R1 à R6	les conditions de l'inventaire des figures qui gardent vraie une figure ou une phrase (section 1.5) ; R1 : le ×1,44 du J1 vient du format v1.2 par le motif de ses requêtes de lecture, sur ce disque
III-$x.y$	le numéro d'une figure dans l'inventaire (`memoire/tome3/INVENTAIRE-figures.md`) ; le numéro imprimé suit les chapitres du PDF, où l'introduction est le chapitre 1

Chapitre 1

Introduction

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, les deux premiers tomes. Le Tome I a mesuré la machine de Stone et démontré que, sur elle, le disque est l'étage le plus lent : ce qui compte est le nombre de bits qu'un jeton doit lire. Le Tome II a construit troncons, qui fait tourner gpt-oss-120b (63,4 Go) avec 384 Mo de mémoire, exactement (logits identiques au bit entre fichier codé et fichier brut), à 2,22 fois la meilleure base mesurée ; ses poids d'experts se lisaient à 3,84 bits par poids sans perte, et la base de la découpe « base + complément » à 2,64 bits avec perte. **Ce tome établit** ce qu'a donné l'idée biomimétique de Stone, le génome des poids : une ronde de quatre équipes qui l'a mise à l'épreuve et a dit, chiffres en main, pourquoi presque tout tombe ; trois mécanismes qui gagnent, tous trois des « synonymes » au sens du code génétique ; leur construction dans le format et le moteur, et le juge qui l'a refaite ; six pistes empruntées à d'autres disciplines ; deux pistes mathématiques, la géométrie tropicale du routeur et les spectres des matrices. **Il reste** ce que la discussion rassemble (chapitre 14) : la queue de la KL, que rien ne vise encore ; le 120b servi par sa base synonyme ; un autre corpus ; le motif de lecture qui fait la vitesse de ce disque.

En bref

Un modèle d'IA est un très long texte de nombres. Stone a proposé de s'inspirer du vivant, qui ne stocke pas un organisme mais un génome, une recette d'où l'organisme se construit. Ce tome raconte ce que cette idée est devenue quand on l'a jugée par la mesure. La première réponse est sévère : une fois connue l'échelle de leur bloc, les poids de gpt-oss se comportent comme du bruit, et un bruit ne se résume pas ; la graine, les motifs partagés, la parenté entre experts, la recette sans perte tombent tous pour cette raison. Mais le vivant a un autre geste que la recette : la redondance du code génétique, où plusieurs codons désignent le même acide aminé. Les trois mécanismes qui gagnent sont de cette sorte. Le premier efface une information que le calcul ne lit jamais (le signe d'un zéro), sans perte, et fait lire 1,80 Go de moins au 120b. Le deuxième se sert de ce que le codeur sait déjà (la classe d'énergie de chaque colonne d'entrée). Le troisième, la mutation synonyme, choisit une approximation des poids dont l'erreur tombe là où les entrées réelles ne regardent pas : servie seule, la base des 768 experts du 20b divise par deux la KL de la base fixe du Tome II. Le tome dit aussi ce qui ne marche pas, aussi précisément que ce qui marche : le mode rapide n'apporte pas la vitesse annoncée, la garde tropicale ne voit pas les bascules en cascade, les spectres ne suivent pas une loi de puissance, et aucune des six pistes empruntées ne tient franchement devant son juge.

1.1 La question, posée par Stone et par la machine

La mission. Stone l'a écrite le 15 octobre 2026, et le Tome I la cite en entier : faire tourner « un modele ultra lourd quelcoque d'une ai [...] sans ram ni rien », en un mot « compresser à l'extreme puis décompresser en un temps record au besoin à l'instant exacte du noeud ». Les deux premiers tomes ont pris la seconde moitié de la phrase au mot : le format `.trc` et le moteur décompressent chaque expert au moment exact où le calcul le demande, puis le libèrent. Ce tome prend la première : compresser plus loin.

L'idée. Le même jour, Stone a ajouté une piste à lui :

> *J'ai penser à du biomimetisme informatique par exemple simjler une forme de molecule d'adn virtuel qui peut stocker une grande qua tutoriel d'informations en quelque sorte creer une nouvelle convention differentes du bit une transcription et implémentation*

puis, le lendemain, sa consigne pour l'étude : « le but est de concevoir un systeme fonctionnel et si il ne voye rien de convaincaint vers l'ADN au terme ils recherche le moyen biomimétique le plus prométeur jusqu'à trouver » (`idees/biomimetisme-genome-des-poids.md`). La conviction de Stone (« j'ai la certitude que ce qu'on cherche se trouve dans du biomimetisme ») est l'hypothèse de travail du tome ; la mesure la juge, et le tome rapporte les deux.

Ce que la machine en fait. Une nouvelle convention de symboles ne crée pas de place : l'ADN écrit avec quatre lettres, soit deux bits par lettre, et aucun alphabet ne descend sous l'entropie du message [1, 2]. La force du vivant n'est pas son alphabet, mais ses mécanismes. Et la machine dit lesquels comptent : le disque est l'étage le plus lent (2,82 Go/s mesurés au Tome I), et la carte attend pendant qu'il lit, avec, selon le calcul de l'idée, de l'ordre de 214 opérations disponibles par octet compressé. Un génome des poids dépense donc du calcul pour économiser des octets, l'échange même que cette machine demande. La question du tome, sous sa forme jugeable, est celle-ci :

> *Existe-t-il un mécanisme, inspiré du vivant, qui fasse lire sur le disque moins de bits par poids d'expert que la frontière du Tome II (3,84 bits sans perte ; 2,64 bits pour la base, avec perte), à une fidélité jugée par la KL sur du texte contre la référence exacte, et qui se transcrive sur la carte plus vite que le disque ne lit ce qu'il remplace ?*

La question se juge à chaque étape par un seuil déclaré avant la mesure (annexe A), et par un juge qui n'a pas fait le travail.

1.2 Ce que « génome » veut dire ici

Le mot est une analogie, et le tome la dit comme telle chaque fois qu'il s'en sert. Dans le vivant, une séquence de bases se lit par triplets, les *codons*, et le code génétique les traduit en acides aminés [3]. Ce code est dégénéré : 64 codons pour une vingtaine d'acides aminés et un signal d'arrêt, si bien que plusieurs codons *synonymes* désignent le même. Une mutation qui change un codon en un synonyme est sans effet sur la protéine ; et le code naturel est rangé de telle sorte que les erreurs les plus probables tombent sur des acides aminés semblables [4, 5]. Tableau 1.1 donne ce que le tome transpose, et où.

Deux mises en garde valent pour tout le tome. Le vivant tolère des erreurs qu'un modèle de langage ne tolère peut-être pas ; et une analogie belle n'est pas un argument : chaque mécanisme se juge par

Table 1.1 : L'analogie du génome, et ce qu'elle désigne dans ce tome. Une analogie choisit une direction de recherche ; elle ne prouve rien.

Dans le vivant	Dans troncons	Chapitre
le génome, recette plutôt que résultat	une graine et des coefficients, ou un brin de bits lu par une table, d'où les poids se reconstruisent	chapitre 4
le ribosome qui lit l'ARN codon après codon	une fenêtre de 12 bits qui glisse le long du brin et rend des poids par une table	chapitre 4
la duplication de gènes	un expert codé comme mutation d'un expert parent	chapitre 5
le codage prédictif, l'élagage synaptique	ne coder que l'écart à une prédiction ; retirer les poids inutiles	chapitre 6
le codon synonyme	le signe d'un zéro MXFP4, que le calcul ne lit jamais	chapitre 9
la mutation synonyme	une base des poids dont l'erreur tombe là où les entrées réelles ne regardent pas	chapitre 10
l'expression à la demande	l'expert décodé au moment du calcul, puis libéré (déjà construit au Tome II)	—

la mesure, pas par la beauté de l'image (`idees/biomimetisme-genome-des-poids.md`, § 6). Le tome emprunte aussi à d'autres disciplines que la biologie (le climat, la physique des verres de spin, la statistique, le cortex) ; ces emprunts sont dits comme tels au chapitre 7.

1.3 Le chemin du tome

Le tome a trois parties, comme le Tome II.

La question et la ronde. Chapitre 2 pose la question dans les termes de la machine : le bloc MXFP4 au bit près, la frontière, la borne distorsion-débit dans le domaine des poids, et les douze familles « génomiques » de la littérature. Chapitre 3 donne l'éclaireur, qui a trouvé où sont le bruit et la structure, et conçu les deux premiers mécanismes synonymes. Les chapitres 4 à 6 donnent les trois autres équipes de la ronde : la recette (le brin lu par codons, la graine), la duplication de gènes, le codage prédictif et l'élagage. Chapitre 7 donne le mimétisme, six pistes venues d'autres disciplines vers le chemin des octets. Chapitre 8 rassemble la ronde sur un seul axe, la KL sur le texte, et dit pourquoi chaque mécanisme gagne ou tombe.

La construction. Chapitre 9 donne le lot A : le codon synonyme du zéro dans le format `.trc`, version 1.2, sans perte. Chapitre 10 donne le lot B : le génome synonyme sur les 768 experts et le mode rapide du moteur. Chapitre 11 donne le juge qui a refait les deux lots par ses propres chemins.

Les pistes et le bilan. Chapitre 12 donne la géométrie tropicale du routeur et sa garde ; chapitre 13 les spectres des matrices ; chapitre 14 la discussion, les verdicts rassemblés, ce qui pourrait être faux à l'échelle du tome et les travaux futurs. Les annexes donnent les protocoles (seuils déclarés et témoins), les outils et les journaux.

Ce qui est allé au Tome II. La construction a rencontré une question qui n'est pas celle du génome : pourquoi deux fichiers du même modèle se lisent à des vitesses différentes sur ce disque. Le flux du placement disque et son juge y ont répondu (le motif des requêtes de lecture que le format impose, `genome/JUGEMENT-placement.md`) ; par la décision de ce juge, leur travail va au Tome II, dans un

chapitre « L'étage disque ». Ce tome n'en garde que la conséquence, la réserve R1, qui fixe comment lire le gain de vitesse du format v1.2 (section 11.6.5).

1.4 Ce que ce tome apporte

1. **La raison des échecs, mesurée.** Une fois l'échelle du bloc connue, les poids sont du bruit (le code voisin n'apporte que 0,005 bit ; les rangées tournées sont gaussiennes à 1,5 % près ; deux experts appariés ont le cosinus de deux experts tirés au hasard) ; quelques colonnes géantes portent la structure ; et l'erreur de sortie d'un expert ne prédit pas la KL, qui a une part lisse et une queue de bascules de routage (chapitre 8).
2. **Trois mécanismes synonymes qui gagnent** : le codon du zéro, sans perte (3,843 4 → 3,706 4 bits par poids sur les tenseurs de référence, dans la comptabilité A_flux, celle du codec de l'étude 03 ; 3,843 1 → 3,706 1 dans celle des codes, A_ctx, que donnent les chapitres 2 et 3 ; 1,80 Go de moins à lire sur le 120b) ; le contexte de la colonne, sans perte (3,684 4 bits) ; la mutation synonyme, avec perte (chapitres 3 et 8).
3. **Un format v1.2 exact**, construit, dont un décodeur écrit par le juge depuis la seule description retrouve chaque expert tiré au bit près, au signe des zéros près, et dont les logits sont identiques au bit sur dix fenêtres que personne n'avait vues ; sur ce disque, le J1 du 120b y gagne ×1,44, par le motif de ses requêtes de lecture (chapitres 9 et 11).
4. **Un mode rapide servi par le moteur** : les 768 experts du 20b servis par leur base synonyme ($\lambda = 0$), 2,67 bits par poids, KL 0,131 8 contre 0,264 0 pour la base fixe, redonnée hors de R1 par le juge, et servie par la carte aussi fidèlement que par la référence f64 (chapitre 10).
5. **Des idées tuées par les nombres**, chacune avec la propriété qui l'a décidée : la graine, les motifs, la duplication, la recette sans perte, l'hebbien, l'élagage, la garde tropicale, la loi de puissance des spectres ; et deux gains de vitesse barrés comme gains du mode rapide (chapitre 14).

1.5 Comment lire ce tome

Le juge : la KL. Presque tous les verdicts du tome sont rendus par un seul nombre, la divergence de Kullback-Leibler (la KL) entre la loi du jeton suivant donnée par le modèle modifié et celle de la référence exacte f64 du Tome II, moyennée sur des positions de texte. Elle est posée en cinq phrases au section 3.5, avant son premier usage, et reprise avec ses deux parts au section 8.1. Les autres noms du travail (les fenêtres R1, les lots, les protocoles P0 à P3, A_ctx et A_flux…) sont dans la table des notations, au début du tome.

La provenance des nombres. Chaque nombre porte sa provenance : [mes.] mesuré ici, par un outil et un journal commités ; [calc.] calculé à partir de nombres nommés ; [litt.] littérature, non reproduite ; et, quand il le faut, [hyp.] pour une hypothèse dite. Chaque nombre vient d'un fichier commité, cité par son chemin (`commececi`) ; aucun n'est recopié de mémoire.

Les phrases barrées. Quand une mesure a démenti un chiffre ou une phrase publiés, même les nôtres, la phrase est barrée ~~comme ceci~~ là où elle était, et l'on dit ce qui l'a remplacée. Le tome en porte beaucoup : c'est la trace que les juges ont fait leur travail.

Les figures. Les figures au style du tome ont un numéro dans l'inventaire des figures (III-$x.y$, le chapitre de l'inventaire et le rang, `memoire/tome3/INVENTAIRE-figures.md`) ; le texte ne s'en

sert que pour une figure prévue et non tracée, et appelle les autres par leur numéro imprimé, qui suit les chapitres du PDF (l'introduction y est le chapitre 1). Chacune se tire des données commitées par un script commité, et sa légende, jugée, ouvre sur l'idée de la figure en une phrase. Les figures du mimétisme et la coupe du routeur sont incluses telles que leurs pistes les ont tracées, hors du style commun, et leur légende le dit ; celles des spectres viennent d'un document à part de leur piste.

Les réserves. Six réserves de l'inventaire des figures traversent le tome ; pour ne pas les confondre avec les dix fenêtres d'évaluation que le Tome II appelle R1, le tome écrit toujours « la réserve R1 ». La réserve R1 : le gain ×1,44 du J1 vient du format v1.2 par le motif de ses requêtes de lecture, propre à ce disque, jamais de « l'état du fichier ». R3 : les mesures faites avec 32 experts substitués et celles faites avec 768 ne se relient par aucun trait. R4 : le rapport de KL de S^{*} vient de la passe à dix fenêtres, l'ancien chiffre à trois fenêtres est barré. R5 : les figures hors du style commun sont dites telles. R6 : le second tirage de P3, en travail dans une autre session, ne donne aucun chiffre au tome. (R2 concerne le Tome II.)

Ce qui pourrait être faux. Chaque chapitre se ferme sur sa propre section ; celle du tome entier est au section 14.4.

Première partie

La question et la ronde

Chapitre 2

La question du génome : le toit, les bits lus par poids, la frontière

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, deux tomes et une idée. Le Tome I a posé le toit : à disque limitant, la vitesse d'un jeton est le débit du disque divisé par les octets qu'il doit lire, et seul le nombre de bits par poids a sur ce toit une pente de -1 sans contrepartie. Le Tome II a construit le moteur et le format `.trc`, qui lit les experts de gpt-oss à **3,8431 bits par poids sans perte** (codes MXFP4 en rANS à table par voie, échelles en écart à la médiane de ligne ; mesure F1 de l'arbitre) et prévoit une base de **2,6441 bits** (signe et quatre classes de magnitude) à **4,29 %** d'erreur quadratique relative sur les poids. L'étude 03 avait mesuré que, sachant l'échelle de leur bloc, les codes sont presque indépendants : le code voisin n'apporte que 0,005 bit. Stone, enfin, avait posé une idée (`idees/biomimetisme-genome-des-poids.md`) et une consigne, le 2026-10-16 : « le but est de concevoir un systeme fonctionnel ». **Ce chapitre établit** la question en bits ; le bloc MXFP4 au bit près, avec la loi de ses quinze valeurs ; la frontière à battre et ce qu'en dit une borne distorsion-débit calculée dans le domaine des poids, posée comme une *borne supérieure* de $R(D)$ et démontrée telle ; l'état de l'art en douze familles, chacune passée à deux filtres (quelle dépendance exploite-t-elle ? quelle erreur mesure-t-elle ?) ; la case que personne n'avait remplie sur un modèle né en 4 bits ; et ce que la lecture prédit, que les quatre chapitres suivants mesurent. **Il reste** les mesures elles-mêmes : l'éclaireur (chapitre 3), la recette (chapitre 4), la duplication (chapitre 5), le codage prédictif et l'élagage (chapitre 6), puis la synthèse de la ronde (chapitre 8). Sources : `genome/00-etat-de-l-art.md` ; `genome/etat-de-l-art-pour-le-memoire.md` ; `genome/01-eclaireur.md` § 3 ; `mesures/genome/etat-de-l-art/journaux/borne-rd.json`, `borne-rd-passe1.json`, `zero-canonique.json`, `zero-temoin-sans-drapeau.json` ; `mesures/genome/etat-de-l-art/sources.csv`, `recherches.csv` ; Tome I, chapitre 3 (le toit).

En bref

Un modèle de langage est un très long texte de nombres : 19 milliards de poids d'experts pour gpt-oss-20b, 115 milliards pour le 120b. L'idée de Stone tient dans une image : le vivant ne stocke pas un organisme, il stocke un génome, quelques milliards de lettres d'où l'organisme se construit. Peut-on, de même, stocker une *recette* des poids plutôt que les poids ? L'image est une analogie, pas une preuve, et ce chapitre la traduit en une question que la machine peut juger : combien de bits faut-il lire sur le disque par poids, et à quelle erreur ? La réponse de la théorie de l'information est sévère : si les poids, vus d'eux-mêmes, se comportent comme des tirages indépendants, aucune recette ne les décrit plus court que leur codage entropique. La lecture de la littérature dit donc d'abord où chaque idée « génomique » parie sur une dépendance, puis

> où la seule marge restante se trouve : non dans les poids, mais dans l'erreur qu'on choisit de mesurer.

2.1 Le toit : ce qu'un jeton doit lire

Le Tome I l'a démontré (théorème du toit, chapitre 3) : sous la seule hypothèse qu'un étage de débit W fait passer au plus $W\tau$ octets en un temps τ, un jeton qui doit lire O octets sur cet étage dure au moins O/W. Pour des poids lus une fois chacun, $O = Nb/8$ (N poids lus par jeton, b bits par poids), d'où

$$\text{jetons/s} \;\le\; \frac{8\,W}{N\,b}, \qquad \frac{\partial \ln(\text{jetons/s})}{\partial \ln b} = -1. \tag{2.1}$$

Sur la machine de Stone, l'étage le plus lent est le disque : 2,82 Go/s mesurés [mes.] (`fondations/journaux/01-machine/debits-mesures.json`, cité par `genome/00-etat-de-l-art.md` § 1). Diviser par deux les bits lus par poids double le toit ; retirer 3,56 % des octets d'experts lus le monte de $1/(1-0{,}0356) - 1 = 3{,}7\,\%$ [calc.]. Toute la quête du génome se juge donc en **bits lus par poids**, à une erreur donnée.

Le calcul n'est pas le goulot. La carte a de l'ordre de 214 opérations libres par octet compressé lu (fondations/01), 926 au débit réel du disque du moteur selon la correction du Tome II (m12) [calc.]. Un mécanisme dont la transcription coûte moins que ce budget est gratuit en temps : il ne se juge qu'à ses bits et à son erreur. Les chapitres de la ronde comptent ce coût chaque fois qu'il n'est pas nul (section 4.10).

2.2 Le bloc MXFP4 au bit près

Les experts de gpt-oss sont publiés en MXFP4 [6], le format à petite virgule flottante de la famille Microscaling [7, 8]. Un bloc porte 32 poids consécutifs d'une rangée dans 17 octets (figure 2.1) : un octet d'échelle E8M0, un exposant seul, et seize octets de codes, deux codes de 4 bits par octet.

Le code E2M1. Un code c de 4 bits se lit $c = (s, E_1E_0, M)$: un bit de signe, deux bits d'exposant, un bit de mantisse. Sa valeur est

$$\text{E2M1}(c) = (-1)^s \times \begin{cases} M/2 & \text{si } E = 0 \quad \text{(sous-normal)}, \\ 2^{E-1}\,(1 + M/2) & \text{si } E \in \{1, 2, 3\}, \end{cases} \tag{2.2}$$

ce qui donne les huit magnitudes $\{0;\ 0{,}5;\ 1;\ 1{,}5;\ 2;\ 3;\ 4;\ 6\}$ [calc.]. Seize motifs pour quinze valeurs : le code $0 = (0, 00, 0)$ et le code $8 = (1, 00, 0)$ valent tous deux zéro, l'un $+0$, l'autre -0. La figure le dit en légende : c'est le premier codon synonyme du tome, et le chapitre suivant en mesure le prix.

La valeur d'un poids. Le poids i du bloc b vaut $w_{32b+i} = 2^{\,e_b - 127}\,\text{E2M1}(c_i)$, e_b l'octet E8M0 du bloc. Les chapitres suivants écrivent souvent la forme équivalente des noyaux de llama.cpp et de `trc-reference`, à table doublée :

$$w = \pm K[m] \cdot 2^{\,e-128}, \qquad K = \{0, 1, 2, 3, 4, 6, 8, 12\}, \tag{2.3}$$

où m est la magnitude sur 3 bits ; les deux écritures donnent les mêmes produits puisque $K[m] = 2\,|\text{E2M1}(c)|$ et $2^{e-128} = 2^{e-127}/2$ [calc.]. Un bloc coûte $8 + 32 \times 4 = 136$ bits, soit 4,25 bits par poids avant tout codage [calc.]. Une rangée de 2 880 poids compte 90 blocs ; une matrice d'expert de gpt-oss, $2\,880 \times 2\,880 = 8\,294\,400$ poids.

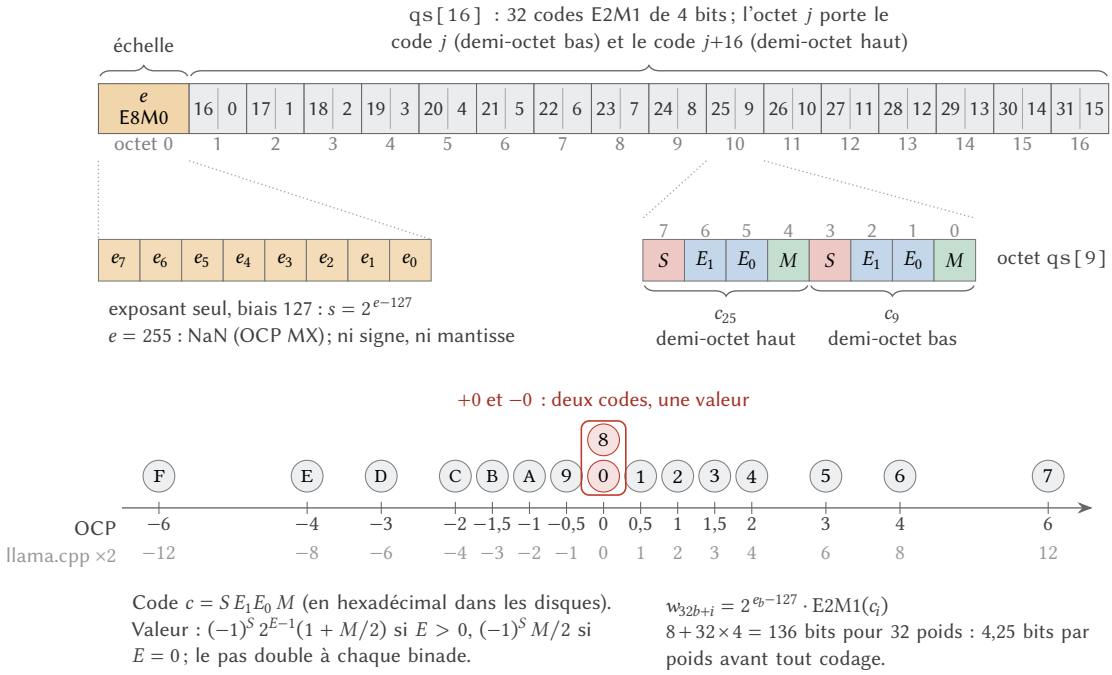


Figure 2.1 : Trente-deux poids partagent un octet d'échelle ; seize motifs de code donnent quinze valeurs, car +0 et −0 sont le même nombre. [schéma] L'échelle e est un octet E8M0 (un exposant seul) ; seize octets portent les codes, le code i dans le demi-octet bas de l'octet i si $i < 16$, dans le demi-octet haut de l'octet $i - 16$ sinon. Chaque code est un flottant E2M1 : un signe, deux bits d'exposant, un bit de mantisse. Au-dessus de l'axe, les valeurs de la spécification OCP ; dessous, en gris, celles de la table doublée de llama.cpp ({0,1,2,3,4,6,8,12}, échelle divisée par deux, mêmes produits), que la roue des codons du chapitre 9 imprime. La paire +0 (code 0), −0 (code 8) est le premier codon synonyme du chapitre 9. Ce que la figure ne montre pas : l'ordre des blocs dans un tenseur (rangée après rangée, 90 blocs par rangée de 2 880 poids).

Données : spécification OCP Microscaling Formats (MX) v1.0 ; llama.cpp, `ggml-common.h` (`block_mxfp4`).

$\lvert K\rvert$ (unités doublées)	0	1	2	3	4	6	8	12
$\lvert$E2M1$\rvert$	0	0,5	1	1,5	2	3	4	6
$P(V = +\lvert K\rvert)$ (%)	13,694	8,601	10,152	5,461	7,883	5,595	3,998	1,462
$P(V = -\lvert K\rvert)$ (%)		8,602	10,153	5,460	7,882	5,595	3,998	1,462

Table 2.1 : La loi des quinze valeurs MXFP4 sur les 12 tenseurs de référence de gpt-oss-20b, ±0 fusionnés [mes.]. Données : `mesures/genome/etat-de-l-art/journaux/borne-rd-passe1.json` (`loi_v`, en unités doublées K) ; $E[V^2] = 17{,}847$ dans ces unités. La loi est symétrique : seul l'écart à la médiane de ligne de l'échelle (le contexte κ du format) et la classe de magnitude portent de l'information au-delà de cette table.

La loi des valeurs. Sur les 12 tenseurs de référence de la frontière (couches 0, 8, 16 et 23 de gpt-oss-20b, matrices gate, up et down, 3 185 049 600 poids), les quinze valeurs ont la loi du tableau 2.1 [mes.] (`mesures/genome/etat-de-l-art/journaux/borne-rd-passe1.json`, champ `loi_v`). Elle est symétrique à $1{,}3 \cdot 10^{-5}$ près, et le zéro y est la valeur la plus fréquente : 13,69 % des poids. Son entropie vaut $H(V) = 3{,}7228$ bits, contre $H(C) = 3{,}8598$ pour les seize codes des mêmes tenseurs (`genome/01-eclaireur.md` § 2.1) : l'écart, 0,137 bit, est le prix du signe des zéros.

2.3 La frontière, et la borne distorsion-débit

2.3.1 Ce que coûte un poids aujourd'hui

Deux points forment la frontière que la ronde devait battre (`genome/00-etat-de-l-art.md`, En bref, point 1 ; mesure F1 de l'arbitre, rapportée au Tome II, chapitre 2) :

- **sans perte** : 3,8431 bits par poids (A_ctx : codes en rANS à table par voie et par contexte d'échelle, échelles E8M0 en écart à la médiane de ligne, tables et en-têtes comptés), chaque flux décodé exactement [mes.] ;
- **avec perte**, la *base* du mode rapide : le signe et quatre classes contiguës de magnitude, {0; 0,5}, {1; 1,5; 2}, {3; 4}, {6} en unités E2M1, chacune reconstruite par son centre de gravité pondéré par s^2 (0,237 166 ; 1,424 298 ; 3,418 577 ; 6 ; `mesures/genome/genome/frontiere.json`), à **2,6441 bits** lus pour une erreur quadratique relative sur les poids $D = \sum s^2(v - \hat{v})^2 / \sum s^2 v^2 =$ **4,2920 %** [mes.].

La base est une *fonction du code* : le code exact se retrouve derrière elle par un complément sans perte, ce que le format réserve pour sa version à deux niveaux. C'est sa force (elle s'emboîte) et sa contrainte (elle est scalaire et fixée d'avance) ; la borne qui suit dit combien coûte cette contrainte.

2.3.2 La borne d'un codeur sans mémoire, dans le domaine des poids

Le modèle. On écrit $w = s\,v$, v tiré i.i.d. selon la loi du tableau 2.1 (±0 fusionnés), $s = 2^{e-128}$ connue du décodeur. La distorsion est l'erreur relative poolée $D = \sum s^2(v - \hat{y})^2 / \sum s^2 v^2$. Un quantificateur unique pour tous les blocs (une seule pente, dite allocation *uniforme*) est exactement la contrainte de la base frontière. La fonction distorsion-débit de la source discrète V sous $d(v, \hat{y}) = (v - \hat{y})^2$ est

$$R(D) = \min_{Q(\hat{y}|v)\ :\ \mathbb{E}\, d(V,\hat{Y}) \le D} I(V;\hat{Y}), \tag{2.4}$$

et le théorème de codage de source avec critère de fidélité [9] dit que tout débit au-dessus de $R(D)$ est atteignable par des codes assez longs, aucun en dessous.

L'algorithme de Blahut-Arimoto. Pour une pente $\beta > 0$ (bits par unité de distorsion), on itère jusqu'au point fixe [10]

$$Q(\hat{y} \mid v) = \frac{q(\hat{y})\, 2^{-\beta(v-\hat{y})^2}}{\sum_{y'} q(y')\, 2^{-\beta(v-y')^2}}, \qquad q(\hat{y}) = \sum_{v} p(v)\, Q(\hat{y} \mid v), \tag{2.5}$$

puis on lit $R = \sum_{v,\hat{y}} p(v) Q(\hat{y} \mid v) \log_2 \frac{Q(\hat{y}|v)}{q(\hat{y})}$ et $D = \sum p(v) Q(\hat{y} \mid v)(v-\hat{y})^2$. Aux pentes faibles, le nombre d'itérations nécessaire croît comme $1/\beta$; c'est là que l'outil a peiné.

L'histoire des passes, telle qu'elle est. La passe 1 de `borne_rd.py` a fait tomber son propre témoin B1 au bout bas de la courbe (à $\beta = 10^{-6}$, le point n'était pas convergé), tandis que B2 tenait (une gaussienne suit 2^{-2R} à 10^{-5} bit) et que sa cassure tombait. La passe 2, avec la correction déclarée après la chute (un point de distorsion supérieure à $\mathbb{E}[v^2]$ est remplacé par le point de débit nul), a fait tenir B1bis et B2, mais 243 points sur 1 619 seulement ont convergé, et la courbe à allocation optimale entre exposants se lisait hors de son domaine : elle n'est pas publiée. La passe 3 n'a calculé que l'allocation uniforme. Dans la zone de 1,8 à 3,2 bits, Blahut-Arimoto n'a pas atteint la tolérance déclarée (10^{-11}) en 200 000 itérations ; par la règle déclarée avant la passe 3, aucun nombre *convergé* n'est donc publié à 2,64 bits ni à 4,29 %. Ce qui reste vrai sans convergence, et qui a été lu *après* la passe 3 (la lecture est dite telle, `genome/01-eclaireur.md` § 3), est le lemme suivant.

Lemme 2.1 (Chaque itéré est un point atteignable)**.** *Pour tout canal de test $Q(\hat{y} \mid v)$ et toute loi q sur les reproductions,*

$$\sum_{v,\hat{y}} p(v)\, Q(\hat{y} \mid v) \log_2 \frac{Q(\hat{y} \mid v)}{q(\hat{y})} \;\geq\; I(V;\hat{Y}) \;\geq\; R\big(\mathbb{E}\, d(V,\hat{Y})\big), \tag{2.6}$$

avec égalité à gauche si et seulement si q est la marge $m(\hat{y}) = \sum_v p(v)Q(\hat{y} \mid v)$. Le point (D, R) lu à n'importe quelle itération de Blahut-Arimoto est donc au-dessus de la courbe : $R(D) \leq R$.

Démonstration. On écrit $\log_2 \frac{Q}{q} = \log_2 \frac{Q}{m} + \log_2 \frac{m}{q}$ sous la somme. Le premier terme donne $\sum p(v)Q(\hat{y} \mid v) \log_2 \frac{Q(\hat{y}|v)}{m(\hat{y})} = I(V;\hat{Y})$, par définition de l'information mutuelle du couple $(V, \hat{Y})$ de loi $p(v)Q(\hat{y} \mid v)$. Le second ne dépend que de $\hat{y}$: $\sum_{\hat{y}} \big(\sum_v p(v)Q(\hat{y} \mid v)\big) \log_2 \frac{m(\hat{y})}{q(\hat{y})} = \sum_{\hat{y}} m(\hat{y}) \log_2 \frac{m(\hat{y})}{q(\hat{y})} = D(m \,\|\, q)$, divergence de Kullback-Leibler [11], positive ou nulle et nulle si et seulement si $q = m$ (inégalité de Gibbs, [2], § 2.6). D'où la première inégalité. La seconde est la définition (2.4) : Q est un canal admissible pour la distorsion $D = \mathbb{E}\, d(V,\hat{Y})$ qu'il réalise, donc $R(D)$, minimum sur ces canaux, est au plus $I(V;\hat{Y})$. ∎

Deux conséquences rendent la figure lisible. D'abord, la courbe tracée par les itérés, convergés ou non, est une **borne supérieure** de $R(D)$, jamais une estimation par en dessous. Ensuite, $R(D)$ est convexe ; tout segment entre deux points atteignables est atteignable par partage du temps (coder une fraction θ des blocs avec le premier code, le reste avec le second donne $(\theta D_1 + (1-\theta)D_2, \theta R_1 + (1-\theta)R_2)$), et l'enveloppe convexe inférieure des itérés reste une borne supérieure. C'est elle qu'on lit à l'erreur et au débit de la frontière.

Ce que la borne dit. Les 40 itérés de l'allocation uniforme, dont 25 convergés, sont tracés en figure 2.2 [mes.] [calc.] (`memoire/figures/tome3/donnees/eclaireur-nombres.csv`, tiré de `borne-rd.json`). Le tableau 2.2 en garde les quatre points que l'éclaireur a publiés. À l'erreur de la frontière, la corde entre deux itérés donne $R(4{,}29\,\%) \leq 2{,}168$ bits, **0,476 bit sous la frontière** ; à son

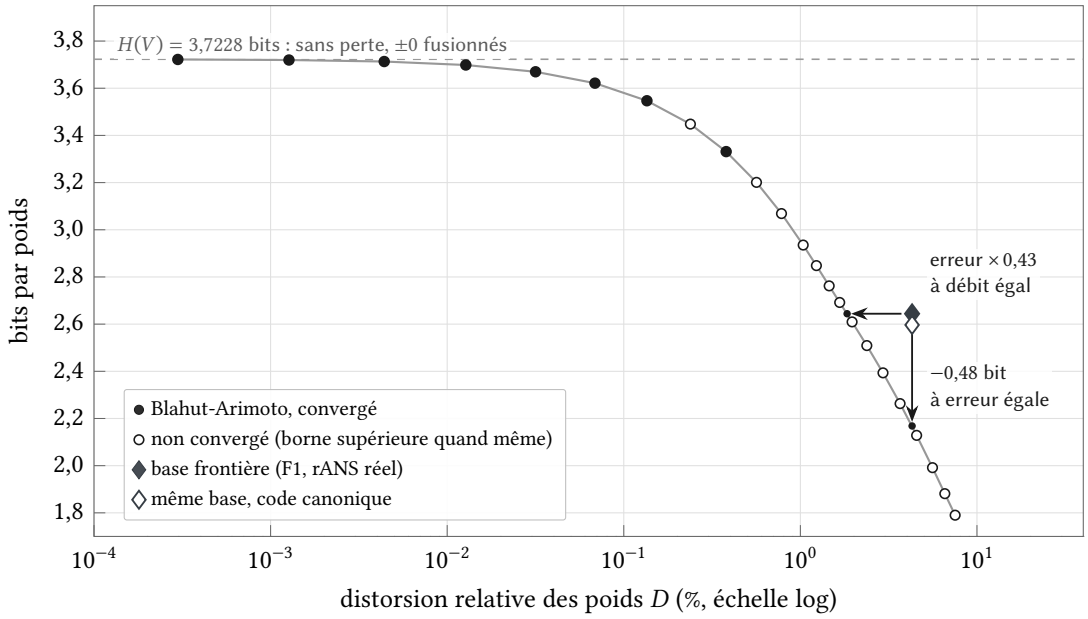


Figure 2.2 : L'écart de la base frontière à la borne distorsion-débit d'un codeur sans mémoire est dans la forme du quantificateur, pas dans une structure des poids. [calc.] La source est le code v à 15 valeurs (±0 fusionnés) multiplié par l'E8M0 de son bloc, un seul quantificateur pour tous les blocs (la contrainte même de la frontière). Chaque itéré de Blahut-Arimoto est un point atteignable, convergé ou non, car $\sum p\,Q \log(Q/q) = I(V;\hat{Y}) + D(\text{marge} \,\|\, q) \geq R(D)$: la courbe est donc une borne supérieure de $R(D)$. Cette lecture a été posée après la troisième passe ; la règle déclarée avant elle ne publiait que des points convergés. Entre 1,8 et 3,4 bits, 15 des 40 itérés n'ont pas atteint la tolérance déclarée : marques creuses. Les cordes entre deux points sont atteignables par partage du temps. À l'erreur de la frontière (4,29 %), la borne vaut au plus 2,17 bits, 0,476 bit sous la frontière (2,6441 bits, rANS réel) ; à son débit, la distorsion vaut au plus 1,84 %. La borne est celle des codes *sachant l'échelle* : les échelles n'y sont pas comptées, alors que les bits de la frontière les comptent (avec son contexte κ). Sur les experts du protocole de $\lambda = 1$ (16 experts), les échelles coûtent 0,0375 bit par poids et les symboles seuls $H_0 = 2{,}6415$ bits ; contre ce H_0, l'écart vaut 0,473 bit au lieu de 0,476, et la conclusion ne change pas. Ce que la figure ne montre pas : une allocation qui donne plus de débit aux blocs lourds (calculée hors de son domaine, non publiée), ni aucune erreur de sortie ou de KL : cette métrique n'est pas un juge (chapitre 8).

Données : `mesures/genome/etat-de-l-art/journaux/borne-rd.json` (`uniforme_precis`), `zero-temoin-sans-drapeau.json`, `zero-canonique.json`, `mesures/genome/etat-de-l-art/passes/frontiere-lam0-damp0.01-e0_8_16_24.json` ; script `memoire/outils/tome3_eclaireur.py`.

Débit (bits par poids)	Distorsion relative D	Lecture
2,1284	4,55 %	à peu près l'erreur de la frontière, 0,51 bit de moins
2,2631	3,67 %	sous la frontière en débit *et* en erreur
2,5091	2,38 %	
2,6095	1,96 %	au débit de la frontière, moins de la moitié de son erreur

Table 2.2 : Quatre points de la passe 3 (allocation uniforme, un seul quantificateur pour tous les blocs), chacun une borne supérieure de $R(D)$ par le lemme 2.1 [mes.]. Données : `mesures/genome/etat-de-l-art/journaux/borne-rd.json` (`uniforme_precis`) ; `genome/01-eclaireur.md` § 3.

débit (2,6441 bits), la distorsion vaut au plus 1,84 %, **0,43 fois son erreur** [calc.]. L'éclaireur écrivait « environ 0,47 bit de moins » et « moins de la moitié de son erreur » (`genome/01-eclaireur.md` § 3) ; le juge des figures a refait la corde par l'enveloppe convexe des 40 itérés et trouvé les mêmes nombres (`memoire/tome3/JUGEMENT-figures2-ronde.md`, M12).

Une précaution de comptabilité. La borne est celle de codes *qui connaissent l'échelle* : les échelles n'y sont pas comptées, alors que les 2,6441 bits de la frontière les comptent (avec son contexte κ). Sur les 16 experts du protocole de l'éclaireur où la frontière a été refaite à l'estimateur d'entropie, les échelles coûtent 0,0375 bit par poids et les symboles seuls $H_0 = 2{,}6415$ bits ; contre ce H_0, l'écart vaut 0,473 bit au lieu de 0,476 [calc.] (`eclaireur-nombres.csv`, d'après `passes/frontiere-lam0-damp0.01-e0_8_16_24.json`). La conclusion ne change pas.

Où est l'écart. Il n'est pas dans une structure des poids : le modèle de la borne les suppose indépendants sachant l'échelle, ce que l'étude 03 a mesuré vrai à 0,005 bit près. Il est dans **la forme du quantificateur**. La frontière est scalaire et contrainte d'être une fonction du code ; la borne est atteinte par des codes longs et libres. L'écart se partage entre deux gains connus. Le premier est celui d'un quantificateur scalaire à contrainte d'entropie, qui choisit ses cellules pour leur coût en bits et non pour leur nombre. Le second est le gain de pavage d'un code vectoriel ou en treillis : à haut débit, une cellule de quantification sphérique en grande dimension a, à volume égal, un moment d'inertie plus petit que le cube d'un quantificateur scalaire uniforme d'un facteur $\pi e/6$, soit au plus

$$10 \log_{10} \frac{\pi e}{6} = 1{,}53 \text{ dB}, \qquad \frac{1}{2} \log_2 \frac{\pi e}{6} = 0{,}2546 \text{ bit par poids} \tag{2.7}$$

à erreur égale [calc.] [12, 13]. Un générateur de poids (graine, automate, dictionnaire) n'a pas d'autre structure à exploiter que celle-là : il ne peut que viser la même borne. Le chapitre de la recette (chapitre 4) retrouve exactement ce partage, mesuré.

2.4 L'état de l'art, en douze familles

2.4.1 Deux filtres

Premier filtre : quelle dépendance ? Une source i.i.d. n'a pas d'autre structure que sa loi. Sa description la plus courte, en moyenne, est son codage entropique [1] ; le théorème vaut pour toute description, programme générateur compris, au sens de la complexité de Kolmogorov, dont l'espérance par symbole tend vers l'entropie pour une source i.i.d. [2, chapitre 14]. Une idée « génomique » ne peut donc gagner sans perte que la dépendance qu'elle exploite, et il faut que cette dépendance existe

dans nos poids. Sur les 72 tenseurs d'experts de gpt-oss-20b, l'étude 03 et la mesure F1 avaient donné [mes.] (`genome/00-etat-de-l-art.md` § 1) : H(code) d'ordre 0 de 3,860 bits sur 4 ; un apport du code précédent de 0,005 bit ; un apport du contexte d'échelle d'environ 0,035 bit (2,642 → 2,607 pour la base) ; les échelles E8M0 sachant la médiane de ligne à 1,01 bit pour 32 poids, soit 0,032 bit par poids.

Second filtre : quelle erreur ? Avec perte, ce qui compte n'est pas $\|\Delta W\|$ mais ce que l'erreur change à la sortie. Pour une matrice W et ses entrées x, de matrice de seconds moments $H = \mathbb{E}[xx^\top]$,

$$\mathbb{E}\,\|\Delta W x\|^2 = \mathbb{E}\big[x^\top \Delta W^\top \Delta W x\big] = \mathbb{E}\,\mathrm{tr}\big(\Delta W x x^\top \Delta W^\top\big) = \mathrm{tr}\big(\Delta W\, H\, \Delta W^\top\big), \tag{2.8}$$

par la cyclicité de la trace et la linéarité de l'espérance [calc.]. Si H était proportionnelle à l'identité, l'erreur de sortie serait l'erreur sur les poids ; elle ne l'est pas : les activations des modèles de langage ont des canaux dominants, et protéger le centième des poids qui les lisent réduit beaucoup l'erreur [14] [litt.]. Toute la marge d'un mécanisme avec perte est dans (2.8).

2.4.2 Les familles

Le tableau 2.3 range les douze familles lues par l'éclaireur, à la source primaire (le résumé de chaque article, par l'API d'arXiv ou de Crossref, le 2026-10-16 ; la formulation publiée pour les travaux dont le mécanisme compte : GPTQ, OBC, QMoE, MatGPTQ, SeedLM, Tied Trit-Planes ; `mesures/genome/etat-de-l-art/sources.csv`). Les chiffres des auteurs sont des chiffres d'éditeur, non reproduits ici [litt.]. Pour chacune : ce qu'elle pose, ce qui nous manque pour l'employer sur des poids nés en MXFP4.

Deux calculs qui jugent la graine avant toute mesure. SeedLM approche un bloc de C poids par k colonnes d'une matrice pseudo-aléatoire. Si V est un sous-espace aléatoire uniforme de dimension k et w un vecteur fixé, $\mathbb{E}\,\|P_V w\|^2 = \frac{k}{C}\|w\|^2$: par invariance par rotation, P_V a la même loi que $O^\top P_0 O$ pour O uniforme et P_0 le projecteur sur les k premières coordonnées, et $\mathbb{E}\,[O^\top P_0 O] = \frac{\mathrm{tr}\,P_0}{C} I = \frac{k}{C} I$ [calc.]. Un bloc de poids presque i.i.d. laisse donc un résidu d'énergie relative voisin de $1 - k/C$. SeedLM le rachète en choisissant la meilleure graine parmi 2^{16} ; mais on ne peut pas gagner plus d'information que l'index qu'on stocke : 16 bits par bloc au plus [calc.]. Le section 4.2 en tire la loi exacte (le meilleur de N sous-espaces suit une loi Bêta) et la confronte aux vrais poids.

2.5 Ce qui n'a jamais été tenté sur un modèle né en 4 bits

L'éclaireur a interrogé l'API d'arXiv le 2026-10-16, requêtes consignées avec leur nombre de réponses (`mesures/genome/etat-de-l-art/recherches.csv`) : « MXFP4 ET gpt-oss », « lossless ET MXFP4 », « gpt-oss ET compression », « Matryoshka ET quantization », « any-precision ET LLM », « DNA ET neural network ET weight compression » (0 réponse), « cellular automata ET weights ET compress » (0), « dictionary learning ET large language model ET weight ET compression », « genetic code ET error ET quantization » (0). Le tableau 2.4 croise les gestes et leur application à un modèle né en 4 bits.

La case prise. Les lignes 4 et 5 réunies, dans le cadre des lignes 2 et 3 : *choisir les symboles de la base emboîtée par rétroaction d'erreur sur les entrées réelles, sous contrainte d'entropie, sans toucher au décodeur ni au code exact.* La raison du choix est un nombre : c'est le seul geste de la liste qui ne parie pas sur une dépendance entre poids, que 03 a mesurée absente (0,005 bit) ; il parie sur une dépendance entre *entrées*, que le moteur peut mesurer. Le vivant a le même geste, et l'éclaireur l'a pris pour guide,

Famille	Ce qu'elle pose (littérature, non reproduit)	Ce qui nous manque
1. Recette à graine	SeedLM [15] : un bloc approché par quelques colonnes pseudo-aléatoires d'un LFSR, graine et coefficients stockés ; PRANC [16], NOLA [17], dimension intrinsèque [18]	PRANC, NOLA et la dimension intrinsèque entraînent *dans* le sous-espace ; SeedLM travaille après coup mais avec perte, sur BF16 ; un sous-espace aléatoire de dimension k garde en moyenne k/C de l'énergie d'un bloc
2. Réseaux générateurs	HyperNetworks [19], HyperNEAT [20], automates cellulaires neuronaux [21], HyperNCA [22]	tous génèrent des poids *entraînés à être générables* ; aucun ne décrit après coup un modèle entraîné librement
3. Parenté et fusion d'experts	MC-SMoE [23], Merging Experts into One [24], HC-SMoE [25], Sub-MoE [26], D^2-MoE [27], MoBE [28], OMP-MoE [29]	tous partent du BF16 et mesurent une précision de tâches ; aucun ne mesure la parenté en bits sur des codes de 4 bits
4. Codage différentiel	BitDelta [30] (le delta d'un affinage en 1 bit), Delta-CoMe [31]	un parent : un affinage en a un, un expert n'en a pas, sauf s'il est né par copie
5. Prédictif et entropique sans perte	cortex prédictif [32], DeepCABAC [33], ZipNN [34], DFloat11 [35], Huff-LLM [36], TRACE [37]	tous gagnent sur l'exposant BF16, très redondant ; un code MXFP4 n'a plus d'exposant par poids, il reste 0,14 bit de redondance par code ($4 - 3{,}86$) et 0,14 sur les échelles, déjà pris
6. Élagage	densité synaptique [38], OBD [39], OBS [40], Deep Compression [41], experts retirés [42, 43]	la sensibilité se lit dans la sortie, pas dans les poids ; OBS est le premier « report d'erreur »
7. Dictionnaires et codes vectoriels	codage parcimonieux [44], AQLM [45], QuIP# [46], QTIP [47], HyperQuant [48], MASA [49]	ils *remplacent* la grille et partent du BF16 ; sur une source i.i.d., ils ne gagnent que le pavage (2.7)
8. Rétroaction d'erreur	OBC [50], GPTQ [51], QMoE [52], AWQ [14], MoEQuant [53], EAQuant [54], neurone sigma-delta [55]	un point de départ né en 4 bits, une grille imposée et emboîtée, le coût en débit ; et la contrainte qu'un complément sans perte rende le code exact derrière la base
9. Formats emboîtés, modèles nés en 4 bits	Matryoshka [56], Any-Precision [57], MatGPTQ [58], MX [7], entraînement MXFP4 [59], gpt-oss [6], Scale-QLoRA [60], Tied Trit-Planes [61], BitNet [62]	MatGPTQ, le plus proche, part du BF16 et tronque les bits forts d'un entier ; Tied Trit-Planes requantifie des experts MXFP4 lus sur SSD, mais à 4,06 bits et sans emboîtement
10. L'ADN au sens propre	calcul [63], stockage [64-66], code génétique correcteur [5], biais d'usage des codons [67]	la densité de l'ADN est physique, pas informationnelle ; ce qui transpose est l'agencement d'un code pour que l'erreur tombe là où le phénotype ne la voit pas
11. L'évolution des codes	EvoPress [68], HyperNEAT	un optimiseur, pas une représentation : il ne crée pas d'information
12. Mémoires réparties et index	représentations holographiques [69], index de l'hippocampe [70]	un index ne rappelle qu'un souvenir déjà stocké : c'est un cache, déjà construit au Tome II

Table 2.3 : Les douze familles « génomiques » de l'état de l'art, lues à la source primaire [litt.]. Synthèse de `genome/00-etat-de-l-art.md` §2 ; adresses, dates de publication et de consultation : `mesures/genome/etat-de-l-art/sources.csv` et `memoire/sources/genome-etat-de-l-art.bib`.

Geste	Fait ailleurs	Sur un modèle né en 4 bits ?
sans perte sur les codes et les échelles MX	ZipNN, DFloat11, TRACE : sur BF16	troncons (03, F1 : 3,84 bits) ; rien d'autre trouvé
base emboîtée dans le code natif, complément exact	Matryoshka, Any-Precision, MatGPTQ : sur entiers, depuis le BF16	troncons (07, F1 : base fonction du code)
requantifier les experts MXFP4 publiés	Tied Trit-Planes (4,06 bits, SSD), ESTS (après élagage et réaffinage)	jamais *sous* 4 bits avec un complément exact
rétroaction d'erreur sur les entrées réelles	OBC, GPTQ, QMoE, MatGPTQ : depuis le BF16	**jamais** pour choisir une base emboîtée dans un code MXFP4
contrainte d'entropie dans la rétroaction	QMoE code l'entropie après coup ; HyperQuant code par Rice	**jamais** dans le choix des symboles d'une base native
graine, hyperréseau, automate, dictionnaire après coup	SeedLM, MoBE, Sub-MoE, D^2-MoE (BF16)	non trouvé ; chapitres 4 et 5

Table 2.4 : Ce qui a été fait ailleurs et ce qui ne l'a pas été sur un modèle né en 4 bits [litt.]. « Jamais » veut dire « non trouvé par les requêtes consignées », pas « inexistant ». Source : `genome/00-etat-de-l-art.md` § 3.

comme analogie : le code génétique est rangé pour que les erreurs ponctuelles probables tombent sur des acides aminés semblables, au point qu'il est « un sur un million » parmi les codes possibles [5] ; le neurone code un signal continu en impulsions en reportant son erreur sur la suite, à la manière d'un modulateur sigma-delta [55]. Le chapitre 3 construit ce mécanisme, la « mutation synonyme », et le mesure.

2.6 Ce que la lecture prédit

Le tableau 2.5 est le pronostic écrit le 2026-10-16, avant toute mesure de la ronde (`genome/00-etat-de-l-art.md` § 4). Il dit, pour chaque famille, la dépendance qu'elle exploite et la mesure de troncons qui la juge. Les chapitres 3 à 6 le confrontent aux nombres ; le chapitre 8 fait le bilan. Ce chapitre n'annonce aucun résultat : il fixe ce qui aurait pu les démentir.

La dernière ligne demande une précision, que la suite du tome rend nécessaire : « dépasser la borne des poids » ne veut pas dire descendre sous $R(D)$ de la figure 2.2, ce qui est impossible dans sa métrique. Cela veut dire *changer de métrique* : une base dont l'erreur sur les poids *monte* peut faire moins d'erreur à la sortie (2.8), et c'est la seule voie par laquelle un mécanisme avec perte gagne quelque chose que la borne des poids ne lui interdit pas. Les chapitres suivants montreront aussi qu'il faut aller plus loin encore, jusqu'à la divergence de Kullback-Leibler sur le texte, parce que l'erreur de sortie d'un expert trompe à son tour.

Ce qui pourrait être faux

- **Seuls des résumés ont été lus**, plus les formulations publiées des six méthodes clés : un chiffre de résumé est un chiffre d'éditeur, jamais reproduit. Un article peut contenir une expérience sur un modèle MXFP4 que son résumé ne dit pas : « jamais tenté » veut dire « non trouvé par les requêtes consignées ».
- **Les recherches n'ont porté que sur arXiv et Crossref** : des actes de conférence non déposés sur arXiv peuvent manquer.

Famille	Dépendance exploitée	Mesure qui la juge	Pronostic
graine, hyperréseau, automate	régularité des poids	0,005 bit pour le voisin (03)	aucun gain sans perte ; avec perte, au mieux la borne $R(D)$ des poids
parenté d'experts, delta	corrélation entre experts	information mutuelle entre experts (chapitre 5)	gain si et seulement si l'information mutuelle est grande
prédictif sans perte	dépendance séquentielle	0,005 bit (03)	au plus 0,005 bit
élagage	poids inutiles	sensibilité mesurée (chapitre 6)	dépend de la sortie, pas des poids
code vectoriel, dictionnaire	forme de la loi	gain de pavage, au plus 0,2546 bit	au plus 0,25 bit, décodeur neuf
mutation synonyme	**corrélation des entrées**	$H = \mathbb{E}[g^2 x x^\top]$ sur les entrées réelles (chapitre 3)	**le seul qui puisse dépasser la borne des poids**

Table 2.5 : Le pronostic de la lecture, déclaré avant les mesures de la ronde [calc.] [litt.]. Source : `genome/0-etat-de-l-art.md` § 4.

- **La quasi-indépendance des codes** vient de 03 (ordre 1, voisin dans la ligne, contexte d'échelle). Une dépendance d'ordre plus élevé, à longue portée ou entre experts n'y est pas mesurée ; les chapitres 5 et 6 la mesurent.
- **L'argument de Kolmogorov est en moyenne sur la source** : un tenseur particulier peut avoir une description courte (la graine qui l'a initialisé, par exemple) ; après l'entraînement, rien ne l'indique, et cette graine n'est pas publiée.
- **La borne de la figure 2.2** suppose les codes indépendants sachant l'échelle ; une dépendance non mesurée la ferait baisser encore. Elle est une borne supérieure par le lemme 2.1 ; la lecture des itérés non convergés comme bornes a été posée *après* la passe 3, et la règle déclarée avant ne publiait que des points convergés. Elle ne compte pas les échelles (0,0375 bit).
- **Le gain de pavage** (0,2546 bit) est une borne asymptotique à haut débit ; à 2 ou 3 bits il est plus petit, ce qui renforce le pronostic.
- **La métrique de la borne** est l'erreur poolée sur les poids, dominée par une petite part des blocs (section 3.2) : elle situe la frontière, elle ne juge aucun mécanisme.

Chapitre 3

L'éclaireur : où est le bruit, où est la structure

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la frontière du chapitre 2 : 3,8431 bits par poids sans perte, la base de 2,6441 bits à 4,29 % d'erreur sur les poids, et des codes presque indépendants sachant leur échelle (0,005 bit pour le voisin). La lecture avait désigné une case vide : choisir les symboles d'une base emboîtée par rétroaction d'erreur sur les entrées réelles, sous contrainte d'entropie, sans toucher au décodeur. **Ce chapitre établit**, avant de concevoir, où sont le bruit et la structure des poids : le signe d'un zéro est une pièce parfaite, et l'énergie se concentre dans quelques blocs, si bien que la métrique des poids n'est pas un juge. Puis deux mécanismes, chacun avec sa démonstration et sa mesure. Le premier, le *codon synonyme du zéro*, bat la frontière sans perte de valeur : 3,7061 bits au lieu de 3,8431 sur les 12 tenseurs de référence, logits de la référence f64 identiques au bit. Le second, la *mutation synonyme*, échoue d'abord par sur-apprentissage, puis, régularisée et sous contrainte d'entropie, bat la frontière avec perte au critère déclaré : 2,563 bits au lieu de 2,642, et une erreur de sortie d'expert divisée par trois à quatre sur des entrées tenues à l'écart. De bout en bout, sa variante sans contrainte d'entropie fait baisser la KL contre la référence de 27 %. **Il reste** une variante qui soit à la fois sous la frontière en bits et sous elle en KL (aucune ne l'est : la KL n'a été mesurée que sur la variante $\lambda = 0$, plus chère de 0,011 bit) ; le portage du codon du zéro dans le format, que fait le chapitre 9 ; le génome synonyme comme second fichier, que fait le chapitre 10. Sources : `genome/01-eclaireur.md` ; `genome/etat-de-l-art-pour-le-memoire.md` § 2 ; `genome/05-synthese.md` §§ 0 et 1.3 ; `mesures/genome/etat-de-l-art/journaux/` (`zero-compte-*`, `zero-canonique`, `temoin-*`, `diagnostic-L0-E0`, `choix-delta`, `kl-*`) et `passes/` ; `mesures/genome/construction/synthese/apparie-eclaireur.json` ; outils sous `outils/genome/eclaireur/`.

En bref

Une mutation synonyme change un codon sans changer la protéine (section 1.2). L'éclaireur a cherché, dans les poids de gpt-oss, l'équivalent de cette redondance, en gardant l'analogie pour ce qu'elle est : une image qui guide, pas un argument. Il en a trouvé deux. La première est dans le format lui-même : MXFP4 a seize codes pour quinze valeurs, parce que +0 et −0 sont le même nombre ; le signe d'un zéro est une information que le calcul ne lit jamais, comme une partie du génome qui ne code rien, et la retirer rend 3,6 % des octets sans changer un seul bit des logits. La seconde est dans la manière de se tromper : une erreur de poids dans une direction que les entrées réelles de l'expert n'excitent pas est invisible à sa sortie. Choisir les symboles de la base pour y ranger l'erreur coûte zéro au décodeur et divise l'erreur de sortie par trois à quatre.

Outil	Rôle	Témoin (déclaré avant)	Résultat [mes.]
`capture/` (Rust, fourche de `trc-reference`)	capture les entrées réelles des experts et le routage, substitue des bases, réécrit un code	logits de la fourche égaux à ceux de `trc-reference` au bit (24 jetons) ; cassure : YaRN sans échelle	0 logit différent ; cassure : 4 826 112 (`temoin-fourche*.json`)
`synonyme.py`	la base frontière et la mutation synonyme ; erreurs de poids et de sortie	T0 : erreur de la frontière sur les 12 tenseurs $= 0{,}042\,919\,745$ à 10^{-6} ; T1 : $H = I$, $\lambda = 0$ rend la frontière ; T2 : $H_0 = 2{,}642$ à 0,001	T0 0,042 919 745 124 98 (cassure 0,4246) ; T1 0 symbole différent (avec le H réel : 1 789 132) ; T2 2,6415 (`temoins*.json`)
`zero-synonyme/` (Rust, fourche de la mesure F1)	codeur rANS réel, décodage exact, avec ou sans $-0 \to +0$	sans le drapeau, redonne F1 au bit	toutes les grandeurs de F1 égales (A_ctx 3,843 137 ; `zero-temoin-sans-drapeau.json`)
`zero_compte.py`	comptes exacts des ±0	H(code) des 12 tenseurs $= 3{,}860$ à 0,001 ; cassure (code 1 compté comme zéro) : l'écart bouge de plus de 0,05	3,8598 ; cassure : 0,1369 → 0,3514
`borne_rd.py`	la borne $R(D)$ du section 2.3	B1 : les bouts de la courbe ; B2 : une gaussienne à 0,05 bit	passe 1 : B1 tombe ; B2 tient (10^{-5} bit) ; cassure tombe ; correction déclarée après (section 2.3)

Table 3.1 : Les instruments de l'éclaireur, leurs témoins et leurs cassures [mes.]. Source : `genome/01-eclaireur.md` § 1 ; journaux sous `mesures/genome/etat-de-l-art/journaux/`.

3.1 Les instruments et leurs témoins

Chaque instrument a son témoin, déclaré avant sa première mesure, dont la réponse est connue d'avance, et sa cassure, qui force le témoin à tomber (tableau 3.1). Les journaux sont sous `mesures/genome/etat-de-l-art/journaux/`, les outils sous `outils/genome/eclaireur/`. Trois noms du tableau viennent du Tome II : `trc-reference` est la référence exacte, le programme qui calcule gpt-oss en double précision (f64) pour juger tous les autres ; une *fourche* en est une copie modifiée pour un besoin de la ronde (ici, capturer les entrées des experts, ou leur substituer une base) ; YaRN [71] est la mise à l'échelle des fréquences de position par laquelle gpt-oss étend son contexte, que la cassure du premier témoin retire, pour vérifier que la comparaison au bit sait tomber.

Les entrées réelles. La fourche de la référence f64 a capturé, à chaque couche, l'état normalisé qui entre dans les experts (en f32) et le routage, sur wiki.test découpé par `trc-jetons` ; ses dix premières fenêtres égalent celles du fichier de logits `.kld` du jugement R1 du Tome II (vérifié par l'outil, `journaux/capture-*.json`). La **calibration** est faite des fenêtres 10 à 21 (6 132 positions), l'**évaluation tenue à l'écart** des fenêtres 0 à 9 (5 110 positions, celles de R1). Chaque position envoie son entrée à 4 experts sur 32 par couche : en moyenne 766 entrées de calibration et 639 d'évaluation par expert [mes.]. Ce nombre, petit devant la dimension 2 880, décide du premier échec de la section 3.4.

	+0 (code 0)	−0 (code 8)	H(signe | zéro)	H(code)	H(valeur)	écart
gpt-oss-20b, 72 tenseurs	6,3920 %	6,3929 %	0,999 999 996 7	3,8638	3,7360	**0,1278**
12 tenseurs de référence	6,8464 %	6,8475 %	0,999 999 99	3,8598	3,7228	0,1369
gpt-oss-120b, 108 tenseurs	6,2779 %	6,2789 %	0,999 999 995 1	3,8667	3,7411	**0,1256**

Table 3.2 : Le signe des zéros dans les codes MXFP4 des experts [mes.]. Entropies en bits par poids (le signe : en bits par zéro). 20b : 19 110 297 600 poids ; 120b : 114 661 785 600. L'écart H(code) − H(valeur) est ce qu'un codeur sans perte paie pour un signe que le calcul ne voit pas. Données : `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`, `zero-compte-120b.json`, `zero-compte-20b-12.json`.

3.2 Où est le bruit, où est la structure

3.2.1 Le signe des zéros : du bruit pur

Les comptes exacts des +0 (code 0) et des −0 (code 8) sur tous les tenseurs d'experts des deux modèles donnent le tableau 3.2 et la figure 3.1. Sur gpt-oss-20b, 12,8 % des codes sont des zéros, moitié +0, moitié −0 ; l'entropie du signe d'un zéro vaut 0,999 999 996 7 bit, à $3 \cdot 10^{-9}$ d'une pièce parfaite [mes.].

La figure 3.1 montre l'écart tenseur par tenseur : il va de 0,117 à 0,196 bit par poids sur le 20b, de 0,115 à 0,239 sur le 120b, au plus haut à la couche 0, et ne descend nulle part sous 0,11 bit. La ligne « tout le modèle » est tracée sous les barres : une première version la posait par-dessus avec un halo blanc qui cachait le sommet de 37 barres, et le juge de la troisième passe des figures l'a fait tomber (GJ1, `memoire/tome3/JUGEMENT-figures3.md` § 6) avant qu'elle soit redessinée (commit `bb01679`). Aucun bloc n'a ses 32 codes nuls (0 sur 597 196 800 pour le 20b, 0 sur 3 583 180 800 pour le 120b) : l'échelle d'un bloc n'est jamais elle-même un synonyme.

Pourquoi le signe d'un zéro est aléatoire [calc.]. Un quantificateur MXFP4 arrondit au plus près. Un poids d'origine de petite magnitude devient ±0 selon son propre signe, et les signes des petits poids sont équiprobables. Le format garde ce signe parce que E2M1 code le signe à part [8] ; le calcul ne le voit jamais, puisque $K[8] = -0 = K[0]$. C'est de l'information héritée de la quantification, le signe d'un poids BF16 qui n'existe plus, sans aucun effet sur la valeur.

3.2.2 L'énergie est concentrée

La métrique de la frontière est l'erreur relative *poolée* sur les poids. Or l'énergie des 12 tenseurs de référence est concentrée dans quelques exposants de bloc (tableau 3.3) : les blocs d'exposant 129 portent 54,8 % de l'énergie pour 5,3 % des poids ; 129 et 130 ensemble, 74,2 % pour 5,8 % ; les huit exposants les plus lourds, 96,9 % pour 14,6 % [mes.] [calc.].

La conséquence est un avertissement que toute la ronde a vérifié à ses dépens. Une allocation du débit qui ne servirait que les blocs lourds « battrait » la frontière dans la métrique poolée tout en détruisant 85 % des poids. **La métrique des poids n'est pas un juge.** D'où le juge de la section 3.4 (l'erreur de sortie des experts sur des entrées tenues à l'écart), puis celui de la section 3.5 (la KL de bout en bout).

3.2.3 La borne d'un codeur sans mémoire

Le troisième regard est la borne distorsion-débit du section 2.3 : à l'erreur de la frontière, un codeur idéal qui ne regarde que les poids lirait au plus 2,168 bits, soit 0,476 bit de moins ; ce gain est dans la

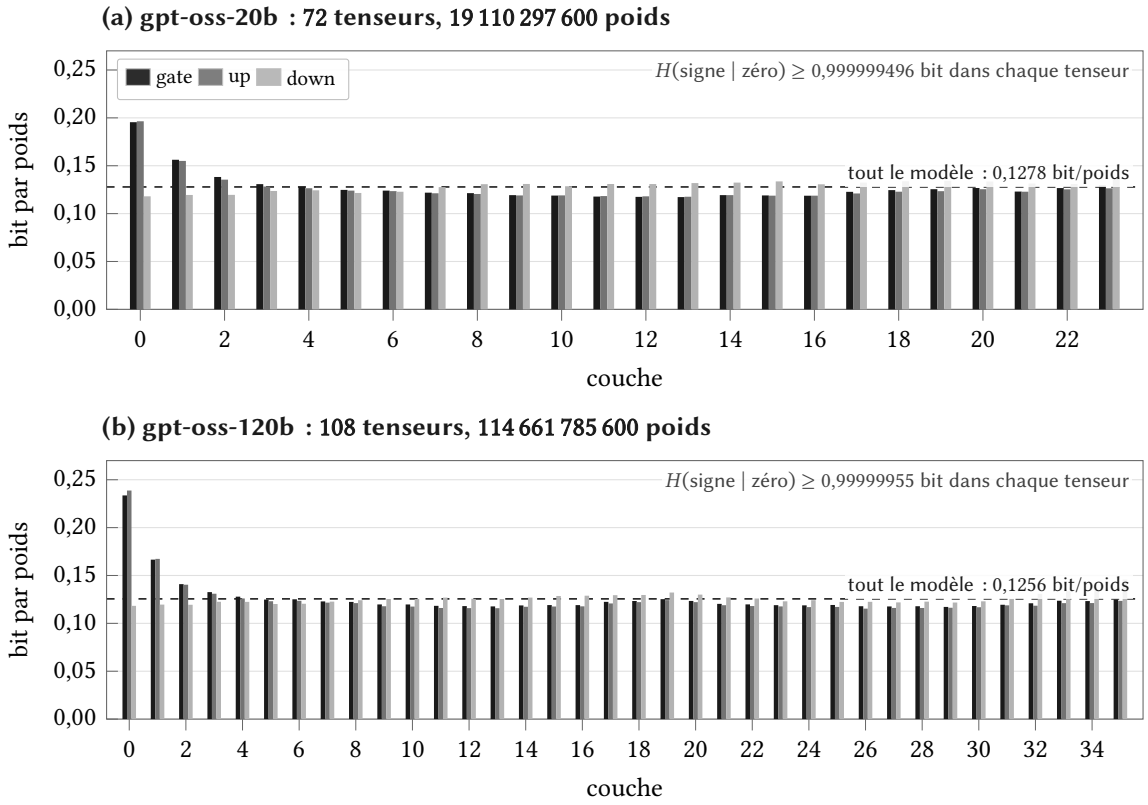


Figure 3.1 : Le signe des zéros est une pièce équilibrée qui coûte un huitième de bit. [mes.] Pour chaque tenseur d'experts, l'écart $H(\text{code}) - H(\text{valeur})$, tiré des comptes exacts des +0 (code 0) et des −0 (code 8) : ce qu'un codeur sans perte paie pour garder un signe que le calcul ne voit jamais ($K[8] = -0 = K[0]$). Sur gpt-oss-20b, les zéros font 6,3920 % de +0 et 6,3929 % de −0 parmi les poids, et $H(\text{signe} \mid \text{zéro}) = 0{,}9999999967$ bit ; par tenseur, l'écart va de 0,117 à 0,196 bit (20b) et de 0,115 à 0,239 (120b), au plus haut à la couche 0. Aucun bloc n'a ses 32 codes nuls (0 sur 597 196 800 ; 0 sur 3 583 180 800) : l'échelle n'est jamais un synonyme. Ce que la figure ne montre pas : les tenseurs d'attention et du routeur (hors MXFP4), ni le débit codé une fois −0 fondu (chapitre 9).

Données : `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`, `zero-compte-120b.json` (`par_tenseur`) ; script `memoire/outils/tome3_eclaireur.py`.

rang	exposant e	part des poids	part de l'énergie	énergie cumulée
1	129	5,29 %	54,80 %	54,8 %
2	130	0,47 %	19,37 %	74,2 %
3	128	3,44 %	8,90 %	83,1 %
4	127	5,37 %	3,48 %	86,6 %
5 à 8	133, 134, 131, 136	0,01 %	10,37 %	96,9 %

Table 3.3 : L'énergie des poids des 12 tenseurs de référence, par exposant E8M0 de bloc, rangée par énergie décroissante [mes.] [calc.]. Énergie d'un exposant : nombre de poids $\times\, 4^{e-128}$, rapporté au total (la loi des codes est la même d'un exposant à l'autre au premier ordre). Données : `mesures/genome/etat-de-l-art/journaux/borne-rd-passe1.json`, champ `exposants` (3 185 049 600 poids) ; la figure cumulative proposée (III-2.2) n'est pas tracée.

forme du quantificateur, pas dans une structure des poids. Les trois regards disent la même chose : vu de lui-même, un poids MXFP4 est du bruit autour de son échelle. Les deux mécanismes qui suivent n'ajoutent donc aucune structure aux poids : le premier retire ce que le calcul ne lit pas, le second déplace l'erreur là où les entrées ne regardent pas.

3.3 Mécanisme 1 : le codon synonyme du zéro

3.3.1 Le mécanisme

Avant le codage entropique, chaque demi-octet 8 (−0) devient 0 (+0). Le décodeur ne change pas : il rend le code canonique, et la table du flux n'a plus de fréquence pour 8 (dans l'alphabet octet de A_ctx, les 256 symboles deviennent 225 possibles, 15×15). Le coût de transcription est nul. L'analogie est celle de l'ADN non codant : une partie du fichier, 0,128 bit par poids, porte un signe que personne ne lit.

3.3.2 Pourquoi les produits ne changent pas

Proposition 3.1 (Exactitude au bit d'un produit scalaire après $-0 \to +0$)**.** *Soit* $y = \mathrm{fl}(\cdots \mathrm{fl}(\mathrm{fl}(t_1 + t_2) + t_3) \cdots + t_n)$ *une somme IEEE 754 en arrondi au plus près [72], dans un ordre fixé, de termes* $t_j = \mathrm{fl}(w_j x_j)$ *avec des* x_j *finis, et* w' *égal à* w *sauf* $w'_j = +0$ *là où* $w_j = -0$*. Si au moins un terme* t_j *est non nul, alors* $y' = y$ *au bit. Si tous sont nuls,* $y, y' \in \{+0, -0\}$ *et* $y = y'$ *en valeur.*

Démonstration. On a $t'_j = t_j$ si $w_j \neq -0$, et $t_j, t'_j \in \{+0, -0\}$ sinon, puisque $(\pm 0) \cdot x_j = \pm 0$ pour x_j fini. Soit S_k et S'_k les sommes partielles. Hypothèse de récurrence : $S_k = S'_k$ au bit, ou bien $S_k, S'_k \in \{\pm 0\}$. Elle vaut pour $k = 1$. Au pas $k + 1$, trois cas. (i) Si $t_{k+1} = t'_{k+1}$ est non nul : quand $S_k = S'_k$, les opérandes sont égaux et la somme aussi ; quand $S_k, S'_k \in \{\pm 0\}$, $\mathrm{fl}(\pm 0 + t) = t$ exactement pour $t \neq 0$, donc $S_{k+1} = S'_{k+1} = t_{k+1}$. (ii) Si $t_{k+1}, t'_{k+1} \in \{\pm 0\}$ et $S_k = S'_k \neq 0$: $\mathrm{fl}(S \pm 0) = S$ des deux côtés. (iii) Si les deux sommes partielles et les deux termes sont nuls, les deux sommes restent dans $\{\pm 0\}$: seul le signe d'un zéro peut différer, puisque $(+0) + (-0) = +0$ mais $(-0) + (-0) = -0$. Au premier terme non nul, (i) recolle les deux suites, et (ii) les garde égales ensuite. Si aucun terme n'est non nul, seul (iii) s'applique. ■

Dans un noyau entier (valeurs E2M1 doublées en entiers signés), −0 et +0 sont déjà le même entier. Le cas d'un produit de 2 880 termes tous nuls n'arrive pas dans gpt-oss, où aucun bloc n'est nul ; s'il arrivait, seul le signe d'un zéro de sortie changerait, puis disparaîtrait au biais, non nul, de l'expert. Pour la référence f64, la même récurrence s'applique à chacun de ses huit accumulateurs, puis à leur réduction en arbre.

La proposition laisse ouvert le signe d'une somme partielle nulle. La *proposition A1* du chapitre 9 le ferme pour le noyau GEMV du moteur : un accumulateur qui part de +0 et avance par FMA en arrondi au plus près ne vaut jamais −0 (une somme exacte nulle de deux termes qui ne sont pas tous deux −0 s'arrondit à +0), si bien qu'un pas sur un code nul y est le même dans les deux calculs, et que l'égalité au bit vaut à chaque pas, même pour une somme entièrement nulle. La proposition de ce chapitre suffit à la référence ; A1 est ce qu'il faut au moteur.

3.3.3 Les mesures

Le seuil était déclaré avant la passe, en tête de `outils/genome/eclaireur/zero-synonyme/src/main.rs` : le codon bat la frontière si A_ctx descend d'au moins 0,05 bit par poids. Le tableau 3.4 donne les mesures, toutes par le codeur rANS réel de F1, chaque flux décodé exactement.

Mesure (12 tenseurs de référence)	sans le codon (F1)	avec	écart
A_ctx, tout compris (bits par poids)	3,843 137	**3,706 142**	−0,136 995 (−3,56 %)
codes A_ctx seuls	3,805 616	3,668 621	−0,136 995
A_flux (format de l'étude 03)	3,843 442	3,706 406	−0,137 036
tronçon moyen (20b, aligné sur 4 Kio)	11 990 368 o	11 564 160 o	−3,55 %
base seule (niveau 0 de B)	2,606 647	2,559 045	−0,047 602
demi-octets réécrits	—	218 097 443 sur 3 185 049 600	6,85 %

Table 3.4 : Le codon synonyme du zéro, mesuré par le codeur rANS réel [mes.]. Données : `mesures/genome/etat-de-l-art/journaux/zero-canonique.json` et `zero-temoin-sans-drapeau.json` ; `genome/01-eclaireur.md` § 4.3. Le codeur rend l'écart d'entropie à 0,000 04 bit près (0,136 995 contre 0,1369).

Mesuré : 0,137 bit, pour un seuil de 0,05. Le codon synonyme du zéro bat la frontière sans perte. Sur les 72 tenseurs d'experts du 20b, l'écart d'entropie vaut 0,1278 bit ; le chapitre suivant de la ronde le mesure aussi au codec réel sur les deux modèles entiers (section 6.1).

La preuve au bit. Sur 24 jetons de la référence f64, avec les 759 416 633 demi-octets −0 réécrits en +0 dans tous les experts lus, **0 logit sur 4 826 112** diffère au bit, écart absolu maximal 0 (`journaux/temoin-zero.json`) ; la cassure fait différer les 4 826 112 (`temoin-zero-casse.json`). De bout en bout sur les 2 550 positions des dix fenêtres de R1, le fichier de logits du modèle canonique est **identique octet pour octet** à celui de la référence : même empreinte SHA-256, `d97fc421…5362a38` (`journaux/kl-zero-canonique.sha256`) [mes.].

Sur gpt-oss-120b [calc.]. 0,1256 bit par poids sur 114 661 785 600 poids d'experts font

$$114\,661\,785\,600 \times 0{,}1256/8 = 1{,}800 \cdot 10^9 \text{ octets :}$$

1,80 Go qu'aucun calcul ne lit, sur les 60,9 Go du fichier sans perte (03), soit −3,0 %. Le codeur réel n'a été passé par l'éclaireur que sur le 20b ; le chapitre 6 l'a passé sur le 120b entier.

Le toit [calc.]. Les octets d'experts lus par jeton baissent de 3,56 % ; à disque limitant, le toit (2.1) monte de $1/(1-0{,}0356)-1 = 3{,}7$ %, sans rien changer d'autre. Le chapitre 9 mesure ce que le moteur en fait sur le disque de Stone, et pourquoi le chiffre mesuré n'est pas celui-là.

3.3.4 Le même codon dans la base

Dans la base (signe et classe), un zéro garde son signe : la classe {0; 0,5} se reconstruit en $\pm 0{,}237\,s$. Sur le code canonique, ce signe n'est plus stocké (−0,0476 bit, tableau 3.4) ; le décodeur peut le *tirer d'un hachage de la position* (`splitmix64` de $rC+c$) sans lire un bit. L'erreur sur les poids est la même au poids près, puisque $|0-(\pm 0{,}237\,s)|$ ne dépend pas du signe. Une suite de Weyl comme hachage échoue (des rangées voisines presque égales) ; `splitmix64` tient (corrélation 0,01, au niveau du bruit). L'erreur de *sortie* de cette variante n'a pas été mesurée : sa passe a été arrêtée après 4 experts (`passes/frontiere0-ARRETEE-4-experts.log`). Elle reste « égale en erreur sur les poids à −0,048 bit, non jugée sur la sortie ».

3.4 Mécanisme 2 : la mutation synonyme

3.4.1 Le vivant, et ce qu'on en transpose

Le code génétique n'est pas une fonction quelconque des codons vers les acides aminés : parmi un million de codes possibles, le code naturel est l'un des meilleurs pour que les erreurs probables tombent sur des acides aminés semblables [5]. Le neurone, lui, code un signal continu en impulsions en reportant son erreur sur la suite, à la manière d'un modulateur sigma-delta [55]. Transposé, comme analogie : le « phénotype » d'une matrice d'expert est sa sortie Wx sur les entrées que le modèle lui donne vraiment ; une erreur de poids dans une direction que ces entrées n'excitent pas est une mutation synonyme. Le mécanisme choisit les symboles de la base pour que l'erreur y aille.

3.4.2 La dérivation

L'erreur de sortie. Pour une matrice W de R lignes et C colonnes, ses entrées x et le poids de routeur g de l'expert, l'erreur de sortie d'une reconstruction $\hat{W}$ vaut, par (2.8),

$$\mathbb{E}\big[g^2\|(W-\hat{W})x\|^2\big] = \mathbb{E}[g^2]\,\operatorname{tr}\big((W-\hat{W})\,H\,(W-\hat{W})^\top\big), \qquad H = \frac{\mathbb{E}[g^2 xx^\top]}{\mathbb{E}[g^2]}. \tag{3.1}$$

La trace est une somme sur les lignes : les lignes de W se traitent indépendamment. Pour une ligne w, on cherche $\min_q (w-q)^\top H (w-q)$, q parcourant les reconstructions permises colonne par colonne, $q_j = \pm c_k\, s_{b(j)}$, c_k l'un des quatre centres de la base et $s_{b(j)}$ l'échelle E8M0 du bloc de la colonne j.

Lemme 3.2 (Rétroaction d'erreur optimale)**.** *Soit w la ligne courante (erreur déjà reportée comprise), j une colonne et F l'ensemble des colonnes encore libres, $j \in F$, H_F la sous-matrice de H sur F. Si l'on fixe la colonne j à q_j et qu'on laisse les autres colonnes de F s'ajuster, l'accroissement minimal de l'erreur quadratique $\delta^\top H_F \delta$ est*

$$\frac{(w_j-q_j)^2}{[H_F^{-1}]_{jj}}, \qquad \text{atteint par } \delta = -\frac{w_j-q_j}{[H_F^{-1}]_{jj}}\, H_F^{-1} e_j, \tag{3.2}$$

où δ est la modification de w sur F ; sur les colonnes libres autres que j, la correction vaut $\delta_{F\setminus j} = -(w_j-q_j)\,[H_F^{-1}]_{F\setminus j,\,j}/[H_F^{-1}]_{jj}$.

Démonstration. On minimise $\delta^\top H_F \delta$ sous la contrainte $e_j^\top(w+\delta) = q_j$, c'est-à-dire $e_j^\top \delta = q_j - w_j$. Le lagrangien est $L(\delta,\mu) = \delta^\top H_F \delta + \mu\,(e_j^\top\delta - (q_j-w_j))$. Son gradient en δ, $2H_F\delta + \mu e_j$, s'annule en $\delta = -\frac{\mu}{2} H_F^{-1} e_j$. La contrainte donne $-\frac{\mu}{2}\,[H_F^{-1}]_{jj} = q_j - w_j$, soit $\frac{\mu}{2} = -\frac{q_j-w_j}{[H_F^{-1}]_{jj}}$, d'où $\delta = \frac{q_j-w_j}{[H_F^{-1}]_{jj}} H_F^{-1} e_j$, qui est l'expression annoncée. L'accroissement vaut $\delta^\top H_F \delta = \frac{(q_j-w_j)^2}{[H_F^{-1}]_{jj}^2}\, e_j^\top H_F^{-1} H_F H_F^{-1} e_j = \frac{(q_j-w_j)^2}{[H_F^{-1}]_{jj}}$. H_F étant définie positive, c'est un minimum. C'est le lemme d'Optimal Brain Surgeon [40], appliqué à la quantification par OBC et GPTQ [50, 51]. ∎

La règle séquentielle, par un seul facteur. Dans l'ordre des colonnes $1,\dots,C$, l'ensemble libre au pas j est $F_j = \{j,\dots,C\}$. Les quantités $[H_{F_j}^{-1}]_{jj}$ et $[H_{F_j}^{-1}]_{F_j j}$ sont lues dans le facteur de Cholesky supérieur U de H^{-1} ($U^\top U = H^{-1}$) : $[H_{F_j}^{-1}]_{jj} = U_{jj}^2$, et la correction des colonnes suivantes vaut $-\frac{w_j-q_j}{U_{jj}}\,U_{j,F_j}$ (GPTQ). Le facteur se calcule par une seule factorisation.

Lemme 3.3 (Le facteur de Cholesky de H^{-1} par renversement)**.** *Soit P la permutation qui renverse l'ordre des colonnes. Si $PHP = LL^\top$, L triangulaire inférieure à diagonale positive, alors $U = PL^{-1}P$ est triangulaire supérieure à diagonale positive et $U^\top U = H^{-1}$: c'est le facteur de Cholesky supérieur de H^{-1}.*

Démonstration. $P = P^\top = P^{-1}$. De $PHP = LL^\top$ vient $H = (PLP)(PLP)^\top = VV^\top$ avec $V = PLP$, triangulaire *supérieure* (renverser l'ordre des lignes et des colonnes d'une matrice triangulaire inférieure la rend supérieure). Alors $H^{-1} = V^{-\top}V^{-1} = U^\top U$ avec $U = V^{-1} = PL^{-1}P$, triangulaire supérieure comme inverse d'une triangulaire supérieure, de diagonale $1/L_{C+1-i,C+1-i} > 0$. Le facteur de Cholesky à diagonale positive est unique : c'est U. ■

Le coût est $n^3/3$ (Cholesky) plus $n^3/3$ (inverse triangulaire), au lieu de n^3 (inverse) plus $n^3/3$ [calc.] ; l'outil l'a vérifié contre le calcul direct, écart relatif $7 \cdot 10^{-14}$ (`outils/genome/eclaireur/synonyme.py`, `facteur_inverse`) [mes.].

La contrainte d'entropie. Le symbole k de la colonne j minimise

$$\left(|w_j| - c_k\, s_{b(j)}\right)^2 + \lambda\, \bar{c}\, U_{jj}^2\, \ell(k), \qquad \ell(k) = -\log_2 p(k), \tag{3.3}$$

p la loi des symboles de la base frontière du tenseur, $\bar{c}$ l'erreur moyenne par poids de la frontière dans les unités de H. Divisé par U_{jj}^2, c'est, par (3.2), « accroissement de l'erreur de sortie $+\,\lambda\,\bar{c}$ bits » : λ est le prix d'un bit en unités d'erreur de frontière. Avec $\lambda = 0$, c'est la rétroaction seule.

Le décodeur ne change pas. Mêmes huit symboles, mêmes quatre centres, mêmes échelles : le mécanisme vit tout entier dans le codeur, et son coût de transcription est nul.

Proposition 3.4 (Le témoin qui sait tomber : $H = I$ rend la frontière)**.** *Si $H = I$ et $\lambda = 0$, la mutation synonyme choisit, pour chaque poids, le symbole de la base frontière.*

Démonstration. Avec $H = I$, $H^{-1} = I$ et $U = I$: aucune correction n'est reportée, et chaque poids choisit le centre le plus proche de $|w_j|/s_{b(j)}$, avec le signe de w_j. Les centres sont les centres de gravité d'une partition *contiguë* des magnitudes, {0; 0,5}, {1; 1,5; 2}, {3; 4}, {6}, de centres 0,237 ; 1,424 ; 3,419 ; 6. Les frontières de Voronoï, milieux de deux centres voisins, sont 0,831 ; 2,421 ; 4,709 [calc.]. Chaque magnitude tombe dans la cellule de sa classe : $0{,}5 < 0{,}831 < 1$, $2 < 2{,}421 < 3$, $4 < 4{,}709 < 6$. Le symbole choisi est donc celui de la fonction frontière. ■

Mesuré (témoin T1) : 0 symbole différent sur 8 294 400 ; avec le H réel, 1 789 132 [mes.].

3.4.3 Le protocole

Experts : couches 0, 8, 16 et 23, experts 0, 4, …, 28 (96 matrices, 796 millions de poids) ; des passes plus tardives sur des sous-ensembles déclarés avant elles. Calibration : fenêtres 10 à 21 ; évaluation tenue à l'écart : fenêtres 0 à 9 ; erreurs de sortie pondérées par g^2. L'erreur de sortie d'*expert* remplace les trois matrices à la fois, avec entre elles la SwiGLU bornée de gpt-oss. **Critère, déclaré avant** (tête de `synonyme.py`) : débit au plus celui de la frontière, au même estimateur, *et* erreur de sortie d'expert plus basse sur chacune des deux moitiés de l'évaluation et dans chacune des quatre couches.

3.4.4 Le premier échec, et son pourquoi : le sur-apprentissage

La passe déclarée ($\lambda = 0$, $\delta = 0{,}01$, l'amortissement de GPTQ : $H + \delta\,\overline{\operatorname{diag} H}\, I$) **ne bat pas** la frontière : débit 2,6789 bits (H_0) contre 2,6381, et, sur l'évaluation, la couche 0 est pire (0,094 et 0,067 d'erreur de sortie d'expert, moitiés 1 et 2, contre 0,059 et 0,052 ; `passes/synonyme-lam0-damp0.01.json`) [mes.].

Le diagnostic, hypothèse et prédictions déclarées avant (`journaux/diagnostic-L0-E0.json`), l'explique par un nombre. Chaque expert ne reçoit qu'environ 766 entrées de calibration en dimension

Matrice gate, couche 0, expert 0	entrées de calibration	erreur de sortie, calibration	erreur de sortie, évaluation
base frontière	—	3,40 %	3,49 %
synonyme, $\delta = 0{,}01$	762	0,21 %	4,38 %
synonyme, $\delta = 0{,}1$	762	0,28 %	3,27 %
synonyme, $\delta = 1$	762	0,72 %	2,74 %
synonyme, $\delta = 10$	762	1,70 %	2,82 %
synonyme, $\delta = 0{,}01$, demi-calibration	381	1,61 %	4,01 %
synonyme, $\delta = 1$, demi-calibration	381	1,39 %	2,82 %

Table 3.5 : Le diagnostic du sur-apprentissage, matrice gate de l'expert 0, couche 0 (499 positions d'évaluation) [mes.]. Données : `mesures/genome/etat-de-l-art/journaux/diagnostic-L0-E0.json` (la figure proposée III-2.3 n'est pas tracée).

2 880 : la covariance estimée est de rang au plus 766, et la rétroaction pousse l'erreur dans les quelque 2 100 directions que l'échantillon ne voit pas, mais que l'évaluation excite. Le tableau 3.5 le montre sur la matrice gate de l'expert 0, couche 0.

Les prédictions (1) et (2) tiennent : l'écart entre calibration et évaluation existe, et il se resserre quand δ croît. La prédiction (3) **tombe** : avec la moitié des entrées, l'évaluation n'empire pas (4,01 % contre 4,38 % à $\delta = 0{,}01$; 2,82 % contre 2,74 % à $\delta = 1$). Le défaut n'est donc pas d'abord le nombre d'entrées, c'est le manque de régularisation de H : à $\delta = 0{,}01$, la rétroaction fait confiance à des directions mal estimées, quelle que soit la taille de l'échantillon dans cette gamme. Sur les 96 matrices, le rapport médian synonyme sur frontière vaut 0,044 en calibration et 0,84 en évaluation. *Le synonyme d'un échantillon n'est pas le synonyme de la loi.*

Le choix de δ sans regarder l'évaluation. Procédure déclarée avant (`journaux/choix-delta.json`) : l'expert 2 des quatre couches, hors des experts mesurés ; la calibration coupée en deux (fenêtres 10 à 15 pour ajuster, 16 à 21 pour juger) ; $\delta \in \{0{,}1;\ 0{,}3;\ 1;\ 3;\ 10\}$. Rapport moyen d'erreur de sortie, synonyme sur frontière : 0,623 ; 0,563 ; **0,538** ; 0,553 ; 0,589 [mes.]. D'où $\delta^* = 1 : H + \overline{\operatorname{diag} H}\, I$, la covariance mesurée rapprochée de l'isotropie à poids égal.

3.4.5 Les résultats

$\lambda = 0$. L'erreur de sortie baisse partout (tableau 3.6), mais le débit monte de 0,011 bit : au critère strict déclaré (débit au plus celui de la frontière), $\lambda = 0$ ne bat pas encore. D'où la contrainte d'entropie.

$\lambda = 1$. Au critère déclaré avant (tableau 3.7), **8 cases sur 8** (deux moitiés, quatre couches), à 0,078 bit de moins : **la mutation synonyme bat la frontière avec perte**. La marge la plus mince est la couche 0 (×0,97 sur la moitié 1).

Ce que cela dit de la structure. L'erreur sur les poids *monte* (de 4,3 à 4,9 %) pendant que l'erreur de sortie est divisée par trois à quatre. La structure exploitable n'est pas dans les poids, elle est dans les *entrées* ; les poids sont du bruit vus d'eux-mêmes, et une base dont l'erreur est rangée dans les directions muettes des entrées en coûte moins. C'est la conclusion du section 2.6, mesurée.

	Frontière	Synonyme $\lambda = 0$	Rapport
débit H_0 des symboles (bits)	2,6381	2,6488	+0,011
débit H(symbole | κ)	2,5953	2,6079	+0,013
erreur relative sur les poids (gate, up, down)	4,40 / 4,25 / 4,40 %	4,71 / 4,55 / 4,76 %	×1,07
erreur de sortie des matrices, évaluation	3,16 / 3,60 / 2,89 %	1,72 / 1,99 / 2,21 %	médiane ×0,59
erreur de sortie d'expert, moitiés 1 / 2	3,03 / 2,53 %	**0,84 / 0,65 %**	×0,28 / ×0,26
couche 0	5,85 / 5,19 %	5,10 / 4,12 %	
couche 8	13,57 / 15,19 %	5,96 / 6,62 %	
couche 16	7,87 / 8,47 %	3,65 / 3,69 %	
couche 23	3,01 / 2,51 %	0,83 / 0,64 %	
experts améliorés	—	32 sur 32	médiane ×0,67

Table 3.6 : La mutation synonyme, $\lambda = 0$, $\delta^* = 1$, 8 experts par couche [mes.]. Données : `mesures/genome/etat-de-l-art/passes/synonyme-lam0-damp1.json` et `.csv`.

	Frontière (mêmes experts)	**Synonyme** $\lambda = 1$	Écart
débit H_0 des symboles (bits)	2,6415	**2,5634**	**−0,078**
débit H(symbole | κ)	2,5989	2,5232	−0,076
débit lu, H_0 + échelles (0,0375)	2,679	**2,601**	−0,078
erreur relative sur les poids	4,26 / 4,20 / 4,44 %	4,83 / 4,59 / 5,20 %	plus haute
erreur de sortie des matrices, évaluation	3,18 / 3,48 / 2,72 %	1,78 / 1,94 / 2,09 %	médiane ×0,60 (43 sur 48)
erreur de sortie d'expert, moitiés 1 / 2	3,94 / 3,14 %	**1,18 / 0,74 %**	×0,30 / ×0,24
couche 0	5,50 / 4,87 %	5,32 / 4,29 %	×0,97 / ×0,88
couche 8	26,80 / 25,11 %	10,70 / 10,44 %	×0,40 / ×0,42
couche 16	4,92 / 5,14 %	2,98 / 3,11 %	×0,61 / ×0,61
couche 23	3,92 / 3,12 %	1,17 / 0,73 %	×0,30 / ×0,23
experts améliorés	—	14 sur 16	médiane ×0,73, pire ×1,10
complément exact H(code | symbole)	1,219	1,689	+0,47

Table 3.7 : La mutation synonyme, $\lambda = 1$, $\delta^* = 1$, experts 0, 8, 16 et 24 de chaque couche (sous-ensemble déclaré avant la passe), contre la frontière refaite sur les mêmes experts [mes.]. Données : `passes/synonyme-lam1-damp1-e0_8_16_24.json`, `passes/frontiere-lam0-damp0.01-e0_8_16_24.json`.

Le prix, en un chiffre. Le complément exact derrière la base coûte 1,689 bit au lieu de 1,219 : la base n'est plus une fonction du code ($H(\text{base} \mid \text{code}) > 0$). Lu en deux niveaux, l'expert exact coûterait $2{,}564 + 1{,}689 = 4{,}25$ bits au lieu de 3,86 [calc.] : *la mutation synonyme n'est pas un niveau 0 de la découpe base + complément*. Son usage est celui d'un **second génome** : un fichier exact (le code canonique, 3,706 bits) et une base synonyme séparée (2,60 bits lus), le disque payant la place des deux (environ 6,3 bits par poids stockés), le moteur n'en lisant qu'un par défaut d'expert. C'est le « mode rapide et mode exact » de l'idée de Stone ; le chapitre 10 le construit.

L'expert aberrant. Couche 8, expert 24 : erreurs de matrices modestes (frontière de 2,4 à 3,9 %), mais erreur de sortie d'expert de **67,6 %** ; synonyme $\lambda = 0$: 18,9 % [mes.]. La sortie de cet expert est petite devant ses termes intermédiaires (compensation entre la SwiGLU et down) : une petite erreur relative en amont devient grande en aval. Il pèse lourd dans la couche 8 et il est le premier candidat à rester exact.

ECSQ sans rétroaction, et la piste à 1 bit. Un quantificateur scalaire à contrainte d'entropie sans rétroaction ($\lambda = 0{,}5$) glisse le long de la courbe : 2,615 bits (H_0), erreur de sortie d'expert 3,18 / 2,72 % contre 3,03 / 2,53 ; il ne bat pas (`passes/ecsq-lam0.5-damp0.01.json`). La mutation synonyme à un bit de magnitude (4 symboles, environ 1,96 bit) a été lancée puis *arrêtée après 2 experts* pour laisser la machine à la KL : 20,5 et 19,5 % d'erreur de sortie d'expert (couche 0, experts 0 et 8) contre 25,7 et 25,1 % pour la frontière à 1 bit sur les mêmes experts (`passes/synonyme1-ARRETEE-2-experts.log`). Deux experts ne font pas une mesure : elle est à refaire en entier.

3.5 Le jugement de bout en bout : la KL contre la référence f64

Le juge, en cinq phrases. À chaque position t d'un texte, un modèle de langue rend une loi de probabilité sur le jeton suivant ; soient p_t celle de la référence f64 du lot R du Tome II (le modèle exact, calculé en double précision) et q_t celle du modèle dont on a remplacé des experts. La **KL de la position** est la divergence de Kullback-Leibler [11]

$$\mathrm{KL}_t = \sum_i p_{t,i} \log \frac{p_{t,i}}{q_{t,i}} \quad \text{(en nats)}, \tag{3.4}$$

nulle quand les deux lois sont égales, et sinon égale, au terme de coupe près, au surcoût moyen, en nats par jeton, de prédire avec q_t un texte tiré de p_t. L'instrument `juge kl` ne somme que les jetons i tels que $\log p_{t,i} > -16$: ceux auxquels la référence donne moins de $e^{-16} \approx 1{,}1 \cdot 10^{-7}$ [calc.] sont laissés de côté, partout et pour tous les modèles ; sans renormalisation, la somme ainsi coupée peut être très légèrement négative. La **KL d'une passe** est la moyenne de KL_t sur les positions notées ; ici, sur les dix fenêtres d'évaluation de wiki.test que le Tome II appelle R1, soit 2 550 positions (255 par fenêtre). C'est le juge de tout le tome : une erreur sur les poids ou à la sortie d'un expert n'en est qu'un indice, et la définition 8.1 la reprend avec ses deux parts, la part lisse et la queue des bascules de routage (section 8.1).

Le protocole (P1 de la ronde). La sous-commande `kl` de la fourche, puis `juge kl` du lot R du Tome II : les 10 fenêtres de R1 (2 550 positions notées), la référence f64, les experts 0, 4, …, 28 des couches 0, 8, 16 et 23 remplacés par leur base, soit **32 experts sur 768**, chacun servi à chaque position qui le route, contre `ref-openai-20b.kld`. La divergence [11] se lit position par position, en nats ;

2 550 positions, 32 experts sur 768 remplacés	Base frontière	**Synonyme $\lambda = 0$**	Écart
KL moyenne (± incertitude du juge)	0,0360 ± 0,0029	**0,0262 ± 0,0026**	−27 %
KL médiane	0,0088	**0,0038**	−56 %
KL, 90e centile	0,061	0,037	−40 %
KL, 99e centile	0,495	0,410	−17 %
KL, 99,9^{e} centile	1,59	1,70	+7 %
premier choix identique (Wilson 95 %)	87,1 % [85,8 ; 88,4]	**90,9 % [89,7 ; 91,9]**	disjoints
perplexité (référence 460,2)	485,9 (+5,4 %)	**468,3 (+1,8 %)**	÷3

Table 3.8 : La KL de bout en bout contre la référence f64 (protocole P1) [mes.]. Données : `mesures/genome/etat-de-l-art/journaux/kl-frontiere-juge.json`, `kl-synonyme-d1-juge.json` ; position par position : `kl-*-positions.csv`.

le premier choix identique se donne avec son intervalle de Wilson [73],

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad z = 1{,}96; \tag{3.5}$$

pour $\hat{p} = 0{,}8714$ et $n = 2\,550$, il donne $[0{,}8578;\ 0{,}8838]$ [calc.], l'intervalle que porte le journal. Seuil, déclaré avant : la mutation synonyme est meilleure de bout en bout si sa KL moyenne est sous celle de la frontière *et* si son taux de premier choix identique est au-dessus.

Quelle variante a été jugée. La passe a été écrite avant le résultat de $\lambda = 1$: elle juge la variante $\lambda = 0$, $\delta^* = 1$, à débit +0,011 bit, dont les erreurs de sortie sont du même ordre. La KL a été mesurée sur la variante $\lambda = 0$, à 2,649 bits ; $\lambda = 1$ (2,563 bits) n'a été jugée qu'à l'erreur de sortie d'expert. La synthèse de la ronde a dû corriger une attribution de cette KL à $\lambda = 1$ (section 8.2). **Aucune variante de la mutation synonyme n'est donc mesurée à la fois sous la frontière en bits et sous elle en KL.**

Le seuil déclaré tient (tableau 3.8) : KL moyenne plus basse, premier choix plus haut, intervalles de Wilson disjoints. L'éclaireur avait comparé les deux moyennes sans les apparier (écart de 0,0098, environ 3,6 fois l'incertitude d'une moyenne). La synthèse les a appariées position par position, depuis les CSV commités (`mesures/genome/construction/synthese/apparie-eclaireur.json`) [calc.] sur [mes.]. Son témoin, déclaré avant, recalcule chaque KL moyenne à 10^{-12} près (0,036 013 763 948 des deux côtés, écart $1{,}8{\cdot}10^{-13}$) et sa cassure (une ligne retirée) tombe. La différence appariée, synonyme moins base, vaut −0,009 845 ; écart-type naïf 0,001 90 (5,2 σ), écart-type par blocs de fenêtres, qui respecte la corrélation des positions d'une même fenêtre (elles partagent leur passé, donc leurs bascules), **0,003 34, soit 2,9 σ**. 2 060 positions sur 2 550 (80,8 %) ont moins de KL avec la mutation synonyme ; le premier choix identique monte de 3,73 ± 0,59 points. La dérivation des deux incertitudes est au chapitre 11.

Ce que la queue dit. Le 99,9^{e} centile ne baisse pas (1,59 → 1,70). Les positions les plus touchées sont celles où le routage bascule (le jugement R1 du Tome II : les bascules de routage sont le premier amplificateur d'une perturbation), et une base meilleure ne les supprime pas. Le chapitre 8 en fait la seconde part de la KL, et le chapitre 12 la mesure.

Mécanisme	Bits par poids lus	Erreur	Verdict au seuil déclaré
frontière sans perte (F1)	3,8431	0	référence
codon synonyme du zéro	**3,7061** (rANS réel, 12 tenseurs) ; −0,128 sur tout le 20b, −0,126 sur tout le 120b	**0** : logits identiques au bit ; `.kld` égal octet pour octet	**bat la frontière sans perte** (seuil 0,05)
base frontière (07, F1)	2,6441 (rANS) ; 2,679 (H_0 + échelles)	poids 4,29 % ; sortie d'expert 3,9 / 3,1 % (16 experts)	référence
base frontière sur code canonique, signe haché	2,5965 (rANS réel)	poids 4,29 % ; sortie non mesurée	égale en poids à −0,048 bit ; non jugée
ECSQ $\lambda = 0{,}5$, sans rétroaction	2,615 (H_0)	sortie d'expert 3,18 / 2,72 % contre 3,03 / 2,53	ne bat pas
synonyme $\lambda = 0$, $\delta = 0{,}01$ (déclarée)	2,679 (H_0)	sortie d'expert 1,03 / 0,79 % ; couche 0 pire	**ne bat pas**
synonyme $\lambda = 0$, $\delta^* = 1$	2,649 (H_0)	sortie d'expert 0,84 / 0,65 % ; KL 0,0262 contre 0,0360	bat en erreur et en KL ; pas au critère strict (+0,011 bit)
synonyme $\lambda = 1$, $\delta^* = 1$	**2,563 (H_0),** −0,078	**sortie d'expert 1,18 / 0,74 % contre 3,94 / 3,14 %**, 8 cases sur 8 ; KL non mesurée	**bat la frontière avec perte**

Table 3.9 : Le verdict de l'éclaireur [mes.]. Source : `genome/01-eclaireur.md` § 7, avec la correction de `genome/05-synthese.md` § 0 (la KL appartient à $\lambda = 0$).

Ce que la KL dit de la frontière elle-même. Remplacer 32 experts sur 768 (4 % des experts, dans 4 couches sur 24) par la base de 2,64 bits coûte déjà une KL de 0,036, six fois le seuil de l'architecture B (0,0055, fondations/07 § 4.2) ; la mutation synonyme la ramène à 0,026, encore quatre fois au-dessus. La base, même synonyme, est un **mode rapide** ou un **brouillon**, pas un remplaçant silencieux du mode exact.

3.6 Le verdict

Vu des poids, gpt-oss est du bruit : codes indépendants à 0,005 bit près, signe des zéros à 1,0000 bit d'entropie, et une borne distorsion-débit que seuls un meilleur quantificateur ou un code vectoriel approchent, sans rien à « générer ». Les deux mécanismes qui gagnent (tableau 3.9) n'ajoutent aucune structure aux poids : le premier **retire de l'information que le calcul ne lit jamais**, le second **déplace l'erreur là où les entrées réelles ne la voient pas**. Les deux sont, au sens de l'analogie, des synonymes : des différences de code sans différence de phénotype. La suite de la ronde les a pris tous les deux : le premier entre dans le format (chapitre 9), le second devient un second fichier (chapitre 10).

Ce qui pourrait être faux

- **Le codon du zéro et l'empreinte des experts.** Le format `.trc` vérifie l'empreinte de chaque expert décodé contre les octets du GGUF ; un expert canonique a d'autres octets. Il faut une empreinte du code canonique : une décision de format, que le chapitre 9 prend.
- **L'exactitude au bit** est démontrée pour un produit scalaire IEEE à ordre fixé et mesurée sur la référence f64 (24 jetons au logit, 2 550 positions à l'empreinte du `.k1d`) ; pour les noyaux du moteur, c'est la proposition A1 du chapitre 9 et sa mesure.
- **Le sous-ensemble.** 4 couches sur 24, 8 puis 4 experts sur 32 par couche ; le gain de la mutation synonyme varie beaucoup d'une couche à l'autre (×0,97 à ×0,23) ; une couche où il s'inverserait n'est pas exclue. La KL ne substitue que 32 experts sur 768.
- **L'estimateur de débit** de la mutation synonyme est l'entropie (H_0 ou $H \mid \kappa$), pas un codeur réel. Pour la frontière, le codeur réel rend 2,6441 contre 2,6066 + échelles ; rien n'indique que l'écart soit différent pour les symboles synonymes, mais ce n'est pas mesuré.
- δ^* a été choisi sur l'expert 2 des quatre couches : d'autres experts que ceux qui sont mesurés, mais des mêmes couches.
- **La calibration et l'évaluation** viennent du même corpus (wiki.test), fenêtres disjointes ; une évaluation sur un autre domaine (code, dialogue) pourrait réduire le gain.
- **Les deux comparaisons** se font sur des sous-ensembles différents ($\lambda = 1$ sur 4 experts par couche, $\lambda = 0$ sur 8), chacune contre sa propre frontière.
- **La KL juge $\lambda = 0$, pas $\lambda = 1$** : le gain de 0,078 bit de $\lambda = 1$ n'est jugé qu'à l'erreur de sortie d'expert, et la ronde a montré que cette erreur ne prédit pas la KL (chapitre 8).
- **La borne du section 2.3** suppose les codes indépendants sachant l'échelle ; une dépendance non mesurée la ferait baisser.

Chapitre 4

Le génome des poids : la recette, la graine, S^*

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la frontière (3,8431 bits sans perte ; base de 2,6441 bits à 4,2920 % d'erreur sur les poids, sur les 12 tenseurs de F1), des codes presque indépendants sachant leur échelle, et la consigne de Stone pour ce sujet, le 2026-10-16 : « le but est de concevoir un systeme fonctionnel et si il ne voye rien de convaincaint vers l'ADN au terme ils recherche le moyen biomimétique le plus prométeur jusqu'à trouver ». La lecture (chapitre 2) prédisait qu'une recette ne gagnerait rien sans perte, et qu'avec perte elle viserait au mieux la borne des poids. **Ce chapitre établit** que la recette littérale (une graine et des coefficients, à la manière de SeedLM) ne bat pas la frontière, et le dit avant la mesure par un lemme sur le meilleur de N sous-espaces aléatoires ; que la même idée, portée à la dimension d'une rangée entière et rendue cherchable par l'algorithme de Viterbi (le *brin lu par codons*), la bat sur les 12 tenseurs en erreur de poids, de 0,20 à 0,60 bit ; qu'au même point elle perd en erreur de sortie, et pourquoi (une table qui écrête les colonnes massives) ; que le système corrigé, S^*, gagne en sortie d'expert 8 fois sur 8, mais que sa KL de bout en bout est **égale** à celle de la base. Il démontre aussi qu'aucune recette suivie d'un complément exact ne descend sous l'entropie des codes. **Il reste** le noyau sur la carte, le format d'une sorte de flux « BRIN », la rétroaction d'erreur entre colonnes combinée au brin, et la KL avec tous les experts codés. Sources : `genome/genome.md` ; `genome/genome-pour-le-memoire.md` ; `mesures/genome/genome/00-seuils-declares.md`, `csv/`, `kl/`, les journaux `*.json` ; `genome/05-synthese.md` §§ 1.2, 2.2, 2.4 ; outils sous `outils/genome/recette/` et `outils/genome/recette-kl/`.

En bref

L'ARN messager se lit trois bases à la fois, à partir d'un point fixe, sans virgule, et chaque codon se traduit par une table fixe et dégénérée (section 1.2). Le chapitre prend cette image au sérieux, comme image : un *brin* de bits stocké pour chaque rangée de matrice, une fenêtre de 12 bits qui glisse le long du brin et rend un poids à chaque pas par une table de 256 valeurs, apprise sur le tenseur, comme le code génétique est fixe pour une espèce. Le meilleur brin se trouve exactement par programmation dynamique. Ce lecteur de codons gagne là où une graine courte perd, parce que sa dimension est illimitée. Mais il gagne en erreur de poids et perd en sortie : sa table écrête les quelques colonnes géantes que les entrées réelles lisent le plus. Une fois ces colonnes gardées à part, il gagne en sortie d'expert, et la divergence sur le texte ne bouge pas. C'est la leçon du chapitre, et de la ronde : chaque juge intermédiaire peut se tromper, et seul le texte tranche.

4.1 La question et son juge

Une recette « marche » si elle lit moins de bits par poids sur le disque à erreur égale ; le gain se lit dans le toit (2.1). Les seuils sont dans `mesures/genome/genome/00-seuils-declares.md`, commités avant toute mesure d'une recette (commit `e777d6d`, puis la précision de l'échantillon, `2698ccd`, avant la première passe).

Le critère W. Sur l'échantillon (experts 0, 8, 16 et 24 de chacun des 12 tenseurs ; une rangée sur quatre, 720 rangées entières par expert, 34 560 rangées, 99,5 millions de poids), à l'erreur relative de la base *mesurée sur les mêmes rangées*, la recette lit moins de bits que la frontière F1 du même tenseur *sur chacun des 12 tenseurs* et en global. Une recette sans effet passerait les 12 avec la probabilité $2^{-12} \approx 2{,}4 \cdot 10^{-4}$ (test du signe) [calc.]. L'erreur est $e = \sum (w - \hat{w})^2 / \sum w^2$ dans le domaine réel des poids, échelle comprise, cumulée sur les rangées du tenseur : la mesure de la frontière. Les critères O (sortie d'une couche d'experts), F (fidélité en fonction du débit), K (KL), C (transcription) et L (complément exact) sont déclarés dans le même fichier ; chacun est jugé là où il se mesure.

4.2 La graine : la recette littérale, et pourquoi elle ne peut pas suffire

4.2.1 Le mécanisme

Un bloc de C poids $w \in \mathbb{R}^C$ est approché par $\hat{w} = U(g)\,t$, où $U(g)$ est une matrice $C \times P$ tirée d'un générateur pseudo-aléatoire par une graine g de K bits, et $t \in \mathbb{R}^P$ des coefficients quantifiés (entiers de 4 bits en complément à deux, un exposant partagé de 4 bits). L'encodeur essaie les 2^K graines, projette w sur chaque sous-espace (pseudo-inverse précalculée par Cholesky), quantifie, garde la meilleure. Débit : $(K + 4 + 4P)/C$ bits par poids. C'est la forme du problème de SeedLM [15] ; le code est écrit de zéro (`outils/genome/recette/`). Deux générateurs : un LFSR de Galois de 16 bits ($x^{16} + x^{14} + x^{13} + x^{11} + 1$, période 65 535 vérifiée par le test `periode_maximale`) et un mélangeur de bits à compteur. Témoin T3 : un bloc engendré exactement par une graine et des coefficients quantifiables se retrouve à erreur nulle (`bloc_engendre_retrouve_exactement`).

4.2.2 Ce que la théorie permet, avant toute mesure

Lemme 4.1 (Le meilleur de N sous-espaces aléatoires)**.** *Soit w une direction uniforme sur la sphère de $\mathbb{R}^C$ et V un sous-espace aléatoire uniforme de dimension P, indépendant de w. La part d'énergie hors de V, $\rho = \|P_{V^\perp} w\|^2 / \|w\|^2$, suit la loi* Beta$(a, b)$ *avec $a = (C - P)/2$ et $b = P/2$.*

Démonstration. Par invariance par rotation, on peut fixer $V = \mathrm{vect}(e_1, \dots, e_P)$ et prendre $w = z/\|z\|$ avec z gaussien standard dans $\mathbb{R}^C$. Alors $\rho = \sum_{i>P} z_i^2 / \sum_i z_i^2 = X/(X+Y)$ avec $X = \sum_{i>P} z_i^2 \sim \chi^2_{C-P}$ et $Y = \sum_{i \le P} z_i^2 \sim \chi^2_P$, indépendants. Le rapport $X/(X+Y)$ de deux khi-deux indépendants à $2a$ et $2b$ degrés de liberté est, par définition, de loi Beta(a, b). ∎

Corollaire 4.2 (Le résidu minimal attendu)**.** *Pour $N = 2^K$ sous-espaces indépendants, de résidus $\rho_1, \dots, \rho_N$, et F la fonction de répartition de* Beta(a, b)*,*

$$\mathbb{E}\big[\min_i \rho_i\big] = \int_0^1 \big(1 - F(\varepsilon)\big)^N \,\mathrm{d}\varepsilon \;\approx\; \Gamma\Big(1 + \frac{1}{a}\Big)\Big(\frac{a\,B(a,b)}{N}\Big)^{1/a}. \tag{4.1}$$

Démonstration. Pour une variable positive, $\mathbb{E}[M] = \int_0^\infty \mathbb{P}(M > \varepsilon)\,\mathrm{d}\varepsilon$; pour $M = \min_i \rho_i$ et des ρ_i indépendants de même loi, $\mathbb{P}(M > \varepsilon) = (1 - F(\varepsilon))^N$, et $\rho \le 1$. Pour l'asymptote, la densité de Beta(a, b)

(C, P, K)	débit (bits par poids)	résidu minimal attendu	borne de Shannon au même débit
(8, 3, 16)	4,00	0,79 %	0,39 %
(12, 4, 16)	3,00	3,82 %	1,56 %
(12, 3, 16)	2,67	**6,32 %**	2,48 %
(14, 4, 16)	2,57	7,07 %	2,83 %
(16, 5, 16)	2,50	7,84 %	3,13 %
(16, 4, 16)	2,25	10,74 %	4,42 %
(8, 1, 16)	3,00	5,35 %	1,56 %

Table 4.1 : Le résidu minimal attendu de la graine sur une source isotrope, par le lemme 4.1 et son corollaire, avant quantification des coefficients [calc.]. La borne de Shannon d'une gaussienne, 2^{-2R}, est donnée au même débit. Données : `mesures/genome/genome/graine-theorie.json` ; `genome/genome.md` § 2.2.

vaut $\varepsilon^{a-1}(1-\varepsilon)^{b-1}/B(a,b) \approx \varepsilon^{a-1}/B(a,b)$ près de 0, d'où $F(\varepsilon) \approx \varepsilon^a/(a\,B(a,b))$. Pour N grand, seule compte la région où F est petite, et $(1-F)^N \approx \exp(-NF) = \exp(-c\,\varepsilon^a)$ avec $c = N/(aB(a,b))$. Le changement de variable $u = c\,\varepsilon^a$ donne $\int_0^\infty e^{-c\varepsilon^a}\mathrm{d}\varepsilon = c^{-1/a}\,\frac{1}{a}\int_0^\infty u^{1/a-1}e^{-u}\,\mathrm{d}u = c^{-1/a}\,\Gamma(1+1/a)$, qui est (4.1). ∎

Le calcul exact par quadrature (`outils/genome/recette/graine_theorie.py`, `graine-theorie.json`), avant toute quantification des coefficients (qui ne peut qu'ajouter de l'erreur), donne le tableau 4.1 [calc.].

Ce que cela dit, avant la mesure. Autour de 2,64 bits, la graine laisse *au moins* 6,3 % d'erreur sur une source isotrope, quand la base en laisse 4,29 % ; pour atteindre 4,29 %, il lui faut près de 3 bits. Le pourquoi tient en une ligne : les K bits de la graine achètent un livre de codes aléatoire de 2^K sous-espaces dans une dimension $C \le 16$. Shannon prouve que le livre de codes aléatoire atteint la fonction débit-distorsion [9], mais *quand la dimension tend vers l'infini* ; à $C = 8$ ou 12, l'écart à la borne est un facteur 2,5 en erreur. Et les $4P + 4$ bits des coefficients décrivent une position fine dans un sous-espace choisi au hasard : ils paient une précision que le choix grossier du sous-espace a déjà perdue.

4.2.3 La mesure

`genome graines` (`graines.jsonl`, `graines.log`) : expert 0 de chacun des 12 tenseurs, 32 rangées réparties sur toute la hauteur, rangées entières ; l'erreur de la base, mesurée sur les mêmes poids, vaut 4,559 % sur cet échantillon (tableau 4.2).

Le lemme tient sur les vrais poids : la mesure suit la prédiction, plus la quantification des coefficients (0,6 à 2 points). Les blocs de poids réels se comportent, pour la graine, comme des directions uniformes : aucune concentration que le meilleur de 2^{16} sous-espaces aurait pu saisir. À l'erreur de la base, la graine demande **3,06 bits par poids** (interpolation en log de l'erreur entre (12, 4) et (10, 3) ; en linéaire, 3,07 [calc.]), contre 2,64 : **elle ne bat pas la frontière**, de 0,42 bit. Elle ne bat même pas le sans-perte (1,4 % d'erreur pour 4 bits, quand 3,84 bits rendent les poids exacts). Le générateur ne compte pas : LFSR et mélangeur rendent 1,58 et 1,41 %, l'écart venant de la corrélation des états successifs d'un LFSR, qui décalent les mêmes bits d'un cran.

(C, P, K), générateur	débit	erreur mesurée	prédiction du lemme 4.1
(8, 3, 16), mélangeur	4,00	1,41 %	0,79 %
(8, 3, 16), LFSR	4,00	1,58 %	0,79 %
(10, 3, 16)	3,20	3,89 %	2,98 %
(12, 4, 16)	3,00	4,89 %	3,82 %
(8, 1, 16), forme-gain	3,00	7,34 %	5,35 %
(12, 3, 16)	2,67	7,51 %	6,32 %
(14, 4, 16)	2,57	8,24 %	7,07 %
(16, 5, 16)	2,50	9,10 %	7,84 %
(12, 2, 16)	2,33	11,76 %	9,99 %
(16, 4, 16)	2,25	12,15 %	10,74 %

Table 4.2 : La graine mesurée sur les vrais poids, contre le lemme 4.1 [mes.] [calc.]. Données : `mesures/genome/genome/graines.jsonl`, `csv/graine-mesure-theorie.csv`.

4.3 La graine qui pousse : le brin lu par codons

4.3.1 L'idée

Si le défaut de la graine est sa dimension, il faut une graine *aussi longue que la rangée*, et un moyen de la chercher. L'ARN messager est lu par le ribosome trois bases à la fois, à partir d'un point fixe, sans virgule [3], et chaque codon se traduit par une table fixe et dégénérée [4]. Transposé, comme analogie de construction :

- le **brin** : une suite de bits stockée sur le disque, une par rangée de matrice ;
- la **fenêtre de lecture** : l bits ; elle avance de k_b bits à chaque pas et rend v poids. Deux pas voisins partagent $l - k_b$ bits : les codons se chevauchent, et c'est là que le brin diffère de l'ARN ;
- le **code génétique** : la fenêtre $s \in \{0, \dots, 2^l - 1\}$ se traduit en v valeurs par $\texttt{table}[\texttt{hache}(s, j) \gg (32 - q)]$, $j < v$: un mélangeur de bits choisit une case d'une table de 2^q valeurs ; 2^l fenêtres pour 2^q sens, un code dégénéré comme le vrai ;
- l'**adaptation de l'organisme** : la table est apprise sur les poids du tenseur (Lloyd : chaque case prend la moyenne pondérée des poids qu'elle sert).

Débit : k_b/v bits par poids, plus la première fenêtre (l bits par rangée). C'est un code en treillis, le treillis « à décalage » de Mao et Gray, repris avec des codes calculés par QTIP [47, 74] ; la quantification en treillis remonte à Marcellin et Fischer [13, 75]. *Ils posent le problème ainsi* : une source gaussienne i.i.d., après une rotation aléatoire. *Ce qui nous manquait*, et qui est dérivé ici : son emploi sur un modèle né en 4 bits contre une base déjà emboîtée dans le format ; la répartition des bits entre rangées par leur énergie, mesurée et décomposée ; la comptabilité complète contre la frontière F1 du format `.trc`. Le code est écrit de zéro (`outils/genome/recette/src/codons.rs`).

4.3.2 Le meilleur brin : Viterbi, et pourquoi il est exact

L'état au pas p est la fenêtre s_p (l bits). La transition $s \to s'$ est permise si et seulement si les $l - k_b$ bits forts de s' sont les $l - k_b$ bits faibles de s, c'est-à-dire $s' = ((s \ll k_b) \mid b) \bmod 2^l$ pour un $b < 2^{k_b}$.

Lemme 4.3 (Les prédécesseurs dans le treillis à décalage)**.** *Les prédécesseurs de s' sont exactement les $s = (t \ll (l - k_b)) \mid u$, $t < 2^{k_b}$, avec $u = s' \gg k_b$. Ils ne dépendent de s' que par u.*

Démonstration. La transition écrit $s' = ((s \ll k_b) \mid b) \bmod 2^l$: les $l - k_b$ bits forts de s' sont les

$l - k_b$ bits faibles de s, et ses k_b bits faibles sont b. Donc $s' \gg k_b = s \bmod 2^{l-k_b}$: u fixe les $l - k_b$ bits faibles de s, et ses k_b bits forts, t, sont libres. Réciproquement, tout s de cette forme mène à s' avec $b = s' \bmod 2^{k_b}$. ■

Proposition 4.4 (Exactitude du Viterbi). *Soit $C_p(s)$ le coût minimal $\sum_{i \le p} \sum_j \omega_{i,j} (x_{i,j} - \hat{x}_{i,j})^2$ d'un brin dont la fenêtre au pas p est s. Alors $C_0(s) = d_0(s)$ et*

$$C_p(s') = M_p(s' \gg k_b) + d_p(s'), \qquad M_p(u) = \min_{t < 2^{k_b}} C_{p-1}\big((t \ll (l - k_b)) \mid u\big), \tag{4.2}$$

avec $d_p(s') = \sum_j \omega_{p,j} (x_{p,j} - \texttt{table}[\texttt{hache}(s', j)])^2$. Le brin de coût minimal s'obtient en gardant l'argmin t de chaque $M_p(u)$ et en remontant depuis $\arg\min_s C_{P-1}(s)$.

Démonstration. Le coût est additif par pas, et le terme du pas p ne dépend que de la fenêtre s_p : c'est le principe d'optimalité de Bellman, sur lequel repose l'algorithme de Viterbi [76]. Un chemin optimal finissant en s' au pas p se compose d'un chemin optimal finissant en un prédécesseur de s' au pas $p - 1$, puis de $d_p(s')$; le lemme 4.3 donne l'ensemble exact de ces prédécesseurs, qui ne dépend que de $u = s' \gg k_b$. La récurrence (4.2) en découle, et la remontée des argmin reconstruit le chemin. ■

Le coût. Par pas, 2^l comparaisons (le minimum sur t, par tranches contiguës de 2^{l-k_b} grâce au lemme) et $2^l \cdot v$ évaluations ; mémoire de retour : 2^{l-k_b} octets par pas. Pour $l = 12$: 4 096 états, environ $2{,}3 \cdot 10^5$ poids par seconde et par forme sur les 16 fils du i7-13620H [mes.].

Les témoins. T2 (comptabilité des bits) : le brin écrit (`ecrire_brin`) puis relu par le seul décodeur (`lire_brin`, qui ne connaît que le brin et la table) rend exactement la reconstruction dont l'erreur est comptée (tests `brin_ecrit_puis_lu_*`). T1 (source connue) : sur 128 rangées gaussiennes i.i.d., $l = 12$, $k_b = 2$, $v = 1$, $q = 8$, table gaussienne sans apprentissage, l'erreur vaut **0,073 05** (`gauss-T1.json`), dans l'intervalle déclaré [0,0625 ; 0,080]. Le bas de l'intervalle est la borne de Shannon $2^{-2 \cdot 2}$: la battre dénoncerait une faute de comptabilité ; le haut est le milieu entre le treillis publié à $l = 16$ (0,069, QTIP) et le meilleur quantificateur vectoriel de dimension 8 publié (0,089). Cassure (table nulle) : 0,996, le témoin tombe (`gauss-T1-casse.json`) [mes.].

Le brin sur une gaussienne vraie. La figure 4.1 donne la courbe du brin seul sur 64 rangées gaussiennes (32 à $l = 16$, sauf la forme (2, 1), sur 64). L'écart à la borne de Shannon reste entre 0,46 et 1,66 dB à $l = 12$, entre 0,29 et 1,08 dB à $l = 16$ [mes.] [calc.] (`memoire/figures/tome3/donnees/genome-nombres.csv`). À 2 bits, la longueur de fenêtre achète l'essentiel de l'écart : 0,0880 ; 0,0785 ; 0,0728 ; 0,0694 ; 0,0675 pour $l = 8$, 10, 12, 14, 16, contre la borne 0,0625. Le mélangeur bon marché (4 opérations) rend 0,0721 contre 0,0728 pour le fort ; la taille de la table compte peu ($q = 6$: 0,0732 ; $q = 10$: 0,0728). Ces deux faits fixent le réglage du reste du chapitre : $l = 12$, $q = 8$.

4.3.3 La rotation : répartir l'information de chaque poids

Une rangée w de 2 880 poids est tournée par morceaux de 64 : $y = \frac{1}{8} H_{64} D\, w$, $D = \mathrm{diag}(\pm 1)$ tirée d'une graine (aucun bit stocké), H_{64} la matrice de Hadamard d'ordre 64.

Proposition 4.5 (La rotation conserve le produit et l'erreur). *Soit R la matrice bloc-diagonale des $\frac{1}{8} H_{64} D_m$. Alors R est orthogonale ; $Wx = (WR^\top)(Rx)$; et pour toute reconstruction $\hat{W}'$ des rangées tournées $W' = WR^\top$, l'erreur se conserve : $\|W - \hat{W}'R\|_F = \|W' - \hat{W}'\|_F$.*

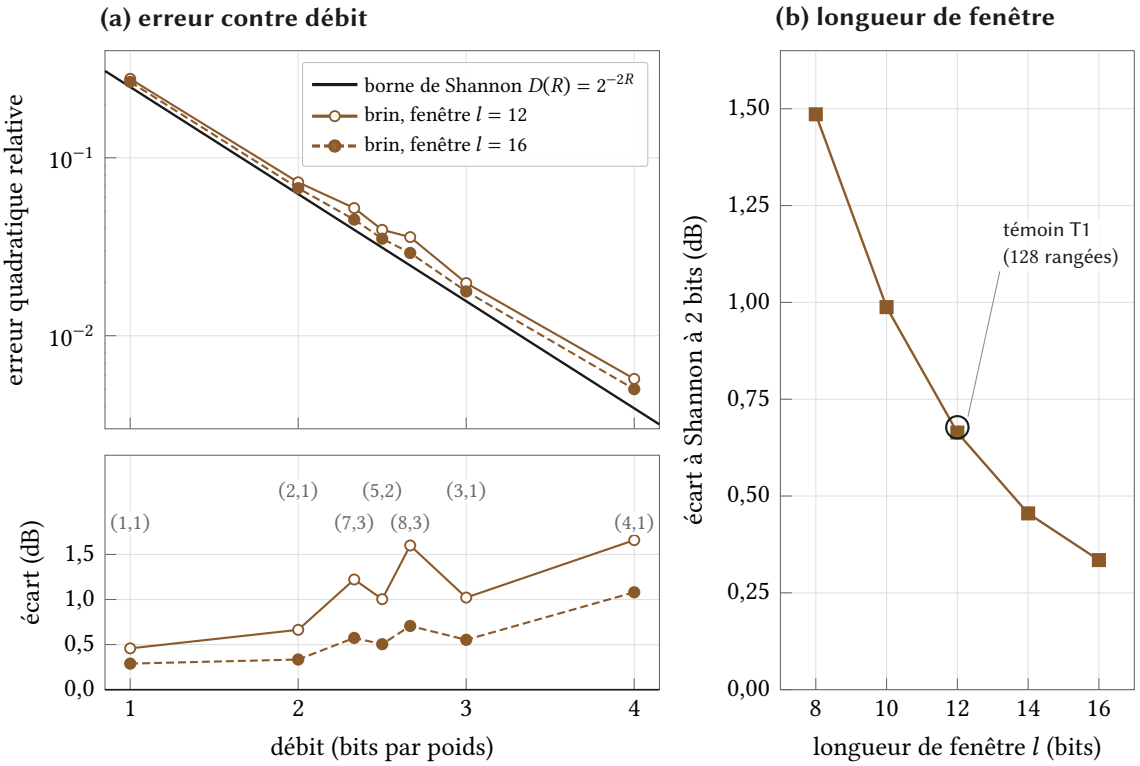


Figure 4.1 : Sur une gaussienne vraie, le brin lu par codons reste à moins de deux décibels de la borne de Shannon, et la longueur de fenêtre achète l'essentiel de l'écart. [mes.] Un brin de bits est lu par une fenêtre glissante de l bits ; un mélangeur haché fait de chaque fenêtre un indice dans une table de 2^q valeurs (ici $q = 8$, quantiles de la gaussienne, sans passe de Lloyd) ; Viterbi choisit les bits. La source est gaussienne i.i.d., en rangées de 2 880 valeurs : 64 rangées pour $l = 12$; pour $l = 16$, 32 rangées (92 160 valeurs), sauf la forme (2, 1), mesurée sur 64. La référence est la fonction distorsion-débit de Shannon $D(R) = 2^{-2R}$, qu'aucun code de longueur finie n'atteint. (a) Les sept formes (bits k_b avancés par pas, valeurs v émises, écrites en haut du panneau du bas), de 1 à 4 bits ; l'écart reste entre 0,46 et 1,66 dB pour $l = 12$ (creux, trait plein) et entre 0,29 et 1,08 dB pour $l = 16$ (pleins, tirets). (b) À 2 bits, la longueur de fenêtre achète l'essentiel de l'écart ($l = 8$: 0,0880 ; $l = 16$: 0,0675, contre la borne 0,0625). À $l = 12$, le mélangeur bon marché rend 0,0721 contre 0,0728 pour le fort, trois passes de Lloyd 0,0722, $q = 6$ et $q = 10$ rendent 0,0732 et 0,0728 (non tracés : ils tombent sur le point $l = 12$) ; le témoin T1, même réglage sur 128 rangées, rend 0,0731 (anneau à l'encre). Ce que la figure ne montre pas : les poids réels (figures suivantes), où la rotation doit d'abord rendre les rangées gaussiennes ; ni le coût du décodage.

Données : `mesures/genome/genome/gauss-balayage.jsonl`, `gauss-T1.json` ; script `memoire/outils/tome3_genome.py`.

Démonstration. $H_{64}H_{64}^\top = 64\,I$ et $D_m D_m^\top = I$, donc $(\frac{1}{8}H_{64}D_m)(\frac{1}{8}H_{64}D_m)^\top = \frac{1}{64}\,H_{64}D_m D_m^\top H_{64}^\top = I$: chaque bloc est orthogonal, et R aussi. Alors $WR^\top Rx = Wx$. Enfin $W - \hat{W}'R = (W' - \hat{W}')R$, et la norme de Frobenius est invariante par multiplication à droite par une matrice orthogonale : $\|AR\|_F^2 = \mathrm{tr}(ARR^\top A^\top) = \|A\|_F^2$. ∎

On stocke donc les rangées tournées, et l'on tourne l'activation *une fois par jeton* : 64×6 additions par morceau (la transformée de Hadamard rapide), soit 6 additions par élément d'entrée, partagées par toutes les rangées de toutes les matrices qui lisent cette entrée. L'outil mesure néanmoins l'erreur dans le domaine d'origine, après la rotation inverse (`detourner`) : c'est le second chemin de cette identité. Pourquoi tourner : un poids aberrant (une échelle E8M0 très au-dessus de la médiane de sa rangée) ne peut être atteint par le brin qu'en sacrifiant ses voisins, qui partagent sa fenêtre. Après rotation, il est réparti sur 64 positions, et la source devient gaussienne, ce pour quoi le treillis est fait. C'est, comme image, la mémoire holographique de la liste de Stone : l'information de chaque élément répartie sur tous. Chaque rangée tournée est divisée par son écart-type σ (gardé sur 16 bits) : la source codée est $x = y/\sigma$, de variance 1.

4.3.4 La répartition des bits entre rangées

Une rangée r a, pour chaque forme k, une erreur $D_r(k)$ et un coût $R_r(k)$ (bits du brin, première fenêtre, 3 bits d'indice de forme, 16 bits de σ). On minimise $\sum_r D_r(k_r)$ sous $\sum_r R_r(k_r) \le R$.

Proposition 4.6 (Le choix lagrangien est optimal pour son budget)**.** *Pour tout $\lambda \ge 0$, le choix $k_r(\lambda) = \arg\min_k D_r(k) + \lambda R_r(k)$, fait rangée par rangée, est optimal pour le budget $R(\lambda) = \sum_r R_r(k_r(\lambda))$ qu'il dépense.*

Démonstration. Si un autre choix k' dépensait au plus $R(\lambda)$ avec une erreur plus petite, alors $\sum_r D_r(k'_r) + \lambda \sum_r R_r(k'_r) < \sum_r D_r(k_r) + \lambda \sum_r R_r(k_r)$, ce qui contredit la minimalité de chaque terme de la somme. (Le résultat porte le nom d'Everett, 1963 ; la source n'est pas relue ici et n'entre pas dans la bibliographie.) ∎

La courbe débit-distorsion de la recette est l'ensemble des points $(R(\lambda), D(\lambda))$ pour 400 valeurs de λ ; les bits à erreur égale se lisent par interpolation linéaire en log(erreur) entre deux points voisins.

Proposition 4.7 (Le gain de la répartition, à haut débit)**.** *Pour des rangées gaussiennes de variances σ_r^2, de distorsions $D_r = \sigma_r^2\, 2^{-2R_r}$ (haut débit, $D_r < \sigma_r^2$), la répartition optimale des bits à débit moyen $\bar{R}$ gagne, à erreur moyenne égale,*

$$\Delta R = \frac{1}{2}\log_2 \frac{\mathrm{MA}(\sigma_r^2)}{\mathrm{MG}(\sigma_r^2)} \quad \textit{bits par poids} \tag{4.3}$$

sur la répartition uniforme, MA et MG les moyennes arithmétique et géométrique des σ_r^2.

Démonstration. Avec n rangées, on minimise

$$\frac{1}{n}\sum_r \sigma_r^2 2^{-2R_r} \quad \text{sous} \quad \frac{1}{n}\sum_r R_r = \bar{R}.$$

Le lagrangien donne $2\ln 2\;\sigma_r^2 2^{-2R_r} = \mu$ pour tout r : toutes les rangées finissent à la même distorsion D (le remplissage inverse de [2, § 10.3.3]). Alors $R_r = \frac{1}{2}\log_2(\sigma_r^2/D)$, et la moyenne donne $\bar{R} = \frac{1}{2}\log_2 \mathrm{MG} - \frac{1}{2}\log_2 D$, soit $D_{\mathrm{opt}} = \mathrm{MG}\cdot 2^{-2\bar{R}}$. La répartition uniforme ($R_r = \bar{R}$) donne $D_{\mathrm{unif}} = \mathrm{MA}\cdot 2^{-2\bar{R}}$. À erreur égale, il faut donc à l'uniforme $\frac{1}{2}\log_2(\mathrm{MA}/\mathrm{MG})$ bits de plus. Par l'inégalité arithmético-géométrique, le gain est positif, nul si toutes les variances sont égales. ∎

Poste	bits	par poids
brin d'une rangée de n poids	$l + (n/v - 1)\,k_b$	$\approx k_b/v$
indice de forme de la rangée	3	0,0010
σ de la rangée (source tournée)	16	0,0056
tables : 5 formes × 256 valeurs × 16 bits, pour 32 experts	20 480	0,000 08
échelles E8M0 (source brute seulement) : δ de F1 + 8/2 880 de médianes		0,036 à 0,043
alignement de chaque matrice d'expert sur 4 096 octets, au pire	32 768	0,0040
exceptions de S^* : compte (8 bits) + 3× (colonne 12 + code 4 + échelle 8) par rangée	80	0,028

Table 4.3 : Tout ce que la recette lit sur le disque, par poste [calc.]. Par poids : divisé par 2 880 pour un poste par rangée (par exemple $80/2\,880 = 0{,}028$), par 8 294 400 pour un poste par matrice ($32\,768/8\,294\,400 = 0{,}0040$). Source : `genome/genome.md` § 4.

La base de B n'en prend rien : elle met 2,64 bits partout. La section 4.6 calcule ce gain sur les vrais tenseurs : de 0,009 à 1,27 bit.

4.4 La comptabilité, au bit près

Tout ce que la recette lit sur le disque est compté (tableau 4.3), par un second chemin en Python (`outils/genome/recette/analyse.py`) qui fait le verdict. La frontière F1 de chaque tenseur est lue dans `f1-par-tenseur.csv` : base $+\delta + 8/2\,880 + 0{,}000\,07$.

Une sous-comptabilité est dite : le journal Rust (`codons-*-l12.json`) ne compte qu'*une* table par tenseur au lieu de cinq (0,000 06 bit par poids de moins) ; le second chemin compte les cinq, et c'est lui qui juge. Les deux diffèrent de moins de 0,0002 bit par tenseur.

4.5 Les résultats sur les poids réels

Témoin S0 : la frontière refaite. La commande `genome frontiere` recompte les 12 tenseurs entiers (3 185 049 600 poids), recalcule les centres de classe (0,237 166 ; 1,424 298 ; 3,418 577 ; 6,0 en unités E2M1) et retrouve l'erreur publiée 0,042 919 745 124 980 à $5{,}8 \cdot 10^{-15}$ près (`frontiere.json`). Cassure (chaque classe lit le centre de la suivante) : 1,2128, le témoin tombe (`frontiere-casse.json`) [mes.].

Le verdict W. Lecture par codons, $l = 12$, $q = 8$, 2 itérations de Lloyd sur les 48 premières rangées échantillonnées de chaque expert, cinq formes $(k_b, v) = (2,1), (7,3), (5,2), (8,3), (3,1)$, soit 2 ; 2,33 ; 2,5 ; 2,67 ; 3 bits par poids. Second chemin : `verdict-*-l12-second-chemin.json`, égal au journal Rust à 10^{-4} bit près. La figure 4.2 trace les courbes, le tableau 4.4 les gains.

12 tenseurs sur 12 : le critère W tient pour la source tournée. En global (une seule λ pour les 34 560 rangées, erreur cumulée 4,257 %), 2,049 bits contre 2,644 ; ce chiffre est dominé par les rangées de grande énergie, et la moyenne par tenseur est celui à retenir. Sur la **source brute** (échelles E8M0 gardées, sans rotation), 11 tenseurs sur 12 gagnent, de 0,15 à 0,57 bit, mais **blk.8.ffn_down tombe** : même à 3 bits partout, la recette y laisse 6,43 % d'erreur, quand la base en laisse 4,63 % à 2,64 bits. Ses rangées portent des blocs dont l'échelle est très au-dessus de la médiane ; le brin ne peut les atteindre qu'en sacrifiant leurs voisins. La rotation règle exactement ce cas (0,231 bit gagné sur ce tenseur) : c'est ce qui l'a fait retenir.

D'où vient le gain : pavage contre répartition. Les formes fixes (toutes les rangées à la même forme) donnent, par interpolation entre formes voisines, le gain du *seul pavage* (`codons-tourne-l`

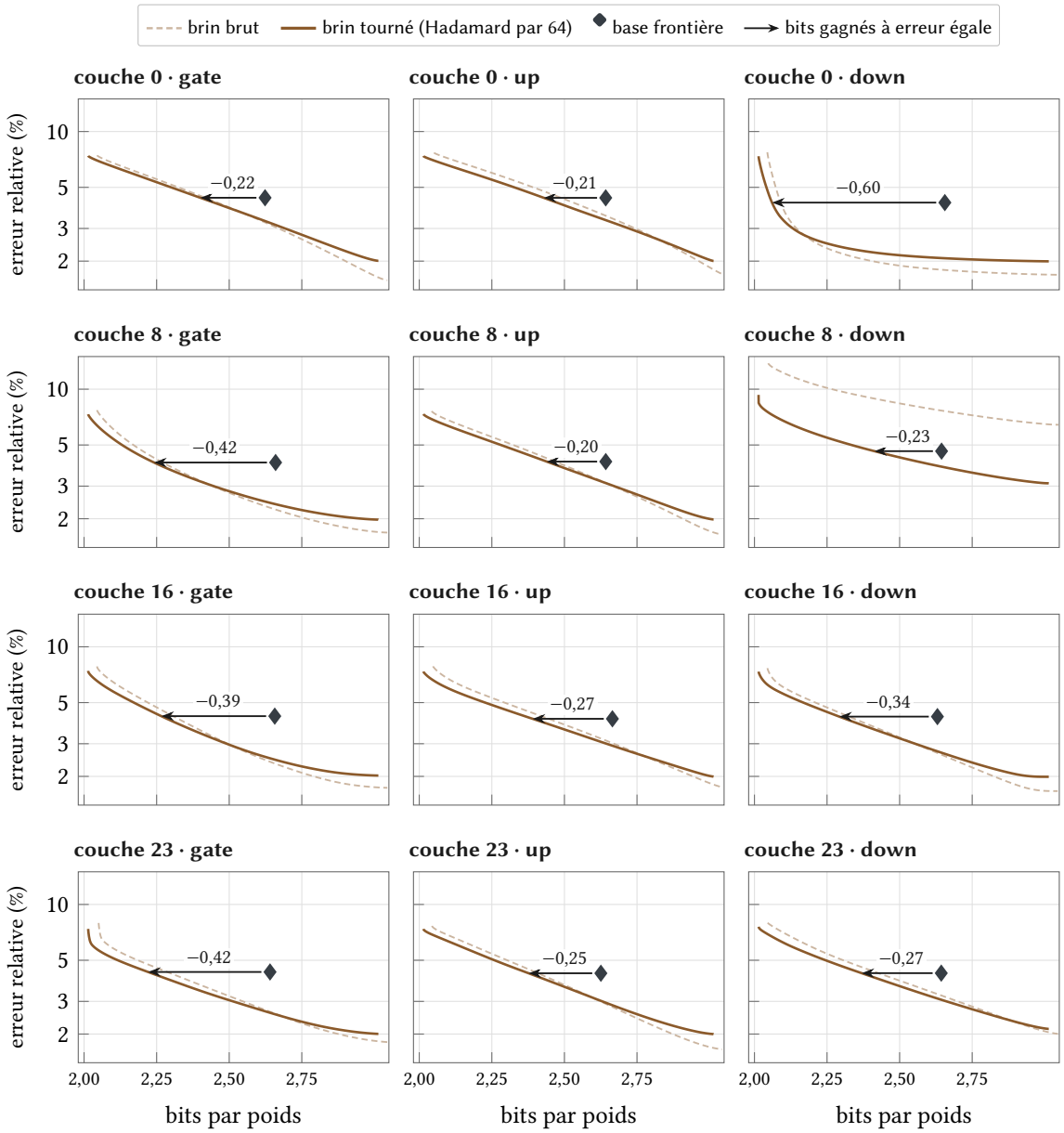


Figure 4.2 : Le brin tourné bat la base en erreur de poids, tenseur par tenseur. [mes.] Douze tenseurs d'experts de gpt-oss-20b (couches 0, 8, 16, 23 ; gate, up, down), fenêtre $l = 12$, chacun mesuré sur l'échantillon du critère W : experts 0, 8, 16 et 24, 2 880 rangées par tenseur, et non le tenseur entier. Chaque courbe balaie la répartition des bits entre rangées (lagrangien, 5 formes) et donne l'erreur quadratique relative des poids $\sum(w - \hat{w})^2 / \sum w^2$, la même grandeur qu'à la figure précédente, contre le débit complet du brin (brin, fenêtre, indice de forme, σ, tables, échelles, alignement : tout compté). En tirets brun clair, le brin sur les rangées brutes ; en brun plein, après la rotation de Hadamard par morceaux de 64, qui rend les rangées gaussiennes. Le losange gris est la base frontière (débit rANS réel, son erreur sur le même échantillon) ; la flèche, les bits que le brin tourné économise à erreur égale : 0,20 à 0,60 bit, 0,319 en moyenne sur les 12 tenseurs. À la couche 8, down, le brin brut n'atteint jamais l'erreur de la base (1 point manquant). Ce que la figure ne montre pas : l'erreur de sortie des experts et la KL, où ce gain ne tient pas (pondération, exceptions et S^* : chapitre 8) ; l'erreur de poids n'est pas un juge.

Données : `mesures/genome/genome/csv/courbes-tourne-l12.csv`, `courbes-brut-l12.csv`, `verdict-par-tenseur.csv`, `mesures/genome/genome/codons-tourne-l12.json` ; script `memoire/outils/tome3_genome.py`.

Tenseur	erreur de la base	frontière (bits)	brin tourné à erreur égale	gain (bits)	gain
blk.0.ffn_gate	4,394 %	2,6234	2,4005	0,223	8,5 %
blk.0.ffn_up	4,389 %	2,6422	2,4291	0,213	8,1 %
blk.0.ffn_down	4,137 %	2,6558	2,0604	0,595	22,4 %
blk.8.ffn_gate	4,022 %	2,6597	2,2410	0,419	15,7 %
blk.8.ffn_up	4,068 %	2,6424	2,4391	0,203	7,7 %
blk.8.ffn_down	4,631 %	2,6445	2,4130	0,231	8,8 %
blk.16.ffn_gate	4,227 %	2,6569	2,2660	0,391	14,7 %
blk.16.ffn_up	4,089 %	2,6653	2,3905	0,275	10,3 %
blk.16.ffn_down	4,203 %	2,6302	2,2920	0,338	12,9 %
blk.23.ffn_gate	4,323 %	2,6406	2,2203	0,420	15,9 %
blk.23.ffn_up	4,254 %	2,6256	2,3768	0,249	9,5 %
blk.23.ffn_down	4,260 %	2,6434	2,3691	0,274	10,4 %
moyenne des 12		**2,6442**	**2,3248**	**0,319**	**12,1 %**

Table 4.4 : Le verdict W, source tournée : bits du brin à l'erreur de la base mesurée sur les mêmes rangées [mes.]. Données : `mesures/genome/genome/csv/verdict-par-tenseur.csv` ; `verdict-tourne-l12-second-chemin.json`.

`12.json`, champ `fixes`) : 0,14 à 0,18 bit sur 11 tenseurs (0,135 en moyenne sur les 12), et −0,14 bit sur blk.8.ffn_down, où la répartition est indispensable. La répartition entre rangées apporte le reste : de 0,03 bit (blk.0.ffn_up) à 0,43 bit (blk.0.ffn_down) [mes.]. Les erreurs des formes fixes sur les poids tournés de blk.0.ffn_gate (0,0739 ; 0,0529 ; 0,0402 ; 0,0364 ; 0,0201) sont celles du brin sur une gaussienne vraie (0,0728 ; 0,0522 ; 0,0394 ; 0,0358 ; 0,0198) à 1,5 % près : **après rotation, les poids sont, pour le brin, indiscernables d'un bruit gaussien**. C'est le partage annoncé au section 2.3 : le gain est le pavage d'un code en treillis, que la base scalaire n'a pas, et la dispersion des énergies, qu'elle ignore.

4.6 Où est le bruit, où est la structure

`outils/genome/recette/structure.py` calcule les valeurs singulières de l'expert 0 de chaque tenseur (matrice 2 880 × 2 880, en f64 exact), contre un témoin : la même matrice dont les entrées sont permutées au hasard, qui détruit toute structure de rang (le spectre suit alors la loi de Marchenko-Pastur ; tableau 4.5).

Le bruit. À l'intérieur d'une rangée, une fois tournée et normalisée, il n'y a rien à prendre au-delà de la variance : le brin y fait exactement ce qu'il fait sur une gaussienne vraie, les codes voisins ne partagent que 0,005 bit, et la graine de dimension 8 à 16 trouve ce qu'elle trouverait dans du bruit. **Aucune recette courte n'engendre ces poids** : c'est la réponse, négative, à la question « les poids sont-ils recettables ? ».

La structure est au second ordre, et elle est grande. L'énergie se concentre sur quelques directions (5 % des directions portent 23 à 68 % de l'énergie, contre 17 à 19 % pour la même matrice permutée ; une seule direction en porte jusqu'à 8,5 %, 22 fois le hasard), donc sur certaines rangées (kurtosis jusqu'à 348, gain de répartition calculé jusqu'à 1,27 bit). C'est elle que la répartition des bits entre rangées encaisse, et que la base de B, qui met le même nombre de bits partout, laisse sur la table.

Tenseur	énergie des 144 premières directions réel / permuté	1re direction réel / permuté	kurtosis des rangées	gain de répartition (haut débit, [calc.])
blk.0.ffn_gate	27,4 / 17,2 %	0,81 / 0,14 %	4,0	0,019
blk.0.ffn_up	23,2 / 17,2 %	0,29 / 0,14 %	3,8	0,009
blk.0.ffn_down	67,7 / 18,9 %	8,48 / 0,38 %	347,7	1,272
blk.8.ffn_gate	45,2 / 17,3 %	1,62 / 0,14 %	12,9	0,572
blk.8.ffn_up	30,0 / 17,2 %	0,41 / 0,14 %	4,9	0,111
blk.8.ffn_down	43,1 / 18,5 %	7,00 / 0,44 %	343,2	0,250
blk.16.ffn_gate	54,2 / 18,1 %	4,60 / 0,39 %	182,9	0,645
blk.16.ffn_up	30,4 / 17,2 %	0,72 / 0,14 %	6,3	0,160
blk.16.ffn_down	43,3 / 17,4 %	1,54 / 0,21 %	42,0	0,343
blk.23.ffn_gate	42,0 / 17,3 %	2,27 / 0,14 %	17,2	0,426
blk.23.ffn_up	33,5 / 17,2 %	1,01 / 0,14 %	7,9	0,130
blk.23.ffn_down	33,3 / 17,7 %	3,45 / 0,19 %	79,7	0,089

Table 4.5 : La structure des matrices d'experts (expert 0 de chaque tenseur) : part d'énergie des 5 % premières directions singulières et de la première, contre la même matrice permutée ; kurtosis des rangées ; gain de répartition par la proposition 4.7 [mes.] [calc.]. Données : `mesures/genome/genome/structure-spectre.json`, `csv/spectre.csv`.

Ce qui reste à prendre. La borne gaussienne avec répartition idéale entre rangées (`borne-shannon-par-tenseur.json`) vaut 1,07 à 2,25 bits à l'erreur de la base ; la recette mesurée est 0,19 à 0,99 bit au-dessus. L'écart vient du brin à $l = 12$, à 0,3 à 0,8 dB de la borne, et de la plus petite forme, 2 bits : le remplissage idéal donnerait 0 ou 1 bit aux rangées les plus faibles. Une forme « zéro » (rangée non stockée), chiffrée sur les mêmes CSV, ferait tomber le débit global à 0,36 bit *à erreur cumulée égale*, ce qui montre surtout que l'erreur cumulée sur les poids se laisse tromper par la répartition (une poignée de rangées de très grande énergie la dominent) ; elle n'est pas retenue. La structure de rang (quelques directions dominantes) n'est pas exploitée : stocker r directions (u, v en f16) coûte $r \cdot 5\,760 \cdot 16/8\,294\,400 = 0{,}011\,r$ bit par poids ; pour blk.0.ffn_down, la première direction (8,5 % de l'énergie) rendrait $\frac{1}{2}\log_2(1/0{,}915) = 0{,}064$ bit à erreur égale pour 0,011 bit de coût ; pour les tenseurs up (0,3 à 1 %), rien [calc.].

Les motifs partagés, combinés. `genome motifs` (`motifs-*.json`) : un bloc de d poids est la somme de M motifs, chacun tiré d'un dictionnaire de 2^b motifs partagé par les 32 experts du tenseur, appris par k-moyennes pondérées étage par étage (quantification résiduelle). Témoin : le quantificateur scalaire à 4 niveaux d'une gaussienne rend 0,1175 [77]. À l'erreur de la base (4,257 %), les motifs demandent **2,90 bits** au mieux (entre $(8, 12, 1)$ à 1,54 bit pour 18,2 % et $(8, 12, 2)$ à 3,05 bits pour 3,65 %) : **ils ne battent pas la frontière**. Un dictionnaire appris en dimension 4 à 8 est un quantificateur vectoriel de petite dimension : même le meilleur réseau de dimension 8 (0,089 à 2 bits sur une gaussienne, QTIP) reste loin du brin (0,073). Partager le dictionnaire entre experts ne coûte rien (0,002 bit) et ne rend rien de plus : les experts n'ont pas de motifs propres que le dictionnaire commun manquerait.

4.7 La recette et la découpe base + complément

Lemme 4.8 (Le complément exact ne rend rien gratuitement)**.** *Soit X les codes MXFP4 d'un tenseur et Y les bits de la recette, fonction déterministe de X. Pour tout codage du complément qui conditionne chaque code X_i sur une fonction Z_i de Y,*

$$R_Y + \sum_i H(X_i \mid Z_i) \;\geq\; H(Y) + H(X \mid Y) \;=\; H(X). \tag{4.4}$$

Démonstration. $H(X,Y) = H(X) + H(Y \mid X) = H(X)$ puisque Y est fonction de X ; et $H(X,Y) = H(Y) + H(X \mid Y)$. D'où l'égalité de droite. Puis $H(X \mid Y) \leq \sum_i H(X_i \mid Y)$ (sous-additivité), $H(X_i \mid Y) \leq H(X_i \mid Z_i)$ (conditionner par moins, Z_i étant fonction de Y, ne réduit pas l'entropie), et $R_Y \geq H(Y)$ (aucun code sans perte ne descend sous l'entropie). ■

Ce que la combinaison perd est $(R_Y - H(Y)) + (\sum_i H(X_i \mid Z_i) - H(X \mid Y))$: l'information sur X que les bits du brin portent *ensemble* et qu'un contexte poids par poids ne voit pas. **Mesuré** (`codons-*-l12.json`, champ `H_code_sachant_recette` : Z_i la reconstruction de la recette ramenée à l'échelle du bloc, en 65 cases de 0,375) : recette + complément + échelles = **4,05 à 4,37 bits** pour la source tournée (4,09 à 4,90 pour la brute), contre 3,80 à 3,83 sans perte pour les mêmes tenseurs. La perte, 0,21 à 0,53 bit, est le second terme. Le critère L tombe, comme le lemme l'annonçait.

Ce que la combinaison donne quand même. Dans l'architecture B, le niveau 0 est lu à chaque défaut de cache et le complément seulement pour les experts admis. Remplacer la base par la recette au niveau 0 divise les octets d'un défaut par $2{,}644/2{,}325 = 1{,}14$ à erreur de poids égale, ou, au débit de la frontière de chaque tenseur, rend une erreur de 2,07 à 3,82 % au lieu de 4,02 à 4,63 % (interpolé sur `courbes-tourne-l12.csv`) ; le complément qui suit coûte alors 0,21 à 0,53 bit de plus qu'avec la base [calc.]. Ces chiffres sont en erreur de poids ; la section suivante dit ce qu'ils valent.

4.8 La sortie des experts : la fidélité, et S^*

Les entrées et la mesure. Les états normalisés et le routage de la référence f64, capturés par l'éclaireur : « calib » (fenêtres 10 à 21, réglage seulement) et « eval » (fenêtres 0 à 9, celles du `.kld` de R1, mesure seulement). Pour l'expert e de la couche ℓ, les 512 premières positions routées vers lui (moins si l'expert est rare). Sortie de l'expert : $y = W_{\text{down}}\,\text{SwiGLU}_{\text{oai}}(W_{\text{gate}}h + b, W_{\text{up}}h + b) + b$ en f64, poids exacts, base ou recette ; erreur relative $\sum \|y - \hat{y}\|^2 / \sum \|y\|^2$.

Au point W, la recette perd en sortie. Couche 0, expert 0 (499 positions) : erreur de poids 4,25 % (base 4,27 %), mais erreur de sortie **7,63 %** contre **6,77 %** pour la base, la recette ne gagnant que sur 146 positions sur 499 (`sortie-tourne-point-W-interrompu.log`) [mes.]. Le critère O, déclaré à ce point, **tombe** sur ce premier expert ; la passe a été arrêtée là pour mesurer ce qui décide vraiment, la fidélité en fonction du débit (protocole F, déclaré avant sa passe dans `00-seuils-declares.md`). *L'erreur quadratique sur les poids ne mesure pas la fidélité* : à erreur de poids égale, l'erreur de la base fait moins de dégâts en sortie que celle du brin.

La pondération. Par la proposition 4.5, la sortie d'une rangée vaut $\sum_c \Delta w_c x_c = \sum_{c'} \Delta w'_{c'} x'_{c'}$ avec $x' = Rx$. Son espérance quadratique est $\sum_{c',c''} \mathbb{E}[\Delta w'_{c'} \Delta w'_{c''}]\, \Sigma'_{c'c''}$, $\Sigma' = R\Sigma R^\top$, $\Sigma = \mathbb{E}[xx^\top]$ des entrées réelles. Pour une erreur *blanche* dans le domaine tourné ($\mathbb{E}[\Delta w'_{c'} \Delta w'_{c''}] = 0$ pour $c' \neq c''$), il reste

$$\mathbb{E}\,\|\Delta w^\top x\|^2 = \sum_{c'} h_{c'}\, \mathbb{E}\big[\Delta w'^2_{c'}\big], \qquad h = \operatorname{diag}\big(R\,\Sigma\,R^\top\big). \tag{4.5}$$

Couche, expert (positions)	base	uniforme à 2,64 b	pondérée à 2,64 b	bits à sortie égale (unif. / pond.)
0, 0 (499)	6,77 %	5,25 %	5,18 %	2,41 / 2,38
0, 16 (512)	5,13 %	3,88 %	3,82 %	2,38 / 2,39
8, 0 (512)	8,30 %	8,80 %	7,91 %	2,68 / 2,60
8, 16 (131)	7,45 %	8,93 %	9,41 %	2,87 / > 2,9
16, 0 (127)	5,72 %	6,02 %	5,40 %	2,68 / 2,58
16, 16 (150)	4,81 %	11,04 %	—	> 2,9 / —

Table 4.6 : La fidélité de sortie du brin, uniforme et pondéré par (4.5), passe F arrêtée après cinq experts et demi, le verdict étant acquis [mes.]. Données : `mesures/genome/genome/fidelite.log`, `verdict-fidelite.json`, `csv/fidelite.csv`.

Pondérer le Viterbi par h (les poids ω de la proposition 4.4) minimise donc l'erreur de sortie attendue sous cette hypothèse (`outils/genome/recette/src/fidelite.rs`, en-tête) [calc.] ; c'est la diagonale de la hessienne de la perte quadratique de couche, le geste de GPTQ réduit à ce que le treillis sait faire.

Le verdict F tombe (tableau 4.6) : la pondération gagne sur 4 experts sur 5, l'uniforme sur 2 sur 6 ; deux experts perdent dans les deux variantes. Le diagnostic (`erreur_colonnes.py`, `erreur-colonnes-ponderee-264.json`) : l'erreur du brin ne se répartit pas sur les colonnes comme celle d'un bruit blanc, et l'hypothèse de (4.5) est fausse là où elle compte. Dans blk.8.ffn_down (expert 16), la colonne 1011 porte **144 fois** l'énergie moyenne d'une colonne et reçoit **334 fois** l'erreur moyenne ; les colonnes de rang 0 modulo 64 en reçoivent 7,6 fois leur part [mes.]. Le mécanisme : une colonne massive se répartit, par la rotation, en 64 valeurs grandes dans chaque rangée ; la table (256 valeurs, quantiles de la source) les écrête ; les erreurs d'écrêtage, de même signe dans toutes les rangées, reviennent par la rotation inverse *sur la colonne massive*. Or une colonne massive de down lit une unité de l'activation intermédiaire, et une colonne massive de gate ou up un canal d'entrée : là où l'erreur fait le plus de mal. La base, elle, rend presque exacts les plus grands poids de chaque bloc (sa classe {6} est exacte). Sur les 15 matrices analysées, 5 ont une colonne dont l'erreur dépasse 10 fois la moyenne, et dans 4 d'entre elles c'est la colonne la plus énergétique de la matrice ; la cinquième (gate, couche 16, expert 0, colonne 2 744, ×29,9) est la colonne la plus énergétique de la matrice up du même expert, qui lit la même entrée.

Le remède, et le système S^*. Garder *à part, exacts*, les 3 poids de plus grande magnitude de chaque rangée (0,1 % ; $8 + 3 \times 24$ bits par rangée, 0,028 bit par poids, comptés) : ce sont, rangée après rangée, les entrées des colonnes massives. Couche 8, expert 16, uniforme : l'erreur de sortie passe de 8,93 % à 6,20 % (base 7,45 %), l'erreur de poids de 4,63 % à 3,16 % (`fidelite-exceptions.log`). Le brin, la rotation, la pondération et ces exceptions forment S^*, déclaré avant sa passe.

Le verdict F(S^*) tient (tableau 4.7) : à 2,64 bits, S^* fait moins d'erreur de sortie que la base sur **8 experts sur 8** (test du signe, $2^{-8} \approx 0{,}004$), et sur chaque expert la majorité des positions le confirme (de 333 sur 512 à 106 sur 106 ; la loi binomiale donne $p < 10^{-5}$ pour chacun). À erreur de sortie égale, S^* lit 2,09 à 2,58 bits, 2,36 en moyenne, contre 2,64 : 0,28 bit de moins par poids lu (10,8 %). C'est moins que les 0,32 bit du critère W : l'erreur de poids exagérait le gain, et la sortie le juge. Le livrable du sujet en tirait « ×1,12 jeton par seconde au toit du disque pour les experts lus au niveau 0 » ; c'est un calcul à erreur de sortie égale, et la section suivante dit que la KL ne le confirme pas.

Couche, expert (positions)	base	S^* à 2,64 b	positions où S^* gagne	S^* à 2,4 b	bits de S^* à sortie égale
0, 0 (499)	6,77 %	**5,09 %**	440 / 499	6,56 %	2,37
0, 16 (512)	5,13 %	**3,65 %**	432 / 512	4,74 %	2,34
8, 0 (512)	8,30 %	**7,79 %**	333 / 512	10,05 %	2,58
8, 16 (131)	7,45 %	**6,72 %**	93 / 131	8,50 %	2,54
16, 0 (127)	5,72 %	**4,57 %**	122 / 127	5,92 %	2,43
16, 16 (150)	4,81 %	**3,26 %**	149 / 150	4,30 %	2,30
23, 0 (150)	5,68 %	**3,15 %**	148 / 150	4,10 %	2,09
23, 16 (106)	3,97 %	**2,56 %**	106 / 106	3,36 %	2,23
moyenne		**rapport 0,747**			**2,36**

Table 4.7 : S^* (rotation, pondération, exceptions 0,1 %) à la sortie des experts, entrées tenues à l'écart [mes.]. Les experts (16, 16), (23, 0) et (23, 16) n'avaient servi à aucun choix. Données : `mesures/genome/genome/fidelite-sstar.log`, `fidelite-sstar.json`, `verdict-fidelite-sstar.json`, `csv/fidelite-sstar.csv`.

4.9 La KL contre la référence : S^* est égal à la base

Le protocole (P2 de la ronde). La fourche de l'éclaireur, augmentée de la substitution en F32 (`outils/genome/recette-kl/`) ; les 8 experts du tableau 4.7 remplacés (couches 0, 8, 16, 23 × experts 0 et 16 : 8 sur 768), base contre S^* à 2,64 bits, et `juge comparer` du lot R, qui apparie les deux KL position par position contre la même vérité (`ref-openai-20b.kld`). Règle du juge : égal si $|\bar{d}| \leq 2\sigma(\bar{d})$; meilleur si $\bar{d} + 2\sigma(\bar{d}) < 0$. Témoin de la fourche : sans expert substitué, une fenêtre, KL au plus 10^{-9} (`kl/temoin-vide*.json`).

Ce qui a été publié, et ce qui l'a remplacé. La première passe portait sur 3 fenêtres (765 positions ; `mesures/genome/genome/kl/comparer-base-contre-recette.json`) : KL 0,0226 pour la base, 0,0301 pour S^*, rapport ~~1,335~~ (un rapport à trois fenêtres, que la passe K10 ramène à 1,018, 0,020 93/0,020 57 [calc.], tableau 4.8), différence appariée +0,0076 ± 0,0047 (1,6 σ), premier choix 91,4 contre 90,3 % [mes.]. La synthèse de la ronde en avait d'abord conclu « S^* a 25 % d'erreur d'expert de moins et ~~33 % de KL de plus~~ » et « ~~tombe~~ », puis l'a barré (`genome/05-synthese.md` §§ 0, 1.2, 2.4) : la passe K10, déclarée avant, a refait la comparaison sur les **10 fenêtres** de R1 (2 550 positions ; `kl/comparer10-base-contre-recette.json`, commit `39e9aa4`). C'est elle qui compte (réserve R4 du tome).

S^* est égal à la base (tableau 4.8) : +0,000 36 ± 0,001 86, rapport de KL 1,018 [calc.]. Ce qui a changé entre les deux passes est le nombre de positions, multiplié par plus de trois : l'écart des 3 premières fenêtres était dans le bruit, et il s'y est fondu. La forme des deux distributions dit pourquoi la moyenne ne bouge pas. S^* réduit la part lisse de la KL (médiane ÷1,21, 99e centile de 0,411 à 0,338) et épaissit l'extrême de sa queue (99,9^e centile de 1,37 à 1,59, maximum de 3,01 à 3,22) : les deux se compensent [mes.]. **25 % d'erreur d'expert en moins, 0 % de KL en moins.** Le chapitre 8 en fait la propriété (c) de la ronde (l'erreur de sortie d'un expert ne prédit pas la KL) et avance une hypothèse, dite comme telle : la pondération de S^* est *diagonale* (4.5), elle ne voit pas la corrélation, dans le domaine tourné, entre les 64 positions qui portent le même canal massif, alors que la base laisse presque intacte la contribution des canaux massifs, et avec elle les marges du routeur des couches suivantes.

P2, 10 fenêtres, 2 550 positions, 8 experts sur 768	base	S^*
KL moyenne (± incertitude)	0,020 57 ± 0,002 12	0,020 93 ± 0,002 35
KL médiane	0,001 84	0,001 52
KL, 90e centile	0,0311	0,0275
KL, 99e centile	0,411	0,338
KL, 99,9e centile	1,367	1,588
KL maximale	3,008	3,216
premier choix identique	91,88 %	91,92 %
perplexité (référence 460,24)	461,62	468,71
différence appariée de KL, S^* moins base	+0,000 36 ± 0,001 86 : **égal**	
différence appariée de premier choix	+0,04 ± 0,56 point : égal	

Table 4.8 : S^* contre la base, la KL de bout en bout sur 10 fenêtres (passe K10) [mes.]. Verdicts du juge : statistique, égal en KL et en premier choix ; littéral, la KL de S^* est un peu au-dessus. Données : `mesures/genome/genome/kl/comparer10-base-contre-recette.json` (et `.csv`, `.txt`).

Étape du décodeur d'un poids (forme $v = 1$)	opérations
fenêtre : registre de 64 bits décalé de k_b, bits neufs insérés, masque de l bits	3
mélangeur fort (xor, puis trois fois multiplication et décalage-xor)	9 (le court : 3)
indice de table (décalage)	1
lecture de la table (256 valeurs en mémoire partagée)	1
multiplication-addition avec l'activation tournée	1
total par poids	**15** (le court : **9**)

Table 4.9 : Le coût de transcription du brin, en opérations par poids [calc.]. Pour $v = 2$ ou 3, la fenêtre se partage entre les v poids du pas : $(3 + 12\,v)/v$ opérations par poids avec le mélangeur fort. Source : `genome/genome.md` § 8.4.

4.10 La transcription

Le budget [calc.]. À 2,32 bits par poids (la moyenne du verdict W), un poids coûte 0,29 octet lu, donc $214 \times 0{,}29 = 62$ opérations de budget (269 au débit du disque selon la correction du Tome II) : le décodeur en prend 15 (tableau 4.9), **le quart** (6 % au débit du disque). Par rangée s'ajoute une multiplication par σ ; par jeton, la rotation de l'activation, 6 additions et un changement de signe par élément d'entrée, partagés par les 2 880 rangées qui le lisent, soit 0,002 opération par poids. Le décodeur rANS de l'étude 03 coûte environ 12 opérations par code de 4 bits : le brin est du même ordre que le décodeur sans perte déjà mesuré.

Mesuré sur le processeur (`genome decoder`, `decodeur-*.json`). Un expert entier (8 294 400 poids, blk.16.ffn_up, expert 0) relu depuis ses 2 880 brins par le seul décodeur, en séries ABBA de 8 contre la déquantification MXFP4 du même expert par le même chemin, machine chargée par les trois autres équipes : forme (5, 2) avec le mélangeur fort, $9{,}6 \cdot 10^7$ poids par seconde contre $10{,}2 \cdot 10^7$ (rapport 0,94) ; avec le court, $7{,}5 \cdot 10^7$ contre $6{,}3 \cdot 10^7$ (1,20) ; forme (2, 1) forte, $6{,}9 \cdot 10^7$ contre $7{,}1 \cdot 10^7$ (0,97) [mes.]. Le brin va aussi vite que la déquantification du format natif sur le même chemin ; la dispersion des passes sous la charge (0,05 à 0,17 s) est de l'ordre de l'écart, et les débits absolus ne valent rien

sous cette charge : seul le rapport compte. Sur la carte, rien n'est mesuré.

4.11 Ce qui reste

1. **Le noyau sur la carte** : une GEMV qui lit le brin (fenêtre glissante dans un registre de 64 bits, mélangeur, table en mémoire partagée), tourne l'activation par morceaux de 64 et accumule ; à mesurer contre les 2,82 Go/s du disque et contre le noyau MXFP4 du moteur.
2. **Le format** : une sorte de flux « BRIN » dans `.trc` (par matrice, l'indice de forme et σ de chaque rangée, puis les brins bout à bout ; les tables résidentes ; la graine de la rotation, constante du format).
3. **La rétroaction d'erreur entre colonnes** combinée au brin (le LDLQ de QuIP et QTIP, ou la mutation synonyme du chapitre 3) : le brin a gagné le pavage et la répartition, les exceptions les colonnes massives ; la rétroaction gagnerait la corrélation des entrées, et c'est la prédiction qui départagerait l'hypothèse du section 4.9. Elle n'est pas mesurée et n'entre pas dans les lots de construction.
4. **La fenêtre** $l = 16$ et des formes plus fines : 0,03 à 0,08 bit de plus sur une gaussienne, au prix d'un codage 16 fois plus lent (hors ligne, une fois).
5. **La structure de rang** : 1 à 8 directions dominantes stockées à part, jusqu'à 0,05 bit net sur les down.
6. **gpt-oss-120b** : refaire W et F ; et **la KL de tous les experts codés** : 19,1 milliards de poids à $2{,}3 \cdot 10^5$ poids par seconde et par forme, environ 23 heures de processeur pour une forme par rangée, 115 pour les cinq [calc.] ; ou un codeur sur la carte.

Ce qui pourrait être faux

- **L'erreur sur les poids n'est pas la qualité**, et l'erreur de sortie non plus. Le critère W est en erreur cumulée sur les poids, que la répartition peut tromper ; le critère F est en erreur de sortie d'expert, que S^* a passé 8 fois sur 8 sans bouger la KL. Seule la KL juge, et elle ne substitue que 8 experts sur 768.
- **L'échantillon** : 4 experts sur 32, une rangée sur 4, 12 tenseurs sur 72, gpt-oss-20b seul ; le 120b n'est pas mesuré (les entropies des codes des deux modèles diffèrent de 0,001 bit, ce qui rend un grand écart improbable, sans le prouver).
- **Les tables sont apprises sur l'échantillon** (48 rangées par expert) et évaluées aussi sur ces rangées ; leur coût est compté, mais un léger sur-apprentissage est possible. La sortie les applique à des experts entiers et à des entrées qu'elles n'ont jamais vues.
- S^* **a été choisi après avoir vu 5 experts sur 8** (les deux remèdes viennent des échecs mesurés) ; les 3 autres n'avaient servi à rien et gagnent aussi, mais le risque d'avoir réglé S^* sur ses propres juges reste. La KL, qui ne s'en laisse pas conter, le rend égal à la base.
- **Peu de positions** pour certains experts (106 à 150 aux couches 16 et 23, 131 pour (8, 16)) : chaque verdict par expert reste significatif par la loi binomiale, mais la taille de l'effet est moins précise. La pondération est estimée sur au plus 1 024 positions routées : pour un expert rare, h est bruitée.
- **L'interpolation en** log(**erreur**) entre deux points de la courbe lagrangienne suppose la courbe localement exponentielle ; avec 400 valeurs de λ, l'erreur d'interpolation est sous 10^{-3} bit.
- **La frontière F1** est mesurée par un codeur réel ; la recette est comptée bit à bit, sans en-tête de tuile autre que l'alignement au pire. Un format qui imposerait des tuiles plus fines au brin

ajouterait leurs en-têtes.
- **La transcription n'est pas mesurée sur la carte** ; la rotation de l'activation est calculée négligeable, pas mesurée dans le moteur (pour down, l'entrée diffère par expert et se tourne pour chaque expert actif).
- **L'hypothèse qui explique la queue de** S^* (pondération diagonale contre canaux massifs) n'est pas mesurée.
- **SeedLM et QTIP** sont lus à la source ; leurs chiffres sur Llama ne sont pas reproduits et ne servent que de repère (QTIP publie 0,069 à 2 bits sur une gaussienne, retrouvé ici à 0,0675 pour $l = 16$ et 0,0731 pour $l = 12$).

Chapitre 5

La duplication de gènes

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, l'hypothèse de Stone (`idees/biomimetisme-genome-des-poids.md` § 5 bis) : « les experts d'une couche (ou de couches voisines) sont parents : stocker un expert parent et, pour chaque descendant, une petite mutation coûte moins de bits que l'expert entier ». La frontière : 3,8431 bits sans perte, 2,6441 bits pour la base à 4,2920 % d'erreur. L'état de l'art (tableau 2.3, famille 3) : des fusions d'experts sur BF16, jugées en précision de tâches, jamais en bits sur des codes de 4 bits ; et la carte du modèle ne dit pas si les experts de gpt-oss sont nés par copie [6]. **Ce chapitre établit** la parenté des experts de gpt-oss-20b, mesurée sur les codes MXFP4 et sur les poids, à une permutation des neurones près (la seule symétrie d'un expert, démontrée), dans la même couche et dans la couche voisine ; son coût exact en bits (une information mutuelle conditionnelle, coûts ajoutés comptés) ; l'arbre de codage le plus économe ; l'erreur de sortie d'une mutation tronquée et d'un paralogue sur des entrées réelles. **La duplication ne bat pas la frontière** : elle existe, elle se mesure, et elle vaut un millième de bit. Une phrase des sources est corrigée ici, sur leurs propres données. **Il reste** la KL de la mutation tronquée (non mesurée, son erreur de sortie suffisant à l'écarter), le coût exact et les paralogues sur gpt-oss-120b, et l'écran sur les 24 couches. Sources : `genome/duplication.md` ; `genome/duplication-derivations.md` ; `genome/duplication-pour-le-memoire.md` ; `mesures/genome/duplication/00-seuils-declares.md`, `csv/`, journaux `*.json` ; outils sous `outils/genome/duplication/`.

En bref

Dans le vivant, un gène se duplique parfois ; les deux copies dérivent ensuite, l'une gardant la fonction, l'autre en prenant une nouvelle, et la famille de gènes qui en naît garde longtemps la trace de sa copie commune [78]. L'image suggère une compression : si les 32 experts d'une couche sont nés d'une copie, stocker un parent et des mutations coûterait moins cher que 32 experts. L'image est une analogie, et la question se pose en bits : combien un expert apprend-il sur un autre, une fois leurs neurones remis dans le même ordre ? La réponse mesurée est : presque rien. Après le meilleur appariement, deux experts appariés se ressemblent autant que deux experts tirés au hasard ; ce qu'ils partagent appartient à toute la couche, à ses colonnes d'entrée, pas à une lignée. Les experts de gpt-oss ont été entraînés à être *différents* : le routeur n'envoie à chacun que les jetons où il est utile.

5.1 La question, posée en bits

Un enfant c codé sachant un parent p déjà décodé coûte $H(c \mid p)$ au lieu de $H(c)$; le gain est l'information mutuelle $I(c; p)$, moins ce qu'il faut stocker pour relier l'enfant au parent. Pour un

réseau, une seule liberté rend deux experts comparables : permuter les neurones cachés, qui ne change pas la fonction (proposition 5.1). La duplication de Stone devient

$$\max_{p,\,\Pi} I(c;\ \Pi p) \quad \text{contre} \quad 3{,}84 \text{ bits par poids.} \tag{5.1}$$

Les notations sont celles du modèle : un expert de gpt-oss-20b à la couche ℓ, indice e ; $d = f = 2\,880$; $W_g, W_u \in \mathbb{R}^{f\times d}$ (gate, up), $W_d \in \mathbb{R}^{d\times f}$ (down), biais $b_g, b_u \in \mathbb{R}^f$, $b_d \in \mathbb{R}^d$. Les seuils sont déclarés dans `mesures/genome/duplication/00-seuils-declares.md`, avec un addendum et des corrections barrées, toutes antérieures à la lecture de l'échantillon.

5.2 La symétrie qui rend la parenté invisible

Proposition 5.1 (Invariance par permutation des neurones)**.** *Soit $\Pi \in \{0,1\}^{f\times f}$ une matrice de permutation. L'expert de poids*

$$(\Pi W_g,\ \Pi W_u,\ W_d\Pi^\top,\ \Pi b_g,\ \Pi b_u,\ b_d)$$

calcule la même fonction que $(W_g, W_u, W_d, b_g, b_u, b_d)$.

Démonstration. L'expert calcule $y = W_d\,\sigma(W_g x + b_g, W_u x + b_u) + b_d$, où σ agit composante par composante : $\sigma(g,u)_k = s(g_k, u_k)$ avec $s(g,u) = \min(g,7)\cdot \text{sigmoïde}(1{,}702\cdot\min(g,7))\cdot(\text{borne}(u,-7,7)+1)$ (la SwiGLU bornée de gpt-oss, `ops::swiglu_oai` de `trc-reference`). Une fonction appliquée composante par composante commute avec toute permutation : $\sigma(\Pi a, \Pi b) = \Pi\,\sigma(a,b)$. Donc $W_d\Pi^\top\sigma(\Pi W_g x + \Pi b_g, \Pi W_u x + \Pi b_u) + b_d = W_d\Pi^\top\Pi\,\sigma(W_g x + b_g, W_u x + b_u) + b_d = y$, puisque $\Pi^\top\Pi = I$. ∎

Conséquences. Deux experts nés d'une même copie puis entraînés peuvent avoir leurs neurones dans un ordre quelconque : comparer W_g^A et W_g^B rangée i contre rangée i ne voit rien. La parenté se juge après le meilleur appariement des neurones, comme on apparie ceux de deux réseaux pour les relier ou les fusionner [79, 80]. Il n'existe pas d'autre symétrie continue : s n'est pas homogène (le seuil 7, la sigmoïde, le « +1 » sur u), donc aucune mise à l'échelle d'un neurone ne se compense par W_d. Un appariement par permutation est la seule liberté.

Ce que la permutation fait aux blocs MXFP4. Les blocs de 32 courent le long de l'entrée de chaque matrice. Pour W_g et W_u (entrée d), Π permute des rangées entières : chaque bloc reste entier. Pour W_d (entrée f), $W_d\Pi^\top$ permute les colonnes : les blocs sont traversés. Un code conditionnel position par position reste exact dans les deux cas (section 5.4).

5.3 La parenté brute : écran, appariement, famille

5.3.1 Deux lemmes pour lire un cosinus

Lemme 5.2 (Le gain gaussien d'un parent)**.** *Soit une rangée enfant x et un parent p, connus du décodeur pour le second, avec $x = \alpha p + z$, $z \perp p$, et $\rho = \cos(x,p)$. Pour une source gaussienne et une distorsion quadratique $D < \sigma_x^2(1-\rho^2)$, coder l'enfant sachant le parent économise*

$$g(\rho) = -\frac{1}{2}\log_2(1-\rho^2) \quad \textit{bit par poids,} \tag{5.2}$$

indépendamment de D.

Démonstration. Par projection orthogonale, $\|x\|^2 = \|\alpha p\|^2 + \|z\|^2$ et $\|\alpha p\|^2 = \rho^2\|x\|^2$, donc $\sigma_z^2 = \sigma_x^2(1-\rho^2)$. La fonction débit-distorsion d'une gaussienne de variance σ^2 vaut $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$ pour $D < \sigma^2$ [9]. L'enfant seul coûte $R_1 = \frac{1}{2}\log_2(\sigma_x^2/D)$; sachant le parent, il reste à coder z, $R_2 = \frac{1}{2}\log_2(\sigma_x^2(1-\rho^2)/D)$, valable tant que $D < \sigma_z^2$. La différence est (5.2). ■

Valeurs [calc.] : $g(0{,}1) = 0{,}0072$; $g(0{,}2) = 0{,}0294$; $g(0{,}3) = 0{,}0680$; $g(0{,}5) = 0{,}2075$; $g(0{,}9) = 1{,}198$ bit. Un cosinus de 0,1 ne vaut qu'un centième de bit.

Lemme 5.3 (Le plancher du hasard)**.** *Pour une cible fixée et N vecteurs indépendants uniformes sur la sphère de $\mathbb{R}^n$, n grand, le maximum des N cosinus en valeur absolue vaut à peu près $\sqrt{2\ln(2N)/n}$.*

Esquisse. Pour u uniforme sur la sphère et e fixé, $\sqrt{n}\,\cos(u,e)$ tend en loi vers une normale standard. Le maximum de N valeurs $|Z|$, soit de $2N$ queues gaussiennes d'un seul côté, se place au premier ordre là où l'espérance du nombre de dépassements vaut 1 : $2N\,\mathbb{P}(Z > t) \approx 1$, et $\mathbb{P}(Z > t) \approx e^{-t^2/2}$ au premier ordre logarithmique, d'où $t \approx \sqrt{2\ln(2N)}$. On divise par $\sqrt{n}$. (Les corrections d'ordre suivant baissent légèrement ce niveau.) ■

Pour le dictionnaire de l'écran ($N = 31 \times 2\,880 = 89\,280$ rangées, $n = 2\,880$) : $\sqrt{2\ln 178\,560/2\,880} = 0{,}0916$ [calc.]. Le témoin de hasard de l'écran rend 0,084 [mes.], du même ordre, un peu dessous parce que les rangées du dictionnaire ne sont pas indépendantes. Tout ρ trouvé au niveau de 0,08 à 0,09 n'est qu'un maximum de corrélations fortuites.

5.3.2 L'écran : une borne haute, sans bijection

`ecran_parente.py` cherche, pour chaque rangée cible (un « gène »), le meilleur $|\cos|$ parmi toutes les rangées de même rôle des 31 autres experts de la couche (89 280 candidats), sans exiger d'appariement un à un : c'est la borne haute d'une parenté gène par gène. Couches 0 à 4 (journal interrompu pour libérer la mémoire de la machine, `ecran-20b-interrompu-couches-0-4.log`) [mes.] : pour gate, $\bar{\rho}$ de 0,12 à 0,21 contre un témoin de hasard de 0,084 (gain gaussien de 0,010 à 0,039 bit) ; pour down, $\bar{\rho}$ de 0,32 à 0,44 contre 0,076 à 0,108 (0,09 à 0,18 bit). Le témoin planté ($\rho_0 = 0{,}9$, `ecran-temoin-planter-0.9.json`) est retrouvé à 0,900 et 1,198 bit, et le dictionnaire de hasard ne le voit pas (0,083).

5.3.3 L'appariement un à un : la vraie parenté d'expert à expert

`paires_arbre.py` : coût $\sum_{\text{rôles}} \cos^2$ entre neurones, affectation par l'algorithme hongrois [81] (l'implémentation de scipy [82]), une seule permutation jointe sur les trois matrices (la fonction l'exige), score = moyenne de $-\frac{1}{2}\log_2(1-\cos^2)$ sur les neurones appariés. Couche 12, quatre experts (tableau 5.1).

Gate et up appariés sont au niveau du hasard ; seul down porte un peu de structure. Le témoin planté est retrouvé au millième de bit : l'instrument lit g.

5.3.4 Le meilleur parent de chaque enfant

L'échantillon déclaré : les enfants 0, 8, 16 et 24 des couches 0, 8, 16 et 23 (16 enfants, 48 matrices). Pour chacun, `mutation.py` cherche le meilleur parent parmi 63 candidats (31 de sa couche, 32 de la couche précédente ; `csv/scores-parente.csv`). La figure 5.1 montre les 880 scores.

Paire	cos gate	cos up	cos down	score (bit par poids)
(0, 2)	0,044	0,034	0,108	0,0053
(0, 3)	0,039	0,034	0,106	0,0050
(2, 3)	0,049	0,034	0,107	0,0049
hasard	0,040	0,041	0,039	0,0016
plantée, $\rho_0 = 0{,}5$	0,500	0,500	0,500	0,2077 (attendu 0,2075)

Table 5.1 : L'appariement hongrois de quatre experts de la couche 12, contre un parent de hasard et une copie plantée [mes.]. Données : `mesures/genome/duplication/paires-temoin-planter-0.5.json`. La permutation plantée est retrouvée à 100 %.

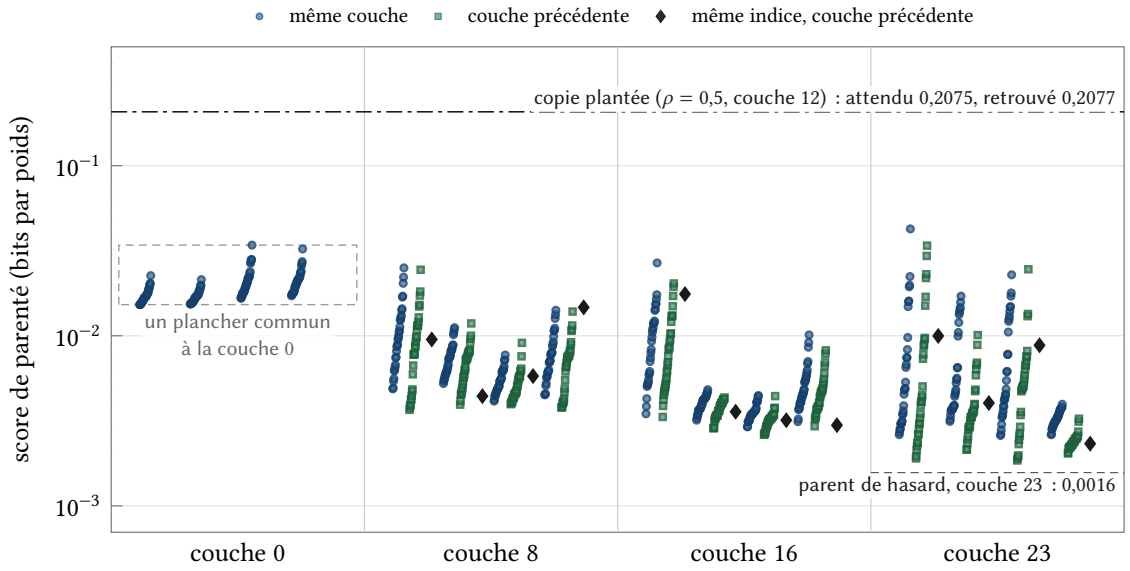


Figure 5.1 : Aucun expert n'est la copie d'un autre : chaque parent candidat reste loin sous une copie plantée. [mes.] Pour 4 experts enfants dans chacune de 4 couches de gpt-oss-20b, le score de parenté de chaque parent candidat : les bits par poids que la meilleure permutation de ses neurones (appariement hongrois sur gate, up et down ensemble) économise sur l'enfant, $-\frac{1}{2}\log_2(1-\rho^2)$ en moyenne sur les trois matrices. Chaque colonne de points est un enfant, ses candidats rangés par score : les 31 autres experts de la même couche (disques bleus), 31 experts de la couche précédente (carrés verts), l'expert de même indice dans la couche précédente (losanges à l'encre). Les 880 scores tiennent entre 0,0018 et 0,0425 bit. Le plus haut (enfant 0, parent 22, couche 23) vaut ×27 le témoin du parent de hasard de sa propre couche (0,0016, 8 paires ; celui de la couche 12, 2 paires, rend 0,0016), et 4,9 fois moins qu'une copie plantée à $\rho = 0{,}5$, que le même instrument retrouve (trait mixte : un contrôle de l'instrument, mesuré à la couche 12). À la couche 0, les 124 candidats de même couche ont tous un score de 0,0153 à 0,0341 (cadre tireté), avec un cosinus des matrices down de 0,23 à 0,31 (médiane de 0,11 aux autres couches) : un plancher commun à toute la couche, une statistique partagée de ses colonnes, pas une parenté ; aucun témoin du hasard n'a été mesuré à cette couche. Ce que la figure ne montre pas : le coût de la permutation et de l'indice du parent (ajouté dans le compte sans perte D1, qui tombe), ni le 120b, mesuré sur une seule couche.

Données : `mesures/genome/duplication/csv/scores-parente.csv`, `paires-temoin-planter-0.5.json`, `arbre-c23.json` ; script `memoire/outils/tome3_duplication.py`.

Ce que la figure montre. Tous les scores tiennent entre 0,0018 et 0,0425 bit par poids [mes.]. Le plus haut, l'enfant (23, 0) et son parent (23, 22), vaut 27 fois le témoin de hasard de sa couche (0,0016, moyenne de 8 paires ; `arbre-c23.json`) et reste 4,9 fois sous une copie plantée à $\rho = 0{,}5$, que le même instrument retrouve. Le meilleur parent sort à peine du lot : son score dépasse la médiane des candidats de 0,0008 à 0,038 bit (`genome/duplication.md` § 2.3). Douze parents sur seize sont dans la même couche, quatre dans la précédente, dont un seul au même indice : (7, 24) pour (8, 24), à 0,0147 bit, la seule trace d'une filiation par indice, et elle est minuscule. Le même indice une couche plus bas n'est pas privilégié : son score est, selon l'enfant, au-dessous ou au-dessus de la médiane des 32 candidats de la couche précédente.

La couche 0, un plancher commun. Dans la couche 0, les quatre enfants choisissent le même parent, (0, 9) : un *moyeu*, pas une lignée. Les 124 candidats de même couche y ont tous un score de 0,0153 à 0,0341, avec un cosinus des matrices down de 0,23 à 0,31 (médiane 0,11 aux autres couches) [mes.] : une statistique partagée par toutes les colonnes de la couche, pas une parenté. La légende le dit, et dit aussi qu'aucun témoin du hasard n'a été mesuré à cette couche ; le coût exact du section 5.4 le mesure par un autre chemin.

5.3.5 La famille : un sous-espace commun

`spectre_famille.py` calcule, pour la couche 12, le débit de Shannon d'une gaussienne de même spectre à $\varepsilon = 4{,}29\,\%$ par remplissage inverse (`spectre-c12.json`) [mes.] : une base commune de la couche (la transformée de Karhunen-Loève de la famille) rendrait 0,06 à 0,07 bit de plus qu'une base diagonale, nets du témoin (gate 0,072, up 0,056, down 0,070). Mais elle coûte $n^2 = 8{,}3 \cdot 10^6$ nombres par couche et par rôle, soit 0,5 bit par poids en f16 répartis sur 32 experts [calc.] : sept fois son gain. Le gain diagonal de down (0,187 bit) est déjà pris par les échelles E8M0, qui suivent la norme de chaque rangée de down.

5.4 Le coût exact d'un expert codé comme mutation

5.4.1 Les énoncés

Proposition 5.4 (Le gain sans perte est une information mutuelle conditionnelle)**.** *Soit C le code E2M1 (16 symboles) d'un poids de l'enfant, κ sa classe d'échelle (la classe du format : signe de e moins la médiane haute de la rangée, 3 classes) et P le code du parent à la position appariée. Un codeur entropique à une table par contexte κ dépense $H(C \mid \kappa)$ bits par poids ; à une table par (κ, P), $H(C \mid \kappa, P)$. Le gain est*

$$H(C \mid \kappa) - H(C \mid \kappa, P) = I(C; P \mid \kappa) \;\geq\; 0, \tag{5.3}$$

nul si et seulement si C et P sont indépendants sachant κ.

Démonstration. C'est la définition de l'information mutuelle conditionnelle, $I(C; P \mid \kappa) = H(C \mid \kappa) - H(C \mid \kappa, P)$; sa positivité et le cas d'égalité sont ceux de [2, th. 2.6.5 et 2.5.2] : elle est une moyenne de divergences de Kullback-Leibler entre $p(c, p' \mid \kappa)$ et $p(c \mid \kappa)p(p' \mid \kappa)$, nulle si et seulement si ces lois coïncident. ∎

Le biais de l'estimateur. H est estimée par comptes exacts (« plug-in »). Son biais vers le bas vaut, au premier ordre, $(K - 1)/(2N \ln 2)$ pour K cases non vides et N échantillons [83]. Pour $H(C \mid \kappa, P)$: $K \leq 3 \times 16 \times 16 = 768$ et $N = 8\,294\,400$, d'où un biais d'au plus $767/(2 \times 8\,294\,400 \times \ln 2) = 6{,}7 \cdot 10^{-5}$ bit [calc.]. L'information mutuelle estimée est gonflée d'au plus ce montant : tout gain sous 10^{-4} bit est indiscernable de zéro.

Les informations ajoutées, comptées [calc.].

– *La permutation* : $\log_2(f!)$ bits par expert, partagés par ses trois matrices.

$$\log_2(2\,880!) = \frac{\mathrm{lgamma}(2\,881)}{\ln 2} = 28\,948{,}6 \text{ bits}; \qquad \text{par poids, } \frac{28\,948{,}6}{3 \times 2\,880^2} = 1{,}163 \cdot 10^{-3}.$$

Second chemin, par Stirling :

$$f \log_2 f - f \log_2 e + \tfrac{1}{2} \log_2(2\pi f) = 33\,096{,}5 - 4\,155{,}0 + 7{,}1 = 28\,948{,}6.$$

– *Les tables* : 768 comptes de 16 bits par matrice pour D1, soit $768 \times 16/8\,294\,400 = 1{,}48 \cdot 10^{-3}$ bit par poids ; 192 comptes pour D2b ($\kappa \times 8$ symboles de base ×8), $3{,}70 \cdot 10^{-4}$.
– Un parent n'est donc utile que si $I(C; P \mid \kappa)$ dépasse $2{,}64 \cdot 10^{-3}$ bit par poids pour D1, $1{,}53 \cdot 10^{-3}$ pour D2b.

Le parent de hasard n'est pas un zéro. Apparier l'enfant à son parent par une permutation tirée au hasard laisse en place les colonnes (pour gate et up, la dimension d'entrée j reste j) : toute structure commune à *tous* les experts selon la colonne survit. Le témoin de hasard mesure cette part commune ; la part propre à la parenté est la différence $I(C; P \mid \kappa) - I(C; P_{\text{hasard}} \mid \kappa)$.

Proposition 5.5 (Le gain à reconstruction identique)**.** *Si la reconstruction de l'enfant est fixée (la base de B, ou un ECSQ seul), d'indices Q, et qu'un contexte parent P' (le symbole de base du parent, ou sa prédiction quantifiée) entre dans le codeur, l'erreur sur les poids et sur la sortie ne change pas d'un bit, et le débit passe de $H(Q \mid \text{ctx})$ à $H(Q \mid \text{ctx}, P')$: le gain est $I(Q; P' \mid \text{ctx})$, avec les mêmes biais et les mêmes coûts qu'à la proposition 5.4.*

Démonstration. La reconstruction est une fonction de Q seul ; ajouter un contexte au codeur entropique ne change que la loi selon laquelle Q est codé, donc que le débit, et la proposition 5.4 s'applique à Q au lieu de C. ■

C'est la seule manière de créditer la parenté sans la confondre avec un changement de quantificateur.

Remarque 5.6 (Pourquoi la prédiction soustraite perd sur une grille discrète). Les poids de l'enfant vivent sur une grille discrète (16 valeurs par bloc). Un quantificateur uniforme dont le pas Δ tombe sur la grille (Δ multiple de 2^{e-128}) rend une erreur *nulle* sur une partie des poids. Le résidu $x - \alpha p$ n'est plus sur la grille : son erreur de quantification vaut environ $\Delta^2/12$ partout. À débit égal, l'ECSQ du résidu perd ce que la grille donnait gratuitement, et ce que le parent rend, $g(\rho)$ du lemme 5.2, est plus petit dès que ρ est petit.

5.4.2 Les mesures

Tableaux par matrice : `csv/d1-sans-perte.csv`, `csv/d2-avec-perte.csv`, `csv/d2b-base.csv` ; verdicts : `verdicts.json` (outil `synthese.py`, dont le témoin `--casser` fait « marcher » D1 partout et tient ; `verdicts-casse.json`). Le tableau 5.2 en donne les moyennes par couche et par rôle.

Lecture (tableau 5.3). Sans perte, le parent rend en moyenne 0,0034 bit par poids sur les codes, dont 0,0016 se retrouvent avec un parent de hasard : la part propre à la parenté est 0,0019 bit, sous les $2{,}64 \cdot 10^{-3}$ de coûts ajoutés. Avec perte, à reconstruction identique, la frontière de 2,6441 bits descendrait au mieux à 2,6429. La mutation soustraite *perd* 0,49 bit, comme la remarque 5.6 l'annonçait. Le plus grand gain d'une seule matrice est 0,0170 bit (gate de (0, 16)), dont 0,0145 se retrouvent au parent de hasard ; le plus grand gain net est 0,0109 bit (gate de (23, 0), parent (23, 22)), puis 0,0098 (gate de (23, 16), parent (22, 13)). Rien n'approche le dixième de bit.

couche	rôle	$H(C \mid \kappa)$	D1 parent	D1 hasard	D2b parent	D2b hasard	ECSQ seule	soustraite	ECSQ + parent
0	gate	3,7686	0,0082	0,0072	0,0064	0,0056	1,564	1,882	1,562
0	up	3,7855	0,0064	0,0060	0,0050	0,0047	1,559	1,929	1,558
0	down	3,8185	0,0002	0,0001	0,0002	0,0000	1,049	1,283	1,043
8	gate	3,8208	0,0045	0,0004	0,0040	0,0003	1,538	2,049	1,533
8	up	3,8005	0,0008	0,0004	0,0006	0,0003	1,655	2,119	1,653
8	down	3,7984	0,0017	0,0001	0,0013	0,0000	1,626	2,168	1,619
16	gate	3,8099	0,0033	0,0004	0,0029	0,0002	1,490	1,999	1,485
16	up	3,8098	0,0007	0,0003	0,0005	0,0002	1,564	2,107	1,562
16	down	3,7988	0,0011	0,0001	0,0009	0,0000	1,568	2,177	1,564
23	gate	3,7978	0,0082	0,0019	0,0069	0,0013	1,449	2,091	1,439
23	up	3,7921	0,0041	0,0018	0,0031	0,0013	1,607	2,123	1,601
23	down	3,7892	0,0020	0,0001	0,0015	0,0000	1,584	2,121	1,579

Table 5.2 : Le coût exact d'un expert codé comme mutation, moyennes de 4 enfants par couche et par rôle, en bits par poids [mes.]. D1 : gain sans perte sur les codes ; D2b : gain à reconstruction identique à la base ; les trois dernières colonnes : débit à 4,29 % d'erreur de poids d'un ECSQ seul, de la mutation soustraite, et de l'ECSQ avec le parent en contexte (sans les échelles, 0,034 bit, ni α, 0,0028). Données : `mesures/genome/duplication/csv/d1-sans-perte.csv`, `d2-avec-perte.csv`, `d2b-base.csv`.

Mesure	Ce qu'elle exige (déclaré)	Résultat [mes.]	Verdict
D1, sans perte	gain $> 2{,}64 \cdot 10^{-3}$ sur les 48 matrices	gain moyen 0,0034, hasard 0,0016, net 0,0019 ; 21 sur 48	**ne marche pas**
D2, mutation soustraite	sous 2,6441 et sous l'ECSQ seule de plus d'un écart-type du hasard ($3{,}0 \cdot 10^{-4}$)	−0,486 bit en moyenne : pire que sans parent	**ne marche pas**
D2c, contexte parent, même reconstruction que l'ECSQ	idem	+0,0016 bit, α compris ; 43 sur 48 battent le hasard de plus d'un écart-type	gain réel, sans portée
D2b, base + parent, même reconstruction que la base	gain $> 1{,}53 \cdot 10^{-3}$ et au-dessus du hasard sur les 48	gain 0,0028, hasard 0,0012 ; 24 sur 48 ; frontière 2,6441 → 2,6429	**ne marche pas**
O, sortie	D2 au plus l'erreur de la base, 4 couches sur 4	base 6,4 %, mutation 8,6 % ; 4 enfants sur 16 sous la base	**ne marche pas**

Table 5.3 : Les verdicts de la duplication contre les seuils déclarés [mes.]. Données : `mesures/genome/duplication/verdicts.json` ; `genome/duplication.md` § 3.

enfant	parent	jetons	base de B poids / sortie	mutation à $\varepsilon \approx 4{,}29\,\%$	ECSQ seule à $\varepsilon \approx 4{,}29\,\%$	mutation tronquée poids / sortie	parent brut sortie
(0, 0)	(0, 9)	762	4,3 / 6,6	4,7 / 7,1	4,8 / 7,0	94,3 / 97,4	217,0
(0, 8)	(0, 9)	698	4,3 / 6,3	4,6 / 6,5	4,7 / 6,7	95,3 / 100,3	222,1
(0, 16)	(0, 9)	1 334	4,6 / 4,5	5,4 / 5,5	4,6 / 5,1	93,4 / 103,6	95,9
(0, 24)	(0, 9)	914	4,5 / 4,8	4,5 / 5,2	4,5 / 4,3	91,3 / 94,0	101,0
(8, 0)	(8, 31)	1 329	4,7 / 8,3	3,9 / 18,5	5,1 / 29,3	92,5 / 120,3	168,3
(8, 8)	(7, 30)	1 761	4,5 / 7,2	4,1 / 11,4	4,1 / 11,3	93,7 / 106,1	121,0
(8, 16)	(7, 11)	300	4,2 / 7,0	3,9 / 22,9	3,9 / 31,5	94,9 / 105,6	144,4
(8, 24)	(7, 24)	928	5,4 / 20,3	4,5 / 12,5	4,2 / 12,8	93,2 / 121,5	255,4
(16, 0)	(16, 17)	330	4,5 / 6,1	4,0 / 11,1	3,9 / 11,4	95,0 / 77,8	113,0
(16, 8)	(16, 24)	240	4,0 / 3,1	3,8 / 2,9	3,8 / 3,1	98,3 / 64,8	113,0
(16, 16)	(16, 6)	147	4,1 / 5,2	4,1 / 8,2	3,8 / 11,1	98,5 / 83,3	135,0
(16, 24)	(16, 11)	1 237	4,2 / 5,5	4,6 / 7,0	4,2 / 7,1	97,6 / 82,9	207,5
(23, 0)	(23, 22)	303	4,7 / 6,2	4,8 / 8,3	3,6 / 6,5	95,5 / 90,0	54,9
(23, 8)	(23, 16)	56	4,5 / 3,3	3,8 / 2,3	3,8 / 2,7	96,1 / 80,0	56,2
(23, 16)	(22, 13)	239	4,7 / 4,4	3,9 / 5,5	4,5 / 5,9	96,2 / 107,1	176,2
(23, 24)	(23, 11)	2 048	3,9 / 3,6	3,8 / 3,5	3,7 / 4,6	99,0 / 24,2	28,6
moyenne			— / **6,4**	— / **8,6**	— / **10,0**	**95,3 / 91,2**	

Table 5.4 : Erreurs relatives en %, poids puis sortie de l'expert, sur les entrées réelles routées vers chaque enfant [mes.]. Données : `mesures/genome/duplication/csv/t-o-sortie.csv` ; `genome/duplication.md` § 4.

5.5 La mutation tronquée et la sortie de l'expert

Entrées : les états normés réellement routés vers chaque enfant (capture de l'éclaireur, 12 fenêtres de 511 jetons, empreintes SHA-256 dans chaque `mutation-cL.json`), au plus 2 048 jetons par enfant ; sortie de l'expert en f64, biais compris ; témoin : les poids exacts rendent une erreur nulle (`csv/t-o-sortie.csv`, colonne `temoin_exact`) [mes.].

Lecture (tableau 5.4).

- **La mutation tronquée** (l'enfant servi par $\alpha\odot$ parent, résidu jeté) garde de 1 à 9 % de l'énergie des poids (4,7 % en moyenne) et rend de 24 à 121 % d'erreur sur la sortie. Les valeurs sous 100 % des couches 16 et 23 viennent des biais : ils restent exacts et portent une grande part de la sortie des experts profonds (pour (23, 24), le parent brut lui-même ne fait que 28,6 %). C'est l'erreur qu'on aurait en ne servant que les biais : la mutation tronquée n'apporte rien.
- **Même erreur de poids, erreur de sortie très différente.** L'ECSQ seule et la mutation tombent à 3,6 à 5,4 % d'erreur de poids, comme la base ; mais l'erreur de sortie de l'ECSQ seule va de 0,63 à 4,5 fois celle de la base : le pas commun efface les blocs de petite échelle, que la base garde. Le chapitre 6 le voit de son côté, par un autre chemin.
- (8, 24) est l'exception où la base elle-même rend 20,3 % : son erreur de sortie n'est pas uniforme non plus. C'est l'expert aberrant de l'éclaireur (section 3.4), vu ici sur d'autres entrées.
- Le pas de quantification pour O est celui du balayage le plus proche de 4,29 % (pas d'interpolation possible sur une reconstruction) : d'où les erreurs de poids de 3,6 à 5,4 %.

5.6 L'arbre le plus économe

Proposition 5.7 (L'ordre de codage optimal est un arbre couvrant de poids maximal)**.** *Si chaque expert se code seul (B bits) ou comme mutation d'un parent déjà décodé ($B - s$ bits, $s \geq 0$ le gain de l'arête, symétrique), le coût minimal d'un ordre de décodage valide de E experts est $BE - \max_T \sum_{(i,j)\in T} s_{ij}$, le maximum portant sur les arbres couvrants T du graphe complet des experts, et l'algorithme de Kruskal [84] le trouve.*

Démonstration. Un ordre de décodage valide donne à chaque expert sauf un (la racine) un parent décodé avant lui : les arêtes « enfant, parent » forment un arbre couvrant orienté depuis la racine, et le coût est $BE - \sum_T s$. Réciproquement, tout arbre couvrant, enraciné en un expert quelconque et parcouru en largeur, donne un ordre valide de même coût, puisque s est symétrique. Minimiser le coût revient donc à maximiser $\sum_T s$. Le choix glouton de Kruskal (prendre l'arête la plus lourde qui ne ferme pas de cycle) est optimal par la propriété de coupe : pour toute coupe du graphe, une arête de poids maximal qui la traverse appartient à un arbre couvrant de poids maximal. (Pour des gains non symétriques, il faudrait l'arborescence de poids maximal de Chu-Liu et Edmonds ; nos scores sont symétriques par construction, $\cos^2$ et appariement.) ■

Le gain par poids de la couche vaut $\sum_T s/E$, et il ne dépasse pas $\frac{E-1}{E} \max s$. `paires_arbre.py--couche23` apparie une à une les 496 paires de la couche la plus « parente » de l'échantillon, puis trace l'arbre (`arbre-c23.json`) [mes.] : scores des paires, 10 % / médiane / 90 % / 99 % : 0,0031 / 0,0038 / 0,0138 / 0,0689 bit ; témoin de hasard (8 paires) : 0,0016 ; arêtes de l'arbre, min / médiane / max : 0,0037 / 0,0101 / 0,0892. **Gain de la couche codée par son arbre : 0,0212 bit par poids**, en borne gaussienne.

Ce que la figure montre. La figure 5.2 dessine l'arbre par sa propre structure, pas par une projection : une étoile autour de l'expert 31, qui porte les quatre arêtes les plus lourdes, (17, 31) à 0,089, (5, 31) à 0,089, (15, 31) à 0,077, (25, 31) à 0,069, puis des branches minces ; 16 experts sur 32 sont des feuilles. L'expert 31 est un moyeu, la seule trace d'une famille dans toutes les mesures. L'encart est l'ombre des mêmes experts par une MDS classique sur $d_{ij} = \log_2(s_{\max}/s_{ij})$; sa fidélité est dite dans la légende et doit l'être dans le texte : le plan garde 30,9 % de la variance des distances, et la corrélation de rangs entre distances du plan et distances vraies vaut 0,61, contre 0,11 pour des distances permutées (`phylogenie-c23.json`) [mes.]. Deux voisins sur cette ombre sont souvent, pas toujours, voisins dans les données. Le juge des figures a noté qu'en figure 5.2 quatre valeurs d'arête sur dix sont plus près d'une autre arête que de la leur (`memoire/tome3/JUGEMENT-figures3.md`, M4, réserve sur une figure qui tient) : la valeur d'une arête se lit dans le CSV, pas à l'œil. Et surtout, la figure n'est pas une généalogie : aucune arête n'est une copie, l'arbre est l'ordre de codage le moins cher, pas une histoire.

L'arête la plus lourde, en vrais bits. `mutation.py--couche23--enfants17--parent23,31` (`mutation-arete-max-c23-e17-p31.json`) [mes.] : D1 parent 0,0222 / 0,0136 / 0,0149 bit (gate, up, down), D1 hasard 0,0003 / 0,0004 / 0,0001, D2b 0,0199 / 0,0108 / 0,0117 ; erreur de poids de la mutation tronquée 83,9 / 91,4 / 91,4 %. Sortie de l'expert 17 : base 10,4 %, mutation à $\varepsilon \approx 4{,}29$ % 15,3 %, mutation tronquée 109,9 %. C'est le meilleur cas trouvé dans le modèle : 0,017 bit de moyenne, 0,014 net des coûts ajoutés, soit 3,84 → 3,83. Le rapport entre le gain réel sur les codes et le score gaussien vaut ici $0{,}017/0{,}089 = 0{,}19$, et $0{,}0076/0{,}0425 = 0{,}18$ pour (23, 0) ; appliqué à l'arbre, il donne environ **0,004 bit par poids pour la couche 23 entière** [calc.], avant les coûts ajoutés ($2{,}6 \cdot 10^{-3}$) : un gain net de l'ordre du millième.

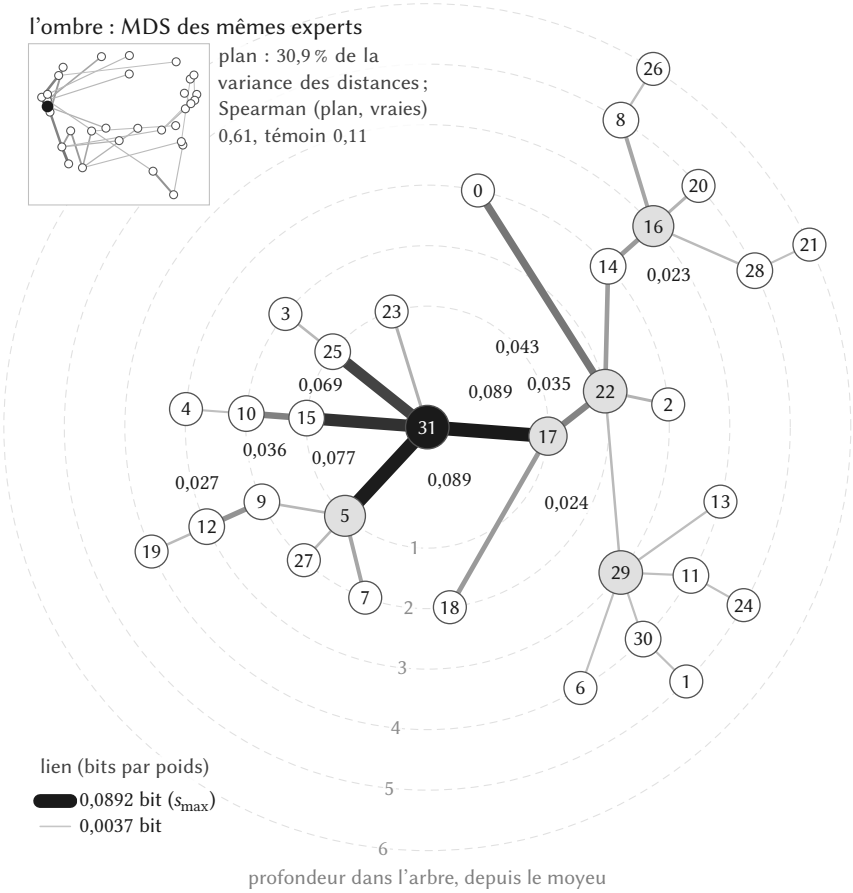


Figure 5.2 : L'ordre de codage le moins cher des 32 experts de la couche 23 est une étoile autour de l'expert 31, puis des branches minces. [mes.] [calc.] Chaque paire d'experts (496 paires) reçoit un score de parenté s_{ij} ; l'arbre couvrant de poids maximal (Kruskal) garde les 31 arêtes qui coûteraient le moins de bits si chaque expert était codé sachant son parent. L'arbre est dessiné par sa propre structure, pas par une projection : le moyeu au centre, chaque expert sur le cercle de sa profondeur, chaque sous-arbre dans un secteur proportionnel à ses feuilles plus une, si bien qu'aucune arête n'en croise une autre (un calcul de dessin, pas une mesure). L'épaisseur et la profondeur du gris des arêtes suivent $s_{ij}/s_{\max}$, et les liens au-dessus du quart de $s_{\max}$ portent leur valeur, posée à côté de l'arête, là où sa boîte ne touche aucun autre trait, disque ni valeur (un calcul de dessin). La taille des disques suit le degré ; les disques teintés ont au moins trois voisins. 3 experts ont le plus haut degré, 5 (22, 29, 31) ; l'expert 31 est le moyeu par le poids (disque à l'encre) : ses arêtes pèsent ensemble 0,335 bit, contre 0,116 pour l'expert 22, et les 4 arêtes les plus lourdes de l'arbre le touchent toutes ; 16 experts sont des feuilles. Échelle : le lien le plus fort vaut 0,089 bit par poids, contre 0,0016 pour un parent de hasard de la même couche. En encart, l'ombre des mêmes experts par une MDS classique sur $d_{ij} = \log_2(s_{\max}/s_{ij})$, le dessin du premier passage ; le plan garde 30,9 % de la variance des distances, et la corrélation de rangs entre distances du plan et distances vraies vaut 0,61 (témoin à distances permutées : 0,11) : deux voisins sur cette page sont souvent, pas toujours, voisins dans les données. Ce que la figure ne montre pas : une généalogie. Aucune arête n'est une copie ; l'arbre est l'ordre de codage le moins cher, pas une histoire.

Données : `mesures/genome/duplication/csv/phylogenie-c23-noeuds.csv`, `phylogenie-c23-aretes.csv`, `phylogenie-c23.json` (`outils/genome/duplication/figure_phylogenie.py`), `arbre-c23.json` ; script `memoire/outils/tome3_duplication.py`.

couche	ε meilleur paralogue	ε, $\beta = 1$	ε hasard	ε, 5e choix	couche : substitué	couche : retiré
0	0,792	1,401	0,945	1,215	0,167	0,133
8	0,842	1,439	0,936	1,239	0,139	0,092
16	0,658	0,749	0,721	0,924	0,071	0,076
23	0,335	0,435	0,467	0,564	0,037	0,037

Table 5.5 : L'erreur relative d'un expert servi par un autre, moyenne sur les experts de chaque couche (32, 32, 31, 31), au plus 96 jetons routés par expert [mes.]. Routage recalculé depuis le routeur F32 du GGUF, égal au routage capturé à 100 % (témoin). Les deux dernières colonnes : erreur relative de la sortie de la *couche* quand l'expert est substitué par son meilleur paralogue, ou retiré (routage renormalisé sur les trois autres). Données : `mesures/genome/duplication/csv/paralogues.csv`, `paralogues-c*.json`.

Sur gpt-oss-120b. `paires_arbre.py--modele120b--couche18--experts8` (28 paires, `paires-120b-c18-8experts.json`) [mes.] : scores de 0,0030 à 0,0054 bit (hasard 0,0016) ; cosinus appariés 0,038 (gate), 0,032 (up), 0,094 (down) contre 0,040 au hasard. Le même tableau que sur le 20b. Le livrable du sujet le dit en barrant sa propre phrase : « ~~gpt-oss-120b n'est pas mesuré~~ » ; il l'est sur une couche et 8 experts sur 128, et le coût exact (D1, D2b) et les paralogues n'y sont pas refaits.

5.7 Les paralogues : servir un expert absent par un autre

Un défaut de cache pourrait se servir sans lire un octet, par un expert déjà résident : un paralogue, au sens de l'analogie. Le vivant tire de ses familles de gènes une robustesse : un gène retiré est souvent compensé par un paralogue [85]. Le routage conscient du cache [86] favorise la réutilisation d'experts déjà en mémoire et se juge en précision de tâches et en vitesse ; on mesure ici la borne fonctionnelle de toute substitution.

Lemme 5.8 (L'erreur d'un paralogue)**.** *Soit y_c la sortie de l'expert manquant et y_s celle d'un expert résident, sur les entrées routées vers c. Avec un gain scalaire β choisi aux moindres carrés,*

$$\varepsilon(c,s) = \min_{\beta} \frac{\sum \|y_c - \beta y_s\|^2}{\sum \|y_c\|^2} = 1 - \rho_f^2, \qquad \rho_f = \frac{\langle y_c, y_s\rangle}{\|y_c\|\,\|y_s\|}, \quad \beta^* = \frac{\langle y_c, y_s\rangle}{\|y_s\|^2}, \tag{5.4}$$

le produit scalaire portant sur tous les jetons et toutes les composantes.

Démonstration. $\|y_c - \beta y_s\|^2 = \|y_c\|^2 - 2\beta\langle y_c, y_s\rangle + \beta^2\|y_s\|^2$ est un trinôme en β, minimal en β^*, où il vaut $\|y_c\|^2 - \langle y_c, y_s\rangle^2/\|y_s\|^2 = \|y_c\|^2(1-\rho_f^2)$. ∎

Pour la sortie de la couche, l'expert c intervient avec son poids de routage $g_c \in (0,1)$: l'erreur relative de la couche est environ $\sum g_c^2\|y_c - y_s\|^2 / \sum \|\sum_j g_j y_j\|^2$, plus petite que ε d'un facteur que la mesure donne. Le paralogue ne lit aucun octet : il bat toute compression en débit. Il ne vaut que si son erreur est au niveau de celle de la base.

Lecture (tableau 5.5). Les sorties des experts se ressemblent davantage en profondeur, mais par une composante *commune à tous* : le témoin de hasard tombe avec elles (0,47 à la couche 23), et ce qui est propre au meilleur paralogue n'est que 0,13 au-dessus du hasard. Un défaut de cache se sert bien mieux par la base de B (erreur d'expert de 5 à 8 %, tableau 5.4) que par le meilleur voisin (33 à 84 %).

Une phrase des sources, corrigée. Le livrable du sujet et la synthèse de la ronde écrivent : « dans la sortie de la couche, ~~substituer n'est jamais meilleur que retirer l'expert~~ (16,7 contre 13,3 %; 13,9 contre 9,2; 7,1 contre 7,6; 3,7 contre 3,7 %) » (`genome/duplication.md` § 0, point 6; `genome/05-synthese.md` § 3.2). Les nombres entre parenthèses la démentent : à la couche 16, la moyenne de l'erreur substituée (0,071) est *sous* celle de l'erreur retirée (0,076). Refait ici depuis `csv/paralogues.csv` [calc.] sur [mes.] : substituer est meilleur que retirer pour 4 experts sur 32 à la couche 0, 5 sur 32 à la couche 8, **25 sur 31 à la couche 16** et **21 sur 31 à la couche 23**, où les deux moyennes sont égales à 10^{-3} près. La phrase juste est donc : *retirer l'expert vaut mieux que le substituer aux couches 0 et 8; aux couches 16 et 23, le meilleur paralogue fait un peu mieux que le retrait pour la majorité des experts, mais d'au plus un demi-point d'erreur de couche en moyenne.* La conclusion du chapitre ne change pas : la barre d'un paralogue n'est pas l'expert absent mais le retrait de l'expert, et le meilleur paralogue ne s'en écarte que de quelques millièmes, loin de ce que rend la base.

5.8 Pourquoi l'idée échoue, et où chercher

Le pourquoi, chiffré. Un gène dupliqué garde sa ressemblance tant que la sélection ne l'en éloigne pas. Les experts de gpt-oss, eux, ont été entraînés pour être *différents* : le routeur n'envoie à un expert que les jetons où il est utile, et deux experts qui se ressembleraient se partageraient les mêmes jetons. La mesure le confirme : après la meilleure permutation, l'information mutuelle entre un enfant et son meilleur parent est de l'ordre de 10^{-3} bit par poids sur 3,84, et ce qui en reste est pour moitié ou davantage une propriété de la couche entière (les colonnes d'entrée, l'espace de sortie de down), pas de la paire. Tronquer la mutation, c'est perdre l'expert : 96 % de son énergie est à lui. *Ils posent le problème ainsi* (Git Re-Basin, Entezari et al., MC-SMoE [23]) : apparier les neurones de deux réseaux ou de deux experts pour les fusionner, jugés en précision de tâches. *Le terme qui nous manquait* : le débit, en bits par poids lus, d'un expert codé sachant un autre. *Celui qu'ils n'ont pas* : la mesure sur des codes de 4 bits, où l'information mutuelle se compte exactement, et la séparation entre la parenté propre et la structure commune à la couche, par le témoin du parent de hasard.

Où est le bruit, où est la structure. (a) Une part de la structure entre experts est **commune à la couche**, pas propre à une paire : le parent de hasard (mêmes colonnes d'entrée, neurones dans un ordre quelconque) reprend 46 % du gain en moyenne, et 88 à 94 % dans la couche 0, où l'état caché porte des canaux aberrants. Ce sont des statistiques par colonne d'entrée partagées par les 32 experts; le chapitre 6 les prend comme contexte de colonne, et c'est le seul contexte qui passe son seuil. (b) La parenté propre croît avec la profondeur (couche 23 : D1 gate 0,0082 contre 0,0019 au hasard; un moyeu, l'expert 31) mais reste sous le centième de bit. (c) Le reste est du bruit pour toute duplication : le meilleur parent explique 1 à 9 % de l'énergie des poids d'un enfant (au plus 16 % pour une seule matrice).

Ce que la mesure a vu en passant. L'ECSQ seule (pas commun, contexte d'exposant) tombe à 0,89 à 1,78 bit (1,52 en moyenne, sans les 0,034 bit d'échelles) à 4,29 % d'erreur de *poids*, loin sous la frontière; mais son erreur de *sortie* varie de 0,63 à 4,5 fois celle de la base. Le levier n'est pas la parenté, une propriété des poids entre eux, mais l'*usage*, une propriété des poids devant leurs entrées : un poids vaut ce que ses entrées réelles le font peser. Le sujet en a tiré son orientation pour la suite (réponse par délégation du 2026-10-17, réversible : c'est le seul mécanisme mesuré ici dont le gain se compte en dixièmes de bit et non en millièmes); le chapitre 6 mesure cette allocation par l'usage, et le chapitre 8 dira ce que la KL en a laissé.

Ce qui pourrait être faux

- **Le parent est choisi par un score gaussien sur les poids**, pas par le gain sur les codes. Un autre parent pourrait rendre un peu plus de D1. Le meilleur score de toute la liste (0,043) borne le gain gaussien de tout parent ; le D1 réel est toujours sous le score.
- **Une seule permutation par paire, jointe sur les trois rôles.** Des permutations par rôle seraient plus libres mais ne décriraient plus le même expert (la fonction exige la même Π). L'écran sans bijection borne ce que toute liberté de plus pourrait rendre.
- **L'échantillon** : 16 enfants sur 768, quatre profondeurs. La couche 23 montre une parenté plus forte que les autres ; les couches 17 à 22 ne sont pas mesurées. L'écran a été interrompu à la couche 4.
- **Les entrées réelles** viennent de 12 fenêtres d'un seul corpus ; au plus 2 048 jetons par enfant, 78 à 96 pour les paralogues.
- **L'estimateur d'entropie** est biaisé vers le bas d'au plus $6{,}7 \cdot 10^{-5}$ bit : négligeable devant les gains rapportés, mais du même ordre que les plus petits.
- **Le lemme 5.3** est une asymptote ; le témoin mesuré (0,084) est un peu sous elle (0,092).
- **La phrase corrigée au section 5.7** l'est sur des moyennes de 31 et 32 experts à 96 jetons au plus ; l'écart entre substituer et retirer aux couches 16 et 23 est de quelques millièmes, sans incertitude publiée.
- **La machine était partagée et chargée** (mémoire libre de 0,7 à 2,6 Go) : les durées des journaux ne mesurent rien, et aucun chiffre de débit n'est rapporté.

Chapitre 6

Codage prédictif et élagage synaptique

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, sur les 12 tenseurs d'experts de la frontière (gpt-oss-20b, couches 0, 8, 16 et 23, gate, up et down ; 3 185 049 600 poids), le codage sans perte lit 3,8431 bits par poids (A_ctx), et la base de B 2,6441 bits pour 4,2920 % d'erreur quadratique relative ; le contexte d'échelle κ (sous, égale, au-dessus de la médiane de la ligne) est déjà dans le format. La lecture prédisait que le codage prédictif sans perte ne gagnerait que ce que le voisin apporte (0,005 bit), et que l'élagage dépendrait de la sortie, pas des poids (tableau 2.5). **Ce chapitre établit** le gain exact des codons synonymes du E2M1, démontré et mesuré au codec réel sur les deux modèles entiers ; que le codage prédictif sans perte, au sens strict, ne retire presque rien, sauf un prédicteur : la classe d'énergie de la *colonne d'entrée* ; un système sans perte en valeur, zéro fondu et colonne, à **3,6844 bits** contre 3,8434, décodé exactement ; le cadre débit-distorsion où l'élagage synaptique n'est que le régime de débit nul d'un remplissage inverse ; que la métrique de la frontière est fausse pour juger une perte ; et que la sortie des experts puis la KL contre la référence font tomber tout ce qui remplit à travers les échelles absolues, même à erreur d'expert égale, pour ne laisser qu'un point avec perte : des partitions relatives à la ligne, tenseur par tenseur, à **0,911 fois les bits** de la base et **KL statistiquement égale**. **Il reste** les noyaux de carte des tables par contexte, la rétroaction d'erreur sur l'alphabet des cellules, un critère de sensibilité jugé par jeton ou contre la KL, et la mesure sur les 24 couches puis sur le 120b. Sources : `genome/predictif-elagage-pour-le-memoire.md` (le livrable du sujet) ; `genome/REPRISE-predictif-elagage.md` ; `mesures/genome/predictif-elagage/` (`resume-12.csv`, `taux-reel-*.json`, `contextes-*.json`, `debit-distorsion-12.*`, `plans/`, `sortie-experts-*`, `kl/`) ; `genome/05-synthese.md` § 1.1 ; outils sous `outils/genome/predictif-elagage/`.

En bref

Deux gestes du système nerveux ont servi de guides, comme analogies. La rétine ne transmet pas l'image, elle transmet ce qui la distingue de ce que ses voisins prédisaient : c'est le codage prédictif [32], et un neurone de l'œil de la mouche ajuste même sa réponse à la loi de ses entrées pour en tirer le plus d'information [87]. Le cerveau en développement, lui, fabrique trop de synapses puis retire celles que l'activité ne sollicite pas : c'est l'élagage synaptique [38]. Transposés aux poids, le premier geste demande ce que les voisins d'un code disent de lui, et la réponse est : presque rien, sauf la colonne par laquelle il lit son entrée. Le second geste se formule exactement en théorie du débit : fixer un prix du bit, et laisser disparaître les synapses qui ne le paient pas. Il gagne beaucoup sur les métriques de poids, et presque tout ce gain s'évanouit quand on juge sur le texte. Ce qui tient est modeste et solide : un signe de zéro qu'on cesse de payer, une carte de colonnes, et une base qui économise 9 % de bits sans dégrader la divergence au-delà du bruit de la mesure.

Tenseurs	sans perte au bit	zéro fondu	gain	journal
12 de la frontière (20b)	3,8434	**3,7064**	0,1370	`taux-reel-12-gptoss20b.json`
72, toutes couches (20b)	3,8509	**3,7230**	0,1280	`taux-reel-toutes-gptoss20b.json`
108, toutes couches (120b, 60,9 Go)	3,8549	**3,7292**	0,1257	`taux-reel-toutes-gptoss120b.json`

Table 6.1 : Le codon synonyme du zéro au codec réel (rANS de l'étude 03, comptabilité de l'arbitre, A_flux), en bits par poids [mes.]. La ligne « sans perte au bit » retrouve exactement A_flux de l'arbitre (3,8434) : second chemin du même codec. Journaux sous `mesures/genome/predictif-elagage/`.

6.1 Les codons synonymes du E2M1

Le fait [mes.]. Sur les 12 tenseurs, $P(+0) = 0{,}068\,46$ et $P(-0) = 0{,}068\,48$; plus largement, $P(+c) = P(-c)$ à 10^{-4} près pour chaque magnitude. Par tenseur, $P(+0)$ et $P(-0)$ diffèrent de moins de 10^{-4}, de 0,059 à 0,098 chacun (`histo-12-gptoss20b.npz`). Le quantificateur qui a produit le MXFP4 de gpt-oss garde le signe de la valeur d'origine même quand sa magnitude s'arrondit à zéro. Or E2M1 a deux codes pour zéro, `0b0000` et `0b1000` ; ggml les lit tous deux comme l'entier 0, la référence f64 comme $+0$ et -0, qui ne changent aucune somme non nulle (proposition 3.1).

Le parallèle biologique, comme image. Les codons synonymes du code génétique (section 1.2) diffèrent souvent par leur troisième base, la position « bancale » de Crick [88], et une mutation qui passe de l'un à l'autre est silencieuse. Les deux codes ± 0 sont deux codons synonymes ; l'information qui les distingue n'a aucun phénotype. Canoniser tout -0 en $+0$ avant le codage entropique revient à réduire l'usage des codons à un seul codon par sens.

Proposition 6.1 (Le gain exact du fondu)**.** *Soit C le code de 4 bits d'un poids et $V = v(C)$ sa valeur, $v(c) = (-1)^{c_3} K[c \,\&\, 7]$, injective sauf sur $\{0, 8\}$. Soit S le bit de signe. Alors*

$$H(C) = H(V) + P(V = 0)\, H(S \mid V = 0). \tag{6.1}$$

Le même énoncé vaut conditionnellement à tout contexte que le décodeur connaît (classe d'échelle κ, octet voisin).

Démonstration. V est une fonction de C, donc $H(V \mid C) = 0$ et $H(C, V) = H(C)$. Par la règle de chaîne, $H(C, V) = H(V) + H(C \mid V)$. Sachant $V = v \neq 0$, C est déterminé (v injective hors de 0) : $H(C \mid V = v) = 0$. Sachant $V = 0$, $C \in \{0, 8\}$ est déterminé par S : $H(C \mid V = 0) = H(S \mid V = 0)$. D'où $H(C \mid V) = \sum_v P(V = v)\, H(C \mid V = v) = P(V = 0)\, H(S \mid V = 0)$. Remplacer partout $H(\cdot)$ par $H(\cdot \mid \kappa)$ ne change aucun pas. ∎

Fondre 8 dans 0 remplace C par C' de même valeur, et $H(C') = H(V)$: le gain est $P(V = 0)\, H(S \mid V = 0)$, soit à peu près $P(V = 0)$ bit quand le signe est une pièce juste. **Calcul posé** [calc.] : $P(V = 0) = 0{,}068\,46 + 0{,}068\,48 = 0{,}136\,94$; $H(S \mid V = 0) = h(0{,}068\,46/0{,}136\,94) = h(0{,}499\,93) = 1{,}0000$ bit, h l'entropie binaire ; gain $= 0{,}136\,94$ bit par poids. Mesuré par l'entropie conditionnelle (contexte κ, tables par tenseur) : 0,136 939 [mes.] ; par le codec réel : 0,1370 [mes.]. La forme du gain par octet, qui sert au format, est reprise au chapitre 9.

Prédicteur, en plus de κ	H (bits par poids)	gain	verdict
aucun (κ seul)	3,7923	—	référence
δ exact (± 2)	3,7915	0,0007	écarté
octet entier (paire de demi-octets du bloc)	3,7885	0,0037	déjà dans A_ctx
code de gauche	3,7846	0,0076	écarté
code du dessus (ligne précédente)	3,7831	0,0091	écarté
expert parent (même place, expert $e-1$)	3,7879	0,0043	écarté
gauche × dessus (867 contextes)	3,7922	0,0000	écarté (tables)
contrainte du maximum de bloc	3,7921	0,0001	écarté
classe de la colonne d'entrée (quartile)	**3,7608**	**0,0314**	**retenu**

Table 6.2 : Le codage prédictif sans perte : ce que chaque contexte retire de l'entropie d'un code, 12 tenseurs, tables comprises [mes.]. Seuil déclaré : 0,015 bit. Données : `mesures/genome/predictif-elagage/contextes-gptoss20b.json`.

Les témoins. Chaque flux est décodé et comparé octet à octet, aucune voie hors de son état final ; chaque demi-octet décodé rend la valeur d'origine ($d = o$, ou $o = 8$ et $d = 0$). Cassure (8 envoyé sur 1) : le témoin de valeur tombe (`taux-reel-12-gptoss20b-casse.json`). De bout en bout, la référence f64 à poids reconstruits (crochet `alterer_expert`) rend, pour le plan identité, des entrées `.kld` identiques octet pour octet à la référence (510 entrées) et `juge kl` = 0 exactement (`kl/identite-2f-octets.json`) ; le plan zéro fondu donne des valeurs identiques, donc la même sortie. Ces mesures rejoignent celles de l'éclaireur par un autre chemin (tableau 3.4) et les étendent au 120b entier.

6.2 Le codage prédictif sans perte : ce que les voisins disent d'un code

Le cadre. Pour un alphabet discret, « prédire puis coder l'écart » s'écrit exactement : coder le code avec une loi conditionnée par un contexte X que le décodeur connaît. Le gain est l'information mutuelle

$$I(C; X \mid \kappa) = H(C \mid \kappa) - H(C \mid \kappa, X). \tag{6.2}$$

L'estimateur par fréquences est biaisé vers le bas ; par le développement de Miller [83], avec des tables par tenseur, le biais sur le gain vaut $(K_X - K_\kappa)(16-1)/(2N_t \ln 2)$ bit par symbole, K le nombre de contextes et $N_t = 265\,420\,800$ poids par tenseur. **Seuil déclaré avant** : 0,015 bit par poids, le coût d'en-tête d'un flux de plus (0,40 % de 3,84 bits, étude 03).

Lecture (tableau 6.2).

- **Les voisins ne prédisent presque rien.** Sachant l'échelle de son bloc, un code MXFP4 est un tirage presque indépendant ; ce que la littérature appelle la « dépendance spatiale » des poids est ici sous 0,01 bit.
- **L'expert parent ne prédit rien** (0,004) : les experts d'une couche ne sont pas des copies mutées les uns des autres au niveau des codes, ce que le chapitre 5 chiffre sur les valeurs, avec permutation.
- **Chaque bloc contient au moins un grand code** ($|\text{E2M1}| \geq 2$) : 0 bloc sur 99 532 800 en est privé ; la règle OCP de l'échelle ($2^{\lfloor \log_2 \max \rfloor - 2}$) est bien suivie, mais la contrainte ne vaut que 10^{-4} bit : un bloc a presque toujours plusieurs grands codes.
- **La colonne d'entrée porte une structure.** Certaines colonnes ont, en moyenne sur toutes les lignes et tous les experts d'un tenseur, des poids plus grands relativement à l'échelle de leur bloc.

Le rapport entre la colonne la plus énergique et la médiane va de 1,11 (blk.0 down) à 2,93 (blk.23 gate), en racine d'énergie. Les entrées de gate et up sont l'état caché, dont certains canaux portent des activations aberrantes ; l'entraînement y a répondu par des poids de taille différente. C'est un fait d'entrée, partagé par les 32 experts : une carte de 2 bits par colonne et par tenseur (720 octets) suffit. C'est la même statistique que le parent de hasard du chapitre 5 reprenait à 88 à 94 % dans la couche 0.

Les témoins, et une correction barrée. Contexte égal au code lui-même : $H = 0$ (tient). Contexte « gauche » permuté au hasard : gain de $6{,}33 \cdot 10^{-5}$ pour un biais attendu de $6{,}26 \cdot 10^{-5}$ (tient, `contextes-temoin-corrige.json`). Le livrable porte une première formule, barrée : « ~~Biais attendu $(51-1)\cdot 15/(2N\ln 2) = 5{,}4\cdot 10^{-6}$ avec N poolé, témoin tombé.~~ » Corrigé le 2026-10-16 : les tables sont par tenseur, le biais se calcule avec N_t et $51-3$ paramètres ajoutés ; avec l'ancienne formule (la cassure du témoin corrigé), le témoin tombe bien (`contextes-temoin-corrige-casse.json`).

Le codec réel, $\kappa\times$ colonne. Flux de demi-octets séparés par contexte, alphabet de 16, zéro fondu, carte et tables comptées (`taux-reel-colonne-12-gptoss20b.json`) [mes.] : **3,6844 bits par poids**, contre 3,7103 pour le même découpage en κ seul (gain réel de la colonne : 0,0259) et 3,7064 pour l'alphabet octet en κ. Par tenseur, la colonne ne rend rien sur blk.23.ffn_down (3,6744 → 3,6741 : l'entrée de down est la sortie d'une SwiGLU, sans canal aberrant) et l'essentiel sur gate et up. Le système sans perte en valeur, zéro fondu et colonne, lit donc **3,6844 bits contre 3,8434**, soit −4,1 % [calc.], et le chapitre 8 montre où ce gain vit (presque tout à la couche 0).

6.3 L'élagage et la quantification prédictive bornée : le cadre débit-distorsion

Le problème. Remplacer chaque code c, dans un bloc d'échelle e, par une cellule s d'une partition P_k des 16 codes ordonnés sur la droite réelle $(-12, \dots, -1, -0, +0, +1, \dots, +12$ en grandeurs E2M1 doublées K), choisie selon un contexte k que le décodeur connaît, reconstruite par $\hat{w} = r_{k,s}\, 2^{e-128}$. Débit : $H(s \mid k)$, une table par tenseur et par contexte. Distorsion : la somme des $(w-\hat{w})^2$. C'est un quantificateur à contrainte d'entropie sur une source discrète [89, 90].

Lemme 6.2 (Le centre optimal d'une cellule)**.** *Pour une cellule fixée, la reconstruction qui minimise $\sum_i 4^{e_i-128}(K_i - r)^2$ est le centre de gravité pondéré $r^* = B/A$, et l'erreur restante vaut $C - B^2/A$, avec $A = \sum_i 4^{e_i}$, $B = \sum_i 4^{e_i}K_i$, $C = \sum_i 4^{e_i}K_i^2$ (à un facteur 4^{-128} près).*

Démonstration. La fonction $r \mapsto C - 2rB + r^2A$ est un trinôme en r, de dérivée $2(Ar - B)$, nulle en B/A ; $A > 0$ en fait un minimum, où la valeur est $C - 2B^2/A + B^2/A = C - B^2/A$. (Le cas vectoriel est celui de Chou, Lookabaugh et Gray [89].) ∎

Lemme 6.3 (La programmation dynamique exacte)**.** *Avec un multiplicateur μ, le coût d'une partition en intervalles des 16 positions ordonnées est une somme de termes qui ne dépendent chacun que de leur cellule, $c(i,j) = D_{[i,j]}/E + \mu\, n_{[i,j]}\big(-\log_2(n_{[i,j]}/n_k)\big)$, E l'énergie du contexte, n les effectifs. La meilleure partition s'obtient exactement par la récurrence*

$$J(0) = 0, \qquad J(j) = \min_{0 \le i < j} J(i) + c(i, j-1), \tag{6.3}$$

en $16 \times 17/2 = 136$ évaluations de cellule, au lieu des 2^{15} partitions.

Famille, à erreur moyenne par tenseur égale à la base (4,337 %)	bits par poids	rapport à la base	part élaguée
base (frontière, même instrument)	2,632	1	—
élagage par magnitude (le reste exact, zéro fondu)	2,665	1,012	—
partitions libres, contexte κ	2,359	0,896	—
partitions libres, δ de ligne borné à ± 3	2,353	0,894	—
zone morte seule, contexte e (élagage lagrangien)	2,318	0,881	—
partitions libres, contexte e (remplissage inverse)	**2,019**	**0,767**	31 %

Table 6.3 : Les familles du cadre débit-distorsion, en erreur de poids [mes.]. Entropies idéales, plus les échelles (0,0375) et les tables ; la base au même instrument vaut 2,6321, contre 2,6441 au codec réel (écart de 0,46 %). Données : `mesures/genome/predictif-elagage/debit-distorsion-12.json` et `.csv`.

Démonstration. Une partition en intervalles de $\{0, \dots, j-1\}$ se termine par un dernier intervalle $[i, j-1]$, précédé d'une partition de $\{0, \dots, i-1\}$. Le coût étant additif par cellule, le coût minimal se décompose comme (6.3) (principe de Bellman). Il y a une cellule possible par paire $i \leq j$ des 16 positions, soit 136, chacune évaluée une fois par le lemme 6.2 sur des sommes cumulées. C'est la segmentation d'histogramme de Muresan et Effros [91], et de Wu et Zhang [92]. Les cellules sont des intervalles parce que, pour la distorsion quadratique, une cellule optimale est convexe sur la droite. ■

Proposition 6.4 (Le remplissage inverse, et l'élagage comme son régime de débit nul)**.** *Un μ commun à tous les contextes réalise le minimum de la distorsion totale sous contrainte de débit total : aux points où chaque contexte est sur sa courbe $D_k(R_k)$ convexe, la condition de Karush-Kuhn-Tucker est $-\partial D_k/\partial R_k = \mu$ pour tout contexte actif, et un contexte dont la pente à débit nul est plus faible que μ reste à débit nul. Les contextes de faible énergie (échelle e petite, colonne peu active) atteignent leur débit nul les premiers : une seule cellule, tout le contexte reconstruit par son centre, presque zéro.*

Démonstration. On minimise $\sum_k D_k(R_k)$ sous $\sum_k R_k \leq R$, $R_k \geq 0$, avec des D_k convexes décroissantes. Le lagrangien $\sum_k D_k(R_k) + \mu(\sum_k R_k - R) - \sum_k \nu_k R_k$ donne, à l'optimum, $D_k'(R_k) + \mu - \nu_k = 0$, $\nu_k \geq 0$, $\nu_k R_k = 0$. Pour un contexte actif ($R_k > 0$), $\nu_k = 0$ et $-D_k'(R_k) = \mu$; pour un contexte inactif, $-D_k'(0) = \mu - \nu_k \leq \mu$. La convexité rend ces conditions suffisantes. C'est le remplissage inverse de [2, th. 10.3.3], transposé à une source discrète par contextes. Un contexte de faible énergie a une pente $-D_k'(0)$ proportionnelle à son énergie : il devient inactif le premier. ■

L'élagage synaptique apparaît ainsi comme le régime de débit nul du remplissage inverse : on ne choisit pas d'élaguer, on choisit un prix du bit, et les synapses qui ne le paient pas disparaissent. C'est le geste d'Optimal Brain Damage [39] et de Deep Compression [41], écrit sur nos cellules.

La métrique, et pourquoi celle de la frontière est fausse ici. La première passe utilisait l'erreur globale de la frontière, $\sum(w - \hat{w})^2 / \sum w^2$ sur les 12 tenseurs. Or la part de l'énergie par tenseur est extrêmement inégale [mes.] : blk.23.ffn_down 91,5 %, blk.16.ffn_down 8,0 %, blk.8.ffn_down 0,40 %, les neuf autres ensemble 0,015 %. Un μ commun sous cette métrique vide les neuf petits tenseurs : 0,32 bit « à 4,29 % » (`ecsq_e_metrique_globale`, pour mémoire), absurde. **Métrique retenue** : la moyenne sur les tenseurs de leur erreur relative propre, chaque matrice comptant autant ; la base y vaut 4,3367 % (de 4,05 à 4,80 % par tenseur).

Pourquoi l'élagage par magnitude perd (tableau 6.3). Élaguer, c'est fondre les petites magnitudes dans le symbole zéro ; le débit ne baisse que de l'entropie de ces symboles, tandis que tous les autres

Famille	erreur de sortie prédite	bits par poids	rapport à la base (2,632)
base	4,489 %	2,632	1
partitions par e (métrique des poids)	4,494 %	2,022	0,768
partitions par e, masses pondérées par l'activité	4,461 %	2,005	0,762
partitions par (e, q)	4,487 %	1,695	0,644

Table 6.4 : L'élagage dépendant de l'activité, à erreur de sortie *prédite* par (6.4) [mes.] [calc.]. À ce point, 40,9 % des poids sont fondus dans la cellule qui contient zéro. Données : `mesures/genome/predictif-elagage/plans/debit-sensibilite-12.json`.

codes gardent leurs 3 bits et plus. À erreur égale, il vaut mieux grossir *toutes* les cellules un peu que d'en vider quelques-unes. Les petites magnitudes portent peu d'énergie, mais aussi peu de bits : 50 % des poids les plus petits d'un tenseur ne portent que 4,3 % de son énergie, et leur retrait ne retire qu'environ la moitié des bits qu'il faudrait. Le remplissage inverse par e, lui, gagne 23 % de bits sur la métrique des poids ; la section 6.5 dit ce qu'il en reste.

6.4 L'élagage dépendant de l'activité

Le cadre. Pour une matrice W et des entrées x de covariance Σ, par (2.8),

$$\mathbb{E}\,\|\Delta W x\|^2 = \operatorname{tr}(\Delta W\,\Sigma\,\Delta W^\top) = \sum_{i,j,j'} \Delta W_{ij}\Delta W_{ij'}\Sigma_{jj'} \;\approx\; \sum_{i,j} \Delta W_{ij}^2\,\Sigma_{jj}, \tag{6.4}$$

approximation diagonale, exacte en moyenne si les erreurs de colonnes différentes sont décorrélées. La sensibilité d'un poids est $s_j = \mathbb{E}[x_j^2]$, l'activité de sa colonne. C'est le critère de Wanda ($|W_{ij}| \cdot \|X_j\|$) [93], voisin de celui de SparseGPT [94], et le terme diagonal d'OBD. L'analogie biologique est explicite : dans le cerveau en développement, l'élagage synaptique dépend de l'activité, une synapse peu sollicitée est retirée la première.

Le système. Les masses du lemme 6.2 deviennent $A = \sum 4^e s_j$; le contexte devient (e, q), q le quartile d'activité de la colonne dans l'expert (carte de 2 bits par colonne et par matrice d'expert, 0,0007 bit par poids, comptée). Les s_j viennent des fenêtres 0 à 4 du corpus du lot R, capturées par la référence f64 (entrées de gate et up : l'état normé routé vers l'expert ; entrée de down : la SwiGLU calculée avec les poids exacts).

Une prédiction, barrée. Le livrable annonçait parmi ses résultats : « ~~L'élagage dépendant de l'activité : 1,695 bit (×0,644) à erreur égale.~~ » Il l'a barré le 2026-10-17 (`genome/predictif-elagage-pour-le-memoire.md` § 0, point 7) : c'est une erreur *prédite* par l'approximation diagonale (tableau 6.4) ; mesuré, ce plan casse l'expert et la KL (0,271, section 6.6).

6.5 Les entrées réelles, et l'erreur à la sortie des experts

Les entrées [mes.]. La référence f64 (`trc-reference`, chemin par lots) a tourné sur les 10 fenêtres de 512 jetons du corpus du lot R avec un crochet de capture (`sonde capture`, `capture-entrees-20b.json`, 2 828 s) : pour les couches 0, 8, 16 et 23, l'état normé après l'attention de chaque position routée vers chaque expert, 20 440 vecteurs de 2 880 par couche (235,5 Mo en f32, hors dépôt). Calibration des plans qui lisent les activations : fenêtres 0 à 4 ; évaluation : fenêtres 5 à 9.

Plan	bits réels	gate	up	down seul	**expert entier**	rapport
base (frontière)	2,6444	1,16 %	2,52 %	2,72 %	**5,57 %**	1
partitions par κ	2,3743	1,12 %	2,70 %	2,37 %	**5,48 %**	0,98
zone morte par e	2,3283	1,44 %	2,15 %	2,51 %	34,8 %	6,2
partitions par e (poids)	2,0178	1,03 %	2,65 %	2,38 %	22,0 %	4,0
partitions par δ de ligne (±3)	2,353 (idéal)	1,14 %	2,67 %	2,42 %	5,48 %	0,98
partitions par e, masses hebbiennes	**2,1815**	0,92 %	2,38 %	2,45 %	**5,70 %**	1,02
partitions par (e, ligne, colonne), hebbiennes	1,7556	2,36 %	4,30 %	6,41 %	9,23 %	1,66
partitions par (e, activité de colonne)	1,6942	—	—	—	(KL 0,271)	—

Table 6.5 : L'erreur relative à la sortie des experts, entrées réelles (fenêtres 5 à 9), moyenne des quatre couches [mes.]. Bits réels : `taux_plan`, décodage exact (la base y donne 2,6444 contre 2,6441 publié par l'arbitre : second chemin). Données : `mesures/genome/predictif-elagage/sortie-experts-12.json`, `sortie-experts-12-hebb.json`, synthèse `resume-12.csv`.

La mesure (`sortie_experts.py`) : pour chaque expert, sortie exacte $y = W_d\,\mathrm{swiglu}(W_g x + b_g, W_u x + b_u) + b_d$ et sortie reconstruite $\hat{y}$ (les trois matrices du plan), erreurs relatives sommées sur les experts (pondération par les jetons) puis moyennées sur les quatre couches. Témoins : plans identité et zéro fondu, erreur nulle exactement (tient).

Par couche, l'erreur d'expert (0 / 8 / 16 / 23) vaut [mes.] : base 5,1 / 7,6 / 6,7 / 2,9 % ; κ 6,1 / 7,6 / 6,5 / 1,8 % ; hebbien par e 5,1 / 8,3 / 7,1 / 2,3 % ; hebbien (e, ligne, colonne) 6,4 / 17,8 / 9,5 / 3,2 %.

Ce que la mesure a révélé (tableau 6.5). Les plans qui remplissent à travers les échelles absolues gardent chaque matrice sous 3 % d'erreur relative, et l'expert entier devient faux à 22 à 35 %. La cause : la quantité que mesure l'erreur relative d'une matrice n'est pas celle qui compte. Une ligne i de gate ou de up agit sur la sortie à travers la colonne i de down, et l'entraînement répartit librement l'échelle entre les deux : le chemin up a une symétrie de réétalonnage, $u_i \to c\,u_i$ et $W_d[:,i] \to W_d[:,i]/c$ ne changent presque rien (presque, parce que la borne de u dans la SwiGLU n'est pas homogène, proposition 5.1). Une ligne de petite échelle dans gate peut donc peser lourd en sortie, et le remplissage inverse par matrice vide précisément ces lignes.

La sensibilité hebbienne. Elle répond au premier ordre (`histo_hebb.py`). En linéarisant la sortie autour des poids exacts,

$$\mathbb{E}\,\|\Delta y\|^2 \approx \sum_{i,j} \Delta W_{ij}^2\, r_i\, s_j, \qquad s_j = \mathbb{E}[x_j^2], \quad r_i = \|W_d[:,i]\|^2\, \mathbb{E}\Big[\Big(\frac{\partial a_i}{\partial g_i}\Big)^2\Big] \tag{6.5}$$

(ou $\partial a_i/\partial u_i$ pour la matrice up), a l'activation intermédiaire. Le premier facteur, s_j, est l'activité de l'entrée (présynaptique) ; le second, r_i, l'influence de la sortie (postsynaptique) ; le poids d'une synapse est leur produit, le produit de Hebb. La factorisation $\mathbb{E}[(\partial a/\partial g)^2 x^2] \approx \mathbb{E}[(\partial a/\partial g)^2]\,\mathbb{E}[x^2]$ néglige leur corrélation, et l'approximation diagonale (6.4) celle des colonnes : deux hypothèses que la mesure juge.

6.5.1 L'élagage hebbien : ce qui tient, ce qui sur-ajuste

– **Masses hebbiennes, contexte e seul** (aucune carte envoyée : r_i et s_j ne servent qu'au codeur, pour choisir les partitions et les centres) : **2,1815 bits réels, erreur d'expert 5,70 %** contre 5,57 % pour la base à 2,6444, soit 0,825 fois les bits pour la même erreur à 2 % près [mes.]. Le même remplissage

Plan	bits réels	KL moyenne	±	médiane	99e centile	même premier choix
base (frontière)	2,6444	0,0636	0,0050	0,0227	0,777	81,6 %
hebbien par e	2,1815	0,195	0,009	0,075	1,54	60,8 %
hebbien (e, ligne, colonne)	1,7556	0,192	0,008	—	—	59,4 %
hebbien (δ, ligne, colonne)	1,9619	0,091	0,005	—	—	73,4 %
partitions par e (poids)	2,0178	0,309	0,011	0,209	1,83	58,5 %
partitions par (e, activité de colonne)	1,6942	0,271	0,009	0,188	1,51	57,1 %
partitions par κ (erreur moyenne égale)	2,3743	0,0804	0,0063	0,0263	1,10	78,4 %
partitions par κ, chaque tenseur à l'erreur de la base	**2,4079**	**0,0702**	0,0059	0,0235	0,911	79,5 %

Table 6.6 : La KL contre la référence f64 (protocole P3 : 128 experts sur 768, 1 275 positions) [mes.]. Données : `mesures/genome/predictif-elagage/kl/kl-12-*-f5-9.json`, synthèse `resume-12.csv`.

inverse sans la sensibilité hebbienne cassait l'expert (22 %) : c'est bien la symétrie d'échelle gate/up ↔ down qui le cassait, et l'influence postsynaptique qui la répare.

- **Avec cartes de lignes et de colonnes** (16 classes, 9 273 cellules) : prédit 4,33 %, mesuré **9,23 %**, la couche 8 à 17,8 %. La prédiction ne tient pas : les quartiles de r_i et s_j sont estimés sur environ 320 entrées par expert (fenêtres 0 à 4), et la factorisation de (6.5) néglige leur corrélation ; un plan qui donne zéro bit aux classes jugées inactives paie au prix fort toute erreur d'estimation. Sur-ajustement mesuré, pas supposé.
- **Variante relative à la ligne** ($\delta\times$ ligne × colonne) : 1,9619 bits réels, KL 0,091, différence appariée +0,027 ± 0,004 contre la base (`kl/comparer-base-hebb_d_lc-f5-9.json`) : perd.

6.6 La KL contre la référence f64

Le protocole (P3 de la ronde). Fenêtres 5 à 9 (1 275 positions notées), seules les 12 matrices de la frontière altérées, c'est-à-dire **128 experts sur 768** (4 couches entières sur 24 ; `genome/05-synthese.md` § 1.1), sonde `alterer` puis `juge kl` du lot R contre la tranche de la référence (`kl_tranche.py`, empreinte dans `kl/tranche-ref-f5-9.json`). Témoin de la chaîne : plan identité, entrées `.kld` identiques octet pour octet à la référence, KL = 0 exactement. La KL absolue de ce protocole ne se compare pas à celle de l'éclaireur (P1, 32 experts) : quatre fois plus d'experts y sont remplacés. Ce qui se compare est le rapport à la base du même protocole et, mieux, la différence appariée position par position, avec la règle du lot R (égal si $\bar{d} \le 2\sigma(\bar{d})$ et $\bar{e} \ge -2\sigma(\bar{e})$ pour le premier choix).

Ce que la KL tranche (tableaux 6.6 et 6.7).

- **Tout plan qui remplit à travers les échelles absolues perd**, de 3 à 5 fois la KL de la base : partitions par e (0,309), élagage par activité de colonne (0,271), hebbien par e (0,195). Les plans qui cassent l'expert cassent le modèle.
- **Même à erreur d'expert égale.** L'hebbien par e fait 5,70 % d'erreur d'expert contre 5,57 % pour la base, et trois fois sa KL (appariée +0,131 ± 0,007, premier choix −20,8 points). L'erreur relative

Plan, contre la base	KL, différence appariée	premier choix (points)	verdict du juge
partitions par κ, erreur moyenne égale	$+0{,}0167 \pm 0{,}0049$ ($3{,}4\,\sigma$)	$-3{,}2 \pm 1{,}0$	pire
partitions par κ, tenseur par tenseur	$\mathbf{+0{,}0066 \pm 0{,}0053}$ ($1{,}2\,\sigma$)	$-2{,}1 \pm 1{,}1$	**égal**
hebbien par e	$+0{,}131 \pm 0{,}007$	$-20{,}8 \pm 1{,}4$	pire
hebbien (e, ligne, colonne)	$+0{,}128 \pm 0{,}007$	$-22{,}2 \pm 1{,}4$	pire
hebbien (δ, ligne, colonne)	$+0{,}027 \pm 0{,}004$	$-8{,}2 \pm 1{,}2$	pire

Table 6.7 : Les comparaisons appariées des plans contre la base, mêmes 1 275 positions, même vérité (`juge comparer`) [mes.]. Données : `mesures/genome/predictif-elagage/kl/comparer-base-*-f5-9.json`.

moyenne à la sortie d'un expert, pondérée par $\|y\|^2$, ne suffit pas à juger : elle est tenue par les jetons et les dimensions de grande sortie ; le remplissage inverse à travers les échelles concentre l'erreur sur ce qui est petit (blocs et lignes de petite échelle, 23 % des poids fondus dans zéro), que la norme RMS des couches suivantes ramène à l'échelle et que la KL voit. La leçon du sujet : à la sortie, il faudrait juger l'erreur relative *par jeton*, ou la KL directement, pas l'énergie totale.

- **À erreur moyenne égale, le plan κ ne vaut pas la base en KL** ($+0{,}0167 \pm 0{,}0049$, $3{,}4\,\sigma$) : il déplace l'erreur vers la couche 0 (6,1 % d'erreur d'expert contre 5,1 %), et une erreur tôt dans le réseau pèse plus dans la KL. D'où le plan « chaque tenseur à l'erreur de la base » (`plans.py --par-tenseur`, un μ par tenseur : domination tenseur par tenseur), à **2,4079 bits réels, 0,911 fois la base**. Apparié contre la base : $+0{,}0066 \pm 0{,}0053$ ($1{,}2\,\sigma$), premier choix $-2{,}1 \pm 1{,}1$ point. **Statistiquement égal à la base** par la règle du lot R ; littéralement un peu au-dessus.

C'est le point avec perte que le sujet retient : 9 % de bits en moins que la base de la frontière, à KL égale dans le bruit de la mesure. La lecture prudente est celle du livrable lui-même : la différence appariée centrale reste positive, et avec plus de positions elle pourrait devenir significative. Le chapitre 8 place ce point sur la frontière commune de la ronde, sous le nom « partitions par κ, égale par la règle ».

6.7 Ce qui reste

- **Les noyaux de carte** : un décodeur à table par (κ, colonne) ou par (e, q), et la reconstruction par table de centres (4 à 16 registres) ; leur branchement dans le codeur du format. La part sans perte (zéro fondu et colonne) est entrée dans le format v1.2 (chapitre 9).
- **La rétroaction d'erreur** (GPTQ, le « codage prédictif en boucle fermée ») sur l'alphabet des cellules, avec la hessienne complète des entrées réelles ; et un critère de sensibilité jugé par jeton, ou appris contre la KL, puisque l'énergie moyenne a trompé.
- **Le plan κ dominé** + colonne + zéro fondu sur les 24 couches, et sa KL ; puis gpt-oss-120b.

Ce qui pourrait être faux

- **Les débits avec perte sont des entropies idéales**, pas un codec, sauf là où la colonne « bits réels » le dit ; l'écart du codec réel mesuré sur la base est de 0,46 %, et il peut être plus grand pour des tables à beaucoup de contextes (2 077 cellules en e, 3 808 en (e, q)) et des cellules rares.
- **L'approximation diagonale** de la sensibilité (6.4) néglige les corrélations entre colonnes d'entrée, et la factorisation hebbienne (6.5) celles entre entrée et dérivée ; la mesure exacte à

la sortie les juge, et en a fait tomber une prédiction (barrée).

- **Les entrées** : 10 fenêtres de 512 jetons d'un seul corpus ; environ 640 entrées par expert et par couche, environ 320 pour calibrer : les quartiles d'activité d'un expert peu routé sont bruités.
- **La KL n'altère que 4 couches sur 24** ; l'effet sur les 24 n'est pas mesuré (la KL croîtrait, pour la base comme pour le plan). Elle porte sur 1 275 positions d'un seul corpus, et l'avantage de 9 % du plan κ dominé est à KL statistiquement *égale*, pas meilleure (différence centrale +0,0066).
- **Le gain des codons synonymes** suppose que tout moteur lise ±0 comme 0 : vrai de ggml (table entière) et de la référence ; pour les noyaux de troncons, c'est la proposition A1 du chapitre 9 et sa mesure.
- **Aucune figure** ne porte ce chapitre : les deux proposées (III-5.1, III-5.2) ne sont pas tracées, et les tableaux en tiennent lieu.

Chapitre 7

Le mimétisme : six pistes du monde vers le pipeline

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la ronde du génome avait tenu trois gains (le codon du zéro, le contexte de colonne, la mutation synonyme) et fermé la garde tropicale globale, la graine, le duplicata, l'hebbien et les spectres (`genome/05-synthese.md` § 2 et § 3). Le moteur n'avait qu'un cache LRU partagé (`trc-cache` n'exporte que `Lru` et `Belady`). Six éclaireurs, chacun chargé d'un domaine du monde (le vivant, la physique, les mathématiques, l'ingénierie, les systèmes humains, la Terre et le cosmos), avaient rendu 45 candidats, dont une bonne part déjà jugée par une mesure. **Ce chapitre établit** le tri de ces candidats en six pistes et ce que chacune a donné devant son juge. P1, la mémoire du cache à deux échelles : le « tombe » mécanique est décidé par 147 jetons produits après la fin du message ; sur la réponse réelle le LFU(32) gagne +0,0185, sur trois blocs : *ouvert*. P2, le cache exclusif : un théorème (l'exclusif à deux étages vaut un LRU de capacité $V + R$) et une réduction de la part disque de 28 et 51 % ; le vrai marché est « 3,2 Go de RAM ou une descente recouverte » : *ouvert*. P3, le routage conditionné par le cache : −10 % de défauts pour une KL de 0,0314 dans le moteur, mais aucune condition ne tient à sa borne défavorable : *à la limite*. P4, le préchargement calibré : la précision garantie tient (0,92 pour environ 1 % d'octets de plus), le gain non (3,3 à 3,5 %) : *ouvert, plutôt du côté de tombe*. P5, le verre de spin : l'optimiseur baisse l'erreur de calibration de 52 % et l'erreur tenue à l'écart de 5 % seulement : *ouvert*, et le mécanisme annoncé ne tient pas au seuil. P6, la fidélité décidée par appel : *ouvert* au décodage, « tient » au préremplissage, mais un « tient » qui ne pouvait presque pas tomber. Chaque piste apporte ses dérivations, posées en entier ici. **Il reste** des générations saines du 120b (P1, P4), les passes ABBA du moteur (P2, P4), le second tirage de P3 (en travail dans une autre session, aucun de ses chiffres ici), les poids réels de P5 (étape 2), la KL de bout en bout de P6 (étape 1). Sources : `genome/mimetisme/tri.md` ; pour chaque piste *k*, `genome/mimetisme/Pk/resultat.md`, `pour-le-memoire.md`, `JUGEMENT.md` (P3 : aussi `JUGEMENT-etage-B.md`, `JUGEMENT-N238.md`, `JUGEMENT-routeur-ligne.md`) ; seuils et journaux sous `mesures/mimetisme/`.

En bref

Le mimétisme demande à d'autres disciplines comment elles ont résolu un problème qui ressemble au nôtre : garder en mémoire ce qui va resservir, laisser partir le reste, décider vite sous incertitude. L'image est celle d'un emprunt : un climatologue sait qu'un réservoir lent intègre le forçage d'un réservoir rapide ; un physicien sait qu'un verre de spin a des vallées où une descente gloutonne s'arrête ; un statisticien sait garantir une précision sans connaître la loi des données. Aucune de ces ressemblances n'est une preuve : chacune propose une mesure, et c'est

> la mesure qui décide. Sur six pistes, aucune ne « tient » franchement devant son juge ; quatre restent ouvertes pour une raison précise et chiffrée (des traces trop courtes ou dégénérées, un recouvrement non mesuré, un simulateur non validé, des poids synthétiques), et deux disent quelque chose de net : un étage de RAM inclusif plus petit que la VRAM ne sert à rien, et un routeur qui favorise les experts présents retire 10 % des défauts pour une KL de l'ordre de 0,03, au bord de ce qui est admis.

7.1 Le tri : quarante-cinq candidats, six pistes

D'où viennent les candidats. Chaque éclaireur a lu son domaine et proposé des mécanismes transposables au chemin des poids, avec, quand c'était possible sans la carte, une première mesure sur les traces du moteur. Le tri (`genome/mimetisme/tri.md`, 2026-10-27) les a dédoublonnés, classés, et a écrit pour chacune des six pistes retenues un protocole complet : données, grandeur, seuil, témoins, second chemin, coût. Les seuils du tri sont commités avant le document (`mesures/mimetisme/tri/00-seuils-declares.md`, commit `ec124f5`) ; un tri ne mesure rien. Critère de classement : gain attendu × nouveauté × faisabilité sur des données existantes, sans carte graphique, avec priorité au goulot réel (octets lus par mot, bascules du routage, préchargement). Contraintes communes : pas de carte, deux fils, priorité normale, RAM visée sous 1,5 Go ; tant que le verrou de la carte existe, aucune lecture lourde de fichier modèle.

Les mêmes mécanismes sous des noms différents. Le dédoublonnage est le premier résultat du tri : six domaines arrivent au même levier sous six noms (tableau 7.1).

Le signal le plus fort est le LFU à mémoire longue : il sort de quatre éclaireurs en positif (vivant, Terre et cosmos, humain, ingénierie) et de deux en négatif (physique : la forme du noyau ne compte pas ; mathématiques : un prédicteur appris n'ajoute que +0,012). Mais toutes ses mesures étaient faites sur des traces de *lecture* : le LFU à décroissance n'avait jamais été mesuré sur une génération, qui est la charge réelle. Et le « LFU −0,075 en génération » d'ingénierie n'était pas le contre-exemple qu'il paraissait : son « lfu » n'a pas de décroissance (le compteur monte, seule la politique « adm » le divise par deux, lu dans `outils/mimetisme/ingenierie/cache2.py`). La question restait ouverte, d'où P1.

Deux familles. Trois des six pistes touchent le goulot sans toucher la qualité (P1, P2, P4 : des octets lus en moins pour la même sortie au bit) ; trois échangent de la qualité contre des octets (P3, P5, P6) et se jugent à la KL. Les secondes partagent une barre : la pente du mode rapide, 0,13 de KL pour −34 % d'octets, linéarisée en 0,038 pour 10 points ; P6 devait fournir la courbe $\mathrm{KL}(f)$ qui manquait à cette pente.

Ce qui est écarté, et pourquoi. Le tri écarte une trentaine de candidats, chacun par un nombre ou une raison écrite (`tri.md` § 4). Les plus parlants : les noyaux de mémoire physiques (Omori-Utsu, Kohlrausch), fermés par la mesure E1 (+0,0051, +0,0028, +0,0044 contre un seuil de +0,015 : la forme ne compte pas, la demi-vie compte, et elle passe dans P1) ; la coupure de Fermi (retirer l'expert de petit poids), fermée par E2 (0,09 % des défauts sous 0,05 ; la porte vers le rapide devient P6) ; la vérification groupée, tuée par son plafond ($G(4) = 1{,}44$, $G(8) = 1{,}84$ pour un brouillon gratuit) ; le prédicteur d'éviction appris (+0,0124 contre le meilleur LFU réglé, sous +0,05) ; les segments à listes fantômes (le SLRU à probatoire de 20 % s'effondre, 0,257 contre 0,738) ; le cache en forme codée, dominé par

Table 7.1 : Le dédoublonnage : les mêmes mécanismes sous des noms de domaines différents (`genome/mimetisme/tri.md` § 1). Le numéro suit le nom de l'éclaireur.

Mécanisme commun	Éclaireurs et noms d'origine	Signal
LFU à demi-vie longue	vivant #1 (ACT-R), Terre #1 (Hasselmann, deux réservoirs), humain #1 et #8 (Anderson et Schooler), ingénierie #2, physique #4 (Omori-Utsu, Kohlrausch), Terre #7 et #4	six domaines, un levier : une demi-vie plus longue que 8 jetons
Routage qui favorise les résidents	mathématiques #1 (cache-conditionnel), Terre #6 (faille collante), humain #5 (substitution d'article, deux choix)	trois domaines
Prédire les experts à venir	mathématiques #2 (prédiction conforme), humain #2 (jurisprudence), vivant #2 (circuit olfactif de la mouche), vivant #4, Terre #5, ingénierie #2	six domaines
Préchargement subordonné à la demande	ingénierie #4 (TCP Nice, LEDBAT), humain #3 (BOLA), mathématiques #2	trois
Choisir la fidélité à la lecture	physique #3 (occupation de Fermi), vivant #5 et #7a (fovéation), humain #6 (triage), ingénierie #6, humain #3, vivant #3	six
Cache exclusif	ingénierie #1 et #5	un, mais mesuré
Plafond de la vérification groupée	mathématiques #5, vivant #6	deux, fermé
Courbe sans falaise	mathématiques #8, ingénierie #7	deux, fermé
Allocation des emplacements entre couches	vivant #7b (Charnov), Terre #8, humain #7	trois, fermé

P2 (−8 % d'octets pour 120 ms de carte par jeton) ; et, de justesse, le remplissage inverse par l'usage (0,094 bit réalisé contre un seuil de 0,126), la meilleure piste de compression non retenue.

Les instruments partagés. Trois règles valent pour toutes les pistes. (i) L'*arrêt séquentiel de groupe* d'O'Brien et Fleming [95], calibré par Monte-Carlo par l'éclaireur humain (constante $c = 2{,}840$) : 6,8 fenêtres sur 10 en moyenne sur cinq séries décisives, avec un plancher de 6 fenêtres, parce qu'une série franchissait la frontière à 4 fenêtres ($t = -5{,}1$) et n'était plus décisive à 10 ($t = -2{,}1$). (ii) La *règle d'appariement* du lot R : un mode est meilleur si $\bar{d} + 2\sigma(\bar{d}) < 0$, σ par blocs de fenêtres. (iii) *Séparer préremplissage et génération dans tout jugement de cache* : les politiques d'admission gagnent en lecture (+0,081) et perdent en génération (−0,094) (ingénierie, H3) ; c'est le piège commun, et P1 et P4 y sont tombés autrement qu'on ne l'attendait (section 7.2.3).

La forme commune des six protocoles. Chaque piste a commité son seuil avant toute lecture de données, et chaque juge a vérifié l'ordre des commits par `git log` ; chaque piste a un témoin dont la réponse est connue d'avance et une cassure qui doit le faire tomber ; chaque chiffre principal a un second chemin ; et chaque piste a été refaite par un juge indépendant, écrit sans le code de la piste. Le verdict de ce chapitre est toujours celui du juge.

7.2 P1 : la mémoire du cache à deux échelles

L'idée. Un cache d'experts décide qui garder. Le LRU garde les plus récents ; le LFU, les plus fréquents. La mémoire humaine oublie en loi de puissance [96], et le climat a deux réservoirs, l'un rapide qui suit le forçage, l'autre lent qui l'intègre [97] ; la version « cache » de cette idée est le LRFU [98], un score qui décroît avec une demi-vie. Si un tel score gagne en génération, c'est le changement le moins cher du projet : un compteur par expert, 4 608 flottants pour le 120b. Les quatre notices sont relues par Crossref le 2026-10-29 ; les articles ne le sont pas, et rien du verdict n'en dépend.

7.2.1 Le score à décroissance exponentielle, estimateur de taux

Définition 7.1 (Score à demi-vie)**.** Soit un expert accédé aux jetons $s_1 < s_2 < \dots$ Pour une demi-vie h (en jetons), son score au jeton t est

$$c_h(t) = \sum_{s_i \le t} 2^{-(t-s_i)/h}, \qquad \text{mis à jour par } c \leftarrow c \cdot 2^{-\Delta t/h} + 1. \tag{7.1}$$

Le cache évince le résident de plus petit score. On pose $\tau = h/\ln 2$, la constante de temps.

Lemme 7.2 (Espérance et variance)**.** *Si les accès forment un processus de Poisson de taux λ par jeton,* $\mathbb{E}[c_h] = \lambda h/\ln 2$ *et* $\mathrm{Var}[c_h] = \lambda h/(2\ln 2)$. *Donc $\hat{r}_h = c_h \ln 2/h$ est un estimateur sans biais de λ, de variance $\lambda \ln 2/(2h) = \lambda/(2\tau)$.*

Démonstration. Le score est un processus de Poisson filtré par le noyau $k(u) = 2^{-u/h} = e^{-u/\tau}$. Par la formule de Campbell, $\mathbb{E}[c_h] = \lambda \int_0^\infty e^{-u/\tau}\,\mathrm{d}u = \lambda\tau$ et $\mathrm{Var}[c_h] = \lambda \int_0^\infty e^{-2u/\tau}\,\mathrm{d}u = \lambda\tau/2$. En multipliant par $\ln 2/h = 1/\tau$: $\mathbb{E}[\hat{r}_h] = \lambda$ et $\mathrm{Var}[\hat{r}_h] = \lambda/(2\tau)$. ∎

L'écart relatif, $\sqrt{\ln 2/(2h\lambda)}$, est le prix d'une mémoire courte : à $\lambda = 0{,}1$ accès par jeton et $h = 8$, il vaut $\sqrt{0{,}6931/1{,}6} = 0{,}66$ [calc.].

Lemme 7.3 (Biais de retard)**.** *Si le taux dérive linéairement, $\lambda(t) = \lambda_0 + \lambda' t$, alors $\mathbb{E}[\hat{r}_h(t)] = \lambda(t) - \lambda'\tau$, et l'erreur quadratique moyenne vaut* $\mathrm{EQM}(\tau) = \lambda/(2\tau) + \lambda'^2\tau^2$.

Démonstration. $\mathbb{E}[\hat{r}_h(t)] = \int_0^\infty \lambda(t-u)\,e^{-u/\tau}\,\mathrm{d}u/\tau = \lambda(t) - \lambda' \int_0^\infty u\,e^{-u/\tau}\,\mathrm{d}u/\tau = \lambda(t) - \lambda'\tau$. L'EQM est la variance du lemme 7.2 plus le carré du biais. ∎

Proposition 7.4 (Horizon optimal)**.** *Pour $0 < \lambda' < \infty$, l'EQM est minimale en $\tau^* = \big(\lambda/(4\lambda'^2)\big)^{1/3}$, qui n'est ni 0 (le LRU) ni ∞ (le LFU).*

Démonstration. $\mathrm{d}\,\mathrm{EQM}/\mathrm{d}\tau = -\lambda/(2\tau^2) + 2\lambda'^2\tau$ s'annule en $\tau^3 = \lambda/(4\lambda'^2)$; la dérivée seconde $\lambda/\tau^3 + 2\lambda'^2$ est positive. ∎

L'horizon croît quand la dérive faiblit et quand le taux est faible. La mesure place le meilleur h, choisi sur les lectures des autres genres, à $h^* = 32$ ($\tau = 46$ jetons) à 326 emplacements, et à 32 ou 64 à 922 [mes.] (`mesures/mimetisme/P1/juge-120b-*.json`, champ `cv`).

Proposition 7.5 (Le continuum LRU–LFU)**.** *Quand $h \to 0$, l'ordre des scores tend vers l'ordre des derniers accès (LRU) ; quand $h \to \infty$, vers l'ordre des comptes (LFU).*

Démonstration. $c_h(t) = 2^{-(t-s_{\text{dernier}})/h}\big(1 + \sum_{i \neq \text{dernier}} 2^{-(s_{\text{dernier}}-s_i)/h}\big)$. Quand $h \to 0$, le second facteur tend vers 1 et le premier ordonne par s_{dernier} ; quand $h \to \infty$, chaque terme de la somme (7.1) tend vers 1 et le score vers le compte. ∎

C'est la famille LRFU [98]. Les témoins le vérifient au bit : à $h = 0{,}001$ le succès égale celui du LRU à 0 près sur huit traces ; à $h = \infty$ il égale le « lfu » de `cache2` à 0 près [mes.] (`mesures/mimetisme/P1/temoins.json`).

Deux réservoirs. Le mélange $M(w) = (1-w)\,\hat{r}_8 + w\,\hat{r}_{128}$ est la sortie d'un filtre linéaire de réponse impulsionnelle $k(u) = (1-w)\frac{\ln 2}{8}2^{-u/8} + w\frac{\ln 2}{128}2^{-u/128}$, somme de deux relaxations, de fonction de transfert

$$K(s) = \frac{1-w}{1 + 8s/\ln 2} + \frac{w}{1 + 128s/\ln 2}, \tag{7.2}$$

deux pôles réels : c'est la structure à deux réservoirs de Hasselmann, dite ici comme analogie de forme. $M(0) = \mathrm{LFU}(8)$ et $M(1) = \mathrm{LFU}(128)$ à 0 près (témoin W3). Le mélange n'est pas meilleur que le LFU(32) seul : −0,0031, +0,1042, −0,0657, +0,0892 contre −0,0176, +0,1136, −0,0449, +0,0981 sur les quatre cas du verdict [mes.].

Du succès à la vitesse. Si le disque borne tout, le temps d'un jeton est proportionnel aux défauts, $v \propto 1/(1-s)$, et un cache de succès s_1 contre s_0 accélère d'un facteur

$$\rho = \frac{1-s_0}{1-s_1}. \tag{7.3}$$

Le seuil déclaré (+0,02 de succès sur 0,2621 de défauts) vaut $\rho = 0{,}2621/0{,}2421 = 1{,}083$; la génération wiki (0,3944 → 0,5080) vaudrait $0{,}6056/0{,}4920 = 1{,}23$, mais sur une boucle ; une fenêtre froide du 120b (LRU 0,45, $\bar{d} = +0{,}094$) vaudrait $0{,}55/0{,}456 = 1{,}21$ [calc.]. (La note de la piste écrit $\rho \approx 1{,}19$ pour ce dernier cas ; le calcul posé donne 1,21.)

7.2.2 Le protocole et le verdict mécanique

Protocole déclaré (`genome/mimetisme/P1/protocole.md`, commit `c861e8f`, après les seuils `ec124f5`). Données : sept traces du 120b (cinq lectures : code, dialogue, prose-en, prose-fr, wiki ; deux générations de 256 jetons : prose-en et wiki), et celles du 20b. Grandeur : le taux de succès global, à la règle de groupe de `trc-cache` (les 4 experts d'une couche sont épinglés pendant leur groupe),

Table 7.2 : P1 : le verdict au seuil déclaré, 120b, 326 emplacements, $h^* = 32$ par validation croisée [mes.]. σ par blocs de 32 jetons. Le juge refait chaque nombre à moins de 10^{-5} par son propre chemin.

Cas	LRU	LFU(32)	d	σ	z	h oracle (non retenu)
génération prose-en	0,7379	0,7204	**−0,0176**	0,0213	−0,8	$h = 8$: +0,0055
génération wiki	0,3944	0,5080	+0,1136	0,0416	2,7	$h = 32$
chaîne prose-en	0,7402	0,6953	−0,0449	0,0224	−2,0	$h = 8$: +0,0021
chaîne wiki	0,3957	0,4938	+0,0981	0,0493	2,0	$h = 32$

Données : `genome/mimetisme/P1/resultat.md` § 2 ; `mesures/mimetisme/P1/juge-120b-326.json` ; juge : `genome/mimetisme/P1/JUGEMENT.md` § 2.

326 et 922 emplacements. h^* choisi par validation croisée sur les lectures des *autres* genres. Seuil : LFU(h^*) − LRU ≥ +0,02 et $z > 2$ sur les deux générations et la chaîne (cache chauffé par la lecture du même genre, puis génération) ; mort sous +0,005 sur l'une.

Témoins [mes.] (`mesures/mimetisme/P1/temoins.json`). W0 : le LRU de la piste égale celui de `cache2` (écart 0 sur 8 traces), et les repères du seuil retombent (0,7379 et 0,45279). W1 : LFU($h = 0{,}001$) = LRU à 0 près. W2 : LFU(∞) = « lfu » de `cache2` (0,66268 et 0,42632). W3 : les extrêmes du mélange, à 0 près. W4, la cassure : la décroissance inversée ($h = 8$) tombe sous le LRU de plus de 0,02 sur chaque trace testée, de justesse sur la génération wiki (0,0269). W5 : la fonction de verdict rend « tient », « tombe », « ouvert », « ouvert » sur quatre écarts fabriqués. W6 : Belady [99] majore toute politique (marges 0,094 et 0,165). Second chemin : un noyau Rust contre le Python, écart de succès au plus $9{,}8 \times 10^{-6}$ sur 108 séries.

Au seuil déclaré, P1 **tombe** : la génération prose-en est sous +0,005 (tableau 7.2). Même le h oracle de cette trace ne rend que +0,0055. À 922 emplacements, rien non plus (+0,0058, +0,0099, −0,0442, −0,0126). Ce que la chaîne montre : chauffer le cache par la lecture du même genre n'aide pas la génération (LRU 0,7379 → 0,7402), et nuit au LFU sans décroissance (prose-en : 0,6627 en génération seule, 0,1411 en chaîne) ; la décroissance sauve le LFU à la frontière lecture–génération (LFU(8) en chaîne : +0,0021 et +0,0856). Les comptes de la lecture deviennent de la pollution quand la génération part dans une autre région d'experts.

7.2.3 Ce que les traces de génération étaient vraiment

La piste a vu, avant son juge, que les deux générations finissent mal : prose-en tombe à 2 jetons distincts par bloc de 32 dès le jeton 128, wiki à 4 ou 5 sur ses 100 derniers jetons (figure 7.1, panneaux du bas). Elle l'a écrit « boucle du décodage glouton ». Le juge a lu les jetons eux-mêmes et corrigé la lecture [mes.] (`genome/mimetisme/P1/JUGEMENT.md` § 4.1) : dans `t120b.prose-en.generation.jetons`, la séquence harmony est complète (<|end|>, <|start|>, <|channel|>, <|message|> aux jetons 47 à 52 et 64 à 69), puis **<|return|> au jeton 109**. Le modèle a fini sa réponse proprement ; c'est le traceur qui a forcé 147 jetons de plus, que nul moteur de conversation ne produit. La phrase de la piste se lit donc ainsi : ~~« boucle du décodage glouton »~~ : jetons produits après <|return|>.

Sur la réponse réelle, tout h donne un gain positif, de +0,012 à +0,022 (tableau 7.3) ; le négatif vient entier de la queue, où deux jetons tiennent dans le LRU. La tête ne fait que trois blocs entiers de 32 : pas de z utilisable, et ce n'est pas un résultat, seulement la preuve que le « tombe » mesure le traceur. La trace wiki n'est pas une conversation non plus : son invite est du texte WikiText brut coupé au

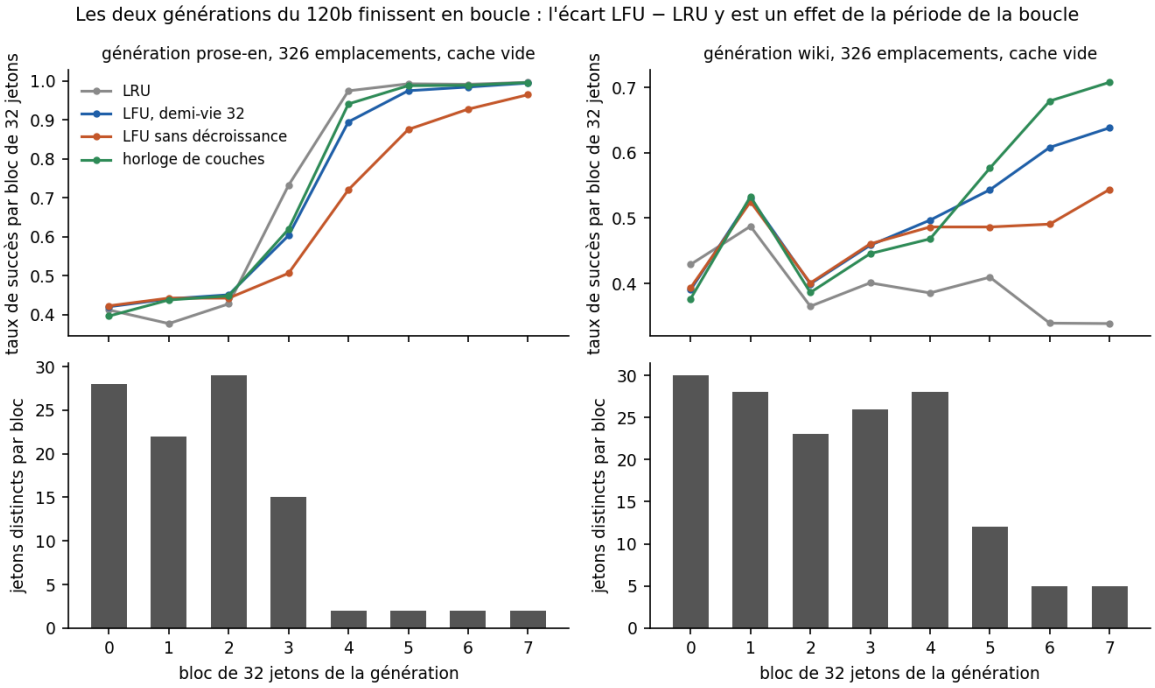


Figure 7.1 : Tout l'écart LFU − LRU des générations du 120b se décide dans leur seconde moitié, où les jetons distincts s'effondrent. [mes.] Haut : succès par bloc de 32 jetons, 326 emplacements, cache vide (LRU, LFU de demi-vie 32, LFU sans décroissance, horloge de couches). Bas : jetons distincts par bloc. Le titre d'origine dit « finissent en boucle » ; le juge a montré que la seconde moitié de prose-en est faite de jetons produits après la fin du message (<|`return`|> au jeton 109), et que wiki est une complétion brute sans gabarit de conversation. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P1/figure-blocs.csv` ; script `outils/mimetisme/P1/figures_p1.py`.

Table 7.3 : P1, génération prose-en du 120b, 326 emplacements, découpée à la fin du message (sonde S1 du juge, déclarée avant calcul) [mes.].

	tête 0–108 (réponse réelle)	queue 109–255 (après la fin)
LRU	0,415	0,977
LFU(8) − LRU	+0,0215	−0,0063
LFU(32) − LRU	**+0,0185**	−0,0443
LFU(∞) − LRU	+0,0134	−0,1409

Données : `genome/mimetisme/P1/JUGEMENT.md` § 4.1 ; `mesures/mimetisme/P1/juge/sondes.json`.

Table 7.4 : P1 : les 42 fenêtres froides de 256 jetons des lectures [mes.]. Les lignes du 120b à 326 sont refaites par le juge (à 10^{-4}) ; les autres ne le sont pas.

Modèle, capacité	LRU par genre	$\bar{d}$	σ	z	LFU gagne	genres $\geq +0{,}02$
120b, 326	0,35 à 0,51	+0,0939	0,0029	32,8	42/42	5/5
120b, 922	0,69 à 0,81	+0,0314	0,0030	10,4	38/42	4/5
20b, 217	0,67 à 0,78	+0,0528	0,0020	27,0	42/42	5/5
20b, 307	0,82 à 0,90	+0,0248	0,0014	18,2	41/42	5/5

Données : `mesures/mimetisme/P1/fenetres-120b.json`, `fenetres-20b.json` ; `genome/mimetisme/P1/resultat.md` §4.

milieu d'un mot, le modèle y fait de la complétion et répète « Here is a summary » ; sur sa tête (jetons 0 à 159), LFU(32) − LRU = +0,041, sur sa queue +0,234 [mes.] : le gain wiki du verdict est lui aussi surtout un effet de boucle.

Un trou de spécification. Le compte d'un expert survit-il à son éviction ? Le protocole ne le disait pas. Le juge, en perdant le compte à l'éviction, obtenait LFU(∞) = 0,2705 au lieu de 0,66268 ; le repère W2 n'est retrouvé, à 10^{-6}, que si le compte de chacun des 4 608 experts est *gardé à vie* [mes.] (`mesures/mimetisme/P1/juge/juge-oubli.json`). C'est la convention mesurée par P1 ; dans le moteur, elle coûte une table de 4 608 compteurs. Le verdict y est robuste : en LFU « dans le cache », la génération prose-en donne −0,0111 et tombe aussi.

7.2.4 Les fenêtres froides : la fréquence, pas la demi-vie

Pour savoir si le gain existe sur un texte qui avance, la piste a déclaré avant calcul une analyse post hoc (`genome/mimetisme/P1/posthoc-declare.md`) : les cinq lectures découpées en 42 fenêtres disjointes de 256 jetons, cache vide au début de chacune, même choix de h^*, même seuil, et l'exigence de plus d'un genre ($\geq +0{,}02$ pour 3 genres sur 5).

À la règle déclarée, l'analyse dirait « tient » (tableau 7.4). Mais *sa cassure échoue* : la décroissance inversée ne tombe pas sous le LRU en fenêtre de 256 jetons (elle le dépasse de +0,0174 au 120b). Ce que la fenêtre voit n'est pas la demi-vie : le LFU *sans* décroissance rend déjà +0,0849, la décroissance n'ajoute que +0,009 ($\sigma = 0{,}0018$). Sur 256 jetons, les comptes n'ont pas le temps de vieillir ; ce qui paie est la fréquence, quelques experts chauds (figure 7.2). Et ce gain ne se transfère pas aux générations saines du 20b : à 217 emplacements, code −0,0015, dialogue +0,0015, prose-fr +0,0524, wiki −0,0297 [mes.]. Le texte imposé d'une lecture et le texte glouton du modèle n'ont pas la même structure d'usage des experts.

L'espace des usages. Pourquoi la lecture prépare-t-elle si mal la génération ? La figure 7.3 représente chaque bloc de 32 jetons par le vecteur de ses usages d'experts (4 608 dimensions, norme 1). La génération forme un bloc à part dans la matrice des similarités cosinus (0,4 à 0,5 avec la lecture, 0,3 pour la queue répétée) ; dans le plan principal (41 et 15 % de la variance pour wiki, 30 et 13 % pour prose-en), la lecture occupe un nuage compact et la génération part sur une trajectoire distincte. Un cache chauffé par la lecture contient la mauvaise région, et les comptes anciens polluent.

Verdict du juge : ouvert. Le chiffre est reproduit au bit par un chemin indépendant, et le « tombe » mécanique est exact au seuil déclaré. Mais il ne juge pas le LFU en génération : il est décidé par 147 jetons d'après la fin. Sur la réponse réelle, le LFU(32) gagne +0,0185, sous le seuil de +0,02 et

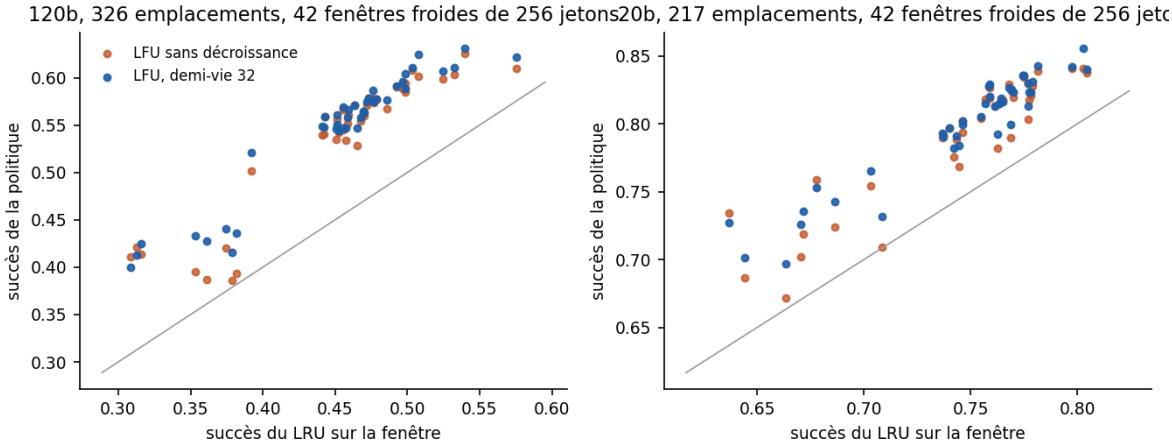


Figure 7.2 : Sur une fenêtre froide de 256 jetons, le LFU bat le LRU 42 fois sur 42, avec ou sans décroissance : c'est la fréquence qui paie. [mes.] Chaque point est une fenêtre ; la diagonale est le LRU. Gauche : 120b, 326 emplacements ; droite : 20b, 217. Le titre du panneau gauche déborde sur le droit dans l'original. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P1/fenetres-120b.csv`, `fenetres-20b.csv` ; script `outils/mimetisme/P1/figures_p1.py`.

sur trois blocs : ni tenu, ni tombé. Pour fermer P1 : au moins quatre générations du 120b arrêtées à `<|return|>`, jugées sur les seuls jetons d'avant la fin, et la convention du compte écrite (gardé à vie).

7.3 P2 : le cache exclusif de la VRAM vers la RAM

L'idée. Dans une hiérarchie de stockage, quand le niveau haut évince un bloc, il peut le *descendre* dans le niveau bas au lieu de le dupliquer : c'est la hiérarchie exclusive de Wong et Wilkes [100]. Le cache du moteur fait l'inverse : dans `Cache::demander`, la RAM est remplie à chaque défaut disque et rafraîchie à chaque succès de la VRAM (lu par le tri et par le juge dans `moteur/crates/trc-cache/src/lib.rs`, lignes 153 à 241). Elle contient donc les experts les plus récents, déjà en VRAM. La simulation de l'éclaireur le disait sans appel : l'inclusif avec $R \leq V$ ajoute 0,0000 de succès.

7.3.1 La pile LRU et le théorème

Définition 7.6 (Distance de pile)**.** La pile LRU est la liste des experts déjà vus, du plus récent (rang 1) au plus ancien. La distance de pile d'un accès à x est le rang de x dans la pile juste avant l'accès (infinie au premier accès). On note $F(C)$ la part des accès de distance de pile au plus C.

Lemme 7.7 (Inclusion, Mattson et al.)**.** *Un cache LRU de capacité C contient, à tout instant, les C premiers éléments de la pile LRU. Un accès y est un succès si et seulement si sa distance de pile est au plus C ; le succès d'un LRU de capacité C vaut donc $F(C)$.*

Démonstration. Par récurrence sur les accès. Un accès de distance $d \leq C$ porte x en tête ; les rangs $1, \dots, d-1$ glissent d'un cran, l'ensemble des C premiers rangs est inchangé : succès. Un accès de distance $d > C$ porte x en tête et pousse le rang C au rang $C+1$, qui est exactement le moins récent du cache, celui que le LRU évince : défaut. ∎

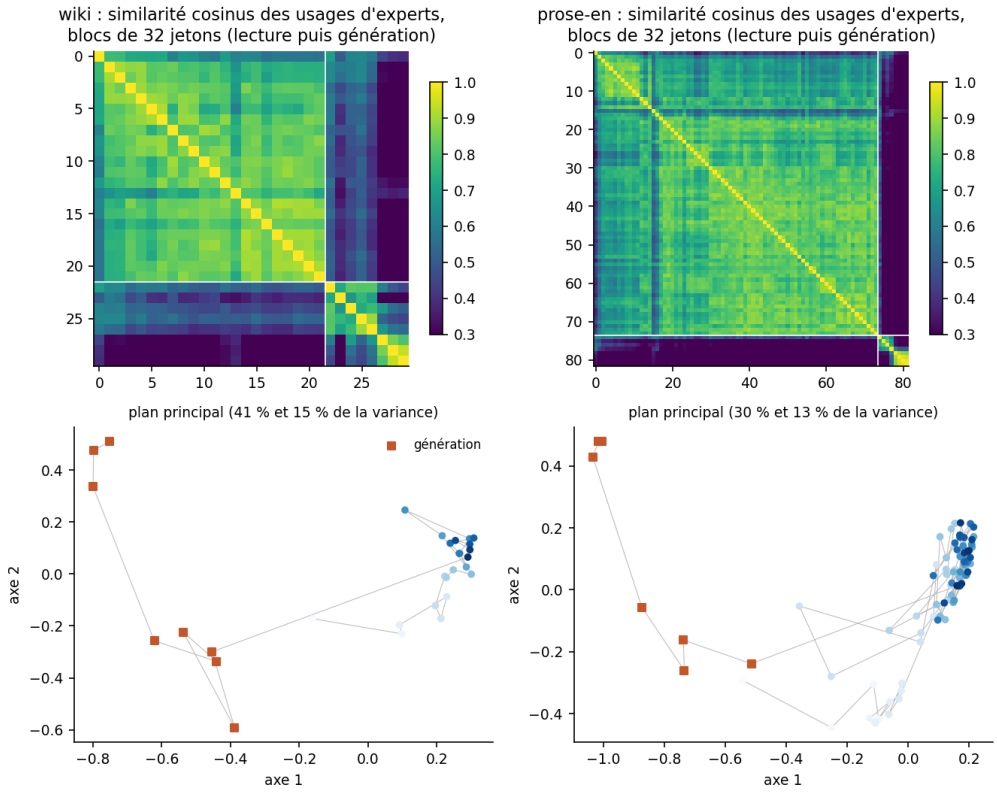


Figure 7.3 : La génération occupe une autre région de l'espace des usages d'experts que la lecture qui la précède. [mes.] Haut : similarité cosinus entre blocs de 32 jetons (lecture puis génération, séparées par le trait blanc). Bas : projection sur les deux premiers axes principaux des vecteurs centrés ; carrés : génération. La queue de génération est faite de jetons d'après la fin du message (juge, section 7.2.3). (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P1/figure-topologie.csv` ; script `outils/mimetisme/P1/figures_p1.py`.

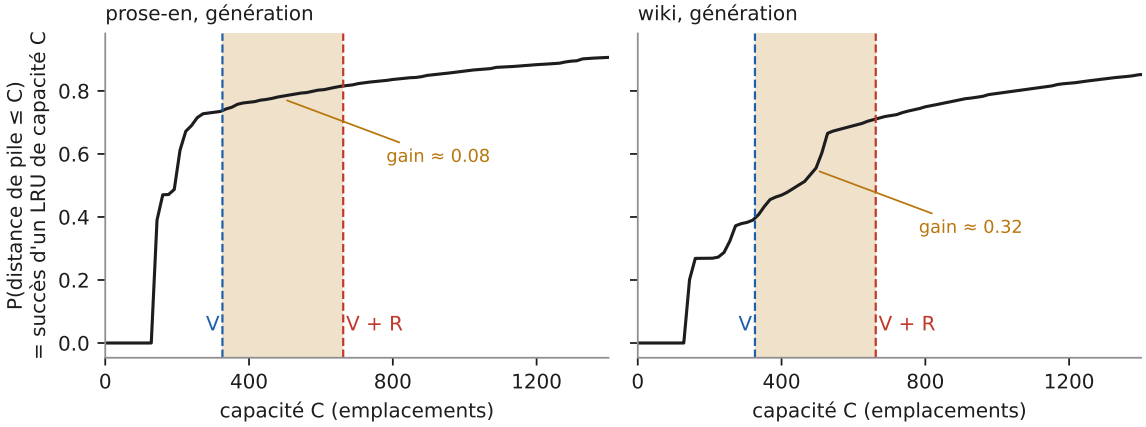


Figure 7.4 : Le gain d'un étage exclusif est la masse des distances de pile dans la bande $(V, V+R]$.
[mes.] [calc.] $F(C)$, succès d'un LRU de capacité C, sur les deux générations du 120b ; bande hachurée pour $V = 326$, $R = 336$. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P2/explore-distances.csv` ; script `outils/mimetisme/P2/figures.py`.

C'est la propriété des algorithmes à pile de Mattson, Gecsei, Slutz et Traiger [101] (notice relue par Crossref par le lot O le 2026-10-15 ; l'article n'est pas relu ici, et la démonstration ci-dessus ne s'appuie pas sur lui).

Théorème 7.8 (L'exclusif à deux étages vaut un LRU de capacité $V+R$)**.** *Soit un étage VRAM de capacité V et un étage RAM de capacité R, tous deux en LRU. Sur un défaut de la VRAM, la victime de la VRAM descend en tête de la RAM ; si l'expert demandé est en RAM, il monte et quitte la RAM ; si la RAM déborde, son plus ancien disparaît. Alors, à tout instant, la VRAM contient les rangs $1, \dots, V$ de la pile, la RAM les rangs $V+1, \dots, V+R$, et le succès total vaut $F(V+R)$.*

Démonstration. Par récurrence sur les accès ; vrai pour la pile vide. Soit un accès à x de rang d. *Si* $d \le V$, x va en tête, les rangs $1, \dots, d-1$ glissent d'un cran ; l'ensemble des rangs $1, \dots, V$ est inchangé, la RAM aussi : succès VRAM. *Si* $V < d \le V+R$, x est en RAM ; il monte et la quitte. L'ancien rang V devient le rang $V+1$: c'est la victime de la VRAM (son plus ancien), qui entre en tête de la RAM, à sa place de rang $V+1$. Les anciens rangs $V+1, \dots, d-1$ glissent aux rangs $V+2, \dots, d$, les rangs au-delà de d ne bougent pas : la RAM est exactement les rangs $V+1, \dots, V+R$ de la nouvelle pile. Succès RAM. *Si* $d > V+R$ (ou au premier accès), x va en tête, la victime de la VRAM (rang V) devient le rang $V+1$ et entre en tête de la RAM ; la RAM déborde d'un élément, son plus ancien, qui est le rang $V+R+1$ de la nouvelle pile, et il sort. Défaut disque. ■

Corollaire 7.9 (La bande de pile)**.** *La part servie par la RAM vaut $F(V+R) - F(V)$, la part disque $1 - F(V+R)$. Le gain de l'exclusif est la masse des distances de pile comprises dans la bande $(V, V+R]$.*

Avec l'épinglage des groupes, la victime est la plus ancienne *hors groupe*, et l'égalité n'est plus exacte. Mesuré (témoin T2, 7 traces, $R = 80$ et 336) : l'écart au LRU de capacité $V+R$ ne dépasse pas $6{,}6 \times 10^{-4}$; sur le 20b sans épinglage, il est exactement 0 [mes.] (`mesures/mimetisme/P2/holdout 20b-t2-sans-epinglage.json`). La figure 7.4 montre le théorème sur les deux générations : le gain est l'aire sous F entre V et $V+R$, 0,08 sur prose-en où la courbe est presque plate après V, 0,32 sur wiki dont l'ensemble de travail est à cheval sur la frontière.

L'inclusif, vu par la même pile. Le juge a mesuré qu'un inclusif dont la RAM compte $R' = V + R = 662$ entrées a *exactement* la part disque de l'exclusif à $R = 336$ (0,184489 et 0,288493 sur les deux générations, épinglage compris), sans aucune descente [mes.] (`mesures/mimetisme/P2/juge/budget-egal.json`). Le même argument de pile l'explique ; la proposition suivante est posée par la rédaction, sous une hypothèse sur la règle du moteur que le juge a lue dans le code mais qui n'est pas mesurée en elle-même.

Proposition 7.10 (L'inclusif de RAM $R' \geq V$)**.** *Supposons, sans épinglage, que la RAM inclusive reçoive chaque expert lu du disque, que la date de chacune de ses entrées soit rafraîchie à chaque accès (servi par la VRAM ou par elle), et qu'elle évince son entrée la plus ancienne. Si $R' \geq V$, la RAM contient à tout instant les rangs $1, \dots, R'$ de la pile, et le succès total vaut $F(R')$.*

Démonstration. Par récurrence ; vrai au départ, tant que la RAM n'est pas pleine (chaque expert vu y est entré à son premier accès). Supposons la RAM égale aux R' premiers rangs ; comme $V \leq R'$, la VRAM (les V premiers rangs, lemme 7.7) y est incluse. Un accès de rang $d \leq R'$ est servi (par la VRAM si $d \leq V$, par la RAM sinon) et rafraîchit l'entrée de x dans la RAM : la RAM reste les R' premiers rangs de la nouvelle pile. Un accès de rang $d > R'$ est un défaut disque ; x entre en RAM et son entrée la plus ancienne, l'ancien rang R', devenu le rang $R' + 1$, sort. ■

D'où, à capacités égales en entrées, $1 - F(662)$ pour l'inclusif à 662 comme pour l'exclusif $(326, 336)$. Pour $R < V$ l'argument ne passe pas : une entrée de la RAM peut vieillir pendant que la VRAM sert des experts absents de la RAM. La piste n'en donne qu'une esquisse, et la mesure en tient lieu : 0 de succès RAM à $R = 80$ et 160 sur les 7 traces (`mesures/mimetisme/P2/grille.csv`, mode inclusif), au plus $7{,}65 \times 10^{-4}$ à $R = V = 326$ [mes.] (`mesures/mimetisme/P2/t1bord.csv`).

7.3.2 Le coût de la descente

Un défaut disque coûte $a = 4{,}1$ ms (11,6 Mo à 2,82 Go/s) ; un défaut servi par la RAM décodée $b = 1{,}09$ ms (13,25 Mo à 12,2 Go/s) ; une descente de la VRAM vers la RAM coûte b aussi [calc.]. Soit φ le nombre de défauts VRAM par jeton et ρ la part d'entre eux servie par la RAM ; chaque défaut VRAM provoque une descente. Par jeton :

$$\begin{aligned} T_{\text{LRU}} &= \varphi\, a, \\ T_{\text{série}} &= \varphi\,\big[(1-\rho)\,a + \rho\, b + b\big], \\ T_{\text{recouvert}} &= \varphi\,\big[(1-\rho)\,a + \rho\, b\big]. \end{aligned} \tag{7.4}$$

Le gain en série vaut $\varphi\,[\rho(a-b) - b]$, positif si et seulement si

$$\rho > \frac{b}{a-b} = \frac{1{,}09}{3{,}01} = 0{,}362. \tag{7.5}$$

Avec une fraction ω de la descente recouverte par le calcul, la rentabilité demande $\rho(a-b) \geq (1-\omega)\,b$, soit $\omega \geq 1 - \rho(a-b)/b$. Les temps mesurés le disent directement : sur prose-en, le LRU seul coûte 154,7 ms par jeton, l'exclusif recouvert 121,1 ms et la descente 39,75 ms, donc il faut $\omega \geq 1 - 33{,}6/39{,}75 = 0{,}154$ [calc.] ; $\rho \approx 0{,}30$ y est sous le seuil (7.5), et $\rho \approx 0{,}52$ sur wiki au-dessus.

7.3.3 La mesure, le verdict et ce que le juge corrige

Seuil déclaré (`genome/mimetisme/P2/seuil-declare.md`, commité avant toute lecture de trace) : réduction relative de la part disque, exclusif contre inclusif, $V = 326$, $R = 336$, d'au moins 15 %

Table 7.5 : P2, $V = 326$, $R = 336$: la piste et le juge [mes.]. Écart sous 10^{-5} sur les parts disque, sous 0,1 % sur les descentes.

Génération		disque inclusif	disque exclusif	réduction	descentes/jeton	descente/345 ms
prose-en	piste	0,2573	0,1845	28,3 %	36,5	11,5 %
	juge	0,257270	0,184489	28,29 %	36,469	11,52 %
wiki	piste	0,5926	0,2885	51,3 %	85,9	**27,2 %**
	juge	0,592556	0,288493	51,31 %	85,938	27,15 %

Données : `genome/mimetisme/P2/resultat.md` § 1 ; `genome/mimetisme/P2/JUGEMENT.md` § 1 ; `mesures/mimetisme/P2/verdict.json`, `juge/verdict-juge.json`.

Table 7.6 : P2 : temps de service des défauts par jeton, $V = 326$, $R = 336$ [calc.] sur [mes.]. Entre parenthèses, le gain sur le LRU seul.

Trace	LRU seul	exclusif en série	exclusif recouvert
prose-en, génération	154,7 ms	160,9 (−6,1)	121,1 (+33,6)
wiki, génération	357,6 ms	313,8 (+43,8)	220,1 (+137,5)

Données : `mesures/mimetisme/P2/net-selectif-120b.csv` (lignes $s = 0$) ; refait par le juge à 1 ms près (J4c).

sur les deux générations, *et* trafic de descente au plus 25 % d'un jeton de 345 ms ; mort sous 5 % sur l'une.

La première condition tient sur les deux générations (28,3 et 51,3 % contre 15 %), la seconde tombe sur wiki (27,2 % contre 25 %) : **ouvert** (tableau 7.5). Rapportée au temps de disque propre à wiki (357,6 ms en LRU seul), la descente vaut encore 26,2 % : le verdict ne bouge pas. Témoins, tous tenus et capables de tomber : $R = 0$ redonne le LRU seul aux entiers près (T0) ; l'inclusif à $R \leq V$ ajoute 0 (T1, tolérance relâchée à 10^{-3} au bord $R = V$, dit avant) ; l'exclusif égale le LRU de $V + R$ à $6{,}63 \times 10^{-4}$ (T2) ; la démotion supprimée redonne le LRU seul, le remplissage au défaut redonne l'inclusif (T3) ; le simulateur de l'éclaireur est retrouvé à $4{,}8 \times 10^{-4}$ (T4). Second chemin : un Rust à horodatages, égal au Python sur 154 cellules. La figure 7.5 montre l'inclusif collé au LRU seul tant que $R \leq V$, puis décrochant ; l'exclusif part dès $R = 80$.

Ce que le juge corrige (`genome/mimetisme/P2/JUGEMENT.md` § 3). (1) La référence du seuil, « commité en 6f3d737 », désigne un commit qui n'est sur aucune branche (version amendée d'un commit qui portait un trailer) ; le commit sur `main` est `6d325d8`, même seconde, contenu identique. (2) La référence « inclusif à $R = 336$ » est dégénérée par construction : « 28 et 51 % contre l'inclusif » se lit « contre rien ». La vraie alternative est l'inclusif de $R' = 662$, qui a la même part disque (proposition 7.10) sans aucune descente ; le vrai marché de l'exclusif est donc d'économiser V entrées de RAM (environ 3,2 Go : 7,68 Go encodés contre 4,45 Go décodés) contre le trafic de descente. En série, cet inclusif bat l'exclusif (+25,7 contre −6,1 ms sur prose-en, +105,0 contre +43,8 sur wiki) ; recouvert, l'exclusif gagne. (3) $V = 922$ n'est pas une option sur la carte de la machine. (4) La démotion sélective (ne descendre que les victimes réutilisées au moins s fois en VRAM), choisie après coup et dite telle, n'est qu'un réglage sur la même courbe : son avantage n'existe qu'en série, et recouverte elle perd sur wiki (253,2 ms contre 220,1). (5) Le théorème n'est pas neuf : c'est l'inclusion de Mattson et l'argument de Wong et Wilkes ; il est juste, et le juge le refait exactement par un troisième chemin (distances de pile par arbre de Fenwick : 6 801 = 6 801 et 10 635 = 10 635 défauts).

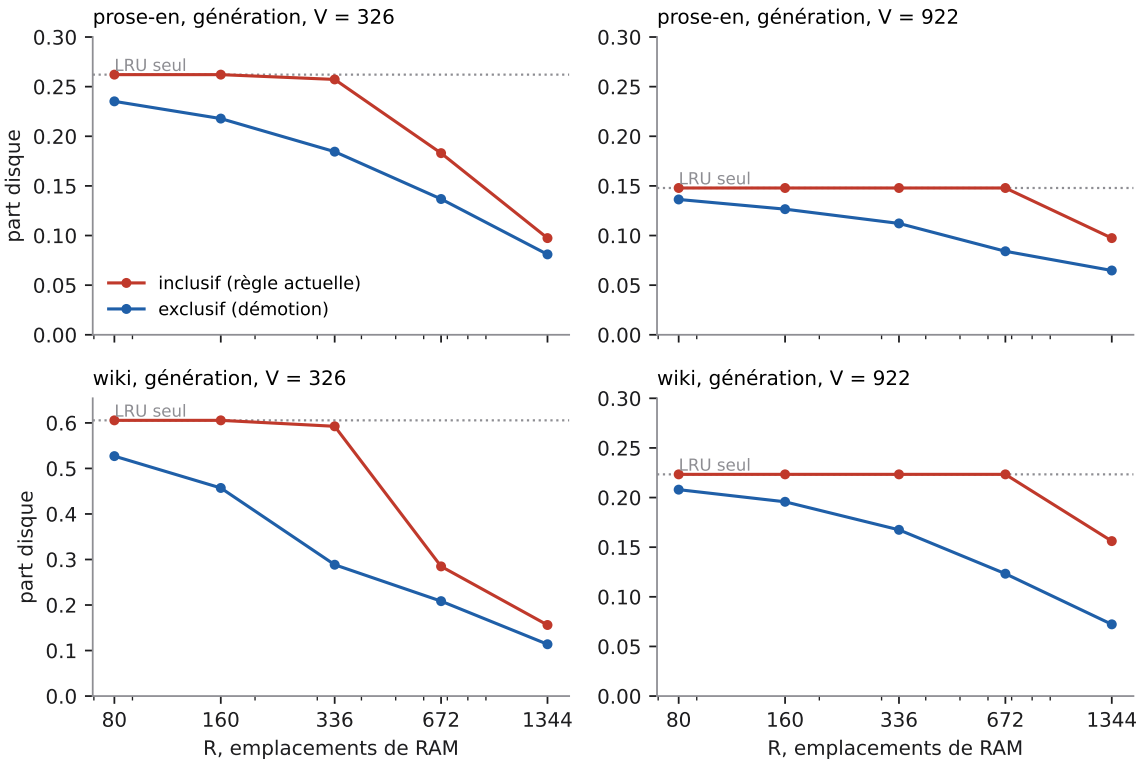


Figure 7.5 : L'étage RAM inclusif ne sert à rien tant que $R \leq V$; l'exclusif sert dès la première entrée. [mes.] Part des accès servis par le disque contre R, deux générations du 120b, $V = 326$ et 922 ; en pointillés, le LRU seul. Le juge rappelle que $V = 922$ entrées décodées font 12,2 Go de VRAM, plus que la carte (8 188 Mio) : ce n'est pas une option sur cette machine. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P2/grille.csv` ; script `outils/mimetisme/P2/figures.py`.

Verdict du juge : ouvert, reproduit. Ce que P2 apporte vraiment : un étage RAM inclusif de taille $R \leq V$ ne sert à rien (le `--ram-tiede` actuel, en simulation) ; l'exclusif atteint la part disque d'un inclusif de $V + R$ entrées avec V entrées de RAM en moins, au prix de 36 à 86 descentes par jeton. La décision de Stone n'est pas « exclusif ou inclusif à R égal », c'est « 3,2 Go de RAM de plus, ou une descente recouverte ». Le nœud reste le recouvrement de la descente, à mesurer dans le moteur en ABBA, contre l'inclusif à $V + R$ qui ne demande aucun code.

7.4 P3 : le routage conditionné par le cache, borné par la marge

L'idée. Skliar et al. ajoutent un biais aux logits des experts déjà en cache avant le top-k du routeur [102]. Trois éclaireurs y sont arrivés par d'autres chemins : la faille sismique qui colle (friction à vitesse et état), la substitution d'un article en rupture de stock, la puissance de deux choix des files d'attente ; ce sont des analogies, et la mesure seule décide. P3 change le *flux de demandes*, pas la politique : Belady est optimal pour un flux fixé, et la politique ne bouge plus (le meilleur prédicteur dépasse le LFU réglé de +0,011 à +0,013, mathématiques M1). C'est aussi la plus risquée des six : une bascule voulue coûte comme une bascule subie, et les bascules portent 96,5 % de la KL sur les positions qui basculent (`genome/05-synthese.md` § 2.2).

Ce chapitre ne cite que ce qui est jugé. L'étage A (boucle ouverte), l'étage B à $N = 146$, l'étage B à $N = 238$ et le routeur biaisé en ligne dans le moteur ont chacun leur jugement commité. Un second tirage est en travail dans une autre session (`genome/mimetisme/P3/REPRISE-second-tirage.md`) : aucun de ses chiffres n'est cité. Un témoin à $\beta = 0{,}3$ hors moteur (`resultat-temoin-b0.3.md`) n'a pas de jugement propre : il n'est pas cité non plus.

7.4.1 La géométrie : une paroi décalée, une bande

Un routeur de gpt-oss-20b rend, par cellule (position t, couche ℓ), 32 logits $z \in \mathbb{R}^{32}$; l'ensemble choisi $S(z)$ est celui des quatre plus grands. Dans $\mathbb{R}^{32}$, les régions $\{z : S(z) = S_0\}$ sont des cônes polyédriques ; leurs parois sont des hyperplans $\{z_i = z_j\}$ de l'arrangement des tresses, et la paroi qui compte pour S est le lieu tropical $\{z : z_{(4)} = z_{(5)}\}$, où s'annule la *marge* $m(z) = z_{(4)} - z_{(5)}$. La marge est, à $\sqrt{2}$ près, la distance de z à la paroi la plus proche qui change S (le chapitre 12 en fait le sujet).

Le biais de Skliar et al. est une *translation* $z' = z + \beta\, \mathbf{1}_R$, où $\mathbf{1}_R$ indique les experts résidents. Il ne change pas la géométrie des régions ; il déplace le point. Pour une paire (a absent, b résident), cela revient à décaler la paroi $z_a = z_b$ en $z_a = z_b + \beta$ (figure 7.6). Toute cellule dont le point tombe dans la bande entre les deux parois change de choix : a était pris, b l'est maintenant. *La part de cellules changées est la masse de probabilité de la bande.*

La masse de la bande, mesurée. La fonction de répartition de la marge sur les 122 640 cellules (24 couches × 5 110 positions) est connue à 3×10^{-6} près, recalculée indépendamment des journaux [mes.] (`mesures/mimetisme/P3/etage-A.json`) :

β (logit)	0,01	0,03	0,1	0,3
$P(m < \beta)$	0,0818	0,2240	0,5632	0,9035
densité moyenne $P(m < \beta)/\beta$	8,2	7,5	5,6	3,0

La masse croît presque linéairement jusqu'à $\beta \approx 0{,}03$ (densité en zéro $g(0) \approx 8$ par unité de logit, cohérente avec la loi presque exponentielle de la marge du chapitre 12), puis sature. L'étendue moyenne des logits vaut 4,136 : $\beta = 0{,}1$ correspond à $\lambda = 0{,}024$ dans la normalisation de l'article.

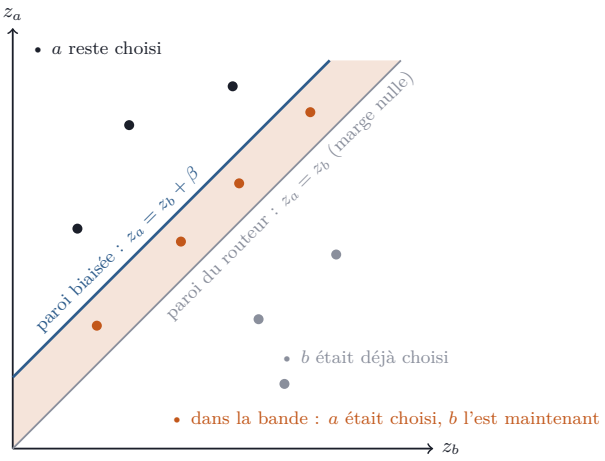


Figure 7.6 : Le biais vers les résidents ne déforme pas les régions du routeur : il décale une paroi, et la bande balayée est la part des cellules qui changent. [schéma] Plan (z_b, z_a) d'un absent a et d'un résident b : paroi du routeur $z_a = z_b$ (marge nulle) et paroi biaisée $z_a = z_b + \beta$. Un schéma, pas une mesure. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)
Données : `genome/mimetisme/P3/figures/fig-paroi.tex`.

Le facteur de résidence. La variante V2 (n'échanger que le 4e absent contre le 5e résident) change une cellule exactement quand $m < \beta$, que le 4e est absent et le 5e résident. Si la marge était indépendante de la résidence,

$$\text{part changée}(\beta) = P(m < \beta) \cdot P(a \text{ absent}) \cdot P(b \text{ résident} \mid a \text{ absent}). \tag{7.6}$$

Le rapport mesuré « part changée / $P(m < \beta)$ » vaut 0,237, 0,232, 0,225, 0,210 aux quatre β à $N = 146$, et 0,222 à 0,188 à $N = 238$ [mes.] : presque constant, c'est bien un facteur de résidence. Avec $P(a \text{ absent})$ lu dans les défauts de rang 4 à $\beta = 0$ (0,544 et 0,301), on en tire, au plus petit β, $P(b \text{ résident} \mid a \text{ absent}) =$ 0,437 et 0,738 [calc.].

Un échange évite plus d'un défaut. Aux petits β, chaque expert échangé évite 1,44 à 1,57 défaut à $N = 146$ et 1,18 à 1,25 à $N = 238$ [mes.] : le défaut du jour, plus l'absent qui n'entre pas dans le cache et n'en évince pas un utile, plus la récence rendue au résident. Le taux de change au point −10 % vaut $0{,}1 \times 216\,397/(0{,}1198 \times 122\,640) = 1{,}47$ défaut par cellule à $N = 146$, et 1,21 à $N = 238$ [calc.].

7.4.2 Le critère de décision en une grandeur

Proposition 7.11 (Le seuil de la pente κ). *Soit $f^*(N)$ la part de cellules à changer pour retirer 10 % des défauts au budget N, et supposons la KL excédentaire linéaire en la part changée,* $\mathrm{KL} = \kappa f$ *(régime des petits β). Le seuil de succès ($\mathrm{KL} \le \varepsilon = 0{,}038$ à −10 %) équivaut à*

$$\kappa \le \kappa_{\max}(N) = \frac{\varepsilon}{f^*(N)}, \tag{7.7}$$

et le seuil de mort ($\mathrm{KL} > 0{,}07$) à $\kappa > 0{,}07/f^*(N)$.

Démonstration. Au point $f = f^*$, $\mathrm{KL} = \kappa f^* \le \varepsilon \iff \kappa \le \varepsilon/f^*$; de même pour 0,07. ∎

Avec $f^*(146) = 11{,}98$ % et $f^*(238) = 7{,}31$ % (franchissements interpolés) : $\kappa_{\max} = 0{,}038/0{,}1198 =$ 0,317 et $0{,}038/0{,}0731 = 0{,}520$; la mort est à $\kappa = 0{,}584$ et 0,958 [calc.]. Le seul κ connu avant l'étage B

Table 7.7 : P3, étage A : défauts évités et part de cellules changées, par β [mes.]. Chaque nombre est refait par le juge, défaut pour défaut, avec un autre lecteur GGUF, un autre produit matriciel et un autre cache.

	$N = 146$ (19 %)	$N = 238$ (31 %)
défauts à $\beta = 0$	216 397 (succès 0,5589)	108 472 (succès 0,7789)
V1 (tous résidents), $\beta = 0{,}03$	−5,1 % ; 6,0 % changées	−7,5 % ; 5,4 %
V1, $\beta = 0{,}06$	−9,9 % ; 11,9 %	−14,3 % ; 10,5 %
V1, $\beta = 0{,}1$	**−16,1** % ; 19,3 %	**−22,6** % ; 16,6 %
V1, $\beta = 0{,}3$	−40,9 % ; 48,2 %	−53,0 % ; 37,7 %
V2 (4e/5e seul), $\beta = 0{,}1$	−10,4 % ; 12,7 %	−15,3 % ; 11,4 %
franchissement de −10 % (V1)	$\beta = 0{,}060$; 12,0 %	$\beta = 0{,}041$; 7,3 %
KL a priori à ce franchissement (0,309× part)	0,037	0,023
$\beta = -0{,}1$ (on favorise les absents)	+17,4 %	+28,2 %
masque au hasard, même densité, $\beta = 0{,}1$	+0,7 %	+1,9 % (juge : +1,48 %)

Données : `genome/mimetisme/P3/resultat.md` § 2 ; `mesures/mimetisme/P3/etage-A.csv`, `affinage.csv` ; juge : `genome/mimetisme/P3/JUGEMENT.md` § 2.

était celui des bascules spontanées entre deux moteurs presque exacts :

$$\kappa_0 = \frac{0{,}0145 - 0{,}00168}{5\,087/122\,640} = \frac{0{,}01282}{0{,}04148} = 0{,}309, \tag{7.8}$$

2,5 % sous $\kappa_{\max}$ à $N = 146$: une marge infime. La forme (7.7) est celle qu'il faut retenir : un seuil sur une pente, ε étant la pente linéarisée du mode rapide fois dix points.

7.4.3 L'étage A : la boucle ouverte

Seuils (`mesures/mimetisme/P3/00-seuils-declares.md`, commit `1606f9f`, seul fichier du commit, avant tout contact avec les données). Modèle : gpt-oss-20b, 5 110 positions (10 fenêtres de 511), 122 640 cellules ; cache LRU global à règle de groupe, $N = 146$ (19 %) et 238 (31 %) ; bascules causales (résidents = contenu du cache juste avant le groupe), poids de mélange non biaisés. Les ensembles sont calculés sur la trajectoire du modèle exact : c'est la *boucle ouverte*.

Le levier a de l'amplitude, et elle vient de la résidence : le masque au hasard de même densité ne gagne rien, le signe inversé perd (tableau 7.7). Aux petits β, un seul expert change par cellule changée (1,00 à 1,02), et l'expert de plus haut logit reste choisi dans 99,98 à 100 % des cellules : le levier ne touche que les derniers rangs. Les courbes V1 et V2 se superposent dans le plan (part changée, défauts évités) (figure 7.7) : la monnaie du levier est le nombre de cellules changées, pas la forme du biais. Si la vitesse suivait 1/défauts, −10, −16 et −23 % de défauts donneraient ×1,11, ×1,19 et ×1,29 [calc.], ce qui n'est pas mesuré.

Témoins [mes.] : W1, les logits recalculés depuis h et le routeur F32 retrouvent l'ensemble de la capture dans 122 640 cellules sur 122 640 (cassure : sans le biais du routeur, 0,6544) ; W2, $\beta = 0$ redonne les défauts du rejeu ; W3, les signes ; W4, la borne E7 respectée (rapports 0,53 à 0,78 ; tolérance de 1,5 non dérivée, dite) ; W5, un second LRU indépendant redonne le compte exact sur les 16 lignes ; W6, causalité par troncature. Verdict déclaré de l'étage A : **ouvert, étage B justifié** (−10 % atteint aux deux budgets, KL a priori sous 0,07).

Ce que le juge de l'étage A ajoute (`genome/mimetisme/P3/JUGEMENT.md` § 4). Les nombres sont exacts. Deux lectures sont trop optimistes. (1) L'étage B, tel qu'il est écrit, impose à la référence

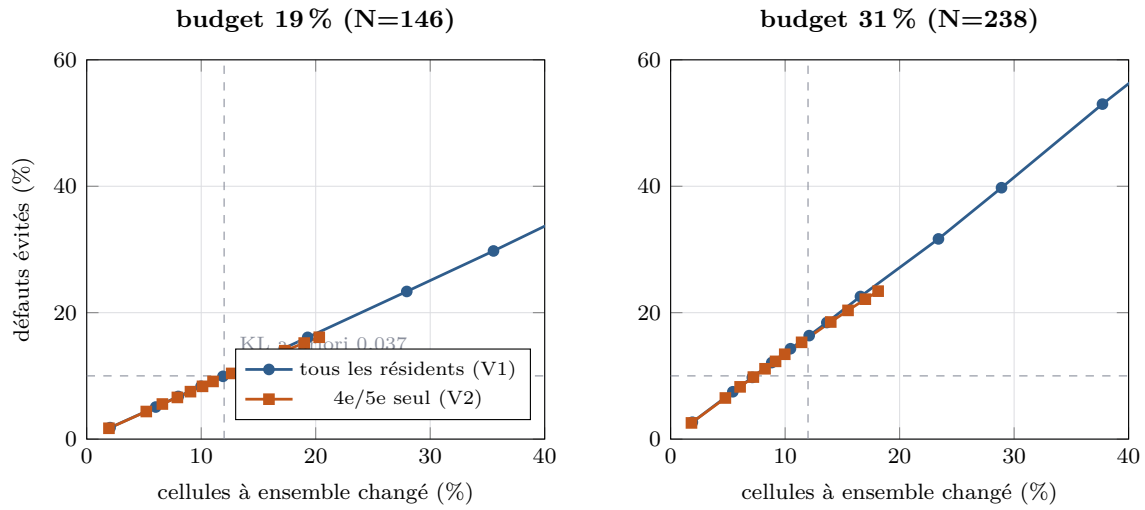


Figure 7.7 : La monnaie du levier est le nombre de cellules changées : les deux variantes du biais tombent sur la même courbe. [mes.] Défauts évités contre part de cellules à ensemble changé, V1 et V2, aux deux budgets ; repères à −10 % de défauts et au franchissement. L'étiquette « KL a priori 0,037 » est la constante des bascules spontanées ; le juge montre qu'elle est probablement trop basse pour un routeur en ligne (0,090 par racine de cascade). Dans l'original, cette étiquette chevauche la légende. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P3/series/*.csv` ; `genome/mimetisme/P3/figures/fig-pareto.tex`, script `outils/mimetisme/P3/p3_series.py`.

f64 les ensembles de la boucle ouverte à *toutes* les cellules : il interdit les bascules secondaires que la bascule voulue déclencherait aux couches suivantes. Or 59 % des bascules du lot R sont des secondaires (3 006 sur 5 087), et $P(\text{bascule en } \ell+1 \mid \text{bascule en } \ell) = 26{,}6\,\%$ contre 3,4 % (chapitre 12). L'étage B mesure donc le coût *direct* des bascules, une borne basse vraisemblable : une KL d'étage B au-dessus de 0,07 tue P3, une KL sous 0,038 ne suffit pas pour « tient ». (2) Le sens de l'erreur de la KL a priori est probablement connu : rapportée aux seules racines des cascades, la constante vaut $0{,}0128/(2\,081/122\,640) = 0{,}755$ au lieu de 0,309 [calc.], d'où une KL a priori de 0,090 à $N = 146$ (au-dessus de la mort) et 0,055 à $N = 238$.

7.4.4 L'étage B : la KL excédentaire mesurée

L'étage B impose, dans la référence f64 du lot R, les ensembles biaisés, et mesure la KL excédentaire et le premier choix de sortie contre la référence. Seuils (déclarés avec l'étage A, rappelés dans `mesures/mimetisme/P3/02-etage-B-declare.md`, commit `c4a037a`, avant toute passe) : « tient » si, au même β, −10 % de défauts, KL ≤ 0,038 et premier choix identique ≥ 95 % ; « tombe » si KL > 0,07 au β à −10 %.

À $N = 146$. À $\beta = 0{,}06$: défauts −9,93 % (194 904), KL 0,0209, premier choix 88,4 % ; à $\beta = 0{,}07$: −11,53 %, KL 0,0239, premier choix 87,6 % [mes.]. Le juge les relit des fichiers `.k1d` par un second lecteur, au dernier chiffre affiché (KL 0,0208902 et 0,0239339), et le témoin $\beta = 0$ tombe à zéro au bit (même empreinte SHA-256 que la référence). La KL tient, le premier choix tombe : **non concluant au seuil littéral**. Le juge a demandé d'où viennent les désaccords de premier choix (tableau 7.8) : en grande partie de quasi-égalités de la référence (la moitié sous 0,1 logit de marge, 86 % sous 0,3, contre

Table 7.8 : P3, étage B à $N = 146$: où tombent les désaccords de premier choix, selon la marge de la référence (log-probabilité du 1er moins celle du 2e) [mes.] par le juge.

	cassure T1 (51)	$\beta = 0{,}06$ (295)	$\beta = 0{,}07$ (317)	toutes positions
désaccords à marge $< 0{,}1$	84,3 %	53,6 %	53,3 %	14,4 % des positions
désaccords à marge $< 0{,}3$	94,1 %	86,1 %	85,8 %	37,2 % des positions
marge médiane des désaccords	0,015	0,087	0,088	(accords : 0,51)
premier choix si marge $\geq 0{,}1$	99,6 %	93,7 %	93,2 %	
premier choix si marge $\geq 0{,}3$	99,8 %	97,4 %	97,2 %	

Données : `genome/mimetisme/P3/JUGEMENT-etage-B.md` § 4 ; `mesures/mimetisme/P3/juge/etage-B/`.

14 et 37 % des positions), mais pas seulement : hors des positions à marge $< 0{,}1$, le premier choix reste à 93,7 et 93,2 %, sous 95 %. Ce découpage est post hoc et ne change pas le verdict.

À $N = 238$, avec un critère pondéré fixé après lecture. Le critère suivant a été écrit *après* avoir lu $N = 146$, et le dit en toutes lettres (`mesures/mimetisme/P3/03-critere-marge-declare.md`, commit `e8bf954`, avant toute passe à $N = 238$) : la lecture « pondérée » exige le premier choix ≥ 95 % sur les seules positions où la marge de la référence est au moins 0,3 logit. Sa raison, indépendante du chiffre lu : prendre le 2e jeton au lieu du 1er coûte au plus la marge en log-probabilité ; une bascule à marge $< 0{,}3$ est bon marché. Un « tient » de $N = 238$ seul est dit *partiel*. Au premier pas qui franchit -10 %, $\beta_1 = 0{,}041$: défauts 97 604, soit $-10{,}0192$ % (20,8 défauts sous le seuil de 97 624,8) ; KL 0,0117202 ; premier choix littéral 91,96 % ; pondéré 98,75 % (20 désaccords sur 1 601 positions) [mes.], reproduits par le juge au dernier chiffre. Verdict déclaré : **pondéré « tient, partiel » ; littéral « non concluant »**.

Le juge de $N = 238$ relève un grave de portée (`genome/mimetisme/P3/JUGEMENT-N238.md` § 5, G1) : sur les données réelles, la condition pondérée *ne mord jamais avant la KL*. Huit fichiers réels de KL $\leq 0{,}029$ la passent tous à 97,1 % ou plus, llama.cpp compris (92,55 % littéral et 98,06 % pondéré contre la même référence) ; le seul fichier réel qui la manque a une KL de 0,243. Le levier à β_1 est plus proche de la référence que llama.cpp au pondéré, et 0,6 point plus loin au littéral. Le « tient » pondéré dit donc, à ce jour, « -10 % de défauts et KL $\leq 0{,}038$ », pas une troisième chose.

7.4.5 Le routeur biaisé en ligne, dans le moteur

Seule l'épreuve en ligne lève la boucle ouverte : le biais s'applique aux logits recalculés à chaque couche sur la trajectoire réelle, avec les résidents tirés du cache du moteur au même instant. Mécanisme, grandeurs, seuils, témoin et cassure sont déclarés avant la moindre ligne de code du moteur (`mesures/mimetisme/P3/06-routeur-ligne-declare.md`, commit `0f38124`, puis le code en `eb78770`) : un noyau `routeur_topk_biais` choisit les quatre experts par `logit + b`, calcule les poids sur les logits non biaisés, derrière un drapeau absent par défaut ; base de comparaison : le même moteur sans biais (P0), 238 emplacements, mêmes fenêtres.

À l'estimation ponctuelle, la lecture pondérée **tient** et la littérale est **non concluante** ; la cassure $\beta = 0{,}3$ tombe, et le témoin $\beta = 0$ est égal au bit (trace et 5 120 empreintes) au moteur d'avant l'option (tableau 7.9). Le juge a lu le noyau ligne à ligne : le biais n'entre que dans la sélection, pas dans les poids ni dans l'ordre ; son émulation sur 20 000 tirages et les données réelles de la couche 0 ne montrent aucune fuite.

Table 7.9 : P3, routeur biaisé en ligne dans le moteur, 238 emplacements, contre le même moteur sans biais [mes.]. Refait par le juge au dernier chiffre affiché ; le pondéré se compte sur les 1 592 positions de marge $\geq 0{,}3$ de P0.

β	défauts (variation)	KL	littéral	pondéré
0 (P0, et P0b)	108 679	0	100 %	100 %
0,041	97 780 (−10,0286 %)	**0,031388**	88,549 %	**96,357 %**
0,05	95 610 (−12,0253 %)	0,037692	87,569 %	96,043 %
0,3 (cassure)	50 911 (−53,1547 %)	0,089069	76,980 %	88,631 %

Données : `genome/mimetisme/P3/JUGEMENT-routeur-ligne.md` § 3 ; `mesures/mimetisme/P3/routeur-ligne/`, `mesures/mimetisme/P3/juge/routeur-ligne/`.

Table 7.10 : P3 en ligne, $\beta_1 = 0{,}041$: part des cellules de la couche (en %) [mes.] par le juge.

couche	0	3	6	9	12	15	18	21	23
changées en ligne	7,38	8,65	11,12	10,02	12,64	17,63	15,19	12,41	20,12
changées en boucle ouverte	7,30	6,87	8,00	5,91	6,32	9,80	8,51	5,13	7,50
bascules naturelles (P0 ≠ réf.)	0,04	0,27	0,57	0,55	0,88	1,57	1,60	1,94	2,95
dérive certaine	0	1,70	3,01	3,72	5,23	7,20	6,01	6,30	11,94
part incohérente des changées	0	19,6	27,1	37,0	41,3	40,8	39,5	50,7	59,3

Données : `genome/mimetisme/P3/JUGEMENT-routeur-ligne.md` § 5 ; `mesures/mimetisme/P3/juge/routeur-ligne/cellules.json`.

Le grave G1 : rien ne tient à sa borne défavorable. Par rééchantillonnage des dix fenêtres, la borne haute de la KL est au-dessus de 0,038 pour 20 graines sur 20 (0,03831 à 0,03849) ; la borne basse du pondéré est sous 95 % pour 14 graines sur 20 (94,96 à 95,02 %) ; la marge de 31 défauts au seuil tombe à −86 si le cache repart vide à chaque fenêtre, et le −10 % n'est pas atteint à trois budgets voisins sur quatre (230 : −9,45 % ; 234 : −9,75 % ; 246 : −9,84 %) [mes.]. « Tient » est une estimation au bord ; « à la limite » est le bon mot.

La dérive, chiffrée couche par couche. En ligne, la part de cellules changées monte avec la profondeur, alors qu'elle est plate en boucle ouverte (tableau 7.10). Le juge sépare ce qui ne peut pas venir du biais seul : une cellule est « incohérente » si un expert entrant était absent du cache réel, ou un sortant présent, ce qui est impossible si ses logits sont ceux de P0. Au moins 39,3 % des cellules changées sont de la dérive certaine (6 207 sur 15 799), 0 en couche 0, 59 % en couche 23 ; 87,5 % d'entre elles ont une couche amont changée au même pas : la dérive passe surtout par le résidu du pas, la cascade de routage, et cinq fois moins par l'historique KV.

Verdict des juges : à la limite, pas une brique. Le levier existe, il vient du cache, et il est mesuré dans le moteur : −10 % de défauts pour une KL de 0,031. Mais aucune des trois conditions ne tient à sa borne défavorable, la porte pondérée ne mord pas avant la KL, et un seul budget, un seul texte de lecture, une seule passe le portent. Le levier ne peut pas servir de brique sans un second tirage (autre texte, autre budget) qui le confirme ; ce tirage est en cours ailleurs, et ce chapitre ne le cite pas.

7.5 P4 : le préchargement à confiance calibrée

L'idée. Le préchargement du projet est mort de sa *précision* : $p = 0{,}692$, +34 % d'octets, −11 % de vitesse (lot O, § 9.2), et le lot O écrivait qu'il ne reviendrait qu'avec $p > 0{,}9$. Deux emprunts : la prédiction conforme, qui garantit la précision d'un sous-ensemble de prédictions sans hypothèse de loi [103, 104] ; et le trafic d'arrière-plan qui n'occupe que la ressource libre et recule à la demande [105]. Le prédicteur est un raisonnement par cas, comme une jurisprudence : les 20 jetons antérieurs du même texte dont le chemin d'experts ressemble le plus votent pour les experts de la couche $\ell_0 + d$.

7.5.1 La garantie, posée

Un jeton t traverse L couches ; à la couche ℓ il demande k experts. Le cache (LRU global) laisse un ensemble de défauts. À la fin du routage de ℓ_0, le prédicteur propose pour $\ell = \ell_0 + d$ des candidats e de confiance $s(e) \in [0, 1]$ (part pondérée des 20 voisins qui portent e). Un *chargement* est un candidat absent du cache avec $s \geq \theta$; sa précision est $p(\theta) = P(e \in S \mid e \text{ chargé}, s \geq \theta)$, sa couverture $u(\theta)$ la part des défauts précédés d'un chargement juste.

Proposition 7.12 (Calibration conforme fendue, séquence fixe)**.** *Sur la moitié de calibration, soit $n(\theta)$ chargements de confiance $\geq \theta$, dont $x(\theta)$ justes, et $\mathrm{LB}(\theta) = B^{-1}(\delta;\, x, n - x + 1)$ la borne de Clopper-Pearson unilatérale [106] de niveau $1 - \delta$. On teste $H_\theta : p(\theta) < 0{,}9$ en descendant la grille depuis le plus haut seuil qui retient au moins 30 chargements, et l'on s'arrête au premier échec ; θ^* est le dernier seuil rejeté. Si les étiquettes sont échangeables, $P\big(p(\theta^*) < 0{,}9\big) \leq \delta$, sans correction de multiplicité.*

Esquisse. Pour chaque θ fixé, $P(\mathrm{LB}(\theta) \geq 0{,}9 \mid p(\theta) < 0{,}9) \leq \delta$ par construction de la borne de Clopper-Pearson. Dans une séquence fixe arrêtée au premier échec, une erreur de famille exige que la *première* hypothèse vraie rencontrée soit rejetée à tort, ce qui arrive avec probabilité au plus δ ; c'est le principe de « Learn then Test » [104]. ∎

Réglage déclaré : $\delta = 0{,}1$ (borne unilatérale à 90 %), grille au pas de 0,02 (`mesures/mimetisme/P4/00-seuils-declares.md`). L'hypothèse d'échangeabilité est fausse pour des jetons ordonnés (l'historique de voisins grandit) ; l'écart va dans le bon sens dans les quatre sessions (précision de test 0,919, 0,905, 0,947, 0,922 contre 0,911, 0,904, 0,918, 0,914 en calibration), sans le démontrer.

7.5.2 Le simulateur et le gain

Le temps d'une couche sans prédiction est $t_\ell = a + n_d\tau + \mathbb{1}[n_d > 0]\, q + c + o$, avec n_d défauts, $\tau = 4{,}1$ ms par lecture, $q = 1{,}0 + 0{,}85$ ms (copie, décodage), $a + c = 2{,}22$ ms de calcul et o un écart d'étalonnage (la base fait 460 ms par jeton). Une lecture de fond ne démarre que dans une fenêtre de calcul mais finit sa course : elle retarde la demande suivante de $\delta_r = \tau - (a + c) = 1{,}9$ ms. Une lecture juste économise τ ; une fausse coûte δ_r. Avec U chargements justes et C chargements sur la moitié de test :

$$\text{économie} = U\tau - C\,\delta_r, \qquad \text{gain} = \frac{\text{économie}}{T_{\text{base}} - \text{économie}}. \tag{7.9}$$

Pour le code : $U = 12\,770$, $C = 13\,892$, $T_{\text{base}} = 516{,}7\ \text{ms} \times 1\,271$ jetons ; économie $= 52\,357 - 26\,395 = 25\,962$ ms, gain $= 25\,962/(656\,726 - 25\,962) = 4{,}12$ % [calc.], contre 3,58 % simulé. Quand $\delta_r \to 0$ on retrouve la formule du tri, qui surestime de 2 à 2,4 fois. La formule du lot O, $f - (1-p)/p \times$ part disque avec $p = 0{,}918$, $f = 80/460 = 0{,}174$ et une part disque de 0,69, donne +11 % : elle suppose que toute lecture juste se cache entière, ce qui est faux quand la fenêtre de calcul (2,2 ms) est plus courte que la lecture (4,1 ms). Plafond (prédicteur parfait) : ×1,21 par le calcul du lot O.

Table 7.11 : P4, 120b, 326 emplacements (7 %), $d = 2$, bras « lire d'avance », moitié de test : la piste / le juge [mes.]. Gains sortis du simulateur, non validé.

Session	θ^*	précision	couverture	octets de plus	gain simulé
code	0,86 / 0,86	0,919 / 0,914	10,9 / 10,9 %	+0,84 / +0,91 %	+3,58 / +3,46 %
dialogue	0,80 / 0,82	0,905 / 0,918	16,9 / 15,1 %	+1,47 / +1,14 %	+4,96 / +4,39 %
prose-en	0,90 / 0,90	0,947 / 0,947	7,9 / 7,9 %	+0,41 / +0,42 %	+2,60 / +2,56 %
prose-fr	0,86 / 0,86	0,922 / 0,920	9,5 / 9,4 %	+0,72 / +0,74 %	+2,89 / +2,82 %

Données : `genome/mimetisme/P4/resultat.md` § 3 ; `genome/mimetisme/P4/JUGEMENT.md` § 1 ; `mesures/mimetisme/P4/juge/juge-120b.json`.

7.5.3 La mesure et le verdict

Seuil déclaré (commits `47339c2`, amendement `4732d35`, avant la première mesure de session) : au moins +5 % de vitesse simulée pour au plus 5 % d'octets de plus, sur 3 genres de lecture sur 4 et en génération, avec une précision réalisée d'au moins 0,85 ; mort sous +2 % ou sous 0,8 de précision. Règle 3 de l'amendement : tant que le simulateur n'est pas validé, le verdict ne peut pas dépasser « ouvert ».

Témoins. T0 (deux LRU égaux) tient. T2, prédicteur parfait : ×1,2247 (v1) puis ×1,1786 (v2), qui manque le ×1,21 ± 2 % de 0,6 point. **T3 manque** : un prédicteur synthétique de précision 0,692 devait redonner le −11 % du moteur ; le simulateur rend +5,2 puis +4,9 %. C'est le seul point mesuré du moteur, et le simulateur ne le retrouve pas : tous ses gains sont des ordres de grandeur, de signe d'erreur inconnu. La cassure par étiquettes mélangées tient : aucune calibration ne trouve de seuil, 7 sessions sur 7.

Ce qui tient : la garantie de précision, 0,905 à 0,947 (0,918 agrégée ; 0,922 chez le juge) pour +0,4 à +1,5 % d'octets, là où le préchargement tué avait 0,692 pour +34 %. Ce qui ne tient pas : le gain, +3,51 % en moyenne (+3,31 % chez le juge), 0 genre sur 4 à +5 % (tableau 7.11). Le bras « épingler sans rien lire » est mort (+0,09 %). La figure 7.8 montre le goulot : tenir 0,9 de précision coûte la couverture, 8 à 17 % des défauts, avec la même pente pour les quatre genres. Sur le 20b à 154 emplacements, les quatre genres passent +5 % (+5,05 à +9,10 %), non refaits par le juge.

Ce que le juge ajoute (`genome/mimetisme/P4/JUGEMENT.md` § 3). (1) La jambe « génération » repose sur les deux traces dégénérées de P1 (section 7.2.3) : le « 98,9 % de succès, rien à précharger » de prose-en est celui d'une boucle de blancs ; sur wiki, la calibration ne trouve aucun seuil parce que sa moitié de calibration (avant la boucle) a une précision de 0,73 à $\theta = 0{,}9$, alors que sa moitié de test (dans la boucle) atteint 0,979. Ni échec ni succès du mécanisme : une donnée non représentative. (2) Le gain dépend de l'ancrage à 460 ms : le code fait 92 défauts par jeton dans le LRU simulé, ce qui dépasse déjà 460 ms et rend l'écart d'étalonnage négatif ; ancré par session, le gain du code tombe à +0,42 % ; la moyenne de lecture va de +3,08 à +3,61 % selon l'ancrage, jamais plus d'un genre sur 4 au-dessus de +5 %. (3) Une phrase du code est inexacte (l'écart o est ajusté sur les quatre mêmes sessions), sans effet sur le gain.

Verdict du juge : ouvert, plutôt du côté de tombe. La conformalité règle le problème de précision qui avait tué le préchargement ; le goulot est maintenant la couverture. Seul arbitre valable : une passe ABBA du moteur avec la porte θ^*, qui validerait aussi le simulateur.

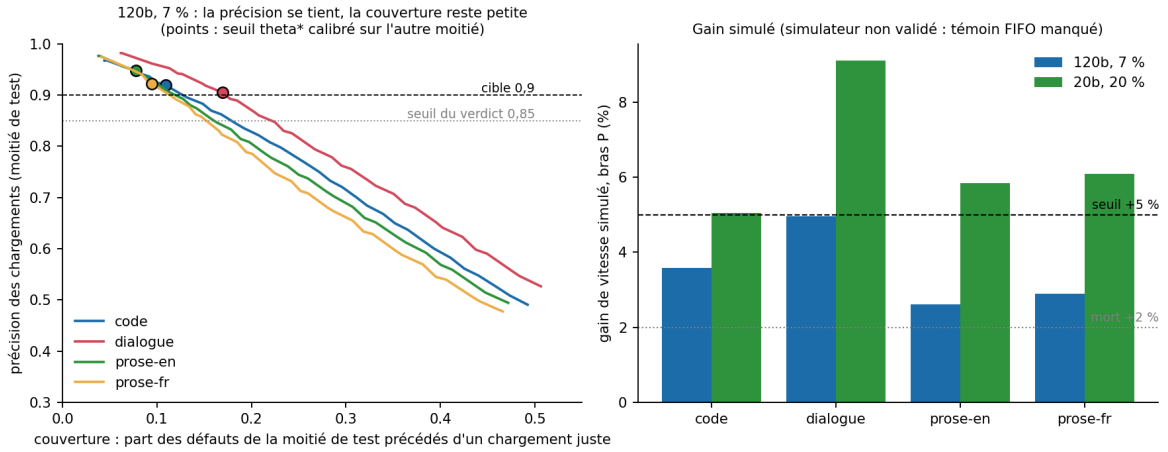


Figure 7.8 : La calibration tient la précision à 0,9 ; c'est la couverture qui reste petite, et avec elle le gain. [mes.] Gauche : courbes précision-couverture de la moitié de test, par session du 120b à 7 % ; points : θ^* calibré sur l'autre moitié. Droite : gain de vitesse simulé, bras « lire d'avance », contre le seuil de +5 % et la mort à +2 % ; simulateur non validé (témoin T3 manqué). (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P4/figures/p4-precision-couverture.csv`, `p4-gains-par-session.csv` ; script `outils/mimetisme/P4/p4_figures.py`.

7.6 P5 : le choix des symboles synonymes comme verre de spin

L'idée. La mutation synonyme du lot B (chapitre 10) range chaque poids d'expert dans l'un de huit niveaux en minimisant une erreur pondérée par un seul passage glouton ; c'est le seul gain avec perte qui ait passé la KL ($-0{,}0098 \pm 0{,}0033$). Le physicien y reconnaît l'énergie d'un verre de spin, et sait que le glouton s'y arrête dans une vallée ; le recuit simulé [107] et l'échange de répliques [108] traversent les barrières. C'est le domaine le plus éloigné du calcul de tous les candidats retenus, et l'éclaireur n'a trouvé aucun recuit ni échange de répliques appliqué à la quantification d'un grand modèle de langue.

7.6.1 Le problème posé

Une rangée de matrice d'expert compte $n = 2\,880$ poids w_j, par blocs de 32 qui partagent une échelle $s_b = 2^{e_b - 128}$ (MXFP4). La mutation synonyme range chaque poids dans l'un des $k = 8$ niveaux $\pm c_m s_j$ ($m = 0, \dots, 3$, $c = (0{,}4743;\ 2{,}8486;\ 6{,}8372;\ 12)$ en grandeurs doublées). Soit q le vecteur des niveaux choisis et $r = w - q$ le résidu. L'erreur de sortie, moyennée sur les entrées x_i de l'expert pondérées par le carré g_i^2 du poids du routeur, et le coût de code sont

$$E(q) = r^\top H r + \lambda' \sum_j \ell(q_j), \qquad H = \frac{\sum_i g_i^2\, x_i x_i^\top}{\sum_i g_i^2} + \delta\, \bar{m}\, I, \tag{7.10}$$

avec $\ell(m) = -\log_2 p_m$ la longueur de code du niveau m, $\bar{m}$ la moyenne de la diagonale de H sans amortissement et δ l'amortissement de GPTQ (valeur déployée $\delta = 1$).

Lemme 7.13 (Variation d'un seul poids). *Si l'on remplace le niveau de la coordonnée j, le résidu change de $\delta r = q_j - q'_j$ en position j seulement, et*

$$E(q') - E(q) = 2\,\delta r\,(Hr)_j + \delta r^2 H_{jj} + \lambda'\big(\ell(q'_j) - \ell(q_j)\big).$$

Démonstration. $r' = r + \delta r\, e_j$, donc $r'^\top H r' - r^\top H r = 2\,\delta r\, e_j^\top H r + \delta r^2\, e_j^\top H e_j$. ∎

Tenir le vecteur Hr à jour coûte n opérations par changement accepté ($Hr \mathrel{+}= \delta r\, H_{:,j}$) ; évaluer les $k-1$ candidats d'une coordonnée coûte $O(k)$. C'est ce qui rend les balayages praticables ; l'énergie tenue de façon incrémentale égale l'énergie recalculée de zéro à $1{,}3\times10^{-15}$ relatif [mes.].

Trois lectures du même objet. (i) *Le plus proche vecteur.* Avec $H=VV^{\top}$, V triangulaire supérieure, $r^{\top}Hr=\sum_i\big(\sum_{j\le i}V_{ji}r_j\big)^2$: le terme i ne dépend que de $r_1,\dots,r_i$. Choisir q_1 qui minimise le premier terme, puis q_2 sachant q_1, etc., est l'algorithme du plan le plus proche de Babai [109], et GPTQ lancé de l'arrière vers l'avant lui est identique [51, 110]. Chaque terme est minimisé exactement sachant le passé, jamais en regardant l'avenir. (ii) *Un verre de spin.* En écrivant le niveau de j par un état de Potts $\sigma_{jm}\in\{0,1\}$, $\sum_m\sigma_{jm}=1$, $r_j=w_j-s_j\sum_m c_m\sigma_{jm}$,

$$E_{\mathrm{err}}=\mathrm{cte}-2\sum_{j,m}h_{jm}\,\sigma_{jm}+\sum_{j,j',m,m'}J^{mm'}_{jj'}\,\sigma_{jm}\sigma_{j'm'},\qquad h_{jm}=s_jc_m(Hw)_j,\ J^{mm'}_{jj'}=H_{jj'}s_js_{j'}c_mc_{m'},\tag{7.11}$$

un modèle de Potts à huit états sur le graphe complet de 2 880 sites, dont les couplages ne sont pas indépendants : H est une somme de N termes de rang un (147 à 1 334 entrées de calibration par expert, d'après les journaux relus par le juge) plus l'amortissement. On cherche un vecteur *discret* presque orthogonal à N motifs x_i : c'est le cadre du perceptron binaire symétrique, dont Aubin, Perkins et Zdeborová conjecturent un paysage « gelé » où les algorithmes locaux butent [111]. Ce rapprochement est un cadre, pas un résultat : leurs motifs sont gaussiens indépendants et leurs poids ±1. (iii) *Une mesure de Gibbs*, $\pi_T(q)\propto e^{-E(q)/T}$.

Proposition 7.14 (Les deux dynamiques préservent π_T)**.** *(a) Le bain thermique sur une coordonnée, qui redessine q_j avec la probabilité $p_m\propto e^{-\Delta E_m/T}$ (ΔE_m du lemme 7.13), vérifie l'équilibre détaillé pour π_T ; à $T\to0$, c'est la descente cyclique de coordonnées. (b) Dans l'échange de répliques à températures $T_1<\cdots<T_{n_t}$, accepter l'échange des états de deux copies a,b avec la probabilité $\min\big(1,e^{(\beta_a-\beta_b)(E_a-E_b)}\big)$, $\beta=1/T$, préserve la loi produit $\prod_a\pi_{T_a}$.*

Démonstration. (a) La loi de tirage est π_T conditionnée aux autres coordonnées : $P(q\to q')=\pi_T(q')/Z_j$, où $Z_j=\sum_m\pi_T(q\text{ avec }j\leftarrow m)$ ne dépend pas du niveau courant de j ; donc $\pi_T(q)\,P(q\to q')=\pi_T(q)\pi_T(q')/Z_j$, symétrique en q,q'. (b) L'échange multiplie le poids de la loi produit par $\dfrac{\pi_{T_a}(q_b)\,\pi_{T_b}(q_a)}{\pi_{T_a}(q_a)\,\pi_{T_b}(q_b)}=e^{-\beta_aE_b-\beta_bE_a+\beta_aE_a+\beta_bE_b}=e^{(\beta_a-\beta_b)(E_a-E_b)}$; la règle de Metropolis sur ce rapport rétablit l'équilibre détaillé. ∎

Le recuit simulé est la même idée sans échange : une seule copie dont T décroît. Réglage de l'étape 1 : 8 températures de 0,02 à 1,0 fois l'erreur moyenne de Babai par poids, échange à chaque balayage, 40 balayages, puis trempe à $T=0$ du meilleur état visité ; toutes les copies partent de Babai.

7.6.2 Les témoins : le verre est réel à petite dimension

Seuils de l'étape 1 commités avant toute lecture de capture (`mesures/mimetisme/P5/00-seuil-declare.md`, commit `ab92f4e`) ; trois addendums, chacun commité une seconde avant son premier journal (vérifié par le juge).

À petite dimension et conditionnement fort, le verre est réel (tableau 7.12) : Babai perd jusqu'à ×33 sur le jouet, 44 % sur un bloc réel de 16 colonnes, et l'échange de répliques le résout. La figure 7.9 en montre une coupe : un bassin que le glouton n'atteint pas. La question de la piste est si ce verre existe encore à $n=2\,880$ et $\delta=1$, où l'amortissement l'adoucit.

Table 7.12 : P5 : les témoins, tous déclarés avant d'être lancés [mes.].

Témoin	Montage	Résultat
W1 force brute	10 poids, 5 niveaux ($5^{10} = 9\,765\,625$ états), 1 000 H corrélés	Babai manque l'optimum sur **847 / 1 000** (excès médian 106 %, jusqu'à ×33) : l'instrument peut tomber
W1, échange de répliques	~~200 balayages, 8 températures~~ ; 3 000 balayages, 12 températures	~~à l'optimum partout~~ : échoue sur 43 / 1 000 ; réglage final : **1 000 / 1 000**
W2 cassures	0 balayage ; températures gelées ; H permuté	Babai exactement ; optimum sur 107 / 847 seulement ; 2 / 847
W3 décodage par sphère	Fincke-Pohst [112], $n = 16$, bloc réel de H	échange = sphère sur 10 / 10 ; Babai non optimal sur 6 / 10 (8 à 44 % au-dessus)
W4	Babai générique contre `synonyme.py::choisir`	0 symbole différent sur 2 048

Données : `genome/mimetisme/P5/resultat.md` § 2 ; `mesures/mimetisme/P5/temoins-*.json`, `W3-sphere-n16.json`.

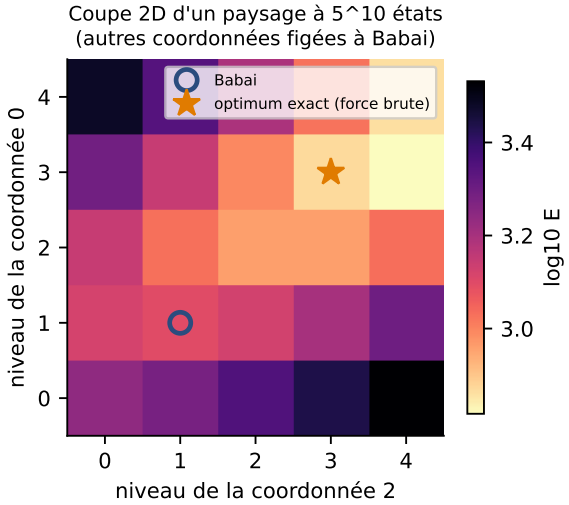


Figure 7.9 : Sur un jouet à dix poids, le glouton s'arrête dans une vallée que l'optimum exact quitte. [mes.] Coupe d'un paysage d'énergie à 5^{10} états (instance 464) selon deux coordonnées, les huit autres figées à la solution de Babai ; cercle : Babai ; étoile : niveaux que l'optimum exact donne à ces deux coordonnées. L'énergie d'une case de la coupe n'est pas celle de l'optimum (1 248 pour Babai, 37,9 pour l'optimum, toutes coordonnées libres). (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P5/figures/paysage-toy.csv` ; script `outils/mimetisme/P5/figures.py`.

Table 7.13 : P5 : gain d'évaluation tenue à l'écart de la descente cyclique ($\lambda = 0$) contre Babai, en %, $\delta = 1$, 64 rangées [mes.]. Le juge refait chaque instance à 4×10^{-14} point, puis sur des poids neufs (graine 2026).

Instance	N calib.	piste	juge, mêmes poids	juge, poids neufs	ΔH_0 (bit)
L0-E0	762	4,74	4,74	0,40	+0,0032
L0-E16	1 334	8,87	8,87	6,87	+0,0045
L8-E0	1 329	7,88	7,88	8,06	+0,0040
L8-E16	300	3,83	3,83	−0,27	+0,0033
L16-E0	330	3,23	3,23	6,87	+0,0040
L16-E16	147	1,43	1,43	3,90	+0,0025
L23-E0	303	7,55	7,55	12,07	+0,0035
L23-E16	239	5,43	5,43	4,07	+0,0035
médiane		5,09	5,09	5,47	
≥ 10 %		0 / 8	0 / 8	1 / 8	

Données : `genome/mimetisme/P5/JUGEMENT.md` § 2 ; `mesures/mimetisme/P5/etape1/gains-d1.csv`, `juge/`.

7.6.3 L'étape 1 : le gain se mesure et ne se transfère pas

Étape 1, sans fichier modèle (le verrou de la carte existait) : le H réel des couches 0, 8, 16, 23, construit depuis les captures d'entrées, avec des poids *synthétiques* gaussiens ; 8 instances (couches × experts 0 et 16), 64 rangées chacune, $\delta = 1$, débit égal à ±0,005 bit. Seuil : baisse de l'erreur d'évaluation tenue à l'écart d'au moins 10 % sur au moins 6 instances sur 8 ; mort sous 3 % en médiane.

Au meilleur des deux λ (à débit égal), la médiane monte à 5,46 % pour la descente cyclique ; la piste publie 6,8 % pour l'échange de répliques. Sur la calibration, la baisse est de 51,5 % (descente) et 54,6 % (échange) ; sur l'objectif optimisé, 6,8 et 7,8 % [mes.]. **Ouvert** : sous le seuil de succès, hors de la zone de mort (tableau 7.13 et figure 7.10). L'optimiseur est bon sur ce qu'on lui donne et ce qu'il apprend ne se transfère presque pas : les rapports objectif sur évaluation sont de 8 à 10.

Pourquoi : l'erreur déplacée dans le noyau de la calibration. Décomposons l'erreur dans la base propre (λ_k, u_k) de H_{entr} : $E = \sum_k e_k \, (u_k^\top H u_k)$, $e_k = \|\Delta W u_k\|^2$. Un optimiseur qui minimise $\sum_k \lambda_k e_k$ à budget d'erreur de poids à peu près fixé *déplace* l'erreur vers les directions de petit λ_k. Or le H de calibration a un rang au plus N, de 147 à 1 334 pour $n = 2\,880$; dans son noyau il vaut zéro et seul l'amortissement voit l'erreur, alors que l'évaluation y place 40 % (Babai) à 47 % (descente) de la sienne. Sur L8-E0 (32 rangées), l'erreur d'évaluation baisse de 38 % dans les trois premiers groupes de directions (0,00602 → 0,00373) et monte de 6 % dans les onze autres (0,01225 → 0,01303) [mes.] (figure 7.11). C'est le mécanisme d'un modèle appris sur N exemples en dimension $n > N$: l'énergie empirique est plate là où l'énergie vraie ne l'est pas.

L'amortissement et la chaleur. Erreur d'évaluation moyenne rapportée à Babai déployé : Babai 1,157, 1,040, 1,000, 1,044 aux δ = 0,1, 0,3, 1, 3 ; descente 1,191, 1,039, 0,946, 0,946 ; échange 1,196, 1,025, 0,935, 0,945 [mes.]. $\delta = 1$ est près de l'optimum de Babai ; l'optimiseur ne déplace pas le meilleur δ, et sur-ajuste en dessous. Un tableau qui compare à Babai *au même* δ rendait « 11 % » à $\delta = 3$: un artefact, barré par la piste elle-même. L'estimateur de Gibbs (un état tiré à T finie généraliserait mieux que le fondamental) a été essayé sur L8-E0 : −8,1, −9,0, −8,9, −7,9, −4,3, −0,3 % aux T/e_b = 0 à 1 ; un

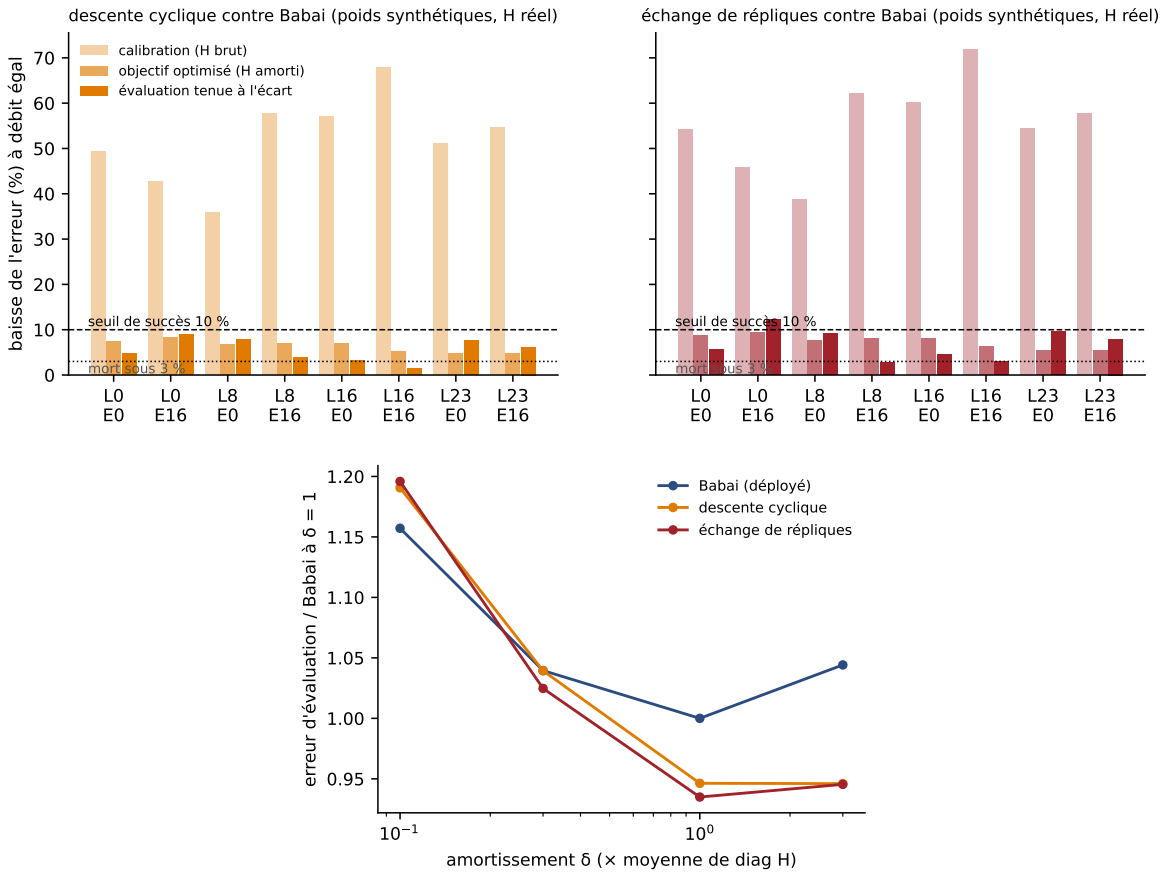


Figure 7.10 : L'optimiseur retire la moitié de l'erreur de calibration et vingt fois moins de l'erreur tenue à l'écart. [mes.] Haut : baisse de l'erreur contre Babai, par instance, sur la calibration, sur l'objectif optimisé et sur l'évaluation tenue à l'écart ; descente cyclique (gauche) et échange de répliques (droite), seuils de succès (10 %) et de mort (3 %). Les barres de l'échange de répliques sont celles de la piste ; le juge ne les reproduit qu'en classe (médiane 4,8 % contre 6,8 %). Bas : erreur d'évaluation moyenne contre l'amortissement δ, rapportée à Babai déployé ($\delta = 1$). Poids synthétiques (étape 1). (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P5/etape1/gains-d1.csv`, `etape1/table-delta.csv` ; script `outils/mimetisme/P5/figures.py`.

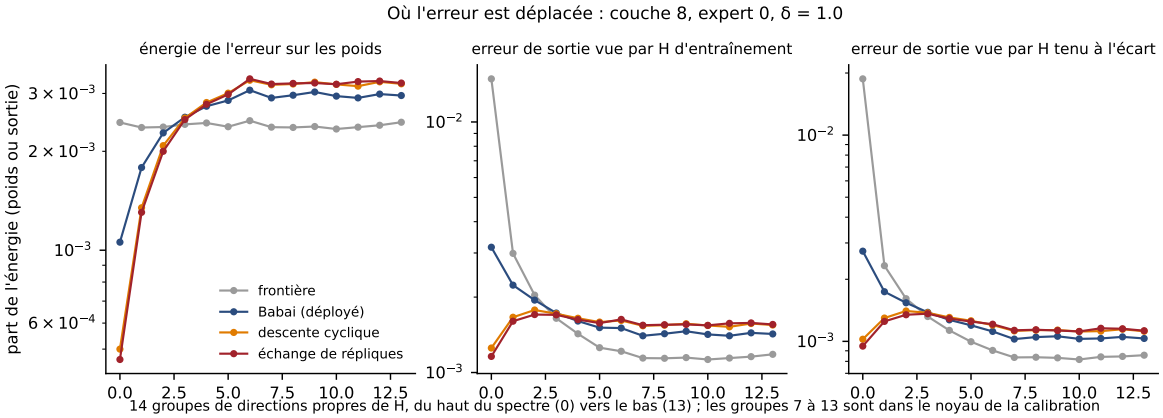


Figure 7.11 : L'optimiseur pousse l'erreur vers les directions que la calibration ne voit pas, et l'évaluation l'y retrouve. [mes.] Couche 8, expert 0, $\delta = 1$: part de l'énergie par groupe de directions propres de H, du haut du spectre (0) vers le bas (13), sur les poids (gauche), vue par H d'entraînement (milieu), vue par H tenu à l'écart (droite) ; frontière, Babai, descente cyclique, échange de répliques. Les groupes 7 à 13 sont dans le noyau de la calibration. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P5/figures/spectre-erreur.csv` ; script `outils/mimetisme/P5/figures.py`.

plateau dans le bruit, puis la dégradation ; pas de gain de généralisation par la chaleur.

En bits. À haute résolution, $D \propto 2^{-2R}$, un rapport d'erreur ρ vaut $\Delta R = -\frac{1}{2}\log_2 \rho$: $\rho = 0{,}935 \to 0{,}049$ bit par poids, $0{,}946 \to 0{,}040$, $0{,}866 \to 0{,}104$, contre $0{,}80 \to 0{,}161$ visés [calc.], vérifiés par le juge. Le coût : la descente prend environ 0,1 s par rangée contre 6 ms pour Babai (×15 à ×30), l'échange environ quinze fois plus.

7.6.4 Le mécanisme annoncé, et ce que le juge en dit

La piste a cherché la cause du non-transfert dans la finitude de l'échantillon de calibration et a déclaré un test (addendum 3, commit `d958a46`) : entraîner sur la calibration plus les fenêtres 0 à 4 de l'évaluation, juger sur les fenêtres 5 à 9 ; « soutenu » si le gain médian de la descente sur le test monte d'au moins 2 points et monte sur au moins 6 instances sur 8. Elle a écrit : « +5,9 à +7,3 % (… +2,6 points ; ~~seuil soutenu tenu~~) ». Le juge (`genome/mimetisme/P5/JUGEMENT.md` § 4) relit la déclaration : les deux médianes publiées côte à côte font +1,41 point, ce qui tombe dans « non tranché » ; le +2,6 est la médiane des montées par instance, une autre grandeur que la déclaration ne fixait pas. Puis il isole N par un test déclaré avant (J5) : *retirer* la moitié des entrées de calibration, même source, même test. Diviser N par 2,3 en médiane ne baisse le gain médian que de 0,43 point, sous le seuil de 0,5 : **non soutenu**. Ce que l'addendum 3 a mesuré mêle « plus de N » et « des entrées prises dans le fichier d'évaluation, voisines du test ». Ce qui est net : N compte pour Babai lui-même (diviser N par 2 lui coûte 6 % d'erreur d'évaluation en médiane).

L'échange de répliques du juge reproduit la baisse de l'objectif de la piste à 0,1 point près sur chaque instance, mais pas son gain d'évaluation : médiane 4,8 %, sous sa propre descente (5,1 %), 0 / 8 au-dessus de 10 % (le 12,3 % de L0-E16 devient 6,3 %). À qualité d'objectif égale, deux échanges tombent sur des états dont l'erreur tenue à l'écart diffère de 2 à 6 points : le « +1,4 point au-dessus de la descente » est du bruit d'optimiseur.

Verdict du juge : ouvert ; le mécanisme ne tient pas au seuil. Le chiffre principal tient pour la descente cyclique, à l'identique et sur des poids neufs ; celui de l'échange ne se reproduit qu'en classe. L'étape 2 (poids réels) n'est pas autorisée par la règle et n'a pas tourné. Le levier « nombre d'entrées de calibration » reste une hypothèse, à mesurer à source fixe et à fenêtres lointaines.

7.7 P6 : la fidélité décidée par appel

L'idée. Les poids de mélange du routeur sont un softmax sur quatre scores, comme les nombres d'occupation de quatre niveaux d'un système physique ; une occupation faible pèse peu dans la sortie. L'image physique (une surface de Fermi) est une analogie : ce qu'elle propose est de servir en *rapide* (le compagnon compressé du lot B) l'expert appelé avec un petit poids, au lieu de le retirer. L'idée de dégrader selon l'importance n'est pas neuve [42, 113], ni celle de raffiner en précision mixte [114] ; ce que P6 devait fournir au projet est la courbe qui manque à tout le lot B, la KL en fonction de la part f des défauts servis en rapide, dont seules les extrémités étaient mesurées (0,0044 et 0,145).

7.7.1 Le modèle et ses grandeurs

Un accès $a = (t, \ell, s)$ appelle l'expert x_a avec le poids w_a ($\sum_s w = 1$). Un *séjour* σ d'un expert dans le cache va de son entrée (un défaut) à son éviction ; $A(\sigma)$ est l'ensemble de ses accès. Le portillon fixe le niveau d'un séjour à son entrée : $\varphi(\sigma) = F$ (rapide) si et seulement si $w_{a_0(\sigma)} < \theta$. Un séjour coûte $\rho = 0{,}718$ expert exact s'il est rapide, 1 sinon. L'énergie perturbée de la règle « accepter » et l'énergie totale sont

$$\mathscr{E} = \sum_{\sigma} \mathbb{1}[\varphi(\sigma) = F] \sum_{a \in A(\sigma)} w_a^2, \qquad \mathscr{E}_{\text{tout}} = \sum_{a} w_a^2. \tag{7.12}$$

Hypothèse 7.15 (Petite perturbation)**.** L'écart de sortie à l'accès a est $w_a\,\varepsilon_x$, avec ε_x un écart d'expert de variance commune ; la variance d'écart totale est alors proportionnelle à l'énergie. Ce n'est pas la KL.

Les octets économisés sont $S = (1 - \rho)\,f$, f la part des séjours rapides ; le tout-rapide donne $S_{\text{tout}} = 0{,}282$ et $\mathscr{E} = \mathscr{E}_{\text{tout}}$. La grandeur de la piste est l'énergie par octet économisé, rapportée à celle du tout-rapide :

$$r(\theta) = \frac{\mathscr{E}/\mathscr{E}_{\text{tout}}}{S/S_{\text{tout}}} = \frac{\mathscr{E}/\mathscr{E}_{\text{tout}}}{f}. \tag{7.13}$$

Proposition 7.16 (La cassure par le hasard)**.** *Si φ est tiré indépendamment des poids avec $P(F) = f$, alors $\mathbb{E}[\mathscr{E}] = f\,\mathscr{E}_{tout}$ et $\mathbb{E}[r] = 1$.*

Démonstration. $\mathscr{E} = \sum_\sigma \varphi(\sigma)\,e(\sigma)$, $e(\sigma) = \sum_{A(\sigma)} w^2$ indépendant de $\varphi(\sigma)$; donc $\mathbb{E}[\mathscr{E}] = f \sum_\sigma e(\sigma) = f\,\mathscr{E}_{\text{tout}}$. ∎

Mesuré : 1,0001 ± 0,023 sur 50 tirages et 7 traces [mes.].

Proposition 7.17 (La porte seule)**.** *Soit $p(\theta) = P(w < \theta)$ et $\hat{\mathscr{E}}(\theta) = \mathbb{E}[w^2;\, w < \theta]/\mathbb{E}[w^2]$. Pour un comptage sans portage (chaque accès porte sa propre étiquette) et une sélection indépendante de l'état du cache, $r_0(\theta) = \hat{\mathscr{E}}(\theta)/p(\theta) = \mathbb{E}[w^2 \mid w < \theta]/\mathbb{E}[w^2] < 1$ dès que la loi de w n'est pas dégénérée.*

Démonstration. $\mathbb{E}[w^2;\, w < \theta] = p(\theta)\,\mathbb{E}[w^2 \mid w < \theta]$; comme w^2 croît sur $[0, 1]$, $\mathbb{E}[w^2 \mid w < \theta] < \mathbb{E}[w^2]$ si $P(w \geq \theta) > 0$ et $P(w < \theta) > 0$. ∎

Table 7.14 : P6, décodage du 120b, $\theta = 0{,}20$, 323 emplacements, LFU de demi-vie 8, $\rho = 0{,}718$: octets économisés S et énergie par octet r [mes.]. Reproduit par le juge à moins de 10^{-4} en S et 10^{-3} en r.

Règle	prose-en S / r	wiki S / r	cumul S / r	tient ?
accepter	10,10 % / 0,725	**7,78 %** / 0,565	8,55 % / 0,688	non
remonter à 0,30	8,22 % / 0,461	6,95 % / 0,391	7,37 % / 0,442	non
remonter à 0,40	9,29 % / 0,573	7,51 % / 0,451	8,10 % / 0,539	non
remonter à 0,50	9,77 % / 0,613	7,74 % / 0,553	8,41 % / 0,612	non
clairvoyant (borne)	8,85 % / 0,140	7,19 % / 0,227	7,74 % / 0,185	hors verdict

Données : `genome/mimetisme/P6/resultat.md` § 2 ; `genome/mimetisme/P6/JUGEMENT.md` § 1 ; `mesures/mimetisme/P6/`, `juge/juge.json`.

Mesuré : $r_0(0{,}20) = 0{,}414$, $r_0(0{,}25) = 0{,}614$ [mes.]. La piste en concluait : ~~« les défauts ne sont pas corrélés aux petits poids ; le gain de la porte vient de la loi de w, non du cache »~~. C'est faux, dit le juge (`genome/mimetisme/P6/JUGEMENT.md` § 4.2) : les défauts se répartissent en 19,4, 23,1, 26,8 et 30,6 % sur les rangs 0 à 3, et 29,6 % des défauts sont sous $\theta = 0{,}20$, contre 24,9 % quand on détruit le rang dans le groupe ; l'état du cache donne à la porte environ 19 % d'octets économisés de plus à θ égal ($29{,}6/24{,}9 = 1{,}19$). Le test de la piste (mélange des vecteurs par couche) gardait le rang : la corrélation passe dans S, pas dans r, qui n'en voit rien.

Le portage. Une étiquette « rapide » fixée à l'entrée vaut pour tout le séjour : $\mathscr{E} = \mathscr{E}_{\text{porte}} + \mathscr{E}_{\text{suite}}$. À $\theta = 0{,}20$ sur les générations du 120b, $\mathscr{E}_{\text{porte}}/\mathscr{E}_{\text{tout}} = 0{,}0476$ et $\mathscr{E}_{\text{suite}}/\mathscr{E}_{\text{tout}} = 0{,}1609$: la suite porte 77 % de l'énergie [mes.]. Avec $f = 0{,}0855/0{,}282 = 0{,}303$, $r = 0{,}2085/0{,}303 = 0{,}688$ [calc.]. Le poids d'entrée prédit mal la suite (corrélation 0,34 et 0,25). La *borne clairvoyante*, qui connaîtrait le plus grand poids que recevra l'expert pendant son séjour ($\varphi^* = F \iff \max_{A(\sigma)} w < \theta$), atteint $r^* = 0{,}185$: le portage coûte un facteur 3,7. La règle « remonter à θ' » relit l'exact entier au premier succès de poids $\geq \theta'$, parce que le fichier rapide n'est pas emboîté dans l'exact.

Variante B, le préremplissage. Dans un lot de n_β jetons, chaque clé $u = (\text{lot}, \ell, x)$ est lue une fois. On lit en rapide les f clés de plus petit score, avec trois scores : le maximum $m_u = \max_j w_j$ (règle A), la masse $M_u = \sum_j w_j$ (règle B), l'énergie $e_u = \sum_j w_j^2$ (règle C).

Proposition 7.18 (L'oracle de l'énergie)**.** *À f fixé, la règle C minimise $\mathscr{E}$ parmi toutes les parties de $f\,|U|$ clés ; elle borne donc A et B par en dessous.*

Démonstration. $\mathscr{E} = \sum_{u \in F} e_u$ est additive ; la partie de $f|U|$ clés de plus petite somme est celle des $f|U|$ plus petits termes. ∎

Le maximum est un mauvais substitut de e_u parce qu'il ignore le nombre d'appels ($m_u^2 \leq e_u \leq n_u m_u^2$) ; la masse $M_u = n_u \bar{w}_u$ les compte.

7.7.2 La mesure, le verdict et ce que le juge corrige

Seuils (`mesures/mimetisme/P6/00-seuils-declares.md`, commit `4d2b915`, amendement `c206831`, avant toute lecture de trace) : décodage, « tient » si sur *chacune* des deux générations du 120b $r \leq 0{,}70$ avec au moins 8 % d'octets économisés à $\theta = 0{,}20$; variante B, « tient » si $r \leq 0{,}70$ à 8 % sur 3 genres sur 4 et $r_B \leq 0{,}9\, r_A$.

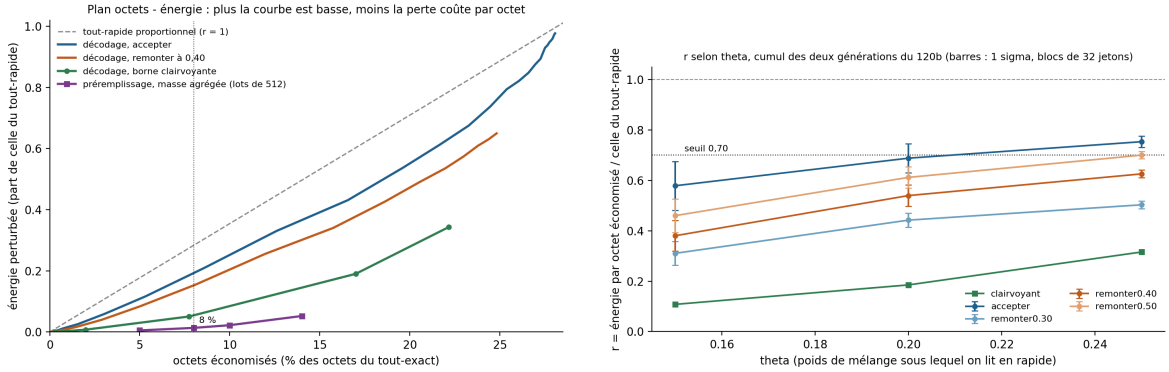


Figure 7.12 : Une porte sur le poids d'appel coûte moins d'énergie par octet que le tout-rapide, et la masse agrégée du préremplissage presque rien ; mais l'énergie n'est pas la KL. [mes.] Gauche : énergie perturbée contre octets économisés (décodage du 120b : accepter, remonter à 0,40, borne clairvoyante ; préremplissage : masse agrégée par lots de 512 ; tirets : le tout-rapide proportionnel, $r = 1$). Droite : r selon θ, cumul des deux générations, barres à 1σ (blocs de 32 jetons), seuil 0,70. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/mimetisme/P6/courbe-plan-decodage.csv`, `courbe-plan-prefill-B.csv` ; script `outils/mimetisme/P6/figures.py`.

Décodage : ouvert. Aucune règle ne passe 8 % sur wiki (7,78 ± 0,38 % pour « accepter »), aucune n'a $r > 0{,}90$ (tableau 7.14). Au cumul des deux traces, « accepter » et « remonter à 0,40 » passeraient, mais la règle déclarée voulait chaque trace : verdict **ouvert**, et la piste n'a pas déplacé son seuil. Le juge ajoute que « ouvert » tient au choix d'un θ fixe plutôt qu'au mécanisme : les poids se tassent autour de 0,2, et S passe de 2,2 % ($\theta = 0{,}15$) à 8,5 % (0,20) puis 16 % (0,25) ; à $S = 8$ % exact, trace par trace, r vaut 0,669 et 0,573, tous deux sous 0,70 (exploratoire, hors seuil).

Variante B : tient, mais ne pouvait presque pas tomber. À 8 % d'octets du préremplissage, $r_B = 0{,}039$ à 0,060 (seuil 0,70) et $r_B/r_A = 0{,}61$ à 0,66 sur les quatre genres du 120b, et r_B de 0,030 à 0,074 au 20b ; le tirage au hasard rend $r = 1{,}00$ [mes.]. Le juge mesure l'oracle de la proposition 7.18 : r_B est à 4 % de lui sur les quatre genres (et à 1,5 à 4,3 % sur huit genres, 120b et 20b). Le seuil « tombe si $r_B \geq r_A$ » ne pouvait guère tomber, puisque le maximum est un moins bon substitut de l'énergie que la somme quelle que soit la loi des poids. À l'étape 0, la masse dit seulement que l'énergie d'un lot se concentre dans les experts souvent appelés ; elle ne dit rien de la KL.

La courbe KL(f), posée comme hypothèse. L'étape 0, celle du plan octets–énergie de la figure 7.12, ne peut qu'éliminer. Sous l'hypothèse linéaire $\mathrm{KL}(f) - \mathrm{KL}(0) = (\mathrm{KL}(1) - \mathrm{KL}(0))\,\mathscr{E}(f)/\mathscr{E}_{\text{tout}}$, avec les extrémités mesurées au 20b (0,0044 et 0,1448), 8 % d'octets du préremplissage coûteraient 0,002 de KL et 14 % de 0,006 à 0,009. Ce n'est pas un résultat : 96,5 % de la KL du mode rapide vient de bascules de routage, non linéaires, et la convexité pourrait aller dans les deux sens. L'étape 1, qui mesurerait les points (f, KL) au 20b, n'a pas été lancée (verrou de la carte, 8,31 Go à lire) ; le juge demande qu'elle garde le maximum, le hasard *et* l'oracle de l'énergie comme comparaisons à octets égaux : si la KL suit l'énergie, masse et oracle donnent la même KL.

Verdict du juge. Décodage du 120b : **ouvert**, reproduit. Variante B : « tient » à l'étape 0, reproduit, mais quasi l'oracle de sa propre mesure : rien n'est dit de la qualité. Le portillon économise 7,8 à 10,1 % des octets lus en génération pour 16 à 26 % de l'énergie du tout-rapide ; seule l'étape 1 dira ce que vaut l'énergie.

Table 7.15 : La carte du monde vers le pipeline : les concepts empruntés, leur étage, leur verdict (`genome/mimetisme/tri.md` § 6, verdicts des pistes mis à jour par leurs juges).

Concept du monde	Domaine	Étage	Verdict
Réservoir lent et rapide (Hasselmann) ; oubli en loi de puissance (ACT-R)	climat ; psychologie	K	P1 : ouvert
Répliques sismiques (Omori-Utsu), processus de Hawkes	sismologie	K	fermé (E1)
Marées, phase périodique de la couche	géophysique	K	petit gain, comparateur de P1
Hiérarchie exclusive (démotion)	stockage	K	P2 : ouvert
Cache compressé	architecture	K	dominé par P2
Nucléation, barrière de germe	thermodynamique	K	fermé (E3)
Allocation par utilité marginale (Charnov, hydraulique, METRIC)	écologie, hydrologie, logistique	K	fermé
Admission par fréquence (TinyLFU) ; segments à listes fantômes	systèmes	K	fermé en génération ; non mesuré
Faille collante ; substitution d'article, deux choix	sismologie ; commerce, files	R	P3 : à la limite
Hypersurface tropicale, marge du top-k	mathématiques	R	garde fermée (chapitre 12) ; bande dans P3
Lissage de Maslov	mathématiques	R	fermé
Prédiction conforme ; jurisprudence	statistique ; droit	L	P4 : ouvert, plutôt tombe
Circuit olfactif de la mouche	neurosciences	L	variante de P4
Trafic d'arrière-plan qui cède (TCP Nice, LEDBAT)	réseaux	L	dans P4, non mesuré seul
Parareal, COLA, onde P	simulation, séismes	L	fermé (aucun octet)
Vérification groupée ; codes à effacement	information	L	fermés
Contrôle à tampon (BOLA)	vidéo	L	non mesuré, après P6
Occupation de Fermi, portillon ; fovéation, triage, queue de Pareto	physique ; vision, médecine, économie	R, L	P6 : ouvert (décodage), étape 0 (B)
Raffinement itératif en précision mixte	calcul scientifique	Q, L	dans P6
Verre de spin, recuit, échange de répliques	physique des désordres	Q	P5 : ouvert
Remplissage inverse par l'usage	débit-distorsion	Q	non retenu, à greffer
Réseaux de tenseurs, holographie, Landauer	matière condensée	F, Q	défavorable, fermé
Graine, génome, duplication, hebbien, spectres	biologie	F, Q	fermés (chapitres 4 à 6, chapitre 13)
Arrêt séquentiel de groupe	essais cliniques	J	instrument

7.8 La carte du monde vers le pipeline

Le tri range chaque concept emprunté sur l'étage du pipeline qu'il touche : **F** le format et le codage entropique, **Q** le choix des symboles, **K** le cache d'experts, **R** le routage, **L** la lecture, l'ordonnancement et le préchargement, **J** le juge et la méthode. Le tableau 7.15 en donne l'essentiel, avec le verdict d'aujourd'hui (la figure III-6.17 qui la dessinerait n'est pas tracée). Le tableau 7.16 rassemble les six pistes devant leurs juges.

Table 7.16 : Les six pistes devant leurs juges.

Piste	Chiffre principal (juge)	Verdict du juge	Ce qui manque pour trancher
P1	LFU(32) − LRU : −0,0176 au seuil ; +0,0185 sur la réponse réelle	ouvert (« tombe » mécanique)	4 générations du 120b arrêtées à `<|return|>`
P2	part disque −28 et −51 % ; descente 11,5 et 27,2 % du jeton	ouvert	recouvrement de la descente (ABBA moteur), contre l'inclusif à $V + R$
P3	−10,03 % de défauts, KL 0,0314 en ligne	à la limite	second tirage (autre texte, autre budget), en cours ailleurs
P4	précision 0,92 ; gain simulé +3,3 %	ouvert, plutôt tombe	ABBA moteur avec la porte θ^*
P5	descente −5,1 % d'erreur tenue à l'écart	ouvert ; mécanisme non tenu	test de N à source fixe ; poids réels (étape 2)
P6	décodage $r = 0{,}69$; préremplissage $r_B = 0{,}04$ à $0{,}06$	ouvert ; B tient à l'étape 0	KL de bout en bout au 20b (étape 1)

7.9 Ce qui pourrait être faux

Ce qui pourrait être faux

- **Deux traces de génération pour le 120b, et elles sont dégénérées.** P1, P2, P4 et P6 jugent la génération sur les deux mêmes traces de 256 jetons ; la moitié de prose-en est faite de jetons produits après la fin du message, et wiki est une complétion brute qui se répète. Les juges de P1 et de P4 l'ont dit ; ceux de P2 et de P6 ne l'ont pas relevé, et leurs chiffres de génération portent la même réserve.
- **Les taux de succès simulés dépassent ceux du moteur** (0,74 contre 0,59 sur prose-en) : tout gain de cache simulé (P1, P2, P4, P6) est une borne haute avant l'ABBA du moteur. Les tailles d'experts y sont égales, la RAM tiède absente.
- **P3 ne repose que sur ses jugements** : un seul budget porte le « tient », un seul texte de lecture, une seule passe en ligne ; le second tirage, en cours ailleurs, peut le confirmer comme le démentir, et ce chapitre ne le connaît pas.
- **Le simulateur de P4 n'est pas validé** (T3 manqué) : ses gains sont des ordres de grandeur de signe d'erreur inconnu.
- **P5 est mesuré sur des poids synthétiques** gaussiens, sur 64 rangées et le seul H d'entrée de gate et up ; les vrais poids ont des colonnes géantes corrélées au H, et l'écart Babai–optimum peut y être plus grand ou plus petit. Aucune affirmation sur la qualité du texte.
- **P6 est mesuré en énergie**, pas en KL ; et son « tient » au préremplissage ne pouvait presque pas tomber.
- **La proposition sur l'inclusif** (proposition 7.10) est posée par la rédaction sous une hypothèse sur la règle du moteur (rafraîchissement de la RAM à chaque accès) lue dans le code par le juge, pas exécutée ; elle explique l'égalité mesurée par le juge, elle ne la remplace pas.
- **Les sources des pistes** : le tri n'a rouvert aucune source externe ; la rédaction a relu le 2026-10-29 les notices de celles que ce chapitre cite (Crossref, arXiv, USENIX, RFC), pas les articles. Les éclaireurs cités par numéro (« vivant #1 ») ne sont connus ici que par leurs rapports.
- **Le classement du tri n'est pas mesuré** : « gain × nouveauté × faisabilité » est un jugement d'ordre ; deux pistes écartées (le remplissage inverse par l'usage, BOLA) pourraient valoir

plus que P4 ou P5 une fois chiffrées.

– **Les figures sont hors du style commun** : tracées par chaque piste, avec leurs propres titres (dont celui de la figure 7.1, que le juge corrige), leurs décimales à point et un titre de panneau qui déborde ; elles sont incluses telles quelles, et les nombres qu'on y lit ont été vérifiés contre les jugements, pas redessinés.

Chapitre 8

La synthèse de la ronde : une frontière commune, deux parts de la KL

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, quatre équipes et quatre livrables, chacun jugé à sa propre mesure : l'éclaireur (chapitre 3) avait sa frontière à 2,644 1 bits par poids et sa mutation synonyme, l'équipe du génome (chapitre 4) sa recette S^*, la duplication (chapitre 5) sa parenté d'experts, l'équipe du codage prédictif et de l'élagage (chapitre 6) ses partitions, son hebbien et son codon synonyme du zéro. Le format `.trc` v1.0 du Tome II était exact au bit, à 3,843 4 bits par poids d'expert (A_flux) ; le moteur rendait au bit les logits du codage brut. Trois passes de KL existaient, sous trois protocoles différents. **Ce chapitre établit** une frontière commune des quatre équipes sur un seul juge, la divergence de Kullback-Leibler sur le texte, en rapport à la base du même protocole et en différence appariée ; la raison de chaque échec, rattachée à l'une de trois propriétés mesurées des poids ; une décomposition de la KL en une part lisse, quadratique et posée ici en entier, et une queue de bascules du routeur ; le cadre tropical qui fait des régions du routeur des polyèdres ; et le cahier des deux lots de construction, avec ses trois énoncés (proposition A1, lemmes A2 et B2) démontrés. **Il reste** la construction elle-même (lot A au chapitre 9, lot B au chapitre 10, leur juge au chapitre 11), la garde tropicale (chapitre 12) et les spectres (chapitre 13), que ce chapitre désigne sans les mesurer. Sources : `genome/05-synthese.md`, `genome/construction-synthese-pour-le-memoire.md`, les fichiers de `mesures/genome/construction/synthese/k10/` écrits par `outils/genome/synthese/frontiere.py --k10`, les journaux du juge de l'éclaireur (`mesures/genome/etat-de-l-art/journaux/kl-*-juge.json`) et de la passe K10 (`mesures/genome/genome/kl/comparer10-base-contre-recette.json`).

En bref

Quatre équipes ont essayé de comprimer les poids de gpt-oss en y cherchant une structure : une graine, des motifs, une parenté, une prédiction. Placées sur un seul axe de qualité, la KL du modèle entier sur un texte réel, presque toutes tombent, et elles tombent pour l'une de trois raisons mesurées : une fois connue l'échelle de son bloc, un poids est du bruit ; quelques colonnes géantes portent ce qui reste de structure ; et l'erreur de sortie d'un expert ne dit pas ce que coûtera cette erreur au texte. Les trois mécanismes qui gagnent ont un air de famille : ce sont des *synonymes*. L'image vient du code génétique, où plusieurs codons donnent le même acide aminé ; elle est une analogie, pas une preuve. Ici, le signe d'un zéro est un codon que le calcul ne lit pas, et on le fond ; la classe de la colonne d'entrée est une information que le décodeur a déjà ; et la mutation synonyme change des codes là où les entrées de l'expert ne regardent pas. Pourquoi l'erreur d'expert trompe : la KL a deux parts. Une part lisse, petite et quadratique, que l'erreur suit ; et une queue, faite des positions où une petite erreur fait basculer le choix des experts

d'une couche suivante, et qui porte l'essentiel de la moyenne. Le routeur découpe l'espace des états en polyèdres ; la queue vit sur leurs faces. Le chapitre finit par le cahier de construction : deux lots, leurs critères déclarés avant, et les trois preuves sur lesquelles ils reposent.

8.1 Un seul juge, et pourquoi des rapports

Le juge. Chaque équipe de la ronde avait commencé par sa propre mesure : l'erreur relative sur les poids, puis l'erreur de sortie d'un expert sur des entrées réelles. Chacune a désigné des gagnants que la mesure suivante a fait tomber (section 8.3). Le juge qui reste est celui du lot R du Tome II : la divergence de Kullback-Leibler [11] de la loi du jeton suivant donnée par le modèle modifié, contre la référence f64, position par position, sur les fenêtres de wiki.test.

Définition 8.1 (La KL d'une position, sa moyenne)**.** À une position t, soient p_t la loi du jeton suivant rendue par la référence f64 et q_t celle du modèle à juger. La KL de la position est $\mathrm{KL}_t = \sum_i p_{t,i} \log(p_{t,i}/q_{t,i})$ (en nats ; l'instrument `juge k1` ne somme que les i tels que $\log p_{t,i} > -16$). La KL d'une passe est la moyenne $\overline{\mathrm{KL}} = \frac{1}{n}\sum_t \mathrm{KL}_t$ sur les n positions notées.

Trois protocoles. Les trois passes de la ronde ne remplacent pas les mêmes experts ni ne notent les mêmes positions (tableau 8.1). Une KL absolue de P3, qui remplace quatre fois plus d'experts, ne se compare pas à une KL de P1 : la base va de 0,0206 à 0,0636 selon le protocole.

Table 8.1 : Les protocoles de KL de la ronde [mes.]. La KL de la base est celle de la frontière de l'éclaireur substituée aux mêmes experts ; ± un écart-type naïf, celui de l'instrument.

Prot.	Équipe	Experts remplacés	Positions notées	KL de la base
P0	(sans perte)	—	logits identiques au bit	0 exactement
P1	éclaireur	32 sur 768 (couches 0, 8, 16, 23 ; experts 0, 4, …, 28)	2 550 (10 fenêtres)	0,0360 ± 0,0029
P2	genome	8 sur 768 (couches 0, 8, 16, 23 ; experts 0, 16)	765 (3 fenêtres)	0,0226 ± 0,0031
P2-10	genome (K10)	les mêmes 8	2 550 (10 fenêtres)	0,0206 ± 0,0021
P3	prédictif-élagage	128 sur 768 (les 12 tenseurs entiers)	1 275 (fenêtres 5 à 9)	0,0636 ± 0,0050

Données : `mesures/genome/construction/synthese/k10/frontiere.csv` (colonnes `kl_base_protocole`, `kl_incertitude`) ; `genome/05-synthese.md` § 1.1.

Ce qui se compare d'un protocole à l'autre est le *rapport* à la base du même protocole ; ce qui se compare à l'intérieur d'un protocole, mieux encore, est la *différence appariée*.

Définition 8.2 (Différence appariée, règle du lot R)**.** Pour deux modèles A (la base) et B jugés aux mêmes positions contre la même vérité, soit $d_t = \mathrm{KL}_t^B - \mathrm{KL}_t^A$ et $\bar{d}$ sa moyenne. Avec $\sigma(\bar{d})$ l'incertitude de $\bar{d}$, B est *meilleur* si $\bar{d} + 2\sigma(\bar{d}) < 0$, *égal* si $|\bar{d}| \le 2\sigma(\bar{d})$, *pire* sinon.

Deux incertitudes pour $\bar{d}$. L'estimateur naïf traite les n positions comme indépendantes : $\sigma_{\text{naïf}} = s_d/\sqrt{n}$, avec s_d l'écart-type empirique des d_t. Or les positions d'une même fenêtre partagent leur passé, donc leurs bascules de routage (section 8.4) : leurs d_t sont corrélés. Le second estimateur, que

`frontiere.py` calcule, prend la fenêtre pour unité. Avec W fenêtres de même taille et $\bar{d}_w$ la moyenne de la fenêtre w,

$$\bar{d} = \frac{1}{W}\sum_{w=1}^{W} \bar{d}_w, \qquad \sigma_{\text{fen}}(\bar{d}) = \frac{1}{\sqrt{W}}\sqrt{\frac{1}{W-1}\sum_{w=1}^{W}(\bar{d}_w - \bar{d})^2}. \tag{8.1}$$

La première égalité tient parce que les fenêtres ont toutes 255 positions notées. Si les positions d'une fenêtre étaient indépendantes, les deux estimateurs auraient la même espérance ; quand elles sont corrélées positivement, σ_{fen} est plus grand, et c'est lui qu'il faut. Sa limite : $W = 10$ blocs, soit 9 degrés de liberté (section 8.8).

La différence appariée de l'éclaireur, calculée ici. L'éclaireur avait comparé deux moyennes sans les apparier (chapitre 3). Ses CSV par position sont commités ; `frontiere.py` les apparie [calc.] sur [mes.].

- *Témoin*, déclaré avant l'exécution : la KL moyenne recalculée depuis chaque CSV égale celle de `juge kl` à 10^{-12} près. Elle vaut 0,036 013 763 948 des deux côtés pour la base et 0,026 168 395 170 pour la mutation synonyme ; le témoin tient. Cassure : la première ligne retirée donne 0,036 021 222 au lieu de 0,036 013 764, et le témoin tombe (`frontiere-casse.log`).
- *Différence* (synonyme moins base) : $\bar{d} = -0{,}009845$; $\sigma_{\text{naïf}} = 0{,}00190$ ($5{,}2\,\sigma$) ; $\sigma_{\text{fen}} = 0{,}00334$ ($2{,}9\,\sigma$). Meilleure significativement par les deux estimateurs.
- 2 060 positions sur 2 550 (80,8 %) ont moins de KL avec la mutation synonyme ; le premier choix identique gagne $+3{,}73 \pm 0{,}59$ points.

Le rapport des deux écarts-types, $0{,}00334/0{,}00190 = 1{,}76$ [calc.], mesure la corrélation à l'intérieur d'une fenêtre : il vaut 1 pour des positions indépendantes. Il dit pourquoi, dans toute la suite, l'incertitude par fenêtres est celle qui juge.

Les débits n'ont pas tous le même estimateur. Codec réel décodé (P0, P3, la frontière de P2), entropie des symboles sans échelles H_0 (P1), comptabilité complète du brin (le génome) : la colonne `estimateur_bits` de la source le dit pour chaque point. À l'intérieur d'un protocole, base et mécanisme ont le même. Pour la frontière, le codec réel rend 2,644 1 bits ; H_0 et les échelles donnent 2,679, parce qu'ils ignorent le contexte κ que le codec exploite ; H_0 seul donne 2,641 5, parce qu'il ignore les échelles (0,037 5 bit). Les deux omissions se compensent presque [calc.] (`genome/05-synthese.md` § 1.1).

8.2 La frontière commune

Tableau 8.2 donne les points jugés à la KL ; la figure 8.1 les place. Le fichier `k10/frontiere.csv` en compte 25 : 12 jugés à la KL sous P1, P2-10 et P3 ; les 2 de la passe P2 à 3 fenêtres, que K10 remplace ; 7 jugés seulement à une métrique intermédiaire (erreur de poids ou de sortie d'expert) ; et les 4 points sans perte, dont la KL est nulle par construction.

Ce que la figure montre. Un seul mécanisme de la ronde passe significativement sous sa base : la mutation synonyme $\lambda = 0$, de l'éclaireur, à ×0,727 de la KL de sa base pour 0,011 bit de plus. Tout le reste est égal ou au-dessus. Les flèches en tirets de la figure partent de la *promesse* de chaque perdant (le rapport 1 qu'annonçait sa métrique intermédiaire) et vont à la KL qu'il a rendue : elles mesurent la distance entre ce que promettait l'erreur de poids ou d'expert et ce que le texte a rendu. Pour l'hebbien par e, réglé pour avoir exactement l'erreur d'expert de la base, la flèche va de 1 à 3,06.

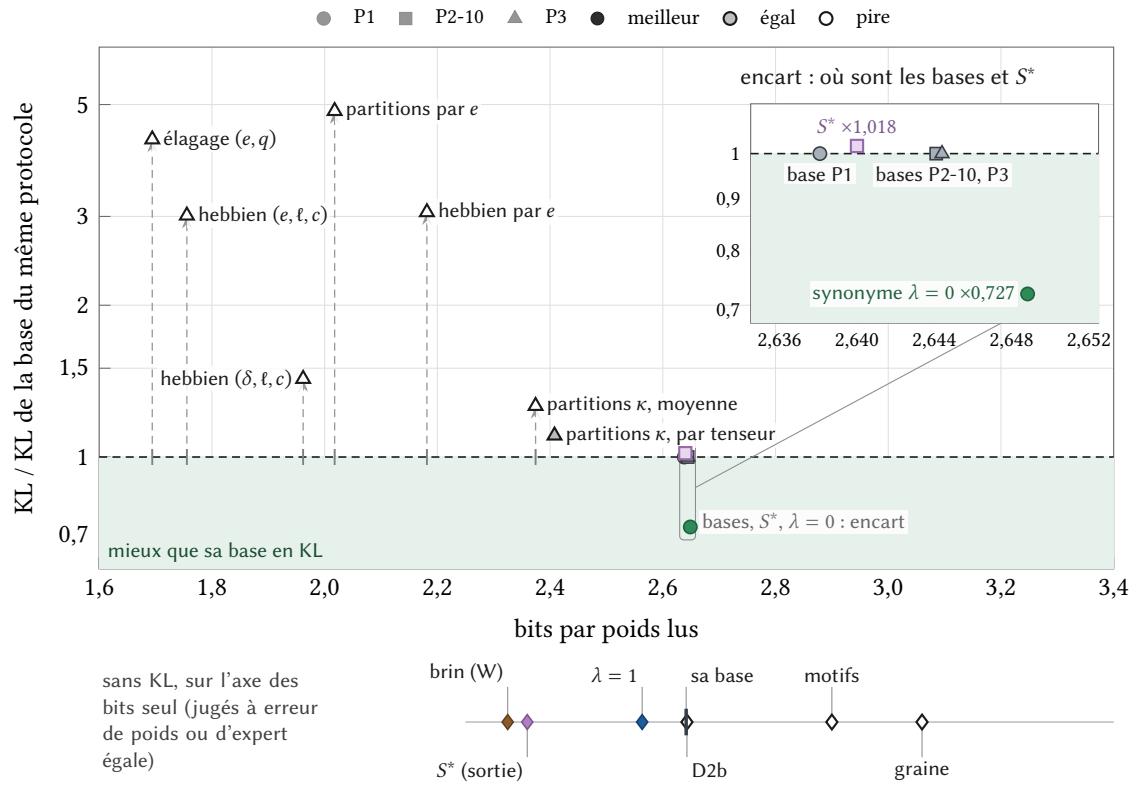


Figure 8.1 : Jugé par la KL sur le texte, un seul mécanisme de la ronde passe sous sa base : la mutation synonyme $\lambda = 0$. [mes.] [calc.] Chaque mécanisme se place par ses bits par poids et par sa KL en rapport à la base de son propre protocole (les protocoles diffèrent par les experts substitués et les fenêtres ; une KL absolue de l'un ne se compare pas à celle d'un autre). La forme dit le protocole (P1 : éclaireur, 32 experts ; P2-10 : génome, 8 experts, 10 fenêtres ; P3 : prédictif-élagage, 12 tenseurs), la couleur la variante (vert : synonyme $\lambda = 0$; violet : S^* ; gris : les bases ; encre : prédictif-élagage), le remplissage le verdict de la règle déclarée, appliquée par le script à la différence appariée $\bar{d} \pm \sigma$ contre la base : meilleur si $\bar{d} + 2\sigma < 0$, égal si $|\bar{d}| \leq 2\sigma$, pire sinon. La mutation synonyme $\lambda = 0$ est meilleure (×0,73 ; $\bar{d} = -0{,}0098 \pm 0{,}0033$, $-2{,}95\,\sigma$), à +0,011 bit. Les partitions par κ tenseur par tenseur rendent une KL égale par la règle (×1,103 ; $+0{,}0066 \pm 0{,}0053$, $+1{,}24\,\sigma$) pour 0,911× les bits : la colonne `verdict` de la source les dit gagnantes, la règle les dit égales, et la figure suit la règle. Deux perdants n'ont pas de différence appariée (partitions par e, élagage (e, q)) : leur verdict est celui de la source, tiré du rapport (×4,85 et ×4,26). Les flèches en tirets vont de la *promesse* de chaque perdant (le rapport 1 qu'annonçait sa métrique intermédiaire, mesurée ou seulement prédite) à la KL qu'il a rendue. S^* est tiré de la passe K10 à 10 fenêtres (×1,018, appariée $+0{,}0004 \pm 0{,}0019$), pas de la passe à 3 fenêtres (réserve R4) ; il se cache sous les bases, que l'encart sépare. En bas, losanges : les 7 points sans KL, sur l'axe des bits seul, pleins s'ils gagnaient à leur propre métrique (erreur de poids ou de sortie d'expert), creux s'ils y tombaient (le trait gris : la base de $\lambda = 1$) ; $\lambda = 1$ y est jugé au protocole P1-sorties (16 des 32 experts de P1, sa propre base à 2,6415 bits) ; sa KL mesurée plus tard à 768 experts est un autre protocole, non tracé ici (réserve R3). Ce que la figure ne montre pas : l'axe sans perte (KL = 0 : frontière 3,8434, codon du zéro 3,7064, contexte de colonne 3,6844, duplication D1 3,8442, qui tombe), tracé tenseur par tenseur au chapitre 9 ; ni les estimateurs de bits, qui diffèrent d'un protocole à l'autre (codec réel, H_0 des symboles, comptabilité complète du brin : colonne `estimateur_bits` de la source).

Données : `mesures/genome/construction/synthese/k10/frontiere.csv` (`outils/genome/synthese/frontiere.py --k10`), `mesures/genome/etat-de-l-art/passes/synonyme-lam1-damp1-e0_8_16_24.json` ; script `memoire/outils/tome3_synthese.py`.

Table 8.2 : La frontière commune : les points jugés à la KL, chacun contre la base de son protocole [mes.] [calc.]. Bits par poids : l'estimateur du protocole ; rapport = $\overline{\mathrm{KL}}/\overline{\mathrm{KL}}_{\mathrm{base}}$; appariée : $\bar{d} \pm \sigma$ (définition 8.2) ; verdict : la règle, appliquée par le script.

Mécanisme	Prot.	Bits	KL	Rapport	Appariée	Même 1er	Verdict
base frontière	P1	2,6381	0,0360	1	—	87,1 %	référence
mutation synonyme $\lambda = 0$	P1	2,6488	**0,0262**	**0,727**	**−0,0098 ± 0,0033**	90,9 %	**meilleure**
base frontière	P2	2,6441	0,0226	1	—	91,4 %	référence
S^*, 3 fenêtres	P2	2,64	0,0301	~~1,335~~	+0,0076 ± 0,0047	90,3 %	égal ; remplacé par P2-10
base frontière	P2-10	2,6441	0,0206	1	—	91,9 %	référence
S^*, 10 fenêtres	P2-10	2,64	0,0209	1,018	+0,0004 ± 0,0019	91,9 %	égal
base frontière	P3	2,6444	0,0636	1	—	81,6 %	référence
partitions par κ, tenseur par tenseur	P3	2,4079	0,0702	1,103	+0,0066 ± 0,0053	79,5 %	égal
partitions par κ, erreur moyenne égale	P3	2,3743	0,0804	1,263	+0,0167 ± 0,0049	78,4 %	pire
hebbien (δ, ℓ, c)	P3	1,9619	0,0909	1,429	+0,0273 ± 0,0039	73,4 %	pire
hebbien par e	P3	2,1815	0,1947	3,059	+0,1310 ± 0,0072	60,8 %	pire
hebbien (e, ℓ, c)	P3	1,7556	0,1917	3,012	+0,1281 ± 0,0067	59,4 %	pire
élagage par activité (e, q)	P3	1,6942	0,2714	4,265	—	57,1 %	tombe (rapport)
partitions par e	P3	2,0178	0,3089	4,854	—	58,5 %	tombe (rapport)

Données : `mesures/genome/construction/synthese/k10/frontiere.csv` ; P1 : `mesures/genome/etat-de-l-art/journaux/kl-*-juge.json` ; P2-10 : `mesures/genome/genome/kl/comparer10-base-contre-recette.json` ; P3 : `mesures/genome/predictif-elagage/resume-12.csv` et `kl/comparer-*.json`.

Deux corrections, dites. La première concerne les partitions par κ tenseur par tenseur. La colonne `verdict` de la source les dit « gagnantes : 0,911 × les bits, KL égale par la règle » ; la règle de la définition 8.2 les dit *égales* en KL (+0,0066 ± 0,0053, 1,2 σ), à 0,911 fois les bits de la base. La figure suit la règle, appliquée par son script ; le premier juge des figures l'avait relevé comme grave (G1, `memoire/tome3/JUGEMENT-figures-ronde.md`), et le second l'a dit levé. « Égale en KL pour 9 % de bits de moins » est une vraie avancée de débit à qualité égale ; ce n'est pas un gain de qualité. La seconde concerne la variante de la mutation synonyme qui porte la KL. La consigne de la synthèse attribuait la KL 0,0262 à la variante $\lambda = 1$: ~~$\lambda = 1$: KL 0,0262 contre 0,0360~~. Le journal (`kl-synonyme-d1-juge.json` : « d1 » pour $\delta^* = 1$, $\lambda = 0$) dit que la KL a été mesurée sur $\lambda = 0$, à 2,649 bits (+0,011 sur la frontière) ; $\lambda = 1$ (2,563 4 bits contre 2,641 5 pour la frontière sur les mêmes experts) n'avait été jugée qu'à l'erreur de sortie d'expert, 8 cases sur 8. À la fin de la ronde, *aucune variante n'était donc mesurée à la fois sous la frontière en bits et sous elle en KL* ; c'est la mesure B0 du cahier (section 8.7.2).

Ce que la figure ne montre pas. Elle garde le protocole à 32 experts sur 768 : la KL de $\lambda = 1$ mesurée plus tard sur les 768 experts (chapitre 10) est un autre protocole, et aucun trait ne relie la figure 8.1 aux points à 768 experts du chapitre 10 (réserve R3). S^* y est placé par la passe K10 à 10 fenêtres, pas par la passe à 3 fenêtres (réserve R4) ; il se cache sous les bases, et l'encart les sépare. L'axe sans perte (KL nulle) n'est plus dans cette figure : il est tracé tenseur par tenseur au chapitre 9 (figure 9.6). Ses trois points, moyennes sur les 12 tenseurs de référence, sont 3,843 4 bits (frontière sans perte), 3,706 4 (codon synonyme du zéro) et 3,684 4 (avec le contexte de colonne) ; la duplication

D1, à 3,844 2, tombe [mes.] (`k10/frontiere.csv`).

Les points sans KL. Jugés à erreur de poids ou de sortie d'expert égale à la base, ils ne se placent que sur l'axe des bits : la graine et ses coefficients (3,06 bits), les motifs partagés (2,90), la duplication D2b (2,642 9), le brin tourné au critère W (2,324 8), S^* à erreur de sortie d'expert égale (2,359 5), et la mutation synonyme $\lambda = 1$ (2,563 4). Le brin et S^* sont pleins dans la figure : ils gagnaient à leur métrique ; la KL les a fait tomber ou les a rendus égaux. C'est l'objet de la section suivante.

8.3 Trois propriétés mesurées des poids

Tout ce qui tombe dans la ronde tombe pour l'une de trois propriétés, chacune mesurée par au moins deux équipes et par deux chemins.

8.3.1 Une fois l'échelle du bloc connue, les poids sont du bruit

Table 8.3 : Propriété (a) : une fois l'échelle du bloc connue, il ne reste presque rien à prédire [mes.].

Mesure	Valeur	Chapitre
information apportée par le code voisin, sachant l'échelle	0,005 bit (gauche 0,0076, dessus 0,0091)	chapitre 6
information apportée par l'expert voisin, même place	0,0043 bit	chapitre 6
cosinus des rangées de deux experts appariés (algorithme hongrois)	0,04 (gate), 0,03 (up), 0,07 à 0,11 (down) ; hasard 0,04	chapitre 5
erreur du brin sur les rangées tournées, contre une gaussienne vraie	0,0739 contre 0,0731 (1,5 % près) ; kurtosis 3,0	chapitre 4
la graine contre le lemme de la loi Bêta	au-dessus de la prédiction de 0,6 à 2 points	chapitre 4
signe des zéros	0,9999999967 bit d'entropie, sans effet sur la valeur	chapitre 3, chapitre 9

Données : `genome/05-synthese.md` § 2.1 (a), qui renvoie aux livrables de chaque équipe.

La graine, les motifs partagés, la duplication et la recette à complément exact cherchaient tous une dépendance entre morceaux du tenseur. La tableau 8.3 dit qu'il n'y en a pas à l'échelle du bloc : sachant l'échelle, un code ne dit presque rien de son voisin ni de l'expert voisin, et une rangée tournée suit une gaussienne à 1,5 % près. Une seule « structure » de ce type est exploitable, et c'est l'inverse d'une structure : le signe d'un zéro, que le calcul ne lit pas, est une pièce juste. Le fondre retire exactement un bit par zéro de signe minoritaire (chapitre 9, section 9.2).

8.3.2 Quelques colonnes géantes portent la structure

Les deux dernières lignes disent la même chose par deux chemins indépendants. Ce que les experts d'une couche partagent n'est pas une parenté d'expert à expert : c'est *une statistique par colonne d'entrée*, commune aux 32 experts parce qu'elle vient de leur entrée commune, l'état caché et ses canaux aberrants à la couche 0. La duplication la voyait comme une « parenté de hasard » (un parent tiré au hasard rend 88 à 94 % du gain d'un vrai parent à la couche 0) ; le codage prédictif l'a prise comme contexte, et c'est le seul contexte qui a passé son seuil.

Table 8.4 : Propriété (b) : la structure qui reste est concentrée dans quelques colonnes et quelques directions [mes.].

Mesure	Valeur	Chapitre
énergie des blocs d'exposant 129 et 130	74,2 % de l'énergie pour 5,8 % des poids	chapitre 3
énergie des 5 % premières directions singulières	23 à 68 % (17 à 19 % pour la matrice permutée) ; kurtosis des rangées jusqu'à 348	chapitre 4
colonne la plus massive (blk.8 down, expert 16, colonne 1011)	×144 l'énergie moyenne d'une colonne ; reçoit ×334 l'erreur moyenne du brin	chapitre 4
part de l'erreur poolée de la frontière tenue par un seul tenseur	91,5 % (blk.23 down)	chapitre 6
gain du contexte de colonne, par tenseur	−0,1416 et −0,1231 bit (couche 0, gate et up) ; sous 0,01 ailleurs	chapitre 9
part du gain d'un parent que rend un parent de hasard	46 % en moyenne, 88 à 94 % à la couche 0	chapitre 5

Données : `genome/05-synthese.md` § 2.1 (b) ; gain de colonne : `mesures/genome/construction/synthese/k10/colonne-par-tenseur.csv`, colonnes `zero_octet_kappa` moins `zero_demi_octet_kappa_colonne`.

Une précision sur le chiffre de la colonne. Le fichier donne, pour blk.0 gate, 3,607 307 bits avec le zéro fondu et le contexte octet κ, 3,611 090 avec l'alphabet demi-octet et κ, et 3,465 690 avec la colonne. Contre le flux réel du format (l'octet), le gain de la colonne vaut $3{,}607307 - 3{,}465690 = 0{,}1416$ bit [calc.] ; contre le flux demi-octet, 0,1454. La synthèse écrivait « −0,142, −0,128 » en mêlant les deux références pour gate et up ; le chapitre 9 et ce chapitre gardent la première, 0,141 6 et 0,123 1, qui est celle de la décision du lot A.

La colonne géante, vue par trois reconstructions. La figure 8.2 prend une matrice, la couche 8, down, expert 16 du 20b, et ses 2 880 colonnes dans l'ordre naturel. La colonne 1011 porte ×143,9 l'énergie moyenne. Le brin pondéré, qui tourne chaque rangée par une transformée de Hadamard de 64 avant de la lire par codons, lui envoie ×334 son erreur moyenne : la rotation répartit la valeur géante en 64 grandes valeurs par rangée, la table de codons les écrête, et ces erreurs d'écrêtage, de même signe dans toutes les rangées, reviennent sur la colonne par la rotation inverse. La base frontière, qui garde exacts les plus grands codes de chaque bloc, n'y met que ×14,0 son erreur moyenne ; S^*, qui met à part les trois plus grands poids de chaque rangée, la ramène à ×1,05. Une seconde pointe, la colonne 960, premier canal du morceau de rotation de la géante (960 à 1023), d'énergie ordinaire (×0,83), reçoit ×310 l'erreur moyenne du brin ; sa cause n'est pas mesurée, et le dessin de la figure s'arrête si un script la déplaçait (le juge de la troisième passe l'a éprouvé : `memoire/tome3/JUGEMENT-figures3.md` § 4.4) [mes.] (`synthese-nombres.csv`).

C'est la propriété (b) dans sa forme la plus nette : une erreur de poids moyenne petite peut cacher une erreur énorme sur une colonne que les entrées excitent beaucoup. Tout mécanisme qui écrête les colonnes massives, ou les noie dans une moyenne, perd à la sortie de l'expert.

8.3.3 L'erreur de sortie d'un expert ne prédit pas la KL

Trois points ont un rapport d'erreur d'expert dans [0,75; 1,02] et des rapports de KL qui vont de 1,02 à 3,06 (figure 8.3). S^* retire 25 % de l'erreur d'expert et rien de la KL ; le seul gagnant en KL divise l'erreur d'expert par près de quatre pour un gain de KL de 27 %. La relation n'est ni monotone ni

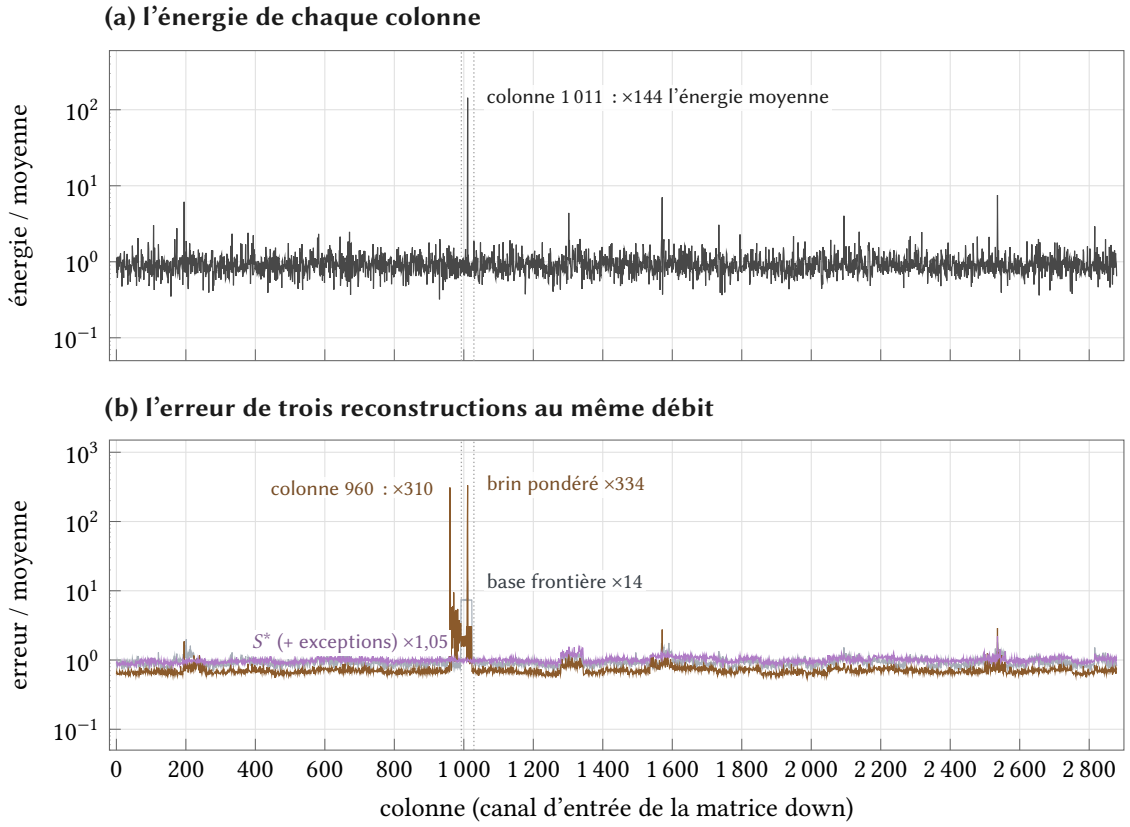


Figure 8.2 : Une colonne géante, et le premier canal de son morceau de rotation, aspirent l'erreur du brin ; la base la contient, S^* l'efface. [mes.] Une matrice : couche 8, down, expert 16 de gpt-oss-20b, ses 2 880 colonnes dans l'ordre naturel. (a) L'énergie $\sum_r w_{rc}^2$ de chaque colonne, divisée par la moyenne des colonnes. (b) L'énergie d'erreur $\sum_r (\hat{w}_{rc} - w_{rc})^2$ de trois reconstructions au même débit (2,64 bits), chacune divisée par sa propre moyenne : la base frontière (gris bleuté), le brin pondéré (brun), S^* (violet). La colonne 1 011 (entre deux pointillés) porte ×144 l'énergie moyenne et reçoit ×334 l'erreur moyenne du brin : la rotation la répartit en 64 grandes valeurs par rangée, la table les écrête, et les erreurs d'écrêtage, de même signe dans toutes les rangées, reviennent sur la colonne par la rotation inverse. La base frontière rend presque exacts les plus grands poids de chaque bloc (×14) ; S^* garde à part, exacts, les trois plus grands de chaque rangée et ramène la colonne à ×1,05. La seconde pointe est la colonne 960, premier canal du morceau de rotation de la géante (960 à 1 023), d'énergie ordinaire (×0,83) : elle reçoit ×310 l'erreur moyenne du brin, contre ×0,87 pour la base et ×1,04 pour S^* ; la cause n'en est pas mesurée. Les 62 autres colonnes du morceau reçoivent ×4,0 la moyenne, et les 44 autres bords de morceau (colonnes $\equiv 0 \pmod{64}$) ×0,70 (au plus ×0,87) : les « bords ×7,6 » de la source sont une moyenne que porte cette seule colonne. Ce que la figure ne montre pas : les autres tenseurs (une seule matrice est en CSV ; les classes de colonne de blk.0 gate restent à tracer), ni le coût des exceptions (compté dans le débit de S^*).

Données : `mesures/genome/genome/csv/colonnes-energie-erreur.csv` (`outils/genome/recette/figures_transformations.py`) ; script `memoire/outils/tome3_synthese.py`.

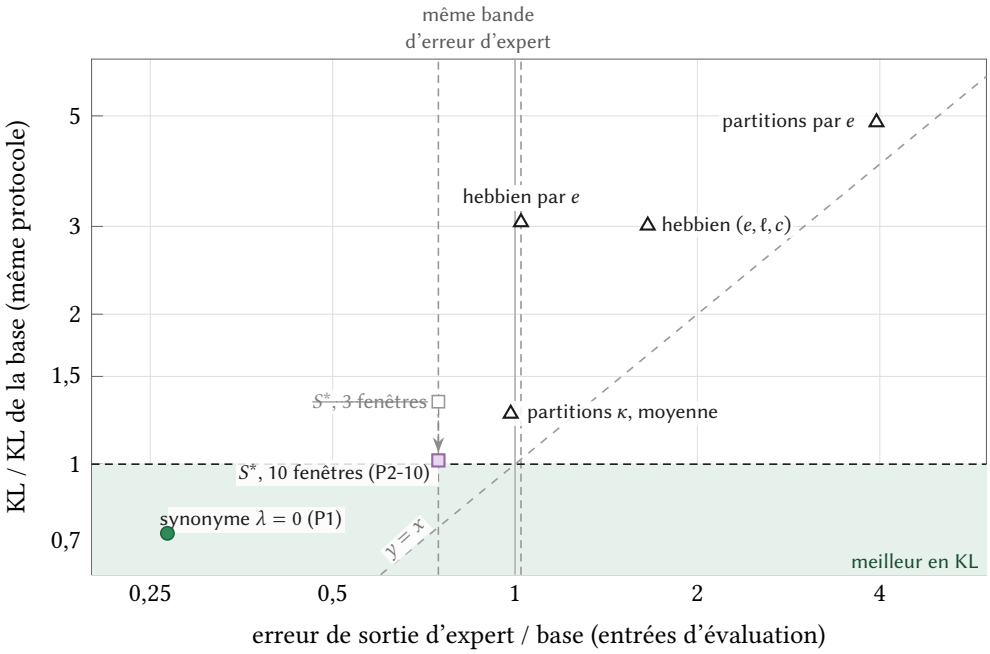


Figure 8.3 : L'erreur de sortie d'un expert ne prédit pas la KL : dans une même bande d'erreur d'expert, la KL va de l'égalité à plusieurs fois la base. [mes.] Chaque point est un mécanisme jugé des deux façons, en rapports à la base de son propre protocole : en abscisse, l'erreur de sortie des experts substitués sur des entrées tenues à l'écart ; en ordonnée, la KL du modèle entier sur le texte. Dans la même bande d'erreur d'expert (0,75 à 1,02), la KL va de ×1,02 à ×3,06 la base ; seule la mutation synonyme $\lambda = 0$ passe sous 1. S^* est placé par la passe K10 à 10 fenêtres (×1,018) ; son rapport à 3 fenêtres, barré dans la synthèse (05 § 2), est montré creux et barré, avec la flèche de la correction (réserve R4). Formes, couleurs et remplissages comme dans la frontière commune : la forme dit le protocole (P1 éclaireur, P2-10 genome, P3 prédictif-élagage), le remplissage le verdict de la même règle. Ce que la figure ne montre pas : l'incertitude de chaque rapport (différences appariées dans la source de la frontière), ni le pourquoi. La synthèse lit la KL comme une part lisse, que l'erreur d'expert suit, et une queue de bascules de routage, qu'elle ne voit pas (figure suivante) ; c'est son hypothèse de travail, appuyée par le routage imposé (Tome II, lot R).

Données : `mesures/genome/construction/synthese/k10/erreur-contre-kl.csv` (`frontiere.py --k10`, `mesures/genome/genome/kl/comparer10-base-contre-recette.json`) ; script `memoire/outils/tome3_synthese.py`.

Table 8.5 : Propriété (c) : le rapport d'erreur de sortie d'expert à la base, contre le rapport de KL [mes.] [calc.].

Mécanisme	Prot.	Erreur d'expert	KL	Verdict à la KL
mutation synonyme $\lambda = 0$	P1	0,267	0,727	meilleure
S^*	P2 ; P2-10	0,747	~~1,335~~ ; 1,018	égal
partitions par κ, erreur moyenne égale	P3	0,984	1,263	pire
hebbien par e	P3	1,023	3,059	pire
hebbien (e, ℓ, c)	P3	1,656	3,012	pire
partitions par e	P3	3,951	4,854	tombe

Données : `mesures/genome/construction/synthese/k10/erreur-contre-kl.csv` ; S^* à 3 fenêtres barré, remplacé par K10 (réserve R4).

proportionnelle. La figure montre aussi la correction de S^* : son rapport à 3 fenêtres, ~~1,335, « 33 % de KL de plus »~~, est porté creux et barré, avec une flèche vers le rapport de la passe K10, 1,018, « la même KL que la base sur 10 fenêtres ($+0{,}0004 \pm 0{,}0019$) ». Ce qui a changé : trois fois plus de positions ; l'écart des trois premières fenêtres était dans le bruit (section 8.5.1).

Pourquoi l'erreur d'expert trompe, c'est la section suivante. Le chiffre qu'il faut retenir d'ici : à erreur d'expert égale, la KL peut tripler.

8.4 Deux parts de la KL

8.4.1 Ce que disent les quantiles, et le moteur

Table 8.6 : Les quantiles de la KL par position [mes.]. La médiane dit la part lisse ; le 99,9^{e} centile et le maximum disent la queue.

	base P1	synonyme P1	base P2	S^* P2	base P2-10	S^* P2-10
moyenne	0,0360	0,0262	0,0226	0,0301	0,0206	0,0209
médiane	0,0088	0,0038 (÷2,3)	0,00172	~~0,00188~~	0,00184	0,00152 (÷1,21)
99e centile	0,495	0,410	0,437	~~0,550~~	0,411	0,338
99,9^{e} centile	1,59	1,70	0,899	~~1,43~~	1,37	1,59
maximum	4,17	3,94	1,20	~~2,79~~	3,01	3,22

Données : P1 : `mesures/genome/etat-de-l-art/journaux/kl-frontiere-juge.json`, `kl-synonyme-d1-juge.json` ; P2 : `genome/05-synthese.md` § 2.2 (`comparer-base-contre-recette.json`) ; P2-10 : `mesures/genome/genome/kl/comparer10-base-contre-recette.json`. Les colonnes de S^* à 3 fenêtres sont barrées par la synthèse elle-même : la passe K10 les remplace.

Le synonyme réduit la médiane d'un facteur 2,3 sans toucher la queue : le 99,9^{e} centile passe même de 1,59 à 1,70. La moyenne ne baisse que d'un facteur 1,38 [calc.] (0,0360/0,0262). La même lecture vaut pour S^* sur 10 fenêtres : la médiane baisse (÷1,21), le 99e centile aussi (0,411 → 0,338), mais l'extrême de la queue monte (1,37 → 1,59 au 99,9^{e} centile, 3,01 → 3,22 au maximum), et la moyenne ne bouge pas. Sur 3 fenêtres, ~~S^* garde la médiane (×1,09) et épaissit la queue (99e 0,437 → 0,550, maximum 1,20 → 2,79)~~ : la queue l'emportait par hasard d'échantillon.

Ce que le moteur dit, entre deux calculs exacts. Le lot R du Tome II a comparé deux moteurs exacts l'un à l'autre (lu dans `genome/05-synthese.md` § 2.2, qui cite le lot R, §§ 2.9 et 2.10) [mes.] : 5 087 bascules de routage sur 122 640 paires (couche, position), soit 4,15 % ; les 45 % de positions qui basculent elles-mêmes portent 96,5 % de la KL ; la marge du routeur, quatrième logit moins cinquième, a pour médiane 0,083 sur toutes les paires et 0,0095 sur celles qui basculent. Et la preuve par l'intervention, déclarée avant sa passe : imposer à l'un des deux calculs les experts choisis par l'autre retire 88 % de la KL (0,0145 → 0,00168 ; médiane $9{,}8 \cdot 10^{-4} \to 2{,}2 \cdot 10^{-4}$; premier choix 92,5 → 97,2 %).

8.4.2 La part lisse : la KL au second ordre

Une perturbation δh de l'état caché, venue des experts approchés, change les logits de sortie z d'une quantité δz. Si aucun routeur ne bascule, δz est une fonction lisse de δh, et la KL qui en résulte est petite et quadratique. C'est un calcul classique de la théorie de l'information [2] ; on le pose ici en entier, parce que tout le chapitre repose sur sa forme.

Proposition 8.3 (La KL au second ordre en les logits)**.** *Soient $z \in \mathbb{R}^V$ des logits, $p = \mathrm{softmax}(z)$, $\delta \in \mathbb{R}^V$ une perturbation et $q = \mathrm{softmax}(z+\delta)$. Alors*

$$\mathrm{KL}(p \,\|\, q) = \tfrac{1}{2}\,\delta^\top F \delta + R, \qquad F = \mathrm{diag}(p) - p\,p^\top, \qquad |R| \le \frac{(\max_i \delta_i - \min_i \delta_i)^3}{24}. \tag{8.2}$$

La forme quadratique vaut $\delta^\top F \delta = \mathrm{Var}_{I\sim p}(\delta_I)$: la variance de la perturbation sous la loi du modèle. F est l'information de Fisher de la loi catégorielle paramétrée par ses logits.

Démonstration. Pas 1 : la KL comme écart d'une fonction convexe. Notons $A(z) = \log \sum_j e^{z_j}$ (le log-partition). Alors $\log p_i = z_i - A(z)$ et $\log q_i = z_i + \delta_i - A(z+\delta)$, d'où

$$\mathrm{KL}(p\|q) = \sum_i p_i(\log p_i - \log q_i) = \sum_i p_i\big(A(z+\delta) - A(z) - \delta_i\big) = A(z+\delta) - A(z) - p^\top\delta,$$

puisque $\sum_i p_i = 1$.

Pas 2 : les dérivées de A. $\partial A/\partial z_i = e^{z_i}/\sum_j e^{z_j} = p_i$, donc $\nabla A(z) = p$. Puis $\partial p_i/\partial z_j = p_i(\mathbb{1}[i=j] - p_j)$, donc $\nabla^2 A(z) = \mathrm{diag}(p) - pp^\top = F$. Plus généralement, le long d'une droite $z + \theta\delta$, les dérivées successives de $\theta \mapsto A(z+\theta\delta)$ sont les cumulants de la variable $X = \delta_I$, $I \sim p_\theta = \mathrm{softmax}(z+\theta\delta)$: la première est $\mathbb{E}_{p_\theta} X$, la deuxième $\mathrm{Var}_{p_\theta} X$, la troisième $\mathbb{E}_{p_\theta}(X - \mathbb{E}X)^3$.

Pas 3 : Taylor avec reste de Lagrange. Il existe $\theta \in (0,1)$ tel que

$$A(z+\delta) = A(z) + p^\top\delta + \tfrac{1}{2}\delta^\top F\delta + \tfrac{1}{6}\,\mathbb{E}_{p_\theta}(X - \mathbb{E}_{p_\theta}X)^3.$$

Reporté dans le pas 1, le terme d'ordre 1 s'annule exactement : $p^\top\delta$, l'espérance de δ_I sous p, se retranche de lui-même. Il reste $\mathrm{KL} = \frac{1}{2}\delta^\top F\delta + R$ avec $R = \frac{1}{6}\mathbb{E}_{p_\theta}(X - \mathbb{E}X)^3$.

Pas 4 : la borne du reste. X prend ses valeurs dans $[a,b]$, $a = \min_i \delta_i$, $b = \max_i \delta_i$. Alors $|X - \mathbb{E}X| \le b - a$, donc $|\mathbb{E}(X-\mathbb{E}X)^3| \le (b-a)\,\mathrm{Var}\,X$, et la variance d'une variable bornée dans $[a,b]$ vaut au plus $(b-a)^2/4$. D'où $|R| \le \frac{1}{6}(b-a)^3/4 = (b-a)^3/24$.

Pas 5 : la variance. $\delta^\top F\delta = \sum_i p_i\delta_i^2 - (\sum_i p_i\delta_i)^2 = \mathrm{Var}_p(\delta_I)$. Elle est invariante si l'on ajoute une constante à tous les δ_i, comme le softmax lui-même. ∎

Remarque 8.4 (Ce que la proposition dit de la part lisse). Si les logits de sortie dépendent linéairement de l'état final par la tête W_o, $\delta z = W_o \delta h_L$, la part lisse vaut $\frac{1}{2}\delta h_L^\top W_o^\top F W_o \delta h_L$: quadratique en la perturbation, et pondérée par la loi du modèle. Une erreur d'expert deux fois plus petite donne, tant que rien ne bascule, une part lisse quatre fois plus petite, et seule compte la composante de l'erreur que $W_o^\top F W_o$ voit. C'est la raison pour laquelle la mutation synonyme, qui pondère son erreur par les entrées réelles de chaque expert ($H = \mathbb{E}[g^2 x x^\top]$, chapitre 10), réduit la médiane : elle range l'erreur dans les directions que les entrées n'excitent pas.

8.4.3 La part de bascule

La proposition ne vaut que si l'ensemble des experts choisis ne change pas. Or le routeur d'une couche ℓ garde les $k = 4$ experts de plus grands logits. Si δh pousse l'écart entre le quatrième et le cinquième au-delà de sa marge $m_\ell = z_{(4)} - z_{(5)}$, l'ensemble change, et la sortie de la couche saute d'une quantité qui ne dépend plus de $\|\delta h\|$, mais de la différence entre deux experts. Or deux experts d'une même couche ne se ressemblent pas : servir un expert par son meilleur paralogue laisse 33 à 84 % de l'énergie de sa sortie en erreur selon la couche (chapitre 5).

Hypothèse 8.5 (La forme de la part de bascule)**.** En notant δs_ℓ la perturbation de l'écart des deux scores en jeu à la couche ℓ,

$$\mathbb{E}[\mathrm{KL}] \;\approx\; \mathbb{E}\Big[\tfrac{1}{2}\,\delta z^\top F\,\delta z\Big] \;+\; \sum_{\ell} \mathbb{P}\big(m_\ell < |\delta s_\ell|\big)\;\mathbb{E}\big[\mathrm{KL} \mid \text{bascule à } \ell\big].$$

Le second terme ne dépend de $\|\delta h\|$ que par une probabilité ; sa valeur conditionnelle dépend de la différence de deux experts, pas de la taille de l'erreur.

C'est une hypothèse de travail, pas une proposition : elle donne la forme, pas les termes. Elle s'appuie sur trois faits mesurés : l'intervention du lot R (imposer le routage retire 88 % de la KL entre deux moteurs exacts), la répartition de la KL (96,5 % sur les positions qui basculent), et les quantiles de la ronde (tableau 8.6). Elle n'a pas été mesurée *pour les mécanismes de la ronde* par le routage imposé : la mesure qui la jugerait, la passe KL de S^* et de la base avec les experts imposés depuis la référence, n'est pas faite (section 8.8). Elle explique ce que la propriété (c) constate : l'erreur d'expert pilote le premier terme, et presque rien du second, qui porte la moyenne.

Deux parts, position par position. La figure 8.4 met chacune des 2 550 positions du protocole P1 en un point : la KL de la base en abscisse, celle de la mutation synonyme en ordonnée, mêmes positions et même vérité. La masse passe sous la diagonale : la médiane est divisée par 2,3, et 2 060 positions sont gagnées. La queue garde sa forme : les 25 plus grandes positions de chacune (le « 1 % ») portent 31 % de la KL moyenne de la base et 38 % de celle du synonyme, si bien que la moyenne ne baisse que de ×1,38. Elle ne garde pas ses positions : 10 seulement des 25 sont communes aux deux queues. C'est ce que prédit l'hypothèse 8.5 : une bascule dépend d'une marge petite et d'une perturbation de signe donné ; quand les poids changent, les perturbations changent, et les positions qui basculent changent avec elles. La queue de la base se répartit sur 9 des 10 fenêtres, au plus 7 dans une seule : les bascules se groupent par fenêtre, par leur passé commun, ce qui justifie l'incertitude par fenêtres de l'équation (8.1) [mes.] (`synthese-nombres.csv`). La figure ne refait pas l'intervention ; ces journaux n'ont pas de colonne de routage, et elle ne dit pas quelles positions ont basculé.

8.4.4 Le routeur comme géométrie tropicale

L'hypothèse 8.5 donne aux bascules le premier rôle ; il faut alors savoir où elles ont lieu. Le routeur d'une couche calcule $z(h) = W_r h + b_r$ (32 logits pour le 20b, 128 pour le 120b) et garde les $k = 4$ plus grands ; h est l'état normé qui entre dans les experts.

Proposition 8.6 (Les régions du routeur sont des polyèdres)**.** *Pour tout ensemble S de k experts, la région $\mathcal{R}_S = \{h \,:\, z_i(h) \geq z_j(h)\ \forall i \in S, j \notin S\}$ est un polyèdre convexe. La fonction $\Phi(h) = \sum_{i \in S(h)} z_i(h)$, somme des k plus grands logits, est convexe et linéaire par morceaux ; c'est un polynôme tropical du semi-anneau* $(\max, +)$*, et les faces de ses morceaux sont portées par les hyperplans $z_i = z_j$.*

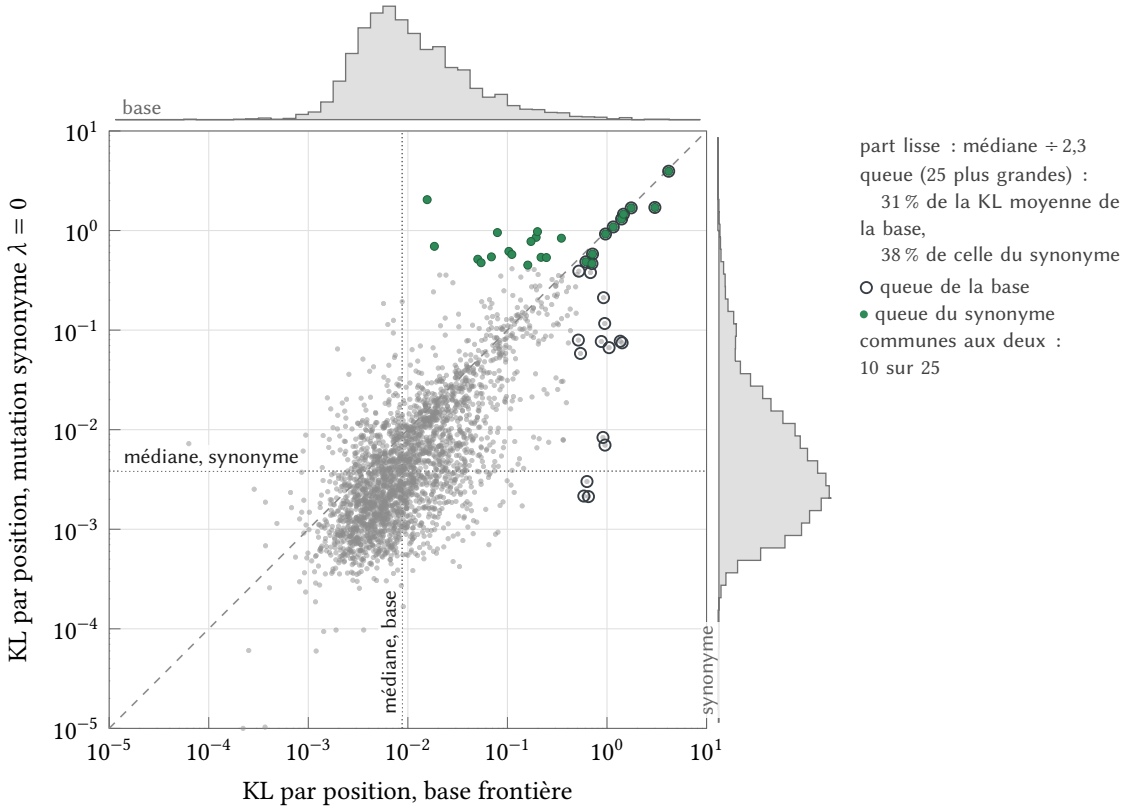


Figure 8.4 : La KL a deux parts : sa masse passe sous la diagonale ; sa queue garde sa forme, pas ses positions. [mes.] [calc.] Les 2 550 positions de la passe KL de l'éclaireur (10 fenêtres de wiki.test, 32 experts sur 768 substitués : protocole P1), la base frontière en abscisse, la mutation synonyme $\lambda = 0$ en ordonnée, mêmes positions et même vérité ; les densités marginales en marge. La masse passe sous la diagonale : la médiane est divisée par 2,3 (de 0,0088 à 0,0038), et le synonyme a moins de KL sur 2 060 positions. La queue garde sa forme : les 25 plus grandes positions de chacun (le « 1 % », $\lfloor 2\,550/100 \rfloor$, soit 0,98 %) portent 31 % de la KL moyenne de la base et 38 % de celle du synonyme, si bien que la moyenne ne baisse que de ×1,38 et que le 99,9^{e} centile monte (de 1,59 à 1,70). Elle ne garde pas ses positions : 10 seulement des 25 sont communes aux deux queues (anneaux : la queue de la base ; disques verts : celle du synonyme). La queue de la base se répartit sur 9 des 10 fenêtres, au plus 7 dans une seule. La synthèse lit la masse comme la part lisse, que l'erreur d'expert suit, et la queue comme des bascules de routage, qui se déplacent quand les poids changent ; c'est son hypothèse de travail, appuyée par l'intervention qui impose le routage (Tome II, lot R, expérience B), que cette figure ne refait pas. La seule KL sous 10^{-5} est négative ($-6{,}9 \cdot 10^{-6}$, bruit d'arrondi) et portée à ce plancher. Ce que la figure ne montre pas : quelles positions ont basculé un expert (ces journaux n'ont pas de colonne de routage), ni les 736 autres experts, laissés exacts.

Données : `mesures/genome/etat-de-l-art/journaux/kl-frontiere-positions.csv`, `kl-synonyme-d1-positions.csv`, `kl-frontiere-juge.json`, `kl-synonyme-d1-juge.json` ; script `memoire/outils/tome3_synthese.py`.

Démonstration. Chaque condition $z_i(h) - z_j(h) \geq 0$ s'écrit $(w_i - w_j)^\top h + (b_i - b_j) \geq 0$: un demi-espace fermé. $\mathscr{R}_S$ en est l'intersection finie, donc un polyèdre convexe. Pour Φ : la somme des k plus grands nombres d'une liste est le maximum, sur les sous-ensembles T de taille k, de $\sum_{i\in T} z_i$; donc $\Phi(h) = \max_{|T|=k} \big(\sum_{i\in T} w_i\big)^\top h + \sum_{i\in T} b_i$, un maximum fini de fonctions affines, convexe et linéaire par morceaux. Dans le semi-anneau où l'addition est max et la multiplication +, c'est un polynôme tropical en les coordonnées de h, de monômes indexés par T. Deux morceaux T, T' se raccordent là où $\sum_T z = \sum_{T'} z$; pour deux voisins qui ne diffèrent que par un échange $i \leftrightarrow j$, c'est l'hyperplan $z_i = z_j$. ■

Le lien entre réseaux linéaires par morceaux et géométrie tropicale est celui de Zhang, Naitzat et Lim [115] : les réseaux ReLU sont des applications rationnelles tropicales, et leurs frontières de décision des hypersurfaces tropicales [litt.]. Le routeur top-k n'est pas un ReLU ; il s'y range par la même algèbre (max, +).

Proposition 8.7 (La marge et la distance)**.** *Soit $h \in \mathscr{R}_S$, et soient* (4) *et* (5) *les indices des quatrième et cinquième logits, de marge $m(h) = z_{(4)}(h) - z_{(5)}(h) \geq 0$. La distance euclidienne de h à l'hyperplan $z_{(4)} = z_{(5)}$ vaut $m(h)/\|w_{(4)} - w_{(5)}\|$. La distance de h au bord de sa région vaut*

$$d(h, \partial\mathscr{R}_S) = \min_{i\in S,\, j\notin S} \frac{z_i(h) - z_j(h)}{\|w_i - w_j\|} \;\leq\; \frac{m(h)}{\|w_{(4)} - w_{(5)}\|}.$$

Démonstration. La distance d'un point h à l'hyperplan $\{u \,:\, a^\top u + c = 0\}$ vaut $|a^\top h + c|/\|a\|$; avec $a = w_{(4)} - w_{(5)}$, $c = b_{(4)} - b_{(5)}$, le numérateur est $m(h)$. Le bord de $\mathscr{R}_S$ est contenu dans la réunion des hyperplans $z_i = z_j$, $i \in S$, $j \notin S$, et pour atteindre la frontière d'un polyèdre convexe depuis un point intérieur, il faut franchir l'un de ses demi-espaces : la distance au bord est le minimum des distances à ces hyperplans (chaque minimum est atteint dans la direction normale, qui ne sort d'aucun autre demi-espace avant, sans quoi celui-ci serait plus proche). La paire ((4), (5)) figure dans ce minimum, d'où l'inégalité. ■

Remarque 8.8. La synthèse écrit que la distance euclidienne « à la frontière la plus proche » vaut $m/\|w_{(4)} - w_{(5)}\|$ (`genome/05-synthese.md` § 3.1). La proposition 8.7 la précise : c'est exactement la distance à l'hyperplan de la paire en jeu, et une borne supérieure de la distance au bord de la région, atteinte quand (4), (5) est aussi la paire la plus proche en distance euclidienne. La marge en unités de logit, celle que mesurent les journaux, est la grandeur qui décide de la bascule ; c'est elle que le chapitre 12 utilise.

Les régions, vues dans l'espace latent. La figure 8.5 projette les 6 132 états de calibration de la couche 23 du 20b (2 880 dimensions) et les colore par leur expert premier. Les états se regroupent par expert premier : dans l'espace d'origine, 48,1 % des 10 plus proches voisins d'un état ont le même expert premier, contre 12,2 % au hasard. Les états de petite marge, au sens de $z_{(1)} - z_{(2)}$ (les 614 du dixième centile, sous 0,0155), se tiennent plus souvent aux bords des groupes : 0,70 de leurs voisins ont un autre expert premier, contre 0,47 au-delà de la médiane dans le plan de la t-SNE, et 0,67 contre 0,43 dans l'espace d'origine. C'est l'image, dans les états réels, de la proposition 8.6 : les états fragiles sont près des faces.

Deux précisions, que la légende jugée donne et qu'il ne faut pas perdre. D'abord, la projection n'est fidèle qu'à ce qu'elle dit : l'ACP garde 21,0 % de la variance, avec une confiance $T(10) = 0{,}792$; la t-SNE, écrite de zéro et déterministe, monte à $T = 0{,}947$, gardée par un seuil déclaré avant la passe, mais ses distances entre groupes et ses aires ne se lisent pas. Ensuite, la marge qui marque les bords

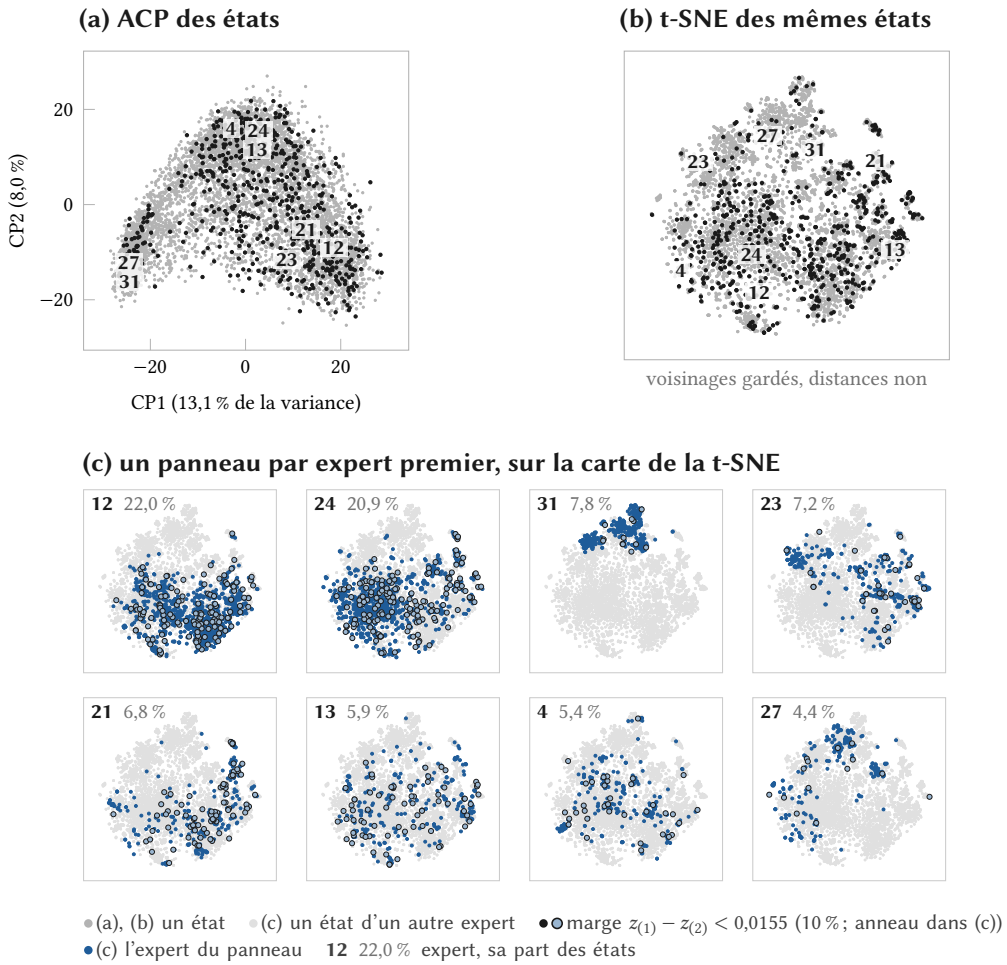


Figure 8.5 : Les 6 132 états de la couche 23 se regroupent par expert premier, et les états de petite marge se tiennent plus souvent aux bords des groupes (0,70 de voisins d'un autre expert, contre 0,47). [mes.] États de calibration de gpt-oss-20b (h après la norme d'après l'attention, 2 880 dimensions, fenêtres 10 à 21 de wiki.test). Expert premier : le plus grand logit du routeur ; marge : $z_{(1)} - z_{(2)}$, l'écart des deux plus grands logits biaisés. En noir (cerclés dans (c)), les 614 états de plus petite marge (10 %, sous 0,0155). (a) ACP centrée, non réduite : 21,0 % de la variance, confiance $T(10) = 0{,}792$, continuité $C(10) = 0{,}917$, Spearman des distances 0,59. (b) t-SNE écrite de zéro, déterministe (perplexité 30) : $T = 0{,}947$, $C = 0{,}959$, Spearman 0,41 ; gardée par le seuil déclaré avant la passe (T et C au moins ceux de l'ACP), que son initialisation par l'ACP atteint déjà : elle le dépasse de 0,155 et de 0,042. Ses distances entre groupes et ses aires ne se lisent pas. Numéros : là où les états du groupe sont le plus serrés (la densité, pas la pureté : dans l'ACP, le groupe y est souvent minoritaire), un peu écartés là où deux noms se recouvraient. (c) Les huit experts premiers les plus fréquents (80,5 % des états ; 19 autres). Part des 10 plus proches voisins de même expert premier : 48,1 % dans l'espace d'origine, 28,7 % (ACP), 44,2 % (t-SNE), 12,2 % au hasard (somme des carrés des parts). Bord d'un état, la part de ses 10 voisins d'un autre expert premier : 0,70 pour les états de petite marge contre 0,47 au-delà de la médiane (plan de la t-SNE), 0,67 contre 0,43 dans l'espace d'origine (juge du premier flux) ; la marge de l'ensemble des quatre experts servis, $z_{(4)} - z_{(5)}$, ne les marque pas (0,51 sous le 10e centile comme 0,51 au-delà de la médiane) et n'est pas dessinée. Témoins : le routeur relu redonne les 6 132 ensembles du journal (sans son biais : 3 482) ; fidélité 1 pour l'ACP complète, $T = 0{,}500$ pour un plan permuté ; bords égaux à ceux du juge (à 9 voisins, ils s'en écartent).

Données : `mesures/genome/tome3/latents/etats-c23.csv`, `etats-c23.json` (graine 20261027), par `outils/genome/tome3/latents_etats.py` ; captures `h-L23.f32`, `routage-L23.bin` (éclaireur) ; bords dans l'espace d'origine `outils/genome/tome3/juge-latents/juge-etats-marges.json` ; `memoire/outils/tome3_latents.py` (témoins `latents-temoins.json`).

est celle du *premier* expert ; la marge de l'ensemble des quatre experts servis, $z_{(4)} - z_{(5)}$, celle de la proposition 8.7, ne les marque pas (0,51 sous le dixième centile comme au-delà de la médiane) : la couleur de la figure est l'expert premier, et une face entre deux ensembles de quatre ne sépare pas forcément deux couleurs. La figure illustre la géométrie ; elle ne mesure pas la garde. Le chapitre 12 reprend ce cadre sur les journaux de routage et le juge.

8.5 Mécanisme par mécanisme

Tableau 8.7 rattache chaque verdict à la propriété qui l'a décidé.

8.5.1 Pourquoi S^* ne gagne pas

S^* est le cas le plus instructif, parce qu'il gagne tout sauf la KL : l'erreur de poids, l'erreur de sortie (8 experts sur 8), et même l'erreur orthogonale au gain par le second chemin de l'équipe (chapitre 4). ~~Sa KL appariée est $+0{,}0076 \pm 0{,}0047$ ($1{,}6\,\sigma$) : égale par la règle, défavorable en estimation, et sa queue est plus épaisse.~~ Sur 3 fenêtres (765 positions), sa KL appariée était $+0{,}0076 \pm 0{,}0047$; sur 10 fenêtres (2 550 positions, passe K10 déclarée avant), elle est de $+0{,}00036 \pm 0{,}00186$: égale à la base, premier choix égal (91,92 contre 91,88 %) [mes.] (`comparer10-base-contre-recette.json`). Vingt-cinq pour cent d'erreur d'expert en moins, zéro pour cent de KL en moins : c'est la propriété (c) dans sa forme la plus nette.

Pourquoi l'extrême de la queue monte est une hypothèse, que la synthèse donne et qu'aucune passe ne départage. La rotation de Hadamard répartit l'erreur de chaque rangée sur 64 colonnes ; la sortie $Wx = (WR^\top)(Rx)$ voit cette erreur à travers Rx, où une activation massive est elle-même répartie sur les 64 positions de son morceau. La pondération de S^* est diagonale ($h = \mathrm{diag}(R\Sigma R^\top)$) : elle ne voit pas la corrélation, dans le domaine tourné, entre les 64 positions qui portent le même canal massif. La base, elle, rend presque exacts les plus grands codes de chaque bloc, et laisse presque intacte la contribution des canaux massifs, ce qui protège les marges du routeur des couches suivantes, qui lisent le même état. La mutation synonyme, qui pondère par le H complet, n'a pas ce défaut, et c'est elle qui gagne. La prédiction qui départagerait, la KL de S^* combiné à la rétroaction par le H complet, n'est ni mesurée ni dans les lots.

8.5.2 La leçon, en une phrase par niveau

- Dans le domaine des poids, il n'y a rien à générer : le gain se prend sur ce que le calcul *ne lit pas* (le signe d'un zéro) ou sur ce que le codeur *sait déjà* (l'échelle, la colonne).
- Dans le domaine des sorties, la moyenne de l'erreur ne suffit pas : ce qui compte est l'endroit où elle tombe. Le synonyme la place là où les entrées ne regardent pas, l'hebbien là où l'énergie est faible ; les deux ont une erreur d'expert basse, seul le premier gagne en KL.
- Dans le domaine du texte, ce qui coûte est la *bascule* : une erreur ne vaut que par sa probabilité de faire franchir une face du routeur à une couche suivante.

8.6 Le contexte d'un symbole de colonne

La propriété (b) a donné un gagnant sans perte : la classe d'énergie de la colonne d'entrée comme contexte du codeur. Le cahier l'a spécifiée au bit près, sous une sorte de flux CODES_COL ; le lot A l'a finalement laissée hors du format, pour un gain trop petit au regard de son prix (section 9.4.1). La spécification reste la référence si un modèle à canaux aberrants plus nombreux la faisait payer ; elle

Table 8.7 : Chaque mécanisme de la ronde, son verdict, et la propriété mesurée qui l'a décidé (a : bruit sachant l'échelle ; b : colonnes géantes ; c : erreur d'expert ≠ KL) [mes.] [calc.].

Mécanisme	Verdict	Propriété	Le chiffre qui le dit
codon synonyme du zéro	gagne, sans perte	(a), à l'envers : il retire du bruit pur au lieu d'en chercher la structure	0,1370 bit sur les 12 tenseurs ; logits identiques au bit
contexte de colonne	gagne, sans perte (couche 0)	(b) : la seule structure de premier ordre qui reste sachant l'échelle est par colonne d'entrée	−0,1416 / −0,1231 bit à la couche 0 ; seuil 0,015
mutation synonyme	meilleure en KL ; débit à confirmer ($\lambda = 1$)	(c), dans le bon sens : pondérée par $H = \mathbb{E}[g^2 x x^\top]$, elle réduit la part lisse	appariée −0,0098 ± 0,0033 ; médiane ÷2,3 ; erreur de poids plus haute ($\lambda = 0$: 4,7 contre 4,4 %)
partitions par κ, par tenseur	égale en KL, 0,911× les bits	relative à la ligne, elle ne déplace pas de bits d'une échelle à l'autre	appariée +0,0066 ± 0,0053 ($1{,}2\,\sigma$)
graine + coefficients	tombe	(a) : un livre de codes aléatoire de dimension 8 à 16 sur des directions uniformes	3,06 bits à l'erreur de la base
motifs partagés	tombe	(a) : quantificateur vectoriel de petite dimension	2,90 bits
recette + complément exact	tombe	(a) : l'entropie des codes est déjà atteinte	4,05 à 4,37 bits contre 3,84
duplication (parent)	tombe	(a) entre experts ; la part qui ressemble à une parenté est (b)	0,0034 bit dont 0,0016 au hasard ; D2b 2,6429
brin lu par codons, critère W	gagne en poids, tombe en sortie d'expert	(b) : la table écrête les colonnes massives	colonne 1011 : énergie ×144, erreur ×334 ; sortie 7,63 contre 6,77 %
S^*	gagne en sortie d'expert, ~~tombe~~ égal en KL	(c) : la part lisse baisse, l'extrême de la queue monte	erreur d'expert ×0,747 ; appariée +0,0004 ± 0,0019
hebbien par e	tombe à erreur d'expert égale	(b) et (c) : l'erreur va sur les blocs de petite échelle, que la norme RMS des couches suivantes ramène à l'échelle	erreur d'expert ×1,02 ; KL ×3,06
partitions par e, élagage par activité	tombent	(b) mal utilisée : la métrique par matrice vide les lignes de petite échelle	KL ×4,3 à ×4,9

Données : `genome/05-synthese.md` § 2.3 ; chiffres dans `k10/frontiere.csv`, `k10/erreur-contre-kl.csv`, `k10/colonne-par-tenseur.csv`.

contient une dérivation qu'il faut poser, parce qu'un décodeur parallèle doit retrouver la colonne de chaque symbole sans rien recalculer.

La disposition de `block_mxfp4`. Un bloc MXFP4 compte 17 octets : l'exposant e (E8M0), puis 16 octets `qs` qui portent 32 codes de 4 bits. L'octet t ($0 \le t < 16$) porte le poids t du bloc dans son demi-octet faible et le poids $16 + t$ dans son demi-octet fort [8, 116]. Une matrice $R \times C$ a $B = C/32$ blocs par ligne ; les blocs se suivent dans l'ordre β (ligne par ligne).

La chaîne. Le flux CODES_COL code les $R \cdot B \cdot 32$ demi-octets canoniques dans l'ordre β ; le symbole n est le demi-octet n, le faible de l'octet $\lfloor n/2 \rfloor$ si n est pair, le fort sinon. Le décodeur rANS entrelacé (chapitre 9 ; Tome II, chapitre du format) produit, sur la voie l du macro-pas m, le mot $w = 32m + l$ de 32 bits, et son sous-pas $k \in \{0, \dots, 7\}$ produit un demi-octet. On pose, pas à pas :

$$n = 8w + k \qquad \text{(un mot porte 8 demi-octets)} \tag{8.3}$$

$$\beta = \lfloor n/32 \rfloor = \lfloor w/4 \rfloor \qquad \text{(32 demi-octets par bloc ; } 8(w \bmod 4) + k < 32\text{)} \tag{8.4}$$

$$u = n \bmod 32 = 8\,(w \bmod 4) + k \qquad \text{(rang du demi-octet dans son bloc)} \tag{8.5}$$

$$t = \lfloor u/2 \rfloor, \quad \text{fort} \iff u \text{ impair} \qquad \text{(l'octet, puis le demi-octet)}$$

$$i(u) = \lfloor u/2 \rfloor + 16\,(u \bmod 2) \qquad \text{(la position du poids dans son bloc)} \tag{8.6}$$

$$\mathrm{col}(n) = (\beta \bmod B) \cdot 32 + i(u) \qquad \text{(la colonne du poids dans sa ligne)} \tag{8.7}$$

$$\mathrm{ctx}(n) = 4\,\kappa(\beta) + q(\mathrm{col}(n)) \in [0, 12) \qquad \text{(le contexte)} \tag{8.8}$$

La deuxième égalité de (8.4) se vérifie : $n = 32\lfloor w/4 \rfloor + 8(w \bmod 4) + k$ et $0 \le 8(w \bmod 4) + k \le 8 \cdot 3 + 7 = 31$. Conséquence : *les 8 demi-octets d'un mot sont dans le même bloc*, donc ils partagent la même classe d'échelle $\kappa(\beta)$ ($\delta < 128$, $= 128$, > 128, celle de la v1.0) ; un seul κ par mot, comme en CODES_CTX, et le décodeur garde sa lecture d'avance de κ. Seule $q(\mathrm{col})$ change d'un sous-pas à l'autre, par la carte des colonnes. Exemple : $w = 6$, $k = 3$ donnent $n = 51$, $\beta = 1$, $u = 19$, l'octet $t = 9$ en son demi-octet fort, $i = 9 + 16 = 25$, et la colonne 25 du deuxième bloc de la ligne, soit $32 + 25 = 57$ si $B > 1$ [calc.].

La carte des colonnes. La classe $q(c) \in \{0, 1, 2, 3\}$ de la colonne c occupe les bits $2(c \bmod 4)$ et $2(c \bmod 4) + 1$ de l'octet $\lfloor c/4 \rfloor$: $C/4$ octets, 720 pour $C = 2880$, un multiple de 8 puisque C est un multiple de 32. Le codeur la calcule pour le tenseur entier, tous experts confondus :

$$E(c) = \sum_{\text{experts, lignes, blocs } b \text{ de la colonne } c} 4^{\delta_b - 128}\, K[\text{code} \,\&\, 7]^2, \qquad K = (0, 1, 2, 3, 4, 6, 8, 12),$$

puis $q(c) = \#\{i : t_i < E(c)\}$, avec t_1, t_2, t_3 les quantiles 0,25, 0,5 et 0,75 des C valeurs triées. Le facteur $4^{\delta - 128}$ est le carré de l'échelle $2^{\delta - 128}$: $E(c)$ est l'énergie de la colonne en unités des valeurs doublées. Le décodeur lit la carte ; il ne la recalcule pas. C'est exactement la carte qui a donné 3,684 4 bits [mes.].

La règle du choix par matrice. CODES_COL coûte au décodeur environ deux fois et demie un flux CODES_CTX (section 8.7.1). D'où une règle normative pour le codeur : CODES_COL seulement si le gain estimé, tables comprises, atteint 0,015 bit par poids, le seuil qu'avait déclaré le codage prédictif pour un flux de plus (0,40 % de 3,84 bits). Sur les 12 tenseurs de référence, deux matrices passent (gate et up de la couche 0), pour 3,684 3 bits en moyenne, contre 3,682 5 en prenant les six matrices qui gagnent quelque chose : la règle prend 92 % du gain de la colonne (0,0221 bit sur 0,0239) pour un décodeur plus cher sur deux matrices au lieu de six [calc.] (`k10/apparie-eclaireur.json`, champ `colonne`).

8.7 Le cahier de construction

La consigne de Stone pour cette étape, le 17 octobre 2026 : « vu qu'on a les conclusions je propose d'avancer avec ce qu'on a on a déja pris beaucoup de temps ». On construit avec les gagnants mesurés, en deux lots dans cet ordre : **A** est sans perte et touche le fichier exact ; **B** ajoute un second fichier, avec perte, et ne touche pas au premier. Chaque pas a son témoin, sa cassure et son seuil, déclarés dans le cahier avant toute construction.

8.7.1 Lot A : ce qu'il porte, et la preuve qu'il ne change aucun bit

Le lot A porte trois choses : le codon synonyme du zéro dans tous les flux de codes (le codeur réécrit tout demi-octet 8 en 0 avant le codage entropique) ; le contexte de colonne, seulement là où il rend au moins 0,015 bit ; l'empreinte de l'expert canonique, sans laquelle le témoin du moteur ne peut plus comparer un expert décodé à son origine.

Table 8.8 : Ce que le lot A devait rendre, prédit par le cahier [calc.] sur [mes.]. Le chapitre 9 le mesure.

	gpt-oss-20b	gpt-oss-120b
poids d'experts	19 110 297 600	114 661 785 600
sans perte v1.0 (A_flux, tous tenseurs)	3,8509 bits	3,8549 bits
zéro fondu (A_flux, tous tenseurs) [mes.]	3,7230 bits (−0,1280)	3,7292 bits (−0,1257)
octets d'experts en moins (zéro seul)	305,6 Mo	1 801,2 Mo
fichier `.trc` v1.0, relevé	11 157 405 696 o	57 730 662 400 o
fichier v1.2 attendu (zéro seul)	≈ 10,85 Go	≈ 55,93 Go
toit du débit à disque limitant	+3,4 %	+3,4 %

Données : `genome/05-synthese.md` § 4.1, d'après `mesures/genome/predictif-elagage/taux-reel-toutes-gptoss\{20b,120b\}.json`.

Le toit, posé. À disque limitant, le temps d'un jeton est proportionnel aux octets lus, et le gain de vitesse au mieux l'inverse du rapport des octets [calc.] :

$$\frac{1}{1-0{,}1257/3{,}8549}-1=\frac{1}{0{,}96739}-1=3{,}37\ \%.$$

L'alignement de chaque tronçon sur 4 096 octets coûte au plus $4\,608\times4\,096=18{,}9$ Mo sur le 120b, soit $18{,}9\cdot10^6\times8/114\,661\,785\,600=0{,}0013$ bit par poids [calc.]. Le chapitre 9 montre que ce toit ne bornait que l'effet des octets, et que la vitesse a suivi une autre variable.

Proposition 8.9 (A1 : la GEMV du moteur rend les mêmes bits)**.** *Le noyau* `produit_bloc` *(*`moteur/crates/trc-noyaux/noyaux/gemv.cuh`*, au commit* `000cef9`*) part de* $s=+0$ *et accumule* $s\leftarrow\mathrm{fmaf}(v,x,s)$ *dans un ordre fixe, avec* $v=$ `valeur_e2m1x2`(c)*, la grandeur doublée du code* c *: un entier de* $\{0,1,2,3,4,6,8,12\}$*, de signe donné par le bit 3, si bien que* $v=-0$ *pour le code 8 et* $+0$ *pour le code 0. Si les entrées* x *sont finies, remplacer le code 8 par le code 0 ne change aucun bit de* s *à aucun pas.*

Démonstration. On suit les deux calculs en parallèle, l'original et le canonique, et l'on montre par récurrence sur les pas que s y est égal au bit et ne vaut jamais -0.

(0) Une grille. Tout binaire32 fini est un multiple entier de 2^{-149} (le plus petit sous-normal). v est un entier ; donc le produit exact $v\cdot x$ est un multiple de 2^{-149}, et la somme exacte $v\cdot x+s$ aussi. Une telle somme, si elle n'est pas nulle, vaut au moins 2^{-149} en valeur absolue ; son arrondi au plus près n'est

donc pas nul (sous le seuil des normaux, tous les multiples de 2^{-149} sont représentables ; au-dessus, l'arrondi au plus près ne rend jamais zéro).

(1) L'accumulateur ne vaut jamais -0. Au départ, $s = +0$. À un pas, fmaf calcule $v \cdot x + s$ exactement et l'arrondit une fois, $s' = \mathrm{RN}(v \cdot x + s)$. Si la somme exacte est nulle, IEEE 754 rend $+0$ en arrondi au plus près, sauf si les deux termes valent tous deux -0 [72] ; or $s \neq -0$ par hypothèse de récurrence, donc $s' = +0$. Si la somme exacte n'est pas nulle, $s' \neq 0$ par (0). Dans les deux cas, $s' \neq -0$.

(2) Un pas sur un code nul est le même dans les deux calculs. Pour $v = \pm 0$ et x fini, $v \cdot x = \pm 0$ exactement. Si $s \neq 0$, $s' = \mathrm{RN}(\pm 0 + s) = s$. Si $s = 0$, alors $s = +0$ par (1), et $s' = +0$ quel que soit le signe de $v \cdot x$, puisque $(-0) + (+0) = +0$ et $(+0) + (+0) = +0$. Dans les deux calculs, $s' = s$.

(3) Les autres pas ont les mêmes opérandes. Un code non nul est le même dans les deux calculs ; x et s sont égaux au bit par récurrence ; fmaf est une fonction de ses opérandes. Donc s' est égal au bit.

Par récurrence sur les pas, s est égal au bit dans les deux calculs à chaque pas. ■

La réduction papillon entre voies (`somme_warp_f32`) additionne ensuite des sommes partielles égales au bit, dans le même ordre ; l'échelle du bloc et le biais sont appliqués après. Conséquence : la sortie de chaque GEMV d'expert, donc chaque logit, est identique au bit. L'argument (0) précise celui de la synthèse (« $|v| \geq 1$ pour un code non nul, donc $|v \cdot x| \geq |x|$ ne s'arrondit pas à zéro ») : ce qui empêche une somme non nulle de s'arrondir à zéro est que tout reste sur la grille $2^{-149}\mathbb{Z}$, et cela tient parce que les grandeurs sont *doublées*, donc entières, et que l'échelle n'entre pas dans l'accumulation. La proposition de l'éclaireur (chapitre 3) laissait le signe d'une somme partielle nulle ; le départ à $+0$ du noyau réel le règle. Le chapitre 9 la mesure sur la carte et sur les logits.

La table des valeurs E2M1, un invariant. Elle est lue en un seul endroit des noyaux, la fonction `valeur_e2m1x2` ; la référence f64 lit le GGUF et n'est pas touchée. Tout nouveau lecteur de codes (un noyau de préremplissage par lots, une GEMM) doit lire le code 8 comme zéro : c'est l'invariant que le lot A a écrit dans l'architecture (§ 5.6).

Le décodeur et la LUT compacte. Si le contexte de colonne entrait dans le format, il faudrait un décodeur à douze contextes par symbole. La LUT normative d'une table occupe $M = 4\,096$ mots de 32 bits, 16 Kio ; CODES_CTX en monte trois par bloc de fils, 48 Kio, la limite de la mémoire partagée statique d'un bloc. Douze contextes demanderaient $12 \times 16 = 192$ Kio, au-delà même des 99 Kio de mémoire partagée dynamique d'un bloc sur sm_89 (guide de programmation CUDA, lu, non relevé sur la carte) [litt.]. En géométrie 2, le symbole tient sur 4 bits ; pour chaque contexte c on range $\mathrm{sym}[c]$, 4 096 symboles deux par octet (2 Kio), et $f[c][16]$, $\mathrm{cum}[c][16]$ en u16 (64 o). Un pas de décodage devient

$$\begin{aligned} \text{fente} &\leftarrow x \,\&\, 4095, \\ s &\leftarrow \big(\mathrm{sym}[c][\text{fente} \gg 1] \gg 4\,(\text{fente} \,\&\, 1)\big) \,\&\, 15, \\ y &\leftarrow f[c][s]\,(x \gg 12) + \text{fente} - \mathrm{cum}[c][s], \end{aligned}$$

puis le renouvellement habituel ($x \leftarrow (y \ll 16) \mid \mathrm{mots}[p]$ si $y < L$, sinon $x \leftarrow y$). Pour 12 contextes : $12 \times (2048 + 64) = 25\,344$ octets, plus la carte (720 o), soit $\approx 25{,}5$ Kio, sous la limite statique [calc.].

Lemme 8.10 (A2 : la LUT compacte rend la même suite)**.** *Pour toute fente* $i \in [0, 4096)$*, la LUT normative donne* $(s, f[s], i - c[s])$ *avec* s *l'unique symbole tel que* $c[s] \leq i < c[s] + f[s]$ *; la LUT compacte range le même* s *en* $\mathrm{sym}[c][i]$ *et lit* $f[s]$ *et* $c[s]$ *dans les tables du même contexte. Les deux pas calculent donc le même* y *et le même renouvellement, et la suite des états, des mots lus et des symboles est la même.*

Démonstration. Les deux LUT sont construites depuis les mêmes fréquences f et cumuls c du contexte ; la normative stocke en i le triplet $(s, f[s], i - c[s])$, la compacte stocke s en i et laisse $f[s]$ et $c[s]$ dans deux petites tables. À état x égal, les deux lisent la même fente $i = x \bmod 4096$, en tirent le même s, et calculent $y = f[s]\lfloor x/4096 \rfloor + (i - c[s])$ par les mêmes valeurs. Le renouvellement ne dépend que de y et du pointeur. Par récurrence sur les pas, états, pointeurs et symboles coïncident. La LUT est une représentation de la table : ce qui est normatif, c'est la suite des symboles décodés. ■

Le coût, posé. Un pas compact fait deux lectures de plus en mémoire partagée et une soustraction de plus qu'un pas de géométrie 1 ; la géométrie 2 fait 8 pas par mot au lieu de 4. Une matrice CODES_COL coûte donc environ 2,5 fois une matrice CODES_CTX [calc.]. Le moteur décode un expert en 0,92 ms (54,2 ms par jeton pour 59 défauts sur le 120b, lot O). Si gate et up d'une seule couche passent en CODES_COL, un expert de cette couche coûte $0{,}92 \times (1 + 2 \times 2{,}5)/3 \approx 1{,}8$ ms ; à $59/36 \approx 1{,}6$ défaut par jeton dans cette couche, c'est $\approx +1{,}5$ ms sur 457 ms par jeton, 0,3 % [calc.], et le décodage se recouvre déjà avec les lectures.

Les critères du lot A, déclarés avant. Ils sont au chapitre 9 avec leurs mesures ; on retient ici leur forme. A-T1 à A-T4 : refus nommés du format, codec et décodeur octet pour octet, empreintes à 100 %. A-C1 : 0 logit différent entre v1.2 et v1.0 sur 10 fenêtres, 20b et 120b (la proposition 8.9) ; la cassure fait différer. A-C2 : gain sur les octets de tronçons d'au moins 0,12 bit par poids d'expert sur chacun des deux modèles (prédit 0,1280 et 0,1257 ; 0,12 laisse 0,0013 bit d'alignement et une marge). A-C3 : pas de régression de vitesse en série ABBA, $r \geq 1 - 2\sigma_{\text{bruit}}$, et un gain n'est annoncé que s'il dépasse $2\sigma_{\text{bruit}}$. A-C4 : le débit de décodage de CODES_COL au moins 10 fois celui du disque. Si A-C4 tombait, la colonne se retirerait et le zéro seul rendrait déjà 86 % du gain du lot sur les 12 tenseurs mesurés (0,1370 sur 0,1591 bit) [calc.].

8.7.2 Lot B : le génome synonyme, et la mesure qui manquait d'abord

Le lot B porte le génome synonyme comme second fichier, `<nom>.rapide.trc`, lu à la place du fichier exact quand le moteur sert un expert en *mode rapide*. Il n'est pas emboîté dans le fichier exact : le complément exact derrière la base synonyme coûterait 1,689 bit au lieu de 1,219 (chapitre 3). C'est le « mode rapide et mode exact » de l'idée de Stone. Le format du compagnon et le codeur sont au chapitre 10 ; on garde ici ce qui décide de sa construction.

La mesure qui manquait. La variante qui bat la frontière en bits ($\lambda = 1$, 2,563 bits H_0) n'avait pas de KL ; celle qui avait une KL meilleure ($\lambda = 0$) n'était pas sous la frontière en bits (section 8.2). Le lot B commence donc par **B0** : la passe KL de l'éclaireur, protocole P1, sur la variante à construire, comparée en appariée à la base frontière. Le critère, déclaré avant : $\lambda = 1$ est retenue seule si elle est meilleure significativement ; égale, elle domine encore la frontière (0,078 bit de moins à KL égale), et les deux variantes sont construites comme deux points du mode rapide, que **B2** départage sur le modèle entier ; pire, $\lambda = 0$ est retenue. Le chapitre 10 et le juge du chapitre 11 disent ce que B0 et B2 ont décidé.

Ce qu'il rendrait. Avec $\lambda = 1$, 2,56 à 2,60 bits par poids lus, le compagnon pèserait $\approx$ 6,14 à 6,24 Go (20b) et $\approx$ 36,9 à 37,4 Go (120b), soit un rapport $f \approx 0{,}69$ à 0,71 des octets d'un tronçon rapide à un tronçon exact v1.2, et un toit de vitesse de ×1,41 à ×1,46 à disque limitant. Il tient en RAM sur le 20b (6,2 + 1,9 Go de résidents sous 16 Go), pas sur le 120b [calc.] sur [mes.] (`genome/05-synthese.md` § 4.7).

Les deux usages. *Les défauts de cache* : une politique « tout défaut en rapide » converge vers le mode rapide pur ; la forme qui garde un sens est à deux niveaux, un expert froid servi en rapide et relu en exact une fois promu. Sur le 120b, le relevé du moteur est de 59 défauts pour 144 appels par jeton, donc une part d'appels servis en rapide $\varphi \le 59/144 = 41\,\%$ au plus fort de la politique [calc.]. *Le brouillon spéculatif* [117] : la vérification de $k+1$ positions par le modèle exact rend le texte exact au bit ; le brouillon ne fait que proposer. Le cahier en a tiré un lemme, qui dit où il paie.

Lemme 8.11 (B2 : le brouillon doit être résident)**.** *En régime limité par le disque, soient b_e les octets d'experts exacts lus par jeton servi simplement, b_d les octets que le brouillon lit sur le disque par jeton proposé, α le taux d'acceptation, et $U(2) = 2 - \rho$ le rapport des octets exacts lus par une passe de vérification de deux positions à ceux d'une seule ($\rho \in [0,1]$: la part des défauts de la seconde position déjà payée par la première). Avec un jeton proposé ($k = 1$), le décodage spéculatif lit moins d'octets par jeton rendu que le décodage simple si et seulement si*

$$\alpha > 1 - \rho + b_d/b_e. \tag{8.9}$$

Démonstration. Un cycle lit b_d octets pour le brouillon et $(2-\rho)\,b_e$ pour la vérification ; il rend en moyenne $1+\alpha$ jetons (le jeton de la vérification, plus le jeton proposé s'il est accepté, avec probabilité α). Le décodage simple lit b_e par jeton. Le cycle gagne si et seulement si $b_d + (2-\rho)\,b_e < (1+\alpha)\,b_e$; en divisant par $b_e > 0$, $b_d/b_e + 2 - \rho < 1 + \alpha$, soit $\alpha > 1 - \rho + b_d/b_e$. ■

Corollaire 8.12 (Ce que la condition coûte)**.** *(i) Comme $\alpha \le 1$, il faut $\rho > b_d/b_e$: un brouillon lu du disque doit être payé par la réutilisation des experts entre positions voisines.*

(ii) Brouillon lu au défaut depuis `<nom>.rapide.trc` : $b_d/b_e \approx f \approx 0{,}7$ (mêmes défauts, tronçons 0,7 fois plus petits) ; il faut $\rho > 0{,}7 + (1-\alpha)$, soit $\rho > 0{,}8$ pour $\alpha = 0{,}9$. Avec un routage indépendant, l'union des experts de deux jetons du 120b pèse $2 - t/E = 2 - 4/128 = 1{,}969$ fois celle d'un seul, donc $\rho \approx 0{,}03$, loin de la condition. Le ρ réel, au cache du moteur, n'est pas mesuré.

(iii) Brouillon résident en RAM ($b_d = 0$) : la condition devient $\alpha > 1 - \rho$, celle de l'idée de Stone. Sur le 120b, le fichier rapide ($\approx$ 37 Go) ne tient pas dans 16 Go ; le seul brouillon résident serait un autre modèle, le 20b en mode rapide (même vocabulaire), dont l'accord avec le 120b n'est pas mesuré.

(iv) Le lemme compte des octets, pas le temps. Le moteur ne tire du disque que 1,55 Go/s utiles sur 1,848 parce qu'il a moins de quatre lectures en vol par couche ; vérifier deux positions double la file. Ce gain-là est borné par $1{,}848/1{,}55 = \times 1{,}19$ et ne dépend pas du brouillon [calc.] sur [mes.].

Démonstration. (i) Si $\rho \le b_d/b_e$, le membre droit de (8.9) vaut au moins 1, et $\alpha \le 1$ ne peut le dépasser. (ii) On remplace b_d/b_e par 0,7 dans (8.9) : $\rho > 1 - \alpha + 0{,}7$. Avec $k = 4$ experts sur $E = 128$ tirés indépendamment pour chaque jeton, un expert de la seconde position est déjà chargé avec probabilité k/E ; l'union compte en moyenne $k + k(1 - k/E)$ experts, soit $2 - k/E$ fois k, et $\rho = k/E = 0{,}03$. (iii) $b_d = 0$ dans (8.9). (iv) Le rapport des débits est un rapport mesuré, indépendant de α. ■

Le lemme 8.11 se lit en une phrase : sur une machine où le disque limite, un brouillon qui lit le disque ne paie presque jamais. La figure qu'elle mérite, le plan (ρ, α) avec la droite (8.9) pour trois valeurs de b_d/b_e (III-7.5 de l'inventaire), n'est pas tracée ; la condition la dit. Le pas B5 du cahier, le brouillon, *ne se construit pas* avant la mesure de ρ sur les traces de routage et de α par l'accord glouton rapide contre exact.

Les pas du lot B, déclarés avant. B0 (la KL de $\lambda = 1$, appariée) ; B1 (le format du compagnon, symboles décodés égaux aux symboles écrits) ; B1c (le codeur sur la carte, égal au codeur Python sur 8 matrices tirées) ; B2 (la KL du modèle 20b entier en mode rapide, meilleure que la frontière

entière, et au niveau de la meilleure requantification de llama.cpp à bits égaux ou inférieurs) ; B3 (la politique à deux niveaux, rejouée sur les traces avant d'être construite) ; B4 (le 120b) ; B5 (le brouillon, sous la condition du lemme 8.11). Le repère de B2 (ii) est une réponse par délégation du 2026-10-17, réversible : la meilleure requantification que llama.cpp produit du même fichier, parce que c'est l'outil de référence que Stone utiliserait sans troncons, gratuit et déjà sur la machine.

8.7.3 Les pistes que les mesures désignent déjà

Aucune des pistes suivantes n'est dans les lots ; chacune porte ce qui est mesuré et la mesure qui la jugerait.

La garde tropicale. La seule piste que les mesures désignent : les bascules dominent la KL entre deux calculs proches, et la marge prédit la bascule (médiane 0,083 sur toutes les paires, 0,0095 sur celles qui basculent). La construction qu'elle suggère : en mode rapide, après les experts rapides de la couche ℓ, regarder la marge du routeur de la couche $\ell + 1$, et refaire la couche ℓ en exact pour les positions sous un seuil τ. Le coût est la part des cellules sous τ, le gain la part des bascules évitées ; les deux se lisent sur les journaux de routage du lot R, sans passe neuve du modèle. Le seuil de décision, déclaré dans le cahier avant toute lecture : la garde vaut d'être construite si elle attrape au moins 80 % des bascules pour au plus 10 % des cellules. Le chapitre 12 la juge.

L'expert pont. Servir un expert manquant par son meilleur paralogue laisse 79, 84, 66 et 33 % de l'énergie de sa sortie en erreur (couches 0, 8, 16, 23), et ~~substituer n'est jamais meilleur que retirer l'expert~~ ; corrigé au chapitre 5 : retirer l'expert vaut mieux que le substituer aux couches 0 et 8 ; aux couches 16 et 23, le meilleur paralogue fait un peu mieux que le retrait pour la majorité des experts, d'au plus un demi-point d'erreur de couche en moyenne (`genome/duplication.md` § 0, point 6, et `genome/05-synthese.md` § 3.2 barrent la même phrase). La barre d'un pont n'est donc pas l'expert absent, c'est le retrait. Rien n'en est mesuré au-delà.

La fractale. Les données de la ronde ne donnaient la part d'énergie qu'à sept rangs par tenseur : trop peu pour une pente. La littérature des spectres à queue lourde [118] donne l'hypothèse à mesurer [litt.] ; le chapitre 13 la mesure. La compression par systèmes de fonctions itérées a le pronostic de la graine, pour la même raison (propriété (a)). Une courbe de Hilbert pour ranger les experts sur le disque ne compte presque pas à la granularité d'un tronçon de 12 Mo [calc.] sur [mes.].

8.8 Ce qui pourrait être faux

Ce qui pourrait être faux

- **Trois protocoles, un seul axe.** Les rapports de KL rapprochent des passes qui n'ont ni les mêmes experts remplacés, ni les mêmes fenêtres, ni le même nombre de positions (765 à 2 550). Un rapport n'est pas invariant au nombre d'experts remplacés : la KL croît avec lui, et pas linéairement, à cause des bascules. La figure 8.1 les montre côte à côte ; elle ne les fond pas en une courbe, et elle garde 32 experts sur 768 pour P1 (réserve R3).
- **Les débits n'ont pas tous le même estimateur** (codec réel, H_0 sans échelles, comptabilité du brin). Entre protocoles, un écart de 0,04 bit peut venir de l'estimateur seul.
- **L'incertitude par fenêtres** repose sur 10 blocs, 9 degrés de liberté. Le verdict de la mutation synonyme (2,9 σ) tient par les deux estimateurs ; une autre découpe pourrait l'élargir.

- **La lecture « part lisse et queue de bascules »** est une hypothèse de travail (hypothèse 8.5). Elle s'appuie sur des mesures directes entre deux moteurs exacts et sur les quantiles de la ronde ; elle n'a pas été mesurée pour les mécanismes de la ronde par le routage imposé. La proposition 8.3, elle, est un calcul exact ; ce qui est hypothèse, c'est la séparation de la KL observée en ses deux termes.
- S^* **égal en KL** repose sur la passe K10, déclarée avant par l'équipe du génome, mais que la synthèse a d'abord recopiée à la main d'un fichier non encore commité ; `frontiere.py --k10` la tire maintenant du journal commité, et ses chiffres sont ceux de ce chapitre. L'explication de sa queue (pondération diagonale dans le domaine tourné) est une hypothèse qu'aucune passe ne départage.
- **La proposition 8.7** précise une phrase de la synthèse ; la marge des journaux est en unités de logit, et la distance euclidienne n'en est une image fidèle que si les normes $\|w_i - w_j\|$ varient peu d'une paire à l'autre, ce qui n'est pas mesuré ici.
- **La figure 8.5** est une projection : 21,0 % de la variance pour l'ACP ; la t-SNE garde les voisinages ($T = 0{,}947$), pas les distances. Elle colore par l'expert premier, et la marge des quatre experts servis n'y marque pas les bords : elle illustre la géométrie du routeur, elle ne prouve pas que les bascules d'un ensemble de quatre s'y voient.
- **Le contexte de colonne** n'est mesuré que sur 12 tenseurs du 20b (4 couches) ; sur le 120b et les autres couches, son gain peut être plus grand ou nul. Le chiffre de gate de la couche 0 dépend de la référence (0,1416 contre le flux octet, 0,1454 contre le flux demi-octet) ; la synthèse les mêlait.
- **La proposition 8.9** couvre le noyau tel qu'il est au commit `000cef9` ; elle repose sur des grandeurs doublées entières et sur une échelle appliquée hors de l'accumulation (la grille de l'argument (0)). Un noyau qui multiplierait l'échelle dans l'accumulation, ou une GEMM sur des poids déquantifiés partant d'un -0, sortirait de la démonstration ; l'invariant de l'architecture le dit.
- **La limite de mémoire partagée** (99 Kio sur option, 48 Kio en statique) est lue dans la documentation de NVIDIA, pas relevée sur la carte ; la LUT compacte tient sous 48 Kio, ce qui rend la spécification indépendante de ce chiffre.
- **Le lemme 8.11** suppose α indépendant de la position et ρ constant ; il compte des octets, pas le temps ; ses chiffres de ρ sont ceux d'un routage indépendant, pas mesurés au cache.
- **Toutes les mesures de la ronde sont sur wiki.test**, calibration et évaluation sur des fenêtres disjointes du même corpus ; sur du code ou du dialogue, le gain de la mutation synonyme peut baisser.
- **Les chiffres du lot R** cités ici (bascules, 88 %) sont lus dans la synthèse, qui les cite ; ce chapitre ne les a pas refaits depuis les journaux du Tome II.

Deuxième partie

La construction

Chapitre 9

Un codon synonyme dans le format (lot A, v1.2)

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, le moteur du Tome II lisait le format `.trc` v1.0 (un niveau, flux CODES_CTX à table par voie) ; les deux fichiers de la machine étaient en v1.0, 11 157 405 696 octets pour gpt-oss-20b et 57 730 662 400 pour gpt-oss-120b. L'équipe du codage prédictif avait mesuré, par des flux rANS réels décodés, que fondre le demi-octet 8 (−0) dans le demi-octet 0 fait passer le 120b de 3,8549 à 3,7292 bits par poids d'expert et le 20b de 3,8509 à 3,7230 (chapitre 6). La synthèse avait démontré que la GEMV du moteur rend les mêmes bits sur les deux codes (proposition 8.9). **Ce chapitre établit** le gain exact du fondu, démontré puis vérifié tenseur par tenseur sur 180 tenseurs ; que tous les lecteurs de codes du projet lisent le code 8 comme 0 ; le format v1.2 au bit près et le lemme qui en garantit l'invariant ; les deux fichiers recodés, exacts en valeur expert par expert, 1 800 486 912 octets de moins sur le 120b ; des logits identiques au bit ; et le jalon J1 refait sur le 120b, ×1,454 en génération, avec ce qui l'explique sur ce disque : le motif des requêtes de lecture que le format impose. **Il reste** à isoler la variable exacte du motif, que personne n'a encore trouvée, et à remesurer le gain sur tout autre disque. Sources : `genome/06-codon-v1-2.md`, `genome/construction-codon-pour-le-memoire.md`, `genome/JUGEMENT-construction.md`, `genome/JUGEMENT-placement.md` ; mesures sous `mesures/genome/construction/codon/`, `mesures/genome/construction/juge/`, `mesures/disque/placement/juge/seance/`.

En bref

Dans le code génétique, plusieurs codons de trois lettres donnent le même acide aminé : GGU, GGC, GGA et GGG donnent tous la glycine. Ces codons sont dits synonymes, et la cellule lit l'un comme l'autre [4, 88]. L'image n'est qu'une analogie, mais elle tombe juste ici. Un poids de gpt-oss tient sur 4 bits : un bit de signe et trois bits de grandeur, seize codes pour quinze valeurs, parce que +0 et −0 sont deux codes du même zéro. Le quantificateur d'origine a gardé le signe des valeurs arrondies à zéro ; ce signe est une pièce de monnaie parfaitement juste, et aucun calcul ne le lit. Le stocker coûte pourtant presque un bit par zéro, et un poids sur huit est un zéro. Le lot A le retire : le codeur réécrit chaque −0 en +0 avant le codage entropique. Le gain se prédit exactement (deux fois la part des −0) et se vérifie à deux dix-millièmes de bit près sur chaque tenseur. Les logits ne changent pas d'un bit, et la preuve en est écrite sur le noyau réel. Le fichier du 120b perd 1,80 Go, et, sur cette machine, le moteur va 1,45 fois plus vite. Ce dernier chiffre n'est pas celui que les octets prédisaient (3,4 % au plus) : il vient de la façon dont le nouveau format découpe ses lectures sur ce disque, et ne vaut que pour lui.

9.1 Seize codes pour quinze valeurs

Le code E2M1. MXFP4 range les poids par blocs de 32 qui partagent une échelle E8M0, 2^{e-127} ; chaque poids est un code de 4 bits au format E2M1 : un bit de signe, deux bits d'exposant, un bit de mantisse [7, 8]. Ses huit grandeurs sont 0, 0,5, 1, 1,5, 2, 3, 4, 6. llama.cpp et le moteur les *doublent*, $K = (0, 1, 2, 3, 4, 6, 8, 12)$, et passent le facteur 2 dans l'échelle, 2^{e-128}. Pour un code $c \in \{0, \dots, 15\}$, de bits $c_3c_2c_1c_0$,

$$v(c) = (-1)^{c_3}\, K[c \,\&\, 7]. \tag{9.1}$$

L'application v est injective sauf sur $\{0, 8\}$: $v(0) = +0$ et $v(8) = -0$, deux codes pour une seule valeur, puisque $-0 = +0$ en IEEE 754 [72]. Seize codes, quinze valeurs.

La pièce du signe. Le quantificateur de gpt-oss garde le signe d'une valeur arrondie à zéro. Sur les 72 tenseurs d'experts de gpt-oss-20b (19 110 297 600 poids), le code +0 a la part 6,3920 % et le code −0 la part 6,3929 % : le signe d'un zéro a une entropie de 0,9999999967 bit [mes.] (`mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`). L'entropie des codes vaut 3,8638 bits, celle des valeurs 3,7360, et la différence, 0,1278 bit par poids, est ce que coûte un signe que rien ne lit. La figure 9.1 montre les seize codes sur la droite repliée des valeurs : avant, deux rayons pour le zéro ; après, un seul, qui porte les deux parts.

9.2 Le gain exact du fondu

Proposition 9.1 (L'entropie d'un code à synonyme)**.** *Soit C le code de 4 bits d'un poids, $V = v(C)$ sa valeur et $S = C_3$ son bit de signe. Alors*

$$H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0). \tag{9.2}$$

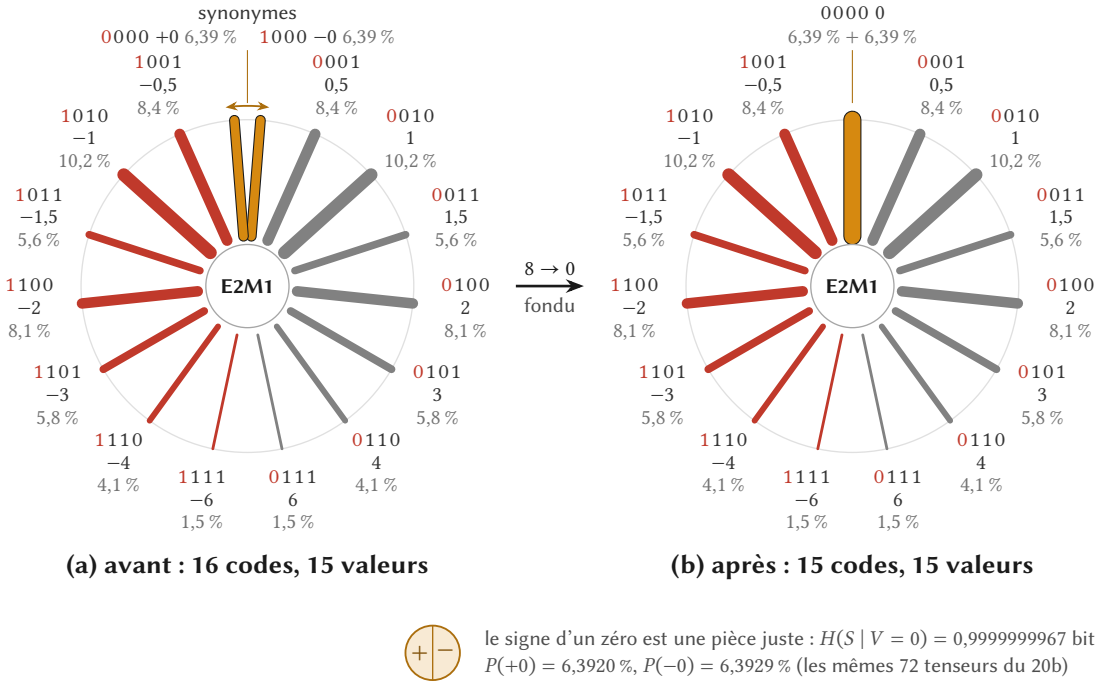
Démonstration. V est une fonction de C, donc $H(V \mid C) = 0$ et $H(C, V) = H(C)$. Par la règle de chaîne [2], $H(C, V) = H(V) + H(C \mid V)$. Sachant $V = v \neq 0$, C est déterminé, puisque v est injective hors de 0 : $H(C \mid V = v) = 0$. Sachant $V = 0$, $C \in \{0, 8\}$, et C est déterminé par S : $H(C \mid V = 0) = H(S \mid V = 0)$. D'où $H(C \mid V) = \sum_v \mathbb{P}(V = v)\, H(C \mid V = v) = \mathbb{P}(V = 0)\, H(S \mid V = 0)$, et $H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0)$. ∎

Corollaire 9.2 (Le gain attendu par poids)**.** *Fondre 8 dans 0 remplace C par $C' = \gamma(C)$, avec $\gamma(8) = 0$ et γ l'identité ailleurs ; C' est en bijection avec V, donc $H(C') = H(V)$. Le gain d'entropie est $\mathbb{P}(V = 0)\, H(S \mid V = 0)$ bit par poids. Quand le signe d'un zéro est une pièce juste, $H(S \mid V = 0) = 1$, les -0 sont la moitié des zéros, et*

$$\textit{gain} = \mathbb{P}(V = 0) = 2 \times \textit{(part des } -0\textit{)}. \tag{9.3}$$

Le résultat vaut conditionnellement à tout contexte X connu du décodeur (classe d'échelle κ, octet voisin) : on remplace partout $H(\cdot)$ par $H(\cdot \mid X)$, et la démonstration ne change pas.

Remarque 9.3 (L'alphabet octet). Le flux CODES_CTX ne code pas des demi-octets mais des octets, deux codes à la fois, avec une table par classe d'échelle. La même démonstration, appliquée à l'octet $O = (C_{\text{bas}}, C_{\text{haut}})$ et à sa valeur $(V_{\text{bas}}, V_{\text{haut}})$, donne un gain de $\mathbb{P}(V_{\text{bas}} = 0)\, H(S_{\text{bas}} \mid \cdot) + \mathbb{P}(V_{\text{haut}} = 0)\, H(S_{\text{haut}} \mid \cdot)$ bits par octet si les signes des zéros sont indépendants de tout le reste, soit $\mathbb{P}(V = 0)$ bit par poids. Le fondu retire de l'alphabet les octets dont un demi-octet vaut 8 : 16 octets de demi-octet fort 8, 16 de demi-octet faible 8, dont un commun (0x88), soit $16 + 16 - 1 = 31$ octets ; il en reste $256 - 31 = 225$ [calc.].



(a) avant : 16 codes, 15 valeurs **(b) après : 15 codes, 15 valeurs**

le signe d'un zéro est une pièce juste : $H(S \mid V = 0) = 0{,}9999999967$ bit
$P(+0) = 6{,}3920\,\%$, $P(-0) = 6{,}3929\,\%$ (les mêmes 72 tenseurs du 20b)

Figure 9.1 : Le fondu 8 → 0 retire l'un des deux codes du zéro : seize codes pour quinze valeurs, puis quinze codes. [mes.] Les quinze valeurs d'E2M1 sont rangées à pas égal (24°) sur un cercle : 0 en haut, valeurs positives dans le sens horaire, ±6 voisins en bas. Ce sont les valeurs de la norme OCP ; la table de llama.cpp les double, le facteur 2 passant dans l'échelle. Le bit rouge est le bit de signe. L'épaisseur d'un rayon croît avec la part mesurée de son code dans les 72 tenseurs d'experts de gpt-oss-20b (19 110 297 600 poids) ; elle est affine en la part (0,4 pt + 44 pt × part) : le rayon du zéro s'épaissit donc de × 1,88 au fondu, et non de × 2. Avant : deux codes, `0000` (+0) et `1000` (−0), pour une valeur ; après : quinze codes, le rayon du zéro porte les deux parts (liseré d'encre). Le compte de `1000` égale celui des −0 fondus par le codeur du lot A, un second outil. La pièce est mesurée sur les mêmes tenseurs : rien ne prédit le signe d'un zéro, et le codeur rANS récupère presque un bit entier par zéro. Analogie, non preuve : les codons synonymes du code génétique (GGU, GGC, GGA, GGG donnent tous la glycine). Ni les classes de contexte κ ni l'alphabet octet du codeur ne sont dessinés ; leur effet sur le gain est l'objet de la figure 9.2.

Données : `mesures/genome/predictif-elagage/histo-toutes-gptoss20b.npz`, `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json` ; témoin `mesures/genome/construction/codon/tailles.json` ; script `memoire/outils/tome3_codon.py`.

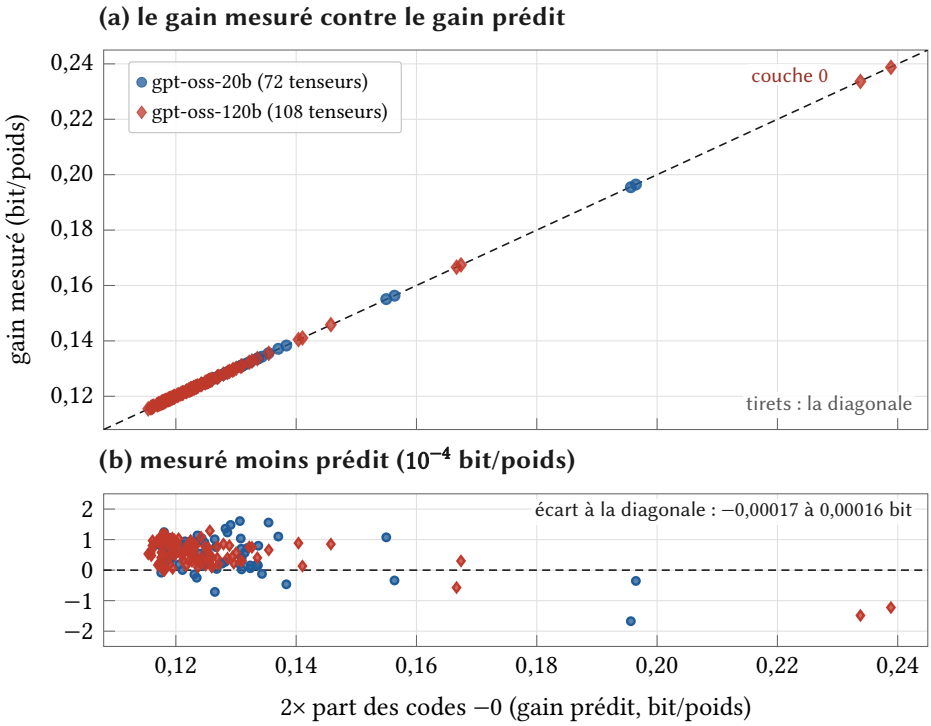


Figure 9.2 : Les 180 tenseurs d'experts des deux modèles tombent sur la diagonale que prédit le corollaire. [mes.] [calc.] (a) Gain mesuré du fondu (bits par poids d'expert, tronçons codés) contre deux fois la part des codes −0. Sous une pièce juste, les −0 sont la moitié des zéros, et le corollaire prédit la diagonale (tirets) ; les deux points de la couche 0 du 120b sont les plus riches en zéros. (b) Les mêmes points, mesuré moins prédit, en 10^{-4} bit par poids : le contexte κ et l'alphabet octet du codeur n'écartent aucun tenseur de plus de −0,00017 à 0,00016 bit.

Données : `mesures/genome/construction/codon/tailles-par-tenseur.csv` ; script `memoire/outils/tome3_codon.py`.

Le chiffre attendu, et le chiffre mesuré. Le codeur du lot A a fondu 1 221 698 587 demi-octets 8 sur le 20b (6,393 % des poids d'experts) et 7 199 491 948 sur le 120b (6,279 %) ; la vérification, qui canonise les tranches du GGUF par son propre chemin, compte les mêmes nombres à l'unité [mes.] (`tailles.json`). Le corollaire prédit donc [calc.]

$$2 \times 0{,}0639288 = 0{,}12786 \text{ bit (20b)}, \qquad 2 \times 0{,}0627889 = 0{,}12558 \text{ bit (120b)},$$

et le gain mesuré sur les tronçons codés vaut 0,12791 et 0,12562 bit (section 9.6). L'écart, quatre à cinq cent-millièmes de bit, est une miette d'arrondi des tables : 225 symboles au lieu de 256 à normaliser sur 4 096 fentes.

Tenseur par tenseur. La figure 9.2 refait la comparaison sur chacun des 180 tenseurs d'experts des deux modèles (72 du 20b, 108 du 120b). L'écart entre le gain mesuré et deux fois la part des −0 reste entre −0,00017 et +0,00016 bit, en moyenne +0,00005 [calc.] sur [mes.] (`tailles-par-tenseur.csv`). C'est un résultat plus fort qu'il n'y paraît : si le signe d'un zéro était prédit par la classe d'échelle ou par l'octet voisin, que le codeur connaît, le gain mesuré serait plus petit que $\mathbb{P}(V = 0)$ (le corollaire conditionnel donnerait $\mathbb{P}(V = 0)\, H(S \mid V = 0, X) < \mathbb{P}(V = 0)$). Les 180 points sur la diagonale disent que, dans chaque tenseur, le signe d'un zéro est une pièce juste que rien ne prédit, et que le codeur rANS [119, 120] en récupère le bit entier.

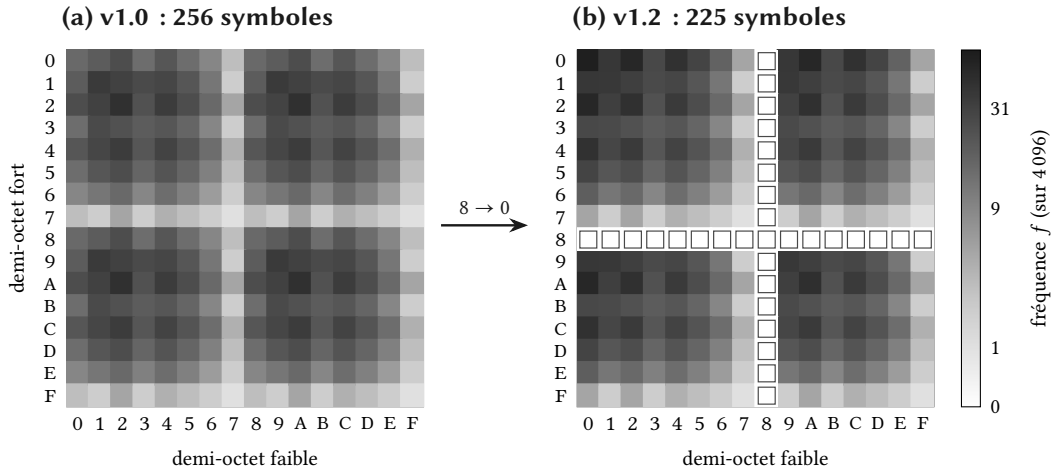


Figure 9.3 : Après le fondu, la ligne et la colonne 8 s'éteignent : il reste 225 octets sur 256. [mes.] La table CODES_CTX ($\kappa = 1$) de gpt-oss-120b, couche 18, matrice up, en grille 16×16 (demi-octet fort en ligne, faible en colonne) ; niveau de gris $\log_{10}(1 + f)$, f la fréquence sur 4 096 (échelle à droite), comme une densité. Cases blanches bordées d'encre : les octets de fréquence nulle, absents. Leur masse passe sur la ligne et la colonne 0. Les lignes 7 et F sont pâles dans les deux panneaux parce que les codes ±6 (±12 dans la table de llama.cpp) sont rares : pâle veut dire rare, blanc bordé veut dire absent. Une table d'une couche ; les autres sont dans le même fichier.

Données : `mesures/genome/construction/codon/tables-octets.csv` ; script `memoire/outils/tome3_codon.py`.

Où sont les zéros. Le gain va de 0,115 bit (120b, couche 26, up) à 0,239 bit (120b, couche 0, up, où 11,9 % des poids sont des −0, donc environ 24 % des zéros) ; la couche 0 est partout la plus riche en zéros (20b : 0,196 bit en gate et en up), puis la couche 1 ; les moyennes par matrice (gate, up, down) sont égales à 0,002 bit près [mes.]. La couche 0 est aussi celle où l'état caché porte des canaux aberrants et où le contexte de colonne rendait ses 0,14 bit (section 8.3.2). Une explication les relie, qui n'est pas testée : une échelle de bloc tirée vers le haut par une colonne géante écrase vers zéro les autres poids du bloc.

Ce que le fondu fait aux tables. La figure 9.3 montre une table CODES_CTX (120b, couche 18, up, $\kappa = 1$) en grille 16×16, avant et après : la ligne et la colonne 8 s'éteignent, et leur masse passe sur la ligne et la colonne 0. Sur les 540 tables CODES_CTX des couches 0, $L/2$ et $L - 1$ des deux modèles, un second lecteur, écrit depuis la seule spécification (`tables_octets.py`), trouve : en v1.0, 540 tables sur 540 donnaient une fente aux octets portant un 8, pour une masse de 412 à 1 039 fentes sur 4 096 (13 % en moyenne) ; en v1.2, aucune [mes.] (`tables-octets.json`). Ces 13 % de fentes rendues aux autres octets sont, en bits, le gain du corollaire.

9.3 Qui lit le code 8, et pourquoi les bits ne changent pas

Le fondu n'est sans perte que si *chaque* lecteur de codes donne au code 8 la valeur 0. Le lot A a relevé tous les lecteurs du dépôt (recherche de `e2m1`, `kvalues`, `& 8`, `& 7` dans `moteur/`).

−0 et +0 ont la même valeur ; la seule question est celle des *bits* d'une somme où un terme passe de −0 à +0. La proposition 8.9 y répond pour la GEMV, en trois temps : une grille (v entier, x binaire32, donc toute somme exacte est un multiple de 2^{-149} et ne s'arrondit pas à zéro si elle n'est pas nulle) ; l'accumulateur, parti de +0, ne vaut jamais −0 ; un pas sur un code nul laisse donc l'accumulateur

Table 9.1 : Les lecteurs de codes et la valeur qu'ils donnent au code 8.

Lecteur	Où	Code 8	Statut
ggml (llama.cpp b11515)	`ggml-common.h`, l. 1124 à 1129 : `kvalues_fp4` (int8) 0, 1, 2, 3, 4, 6, 8, 12, 0, −1, … , −12	0 (entier)	lu à la source le 2026-10-17, licence MIT [litt.]
GEMV du moteur	`trc-noyaux/noyaux/gemv.cuh`, `valeur_e2m1x2` (l. 33)	−0 (f32)	seul lecteur des valeurs dans les noyaux
décodeur CUDA	`decodeur.cuh`	ne lit pas les valeurs	indifférent
référence f64	`trc-reference/src/poids.rs`, `K_MXFP4[8]` (l. 24)	−0 (f64)	lit le GGUF, pas le `.trc`
bancs et témoins	`banc_k.cu` l. 1049, `tests/carte.rs` l. 379	−0	témoins seulement

Données : `genome/06-codon-v1-2.md` § 1 ; ggml : `https://raw.githubusercontent.com/ggml-org/llama.cpp/b11515/ggml/src/ggml-common.h`, SHA-256 0061131b…, consulté le 2026-10-17.

inchangé dans les deux calculs. La figure 9.4 en dessine la machine à états, et sa mesure.

La mesure sur la carte. `trc-noyaux/tests/carte.rs::gemv_lit_le_code_8_comme_0` prend un expert synthétique aux dimensions de gpt-oss (trois matrices 2880×2880, soit $3 \times 2880 = 8640$ lignes de sortie) dont le signe est tiré pour toute grandeur, la grandeur 0 comprise : il porte 2 486 216 demi-octets 8. On le compare à sa copie canonisée, avec des x quelconques : mantisses pleines, exposants de 2^{-10} à 2^{10}, un x sur 97 à $+0$ et un sur 97 à -0 (le cas qui exerce le pas (2) de la démonstration), sommes non exactes en f32. Résultat : **8 640 lignes, 0 différente au bit**. Cassure : un code 0 de la copie, en face d'un x non nul, mis à 1, et la ligne 0 diffère [mes.] (`essais-espace-2026-10-17-a.log`).

La référence f64. `trc-reference/tests/code_8.rs` prend une matrice MXFP4 de 64×2880 (un tiers des grandeurs à 0, signe tiré) et sa copie canonisée : chaque poids déquantifié est égal en valeur, et les sorties de `Matrice::produit` (huit accumulateurs partis de $+0$, FMA, réduction en arbre fixe) sont égales au bit sur 5 vecteurs, zéros des deux signes compris ; la cassure (un code 0 changé en 1) change la sortie [mes.]. La référence lit le GGUF, elle n'est pas touchée par le format ; ce témoin établit seulement qu'elle aussi donne au code 8 la valeur de ggml.

Corollaire 9.4 (La référence f64)**.** *Dans la référence f64, chaque accumulateur part de* $+0$ *et avance par* $\mathrm{acc} \leftarrow \mathrm{RN}(d \cdot x + \mathrm{acc})$*, avec* $d = K[c] \cdot 2^{e-128}$ *le poids déquantifié. Si aucune somme exacte non nulle* $d \cdot x + \mathrm{acc}$ *ne s'arrondit à zéro, la récurrence de la proposition 8.9 s'applique en binaire64 : l'accumulateur ne vaut jamais* -0*, un pas sur un poids* ± 0 *le laisse inchangé, et la réduction finale additionne des accumulateurs égaux au bit.*

Démonstration. Les pas (1), (2) et (3) de la démonstration de la proposition 8.9 n'utilisent la grille que pour établir qu'une somme exacte non nulle ne s'arrondit pas à zéro ; c'est ici l'hypothèse. Le reste est la même récurrence. ∎

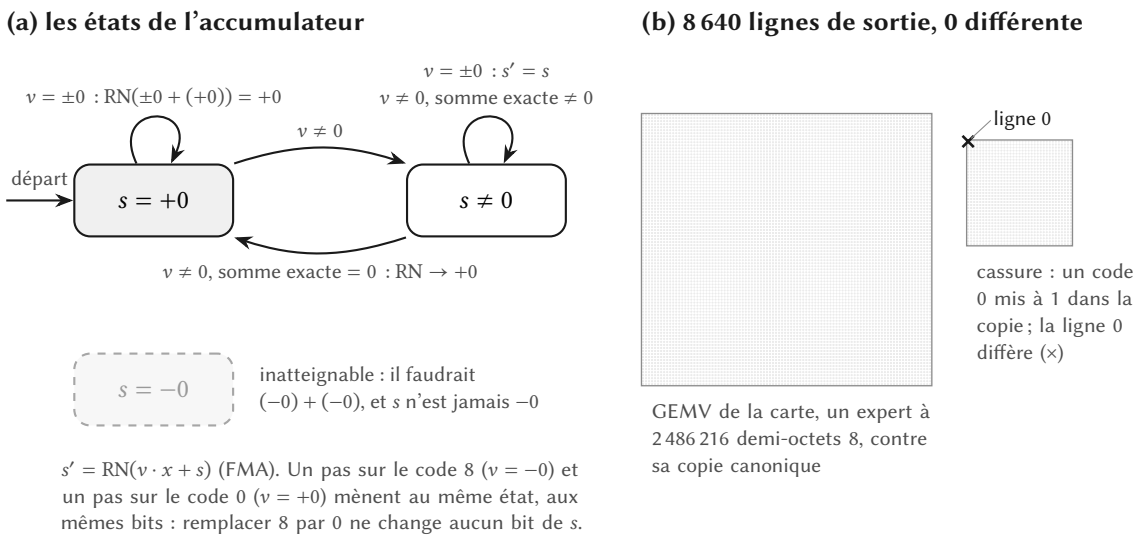


Figure 9.4 : Remplacer le code 8 par le code 0 ne change aucun bit de la GEMV, et l'instrument qui le dit sait tomber. [schéma] [mes.] Proposition A1. (a) Schéma : les états de l'accumulateur s de la GEMV du moteur, qui part de +0 et avance par $s' = \mathrm{RN}(v \cdot x + s)$ dans un ordre fixe ; −0 n'est jamais atteint, car l'arrondi au plus près d'IEEE 754 rend +0 pour une somme exacte nulle sauf si les deux termes valent −0. (b) Mesure (test `gemv_lit_le_code_8_comme_0`, ok ; comptes de 06 § 1) : un expert synthétique aux dimensions de gpt-oss (trois matrices 2 880 × 2 880), des entrées à mantisses pleines, exposants de 2^{-10} à 2^{10} et zéros des deux signes, des sommes non exactes ; une case par ligne de sortie. La petite grille est la cassure (un code 0 mis à 1 dans la copie) : la ligne 0 diffère, marquée d'une croix. La démonstration suppose des entrées finies ; avec ±∞ ou NaN, les deux sorties valent NaN.

Données : `trc-noyaux/tests/carte.rs::gemv_lit_le_code_8_comme_0`, `mesures/genome/construction/codon/essais-espace-2026-10-17-a.log` ; comptes `genome/06-codon-v1-2.md` § 1, `decisions/ARCHITECTURE.md` § 4.16.

L'hypothèse du corollaire n'est pas gratuite, et c'est une précision que ce chapitre ajoute à la note du lot A [calc.]. Dans la GEMV, l'échelle reste hors de l'accumulation et les grandeurs sont entières : la grille $2^{-149}\mathbb{Z}$ tient toujours. Dans la référence, l'échelle est déjà dans d ; un produit $d \cdot x$ plus petit que la moitié du plus petit sous-normal (2^{-1075}) s'arrondirait à un zéro signé, et l'accumulateur pourrait valoir -0. Il faudrait pour cela une activation de l'ordre de 2^{-940} devant la plus petite échelle : rien de tel n'existe dans un modèle qui rend des logits finis, et le témoin le mesure sur ses cinq vecteurs. Mais la démonstration de la référence tient sous cette hypothèse, pas sans elle.

La limite de la proposition. Elle suppose les x finis. Si un x vaut $\pm\infty$ ou NaN, $v \cdot x$ vaut NaN pour $v = \pm 0$ dans les deux calculs, et la sortie est NaN des deux côtés ; seuls le bit de signe ou la charge d'un NaN pourraient différer, ce qui n'est pas un logit utilisable. Le moteur n'a produit aucun NaN sur les 10 240 pas évalués par le lot A.

L'invariant écrit. Le lot A l'a porté dans l'architecture (§ 5.6) : tout lecteur de codes lit le code 8 comme zéro ; tout noyau neuf qui lit des codes (préremplissage par lots, GEMM) le garde, et son témoin compare un expert à sa copie canonisée.

Lemme 9.5 (L'invariant des tables suffit)**.** *Si la table d'un contexte donne $f[s] = 0$ à un symbole s, aucun flux, juste ou corrompu, ne fait produire s au décodeur normatif.*

Démonstration. Le décodeur lit la fente $i = x \bmod 4096$ de son état et rend le symbole qui la porte. Par construction de la LUT, la fente i porte le symbole s tel que $c[s] \le i < c[s] + f[s]$, avec c les fréquences cumulées. Si $f[s] = 0$, l'intervalle $[c[s], c[s])$ est vide, et aucune fente ne porte s. L'état x, quel qu'il soit, et donc quels que soient les mots lus, ne désigne qu'une fente. ■

La LUT compacte de la carte et la LUT normative du processeur sont construites depuis les mêmes fréquences, lues dans la même section TABLES ; le lemme vaut pour les deux. Il a une conséquence d'ingénierie : pour garantir qu'aucun flux v1.2 ne produit un demi-octet 8, il suffit de vérifier les tables à l'ouverture, une fois, et non chaque symbole décodé. Le contrôle coûte, pour le 120b, 36 couches × 3 matrices × 3 contextes × 31 octets = 10 044 lectures [calc.].

9.4 Le format `.trc` v1.2 au bit près

La décision est écrite dans l'architecture (§ 4.16), parce que toute modification du format passe par ce document. La consigne parlait de « v1.1 » pour les fichiers actuels : ils sont en v1.0, et la v1.1 (deux niveaux, la base de B et son complément) est réservée et n'est écrite par aucun codeur. Toutes les comparaisons opposent donc v1.2 à v1.0 ; le lecteur accepte 1.0, 1.1 (en-tête) et 1.2.

La figure 9.5 montre ces quatre champs à leur place dans le fichier, au décalage près ; le reste du fichier ne change pas.

L'expert canonique et ses deux empreintes. L'expert canonique est la suite d'octets de l'expert d'origine (pour $j = 0, 1, 2$, les $R \cdot B$ blocs de 17 octets $[e, \texttt{qs}]$ dans l'ordre β, puis les biais) où chaque demi-octet 8 des `qs` est remplacé par 0 ; les échelles et les biais ne changent pas. Le répertoire garde deux empreintes par expert : l'empreinte d'origine (décalage 24), celle des octets du GGUF, qui garde la preuve de la provenance ; et l'empreinte canonique (décalage 56), celle de ce que le décodeur doit rendre. Un fichier v1.2 n'est plus égal au GGUF au bit ; il l'est *en valeur*, et le vérificateur doit comparer en valeur, pas seulement en octets (section 9.6).

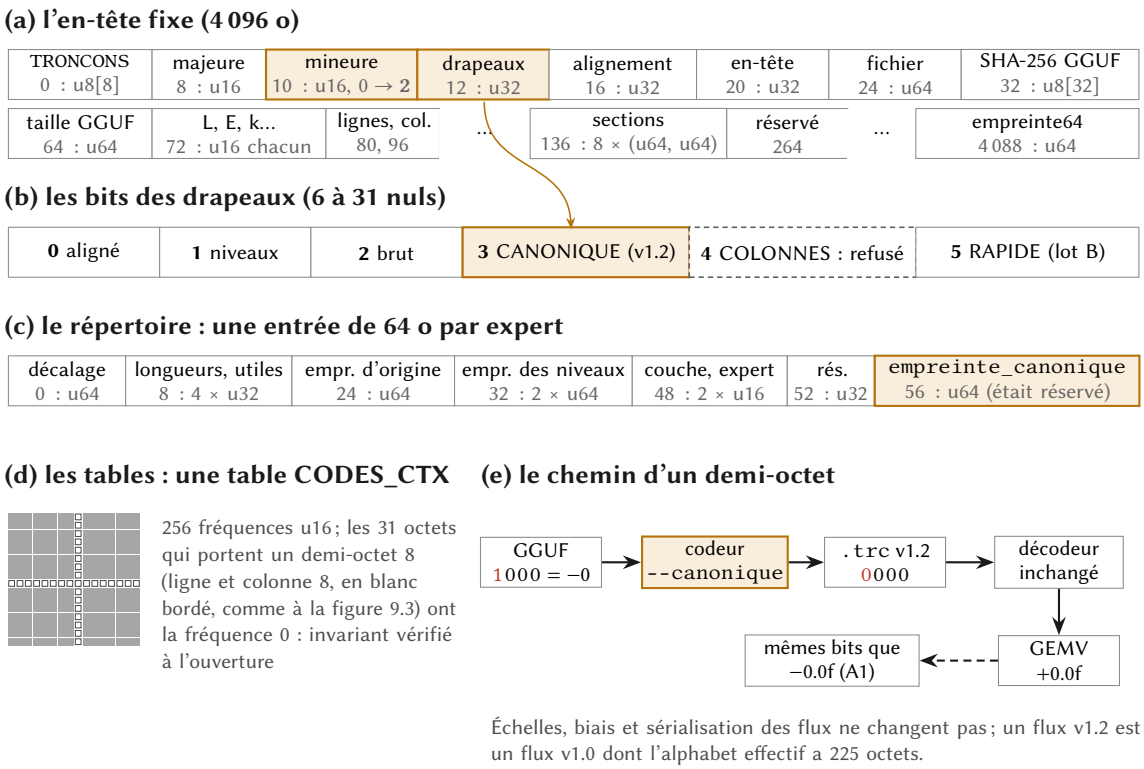


Figure 9.5 : Quatre champs changent de la v1.0 à la v1.2, surlignés de l'orange de la v1.2 ; le reste du fichier ne change pas. [schéma] Seconde ligne de chaque case : le décalage en octets, puis le type. (a) La version mineure (décalage 10) mise à 2 et (b) le bit 3 CANONIQUE du mot des drapeaux ; (c) le champ réservé du décalage 56 de chaque entrée du répertoire, qui devient l'empreinte de l'expert canonique (`empreinte_canonique` est l'identifiant du code, gardé tel quel) ; (d) les tables CODES_CTX, inchangées en forme mais bâties sur les codes canoniques. En (a), les trois tailles sont l'alignement (16), la taille de l'en-tête (20) et celle du fichier (24) ; L, E, k, dès le décalage 72, sont les couches, les experts par couche et les experts actifs ; les lignes et les colonnes des matrices sont aux décalages 80 et 96, la table des huit sections, paires (décalage, longueur), au décalage 136. Le bit 4 (le contexte de colonne du cahier) reste hors de la v1.2 et est refusé (figure 9.6) ; le bit 5 est le compagnon rapide du lot B, exclusif du bit 3. L'empreinte d'origine (décalage 24 d'une entrée) reste celle des octets du GGUF : un fichier v1.2 garde la preuve de sa provenance. Les largeurs des champs ne sont pas à l'échelle.

Données : `decisions/ARCHITECTURE.md` §§ 4.3, 4.7, 4.16 ; `genome/06-codon-v1-2.md` § 2.

Table 9.2 : Ce qui change de la v1.0 à la v1.2, et rien d'autre.

Où	Décalage	v1.0	v1.2
en-tête, version mineure	10 (u16)	0	**2**
en-tête, drapeaux	12 (u32)	bit 0 (aligné) ; bit 1 (niveaux) ; bit 2 (brut)	+ **bit 3 CANONIQUE**
répertoire, chaque entrée	56 (u64)	réservé, 0	**`empreinte_canonique`** : empreinte64 de l'expert canonique
tables CODES_CTX	—	256 fréquences	les mêmes, bâties sur les codes canoniques ; fréquence 0 aux 31 octets qui portent un 8, vérifiée à l'ouverture

Données : `decisions/ARCHITECTURE.md` § 4.16 ; `genome/06-codon-v1-2.md` § 2.

Les refus. `trc-format` refuse, chaque refus avec son témoin et sa cassure dans `tests/aller_retour.rs` : le bit 3 sous une version mineure inférieure à 2 ; le bit 3 sur un fichier brut (le chemin de référence des témoins C3 et C12 garde les octets du GGUF tels quels) ou à deux niveaux (la v1.2 ne définit pas la forme canonique de BASE_CTX et COMPL_CTX) ; une empreinte canonique non nulle sans le bit 3 ; une table CODES_CTX d'un fichier canonique qui donne une fente à un octet portant un 8 ; le bit 4 (le COLONNES du cahier) et toute version 1.3 ou plus.

Ce qui ne change pas. La sérialisation des flux, le décodage normatif, les noyaux du décodeur, la taille des tables (3×512 octets par matrice), le format des parties. *Un flux v1.2 est un flux v1.0 dont l'alphabet effectif a 225 octets au lieu de 256.* C'est pourquoi le décodeur de la carte et la GEMV n'ont pas une ligne de changée, et pourquoi le risque du lot tient dans le codeur, le vérificateur et le témoin du moteur. Le codon synonyme se porte dans un format par les tables, pas par le décodeur : c'est la leçon d'ingénierie du lot, et elle repose sur le lemme 9.5.

9.4.1 Pourquoi la colonne reste dehors

Le cahier proposait d'ajouter le contexte de colonne d'entrée (section 8.6), « s'il tient dans le format sans risque ». Le lot A l'a laissé dehors (réponse par délégation du 2026-10-17, réversible ; raison en une ligne : le gain ne paie pas le risque).

Le chiffre. Sur les 12 tenseurs mesurés du 20b, seules gate et up de la couche 0 passent le seuil de 0,015 bit par poids, avec −0,1416 et −0,1231 bit ; les couches 8 et 23 rendent de 0,005 à 0,010 bit, la couche 16 rien (figure 9.6, panneau a). Les barres négatives du panneau sont surtout le coût du passage de l'alphabet octet à l'alphabet demi-octet, de 0,0029 à 0,0050 bit. Rapporté au modèle entier, où chaque matrice d'une couche compte 265 420 800 poids et le 20b $24 \times 3 \times 265\,420\,800 = 19\,110\,297\,600$ poids d'experts [calc.] :

$$\frac{(0{,}1416 + 0{,}1231) \times 265\,420\,800}{19\,110\,297\,600} = 0{,}0037 \text{ bit par poids}, \qquad \frac{0{,}2647 \times 265\,420\,800}{8} \approx 8{,}78 \text{ Mo},$$

soit 0,08 % des 11,2 Go du fichier, contre 0,128 bit pour le zéro (panneau b). Le prix, de l'autre côté : une sorte de flux neuve (géométrie 2, douze contextes par symbole), un noyau de décodage neuf avec sa LUT compacte, un montage de LUT neuf dans l'ordonnanceur, un second décodeur indépendant à étendre, et un décodage 2,5 fois plus cher sur les matrices concernées (section 8.7.1). Le panneau (c) redonne l'axe sans perte de la frontière commune, que la figure 8.1 ne porte plus : 3,8434, puis 3,7064

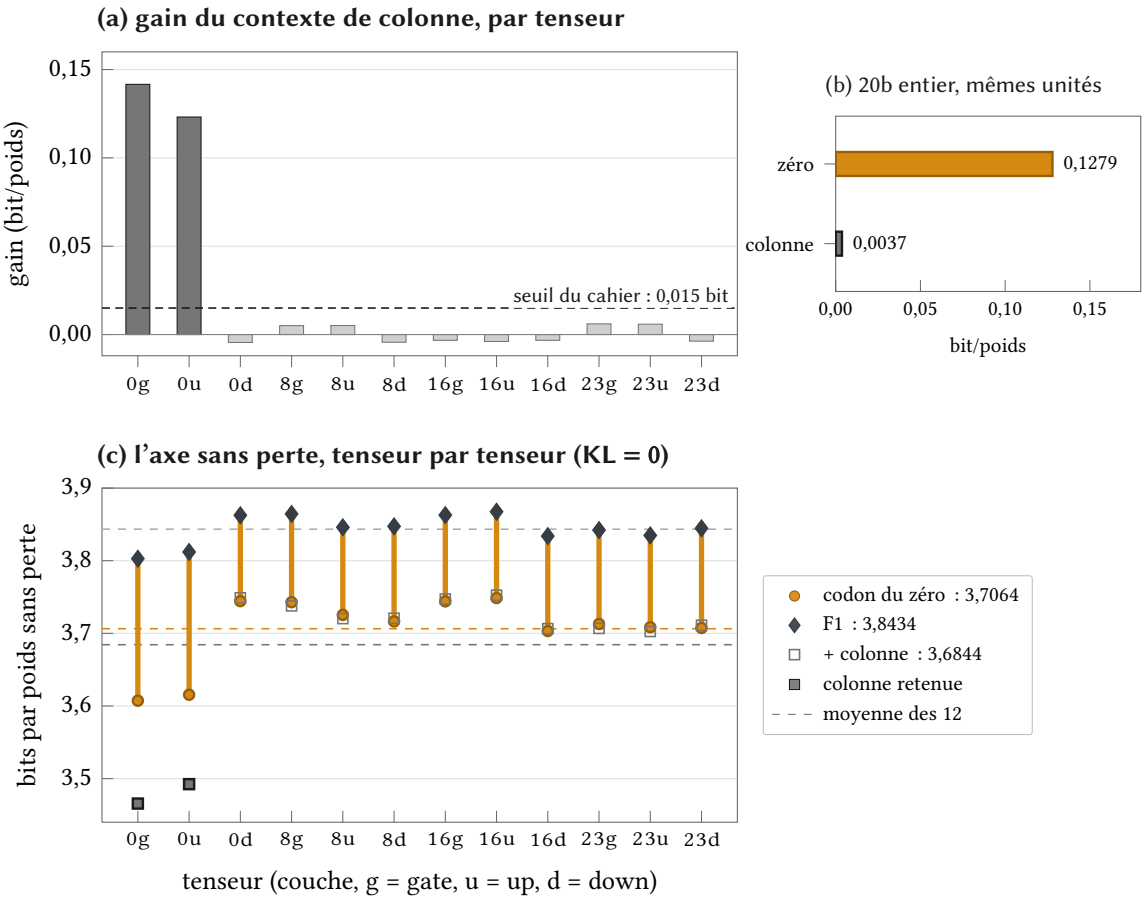


Figure 9.6 : Le contexte de colonne gagne sur deux tenseurs de la couche 0 ; sur le modèle entier, il pèse peu à côté du codon du zéro. [mes.] [calc.] Douze tenseurs de gpt-oss-20b (quatre profondeurs seulement : 0, 8, 16, 23). (a) Le débit gagné en ajoutant la classe de la colonne d'entrée au contexte octet κ, en bits par poids (codec réel : flux codés par le rANS et décodés exactement ; aucun fichier `.trc` écrit). Ce gain contient le coût du passage de l'alphabet octet à l'alphabet demi-octet, de 0,0029 à 0,0050 bit sur les douze tenseurs : les barres négatives sont surtout ce coût. Seules les matrices gate et up de la couche 0 passent le seuil de 0,015 bit par poids déclaré par le cahier (05 § 4.3) ; ailleurs le gain est de quelques millièmes au plus, ou négatif. (b) Dans les mêmes unités, mais pas sur la même longueur : ces deux tenseurs rapportés au modèle entier, 0,0037 bit par poids (8,78 Mo), contre le gain mesuré du codon du zéro, 0,1279 bit. (c) L'axe sans perte (KL = 0 exactement, logits égaux au bit), codec réel : losanges gris, la frontière F1 ; orange, après le codon du zéro (la barre est son gain) ; carrés gris foncé, avec le contexte de colonne, pleins là où la règle le retient ; il perd sur 6 des 12. Tirets : les moyennes des 12, les trois points sans perte de la frontière commune du chapitre 8. Ces douze comptent 3 tenseurs de la couche 0 (3 sur 72 dans le modèle) : la colonne y gagne 0,0220 bit en moyenne, contre 0,0037 pour le modèle entier en (b). Non montré : le prix de l'autre côté, une sorte de flux neuve, un noyau de décodage neuf et un second décodeur à étendre (06 § 2).

Données : `mesures/genome/construction/synthese/colonne-par-tenseur.csv`, `mesures/genome/construction/codon/tailles.json` ; témoin `mesures/genome/construction/synthese/k10/frontiere.csv` ; script `memoire/outils/tome3_codon.py`.

avec le zéro, puis 3,6844 avec la colonne, en moyenne sur les 12 tenseurs. Ces 12 tenseurs comptent 3 tenseurs de la couche 0 sur 12, contre 3 sur 72 dans le modèle : c'est pourquoi la colonne y gagne 0,0220 bit en moyenne, contre 0,0037 sur le modèle entier. Une phrase de la première version de la légende disait que la colonne n'était « pas un fichier codé » : elle était fausse (le gain vient d'un codec réel, des flux rANS codés et décodés exactement) ; le juge l'a relevée (GN2) et la légende jugée est corrigée.

Les trois autres réponses par délégation du lot, réversibles : le drapeau CANONIQUE est refusé sur un fichier brut (le brut est le chemin de référence des témoins et doit garder les octets du GGUF) ; `--canonique` n'est pas l'option par défaut du codeur (sans elle, il écrit la v1.0 octet pour octet, ce qui garde le témoin d'identité) ; les fichiers neufs s'appellent `<modèle>-v12.trc`, à côté des v1.0, sans rien effacer.

9.5 Ce qui a été écrit, et ses témoins

Table 9.3 : Les fichiers du lot A, et ce qui y change. Le décodeur CUDA et la GEMV ne changent pas.

Fichier	Ce qui change
`decisions/ARCHITECTURE.md`	§ 4.15 (la v1.2 dans la liste), § 4.16 (la décision), § 5.6 (l'invariant du code 8)
`trc-format/src/lib.rs`	version 2, `DRAPEAU_CANONIQUE = 8` ; canonisation d'un demi-octet et d'un octet ; refus du bit 3 ; `empreinte_canonique` ; vérification des tables à l'ouverture
`trc-format/tests/aller_retour.rs`	aller-retour v1.2 ; sept refus nommés, deux acceptations témoins
`trc-codec/src/lib.rs`, `tests/rans.rs`	`canoniser_qs` (rend le nombre de −0 fondus) ; le codage et le décodage sont ceux de la v1.0
`trc-codeur` (`coder`, `verifier`, `main`)	option `--canonique` ; deux empreintes par expert ; vérification en valeur par un second chemin ; cassure `--casser-8`
`trc-ordo/src/lib.rs`, `banc-chemin`	mode témoin contre l'empreinte canonique (boucle de pas et préremplissage par flux)
`trc-noyaux/tests/carte.rs`, `trc-reference/tests/code_8.rs`	la proposition 8.9 sur la carte ; la référence et le code 8

Données : `genome/06-codon-v1-2.md` § 3.

Le fondu lui-même tient en trois fonctions, citées telles qu'elles sont commitées (listing 9.1) : un demi-octet 8 devient 0, un octet est canonisé demi-octet par demi-octet, et le codeur canonise les `qs` d'une matrice en comptant les −0 qu'il fond, le nombre que vérifie ensuite le témoin. Rien d'autre du codage ne change.

Listing 9.1 : Le fondu −0 → +0 dans le code : `moteur/crates/trc-format/src/lib.rs`, lignes 67 à 77, puis `moteur/crates/trc-codec/src/lib.rs`, lignes 317 à 327 (extraits au bit, vérifiés par comparaison ligne à ligne).

```rust
/// Le demi-octet canonique : 8 −(0) devient 0 (+0) ; tout autre est inchangé.
#[inline]
pub const fn canoniser_demi(c: u8) -> u8 {
    if c == 8 { 0 } else { c }
}
```

```rust

/// L'octet canonique : ses deux demi-octets canonisés.
#[inline]
pub const fn canoniser_octet(o: u8) -> u8 {
    canoniser_demi(o & 15) | (canoniser_demi(o >> 4) << 4)
}

/// Canonise des `qs` en place : chaque demi-octet 8 -(0) devient 0 (+0). Rend le nombre de
/// demi-octets changés.
pub fn canoniser_qs(qs: &mut [u8]) -> u64 {
    let mut n = 0u64;
    for q in qs.iter_mut() {
        let c = canoniser_octet(*q);
        n += ((*q & 15) == 8) as u64 + ((*q >> 4) == 8) as u64;
        *q = c;
    }
    n
}
```

Les essais. `cargo test --offline --workspace`, sous le verrou de la carte : 82 essais, 0 échec [mes.] (`essais-espace-2026-10-17-a.log`). Chaque témoin neuf tombe quand on le casse (`mesures/genome/construction/codon/tests-casses.txt`) : le contrôle des tables canoniques retiré fait échouer `cassures_v12_refusees` ; le contrôle « empreinte canonique nulle sans drapeau » retiré, de même ; le bit 3 accepté sous la v1.1 fait échouer deux essais ; `canoniser_qs` rendue sans effet fait échouer `matrice_canonique_se_reconstruit_et_gagne`.

Le codeur modifié écrit toujours la v1.0 au bit. Le 20b recodé sans `--canonique` par le codeur modifié a la SHA-256 `e8bd6ae1`… [121], celle de `gpt-oss-20b.trc` relevée par le lot F le 2026-10-15, par un chemin indépendant (`Get-FileHash`) [mes.] (`journaux/recoder.log`).

9.6 Les deux fichiers recodés

Codeur : `trc-codeur coder … --canonique --fils 6`, la SHA-256 du GGUF recalculée par le codeur et égale à l'attendue ; 361 s pour le 20b, 1 395 s pour le 120b. Les fichiers v1.0 restent en place ; les v1.2 sont neufs, `gpt-oss-20b-v12.trc` (SHA-256 `ef1cf740`…) et `gpt-oss-120b-v12.trc` (`97c292ae`…).

C1 en valeur. 768 experts sur 768 et 4 608 sur 4 608 sont exacts, chacun des huit contrôles à zéro faute : empreinte de la partie, décodage, octets contre le GGUF canonisé par un second chemin, empreinte du décodé contre la canonique, empreinte des tranches brutes contre l'origine, empreinte des tranches canonisées contre la canonique, aucun 8, égalité en valeur [mes.] (`journaux/verifier-\{20b,120b\}-v12.json` ; 274 et 781 s). Deux cassures sur le 20b : `--casser 5 7` (un bit de la première médiane retourné) rend faux exactement l'expert (5, 7) ; `--casser-8 11 3` (un demi-octet 0 du décodé changé en 8) rend faux exactement l'expert (11, 3), avec les contrôles « octets », « empreinte du décodé » et « aucun 8 » tombés, et le contrôle « valeur » *tenu*. L'instrument distingue l'égalité des octets de l'égalité des valeurs, et c'est exactement ce que la v1.2 change.

Table 9.4 : Ce que le fondu rend sur le disque [mes.].

	gpt-oss-20b	gpt-oss-120b
poids d'experts	19 110 297 600	114 661 785 600
bits par poids, flux, v1.0 → v1.2	3,85057 → 3,72266	3,85456 → 3,72892
octets de tronçons, v1.0	9 226 493 952	55 416 086 528
octets de tronçons, v1.2	8 920 932 352	53 615 599 616
gain par poids d'expert (A-C2 : ≥ 0,12)	**0,12791 bit**	**0,12562 bit**
octets en moins	305 561 600	1 800 486 912
rapport des tronçons v1.2 / v1.0	0,96688	0,96751
fichier `.trc`, v1.0	11 157 405 696	57 730 662 400
fichier `.trc`, v1.2	10 851 844 096	55 930 175 488
plus grand tronçon, v1.0 → v1.2	12 144 640 → 11 755 520	12 251 136 → 11 878 400

Données : `mesures/genome/construction/codon/tailles.json`, `tailles-par-tenseur.csv` (`outils/genome/construction/codon/analyse_codon.pytailles` ; témoin : la somme par tenseur égale le total du journal, et un octet ajouté le fait tomber).

Les calculs, posés. Le gain par poids d'expert est la différence des octets de tronçons, en bits, divisée par le nombre de poids d'experts [calc.] :

$$\text{20b} : \quad \frac{(9\,226\,493\,952 - 8\,920\,932\,352) \times 8}{19\,110\,297\,600} = \frac{2\,444\,492\,800}{19\,110\,297\,600} = 0{,}12791 \text{ bit},$$

$$\text{120b} : \quad \frac{(55\,416\,086\,528 - 53\,615\,599\,616) \times 8}{114\,661\,785\,600} = \frac{14\,403\,895\,296}{114\,661\,785\,600} = 0{,}12562 \text{ bit}.$$

A-C2, déclaré à 0,12 bit, tient sur les deux. Le cahier attendait ≈ 10,85 Go et ≈ 55,93 Go : les fichiers font 10 851 844 096 et 55 930 175 488 octets. L'équipe du codage prédictif avait mesuré 0,1280 et 0,1257 bit sur des flux par tenseur entier ; découpés par expert dans le format, on mesure 0,12791 et 0,12562, l'écart (moins de 0,0001 bit) étant celui des tuiles partielles et de l'alignement. Le toit de vitesse que les octets permettent, à disque limitant, vaut $1/0{,}96751 - 1 = 3{,}36\,\%$ sur le 120b et $1/0{,}96688 - 1 = 3{,}43\,\%$ sur le 20b [calc.] ; c'est le « 3,4 % » du cahier.

Les logits au bit (A-C1). `troncons evaluer --flux`, 10 fenêtres de 512 jetons, l'empreinte de 64 bits des logits f32 de chacun des 5 120 pas.

Sur le 120b, la comparaison est plus forte que demandée : la v1.2, évaluée par le préremplissage par flux avec le programme du 17 octobre, rend au bit les logits que la v1.0 rendait par la boucle de pas avec le programme du 15. Le format, le programme et le chemin d'évaluation ont changé ; aucun bit des logits. Une découverte en chemin : la première passe de la cassure a été refusée en 8 s, avant tout logit, par la garde de production du moteur (« couche 0, expert 30 : empreinte des 11 234 960 octets utiles lus … au lieu de … ») ; c'est un témoin de la garde, gardé, et la cassure des logits a été refaite `--sans-garde`, déclaration ajoutée au pilote avant la seconde passe. Une cassure doit traverser toutes les gardes en amont de l'instrument qu'elle éprouve.

9.6.1 Le codon par le décodeur du juge

Le juge de la construction (chapitre 11) n'avait rien construit. Il a écrit son propre décodeur depuis la seule description du format, et un lecteur GGUF propre, sans ouvrir le code du lot [mes.] (`genome/JUGEMENT-construction.md` § 2).

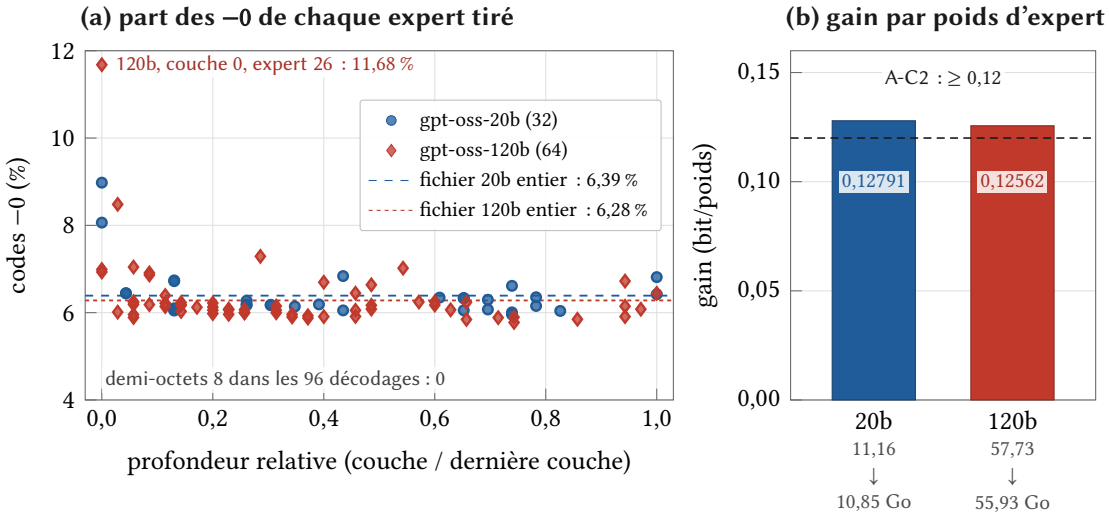


Figure 9.7 : Le décodeur propre du juge retrouve l'expert canonique dans les 96 experts tirés, et le gain des deux fichiers passe le seuil déclaré avant. [mes.] (a) Les 96 experts tirés des deux fichiers v1.2 et décodés par le juge : part des codes −0 de l'expert du GGUF, selon la profondeur relative ; en tirets, la part sur le fichier entier. Les 96 se décodent en l'expert canonique, sans aucun demi-octet 8 ; les 40 témoins v1.0 se décodent en les octets du GGUF (40/40). L'ensemble des demi-octets changés entre v1.0 et v1.2 est exactement celui des −0 (note du juge § 1.2). La part des −0 varie d'un expert à l'autre : la plus haute, 11,68 %, est à la couche 0 du 120b ; le gain du fichier n'en dépend pas. L'axe est tiré des données : aucun expert n'est hors du cadre. (b) Le gain par poids d'expert des quatre fichiers lus en entier (tronçons alignés), leurs tailles en dessous, contre le seuil A-C2 déclaré avant.

Données : `mesures/genome/construction/juge/J-T1-echantillon.csv`, `J-T0-fichiers.json` ; `mesures/genome/construction/codon/tailles.json` ; script `memoire/outils/tome3_juge.py`.

Table 9.5 : A-C1 : les logits de la v1.2 contre ceux de la v1.0 [mes.].

	gpt-oss-20b	gpt-oss-120b
témoin du programme : v1.0 d'aujourd'hui contre le CSV du 2026-10-15	5 120 / 5 120 égales	— (voir la ligne suivante)
v1.2 contre v1.0	**5 120 / 5 120 égales**	**5 120 / 5 120 égales** au CSV du 2026-10-15
`.kld`	égaux octet pour octet	égaux octet pour octet à celui du 2026-10-15
PPL des positions notées	461,978	422,206
cassure (médiane faussée, couche 0, expert 30)	512 lignes sur 512 différentes	—

Données : `genome/06-codon-v1-2.md` § 7 ; `outils/genome/construction/codon/pilote_codon.py`, phases `c1-20b`, `c1-120b`.

- **J-T0, les quatre fichiers entiers** : 0 faute de structure ; les huit tailles du tableau 9.4 redonnées à l'octet ; 0 table CODES_CTX sur 540 en v1.2 avec une fente pour un 8 (540 sur 540 en v1.0).
- **J-T1, 96 experts tirés** (32 du 20b, 64 du 120b, tirage commité avant tout décodage) : 96 sur 96 tiennent les huit contrôles ; le témoin v1.0, 40 sur 40 ; les demi-octets qui changent entre le décodé v1.0 et le décodé v1.2 sont exactement les −0 du GGUF, ni un de plus ni un de moins (par exemple 1 746 601 changés pour 1 746 601 −0 dans l'expert 109 de la couche 19 du 120b) ; trois cassures tombent. La part des −0 dans l'échantillon, 6,42 ± 0,11 % (20b) et 6,33 ± 0,10 % (120b), est à 0,3 et 0,5 écart-type de celle des fichiers ; la plus haute, 11,68 %, est à la couche 0 du 120b (figure 9.7).
- **J-C1, sur dix fenêtres que personne n'avait vues** : 0 empreinte de logits différente sur 5 120 sur le 20b, 0 sur 5 120 sur le 120b ; `.kld` égaux ; cassure (le signe du biais de la rangée 0 de gate retourné pour les 32 experts de la couche 0) : 512 sur 512 différentes dès la position 0. La proposition 8.9 est vraie du moteur réel, sur un texte neuf.

Ce que l'échantillon ne couvre pas, le juge l'écrit : la probabilité qu'un défaut touchant au moins 5 % des experts échappe à 32 tirages vaut $0{,}95^{32} \approx 19\,\%$, à 64 tirages $0{,}95^{64} \approx 3{,}7\,\%$ [calc.]. Le vérificateur du lot A a couvert 100 % des experts par ses deux chemins ; le tirage juge ce vérificateur, il ne le remplace pas.

9.7 Le jalon J1 refait sur le 120b

Le protocole. Huit processus neufs, A B B A A B B A, A = `gpt-oss-120b.trc` (v1.0 d'origine, écrit le 15 octobre), B = `gpt-oss-120b-v12.trc` ; `mesurer--flux--emplacements326`, l'invite wiki-3000, 256 jetons gloutons ; sur secteur, aucun llama.cpp avant chaque passe, verrou de la carte. Le nombre d'emplacements du cache est fixé à 326, celui du J1 : il se calcule sinon à l'ouverture depuis la VRAM libre et la taille des tampons d'arrivée (12× le plus grand tronçon), et un emplacement de plus changerait le cache, donc les octets lus, et mêlerait deux effets. Critères, déclarés avant (commit `a50c380`) : A-C3, pas de régression si $r \geq 1 - 2\sigma$, gain annoncé seulement si $r - 1 > 2\sigma$, σ l'écart-type relatif des quatre passes v1.0 ; jetons égaux ; rapport des octets publié (prédit 0,967) ; mémoire de pointe $|B - A| \leq 2 \times$ bruit + 0,5 Mo.

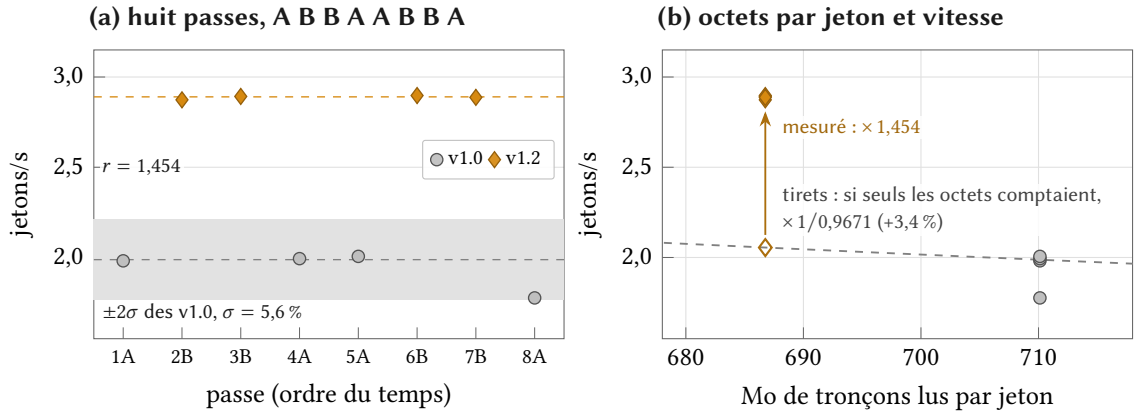


Figure 9.8 : Sur le 120b, la v1.2 va ×1,454 plus vite que la v1.0, et ses octets ne lui donneraient que +3,4 % au plus. [mes.] gpt-oss-120b, séance s2 du lot A, machine calme, huit processus neufs, les mêmes 256 jetons dans tous. (a) Les passes dans l'ordre du temps : médianes 1,988 (v1.0) et 2,890 (v1.2) jetons/s, en tirets ; bande grise, $\pm 2\sigma$ des passes v1.0. (b) Les octets lus par jeton sont déterministes, ×0,9671 ; si leur quantité seule comptait, la v1.2 serait sur la courbe en tirets, au losange creux (+3,4 % au plus). D'où vient le reste : figure 9.9.

Données : `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json` ; script `memoire/outils/tome3_codon.py`.

La séance s1, sous charge. Le 17 octobre, de 10 h 24 à 10 h 50, pendant que d'autres équipes lisaient le disque : médianes 1,558 et 1,732 jetons/s, $r = 1{,}112$, mais $\sigma = 21{,}6\,\%$ (les passes 5 à 7 sont toutes lentes, quel que soit le fichier). Pas de régression ; gain non annoncé ($0{,}11 < 2\sigma = 0{,}43$). Dans les quatre premières passes, calmes, la v1.2 allait 1,4 à 1,7 fois plus vite : 3,3 % d'octets en moins ne peuvent pas rendre 40 %. Le lot a cherché un témoin qui juge le format seul : une copie neuve du v1.0, puis le v1.0 recodé par le même codeur le même jour que la v1.2, au même contenu au bit (SHA-256 égale à celle du v1.0 d'origine). Les deux se sont révélés confondus par leur propre fragmentation, relevée par `fsutil file queryextents` [122] : 289 545 fragments pour la copie, 47 745 dans l'échantillon pour le v1.0 recodé, contre 65 au v1.0 d'origine et 233 à la v1.2.

La séance s2, sur une machine calme. Déclarée avant ses passes (commit `71905dd`) ; nuit du 21 au 22 octobre ; aucune des 22 passes sous charge.

Table 9.6 : Le J1 du 120b, séance s2, en ABBA [mes.]. A = v1.0 d'origine, B = v1.2.

passe	1 A	2 B	3 B	4 A	5 A	6 B	7 B	8 A
jetons/s	1,981	2,873	2,892	1,994	2,006	2,897	2,887	1,776
débit vu par le moteur (Go/s)	1,81	2,63	2,66	1,82	1,83	2,67	2,65	1,61

Données : `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json`.

- **Jetons** : les 256 identifiants égaux dans les huit passes, texte `e4c6dbac…`, celui de s1 et de toutes les passes du 120b des 15 et 16 octobre ; 59,08 défauts par jeton, taux de succès du cache 0,5897, 326 emplacements dans toutes les passes. Le format ne change ni le routage ni le cache.
- **Octets** de tronçons par jeton, déterministes : 686 756 916/710 118 798 = 0,9671, pour une prédiction de 0,967 ; le rapport des tronçons du fichier entier vaut 0,9675, et les experts que ce texte appelle sont un peu plus riches en zéros que la moyenne.

- **Vitesse** : médianes 1,988 (v1.0) et 2,890 (v1.2) jetons/s, $r = 2{,}890/1{,}988 = 1{,}454$ [calc.], $\sigma = 5{,}6\,\%$; rapports appariés 1,450, 1,451, 1,444 et 1,626. Au seuil déclaré : pas de régression, et **gain annoncé** ($r - 1 = 0{,}454 > 2\sigma = 0{,}113$).
- **Mémoire de pointe** : médianes 388,67 et 382,21 Mo, −6,45 Mo (s1 : −6,41). 4 472 832 octets s'expliquent par les 12 tampons d'arrivée épinglés, dimensionnés sur le plus grand tronçon : $12 \times (12\,251\,136 - 11\,878\,400) = 4\,472\,832$ [calc.] ; le reste, ≈ 1,9 Mo, n'est pas expliqué. Le seuil tient au sens littéral seulement parce que la première passe de la séance (468,4 Mo) fait un bruit de 79,8 Mo ; sans elle, les passes v1.0 vont de 388,60 à 388,70 Mo et l'écart est net.

La figure 9.8 le montre en deux panneaux : les huit passes dans l'ordre du temps, et, à droite, les octets contre la vitesse. Si la quantité d'octets seule comptait, la v1.2 serait au losange creux, à 3,4 % au plus. Elle est à 45 %.

9.7.1 D'où vient le gain : le motif des requêtes de lecture

L'étage disque seul. Dans la même séance s2, `troncons disque --n 1500`, sans calcul, A B C C B A, lit A (v1.0 d'origine, 65 fragments) à 1,742 et 1,741 Go/s, B (v1.2, 233 fragments) à 2,254 et 2,391, et C (le v1.0 recodé, égal à A au bit) à 1,356 et 1,331 [mes.] (`disque-s2.json`). Le J1 refait avec C contre B donne $r = 1{,}595$, $\sigma = 4{,}2\,\%$.

La première lecture, et sa barre. Le lot A en avait tiré une conclusion prudente, que le juge de la construction avait reproduite, et qui est fausse dans son attribution :

> ~~Ce gain n'est pas attribuable au format. Deux fichiers au même contenu au bit (A et C) se lisent à 1,74 et 1,34 Go/s à l'étage disque, et rendent 1,99 et 1,81 jetons/s : la disposition physique d'un fichier sur ce NVMe déplace la vitesse de 30 % sans toucher un octet du contenu. […] La part du format, elle, est bornée par le rapport des octets lus : au plus +3,4 % à disque limitant.~~

Ce qui a changé, le 2026-10-27 : le juge du placement disque (`genome/JUGEMENT-placement.md`) a lu *un même fichier* avec les frontières de requête de l'autre format, sans rien réécrire. La borne de 3,4 % ne vaut que pour les octets ; elle ne borne pas l'effet du motif. L'écart entre A et C, lui, a une autre cause : la fragmentation extrême de C. Les `06-codon-v1-2.md` § 8, la note du lot pour le mémoire et le jugement de la construction (§§ 0 et 6) portent la même correction ; ~~une ligne du tableau du § 8 de ce jugement garde l'ancienne attribution sans barre~~, et ce chapitre suit la correction (section 9.9). Depuis le 2026-10-29, cette ligne est barrée elle aussi, avec ce qui a changé (`genome/JUGEMENT-construction.md` § 8).

Table 9.7 : Le motif croisé (gpt-oss-20b, étage disque seul, séance J-bis du juge du placement, 26 octobre) : le même fichier lu avec les frontières de requête de l'autre format, Go/s [mes.].

Cas	Passes	Médiane
A20, motif natif (v1.0)	1,792 ; 1,793 ; 1,813 ; 1,809	1,801
A20, motif de B20	2,509 ; 2,493 ; 2,483 ; 2,564	2,501
B20, motif natif (v1.2)	2,511 ; 2,487 ; 2,556 ; 2,545	2,528
B20, motif de A20	1,811 ; 1,829 ; 1,805 ; 1,774	1,808

Données : `mesures/disque/placement/juge/seance/Jb3-*` ; `genome/JUGEMENT-placement.md` § 5.3.

Ce que montre le motif croisé. Aux seuils déclarés avant ($e_{\text{soi}} \leq 3{,}2\,\%$, $S_j = 11{,}8\,\%$), A20 lu au motif de B20 va ×1,389 plus vite que lu au sien, et B20 lu au motif de A20 va ×0,715 : *la lenteur suit le motif*, pas le fichier. B20, le fichier v1.2, lu avec les frontières de la v1.0, va comme A20 (1,808 contre 1,801). Deux expériences de plus le précisent. J9 : A20 lu avec des requêtes raccourcies de 3,5 % va ×1,421, B20 avec des requêtes allongées de 3,5 % ×0,713 ; le pas de la fenêtre de destination ne décide pas. J10 et J11 : couper chaque tronçon v1.0 en deux requêtes de 6 Mo, mêmes octets, ne le rend pas rapide, il le ralentit (×0,851 à huit lectures en vol, ×0,882 à quatre) : *la longueur seule n'est pas la variable*. Ce qui sépare les motifs lents des rapides, dans les huit cas du juge, est descriptif : sont lentes les requêtes qui finissent exactement à une fin de tronçon v1.0, ou qui débordent de 3,5 % une fin de tronçon v1.2 ; sont rapides celles qui s'arrêtent avant. Cette description n'est pas un mécanisme ; la variable exacte n'est pas isolée.

La cascade, dans une séance à trois fichiers. Le juge de la construction a mis les trois fichiers du 120b dans un seul plan équilibré, A B R R B A A B R R B A, A = v1.0 d'origine, B = v1.2, R = le compagnon rapide frontière ; aucune des 36 passes sous charge [mes.] (`genome/JUGEMENT-construction.md` § 6.2). B contre A : médianes 2,889 et 2,004 jetons/s, $r(B/A) = 1{,}442$, $\sigma = 5{,}8\,\%$; le J1 du lot A (1,454) se reproduit. L'étage disque seul, dans la même séance, lit B 1,37 fois plus vite que A. Le panneau (b) de la figure 9.9 partage ce rapport en deux facteurs [calc.] :

$$r(B/A) = \underbrace{\frac{1}{0{,}9671}}_{\text{octets : } 1{,}034} \times \underbrace{\frac{1{,}442}{1{,}034}}_{\text{reste : } 1{,}394} .$$

Le premier facteur est un toit : ce que les octets en moins rendraient si le disque limitait seul. Le second est un reste, donc une borne basse de l'effet du motif. Le compagnon rapide ajoute ×1,284 contre la v1.2 ; c'est l'affaire du chapitre 10, et sa qualité n'est mesurée que contre le moteur exact du 120b, un proxy.

Le v1.0 recodé. L'encart du panneau (a) règle le cas de C : A120 et C120 ont le même contenu au bit et le même répertoire (même SHA-256, même motif) ; C120 est plus lent à cause de ses 267 473 extents, contre 65 pour A120. Sur ses 800 tronçons d'un seul extent, il se lit ×1,26 plus vite que sur 1 200 de ses tronçons tirés au hasard, et comme A120 sur les mêmes tronçons (0,980 au rapport des moyennes) [mes.] (`genome/JUGEMENT-placement.md` §§ 5.9 et 5.10). Les deux causes sont donc séparées : la fragmentation explique A contre C ; le motif explique A contre B.

Ce que le J1 dit, en une phrase. Sur ce portable, le format v1.2 rend le moteur environ 45 % plus rapide sur le 120b, par le motif de ses requêtes de lecture et non par ses 3,3 % d'octets ; ce gain ne se transporte pas à un autre disque sans y être remesuré. Ce que le lot avait proposé pour l'amplification de 1,33 (étage disque) à 1,45 (moteur) reste une hypothèse : la carte, qui attend plus le disque, tournerait à des horloges plus basses ; les temps de carte mesurés par événements (copie 91 à 99 ms par jeton contre 66 à 68 ; décodage 61 à 67 ms contre 42 à 44) varient bien plus que les octets.

9.8 Ce qui reste

– **Isoler la variable du motif**, à l'étage disque, par des motifs déclarés avant : chaque tronçon v1.0 lu en une requête de 0,965 l puis une requête de la fin ; raccourci par le début ; le motif de B20 décalé de 64 Kio ; les mêmes sur A120 et C120 (`genome/JUGEMENT-placement.md` § 12). Un levier de

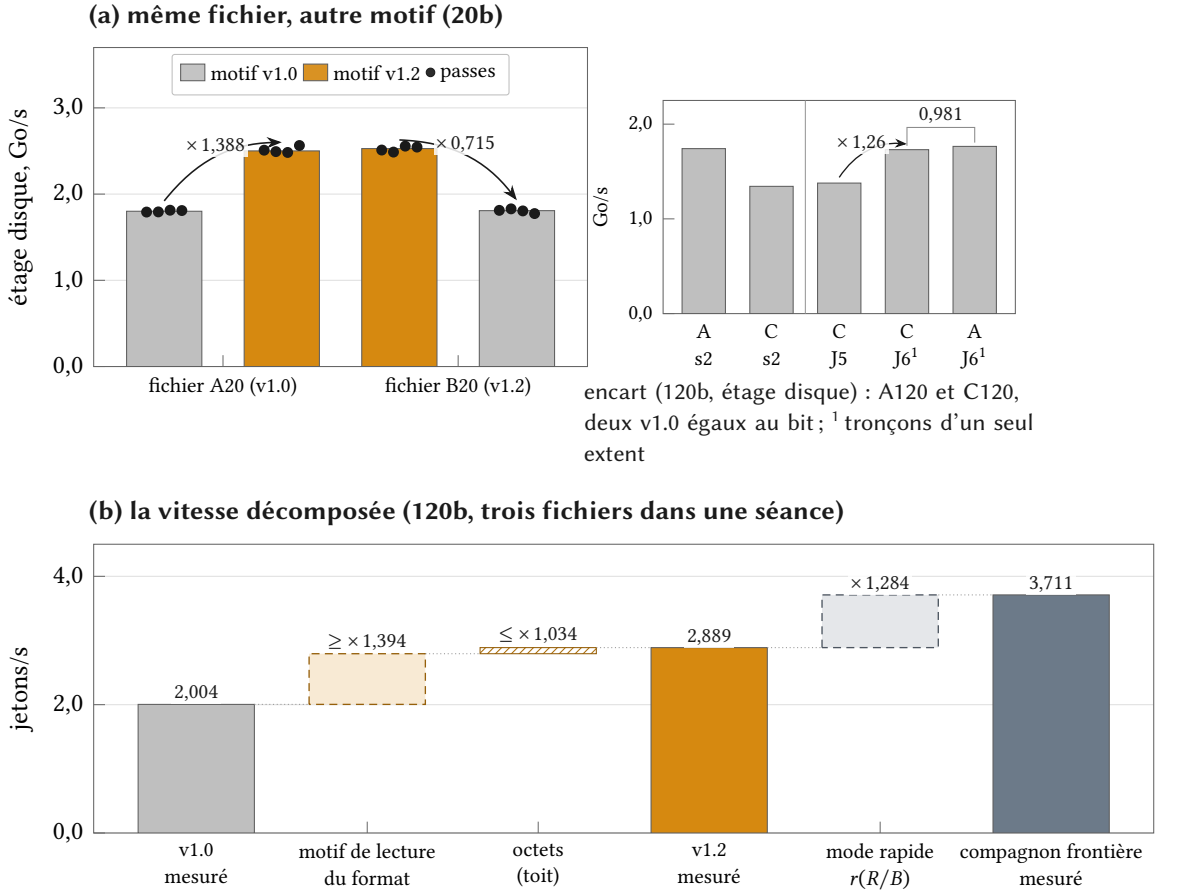


Figure 9.9 : Le gain vient du motif de lecture qu'impose le format v1.2, sur ce disque ; le 20b et le 120b se partagent la figure par panneau. [mes.] La suite de la figure 9.8. (a) gpt-oss-20b, étage disque seul, séance J-bis du juge du placement : le même fichier lu avec les frontières de requête de l'autre format prend la vitesse de ce format, A20 × 1,388, B20 × 0,715 (barres : médianes ; points : les quatre passes de chaque cas, aucune écartée ; couleur : le motif de lecture ; abscisse : le fichier). Encart, gpt-oss-120b : A120 et C120 ont le même contenu au bit ; en s2, C120 est plus lent à cause de sa fragmentation (267 473 extents contre 65). Sur ses tronçons d'un seul extent (J6, 800 tronçons), il se lit × 1,26 plus vite que sur 1 200 de ses tronçons tirés au hasard (J5 ; médianes de 2 et 3 passes), et comme A120 sur les mêmes tronçons (rapport 0,981 des médianes ; 0,980 des moyennes, la valeur qu'écrit le juge du placement). Les barres s2 viennent de `troncons disque`, les barres J5 et J6 du lecteur Python du juge du placement, plus lent de 2 à 5 % (`genome/JUGEMENT-placement.md` § 5.2) : chaque rapport écrit compare deux barres du même lecteur. (b) La séance équilibrée à trois fichiers du 120b du juge de la construction : $r(B/A) = 1{,}442$ se partage entre les octets (hachuré : un toit) et le motif de lecture du format, un reste, donc une borne basse (l'étage disque seul, dans la même séance, donne $B/A = 1{,}370$) ; le compagnon rapide, le compagnon frontière du 120b, avec perte (sa KL, 0,140 ± 0,032, 2σ par fenêtres, est mesurée contre le moteur exact du 120b, un proxy, pas contre la référence f64 ; la figure 10.6 montre les variantes du 20b), ajoute × 1,284 contre la v1.2. Le gain du J1 est un gain du format v1.2 sur cette machine, par le motif de ses requêtes de lecture, non par ses octets ; la variable exacte du motif n'est pas isolée (J10, J11). Il ne se transporte pas à un autre disque sans y être remesuré.

Données : `mesures/disque/placement/juge/seance/Jb3-*`, `J5-*`, `J6-*`, `J1-cartes.json` (`genome/JUGEMENT-placement.md` §§ 5.3, 5.9–5.10) ; `mesures/genome/construction/codon/disque-s2.json` ; `mesures/genome/construction/juge/figures/vitesse-passes.csv`, `disque-passes.csv` (`genome/JUGEMENT-construction.md` § 6.2) ; script `memoire/outils/tome3_codon.py`.

lecture ne se mesure de bout en bout qu'une fois trouvé. Le témoin « décisif » que proposait le lot A (deux copies neuves écrites le même jour, environ 113 Go libres) n'est plus nécessaire : le motif croisé, un même fichier lu aux deux motifs, en tient lieu sans rien réécrire.

- **Remesurer** le gain de vitesse sur tout autre disque : le chiffre de ce chapitre est celui de ce NVMe, derrière ce pilote, sous ce chiffrement.
- **Un second décodeur indépendant** : le cahier demandait d'étendre à la v1.2 le décodeur de l'épreuve J1 du Tome II ; le juge de la construction a écrit le sien depuis la seule description du format, et il lit la v1.2. Ce chapitre le tient pour le second chemin (décision du rédacteur, réversible) ; le décodeur de l'épreuve, lui, n'est pas étendu.
- **Décider** si les v1.2 remplacent les v1.0 comme fichiers par défaut du moteur : c'est un choix de produit, laissé à Stone ; aucun fichier n'a été effacé, et 57,7 + 11,2 Go restent occupés par les v1.0.

9.9 Ce qui pourrait être faux

Ce qui pourrait être faux

- **La proposition 8.9 suppose des x finis** et le noyau tel qu'il est au commit `000cef9` : grandeurs doublées entières, échelle hors de l'accumulation, départ à +0. Un noyau futur qui lirait les codes autrement (une table où l'entrée 8 vaudrait autre chose que ±0, une GEMM partant d'un accumulateur −0, une échelle dans l'accumulation) pourrait casser l'égalité ; le témoin à reprendre est `gemv_lit_le_code_8_comme_0`.
- **Le corollaire 9.4** tient sous une hypothèse de non-sous-dépassement que ni le lot ni la note ne disaient ; elle est sans danger pour des activations de modèle, mais elle n'est pas démontrée pour toute entrée, seulement mesurée sur cinq vecteurs.
- **Le gain est celui de gpt-oss.** Il vaut deux fois la part des −0 ; un modèle MXFP4 quantifié par un outil qui écrit toujours +0 n'a rien à gagner (la v1.2 lui coûte zéro octet) ; un modèle où le signe d'un zéro serait prévisible rendrait moins. Les 180 tenseurs disent que ce n'est pas le cas ici.
- **Les emplacements égaux du J1** isolent le format, pas le produit : avec le nombre d'emplacements par défaut, le plus grand tronçon plus court de 372 736 octets libère $12 \times 372\,736 = 4\,472\,832$ octets de VRAM, moins d'un emplacement (13 254 144 octets) ; le défaut ne change donc probablement pas, mais ce n'est pas mesuré.
- **L'attribution du gain de vitesse** repose sur le motif croisé du juge du placement, mesuré à l'étage disque sur le 20b ; sur le 120b, la cascade du juge de la construction ne donne qu'une borne basse de l'effet du motif (le reste après le toit des octets). La variable exacte du motif n'est pas isolée, et son mécanisme (pile de stockage, contrôleur, repli hors de la SLC) n'est pas observable d'ici.
- **Une ligne du tableau « ce qui tient » du jugement de la construction** (§ 8) ~~écrit encore « pour les fichiers, pas pour le format », sans barre, alors que ses §§ 0 et 6 sont corrigés~~ ; corrigé le 2026-10-29 : elle est barrée, avec ce qui a changé ; ce chapitre suit la correction, d'après `genome/JUGEMENT-placement.md` § 12 point 3.
- **Une séance par mesure de vitesse.** Les rapports valent pour l'état de ces fichiers ces jours-là ; le motif croisé a été mesuré quatre jours après la séance principale du juge, après une réoptimisation du disque, et le rapport B20/A20 n'a pas bougé (1,398 et 1,362 le 22 ; 1,404 et 1,406 le 26), ce qui rassure sans prouver.
- **Le reste de la mémoire** (≈ 1,9 Mo sur 6,45) n'est pas expliqué ; l'hypothèse (d'autres tampons

dimensionnés sur le plus grand tronçon) n'est pas vérifiée.
- **Le contexte de colonne** est laissé dehors sur un chiffre mesuré sur 12 tenseurs de 4 couches du 20b ; sur le 120b et les autres couches, son gain n'est pas mesuré.
- **L'analogie du codon** est une analogie : un codon synonyme du code génétique n'est pas toujours neutre (usage préférentiel des codons, vitesse de traduction) ; ici, le signe d'un zéro l'est exactement, par la proposition 8.9.

Chapitre 10

Le génome synonyme, mode rapide (lot B)

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la mutation synonyme (choisir le symbole de base de chaque poids par une rétroaction d'erreur pondérée par les entrées réelles, sous une contrainte d'entropie) n'était mesurée que sur 32 experts sur 768 de gpt-oss-20b : la variante $\lambda = 0$ en KL de bout en bout ($-0{,}009\,8 \pm 0{,}003\,3$, appariée, contre la frontière), la variante $\lambda = 1$ en erreur de sortie d'expert seulement (chapitre 8). Aucun fichier, aucun noyau, aucune option du moteur ne la servait. **Ce chapitre établit** la mutation synonyme écrite pour les 768 experts, sa KL de bout en bout avec tous les experts substitués, le départage des deux variantes $\lambda = 0$ et $\lambda = 1$ par une règle déclarée avant, et le *mode rapide* du moteur : un fichier compagnon `<nom>.rapide.trc` décodé par le décodeur du mode exact, un noyau `gemv_base`, une option `--rapide`, chacun avec son témoin. Il établit aussi, par le juge, ce que le mode rapide ne rend pas : deux gains de vitesse publiés par le lot, barrés comme gains du mode. **Il reste** la mutation synonyme du 120b (ni calibration, ni juge f64), un autre corpus que wiki.test, la queue de la KL (les bascules de routage) et la décision de la politique mixte des défauts, à reprendre avec les rapports du juge (section 10.9). Sources : `genome/07-genome-synonyme.md`, `genome/construction-synonyme-pour-le-memoire.md`, `genome/JUGEMENT-construction.md`, `mesures/genome/construction/synonyme/` (seuils : `00-seuils-declares.md`) et `mesures/genome/construction/juge/`.

En bref

Dans le vivant, une mutation est dite *synonyme* quand elle change une lettre d'un codon sans changer l'acide aminé que ce codon désigne : la séquence change, la protéine non. L'image sert ici d'analogie, et de rien de plus. Une matrice d'un expert de gpt-oss n'est jugée que par ce qu'elle fait de ses entrées réelles ; une erreur de poids dans une direction que ces entrées n'excitent jamais ne change presque pas la sortie. La mutation synonyme choisit donc, poids après poids, une version approchée de la matrice (sa *base*, trois bits de symbole par poids) dont l'erreur va justement dans ces directions muettes. Elle n'invente pas de nouveau décodeur : mêmes huit symboles, mêmes quatre niveaux, mêmes échelles que la base « frontière » du Tome II, où le symbole est une simple fonction du code. Tout le travail est au codeur. Sur les 768 experts du 20b, servis tous par leur base, la KL de bout en bout tombe de 0,264 0 (frontière) à 0,131 8 (synonyme, $\lambda = 0$), pour 2,67 bits par poids d'expert au lieu de 3,86 pour le fichier exact ; un juge qui ne l'a pas construit le redonne sur dix fenêtres de texte que personne n'avait vues. Le moteur sert ce mode rapide, sur la carte, aussi fidèlement que la référence en double précision. Ce qu'il ne rend pas, c'est de la vitesse en génération : à fichiers comparables, ×1,28 sur le 120b et ×0,94 sur le 20b ; les ×2,00 et ×1,24 d'abord publiés comparaient un compagnon neuf à un ancien fichier

> exact que ce disque lit plus lentement à cause du motif de ses requêtes.

10.1 Ce qui est construit

Le vocabulaire. Le format exact du Tome II sert chaque poids d'expert en MXFP4 : un code E2M1 de quatre bits (un signe, trois bits de grandeur) sous une échelle E8M0 commune à un bloc de 32 poids [123]. La découpe *base + complément* du Tome I range les huit grandeurs E2M1 en quatre *classes* contiguës ; la *base* d'un poids est son signe et sa classe $k \in \{0,1,2,3\}$ (huit symboles, trois bits de symbole), et le *complément* dit quelle grandeur de la classe il porte. Servir la base seule, c'est remplacer chaque poids par $\pm c_k \cdot s_b$, où c_k est le niveau (le centre) de la classe et s_b l'échelle du bloc : c'est le *mode rapide*, avec perte, dont l'intérêt est de lire moins d'octets. Deux manières de choisir la base :

- la **frontière** : le symbole est une fonction du code exact (la classe où tombe sa grandeur) ; c'est la base du Tome II, sans calibration ;
- la **synonyme** : le symbole est choisi par le codeur pour que l'erreur de *sortie* soit petite sur les entrées réelles, quitte à prendre une classe voisine de celle du code ; le décodeur ne voit pas la différence.

Deux réglages de la synonyme sont mesurés : $\lambda = 0$, la rétroaction d'erreur seule, et $\lambda = 1$, la même sous une contrainte d'entropie qui pénalise les symboles rares (section 10.2.2). Tous deux portent un amortissement $\delta^\star = 1$ de la matrice des entrées (équation (10.2)).

Les pièces. Tableau 10.1 les donne, chacune avec le témoin déclaré avant sa mesure (`mesures/genome/construction/synonyme/00-seuils-declares.md`, commits du 17 octobre 2026 entre 8 h 19 et 12 h 14, puis l'amendement de la séance de vitesse le 21 à 23 h 59).

Une décision, et son prix. Le cahier de construction prévoyait un codeur en Rust sur la carte. Le lot a codé les 768 experts avec le code Python de l'éclaireur, sans en changer une ligne, à environ 54 s par expert sur un processus, trois à six processus en parallèle (*réponse par délégation du 2026-10-17, réversible*). La raison : c'est la seule voie qui rend *exactement* la variante dont l'erreur avait été mesurée, et W1 le prouve au bit. Le prix est le 120b, où ce codeur coûterait des dizaines d'heures (section 10.8).

10.2 Le mécanisme

10.2.1 L'erreur de sortie et la matrice des entrées

Soit une matrice d'expert $W \in \mathbb{R}^{R\times C}$ ($R = C = 2\,880$ pour les trois matrices gate, up et down de gpt-oss, [6]), ses entrées réelles $x \in \mathbb{R}^C$ et le poids g que le routeur donne à l'expert pour chacune. L'erreur que fait une reconstruction $\hat{W}$, à la sortie et sur la loi des entrées pondérée comme le modèle pondère l'expert, vaut

$$\mathscr{E}(\hat{W}) = \frac{\mathrm{E}\left[g^2\,\|(W-\hat{W})\,x\|^2\right]}{\mathrm{E}[g^2]} = \mathrm{tr}\left(\Delta W\, H\, \Delta W^\top\right), \qquad \Delta W = W - \hat{W}, \tag{10.1}$$

avec

$$H = \frac{\mathrm{E}[g^2\, xx^\top]}{\mathrm{E}[g^2]} \qquad \text{et, pour le codeur,} \qquad H_\delta = H + \delta^\star\, \overline{\mathrm{diag}\, H}\; I, \quad \delta^\star = 1. \tag{10.2}$$

Le calcul. $\|\Delta W x\|^2 = x^\top \Delta W^\top \Delta W x = \mathrm{tr}(\Delta W\, xx^\top \Delta W^\top)$; la trace et l'espérance sont linéaires, d'où $\mathrm{E}[g^2\|\Delta W x\|^2] = \mathrm{tr}(\Delta W\, \mathrm{E}[g^2 xx^\top]\, \Delta W^\top)$, et la division par $\mathrm{E}[g^2]$ donne (10.1). La trace se coupe en

Table 10.1 : Les pièces du lot B, chacune avec le témoin déclaré avant sa mesure et le résultat [mes.].

Pièce	Ce qu'elle fait	Témoin et cassure	Résultat
codage des 768 experts	`coder_768.sh` appelle `synonyme.py ecrire-sym`, le code de l'éclaireur, *inchangé*	W1 : refaire L00-E00 ($\lambda = 0$) rend le fichier de l'éclaireur octet pour octet ; cassure : $\delta = 0{,}99$	SHA-256 égales (`9079dcf1…`) ; cassure : 117 501 symboles différents
second chemin des symboles	`verifier_sym.py` relit les `.sym`, reconstruit, recalcule les erreurs publiées	W2 : égales au journal de l'éclaireur à 10^{-9} relatif (16 experts, couches 0, 8, 16, 23) ; cassure : un bit de classe sur une ligne	écart maximal $4{,}4 \cdot 10^{-16}$; cassure : $7{,}2 \cdot 10^{-3}$, tombe
noyau du mode rapide	`gemv_base` (`gemv.cuh`, lecteur de valeurs en paramètre)	carte contre référence f64 sur un cas exact par construction (proposition 10.4) ; mode exact avant et après	0 ligne différente sur 2 880 ; mode exact identique au bit
drapeau RAPIDE	bit 5 de l'en-tête, version 1.2	refus sous 1.2, avec brut, avec canonique	4 cas sur 4
empaqueteur et vérificateur	`trc-rapide coder`, `trc-rapide verifier`	S3-T1 (section 10.6) ; cassure `--casser C X`	l'expert visé, et lui seul
option du moteur	`troncons … --rapide`	un fichier rapide sans `--rapide`, un exact avec	refusés tous deux, erreur nommée

Données : `mesures/genome/construction/synonyme/codage/W1-reproduction.json`, `codage/W2-*.json`, `rapide/refus-*.log` ; test `gemv_base_exacte_au_bit_et_mode_exact_inchange` ; test `drapeau_rapide_v12`.

lignes : $\mathrm{tr}(\Delta W H \Delta W^\top) = \sum_r \delta_r^\top H \delta_r$, où δ_r est la ligne r de ΔW. *Les lignes sont donc des problèmes indépendants*, et le codeur traite chaque ligne w seule : $\min_q (w-q)^\top H (w-q)$, q parcourant les reconstructions permises colonne par colonne.

Les entrées de calibration sont celles de l'éclaireur : les fenêtres 10 à 21 de wiki.test (6 132 positions), disjointes des dix fenêtres d'évaluation R1 (fenêtres 0 à 9), et pondérées par le carré du poids du routeur. Un expert donné ne voit que les positions que le routeur lui envoie : 1 329 pour l'expert 0 de la couche 8, en dimension 2 880. Sa matrice H est donc de rang 1 329 au plus, et c'est ce qui rend l'amortissement nécessaire (section 10.2.3).

10.2.2 Le choix des symboles

Lemme 10.1 (Rétroaction d'erreur optimale, OBS)**.** *Soit $H \succ 0$ et une ligne w. Si l'on fixe la colonne j à une valeur q_j et qu'on laisse libres les colonnes suivantes F, l'accroissement minimal de $(w-q)^\top H (w-q)$ vaut $(w_j - q_j)^2 / [H_F^{-1}]_{jj}$, et il est atteint en corrigeant les colonnes libres de $\delta_F = -(w_j - q_j)\,[H_F^{-1}]_{Fj} / [H_F^{-1}]_{jj}$ [40, 50].*

Démonstration. On cherche la plus petite perturbation δ (portée par j et F) telle que $(w+\delta)_j = q_j$, au sens de la forme $\delta^\top H \delta$ restreinte à ces colonnes. Le lagrangien $\frac{1}{2}\delta^\top H \delta + \mu\,(e_j^\top \delta + w_j - q_j)$ s'annule en gradient pour $H\delta = -\mu e_j$, soit $\delta = -\mu H^{-1} e_j$; la contrainte donne $-\mu [H^{-1}]_{jj} + w_j - q_j = 0$, donc $\mu = (w_j - q_j)/[H^{-1}]_{jj}$ et

$$\delta = -\frac{w_j - q_j}{[H^{-1}]_{jj}}\, H^{-1} e_j, \qquad \delta^\top H \delta = \frac{(w_j - q_j)^2}{[H^{-1}]_{jj}^2}\, e_j^\top H^{-1} H H^{-1} e_j = \frac{(w_j - q_j)^2}{[H^{-1}]_{jj}}.$$

Toutes les inverses sont prises sur le bloc des colonnes $\{j\} \cup F$, d'où l'indice F de l'énoncé. ∎

Une seule factorisation. Dans l'ordre des colonnes $1, \dots, C$, les quantités $[H_F^{-1}]_{jj}$ et $[H_F^{-1}]_{Fj}$ du lemme sont les lignes du facteur de Cholesky *supérieur* U de H^{-1} ($U^\top U = H^{-1}$) : $[H_F^{-1}]_{jj} = U_{jj}^2$ et la correction vaut $-\big((w_j - q_j)/U_{jj}\big)\,U_{j,F}$ [51]. L'éclaireur obtient U sans inverser H : si P renverse l'ordre des colonnes et $PHP = LL^\top$, alors $U = PL^{-1}P$. En effet $H^{-1} = P(LL^\top)^{-1}P = PL^{-\top}L^{-1}P = (PL^{-1}P)^\top(PL^{-1}P)$ puisque $P^2 = I$ et $P^\top = P$, et $PL^{-1}P$ est triangulaire supérieure parce que L^{-1} est triangulaire inférieure et que P renverse lignes et colonnes. La démonstration est en tête de la fonction `facteur_inverse` ; le facteur ainsi obtenu s'écarte de $7 \cdot 10^{-14}$ du facteur direct (chapitre 3).

La règle de choix. La colonne j choisit le symbole (signe $\pm$, classe k) qui minimise

$$k_j = \operatorname*{argmin}_k \big(|\tilde{w}_j| - c_k\, s_b\big)^2 + \lambda\, \bar{c}\, U_{jj}^2\, \ell(\pm, k), \qquad \ell = -\log_2 p, \tag{10.3}$$

où $\tilde{w}_j$ est le poids déjà corrigé par les colonnes précédentes, p la loi des symboles de la base frontière du même tenseur, et $\bar{c} = \operatorname{tr}(\Delta W_F H \Delta W_F^\top)/(RC)$ l'erreur moyenne d'un poids de la frontière dans les unités de H. Divisé par U_{jj}^2, le premier terme est l'accroissement de l'erreur de sortie (lemme 10.1) et le second vaut $\lambda\,\bar{c}$ fois la longueur de code : λ se lit en « fractions de l'erreur moyenne d'un poids, par bit ». Puis l'erreur commise est reportée sur les colonnes suivantes,

$$\varepsilon_j = \pm\,\frac{|\tilde{w}_j| - c_{k_j} s_b}{U_{jj}}, \qquad \tilde{w}_F \leftarrow \tilde{w}_F - \varepsilon_j\, U_{j,F}.$$

$\lambda = 0$ est la rétroaction seule ; $\lambda = 1$ ajoute la contrainte d'entropie. C'est mot pour mot ce que fait le code de l'éclaireur (`outils/genome/eclaireur/synonyme.py`, fonctions `hessienne`, `facteur_inverse`, `choisir`), et le lot B ne l'a pas modifié.

Le témoin qui sait tomber. Avec $H = I$ et $\lambda = 0$, $U = I$: aucune correction, chaque poids prend le centre le plus proche. Les centres étant les centres de gravité de classes contiguës, le centre le plus proche d'une grandeur est celui de sa classe, et la synonyme rend *exactement* la frontière (0 symbole différent ; avec le H réel, 1 789 132 symboles différents sur les tenseurs de l'éclaireur, chapitre 3).

Le décodeur ne change pas. Mêmes huit symboles, mêmes quatre centres, mêmes échelles : le mécanisme vit tout entier dans le codeur, et son coût de transcription est nul. C'est ce que dit l'analogie, dans la mesure où elle dit quelque chose : le « ribosome » (le décodeur) lit un codon synonyme sans le savoir.

10.2.3 La transformation dans la base propre de H

Proposition 10.2 (L'erreur de sortie dans la base propre)**.** *Soit (u_k, λ_k) les couples propres de la matrice symétrique positive H et $e_k = \|\Delta W u_k\|^2$ l'énergie de l'erreur dans la direction u_k. Alors*

$$\mathscr{E}(\hat{W}) = \operatorname{tr}(\Delta W H \Delta W^\top) = \sum_k \lambda_k\, e_k, \qquad \|\Delta W\|_F^2 = \sum_k e_k.$$

Démonstration. $H = \sum_k \lambda_k u_k u_k^\top$, donc $\operatorname{tr}(\Delta W H \Delta W^\top) = \sum_k \lambda_k \operatorname{tr}(\Delta W u_k u_k^\top \Delta W^\top) = \sum_k \lambda_k \|\Delta W u_k\|^2$. Les u_k formant une base orthonormée, $\sum_k u_k u_k^\top = I$, et le même calcul avec $\lambda_k = 1$ donne la norme de Frobenius. ∎

Lecture. La proposition dit ce que fait la mutation synonyme : elle abaisse $\sum_k \lambda_k e_k$ en *déplaçant* l'énergie e_k vers les directions où λ_k est petite, au prix d'une énergie totale $\sum_k e_k$ plus grande, c'est-à-dire d'une erreur *sur les poids* plus haute. Elle est synonyme pour la sortie, pas pour les poids.

Figure 10.1 le montre sur une matrice, avant et après : la couche 8, l'expert 0, la matrice gate du 20b, frontière contre synonyme $\lambda = 1$ [mes.]. La matrice H de calibration y est de rang 1 329 : les 1 551 directions au-delà, sur 2 880, ne pèsent rien en calibration, mais les entrées d'évaluation les pèsent encore (au moins 0,119). L'amortissement $\delta^\star = 1$ ajoute à chaque direction $\overline{\operatorname{diag} H} = \operatorname{tr} H/2\,880 = 0{,}41$, plus que ce poids d'évaluation des directions muettes ; seules 433 valeurs propres le dépassent. Le synonyme augmente l'erreur sur les poids de 20 % ($\sum_k e_k$: 40,07 → 48,01) et la déplace vers les directions muettes (de 54 % à 61 % de $\sum_k e_k$ au-delà du rang). L'erreur de sortie, elle, baisse :

$$\begin{aligned} \textstyle\sum_k \lambda_k e_k &: 16{,}50 \to 6{,}15\ (\div 2{,}68) \text{ en calibration,} \\ \textstyle\sum_k (u_k^\top H_{\text{év}} u_k)\, e_k &: 16{,}09 \to 10{,}28\ (\div 1{,}57) \text{ vue par l'évaluation.} \end{aligned}$$

L'écart entre ÷2,68 et ÷1,57 est le sur-apprentissage qui reste à $\delta^\star = 1$: le coût de la frontière vu par l'évaluation reste à son coût de calibration (× 0,975), celui du synonyme monte (× 1,67). L'éclaireur l'avait dit d'une phrase : le synonyme d'un échantillon n'est pas le synonyme de la loi. Une remarque de lecture, venue du juge des figures : l'aire ombrée entre les deux courbes du panneau (b′) *n'est pas* la mesure de ce sur-apprentissage, puisque la frontière en a une aussi (0,17, contre 0,32 pour le synonyme) ; la part cumulée en calibration atteint 1 au rang de H pour toute variante.

Une rangée, vue de près. Figure 10.2 descend au niveau des poids, dans la même matrice : une rangée de 32 poids (ligne 2 118, bloc 52), tirée parmi les blocs dont le nombre de symboles changés égale la médiane sur la matrice, donc typique par construction [mes.]. La mutation y change 4 symboles sur 32 et en éloigne 3 de leur valeur exacte (le quatrième, un zéro, change de signe à la même distance) ; l'erreur sur les poids du bloc monte de 7,39 à 11,27 (en unités de s_b^2). Sur la matrice entière, 17,5 % des symboles changent (5,59 par bloc en moyenne). Une rangée ne peut pas montrer où va l'erreur : c'est la base propre (figure 10.1) qui le montre.

10.2.4 L'erreur dans les directions des entrées

Figure 10.3 regarde la même matrice depuis ses entrées : les 1 329 entrées réelles de calibration de l'expert, projetées sur les premiers axes propres de H ($\lambda_1 = 279{,}5$, $\lambda_2 = 33{,}5$, $\lambda_3 = 21{,}7$, 28,2 % de la trace), et, dans chaque direction unité v d'un plan, l'erreur de sortie $\|\Delta W v\|$ pour une entrée unité de direction v [mes.] [calc.]. Dans un plan $B = [u_1\ u_k]$, cette courbe est la fonction d'appui de l'ellipse $M^{1/2}$, $M = B^\top \Delta W^\top \Delta W B$: pour $v = Ba$, $\|a\| = 1$, $\|\Delta W v\|^2 = a^\top M a$. La frontière est presque un cercle : elle répartit son erreur à plat. La synonyme l'écrase le long de u_1, où se projettent les entrées : 90 % des projections sont à moins de 30° de u_1 dans le plan (u_1, u_2) et de 24° dans le plan (u_1, u_3), et pour une entrée unité de ces cônes l'erreur du synonyme vaut au plus 0,133 et 0,135 de celle de la frontière. Le long des axes eux-mêmes, le rapport vaut 0,082 (u_1), 0,217 (u_2) et 0,269 (u_3).

Ce que la borne de 0,135 ne dit pas. Elle porte sur les *projections* des entrées dans ces plans, pas sur les entrées entières : 72 % de la norme au carré d'une entrée est hors des trois premiers axes, et une entrée fait un angle médian de 59° avec le plan (u_1, u_2). Sur les 1 329 entrées réelles dans leurs 2 880 dimensions, le rapport $\|\Delta W_s x\|/\|\Delta W_f x\|$ vaut 0,61 en médiane, de 0,49 à 0,81, et aucune n'est sous 0,135. La première version de la figure suggérait l'inverse ; le second juge des espaces latents l'a relevé, et la légende jugée dit maintenant les deux nombres. Dans les 2 877 autres directions, pondérée par les entrées, l'erreur de sortie baisse aussi, de 11,96 à 6,08, et ces directions portent 57 % du gain de sortie : le gain n'est pas seulement là où la figure regarde.

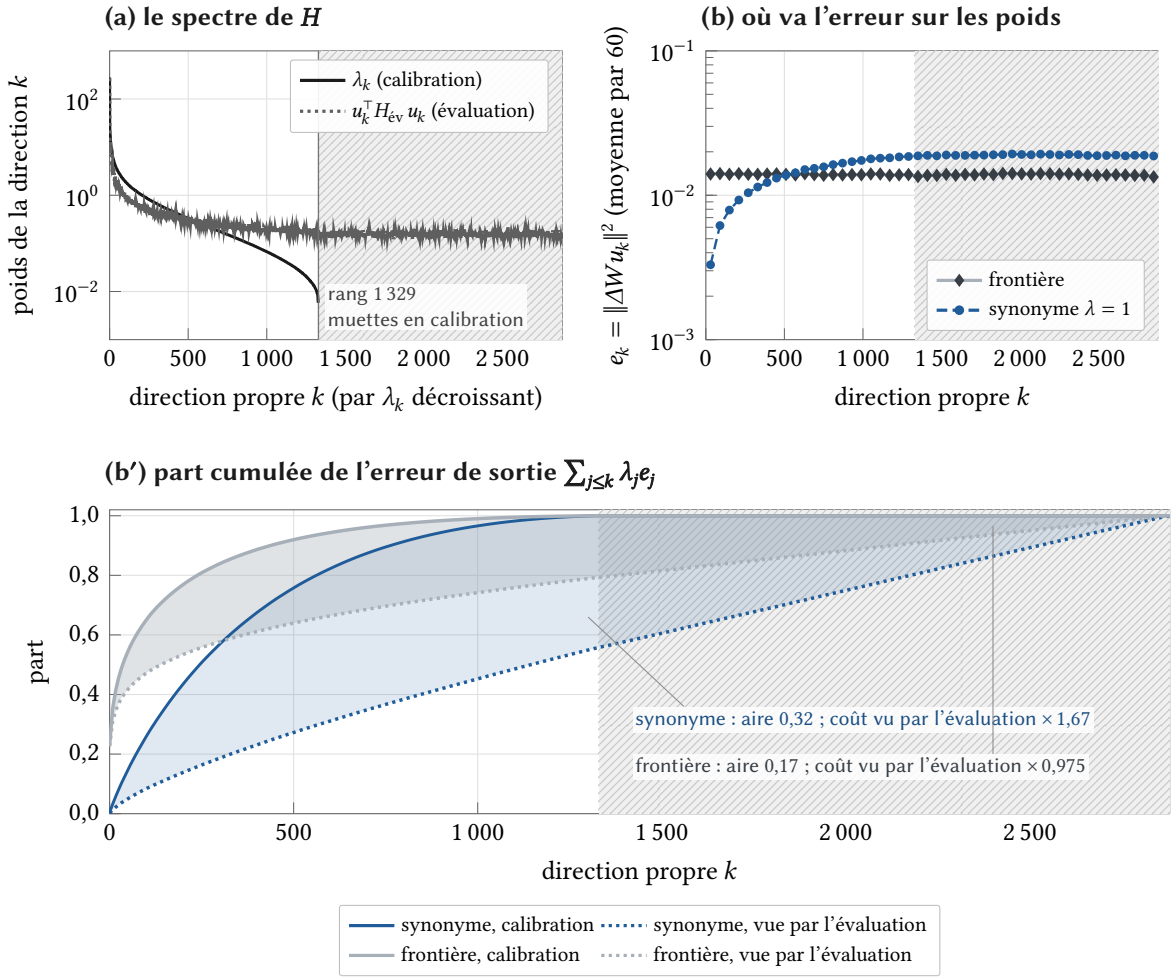


Figure 10.1 : Le synonyme baisse le coût de calibration en poussant l'erreur vers les directions que la calibration ne voit pas ; l'évaluation, elle, les voit. [mes.] [calc.] Avant (frontière) et après (synonyme $\lambda = 1$), pour une matrice : couche 8, expert 0, gate de gpt-oss-20b ; $H = \mathrm{E}[g^2 xx^\top]/\mathrm{E}[g^2]$ sur les entrées de calibration ; le coût de sortie vaut $\mathrm{tr}(\Delta W H \Delta W^\top) = \sum_k \lambda_k e_k$. (a) Le H de calibration est de rang 1 329 (le nombre d'entrées de calibration) : les 1 551 directions au-delà (hachurées), sur 2 880, ne pèsent rien en calibration, mais les entrées d'évaluation les pèsent encore (au moins 0,119). Le codeur, lui, a minimisé le coût avec $H + \delta^* \overline{\mathrm{diag}\, H}\, I$, $\delta^* = 1$ (07 § 2) : 0,41 ajouté à chaque direction, plus que le poids d'évaluation des directions muettes ; seules 433 valeurs propres le dépassent. (b) Le synonyme augmente l'erreur sur les poids, $\sum e_k$ 40,07 → 48,01 (+20 %), et la déplace vers les directions muettes (54 % → 61 % de $\sum e_k$ au-delà du rang). (b′) La part cumulée de l'erreur de sortie, en calibration (trait plein) et vue par les entrées d'évaluation (pointillés). Le coût de sortie baisse de × 1/2,68 en calibration (16,50 → 6,15) mais seulement de × 1/1,57 vu par l'évaluation (16,09 → 10,28) : le coût de la frontière vu par l'évaluation reste à son coût de calibration (× 0,975), celui du synonyme monte de × 1,67 ; c'est le sur-apprentissage résiduel. L'aire ombrée entre les deux courbes n'en est pas la mesure : la frontière en a une aussi (0,17, contre 0,32 pour le synonyme), parce que la part en calibration atteint 1 au rang de H, pour toute variante. Une matrice d'un expert ; les deux autres spectres du même dossier ne sont pas tracés.

Données : `mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv` ; script `memoire/outils/tome3_synonyme.py`.

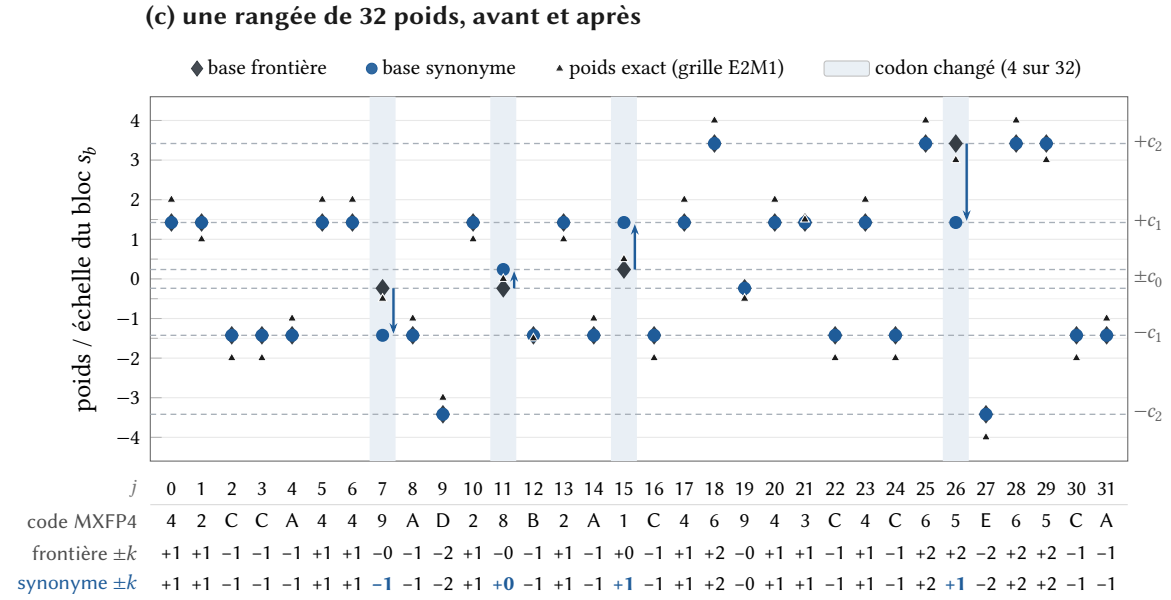


Figure 10.2 : La mutation synonyme change 4 des 32 symboles de cette rangée et en éloigne 3 de leur valeur exacte (l'autre, un zéro, change de signe à la même distance) : elle est synonyme pour la sortie, pas pour les poids. [mes.] Panneau (c) de la figure 10.1 : une rangée de 32 poids de la même matrice que les panneaux (a), (b) et (b′) (gpt-oss-20b, couche 8, expert 0, gate) : ligne 2 118, bloc 52 (colonnes 1 664 à 1 695), en unités de l'échelle du bloc $s_b = 2^{-9}$. Triangles noirs : le poids exact, sur la grille E2M1 ; losanges gris : la base frontière (son symbole est une fonction du code) ; ronds bleus : la base synonyme ($\lambda = 1$, $\delta^* = 1$), telle que la servent les fichiers `.sym` du lot B ; une flèche, à côté des marques, va de la valeur frontière à la valeur synonyme là où le symbole change. Tirets : les niveaux $\pm c_k$ de la base ($c_0 = 0{,}237$, $c_1 = 1{,}424$, $c_2 = 3{,}419$; $c_3 = 6{,}000$ sort du cadre), les mêmes pour les deux bases. Dans la bande, j est la colonne dans le bloc, puis le code MXFP4 (hexadécimal), puis le symbole de chaque base : « $+k$ » et « $-k$ » notent le niveau $\pm c_k$ ($k = 0$ à 3), pas un zéro signé. Le bloc est tiré (graine) parmi les 26 155 blocs dont le nombre de symboles changés égale la médiane sur la matrice (4 sur 32 ; moyenne 5,59, 17,5 % des symboles de la matrice) : typique par construction. Dans ce bloc, le synonyme augmente l'erreur sur les poids (11,27 contre 7,39, en s_b^2) ; sur la matrice entière, l'énergie de l'erreur monte de 20 % (40,07 à 48,01) quand l'erreur de sortie baisse 2,7 fois en calibration (16,50 à 6,15) et 1,57 fois vue par l'évaluation. Non montré : la sortie, que seuls (a), (b) et (b′) montrent ; une rangée ne peut pas montrer où va l'erreur (vers les directions muettes de H, panneau (b)). Témoin : les symboles lus redonnent $\|W - \hat{W}\|_F^2 = \sum_k e_k$ du fichier du lot B à $1{,}8 \times 10^{-12}$ relatif près (les fichiers d'un autre expert : un écart relatif de 41).

Données : `mesures/genome/tome3/latents/rangee-L08-E00-gate.csv`, `rangee-L08-E00-gate.json` (graine 20261027), par `outils/genome/tome3/latents_rangee.py` ; `mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv` ; `memoire/outils/tome3_latents.py`.

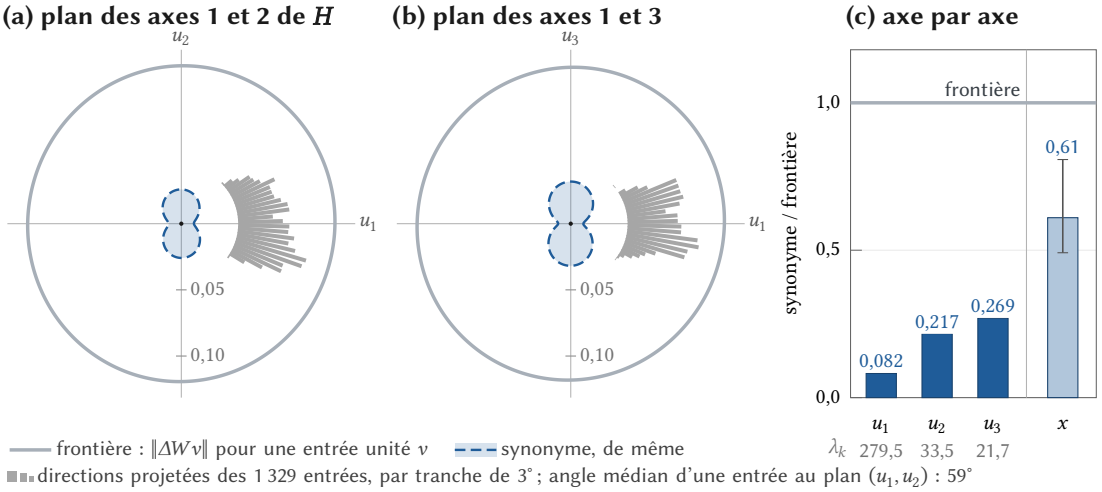


Figure 10.3 : Dans les plans des premiers axes de H, où se projettent les entrées de l'expert, le synonyme ramène l'erreur de sortie à au plus 0,135 de celle de la frontière ; sur les entrées entières, dont 72 % de la norme au carré est hors de ces trois axes, à 0,61 (médiane ; de 0,49 à 0,81). [mes.] [calc.] Expert 0 de la couche 8 de gpt-oss-20b, matrice gate, et ses 1 329 entrées réelles de calibration x (les positions que le routeur lui envoie, fenêtres 10 à 21 de wiki.test), sur les premiers axes propres u_k de $H = \sum g^2 x x^\top / \sum g^2$ (non amortie, la H de la figure 10.1 ; $\lambda_1 = 279{,}5$, $\lambda_2 = 33{,}5$, $\lambda_3 = 21{,}7$, 28,2 % de la trace). (a), (b) Dans chaque direction unité v du plan, chaque courbe fermée atteint $\|\Delta W v\|$, l'erreur de sortie pour une entrée unité de direction v, avec $\Delta W = W - \hat{W}$ pour cette base : la fonction d'appui de l'ellipse $M^{1/2}$, $M = B^\top \Delta W^\top \Delta W B$, $B = [u_1\ u_k]$; l'échelle radiale, dans les unités de la sortie, est graduée sur le demi-axe bas. La frontière est presque un cercle : elle répartit son erreur à plat. Dans l'anneau entre les deux courbes, la rose compte les directions projetées des entrées, leurs projections sur le plan (longueur proportionnelle au compte ; sa place radiale n'a pas d'unité) : 90 % des projections sont à moins de ±30° de u_1 dans (a), de ±24° dans (b) ; pour une entrée unité de ce cône, l'erreur du synonyme vaut au plus 0,133 (a) et 0,135 (b) de celle de la frontière. H n'est pas centrée : u_1 porte la moyenne des entrées, toutes près de $u_1^\top x = 16{,}5$ (écart-type 1,84, contre 5,98 le long de u_2), et λ_1 doit l'essentiel à cette moyenne ($16{,}5^2 \approx 272$). (c) Le rapport synonyme / frontière de $\|\Delta W u_k\|$ le long de chaque axe : 0,082 le long de u_1, l'axe où les entrées pèsent le plus (leur moyenne), 0,217 le long de u_2, 0,269 le long de u_3 (frontière 0,116, 0,119, 0,118). À droite du filet, le même rapport pour les 1 329 entrées réelles dans leurs 2 880 dimensions, $\|\Delta W_s x\| / \|\Delta W_f x\|$: médiane 0,61, de 0,49 à 0,81 (moustache), 0 % d'entre elles sous 0,135 ; une entrée fait un angle médian de 59° avec le plan (u_1, u_2). Ces rapports viennent du journal du juge du second flux, refaits depuis le GGUF, les fichiers `.sym` et les captures ; un témoin les relie aux fichiers commités : leur rapport quadratique pondéré par g^2 égale $\sqrt{\sum_k \lambda_k e_k^{\text{syn}} / \sum_k \lambda_k e_k^{\text{fr}}}$ du fichier du lot B à 10^{-9} près (avec λ_1 faux de 1 %, il s'en écarte). Non montrés : les 2 877 autres directions, où le synonyme met l'erreur sur les poids qu'il retire ici (panneau (b) de la figure 10.1) ; pondérée par les entrées, l'erreur de sortie y baisse aussi, de 11,96 à 6,08, et ces directions portent 57 % du gain de sortie ; les entrées d'évaluation tenues à l'écart, sur lesquelles le gain se réduit (1,57 fois contre 2,7). Les courbes sont calculées depuis les ellipses commitées par le lot : M se retrouve de leurs 181 points par moindres carrés et redonne les e_k et les demi-axes du fichier du lot à $2{,}2 \times 10^{-7}$ relatif près (avec un facteur d'échelle faux de 1 %, elle s'en écarte). Témoin du lot : λ_1 à λ_3 et e_1 à e_3 égalent ceux du fichier du lot B à $3{,}4 \times 10^{-10}$ relatif près (tirée des entrées d'évaluation, λ_1 bouge de 5,5 %).

Données : `mesures/genome/tome3/latents/nuage-L08-E00-gate-points.csv`, `nuage-L08-E00-gate-ellipses.csv`, `nuage-L08-E00-gate.json`, par `outils/genome/tome3/latents_nuage.py` ; entrées réelles `outils/genome/tome3/juge2-latents/journaux/juge2-nuage-rangee.json` (`entrees_reelles`) ; `memoire/outils/tome3_latents.py` (témoins `latents-temoins.json`).

10.3 Le mode rapide ne touche pas le mode exact

Le mode rapide ajoute un chemin dans un moteur dont l'exactitude au bit est démontrée et mesurée (Tome II). Deux propositions disent ce que l'ajout garde, et ce que le noyau neuf garantit.

Proposition 10.3 (B1 : le mode exact est inchangé)**.** *Le noyau* `gemv_4_lignes` *est paramétré par le lecteur des valeurs V (*`moteur/crates/trc-noyaux/noyaux/gemv.cuh`*). Pour V =* `ValeurE2M1`*, les sorties du mode exact sont identiques au bit à celles d'avant l'ajout ; et un fichier rapide se décode par le même décodeur rANS, les mêmes tables et la même grille que le mode exact.*

Démonstration. Pour $V =$ `ValeurE2M1`, la suite des `fmaf` a les mêmes opérandes dans le même ordre qu'avant le paramètre : même valeur `valeur_e2m1x2(c)` pour chaque demi-octet, même parcours (mot t, octet k, demi-octet faible puis fort), même réduction en papillon. Une suite d'opérations flottantes identique en opérandes et en ordre rend des résultats identiques au bit ; le paramètre se résout à la compilation et n'ajoute aucune opération. Le décodeur rANS, lui, ne lit jamais la valeur d'un symbole (Tome II, spécification § 4.9) : il rend des demi-octets ; c'est le drapeau RAPIDE de l'en-tête qui dit qu'ils sont des symboles de base, et c'est la GEMV qui les lit autrement. Témoin de carte : le mode exact avant et après un passage en mode rapide, sur les mêmes octets, rend les mêmes sorties au bit (test `gemv_base_exacte_au_bit_et_mode_exact_inchange`) [mes.]. ■

Proposition 10.4 (B2 : `gemv_base` est déterministe, et exact sur un témoin)**.** *`gemv_base` calcule $y = \hat{W}x$ avec $\hat{W}_{rc} = \pm c_k \cdot 2^{e-128}$ dans le même découpage et le même ordre que* `gemv_mxfp4`*. Si les centres sont des multiples de $\frac{1}{2}$ au plus égaux à 12, les échelles dans $[2^{-11}, 2^{-5}]$ et les entrées dans $\{-2, \dots, 2\}$, la sortie de la carte égale au bit la référence en double précision.*

Démonstration. Le déterminisme. La valeur $v(c) = \pm c_{c\,\&\,3}$ est choisie par des sélections, sans lecture indexée ni divergence d'accès mémoire, et l'ordre des opérations est fixé par le découpage, comme pour `gemv_mxfp4` (Tome II, chapitre des noyaux). *L'exactitude sur le témoin.* Un terme vaut $c \cdot s \cdot x$ avec $c = a/2$ (a entier, $|a| \le 24$), $s = 2^{-m}$ ($5 \le m \le 11$) et x entier, $|x| \le 2$: c'est un multiple de $2^{-(m+1)}$, donc de 2^{-12}, et $|csx| \le 12 \cdot 2^{-5} \cdot 2 = 0{,}75$. Une ligne de 2 880 termes reste sous $2\,880 \times 0{,}75 = 2\,160 < 2^{12}$, et toute somme partielle aussi, dans tout ordre. Un multiple de 2^{-12} de valeur absolue sous 2^{12} s'écrit sur $12 + 12 = 24$ bits significatifs : il est représentable exactement en f32 (24 bits de mantisse). Chaque produit et chaque addition tombent donc sur un flottant f32 exact, aucune opération n'arrondit, et le résultat ne dépend ni de l'ordre ni de la précision : il égale la somme exacte, que la référence f64 calcule aussi exactement. Mesuré : 0 ligne différente sur 2 880 [mes.]. ■

Le témoin de B2 est choisi pour pouvoir tomber : la cassure (le mode exact lancé sur les mêmes octets, qui lit des codes et non des symboles) rend des lignes différentes, et tombe [mes.].

10.4 Le compagnon `<nom>.rapide.trc`, au bit près

Le compagnon est un fichier `.trc` ordinaire (Tome II, chapitre du format), avec ce que donne tableau 10.2 ; tout le reste est le format exact. Le schéma de la figure prévue (III-9.5) n'a pas été tracé : le tableau en tient lieu.

La partition et les centres. La clé `.partition` range les huit grandeurs E2M1, en grandeurs doublées $\{0, 1, 2, 3, 4, 6, 8, 12\}$, dans les classes 0 0 1 1 1 2 2 3 : $\{0, 1\}, \{2, 3, 4\}, \{6, 8\}, \{12\}$. Chaque centre est dans sa classe, et le dernier est la grandeur 12 elle-même ; le facteur 2^{e-128} au lieu de 2^{e-127} compense le doublement. Ce sont ces classes contiguës qui rendent vrai le témoin $H = I$ (section 10.2.2).

Table 10.2 : Le compagnon `<nom>.rapide.trc`, champ par champ : ce qui diffère du fichier exact, et pourquoi.

Élément	Valeur	Raison
version	1.2	le drapeau RAPIDE est défini en v1.2
drapeaux	bit 0 ALIGNÉ + **bit 5 RAPIDE** ; bits 1 (niveaux), 2 (brut), 3 (canonique) refusés avec lui	un lecteur 1.0 ou 1.1 refuse le fichier : le mode exact ne peut pas lire des symboles par erreur
MÉTA	les paires du GGUF, recopiées du fichier exact, plus `troncons.rapide.codeur`, `.variante` (synonyme ou frontière), `.exact` (empreinte64 de l'en-tête du fichier exact, en hexadécimal), `.partition` (`0 0 1 1 1 2 2 3`), `.centres`, `.lambda`, `.amortissement`, `.calibration`	provenance ; la vérification contrôle le lien `.exact`
RÉSIDENTS	recopiés octet pour octet du fichier exact	le compagnon s'ouvre seul
TABLES	par (couche, matrice) : DELTA (contexte 0), CODES_CTX (contextes 0, 1, 2 : comptes des demi-octets de base sur les 32 experts), **CENTRES** (contexte 0 : 4 f32, grandeurs doublées)	même décodeur, mêmes tables ; la GEMV lit les centres
tronçon	MEDIANES, BIAIS, DELTA, CODES_CTX pour les 3 matrices (12 flux, comme en v1.0)	échelles et biais de l'expert exact
demi-octet	bit 3 : le signe ; bits 1-0 : la classe k ; bit 2 : nul. L'octet t d'un bloc porte la colonne $32b + t$ (demi-octet faible) et $32b + 16 + t$ (fort)	disposition de `block_mxfp4`
valeur servie	$\pm c_k \cdot 2^{e-128}$, $c = (0{,}474\,331, 2{,}848\,596, 6{,}837\,154, 12{,}0)$ en grandeurs doublées	`gemv_base`
empreinte d'origine	empreinte64 de l'*expert rapide* (blocs [e, demi-octets de base], puis biais)	le mode témoin du moteur la vérifie sans changement

Données : `genome/07-genome-synonyme.md`, § 3 ; `genome/construction-synonyme-pour-le-memoire.md`, § 2.5.

Trois écarts au cahier , chacun pour ne toucher aucun chemin du mode exact (*réponses par délégation du 2026-10-17, réversibles*) : (1) la sorte de flux CODES_CTX au lieu d'une sorte neuve BASE_CTX, pour que le décodeur de la carte, ses tables et sa grille restent ceux du mode exact, mot pour mot, le drapeau portant le sens ; (2) les résidents recopiés au lieu d'une section vide, pour que le moteur ouvre le compagnon comme tout fichier (coût : 1,9 Go de disque) ; (3) l'empreinte d'origine prise sur l'expert rapide, la provenance de l'expert exact restant dans l'en-tête (SHA-256 du GGUF) et dans `.exact`.

10.5 Les mesures de la qualité

10.5.1 Le protocole et ses témoins

Déclaré avant : la référence f64 du lot R avec substitution (`genome-capture kl --sub`), les 768 experts servis par leur base, sur les dix fenêtres de R1 (2 550 positions notées, tenues à l'écart de la calibration), jugée par `juge kl` contre la référence exacte ; la différence appariée position par position, et son écart-type *par fenêtres* (les dix moyennes de fenêtre, $\sigma = s/\sqrt{10}$), comme dans toute la quête. La KL est celle de Kullback et Leibler [11], au sens et avec la coupe du Tome II ; le premier choix porte son intervalle de Wilson [73].

Deux témoins ont été déclarés mal, et corrigés à leur place, barrés, avec l'heure :

– ~~le journal de chaque passe porte 768 experts substitués~~ : corrigé le 17 octobre à 10 h 50, après la passe frontière, qui porte 732. Le compteur compte les experts *chargés*, pas les fichiers présents, et 36 experts ne sont routés par aucune des 2 550 positions. Le témoin juste est « substitués = lus » : frontière 732 sur 732, synonyme 725 sur 725, $\lambda = 0$ 729 sur 729 ; les ensembles diffèrent parce que le routage dépend des experts servis. Cassure connue : la passe de l'éclaireur, 32 substitués sur 731 lus [mes.].
– ~~la KL recalculée depuis le CSV égale celle du juge à 10^{-12} près~~ : corrigé à 11 h 35, après une chute sans cause sur la passe synonyme. Le CSV porte dix chiffres significatifs, soit au plus $5 \cdot 10^{-10}$ d'erreur relative par valeur ; le seuil absolu venait des KL de 0,03 de l'éclaireur. Le seuil juste est 10^{-9} relatif : écarts mesurés $3{,}5 \cdot 10^{-12}$, $8 \cdot 10^{-12}$ et $5 \cdot 10^{-12}$; la cassure (une ligne retirée) tombe [mes.].

La leçon est celle du lot : un témoin de compteur se déclare sur ce que le compteur compte.

10.5.2 Le repère de l'outil de référence

llama.cpp b11515 [124] ne sait requantifier les experts de gpt-oss sous 2,6 bits qu'en Q2_0 (2,25 bits par poids, groupes de 64) : ses k-quants et ses IQ ont des super-blocs de 256 poids, que 2 880 ne divise pas. Experts seuls requantifiés (le reste du GGUF inchangé, 6 979,93 Mio contre 11 536,18), dix fenêtres de R1 sur le processeur, options de la base « processeur » : KL 1,597 ± 0,019, médiane 1,314, premier choix 13,6 %, perplexité 881,2 [mes.]. C'est un repère faible, qu'il faut dire tel : un arrondi au plus proche à deux bits, sans matrice d'importance, détruit presque le modèle.

10.5.3 La KL de bout en bout, les 768 experts substitués

Table 10.3 : La KL de bout en bout sur R1, les 768 experts de gpt-oss-20b servis par leur base (référence f64 substituée, 2 550 positions) [mes.]. L'incertitude naïve (par positions) est celle que 07 publie ; celle par fenêtres est la juste (section 11.5.3).

Variante	KL	$\pm\sigma$ naïf	$\pm\sigma$ fenêtres	médiane	99,9^{e} c.	premier choix	perplexité
frontière	0,264 0	0,006 6	0,037 6	0,163	4,07	47,8 %	274,1
synonyme $\lambda = 1$	0,152 2	0,006 2	0,020 3	0,061 5	3,93	67,5 %	466,5
synonyme $\lambda = 0$	0,131 8	0,005 5	0,018 2	0,055 9	3,37	69,9 %	411,0
llama.cpp Q2_0 (repère)	1,597	0,019	0,173	1,314	6,78	13,6 %	881,2

Données : `mesures/genome/construction/synonyme/kl/kl-\{frontiere,synonyme,synonyme-10\}-768-juge.json`, `repere/kl-q2_0-juge.json` ; σ par fenêtres recalculé depuis les fichiers par position par `memoire/outils/tome3_synonyme.py` ; perplexité de la référence exacte : 460,2.

Ce que disent ces nombres (tableau 10.3) :

– Servir *tous* les experts par la base de la frontière fait une KL de 0,264, sept fois celle de 32 experts substitués (0,036, protocole de l'éclaireur) : la KL croît moins vite que le nombre d'experts substitués (×24 d'experts, ×7 de KL), ce qu'on attend d'erreurs dont l'essentiel passe par des bascules de routage, qui saturent [calc.]. Les deux nombres sont de deux protocoles ; on les compare ici par leur rapport, on ne les joint par aucun trait (réserve R3).
– La synonyme garde son avantage relatif quand on passe de 32 à 768 experts. Le lot écrivait que « le gain ne fond pas ; il grandit », en comparant $\lambda = 0$ à 32 experts (rapport à la frontière 0,727)

à $\lambda = 1$ à 768 (0,576) : deux variantes. À variante égale, $\lambda = 0$ passe de 0,727 à 0,499, et la phrase reste vraie (constat M5 du juge).
– Les incertitudes absolues publiées par 07 sont naïves : par positions, elles ignorent que les 255 positions d'une fenêtre partagent leur passé. Par fenêtres, elles sont de 3,2 à 8,9 fois plus larges selon la série (constat M1 du juge ; dérivation au section 11.5.3). Aucun verdict n'en dépend : tous reposent sur des différences appariées prises avec σ par fenêtres.

Les quantiles (frontière → synonyme $\lambda = 1$) : 90e centile 0,565 → 0,328 ; 99e 1,457 → 1,295 ; maximum 6,49 → 6,94 [mes.]. La part lisse baisse beaucoup, la queue peu : médiane ÷ 2,65, 90e centile ÷ 1,72, 99e ÷ 1,12, 99,9^{e} ÷ 1,04 [calc.]. Plus on va dans la queue, moins le synonyme y fait : ce sont les bascules de routage, que la pondération par les entrées d'un seul expert ne voit pas (chapitre 8).

10.5.4 Le critère B2, et le départage des deux λ

B2 (i), synonyme $\lambda = 1$ contre la frontière. La règle, déclarée avant : $d = \mathrm{KL}_{\text{synonyme}} - \mathrm{KL}_{\text{frontière}}$ position par position ; meilleure si $\bar{d} + 2\sigma < 0$, égale si $|\bar{d}| \leq 2\sigma$, pire sinon. Mesuré : $\bar{d} = -0{,}111\,8$, σ par fenêtres 0,029 6 (naïf 0,004 5), $\bar{d} + 2\sigma = -0{,}052 < 0$: **meilleure** ($3{,}8\,\sigma$), négative dans les dix fenêtres, de −0,036 (fenêtre 4) à −0,345 (fenêtre 7) ; premier choix apparié +0,197 ± 0,011 [mes.]. La porte du pas suivant (le mode rapide dans le moteur), déclarée « si (i) tient », est ouverte.

B0, la mesure qui manquait d'abord. Faite *après* la passe à 768 experts, sur le protocole de l'éclaireur (32 experts, 2 550 positions ; les symboles $\lambda = 1$ des 32 experts sont un sous-ensemble des 768) [mes.] :

32 experts (éclaireur)	KL	contre la frontière	contre $\lambda = 0$	premier choix
frontière	0,036 0	—		87,1 %
$\lambda = 0$ (éclaireur)	0,026 2	−0,009 8 ± 0,003 3, meilleure	—	90,9 %
$\lambda = 1$	0,035 0	−0,001 0 ± 0,002 2, égale	+0,008 8 ± 0,003 2, pire	87,7 %

À 32 experts, la contrainte d'entropie rend presque tout le gain de KL de $\lambda = 0$ contre ses 0,078 bit ; à 768, $\lambda = 1$ bat la frontière de 42 %. Les deux faits ne se contredisent pas (des experts différents, un régime de bascules différent), mais ils disent que la comparaison décisive est $\lambda = 0$ contre $\lambda = 1$ *à 768 experts*. La règle en a été déclarée le 17 octobre à 12 h 15, après B0 et avant le codage $\lambda = 0$ (commit `ff36671`) : $d = \mathrm{KL}_{\lambda=0} - \mathrm{KL}_{\lambda=1}$; si $\bar{d} + 2\sigma < 0$, $\lambda = 0$ devient la variante du mode rapide ; sinon $\lambda = 1$ reste, qui a 0,07 bit de moins.

B2 entre les deux λ. Codage $\lambda = 0$, $\delta^{\star} = 1$ des 768 experts (768 fichiers de 12 441 636 octets, soit $4 + 32 + 3 \times 2\,880 \times 1\,440$), puis la même passe KL (le 21 octobre, 1 554 s). Mesuré : $\bar{d} = -0{,}020\,4$, σ par fenêtres 0,009 7 (naïf 0,003 8), $\bar{d} + 2\sigma = -0{,}000\,9 < 0$: $\lambda = 0$ **meilleure, de justesse** ($2{,}09\,\sigma$). Neuf fenêtres sur dix vont dans ce sens ; la fenêtre 7 va dans l'autre (+0,061 : 0,113 → 0,174). $\lambda = 0$ a moins de KL sur 1 706 positions sur 2 550 ; premier choix apparié +2,4 ± 0,8 points ; contre la frontière, −0,132 2 ± 0,023 4 ($5{,}6\,\sigma$) et +22,1 ± 1,1 points de premier choix [mes.]. Par la règle, $\lambda = 0$ devient la variante du mode rapide.

Le juge, sur dix fenêtres neuves. Le juge de la construction a tiré dix fenêtres de wiki.test que ni la calibration ni R1 ne touchent (section 11.3) et refait B2 par le moteur : $\bar{d} = -0{,}030\,3 \pm 0{,}008\,4$, $3{,}6\,\sigma$, neuf fenêtres sur dix (la fenêtre 0 contre, +0,016) ; et $\lambda = 0$ contre la frontière, par la référence

substituée : $-0{,}156\,5 \pm 0{,}033\,4$ ($4{,}7\,\sigma$), dix fenêtres sur dix [mes.]. Sa prédiction déclarée, « égales », est démentie : le choix de $\lambda = 0$ n'était pas un coup de chance des fenêtres de R1. Tableau 10.4 donne chaque fenêtre ; figure 10.4 les dessine.

Table 10.4 : Les différences appariées $\bar{d}$ de chaque fenêtre (négatives quand la seconde variante gagne) [mes.]. R1 : lot B, référence f64 substituée. Fenêtres neuves : le juge ; $\lambda = 0$ contre $\lambda = 1$ par le moteur, $\lambda = 0$ contre la frontière par la référence substituée.

Fenêtre	0	1	2	3	4	5	6	7	8	9	z
R1, $\lambda = 1$ – frontière	−0,108	−0,047	−0,116	−0,073	−0,036	−0,079	−0,069	−0,345	−0,053	−0,193	3,8
R1, $\lambda = 0$ – $\lambda = 1$	−0,028	−0,019	−0,048	−0,042	−0,037	−0,020	−0,013	+0,061	−0,020	−0,039	2,1
neuves, $\lambda = 0$ – $\lambda = 1$	+0,016	−0,024	−0,092	−0,031	−0,019	−0,028	−0,029	−0,037	−0,037	−0,021	3,6
neuves, $\lambda = 0$ – frontière	−0,101	−0,267	−0,056	−0,154	−0,085	−0,096	−0,334	−0,118	−0,050	−0,304	4,7

Données : `mesures/genome/construction/synonyme/kl/apparie-768-par-fenetre.csv`, `apparie-l1-contre-l0-768-par-fenetre.csv` ; `mesures/genome/construction/juge/kl/apparie-neuf-moteur-l0-contre-l1-par-fenetre.csv`, `apparie-neuf-sub-l0-contre-frontiere-par-fenetre.csv`. Les fenêtres neuves ne sont pas les fenêtres de R1 du même numéro.

Lire la forêt. Figure 10.4 met les trois comparaisons sur un seul axe. Contre la frontière, les deux synonymes gagnent avec une marge que personne ne discute, sur les deux jeux de fenêtres. Entre eux, l'écart est bien plus petit, et la bande $\pm 2\sigma$ de R1 frôle le zéro : c'est le « de justesse » du lot, et c'est la même règle qui donne les deux verdicts. Une remarque de chemin : le premier panneau des fenêtres neuves met $\lambda = 1$ par le moteur contre la frontière par la référence substituée, deux chemins que le juge publie « à titre indicatif seulement » ($-0{,}126\,0 \pm 0{,}036\,6$) ; sur R1, où les deux chemins existent, ce mélange déplace $\bar{d}$ de 0,001 4. Les fenêtres neuves sont de wiki.test, le corpus de R1 [125] : elles éprouvent un autre texte, pas une autre sorte de texte.

10.5.5 La queue de la KL

Figure 10.5 trace la fonction de survie $P(\mathrm{KL} > t)$ des 2 550 positions pour les trois variantes et le repère. Elle montre d'un coup d'œil les deux parts de la KL que la synthèse avait séparées (chapitre 8) : la part lisse, où le synonyme divise la médiane par 2,66 ($\lambda = 1$) et 2,92 ($\lambda = 0$), et la queue des bascules de routage, où les courbes se rejoignent.

La queue a fait écrire au lot deux phrases que la mesure a reprises, et qui restent barrées à leur place :

- ~~La queue ne bouge pas~~ (07, § 0) : barrée par le lot lui-même le 22 octobre, parce que vraie pour $\lambda = 1$ seulement.
- « Avec $\lambda = 0$, la queue baisse aussi. » Contre la *frontière*, elle tient : 99e centile 1,457 → 1,081 (−26 %) sur R1, 1,877 → 1,397 (−26 %) sur les fenêtres neuves par la référence substituée ; 99,9^{e} −17 % et −18 % [mes.]. Contre $\lambda = 1$, ~~le centile 99 baisse de 17 % et le centile 99,9 de 14 %~~ : sur les fenêtres neuves, le 99e ne baisse que de 5 % et le 99,9^{e} *monte* de 6 % (3,61 contre 3,40, moteur) ; barré par le juge (constat M2).

La raison est arithmétique : un 99,9^{e} centile de 2 550 positions ne laisse que trois positions au-dessus de lui [calc.]. Ce n'est pas une grandeur qu'on oppose d'une variante à l'autre à cette taille d'échantillon. La légende de figure 10.5 le dit, et donne la baisse au 99,9^{e} centile contre la frontière par jeu et par chemin : $\lambda = 1$ 3,4 % sur R1 mais 20,8 % sur les fenêtres neuves, $\lambda = 0$ 17,2 % et 15,9 % (synonymes par le moteur sur les fenêtres neuves ; d'où 15,9 % ici et 18 % par la référence substituée).

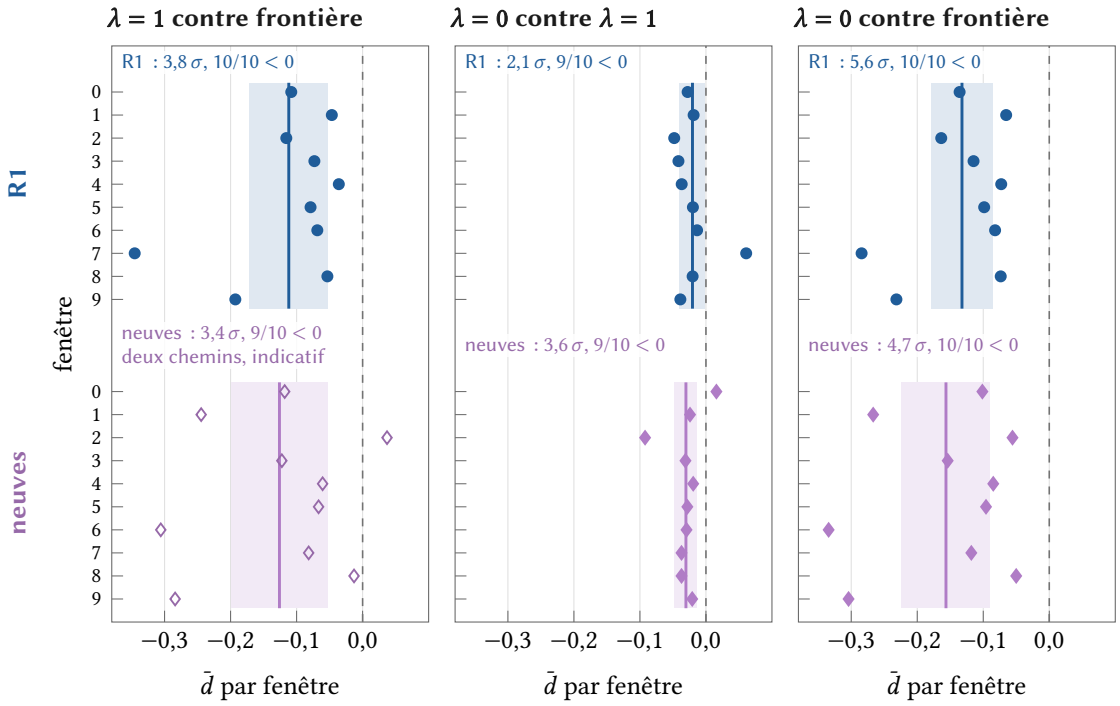


Figure 10.4 : Les deux synonymes battent la frontière sur les deux jeux de fenêtres, chacun dans au moins 9 fenêtres sur 10 ; entre eux, l'écart est bien plus petit, et $\lambda = 0$ ne passe que de justesse sur R1. [mes.] Différences appariées des KL moyennes par fenêtre, $\bar{d} = \overline{\mathrm{KL}}_b - \overline{\mathrm{KL}}_a$, négatives quand la seconde variante gagne ; un seul axe pour les trois panneaux, aucun point hors du cadre. En haut (cercles), les dix fenêtres de R1 par le lot B, par la référence f64 substituée (pour $\lambda = 0$ contre la frontière, le recalcul du juge par le même chemin). En bas (losanges), dix fenêtres neuves du juge : le panneau 1 met $\lambda = 1$ par le moteur contre la frontière par la référence substituée, deux chemins, publiés à titre indicatif seulement (JUGEMENT-construction § 9 ; losanges creux ; sur R1, où les deux chemins existent, ce mélange déplace $\bar{d}$ de 0,0014) ; le panneau 2 par le moteur ; le panneau 3 par la référence substituée. Trait vertical et bande : moyenne et $\pm 2\sigma$, σ l'écart-type des dix fenêtres divisé par $\sqrt{10}$. $\lambda = 0$ contre $\lambda = 1$ passe de justesse sur R1 (2,1 σ, la fenêtre 7 contre) et avec plus de marge sur les fenêtres neuves (3,6 σ). Les fenêtres neuves sont tirées de wiki.test, le corpus de R1 : elles éprouvent un autre texte, pas une autre sorte de texte.

Données : `mesures/genome/construction/synonyme/kl/apparie-*-par-fenetre.csv`, `mesures/genome/construction/juge/figures/apparie-par-fenetre.csv` ; mesure du mélange `mesures/genome/construction/synonyme/moteur/kl-moteur-*-positions.csv` ; script `memoire/outils/tome3_synonyme.py`.

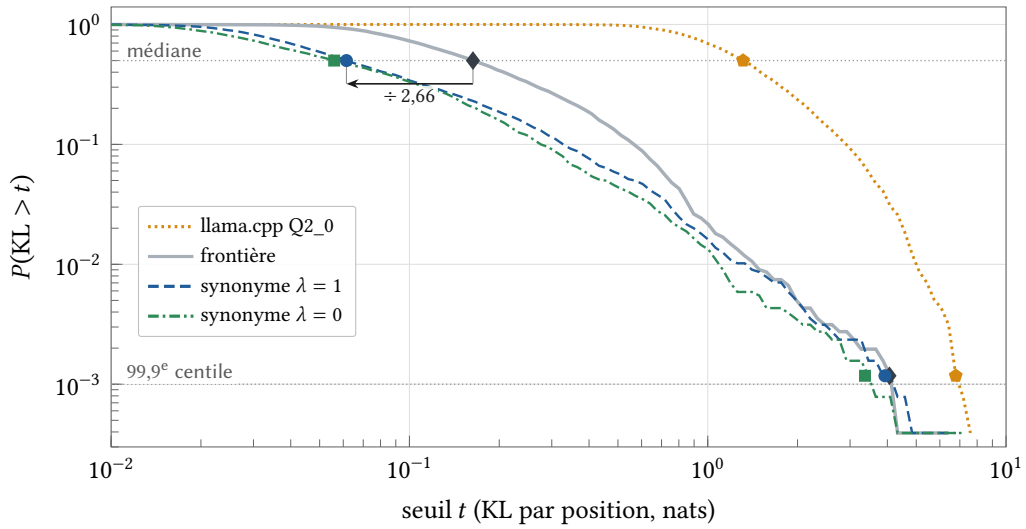


Figure 10.5 : Le synonyme divise la KL médiane par près de trois ; la queue repose sur trop peu de positions pour classer les variantes. [mes.] Fonction de survie $P(\mathrm{KL} > t)$ sur les 2 550 positions notées de R1 (10 fenêtres), les 768 experts de gpt-oss-20b substitués dans la référence f64 ; sur les lignes pointillées, les médianes et les 99,9^{es} centiles, chacun à la marque de sa série. La médiane de la frontière (0,163) est divisée par 2,66 avec $\lambda = 1$ (0,061) et par 2,92 avec $\lambda = 0$ (0,056). Chaque 99,9^{e} centile ne laisse que 3 positions au-dessus de lui : contre la frontière, $\lambda = 1$ l'abaisse de 3,4 % sur R1 mais de 20,8 % sur les dix fenêtres neuves du juge, $\lambda = 0$ de 17,2 % et de 15,9 % (fenêtres neuves : frontière par la référence substituée, synonymes par le moteur). Entre $\lambda = 0$ et $\lambda = 1$, la queue n'est pas comparable à cette taille (barré, JUGEMENT-construction M2). À gauche de 10^{-2} : au plus 0,7 % des positions d'une série. Q2_0 est calculé par llama.cpp sur le processeur, un autre chemin (un repère). Les positions d'une fenêtre partagent leur passé et ne sont pas indépendantes (figure 10.4).

Données : `mesures/genome/construction/synonyme/kl/kl-*-768-positions.csv`, `repere/kl-q2_0-positions.csv`, `kl/kl-*-768-juge.json` ; `mesures/genome/construction/juge/kl/kl-neuf-*-positions.csv` ; script `memoire/outils/tome3_synonyme.py`.

10.5.6 La perplexité ne juge pas la fidélité

La perplexité de la frontière *baisse* (274,1 contre 460,2 pour le modèle exact) pendant que sa KL est la pire : sur un texte brut, hors du format de conversation pour lequel le modèle est réglé, une distribution déformée peut mieux prédire le jeton suivant que le modèle exact. Celle du synonyme $\lambda = 1$ reste à +1,4 % de l'exact (logarithme du rapport 0,013 ± 0,016) ; celle de $\lambda = 0$ descend (411,0 ; −0,113 ± 0,015) [mes.]. Le lot en avait tiré une phrase : ~~$\lambda = 0$ déforme la distribution dans une direction privilégiée que $\lambda = 1$ ne prend pas~~. Sur les fenêtres neuves, le sens s'inverse : $\lambda = 0$ est à +2,4 % de l'exact (402,1 contre 392,5) et $\lambda = 1$ à +15,6 % [mes.] ; barré par le juge (constat M3). La leçon de la synthèse se confirme une fois de plus : seule la KL juge la fidélité.

10.5.7 Le débit des symboles

Les entropies des symboles sur les 72 tenseurs entiers des 768 experts, par l'estimateur de l'éclaireur (chapitre 3) [mes.] :

72 tenseurs, 768 experts	H_0 des symboles	H(symbole | κ)
frontière	2,641 3	2,599 2
synonyme $\lambda = 1$	2,568 3 (−0,073 0)	2,527 5 (−0,071 7)

La prédiction de l'éclaireur, sur 16 experts, était −0,078 bit. Par rôle (moyennes de H_0, synonyme contre frontière) : gate 2,556 contre 2,645 (−0,089), up 2,630 contre 2,643 (−0,013), down 2,518 contre 2,635 (−0,117), avec −0,62, −0,46 et −0,35 bit sur les down des couches 0, 1 et 2. La matrice up, qui voit les mêmes entrées que gate, ne cède presque pas de bits à la contrainte d'entropie ; pourquoi n'est pas mesuré [hyp. : l'erreur de la frontière de up, dans les unités de H, est petite devant ses écarts de longueur de code, et $\bar{c}$ normalise par elle]. Le codeur rANS réel du compagnon [119] rend 2,586 1 bits par poids contre 2,659 3 (−0,073 3) : la même différence que l'entropie, biais f32, échelles et médianes compris dans les deux.

Pour $\lambda = 0$, le compagnon réel coûte 2,671 7 bits par poids d'expert : +0,012 4 sur la frontière et +0,085 6 sur $\lambda = 1$ [mes.]. Sans contrainte d'entropie, la rétroaction choisit des symboles un peu moins prévisibles que la fonction du code : $\lambda = 0$ paie sa KL en bits, là où $\lambda = 1$ gagnait les deux à la fois contre la frontière.

10.5.8 La frontière des variantes

Figure 10.6 place les variantes dans le plan (bits par poids du compagnon réel, KL de bout en bout) : $\lambda = 1$ domine la frontière sur les deux axes ; $\lambda = 0$ achète 0,020 4 de KL avec 0,085 6 bit. Le moteur exact est hors du cadre, à 3,86 bits utiles par poids d'expert et une KL de 0,003 7 (07, § 4.3, écrit 3,85 bits ; la figure prend les bits utiles du fichier v1.0 mesurés par le juge, `mesures/genome/construction/juge/J-T0-fichiers.json`, la définition prise pour les compagnons), et Q2_0 aussi, à 2,25 bits et 1,597. Les barres sont $\pm 2\sigma$ *par fenêtres*.

Réserve R3. Ces points sont mesurés avec les 768 experts substitués ; ceux de la ronde (chapitre 8, la frontière commune) le sont avec 32 experts, et rien ne relie les uns aux autres par un trait. Une KL à 32 experts substitués et une KL à 768 ne mesurent pas la même chose : la première est la part d'un sous-ensemble, la seconde le modèle entier servi par sa base.

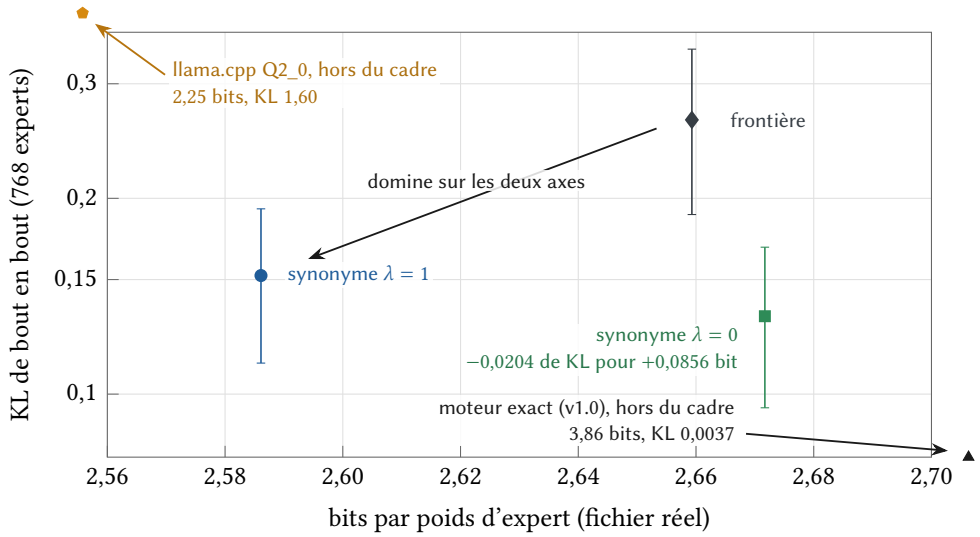


Figure 10.6 : Le synonyme $\lambda = 1$ domine la frontière sur les deux axes ; $\lambda = 0$ achète de la KL avec des bits. [mes.] Les 768 experts de gpt-oss-20b substitués : bits par poids d'expert du fichier compagnon réel (bits utiles) contre la KL de bout en bout sur les dix fenêtres de R1, en échelle logarithmique ; barres : $\pm 2\sigma$ par fenêtres, recalculé depuis les fichiers par position (pour la frontière $\sigma = 0{,}0376$, contre 0,0066 par positions : le σ par positions publié dans 07 est de 3,2 à 8,9 fois trop petit). $\lambda = 0$ achète 0,0204 de KL avec 0,0856 bit. Repères de marge, hors du cadre (flèches) : llama.cpp Q2_0 (sa KL calculée par llama.cpp sur le processeur, un autre chemin que la référence substituée) et le moteur exact contre la référence f64 (bits utiles du fichier v1.0, la définition prise pour les compagnons). Réserve R3 : ces points sont mesurés avec les 768 experts substitués ; ceux de la ronde (chapitre 8) le sont avec 32 experts, et rien ne les relie à ceux-ci.

Données : `mesures/genome/construction/synonyme/figures/variantes-768.csv`, `kl/kl-*-768-positions.csv`, `repere/kl-q2_0-positions.csv` ; `mesures/moteur/O/journaux/o3/c4-20b-comparer.json` ; `mesures/genome/construction/juge/J-T0-fichiers.json` ; script `memoire/outils/tome3_synonyme.py`.

10.6 Le mode rapide dans le moteur

S3-T1, le fichier. `trc-rapide verifier` relit chaque compagnon : pour les 768 experts, les demi-octets décodés égalent ceux du `.sym`, les échelles et les biais ceux de l'expert exact, les empreintes celles du répertoire, et le lien MÉTA vers l'en-tête exact tient. Cassures (`--casser C X`) : L7 E13 pour la frontière, L19 E2 pour $\lambda = 1$, L11 E27 pour $\lambda = 0$; chaque fois un expert faux, le visé [mes.]. Le compagnon frontière fait 8 285 093 888 octets (2,659 3 bits utiles par poids, 2,660 0 aligné) contre 11 157 405 696 pour le fichier exact v1.0 ; celui de $\lambda = 0$, 8 314 617 856 octets, écrit en 301 s.

S3-T2, le moteur. `troncons generer --rapide --temoin` sur le compagnon frontière, invite de la série O4, 32 jetons : 5 856 experts décodés sur la carte, redescendus et vérifiés, 0 erreur ; le texte est cohérent [mes.].

S3-K, les deux chemins du mode rapide. Le moteur (f32, carte) et la référence f64 rendent-ils le même modèle rapide ? Seuils déclarés : KL(référence f64 rapide ‖ moteur rapide) $\leq$ 0,011 et premier choix identique $\geq$ 95 %. Leur source : l'écart mesuré entre le moteur exact et la référence exacte sur le 20b, 0,003 68 $\pm$ 0,000 46 et 97,18 % (critère C4 du Tome II), multiplié par trois pour l'amplification par les bascules d'un modèle plus loin de l'exact ($3 \times 0{,}003\,68 = 0{,}011$) [calc.].

Table 10.5 : Le mode rapide servi par le moteur (gpt-oss-20b, 768 experts, R1) : bits, taille, KL contre la référence exacte et les deux chemins (S3-K) [mes.].

Mode servi	bits par poids	fichier	KL contre la référence	premier choix	S3-K
frontière (témoin)	2,659 3	8,29 Go	0,262 6	47,6 %	0,002 07 ; 98,5 %
synonyme $\lambda = 1$	2,586 1	8,11 Go	0,152 2	67,4 %	0,001 71 ; 98,2 %
synonyme $\lambda = 0$ (retenue par B2)	2,671 7	8,31 Go	0,131 3	69,7 %	0,003 8 ; 97,8 %
exact (v1.0)	3,86	11,16 Go	0,003 7	97,2 %	—

Données : `mesures/genome/construction/synonyme/rapide/coder-*.json`, `moteur/kl-moteur-*-juge.json`, `moteur/S3K-*.json` ; exact : critère C4 du Tome II et `mesures/genome/construction/juge/J-T0-fichiers.json`.

S3-K tient pour les trois variantes, et chaque cassure (la référence f64 *frontière* contre le moteur synonyme) tombe : 0,190 et 50,9 % de premier choix pour $\lambda = 1$, 0,149 et 55,0 % pour $\lambda = 0$ [mes.]. Le moteur sert le mode rapide aussi fidèlement qu'il sert le mode exact, et sa KL contre la référence exacte égale celle de la référence substituée à l'écart des deux chemins près (0,262 6 contre 0,264 0 ; 0,152 2 et 0,152 2 ; 0,131 3 contre 0,131 8). Sur les fenêtres neuves, le juge redonne S3-K pour $\lambda = 0$: 0,003 7 et 97,8 % (section 11.5).

10.7 La vitesse, et ce que le juge a barré

10.7.1 Les séances du lot

La règle déclarée : `troncons mesurer`, invite et options de la série O4 (708 jetons d'invite, 256 jetons gloutons, repos de 10 s), séance A B B A A B B A, A le fichier exact v1.0, B le compagnon ; secteur vérifié avant chaque passe, verrou de la carte tenu ; bruit de référence σ, l'écart relatif des passes A entre elles ; gain annoncé si le rapport médian B/A dépasse $1 + 2\sigma$. Prédit au toit du disque : ×1,41 à ×1,45 (20b), ×1,43 à ×1,46 (120b).

Séance s1 (20b) : publiée, non concluante. Elle a tourné pendant la passe KL B0 (six fils), le calcul du débit et l'écriture du compagnon du 120b, qui lit 57 Go sur le même disque. Passes A : 2,64, 2,47, 3,18, 3,61 jetons/s (médiane 2,91) ; passes B : 3,43, 3,49, 3,36, 3,39 (3,41). Rapport 1,17, $\sigma_A = 17{,}8\,\%$, seuil 1,36 : pas de gain annoncé [mes.]. Le moteur exact y tournait à 2,5 à 3,6 jetons/s contre 10,9 dans la série O4 sur une machine libre.

Séance s2 (20b), machine calme (le 22 octobre, de 1 h 11 à 1 h 25 ; deux relevés vides des autres flux au départ, aucune passe sous charge) : A le v1.0 exact, B le compagnon $\lambda = 0$ [mes.].

	passes A (exact v1.0)	passes B (rapide $\lambda = 0$)
génération, jetons/s	9,39 · 9,69 · 9,83 · 9,39	5,46 · 11,83 · 12,22 · 11,85
médiane	9,54	11,84
préremplissage (708 jetons), jetons/s	10,8 à 11,4	16,9 à 17,7
octets lus par pas de génération	0,116 4 Go	0,115 2 Go
succès en VRAM ; défauts sur 255 pas	0,899 ; 2 472	0,856 ; 3 531
débit par octet en génération	1,12 Go/s	1,39 Go/s

$\sigma_A = 2{,}3\,\%$, rapport 1,241, seuil 1,046 : gain annoncé par la règle, ~~×1,24 en génération~~, et ×1,57 au préremplissage (publié, non déclaré comme critère). Les deux fichiers ne génèrent pas le même texte (le mode rapide est avec perte ; le texte B part dans une boucle), donc les experts routés diffèrent ; par pas, les octets lus sont presque égaux, et le gain vient d'un débit par octet plus haut. Le lot en proposait une cause : ~~le cache de fichiers du système, qui garde une plus grande part d'un fichier de 8,3 Go que d'un fichier de 11,2 Go~~ [hyp.].

120b-V, machine calme (le 22 octobre, de 2 h 34 à 3 h 38) : A le fichier exact v1.0 (57,73 Go), B le compagnon frontière (40,45 Go) [mes.].

	passes A (exact v1.0)	passes B (rapide frontière)
génération, jetons/s	2,08 · 1,87 · 2,12 · 1,88	3,95 · 3,97 · 3,98 · 3,97
médiane	1,98	3,97
préremplissage, jetons/s	1,43 à 1,56	2,99 à 3,01
octets lus par pas de génération	0,707 Go	0,464 Go
débit par octet en génération	1,32 Go/s à 1,50 Go/s	1,83 Go/s à 1,85 Go/s
succès en VRAM	0,591	0,610

$\sigma_A = 6{,}6\,\%$, rapport 2,00, seuil 1,13 : gain annoncé par la règle, ~~×2,00~~, au-delà du prédit au toit du disque.

10.7.2 Une identité, et ce qu'elle sépare

Proposition 10.5 (B3 : la décomposition d'un rapport de vitesses)**.** *Soit, pour une passe X, v_X ses jetons par seconde de génération, o_X les octets lus par pas et $d_X = o_X/t_X$ son débit par octet, $t_X = 1/v_X$ le temps d'un pas. Alors, exactement,*

$$\frac{v_B}{v_A} = \frac{o_A}{o_B} \cdot \frac{d_B}{d_A}.$$

Démonstration. $d_X = o_X v_X$, donc $v_X = d_X/o_X$, et le quotient des deux donne l'identité. ∎

Le premier facteur est ce qu'un format plus petit peut rendre à débit égal : son *toit des octets*. Le second dit si le même octet a été lu plus vite. Sur le 120b, 1,524×1,31 = 2,00 ; sur le 20b, 1,010×1,24 = 1,25 (les médianes donnent 1,24) [calc.]. Le lot laissait le ×1,31 du 120b « non mesuré » et le ×1,24 du 20b « [hyp.] le cache de fichiers ». L'identité ne dit pas d'où vient le second facteur ; elle dit seulement qu'il ne vient pas des octets, donc pas du mécanisme qu'on croyait mesurer.

10.7.3 Ce que le juge a mesuré

Le juge a mis *trois* fichiers de chaque modèle dans une même séance équilibrée (A B R R B A A B R R B A) : A le v1.0 d'origine, B le v1.2 exact, R le compagnon rapide ; et il a mesuré l'étage disque seul de chaque fichier dans la même séance. Alors $r(R/A) = r(R/B) \cdot r(B/A)$, et $r(R/B)$ dit ce que vaut le mode rapide contre un fichier exact que l'étage disque lit à la même vitesse (B et R : 0,956 et 1,009 fois, sous le seuil de séparation) [mes.]. Le plan, sa démonstration et ses seuils sont au section 11.6 ; le résultat tient en quatre nombres :

	$r(R/A)$	$r(R/B)$	toit des octets o_B/o_R	étage disque R/A
120b (compagnon frontière)	1,851	1,284	1,47	1,31
20b (compagnon $\lambda = 0$)	1,220	0,942	0,98	1,39

- **Les mesures du lot se redonnent** : $r(R/A) = 1{,}220$ sur le 20b contre 1,241 ; 1,851 sur le 120b, aux réglages du J1 (`--flux`, 326 emplacements) au lieu de ceux du lot, ce qui suffit à l'écart avec 2,00.
- **Leur attribution au mode rapide tombe** (graves G1 et G2 du juge). Sur le 120b, le mode rapide frontière rend ×1,28 en génération pour un toit des octets de ×1,47, et ×1,12 au préremplissage ; sur le 20b, il génère 6 % *moins vite* que le v1.2 exact, parce qu'il lit autant d'octets par pas (115,1 Mo contre 112,5) : son routage, avec perte, manque plus souvent la VRAM (0,856 contre 0,899). Il ne gagne qu'au préremplissage (×1,23), où le texte est imposé et où les octets suivent la taille des experts.
- **La cause avancée est impossible.** Le moteur lit chaque tronçon sans passer par le cache du système (lectures sans tampon de Windows), et le disque lit 1,000 0 à 1,005 7 fois les octets demandés ; aucun cache de fichiers n'est sur le chemin.
- **Le second facteur de B3 est mesuré** : c'est l'étage disque, qui lit le compagnon 1,31 (120b) et 1,39 (20b) fois plus vite que le v1.0 d'origine à octets égaux par lecture.

Ce qui fait l'étage disque (réserve R1). Sur ce disque, l'écart suit *le motif des requêtes de lecture que le format impose* (frontières et longueurs des tronçons ; réserve R1, section 11.6.5, où sont la lecture démentie et ses mesures). Pour ce chapitre, cela change le mot, pas les verdicts : B et R ont tous deux un motif rapide (environ 11,6 et 8,3 Mo par requête), A le motif lent du v1.0 (environ 12,0 Mo), et $r(R/B)$ compare deux motifs rapides. Ces rapports valent pour ce disque, ce jour-là ; ils ne se transportent pas à une autre machine sans y être remesurés. Le gain du v1.2 sur le v1.0 lui-même est au chapitre 9.

Ce que le juge a relevé de plus. La règle de vitesse du lot ne regardait que le bruit des passes A ($\sigma_A = 2{,}3$ % sur le 20b) ; les passes B avaient $\sigma_B = 31$ % à cause de la première (5,46 jetons/s, juste après l'écriture du compagnon). Avec $\sigma = \max(\sigma_A, \sigma_B)$, le gain n'aurait pas été annoncé (constat M4). La règle déclarée a été suivie ; elle était trop étroite pour voir que le fichier B changeait de comportement entre sa première et sa deuxième lecture. La séance du juge prend $\sigma = \max(\sigma_A, \sigma_B, \sigma_R)$.

10.8 Le 120b

Pourquoi la mutation synonyme n'y est pas écrite. Trois manques, chacun chiffré [calc.] :

1. *Aucune entrée de calibration* : les captures de l'éclaireur sont celles du 20b ; il faudrait la passe de capture de la référence f64 sur le 120b (36 couches, 128 experts ; celle du 20b a pris 1 966 s pour 12 fenêtres et 9,95 Go d'experts lus).
2. *Le codage* : 4 608 experts à environ 54 s chacun (les matrices du 120b ont les mêmes dimensions) font $4\,608 \times 54 = 248\,832$ secondes, environ 69 h sur un processus, environ 23 h avec le gain de parallélisme vu sur le 20b ($\approx \times 3$). C'est le cas pour lequel le cahier prévoit le codeur sur la carte.
3. *Aucun juge* : la référence f64 du 120b sur les dix fenêtres n'existe pas (Tome II, ce qui manque à l'épreuve) ; sans elle, pas de KL de bout en bout.

Ce qui se mesure quand même (seuils déclarés le 17 octobre à 12 h 10, avant ces mesures) : le compagnon *frontière* du 120b, sans calibration, qui a la taille du mode rapide à 0,07 bit près, donc sa vitesse.

- **120b-T1** : dans la passe à cassure, 4 607 experts justes sur 4 608, le seul faux est le visé (L30 E77) ; lien MÉTA tenu [mes.].
- **120b-T2** : `troncons generer --rapide --temoin`, 32 jetons : 56 671 experts décodés sur la carte, redescendus et vérifiés, 0 erreur, texte cohérent ; 471 Go lus en 2 010 s (le mode témoin redescend chaque expert : ce n'est pas une vitesse) [mes.].
- **Un proxy de qualité, validé d'abord là où le vrai juge existe.** Sur le 20b, le moteur exact contre la référence f64 redonne C4 au dernier chiffre (0,003 68 ± 0,000 46, 97,18 %), et KL(moteur exact ‖ moteur rapide) égale la KL contre la référence f64 à 0,002 près : frontière 0,264 5 contre 0,262 6, $\lambda = 1$ 0,153 3 contre 0,152 2, $\lambda = 0$ 0,133 1 contre 0,131 3 [mes.]. L'écart du moteur exact à la référence (0,003 7) est petit devant les KL du mode rapide : le proxy les mesure à 1,5 % près.
- **Sur le 120b** (dix fenêtres de R1, `--flux` comme l'épreuve J1) : KL(moteur exact ‖ moteur rapide frontière) = 0,140 4 ± 0,004 2 (médiane 0,084 ; 99e centile 0,98 ; 99,9^{e} 2,22), premier choix 67,5 %, perplexité 401,4 contre 422,2 pour le moteur exact. Témoins : le moteur exact du 120b rend l'empreinte de CSV de l'épreuve J1 (`a80f5f04…9498`) et sa perplexité (422,21) ; exact contre exact : 0 [mes.]. Le ± est naïf, comme ceux de 07.

La même base frontière coûte deux fois moins de KL au 120b qu'au 20b (0,140 contre 0,265) : plus d'experts (128 contre 32, quatre routés chaque fois), et une part plus petite de chaque sortie passe par un expert donné [hyp., non mesurée]. Si le rapport synonyme sur frontière du 20b (0,50 pour $\lambda = 0$) se transportait, le synonyme du 120b serait vers 0,07 ; c'est une extrapolation, pas une mesure.

10.9 Ce qui reste

1. **La mutation synonyme du 120b** : la passe de capture de la référence f64 sur le 120b, le codeur sur la carte, puis la même KL de bout en bout avec la référence f64 du 120b sur les dix fenêtres.
2. **La politique mixte des défauts** (servir la base d'un expert absent pendant que l'exact arrive) est à reprendre avec $r(R/B)$ et non $r(R/A)$: sur le 120b, ×1,28 en génération pour une KL de 0,14 (proxy) ; sur le 20b, rien en génération, ×1,23 au préremplissage. Le cahier demandait de la simuler sur les traces avant de la construire ; c'est d'autant plus nécessaire.
3. **Un autre corpus** (code, dialogue) : les fenêtres neuves du juge sont de wiki.test, comme R1 et la calibration.
4. **La queue de la KL** (99,9^{e} centile 3,37 avec $\lambda = 0$) : ce sont les bascules de routage. La garde

tropicale était la pièce qui la viserait ; elle tombe au seuil déclaré (chapitre 12), ce qui laisse cette queue sans mécanisme.

5. **Le mode rapide comme brouillon** d'un décodage spéculatif vérifié par le mode exact, dont le texte serait celui du modèle exact au bit : une idée de Stone (« un genre de puissance de raisonnement différent »), non mesurée ici.

10.10 Ce qui pourrait être faux

Ce qui pourrait être faux

- **B2 entre les deux λ s'est joué de justesse sur R1** ($\bar{d} + 2\sigma = -0{,}000\,9$, une fenêtre sur dix contre). Avec dix fenêtres, σ par fenêtres est lui-même incertain d'environ 24 % ($1/\sqrt{2 \cdot 9}$) [calc.]. Le juge l'a redonné à $3{,}6\,\sigma$ sur dix fenêtres neuves, mais toujours de wiki.test et par un seul chemin pour $\lambda = 1$ (le moteur ; la référence substituée $\lambda = 1$ n'y a pas été passée). Une autre sorte de texte pourrait rendre « égales », et la règle garderait alors $\lambda = 1$. Les deux compagnons restent écrits, vérifiés et servis.
- **Les experts non routés** : de 36 à 39 experts ne sont routés par aucune des 2 550 positions de R1 sous substitution, et 11 par aucune des vingt fenêtres sous la référence exacte (figure 11.1). Leurs fichiers sont vérifiés (S3-T1) ; aucune KL ne juge leur base.
- **Le spectre et le nuage sont une matrice d'un expert** (L8, E0, gate) : la forme de la transformation y est vue, pas établie pour les 2 304 matrices. Les deux autres spectres du même dossier ne sont pas tracés.
- **Les incertitudes absolues de 07** sont naïves ; celles de tableau 10.3 par fenêtres sont justes pour des fenêtres indépendantes, ce que dix fenêtres d'un même article ne sont peut-être pas tout à fait.
- **Le repère llama.cpp est faible** (Q2_0 sans matrice d'importance) ; une requantification meilleure à 2,6 bits ou moins n'existe pas dans b11515 pour des lignes de 2 880. Battre ce repère ne dit rien de la place de la synonyme dans l'état de l'art.
- **La vitesse** : une séance par modèle et par juge, sur un seul disque, un seul matin ; les rapports $r(R/B)$ reposent sur le fait, mesuré dans la séance, que B et R se lisent à la même vitesse à l'étage disque. Si ce n'était plus vrai un autre jour, il faudrait refaire la séance, pas reprendre le chiffre. Et sur une machine où le motif v1.0 ne serait pas lent, $r(R/A)$ lui-même changerait.
- **Le 120b** n'a que la frontière (sans calibration) ; son proxy de qualité est une KL entre deux sorties du moteur, sans la référence f64, validé sur un autre modèle.
- **L'analogie** du codon synonyme décrit le mécanisme ; elle ne le prouve pas. Ce qui le prouve est la KL, sur un texte tenu à l'écart.

Chapitre 11

Le juge de la construction

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, deux lots qui se disent finis, et des chiffres produits par leurs seuls auteurs, sur les fenêtres où leurs mécanismes ont été mis au point. Le lot A publie un format `.trc` v1.2 sans perte (le signe d'un zéro fondu, 1,80 Go de moins sur le 120b), des logits identiques au bit sur R1, et un J1 du 120b 1,45 fois plus rapide en v1.2, qu'il n'attribuait pas lui-même au format (chapitre 9) ; le juge du placement l'a depuis attribué au format v1.2, par le motif de ses requêtes de lecture, propre à ce disque (la réserve R1, section 9.7.1 et section 11.6.5). Le lot B publie le génome synonyme sur les 768 experts du 20b, la règle B2 qui retient $\lambda = 0$ de justesse, un mode rapide dans le moteur et deux gains de vitesse, ×1,24 et ×2,00 (chapitre 10). Le SOCLE exige que le travail soit relu par un juge qui ne l'a pas fait. **Ce chapitre établit**, par les chemins propres d'un juge (un décodeur du format écrit depuis sa seule description, un lecteur GGUF à lui, une KL calculée par son code, dix fenêtres de texte que les lots n'ont jamais vues, une séance de vitesse à trois fichiers), ce qui tient, ce qui tombe, et ce qui est vrai mais mal attribué ; et il pose en entier les cinq dérivations sur lesquelles reposent ses verdicts. **Il reste** la décision de la politique mixte des défauts, un autre corpus, et la variable exacte du motif de lecture qui fait la vitesse de ce disque (section 11.10). Sources : `genome/JUGEMENT-construction.md`, `genome/construction-juge-pour-le-memoire.md`, `genome/JUGEMENT-placement.md`, `mesures/genome/construction/juge/` (seuils : `00-seuils-declares.md`) et `outils/genome/construction/juge/`.

En bref

Un juge refait. Il ne relit pas les calculs des autres : il en fait d'autres, par des chemins qui ne partagent avec ceux des lots que ce qu'ils doivent partager, la description du format et les fichiers jugés. Celui de la construction n'a écrit ni la décision du format v1.2, ni le codeur, ni le compagnon rapide, ni le moteur. Il a déclaré ses seuils le 22 octobre 2026 vers 4 h, quand aucun de ses outils n'avait encore lu un octet d'un modèle. Son verdict tient en deux phrases. La construction livre ce qu'elle promettait *en octets et en qualité* : le codon synonyme du zéro est sans perte et exact au bit sur deux modèles et sur un texte neuf ; le génome synonyme $\lambda = 0$ bat la frontière et $\lambda = 1$ sur ce texte neuf, plus nettement que sur le texte où il avait été choisi. Elle ne livre pas le gain de vitesse annoncé pour le mode rapide : les deux rapports publiés se redonnent, mais ils mesuraient surtout la manière dont ce disque lit l'ancien format. Ce chapitre montre comment le juge le sépare, et pourquoi on peut le croire.

11.1 Ce que le juge a refait, et par quel chemin

Ses outils. Tout est sous `outils/genome/construction/juge/` ; le moteur, `juge` et `genome-capture` sont construits par lui depuis `main` (commit `f136e27`) dans sa propre cible. Son décodeur du format (`decodeur_juge.py`, Python et numpy) lit l'en-tête, MÉTA, RÉSIDENTS, TABLES, RÉPERTOIRE et les parties de tronçon depuis la seule spécification (`decisions/ARCHITECTURE.md`, §§ 4.2 à 4.12 et 4.16), et le GGUF par son propre lecteur (v3 : paires, informations des tenseurs, alignement). Il n'a ouvert ni les crates du format, ni le codec, ni le décodeur de l'épreuve J1, ni les outils du lot A avant de l'écrire. Une seule erreur au premier lancement, dans son lecteur de MÉTA : `self.p += self.u64()` perd les 8 octets de la longueur, parce que Python lit `self.p` avant d'appeler `u64`. Corrigée et dite dans le commit `f136e27`, elle ne touchait que le contrôle de MÉTA, pas le décodage.

Ses verdicts , chiffre par chiffre (tableau 11.1).

Table 11.1 : Ce que le juge de la construction a refait, par quel chemin, et son verdict [mes.]. Les attributions barrées le sont dans le jugement lui-même.

Chiffre publié	Chemin du juge	Résultat	Verdict
tailles des `.trc` v1.2, gain 0,127 91 et 0,125 62 bit	son lecteur du format, fichiers entiers	redonnées à l'octet	tient
C1 en valeur, 768 + 4 608 experts	son décodeur, 96 experts tirés et 40 témoins v1.0	96/96, 40/40 ; trois cassures tombent	tient
aucune table v1.2 avec une fente pour un 8	son lecteur des tables	0 sur 540 (540 sur 540 en v1.0)	tient
logits au bit, 20b et 120b	dix fenêtres *neuves*, son script, une cassure neuve	0/5 120 et 0/5 120 ; cassure 512/512	tient
J1 : 2,89 contre 1,99 jetons/s, $r = 1{,}454$	séance à trois fichiers	2,889 contre 2,004, $r = 1{,}442$	tient ; attribution corrigée (R1)
KL de R1 ; B2 $-0{,}020\,4 \pm 0{,}009\,7$	sa KL sur les fichiers du lot B	redonnées	tient
B2 : $\lambda = 0$ meilleure ($2{,}09\,\sigma$)	fenêtres neuves, moteur	$-0{,}030\,3 \pm 0{,}008\,4$, $3{,}6\,\sigma$	tient, renforcé
$\lambda = 0$ meilleure que la frontière	fenêtres neuves, référence substituée	$-0{,}156\,5 \pm 0{,}033\,4$, $4{,}7\,\sigma$	tient
S3-K : moteur rapide = référence substituée	fenêtres neuves	0,003 7 ; 97,8 %	tient
« la queue baisse aussi », $\lambda = 0$ contre $\lambda = 1$	fenêtres neuves	99e −5 % ; $99{,}9^{e}$ +6 %	barré (M2)
« la queue baisse aussi », contre la frontière	fenêtres neuves, référence substituée	99e −26 %, $99{,}9^{e}$ −18 %	tient
20b ×1,24, gain du mode rapide	séance à trois fichiers	$r(R/A) = 1{,}220$; $r(R/B) = 0{,}942$	barré (G2)
120b ×2,00, gain du mode rapide	séance à trois fichiers	$r(R/A) = 1{,}851$; $r(R/B) = 1{,}284$	barré (G1)
cause du ×1,24 : le cache de fichiers	lecture de `trc-es`, étage disque	lectures sans tampon ; disque R/A 1,39	barré (G2)

Données : `genome/JUGEMENT-construction.md`, §§ 1 et 8.

11.2 Le .trc v1.2 par le décodeur du juge

11.2.1 Le décodage rANS vectorisé est le décodage normatif

Un flux rANS du format est découpé en tuiles indépendantes de 32 voies [119, 120]. Le décodage normatif (spécification § 4.9.2) parcourt les macro-pas m d'une tuile, puis ses sous-pas k, puis ses voies $l = 0, \dots, 31$ *dans l'ordre*. En Python, une matrice de codes de 2 880 × 2 880 poids demanderait 4 147 200 pas séquentiels ; le juge les fait en 1 024 itérations (256 macro-pas × 4 sous-pas), sur un tableau de $T \times 32$ états à la fois. Il faut alors prouver que cette forme est le décodage normatif.

La notation. Une tuile a 32 voies d'états $x_l \in [L, 2^{32})$ et un pointeur p dans le tableau partagé `mots`. Un pas de la voie l calcule, avec la table c du contexte de son mot : fente $= x_l \bmod M$; $s = \mathrm{sym}_c(\text{fente})$; $y_l = f_c(s)\,\lfloor x_l / M \rfloor + \text{fente} - \mathrm{cum}_c(s)$; si $y_l < L$, $x_l \leftarrow 2^{16} y_l + \texttt{mots}[p]$ et $p \leftarrow p + 1$, sinon $x_l \leftarrow y_l$. Le format fixe $M = 2^{12}$ et $L = 2^{16}$ (Tome II, chapitre du format).

Lemme 11.1 (J1 : les voies sont indépendantes à un sous-pas)**.** *Au sous-pas (m, k), y_l ne dépend que de x_l et de la table du contexte du mot $32m + l$.*

Démonstration. Les seules entrées du calcul de y_l sont x_l, la fente (fonction de x_l) et la table ; aucune autre voie n'y entre, et le pointeur p n'intervient qu'après, dans la renormalisation. ■

Proposition 11.2 (J2 : l'attribution des mots est une somme préfixe)**.** *Soit $r_l = [y_l < L]$ et $R_l = \sum_{l' \le l} r_{l'}$. Le traitement séquentiel $l = 0, \dots, 31$ donne à toute voie l telle que $r_l = 1$ le mot $\texttt{mots}[p_0 + R_l - 1]$, où p_0 est le pointeur au début du sous-pas, et laisse le pointeur à $p_0 + R_{31}$.*

Démonstration. Par récurrence sur l. Avant la voie l, le pointeur vaut $p_0 + R_{l-1}$ (avec $R_{-1} = 0$) : chacune des voies précédentes qui a renouvelé l'a avancé d'une unité. Si $r_l = 1$, la voie lit $\texttt{mots}[p_0 + R_{l-1}] = \texttt{mots}[p_0 + R_l - 1]$ et porte le pointeur à $p_0 + R_l$; sinon il reste à $p_0 + R_{l-1} = p_0 + R_l$. Au terme, il vaut $p_0 + R_{31}$. ■

Remarque 11.3 (Un seul mot par voie et par sous-pas)*.* La proposition suppose qu'une voie qui renouvelle ne lit qu'un mot. C'est vrai : avant le pas, $x_l \ge L = 2^{16}$, donc $y_l \ge \lfloor x_l / 2^{12} \rfloor \ge 2^4$ puisque $f_c(s) \ge 1$ et $\text{fente} - \mathrm{cum}_c(s) \ge 0$; si $y_l < L$, le nouvel état $2^{16} y_l + w \ge 2^{20} \ge L$. Le Tome II pose le même argument dans le lemme du décodeur vectorisé de l'épreuve J1.

Par le lemme J1, tous les y_l d'un sous-pas se calculent avant toute écriture ; par la proposition J2, les nouveaux états aussi, par la somme préfixe `cumsum(renouvelle)` sur l'axe des voies. Les tuiles étant indépendantes (chacune part de son début et de ses 32 états), le décodeur traite les $T \times 32$ voies de toutes les tuiles d'un flux à chaque sous-pas. La fin de tuile (toute voie revenue sur L) se contrôle à la fin. Une matrice de codes se décode ainsi en 0,5 s environ, un expert entier en 1,8 s à 2,1 s, contrôles compris [mes.].

11.2.2 J-T0 : les quatre fichiers entiers

Le gain sur les parties alignées vaut 0,127 91 bit par poids d'expert sur le 20b et 0,125 62 sur le 120b ; le seuil A-C2 du lot A ($\ge 0{,}12$, repris tel quel) tient sur les deux [mes.]. Le calcul, pour le 20b : $8 \times (9\,226\,493\,952 - 8\,920\,932\,352) = 8 \times 305\,561\,600$ bits, divisés par les $24 \times 32 \times 3 \times 2\,880^2 = 19\,110\,297\,600$ poids d'experts, font 0,127 91 [calc.]. Les huit tailles du lot A sont redonnées à l'octet, et ses 540 tables aussi (540 sur 540 en v1.0, 0 en v1.2). L'empreinte de la table des RÉSIDENTS se trouve à la fin exacte de la zone de données, sans bourrage : la description (« après la zone de données ») se lit ainsi, et le fichier y obéit.

Table 11.2 : J-T0 : les quatre fichiers `.trc`, lus en entier par le lecteur du juge [mes.]. Fautes de structure : en-tête, empreintes de l'en-tête, de MÉTA, de la table des RÉSIDENTS, de TABLES et du RÉPERTOIRE, entrées, taille déclarée égale à la taille sur le disque.

	20b v1.0	20b v1.2	120b v1.0	120b v1.2
fautes de structure	0	0	0	0
fichier (octets)	11 157 405 696	10 851 844 096	57 730 662 400	55 930 175 488
parties de tronçon, alignées	9 226 493 952	8 920 932 352	55 416 086 528	53 615 599 616
octets utiles des tronçons	9 224 936 032	8 919 387 648	55 406 705 152	53 606 030 768
plus grand tronçon	12 144 640	11 755 520	12 251 136	11 878 400
tables CODES_CTX avec une fente pour un 8	216 sur 216	**0 sur 216**	324 sur 324	**0 sur 324**

Données : `mesures/genome/construction/juge/J-T0-fichiers.json`.

11.2.3 J-T1 : 96 experts tirés, décodés par le juge

Le tirage est écrit et commité avant tout décodage (`tirage-experts.json`, commit `93a88a5`) : 32 experts du 20b puis 64 du 120b, sans remise, `Generator(PCG64(20261022))`. Pour chacun, dans le v1.2, huit contrôles : (a) chaque voie de chaque tuile finit sur L ; (b) les blocs reconstruits $[e, qs]$ égaux octet pour octet aux blocs du GGUF canonisés par le code du juge (demi-octet $8 \to 0$) ; (c) les biais égaux au bit ; (d) chaque poids égal *en valeur* au poids du GGUF non canonisé ; (e) l'empreinte de l'expert canonique reconstruit égale à `empreinte_canonique` ; (f) l'empreinte des blocs du GGUF tels quels égale à `empreinte_origine` ; (g) l'empreinte de la partie égale à celle du répertoire ; (h) aucun demi-octet 8.

- **96 experts sur 96 tiennent les huit contrôles** [mes.].
- **Le témoin** (réponse connue d'avance) : les 32 experts du 20b et les 8 premiers tirés du 120b, décodés dans le *v1.0*, rendent le GGUF tel quel (40 sur 40) ; et pour chacun, les demi-octets qui changent entre le décodé v1.0 et le décodé v1.2 sont exactement les -0 du GGUF (L19 E109 du 120b : 1 746 601 changés, 1 746 601 -0) [mes.].
- **Les cassures**, toutes tombent : un mot du flux CODES_CTX changé (voies hors de L, octets, valeurs et empreinte canonique faux) ; la canonisation retirée du lecteur du GGUF (octets et empreinte canonique du côté GGUF faux ; l'empreinte du décodé, qui ne lit pas le GGUF, tient : le juge l'avait annoncée tombante, il le dit) ; un bit du premier biais de la matrice 2 (biais, empreinte de la partie et empreinte canonique faux) [mes.].
- La part des -0 dans l'échantillon, $6{,}42 \pm 0{,}11$ % (20b) et $6{,}33 \pm 0{,}10$ % (120b), contre 6,393 % et 6,279 % sur les fichiers : à 0,3 et 0,5 écart-type.

La figure de ces décodages (figure 9.7) est au chapitre du codon, avec qui elle forme le verdict du lot A.

11.2.4 Ce que le témoin v1.0 → v1.2 prouve

Soit $D_0(x)$ et $D_2(x)$ les demi-octets décodés de l'expert x dans le v1.0 et le v1.2, $G(x)$ ceux du GGUF, et $\kappa(g) = g$ si $g \neq 8$, 0 sinon (le code 8 est le -0 d'E2M1). Le témoin mesure $D_0(x) = G(x)$, et le contrôle (b) mesure $D_2(x) = \kappa(G(x))$. Alors

$$\{i \,:\, D_0(x)_i \neq D_2(x)_i\} = \{i \,:\, G(x)_i \neq \kappa(G(x)_i)\} = \{i \,:\, G(x)_i = 8\} \,:$$

l'ensemble des demi-octets changés est *exactement* celui des -0, ce que le journal vérifie expert par expert. C'est la forme mesurée de la phrase « la v1.2 ne change que le signe des zéros », et, avec (d), de « la v1.2 est sans perte en valeur ».

Verdict J-T : tient. Le `.trc` v1.2 est exactement ce que le format décrit, et chaque expert tiré est l'expert du GGUF au bit près, au signe des zéros près.

11.3 Les fenêtres neuves, et les experts qu'elles appellent

Le tirage. Les lots évaluaient sur R1, les dix premières fenêtres de 512 jetons de wiki.test (jetons 0 à 5 119) [125], et calibraient le génome synonyme sur les fenêtres 10 à 21 (jetons 5 120 à 11 263). Le juge écarte les 120 000 premiers octets de `wiki.test.raw` (SHA-256 `173c87a5…dd08`), qui comptent 27 680 jetons : ils couvrent R1, la calibration, et deux fois autant de marge [mes.]. Il tire au-delà dix débuts de ligne de paragraphe (graine 20 261 022 + 1), à plus de 12 000 octets les uns des autres : aux octets 767 283, 511 976, 615 155, 854 814, 1 148 495, 1 228 294, 1 098 980, 1 240 966, 421 030 et 358 205. Chaque extrait (9,0 à 10,4 Ko) passe par `llama-perplexity` (processeur seul, options de la base, une fenêtre), et les dix fichiers sont fondus en un seul `base-juge-20b.kld`. Sa taille se vérifie au calcul : $20 + 4 \times 5\,120 + 2\,550 \times 402\,184 = 1\,025\,589\,700$ octets, la taille de R1 ; chaque fenêtre est égale à sa source octet pour octet, et une fenêtre décalée ne l'égale pas (cassure) [mes.] [calc.]. Le format de `llama-perplexity` se fond par simple concaténation des jetons puis des entrées : tous les outils du projet lisent le fichier fondu sans changement.

La référence. La référence f64 de ces fenêtres (`juge reference --yarn openai`, l'appel de R1) a pris 2 168 s et routé 755 experts distincts (R1 : 734). KL(llama.cpp processeur ‖ référence f64) y vaut 0,033 9 (R1 : 0,025 5) : des fenêtres un peu plus difficiles, rien d'anormal [mes.].

Les experts vus. Figure 11.1 dresse la carte : chaque case de la grille 24 × 32 du 20b, colorée selon les fenêtres qui lui ont routé des jetons sous la référence exacte. 732 experts sont routés par les deux jeux, 2 par R1 seul, 23 par les fenêtres neuves seules (465 appels sur 490 560), et 11 par aucun ; $732 + 2 + 23 + 11 = 768$ [mes.] [calc.]. Le lot B en comptait de 36 à 39 non routés sur R1 sous substitution ; les fenêtres neuves en jugent une partie (la passe f64 substituée $\lambda = 0$ y charge 756 experts).

Ce que la carte dit, et ce qu'elle ne dit pas. Les 11 cases barrées ne sont pas des experts « non vérifiés » : leur décodage l'est en valeur, comme celui des 757 autres (critère C1 du lot A, 768 experts sur 768, chapitre 9). Il leur manque une KL, pas une mesure : aucune mesure de KL de la construction ne les a appelés, sous le routage de la référence. Un changement confiné à ces 11 experts ne bougerait aucune KL des chapitres 9 à 11 mesurée avec ce routage ; mais une variante dont le routage bascule pourrait les appeler. La première légende de la figure disait plus que cela ; le second juge des figures l'a fait réécrire.

11.4 J-C1 : les logits au bit sur les fenêtres neuves

`troncons evaluer --flux --empreintes`, dix fenêtres neuves, moteur construit par le juge ; comparaison par son script, pas par celui du lot A. Seuil repris du lot A (proposition A1, critère A-C1) : 0 empreinte de logits différente sur 5 120 pas par modèle, et les `.kld` de sortie égaux octet pour octet (SHA-256).

Verdict J-C1 : tient , sur deux modèles et sur un texte que personne n'avait vu. La proposition A1 du lot A (chapitre 9) est vraie du moteur réel. La cassure a été choisie pour toucher chaque position (chaque position route quatre experts de la couche 0 parmi 32, et tous ont leur biais retourné) ; la copie cassée est effacée après.

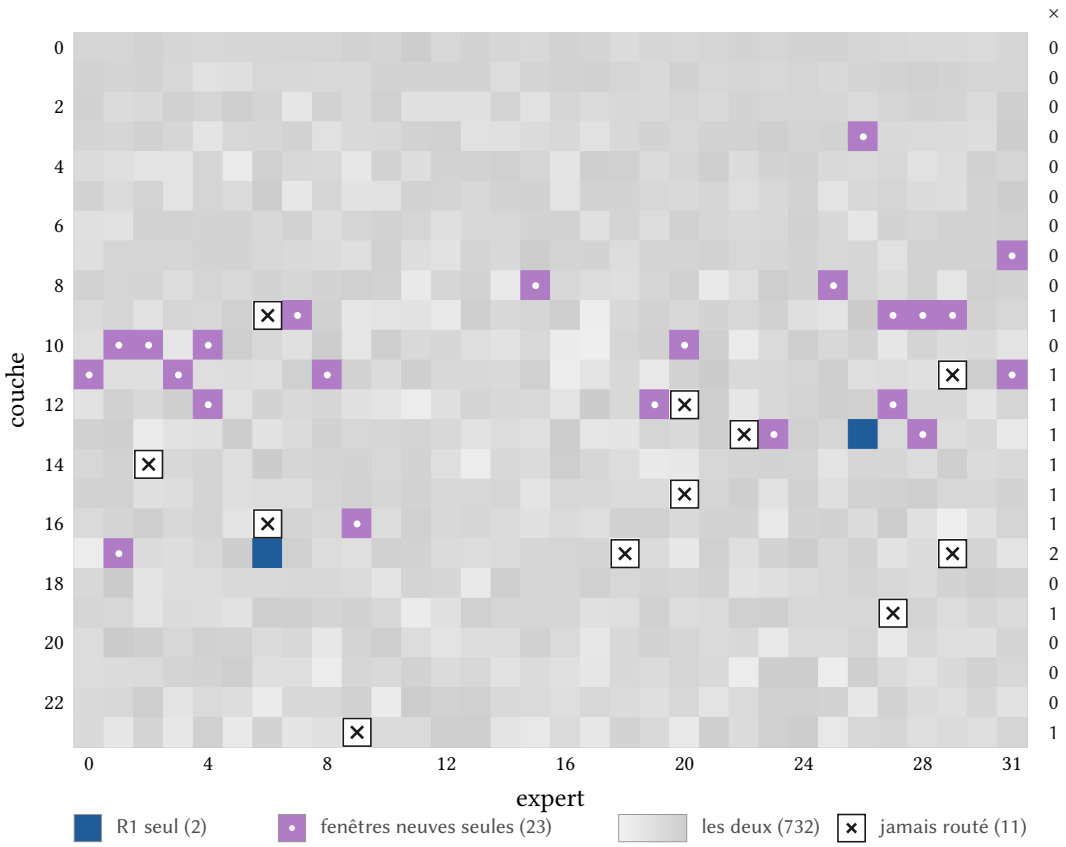


Figure 11.1 : 11 experts de gpt-oss-20b n'ont jamais été routés, sur aucun des deux jeux de fenêtres : aucune mesure de KL de la construction ne les a appelés (leur décodage, lui, est vérifié en valeur : C1, 768 experts sur 768). [mes.] Les 24 × 32 experts de gpt-oss-20b, chaque case colorée selon les fenêtres qui lui ont routé des jetons : les dix fenêtres de R1 seules (bleu), les dix fenêtres neuves du juge seules (violet, point blanc), les deux (gris, plus foncé avec le nombre d'appels, en échelle logarithmique), ou aucune (encadrée et barrée ; leur compte par couche en marge). Un appel est un jeton routé vers un expert ; une fenêtre route ses 511 jetons, dont 256 de contexte non notés (490 560 appels en tout sur chaque jeu de fenêtres ; au plus 4 764 pour un expert sur un jeu, 9 331 sur les deux réunis). C'est la carte de l'espace que le routeur parcourt sur du texte réel, sous le routage de la référence : un changement confiné aux 11 experts barrés ne bougerait aucune KL des chapitres 9 à 11 mesurée avec ce routage, mais une variante dont le routage bascule pourrait les appeler.

Données : `mesures/genome/construction/juge/figures/routage-grille.csv`, `J-T0-fichiers.json` (*k*) ; `mesures/genome/construction/synonyme/kl/kl-frontiere-768-positions.csv` (positions notées) ; `mesures/genome/construction/codon/tailles.json` (C1) ; script `memoire/outils/tome3_juge.py`.

Table 11.3 : J-C1 : le v1.2 contre le v1.0, logits sur dix fenêtres neuves [mes.].

	20b	120b
v1.2 contre v1.0 : empreintes de logits différentes	**0 sur** 5 120	**0 sur** 5 120
`.kld` de sortie	égaux (`1aff1a6a…29ef`)	égaux (`709367e7…8a73`)
témoin : le v1.2 évalué deux fois	CSV identiques (`fc427e9f…469b`)	—
cassure : le signe du biais de la rangée 0 de gate retourné pour les 32 experts de la couche 0, `--sans-garde`, fenêtre 0	**512 sur 512 différentes**, dès la position 0 ; 73 premiers choix changés	—

Données : `mesures/genome/construction/juge/logits/comparer-*.json`, `logits/evaluer-*.\{csv,json,log\}`.

11.5 J-K : la KL du génome synonyme hors de R1

11.5.1 La KL du juge, et son témoin

Par position notée, le juge calcule

$$\mathrm{KL}(P\|Q) = \sum_{i\,:\,\log p_i > -16} p_i\,(\log p_i - \log q_i),$$

P et Q décodés de leur codage sur 16 bits, sans renormalisation : c'est la convention du lot R (Tome II, la référence et le juge), où un fichier contre lui-même rend 0 [11]. **Témoin** : sur une paire publiée par le lot B (R1, $\lambda = 0$), sa KL vaut 0,131 752 831 contre 0,131 752 832 pour `juge kl` ($1{,}1 \cdot 10^{-8}$ relatif, seuil 10^{-6}) ; **cassure** : Q décalé d'une position, 1,634 [mes.]. R1 recalculé par son code depuis les fichiers du lot B : frontière 0,263 99, $\lambda = 1$ 0,152 16, $\lambda = 0$ 0,131 75 ; appariées $-0{,}111\,83 \pm 0{,}029\,62$, $-0{,}020\,40 \pm 0{,}009\,74$ ($\bar{d} + 2\sigma = -0{,}000\,92$), $-0{,}132\,24 \pm 0{,}023\,43$: les chiffres du lot B sont redonnés à quatre chiffres et plus [mes.].

Sur les fenêtres neuves, deux chemins : le *moteur* (`troncons evaluer --flux --rapide` sur les compagnons, le produit tel qu'il est servi) et la *référence f64 substituée* (`genome-capture kl --sub`, le chemin des lots). La vérité est la référence f64 des fenêtres neuves.

11.5.2 Les nombres

Table 11.4 : J-K : la KL contre la référence f64 sur dix fenêtres neuves (2 550 positions), les 768 experts substitués [mes.]. Perplexité de la référence : 392,5.

Fenêtres neuves	KL	σ naïf	σ fenêtres	médiane	99,9^{e} c.	premier choix	perplexité
moteur exact, v1.2	0,004 37	0,000 8	0,002 1	0,000 03	0,557	98,0 %	394,0
$\lambda = 0$, moteur rapide	0,144 8	0,006 0	0,016 4	0,060 8	3,61	71,7 %	402,1
$\lambda = 0$, f64 substituée	0,144 6	0,005 9	0,016 5	0,061 7	3,51	71,3 %	399,5
$\lambda = 1$, moteur rapide	0,175 1	0,006 2	0,018 6	0,075 5	3,40	67,6 %	453,7
frontière, f64 substituée	0,301 1	0,007 4	0,041 5	0,191 7	4,30	51,5 %	239,8

Données : `mesures/genome/construction/juge/kl/kl-neuf-*.json` ; $\lambda = 1$ par la référence substituée : non passé (section 11.11).

Les verdicts déclarés.

- **J-K2, B2 refait hors de R1** (la règle du lot B, reprise telle quelle) : par le moteur, $\bar{d} = \mathrm{KL}_{\lambda=0} - \mathrm{KL}_{\lambda=1} = -0{,}030\,3$, σ par fenêtres 0,008 4, $\bar{d} + 2\sigma = -0{,}013\,6 < 0$ (3,62 σ, contre 2,09 sur R1) ; neuf fenêtres sur dix (la fenêtre 0 contre, +0,016) ; $\lambda = 0$ a moins de KL sur 1 858 positions sur 2 550 ; premier choix +4,1 ± 1,1 points. La prédiction du juge (« égales ») est démentie.
- **J-K1, $\lambda = 0$ contre la frontière**, par la référence substituée : $-0{,}156\,5 \pm 0{,}033\,4$ (4,7 σ), les dix fenêtres, 2 297 positions sur 2 550, premier choix +19,8 ± 3,2 points ; rapport $\lambda = 0$ sur frontière 0,480 (R1 : 0,499). Tient, comme prédit.
- **J-K3, le moteur rapide contre la f64 substituée** (seuils S3-K du lot B) : 0,003 7 ≤ 0,011 et 97,8 % ≥ 95 %, tient ; les deux chemins rendent la même KL contre la vérité, 0,144 8 et 0,144 6, différence appariée $+0{,}000\,2 \pm 0{,}000\,6$.
- **J-K4, le mode exact** : KL 0,004 37, premier choix 98,0 % (R1 : 0,003 68 et 97,2 %) ; llama.cpp sur le processeur y fait 0,033 9. La forme de C4 tient contre llama.cpp processeur ; contre llama.cpp sur la carte, le comparant exact de C4, non mesuré ici.

Un seul chemin pour $\lambda = 1$. La référence substituée $\lambda = 1$ n'a pas été passée sur les fenêtres neuves (le temps : de 47 à 52 min par passe sous la charge de la machine, et le flux du placement disque attendait le calme pour sa séance). B2 y est donc jugé par le moteur seul. Le juge le justifie par deux accords mesurés : les deux chemins rendent la même KL pour $\lambda = 0$ à 0,000 2 près sur ces fenêtres, et pour $\lambda = 1$ sur R1 (0,152 2 par les deux, lot B). Mélanger les chemins dans une différence appariée ($\lambda = 1$ moteur contre frontière f64 : $-0{,}126\,0 \pm 0{,}036\,6$) n'est publié qu'à titre indicatif ; c'est le panneau aux losanges creux de figure 10.4.

Ce que les fenêtres neuves disent de plus. *Le niveau* : $\lambda = 0$ fait 0,144 8 sur les fenêtres neuves et 0,131 8 sur R1 ; le moteur exact 0,004 4 et 0,003 7 ; la référence llama.cpp 0,033 9 et 0,025 5. Ces fenêtres sont plus difficiles pour tout le monde, de 10 à 33 % ; le génome synonyme ne s'y dégrade pas plus que le reste [calc.]. *La queue* et *la perplexité* : deux phrases du lot B tombent, barrées au chapitre 10 (constats M2 et M3). *Les experts* : les fenêtres neuves font jouer 23 experts que R1 n'avait jamais routés ; leur part des appels est petite (0,1 %), et la KL de $\lambda = 0$ les inclut.

11.5.3 L'incertitude d'une KL moyenne : par positions ou par fenêtres

Les 255 positions notées d'une fenêtre partagent leur passé, donc leurs bascules de routage : elles ne sont pas indépendantes. Deux écarts-types de la moyenne s'opposent :

$$\sigma_f = \frac{s(\bar{K}_1, \dots, \bar{K}_{10})}{\sqrt{10}}, \qquad \sigma_n = \frac{s(K_1, \dots, K_{2\,550})}{\sqrt{2\,550}},$$

le premier sur les dix moyennes de fenêtre, le second sur les positions comme si elles étaient indépendantes. Leur rapport définit l'*effet de plan* et le nombre de positions efficaces :

$$\mathrm{deff} = \left(\frac{\sigma_f}{\sigma_n}\right)^2, \qquad n_{\mathrm{eff}} = \frac{n}{\mathrm{deff}}.$$

Proposition 11.4 (L'effet de plan d'un échantillon en grappes)**.** *Soit n grappes (les fenêtres) de m positions, chaque position de variance s^2, deux positions d'une même grappe de corrélation ρ, et deux positions de grappes différentes indépendantes. Alors la variance de la moyenne générale vaut $\frac{s^2}{nm}\big(1 + (m-1)\rho\big)$, et* $\mathrm{deff} = 1 + (m-1)\rho$.

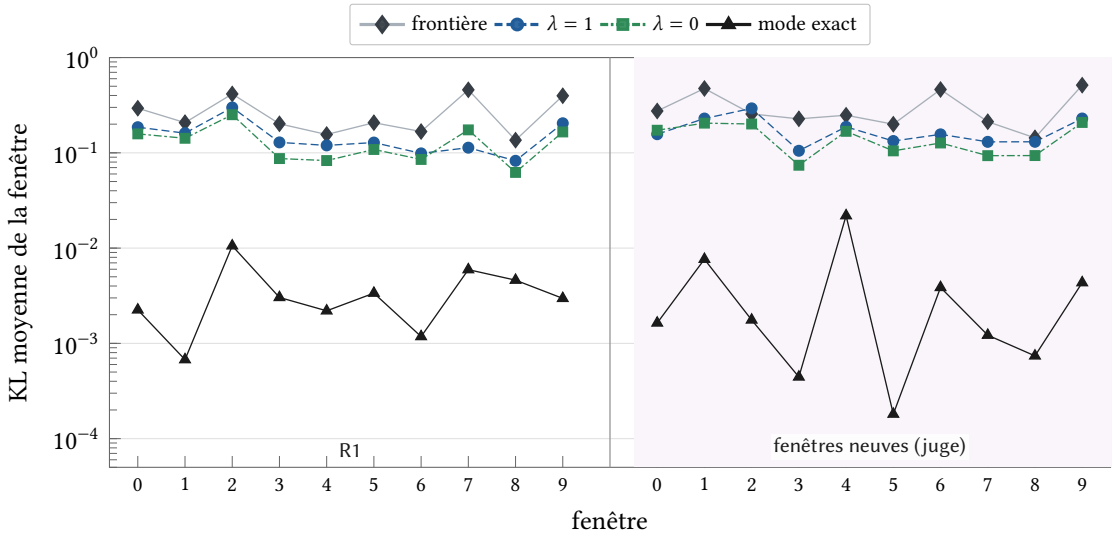


Figure 11.2 : Certaines fenêtres sont plus difficiles que d'autres pour toutes les variantes à la fois ; le mode exact reste loin au-dessous de toutes. [mes.] KL moyenne de chaque fenêtre (255 positions notées) pour la frontière, les deux synonymes et le mode exact, sur les dix fenêtres de R1 et les dix fenêtres neuves du juge (fond teinté). Les traits fins joignent des fenêtres qui ne se suivent pas : ils ne montrent que le parallélisme des variantes. Chemins : sur R1, les variantes par la référence f64 substituée et le mode exact depuis le fichier par position du lot O ; sur les fenêtres neuves, $\lambda = 0$, $\lambda = 1$ et le mode exact par le moteur, la frontière par la référence substituée. Un seul chemin est tracé par série et par jeu. Sur R1, les trois variantes sont aussi passées par le moteur : moteur moins référence substituée, −0,0014 en moyenne pour la frontière, 0,0001 pour $\lambda = 1$, −0,0004 pour $\lambda = 0$, de −0,0076 à 0,0029 par fenêtre sur les trois ; sur les fenêtres neuves, $\lambda = 0$ est passé par les deux, 0,0002 en moyenne. La frontière va de 0,136 à 0,459 sur R1 : cet écart d'une fenêtre à l'autre, commun à toutes les variantes, rend l'écart-type par fenêtres ($\sigma_f = 0{,}0376$ pour la frontière) bien plus grand que l'écart-type naïf par positions, et l'appariement le retire (figure 10.4). Les fenêtres neuves sont tirées de wiki.test, le corpus de R1.

Données : `mesures/genome/construction/juge/figures/kl-par-fenetre.csv` ; `mesures/moteur/O/journaux/o3/c4-20b-comparer.csv` ; `mesures/genome/construction/synonyme/moteur/kl-moteur-frontiere-positions.csv` ; script `memoire/outils/tome3_juge.py`.

Démonstration. La moyenne d'une grappe, $\bar{K}_g = \frac{1}{m}\sum_i K_{gi}$, a pour variance $\frac{1}{m^2}\big(ms^2 + m(m-1)\rho s^2\big) = \frac{s^2}{m}\big(1 + (m-1)\rho\big)$: m variances et $m(m-1)$ covariances. Les grappes étant indépendantes, la moyenne des n moyennes a n fois moins de variance. La variance naïve, qui ignore les covariances, vaut $s^2/(nm)$; le quotient est $1 + (m-1)\rho$. ■

Sur R1, frontière. $\sigma_f = 0{,}037\,6$, $\sigma_n = 0{,}006\,6$: deff ≈ 32 et $n_{\text{eff}} \approx 79$ [calc.]. Une fenêtre de 255 positions vaut donc environ huit positions indépendantes (255/32), et par la proposition, la corrélation moyenne de deux positions d'une même fenêtre est $\rho \approx (32-1)/254 \approx 0{,}12$ [calc.]. Figure 11.2 montre d'où elle vient : certaines fenêtres sont plus difficiles que d'autres pour toutes les variantes à la fois (la frontière va de 0,136 à 0,459 d'une fenêtre de R1 à l'autre), et le mode exact reste loin au-dessous de toutes (de 0,000 2 à 0,021 9).

Pour une différence appariée , l'effet de plan est bien plus petit, parce que l'appariement retire la difficulté propre à chaque fenêtre, commune aux deux variantes : pour $\lambda = 0$ contre $\lambda = 1$ sur R1,

$\sigma_f = 0{,}009\,74$ et $\sigma_n = 0{,}003\,85$, deff $\approx 6{,}4$ [calc.] (`mesures/genome/construction/synonyme/kl/B2-synonyme-11-contre-10-768.json`). C'est pourquoi les lots ont raison de juger par des différences appariées, et pourquoi une KL absolue publiée avec σ_n paraît de trois à six fois plus précise qu'elle n'est (constat M1). La recette du juge : publier une KL absolue avec son σ par fenêtres.

11.5.4 Refaire un résultat à deux écarts-types

Pourquoi le juge prédisait-il « égales » ? Soit une différence appariée mesurée à $z = \bar{d}/\sigma$ écarts-types, et supposons que la vraie différence vaille exactement $\bar{d}$, et que la seconde série de dix fenêtres ait le même σ. L'estimateur neuf suit à peu près $\mathcal{N}(\bar{d}, \sigma^2)$; avec $\bar{d} = -z\sigma$, la probabilité qu'il passe à nouveau le seuil $\bar{d}' + 2\sigma < 0$ vaut

$$P\big(\bar{d}' < -2\sigma\big) = \Phi\Big(\frac{-2\sigma + z\sigma}{\sigma}\Big) = \Phi(z - 2).$$

Pour $z = 2{,}09$, $\Phi(0{,}09) \approx 0{,}54$ [calc.] : à peine plus d'une chance sur deux. Et si la vraie différence est plus petite que $\bar{d}$, ce qui est probable pour un résultat retenu *parce qu'il* passait le seuil, la probabilité baisse. La prédiction suivait ce calcul ; la mesure l'a démentie, $z = 3{,}62$ sur les fenêtres neuves. Le juge n'a pas changé sa prédiction après coup : elle était déclarée, et la mesure a décidé.

11.6 J-V : la vitesse, et ce qu'elle sépare

11.6.1 Le piège

Les deux lots ont chacun comparé un fichier neuf à un fichier ancien : le lot A le v1.2, écrit le 17 octobre, au v1.0 d'origine, écrit le 15 ; le lot B ses compagnons, écrits le 17 et le 22, au même v1.0 d'origine. Le lot A a vu le piège et l'a dit : deux v1.0 identiques au bit se lisaient à 1,74 et 1,34 Go/s. Le lot B ne l'a pas vu. Si deux fichiers du même modèle se lisent à des vitesses différentes pour une raison qui n'est pas leur mécanisme, un rapport de vitesses entre eux ne mesure pas le mécanisme.

11.6.2 La séance à trois fichiers

Le juge met les trois fichiers de chaque modèle dans une seule séance : A le v1.0 d'origine, B le v1.2 (même écrivain que les compagnons, écrit comme eux après le 15), R le compagnon rapide, dans l'ordre

A B R R B A A B R R B A.

Proposition 11.5 (Le plan équilibre une dérive linéaire)**.** *Si la vitesse d'une passe de rang i du fichier X vaut $v_X\,(1 + a + b\,i)$, le rapport des moyennes des quatre passes de deux fichiers vaut v_X/v_Y, sans terme de dérive. Une dérive quadratique, elle, n'est pas équilibrée.*

Démonstration. Les rangs sont A $\{1, 6, 7, 12\}$, B $\{2, 5, 8, 11\}$, R $\{3, 4, 9, 10\}$; leurs sommes valent toutes 26, leurs moyennes $\bar{\imath} = 6{,}5$. La moyenne des quatre passes de X vaut $v_X\,(1 + a + b\,\bar{\imath}) = v_X\,(1 + a + 6{,}5\,b)$, le même facteur pour les trois fichiers, qui disparaît du rapport. Les sommes des carrés des rangs valent $1 + 36 + 49 + 144 = 230$, $4 + 25 + 64 + 121 = 214$ et $9 + 16 + 81 + 100 = 206$: un terme en $c\,i^2$ laisserait un biais. ∎

La règle lit des *médianes* de quatre passes, c'est-à-dire la moyenne des deux du milieu. Sous une dérive monotone qui domine le bruit, les passes du milieu sont celles des rangs 6 et 7 pour A, 5 et 8 pour B, 4 et 9 pour R, dont la moyenne vaut encore 6,5 : la médiane est équilibrée dans ce cas aussi [calc.].

La décomposition. Avec trois fichiers d'une même séance,

$$r(R/A) = r(R/B) \cdot r(B/A).$$

$r(B/A)$ mesure ce que le lot A a mesuré, $r(R/A)$ ce que le lot B a mesuré, et $r(R/B)$ ce que vaut le mode rapide contre un fichier exact que l'étage disque lit à la même vitesse. Le toit qu'un format plus petit peut rendre, disque limitant, est le rapport des octets lus par pas ($o_A/o_B = 1/0{,}967\,1 = 1{,}034$ pour la v1.2) ; tout $r(B/A)$ au-delà vient d'ailleurs que des octets. L'étage disque seul (`troncons disque --n 1500`, l'instrument du lot A) passe d'abord, A B R R B A, et dit lequel des facteurs n'est pas le mécanisme.

Le seuil. Avec $\sigma = \max(\sigma_A, \sigma_B, \sigma_R)$, écarts-types relatifs des quatre passes, un gain publié g est un gain du mode rapide si $r(R/B) \geq g\,(1 - 2\sigma)$; $(1 - 2\sigma)$ est la borne basse d'un rapport de médianes à deux écarts-types de son bruit. Les conditions : secteur à chaque passe ; relevé des autres flux avant et après chaque passe (plus de 5 % d'un cœur ou plus de 8 Mo lus en 3 s : « sous charge », la passe ne conclut pas) ; départ sur deux relevés vides à une minute d'écart ; verrou tenu. **Aucune des 36 passes n'a été sous charge** [mes.].

11.6.3 Le 120b

Réglages du J1 pour les trois fichiers : `mesurer --flux --emplacements 326`, invite wiki-3000, 256 jetons.

Table 11.5 : J-V, le 120b : la séance à trois fichiers [mes.].

	A, v1.0 d'origine	B, v1.2	R, rapide frontière
génération, jetons/s	1,995 · 1,785 · 2,013 · 2,019	2,892 · 2,886 · 2,900 · 2,868	3,712 · 3,709 · 3,698 · 3,719
médiane ; σ relatif	2,004 ; 5,8 %	2,889 ; 0,5 %	3,711 ; 0,2 %
octets de tronçons par pas	710,1 Mo	686,8 Mo (× 0,967 1)	468,5 Mo
débit par octet en génération	1,42 Go/s	1,98 Go/s	1,74 Go/s
succès en VRAM	0,590	0,590	0,607
texte (256 jetons)	`e4c6dbac`…	`e4c6dbac`… (le même)	`2cf1d474`… (celui du lot B)
mémoire de pointe	388,7 Mo	382,3 Mo	342,4 Mo
étage disque seul (2 passes)	1,772 · 1,744 Go/s	2,394 · 2,422	2,324 · 2,279

Données : `mesures/genome/construction/juge/vitesse/seance-120b.\{json,log\}`, `vitesse/analyse-120b.json`, `figures/vitesse-passes.csv`, `figures/disque-passes.csv`.

- **J-V1 (A-C3 du lot A, repris)** : $r(B/A) = 1{,}442$, $\sigma = 5{,}8\,\%$: pas de régression, gain annoncé ; le lot A publiait 1,454. Le J1 se reproduit ; les octets ne baissent que de 3,3 % (toit +3,4 %), et l'étage disque lit B 1,37 fois plus vite que A.
- **J-V2** : $r(R/B) = 1{,}284$; seuil $2{,}00 \times (1 - 2\sigma) = 1{,}769$, avec $\sigma = 0{,}057\,64$ (arrondi à 5,8 % dans le jugement ; `mesures/genome/construction/juge/vitesse/analyse-120b.json`) : **le ×2,00 n'est pas un gain du mode rapide.** Contre un fichier exact lu à la même vitesse par l'étage disque, le mode rapide frontière rend ×1,28 en génération, pour un toit des octets de 686,8/468,5 = ×1,47 [calc.]. La prédiction du juge (×1,38) était trop haute : le débit par octet du compagnon (1,74 Go/s) reste sous celui du v1.2 (1,98), des lectures plus courtes (8,3 Mo au lieu de 11,9 par tronçon) [hyp. pour la cause].

– **D'où vient le reste du** ×2,00 : $r(R/A) = 1{,}851$ ici (le lot B mesurait 2,00 avec d'autres réglages). Le lot B décomposait son rapport en ×1,524 d'octets et un ×1,31 de débit par octet « que les compteurs ne séparent pas » (proposition 10.5). L'étage disque seul rend R/A = 1,309 : ce ×1,31 est ~~l'état du fichier A~~ le motif de lecture v1.0 du fichier A (corrigé le 27 octobre, réserve R1), pas le mode.

11.6.4 Le 20b

Réglages de la séance s2 du lot B : `mesurer`, emplacements par défaut, 256 jetons.

Table 11.6 : J-V, le 20b : la séance à trois fichiers [mes.].

	A, v1.0 d'origine	B, v1.2	R, rapide $\lambda = 0$
génération, médiane ; σ	9,395 ; 0,4 %	12,175 ; 0,2 %	11,466 ; 0,6 %
préremplissage (708 jetons)	10,9 jetons/s	13,7 jetons/s	16,9 jetons/s
octets de tronçons par pas	116,4 Mo	112,5 Mo	115,1 Mo
succès en VRAM	0,899	0,899	0,856
débit par octet en génération	1,09 Go/s	1,37 Go/s	1,32 Go/s
texte	`1b14e44a`…	`1b14e44a`… (le même)	`18be41e4`… (celui du lot B)
étage disque seul	1,786 · 1,775 Go/s	2,461 · 2,443	2,475 · 2,472

Données : `mesures/genome/construction/juge/vitesse/seance-20b.\{json,log\}`, `vitesse/analyse-20b.json`.

– $r(R/A) = 1{,}220$: la mesure du lot B (1,241) se reproduit à 1,7 % près.
– **J-V3** : $r(R/B) = 0{,}942$; seuil $1{,}241 \times (1 - 2 \times 0{,}0056) = 1{,}227$: **le** ×1,24 **n'est pas un gain du mode rapide.** Contre le v1.2, le mode rapide $\lambda = 0$ génère 6 % moins vite : il lit autant d'octets par pas (toit ×0,98), parce que son routage, avec perte, manque plus souvent la VRAM. Il ne gagne qu'au préremplissage (×1,23), où le texte est imposé.
– **La cause avancée par le lot B est impossible** : le moteur lit chaque tronçon sans tampon (`trc-es`, `FILE_FLAG_NO_BUFFERING | FILE_FLAG_OVERLAPPED`), et le flux du placement a mesuré que le disque lit 1,000 0 à 1,005 7 fois les octets demandés. La prédiction déclarée du juge le tenait déjà pour faux « par construction » ; la séance l'a jugé. L'étage disque lit R 1,39 fois plus vite que A, B 1,38 fois.
– $r(B/A) = 1{,}296$ sur le 20b : la v1.2 va aussi 30 % plus vite que son v1.0, pour 3,4 % d'octets en moins ; même cause, l'étage disque.

11.6.5 Ce qui fait l'étage disque : la réserve R1

Le juge de la construction avait d'abord attribué l'écart de l'étage disque à l'état physique des fichiers, après une lecture du flux du placement disque : ~~c'est l'état des fichiers, pas le format (+3,4 % au plus)~~. Le juge du placement a fait tomber cette lecture le 27 octobre (`genome/JUGEMENT-placement.md`, §§ 0, 5.3, 5.6, 5.7), et le jugement de la construction porte la correction à chaque endroit, barres comprises. Sur ce disque (NVMe derrière Intel RST VMD, sous BitLocker), l'écart entre v1.0 et v1.2 suit *le motif des requêtes de lecture que le format impose* : un même fichier lu avec les frontières de l'autre format change de vitesse, et la variable exacte du motif n'est pas isolée (section 9.7.1).

Ce que cela fixe, pour tout le tome :

– **Le** ×1,44 **du J1 est un gain du format v1.2**, par le motif de ses requêtes de lecture, *propre à ce disque* ; il ne vient ni de ses 3,3 % d'octets en moins, ni de « l'état du fichier ». Il ne se transporte pas à un autre disque sans y être remesuré.

- **Les verdicts du mode rapide gardent leur sens** : B et R ont tous deux un motif rapide (environ 11,6 et 8,3 Mo par requête), A le motif lent du v1.0 (environ 12,0 Mo) ; $r(R/B)$ compare deux motifs rapides. Partout où le jugement écrivait « à fichier d'âge comparable » et « l'état du fichier A », on lit « à motif de lecture comparable » et « le motif v1.0 du fichier A ».
- **Le levier qui marche aujourd'hui** est de recoder les fichiers v1.0 en v1.2 (×1,30 à ×1,44 entre deux fichiers du même modèle). Le chapitre du codon le développe (chapitre 9) ; le placement disque est l'objet d'un chapitre du Tome II, « L'étage disque », par décision du juge du placement.

11.7 La lecture : ce qui est faux, trop beau, ou mal attribué

Le juge a lu les deux livrables phrase à phrase contre leurs journaux, et cherché, pour chaque chiffre de leurs « En bref », le fichier commité qui le porte. Deux constats sont *graves* : ils changent ce que le projet ferait ensuite. Les autres sont mineurs : ils ne renversent aucune conclusion, mais ils rendent un chiffre plus beau qu'il n'est.

G1, le « 120b ×2,00 » du mode rapide. Bien mesuré, faux comme gain du mode : le fichier exact de la comparaison est le v1.0 du 15 octobre, que l'étage disque lit 1,31 fois moins vite que le compagnon à octets égaux par lecture ; contre le v1.2, le mode rapide rend ×1,28. Le lot B avait isolé ce ×1,31 et l'avait laissé « non mesuré » ; il est mesuré. *Ce que cela change* : le cahier construisait la politique mixte des défauts sur ce gain ; ×1,28 pour une KL de 0,14 n'est pas ×2,00 pour la même KL.

G2, le « 20b ×1,24 ». Même mesure juste, même attribution fausse, et ici le mode rapide ne gagne rien en génération. Le lot B disposait des deux faits pour le voir : le moteur lisait presque les mêmes octets par pas, et la première passe B, juste après l'écriture du compagnon et une passe A, était deux fois plus lente que les suivantes (5,46 contre 11,8 jetons/s), ce qu'aucun cache de lecture n'explique. *Ce que cela change* : sur le 20b, le mode rapide est un mode de préremplissage, pas de génération.

Les mineurs. M1 : les ± des KL absolues du lot B sont naïfs, trois à six fois trop étroits (section 11.5.3) ; les verdicts appariés, eux, sont justes. M2 : « la queue baisse aussi » avec $\lambda = 0$ ne se reproduit pas contre $\lambda = 1$, et se reproduit contre la frontière. M3 : le sens de la perplexité de $\lambda = 0$ s'inverse hors de R1. M4 : la règle de vitesse du lot B ne regardait que le bruit des passes A ; avec $\sigma = \max(\sigma_A, \sigma_B)$ ($\sigma_B = 31\ \%$), le gain n'aurait pas été annoncé. M5 : « le gain ne fond pas, il grandit » comparait deux variantes ; à variante égale, la phrase reste vraie.

Ce que le juge a vérifié de plus, et qui tient. Le gain par tenseur du lot A vaut deux fois la part des −0 à −0,000 167 … 0,000 161 bit près sur les 180 tenseurs (parts 6,393 % et 6,279 %, recalculées) ; les huit passes de la séance s2 du lot A redonnent 1,988 et 2,890 jetons/s et $r = 1{,}454$; les seize passes de s2 du lot B redonnent 9,54, 11,84, 1,98 et 3,97 jetons/s et leurs octets par pas ; la correction barrée de la synthèse (la KL 0,026 2 est celle de $\lambda = 0$) est juste ; aucune ligne d'attribution ni adresse électronique dans les documents des deux lots.

11.8 Ce qui tient selon le juge, et ce qui est barré

Ce qui est barré , chaque fois là où il était publié (le juge a porté ses barres dans le livrable 07 et dans sa note pour le mémoire, sans rien retirer : *réponse par délégation du 2026-10-22, réversible*) :

Table 11.7 : Ce qui tient selon le juge de la construction, et par quel chemin il l'a refait [mes.].

Chiffre	Lot	Chemin du juge
`.trc` v1.2 : 10 851 844 096 et 55 930 175 488 octets, −1 800 486 912 octets ; 0,127 91 et 0,125 62 bit par poids	A	son lecteur du format, fichiers entiers
chaque expert v1.2 est l'expert du GGUF au signe des zéros près ; aucun 8 ; trois empreintes	A	son décodeur, 96 experts tirés ; témoin v1.0 40 sur 40
aucune table v1.2 ne peut produire un 8 (0 sur 540)	A	son lecteur des tables
logits identiques au bit, v1.2 contre v1.0, sur les deux modèles	A	dix fenêtres neuves ; cassure 512 sur 512
J1 du 120b : v1.2 1,44 fois plus rapide que le v1.0 d'origine, ~~pour les fichiers, pas pour le format~~ par le format, au motif de ses requêtes, sur ce disque (réserve R1)	A	séance à trois fichiers : 1,442
$\lambda = 0$ bat $\lambda = 1$ (B2)	B	fenêtres neuves, moteur : −0,030 3 ± 0,008 4 (3,6 σ)
le moteur sert le mode rapide aussi fidèlement que la f64 substituée	B	fenêtres neuves : 0,003 7 ; 97,8 %
le mode exact est inchangé, et proche de la f64	A, B	fenêtres neuves : 0,004 4 ; 98,0 %
R1 : 0,264 0, 0,152 2, 0,131 8 ; appariées −0,111 8, −0,020 4, −0,132 2	B	sa KL sur leurs fichiers
$\lambda = 0$ meilleure que la frontière, et sa queue plus basse	B	fenêtres neuves, f64 substituée : −0,156 5 ± 0,033 4 (4,7 σ) ; 99e −26 %, 99,9^{e} −18 %

Données : `genome/JUGEMENT-construction.md`, § 8, avec la correction du 27 octobre (§ 6.1).

- ~~120b ×2,00 (gain du mode rapide)~~ → ×1,28 contre un fichier exact lu à la même vitesse par l'étage disque ; le ×2,00 reste vrai comme comparaison de deux fichiers.
- ~~20b ×1,24 (gain du mode rapide)~~ → ×0,94 en génération contre le v1.2 ; ×1,23 au préremplissage.
- ~~cause : le cache de fichiers du système~~ → impossible (lectures sans tampon) ; cause mesurée : l'étage disque, ~~l'état physique du fichier exact~~ le motif de lecture v1.0 (R1).
- ~~avec $\lambda = 0$, la queue baisse aussi au centile 99,9, contre $\lambda = 1$~~ → non reproduit hors de R1 ; pas une grandeur mesurable à 2 550 positions.
- ~~$\lambda = 0$ déforme la distribution dans une direction privilégiée~~ → le sens de la perplexité s'inverse sur d'autres fenêtres.
- Les ± des KL absolues ne sont pas barrés : ils sont *requalifiés*, naïfs.

11.9 Ce que la méthode du juge apprend

- **Une séance à trois fichiers sépare le format, le mode et le fichier** : $r(R/A) = r(R/B) \cdot r(B/A)$, et l'étage disque mesuré dans la même séance dit lequel des facteurs n'est pas le mécanisme. Une heure par modèle ; elle a fait tomber deux chiffres.
- **Avant d'attribuer un gain de vitesse à un mécanisme, mesurer l'étage disque de chaque fichier dans la même séance** : une minute.
- **Un décodeur de juge écrit depuis la seule description, dans un autre langage, vectorisé par somme préfixe** : deux secondes par expert, juste au bit du premier coup sur le décodage.
- **Des fenêtres neuves tirées loin de la région déjà vue, fondues en un seul fichier** que tous les outils lisent sans changement.
- **Ne pas comparer un 99,9^{e} centile de** 2 550 **positions d'une variante à l'autre** : il tient à deux ou trois positions.

- **Le texte d'un verrou compte** : le premier verrou du juge disait « pas une séance de vitesse », et le flux des pistes, qui cherchait le mot « vitesse », s'est mis en pause pour rien.
- **Arrêter une boucle bash qui lance un programme natif sous Windows ne tue pas le programme** : la passe f64 en cours a fini seule, et sa sortie complète a été gardée.

11.10 Ce qui reste

1. **La politique mixte des défauts** est à reprendre avec $r(R/B)$: sur le 120b, ×1,28 en génération pour une KL de 0,14 (proxy) ; sur le 20b, rien en génération. La simuler sur les traces avant de la construire.
2. **Le levier de vitesse que la construction a trouvé n'est pas un mécanisme du génome** : c'est le motif des requêtes de lecture du format sur ce disque. La variable exacte reste à isoler, à l'étage disque, par des motifs déclarés avant (juge du placement, § 12).
3. **Un autre corpus** (code, dialogue) pour la KL du génome synonyme, et la mutation synonyme du 120b.
4. Les fichiers lourds du juge (environ 13 Go de `.kld` hors du dépôt) sont régénérables par les outils commités ; ils peuvent être effacés quand Stone le veut.

11.11 Ce qui pourrait être faux

Ce qui pourrait être faux

- **Une séance par modèle, un seul matin** (le 22 octobre, de 6 h 17 à 7 h 11 pour le moteur). Les rapports $r(R/B)$ reposent sur le fait, mesuré dans la même séance, que B et R se lisent à la même vitesse à l'étage disque (R/B = 0,956 et 1,009, sous le seuil de séparation de 11,8 %) ; si ce n'était plus vrai un autre jour, il faudrait refaire la séance, pas reprendre le chiffre.
- **Les réglages ne sont pas tous ceux du lot B** : sur le 120b, ceux du J1 pour les trois fichiers ; le $r(R/A)$ du juge (1,85) diffère de celui du lot (2,00) pour cette raison au moins. Ce qui juge l'attribution est $r(R/B)$, mesuré aux mêmes réglages pour R et B.
- **Les fenêtres neuves sont de wiki.test**, comme R1 et la calibration : un autre texte, pas une autre sorte de texte.
- **Le décodeur du juge lit la même description que les auteurs du codeur.** Un contresens partagé ne serait pas vu par la comparaison des octets décodés… sauf qu'il l'aurait été : le GGUF est lu par un lecteur sans rien du format `.trc`, et les empreintes sont des SHA-256 calculés du côté du juge [121]. Ce qui n'est pas couvert : les parties du format que ni le v1.0 ni le v1.2 n'exercent (niveaux, brut, colonnes).
- **96 experts sur** 5 376 **(**1,8 %**)** par le chemin du juge. La probabilité qu'un défaut touchant au moins 5 % des experts d'un modèle échappe à 32 tirages vaut $0{,}95^{32} \approx 19\,\%$ (20b), et à 64 tirages $0{,}95^{64} \approx 3{,}75\,\%$ (120b) [calc.] (la source écrit 3,7 %, un arrondi trop bas). Le vérificateur du lot A, lui, couvre 100 % des experts ; le tirage juge ce vérificateur, il ne le remplace pas.
- **La KL du juge partage la convention du lot R** (coupe à $\log p > -16$, sans renormalisation) : indépendante du code, pas de la convention. Les différences appariées n'en dépendent presque pas.
- **Les prédictions du juge se sont trompées deux fois** (B2 « égales », mesuré $3{,}6\,\sigma$; $r(R/B)$ du 120b ≈ 1,38, mesuré 1,28) ; elles étaient déclarées avant, et la mesure a décidé.
- $\lambda = 1$ **par la référence substituée n'a pas été passé** sur les fenêtres neuves ; B2 n'y est jugé

que par le moteur, la frontière que par la référence substituée.
- **Le moteur et `genome-capture`** que le juge a fait tourner sont construits depuis le même code que ceux des lots : il juge les fichiers et les affirmations, pas un second moteur. Le second chemin des logits reste la référence f64 du lot R.
- **La réserve R1 elle-même** : le motif de lecture est établi sur ce disque, par le juge du placement, à l'étage disque ; la variable exacte n'est pas isolée, et une autre lecture pourrait la remplacer encore. Les verdicts de ce chapitre ne dépendent que de ce que l'étage disque mesure dans la séance, pas de son explication.

Troisième partie

Les pistes et le bilan

Chapitre 12

La géométrie tropicale du routeur

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, le lot R du Tome II avait montré que les bascules de routage portent l'essentiel de la KL entre deux calculs proches : imposer le routage de la référence en retire 88 %, et 96,5 % de la KL se trouve sur les positions qui basculent ; la marge du routeur, l'écart entre son 4e et son 5e logit, valait 0,083 en médiane sur toutes les cellules et 0,0095 sur celles qui basculent. La synthèse de la ronde (`genome/05-synthese.md` § 3.1) en avait tiré une idée de Stone : une *garde* qui lirait cette marge et referait en exact les seules cellules fragiles. Elle restait à juger. **Ce chapitre établit** que la garde tombe au seuil déclaré avant la lecture : pour 10 % des cellules signalées, elle attrape 57,1 % des bascules (2 905 sur 5 087), loin des 80 % exigés, et 80 % en coûteraient 26,6 %. Il pose la géométrie qui fait de la marge une distance (le routeur est un polynôme tropical, ses régions des polyèdres), mesure les lois de la marge (presque exponentielle), de la bascule ($\Phi(-m/\sigma)$, palier $\frac{1}{2}$) et de la propagation (26,6 % contre 3,4 %), démontre une forme close de la courbe de la garde, qui ne dépend que d'un nombre sans dimension κ, et dit, avec son juge, ce que ce théorème n'explique pas. Deux explorations, après le verdict et dites telles : la garde attrape 85,2 % des racines de cascade ; une garde qui se souvient du conflit gagne +1,61 point, sous les 5 déclarés. **Il reste** un journal de routage du mode rapide, pour connaître son κ et juger la garde des racines, et la mesure de la réparation : refaire une couche en exact éteint-il la racine et ses suites ? Sources : `genome/pistes/tropicale/garde-tropicale.md`, `pour-le-memoire.md`, `seuil-declare.md`, `seuil-declare-indices.md`, `JUGEMENT.md` ; `mesures/pistes/tropicale/` (journaux, figures, `indices/`, `juge/`) ; outils `outils/pistes/tropicale/`.

En bref

Le routeur d'une couche donne une note à chacun des 32 experts et garde les quatre meilleurs. L'espace de ses entrées est donc découpé en régions, une par quatuor, et ces régions sont des polyèdres : des pièces à faces planes. Une entrée proche d'une face est fragile : un rien la fait changer de quatuor, et ce changement, une « bascule », coûte cher en qualité. La marge mesure cette proximité. L'idée de la garde est celle d'un contrôleur qui ne surveille que les cas limites ; l'image est empruntée au cortex cingulaire, qui surveille le conflit entre deux réponses concurrentes, et ce n'est qu'une image. La mesure dit qu'elle surveille bien les premières couches et mal les dernières, parce qu'une bascule en entraîne d'autres, loin des faces, que la marge ne peut pas voir.

12.1 La question et le seuil déclaré

Le routeur de la couche ℓ calcule $z = W_r h + b_r$ (32 logits) et garde les quatre plus grands. Le mode rapide (chapitre 10) remplace des experts exacts par des experts compressés ; l'erreur qu'il introduit

déplace l'état h et, près d'une face, fait basculer le choix de la couche suivante. *La garde* : en mode rapide, après les experts rapides de la couche ℓ, le moteur lit la marge du routeur de $\ell + 1$; sous un seuil τ, il refait la couche ℓ en exact pour cette position.

Son coût est la part des cellules (couche, position) signalées, $c(\tau)$; son gain la part des bascules signalées, $b(\tau)$. **Seuil déclaré** (`genome/pistes/tropicale/seuil-declare.md`, commit `69260f1`, avant toute lecture) : la garde vaut d'être construite si elle attrape au moins 80 % des bascules en signalant au plus 10 % des cellules. Le 10 % dérive du coût : un défaut d'expert exact sur 10 % des cellules coûte au plus 10 % des octets du mode exact. La garde tourne dans le calcul rapide et ne voit que sa propre marge ; la marge qui décide est celle de B, le calcul qui s'écarte de la référence A. La prédiction, écrite dans la même déclaration : 55 à 65 %, donc un échec.

Le juge a vérifié l'ordre par `git log` : le seuil (`69260f1`, 2026-10-21 23:22:18, seul commit de ce fichier) précède les premiers outils de lecture (`4b2a8eb`, 23:23:32) et le verdict (`77d8110`, 23:25:17) ; git ne prouve pas qu'aucune lecture exploratoire n'a eu lieu avant, et l'horloge des notes de la piste est fausse de quelques minutes (`JUGEMENT.md` § 1).

12.2 Le routeur comme polynôme tropical

Soit $z(h) = W_r h + b_r \in \mathbb{R}^{32}$, $W_r = [w_1 \dots w_{32}]^\top$. Pour un ensemble S de $k = 4$ indices, la région

$$P_S = \{\, h \,:\, z_i(h) \geq z_j(h),\ \forall i \in S,\ \forall j \notin S \,\} \tag{12.1}$$

est une intersection de $k(32-k) = 112$ demi-espaces : un polyèdre convexe, éventuellement vide. Les P_S recouvrent l'espace et ne se chevauchent que par leurs bords.

Proposition 12.1 (Le top-k est un polynôme tropical)**.** *La fonction $F_k(h) = \max_{|S|=k} \sum_{i\in S} z_i(h)$ est un polynôme du semi-anneau* $(\max, +)$ *en les formes affines z_i ; elle est convexe et affine par morceaux, ses morceaux sont les P_S, et son lieu de non-différentiabilité, son hypersurface tropicale, est la réunion des faces $\{z_i = z_j\}$ où i est au rang 4 et j au rang 5.*

Démonstration. Dans le semi-anneau tropical, la « somme » est le max et le « produit » est + : F_k est la somme tropicale des monômes $\bigodot_{i\in S} z_i$, donc un polynôme tropical ; maximum de fonctions affines, elle est convexe et affine par morceaux. Sur l'intérieur de P_S, le max est atteint par le seul S (tout autre ensemble remplace un $i \in S$ par un $j \notin S$ avec $z_j < z_i$), et F_k y est affine de gradient $\sum_{i\in S} w_i$. Deux ensembles atteignent le max à la fois exactement quand on peut échanger un élément de rang 4 contre un élément de rang 5 sans changer la somme, c'est-à-dire sur $\{z_{(4)} = z_{(5)}\}$. ∎

C'est le cadre de Zhang, Naitzat et Lim pour les réseaux ReLU [115] ; le top-k s'y range par la même algèbre.

Définition 12.2 (Marge)**.** La marge d'une cellule est $m(h) = z_{(4)}(h) - z_{(5)}(h) \geq 0$, l'écart entre le 4e et le 5e logit. Elle est nulle exactement sur l'hypersurface tropicale.

Proposition 12.3 (La marge comme distance)**.** *Soit h intérieur à P_S. (a) Dans l'espace des logits, la distance de z à la frontière de sa région vaut $m/\sqrt{2}$. (b) Dans l'espace des états, la distance de h à la frontière de P_S vaut*

$$d(h, \partial P_S) = \min_{i\in S,\, j\notin S} \frac{z_i(h) - z_j(h)}{\|w_i - w_j\|} \;\geq\; \frac{m(h)}{\max_{i\in S,\, j\notin S} \|w_i - w_j\|}, \tag{12.2}$$

et la distance à la face du couple (i, j) des rangs 4 et 5 vaut exactement $m/\|w_i - w_j\|$.

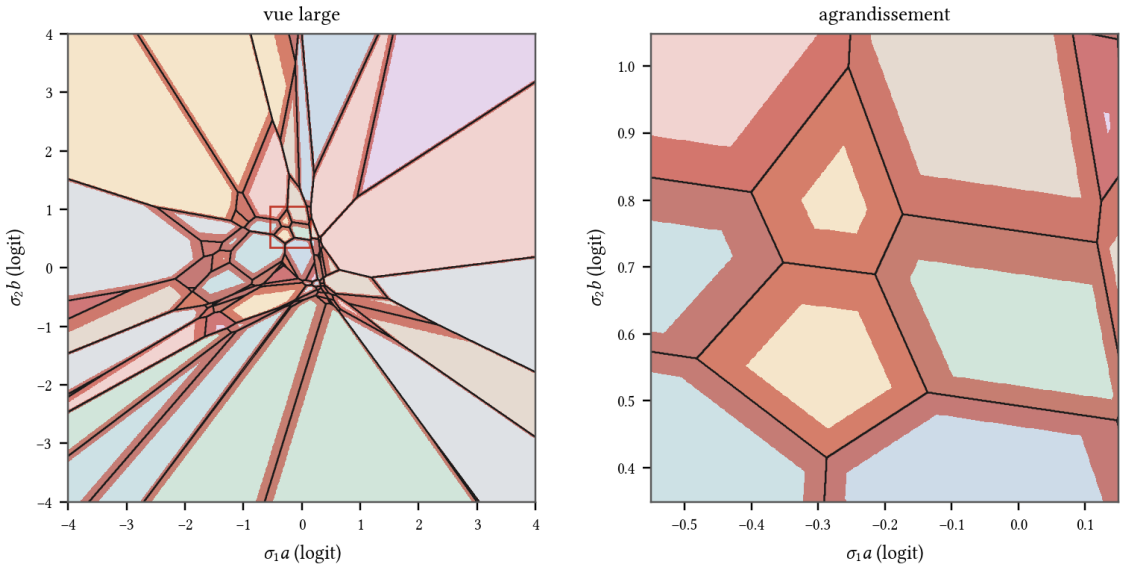


Figure 12.1 : Le routeur réel découpe l'espace des états en polyèdres ; la bande de garde suit leurs faces, plus large là où deux experts voisins se ressemblent. [calc.] Les régions top-4 du routeur de la couche 12 de gpt-oss-20b, dans le plan des deux premières directions singulières à droite de W_r ($h = au + bv$, par l'origine de l'espace des états normalisés) ; en rouge, la bande $\{m < \tau_{10}\}$. Gauche : 92 régions sur 8×8 logits ; droite : l'agrandissement d'une grappe ($0{,}7 \times 0{,}7$ logit). Une coupe à deux dimensions d'un espace à 2 880 : la géométrie y est vraie, les aires ne sont pas des probabilités, et le cadrage de l'agrandissement est un choix. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/pistes/tropicale/figures/coupe-routeur-c12-fr.png` ; script `outils/pistes/tropicale/figures.py`.

Démonstration. La distance d'un point intérieur à la frontière d'un polyèdre convexe est le minimum de ses distances aux hyperplans qui le définissent : le point le plus proche de la frontière est sur l'un d'eux, et un hyperplan redondant, qui ne porte pas de face, est au-delà de la frontière. La distance de h à $\{z_i = z_j\}$ est $(z_i - z_j)/\|w_i - w_j\|$, d'où l'égalité de (b) ; chaque numérateur est au moins m, d'où la borne. (a) est le même calcul dans les coordonnées z, où $w_i - w_j$ devient $e_i - e_j$, de norme $\sqrt{2}$, et le plus petit numérateur est m. ∎

La note de la piste écrit que $m/\|w_i - w_j\|$, pour le couple des rangs 4 et 5, borne par en dessous la distance à toute frontière de la région. C'est vrai de cette face, et dans l'espace des logits ; dans l'espace des états, un autre couple dont les vecteurs de routeur diffèrent davantage peut être plus proche, et la borne générale est celle de (12.2). La bande de garde $\{m < \tau\}$ est donc, le long de la face du couple (i, j), une tranche d'épaisseur $\tau/\|w_i - w_j\|$ de chaque côté, et de demi-épaisseur $\tau/\|P(w_i - w_j)\|$ dans une coupe par un plan de projecteur P (figure 12.1) : large là où deux experts voisins se ressemblent dans le plan.

12.3 Les données et les témoins

Les données. Trois journaux de routage du lot R (commit `21453da`, empreintes SHA-256 dans la déclaration), gpt-oss-20b, 24 couches × 5 110 positions (10 fenêtres de 511), soit 122 640 cellules : A (`ref-openai-bis`), B (`ref-llamacpp-bis`), C (la rampe de llama.cpp avec le routage de A imposé). Une bascule est une cellule où les *ensembles* de quatre experts de A et B diffèrent : 5 087.

Table 12.1 : La garde tropicale : les témoins de l'instrument, chacun avec sa cassure [mes.].

Témoin	Réponse connue	Obtenu	Cassure
T1, la lecture	5 087 bascules et les 8 quantiles de marge du résumé du lot R	au dernier chiffre de f32	un octet d'expert changé : 5 088 ; pas de 7 octets : refusé
T2, la réponse d'avance	A contre C (routage imposé) : 0 bascule	0	un expert changé : 1
T3, l'instrument discrimine	marges permutées : $b(\tau_{10}) = 10 \pm 2\,\%$; marges oracles : $b = 100\,\%$ pour $c \leq 4{,}2\,\%$	10,05 % ; 100 % à 4,15 %	(sa propre cassure)

Données : `mesures/pistes/tropicale/temoins.json` ; `genome/pistes/tropicale/garde-tropicale.md` § 2.

Table 12.2 : La courbe de la garde : cellules et bascules sous le seuil τ de la marge de B [mes.]. Le juge la refait par un classement explicite des 122 640 cellules, sans seuil interpolé, à l'unité.

τ	cellules sous τ	part des cellules c	bascules sous τ	part des bascules b
10^{-4}	106	0,09 %	58	1,1 %
$3{,}16 \cdot 10^{-4}$	336	0,27 %	168	3,3 %
10^{-3}	995	0,81 %	477	9,4 %
$3{,}16 \cdot 10^{-3}$	3 238	2,64 %	1 265	24,9 %
10^{-2}	10 089	8,23 %	2 654	52,2 %
$\tau_{10} = 0{,}012410$	**12 264**	**10,00 %**	**2 905**	**57,1 %**
$1{,}58 \cdot 10^{-2}$	15 406	12,56 %	3 167	62,3 %
$3{,}16 \cdot 10^{-2}$	28 733	23,43 %	3 932	77,3 %
0,0366 (80 % des bascules)	32 623	26,60 %	4 070	80,0 %
10^{-1}	69 090	56,34 %	4 829	94,9 %

Données : `mesures/pistes/tropicale/courbe-garde.csv`, `garde-resume.json`, `second-chemin.json` ; juge : `mesures/pistes/tropicale/juge/juge.json`.

(Comparer les listes ordonnées en compterait 11 028 ; le top-4 est un ensemble, l'ordre interne ne change pas les experts appelés.) Ce sont les seuls journaux du dépôt qui portent une marge. **Aucun journal de routage du mode rapide n'existe** : la perturbation mesurée ici est celle de deux calculs exacts, la rampe de position d'OpenAI et celle de llama.cpp.

T3 est sa propre cassure : le même instrument rend 10 % quand la marge ne sait rien et 100 % quand elle sait tout (tableau 12.1). Le juge a refait T2 et T3 par son code, et ajouté les siens : un octet changé sur une cellule sans bascule donne 5 088 ; ses marges permutées donnent 10,4 % ; et un témoin J5, qui met 600 bascules non attrapées à marge nulle, attend $2\,905 + 600 - 64 = 3\,441$ attrapées et les obtient. Sa première écriture attendait ~~$2\,905 + 600 = 3\,505$~~ : elle oubliait que les 600 cellules entrées en tête en chassent 600, dont 64 bascules (`JUGEMENT.md` § 3).

12.4 Le verdict : la courbe de la garde

Au seuil déclaré, la garde tombe : au décile $\tau_{10} = 0{,}012410$, elle signale 12 264 cellules et attrape 2 905 bascules sur 5 087, 57,1 % (tableau 12.2 et figure 12.2). Pour 80 % il faudrait 32 623 cellules, 26,6 %. La prédiction écrite avant (55 à 65 %) tient. Le second chemin (une lecture en Python pur, par la

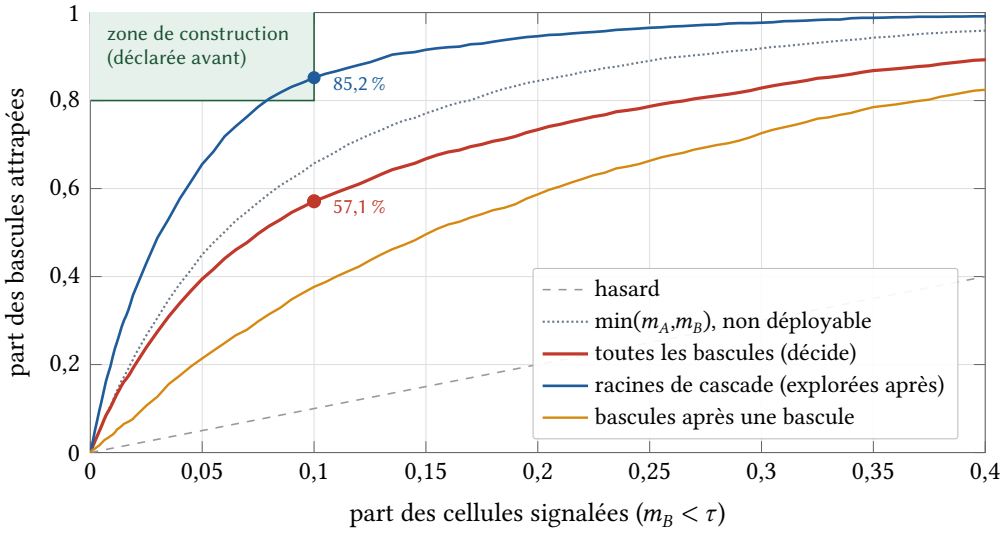


Figure 12.2 : La garde attrape 57,1 % des bascules pour 10 % des cellules, loin de la zone de construction déclarée ; elle attrape 85,2 % des racines de cascade. [mes.] Part des bascules attrapées contre part des cellules signalées ($m_B < \tau$) : toutes les bascules (le verdict), les racines de cascade et les bascules qui suivent une bascule (explorées après le verdict), l'oracle non déployable $\min(m_A, m_B)$, et le hasard. Zone verte : la zone de construction déclarée avant la lecture ($c \leq 10\,\%$, $b \geq 80\,\%$).

Données : `mesures/pistes/tropicale/figures/fig-courbe.csv` ; script `outils/pistes/tropicale/figures.py`.

fonction de lecture du lot R, comptage par boucles) rend les mêmes 2 905 bascules sur les mêmes 12 264 cellules ; le juge, par son propre code et un classement explicite (un seul ex aequo à la frontière, sans ambiguïté), aussi. Le seuil τ_{10} est le décile des marges mesurées sur les journaux eux-mêmes : cela avantage la garde, qui tombe quand même.

Les variantes, qui ne décident pas. La marge de A au lieu de B : 56,1 % (aire sous la courbe ROC 0,862 pour les deux). L'oracle non déployable $\min(m_A, m_B)$: 65,7 %, et 80 % pour 16,9 % des cellules ; même lui manque le seuil. Les seules positions notées par le juge du lot R : 53,4 %. Un contrôle croisé sous une autre perturbation (le moteur de la carte contre la référence, 22 bascules) attrape 9 bascules sur 22 ; il ne juge rien (intervalle à 95 % de 21 à 64 %), mais ne contredit pas [mes.].

La profondeur. La garde voit bien les premières couches et mal les dernières (tableau 12.3 et figure 12.3). L'échelle robuste de la perturbation de l'écart, $\delta = m_A - m_B$ sur les cellules sans bascule à grande marge ($m_B > 0{,}1$), monte de 0,0022 à la couche 0 à 0,0059–0,0086 aux couches 9 à 23. Une garde réglée couche par couche (τ_ℓ appris sur cinq fenêtres, jugé sur les cinq autres, puis l'inverse) ne gagne rien : 55,4 et 56,0 % contre 55,9 et 58,7 % pour le τ global sur les mêmes moitiés [mes.].

12.5 Pourquoi elle tombe

12.5.1 La loi de la marge : presque exponentielle

Lemme 12.4 (Représentation de Rényi). *Si $X_1, \ldots, X_n$ sont exponentielles indépendantes de taux λ, rangées en ordre décroissant $X_{(1)} \geq \ldots \geq X_{(n)}$, les espacements $D_k = X_{(k)} - X_{(k+1)}$, $k = 1, \ldots, n$, avec $X_{(n+1)} = 0$, sont indépendants et D_k suit une loi exponentielle de taux $k\lambda$.*

Table 12.3 : La garde par couche, au τ_{10} global [mes.]. Les fourchettes sont refaites par le juge.

Couches	taux de bascule	$b(\tau_{10})$	marge médiane aux bascules	aire ROC
0	0,72 %	100 %	0,0013	0,991
1 à 4	1,3 à 1,9 %	86,0 à 91,0 %	0,0029 à 0,0045	0,956 à 0,961
5 à 7	2,1 à 3,1 %	73,9 à 79,6 %	0,0042 à 0,0060	0,930 à 0,957
8 à 14	3,3 à 5,0 %	51,2 à 58,6 %	0,0092 à 0,0113	0,851 à 0,896
15 à 23	4,7 à 8,1 %	37,8 à 60,5 %	0,0084 à 0,0199	0,771 à 0,864

Données : `mesures/pistes/tropicale/decile-par-couche.csv`.

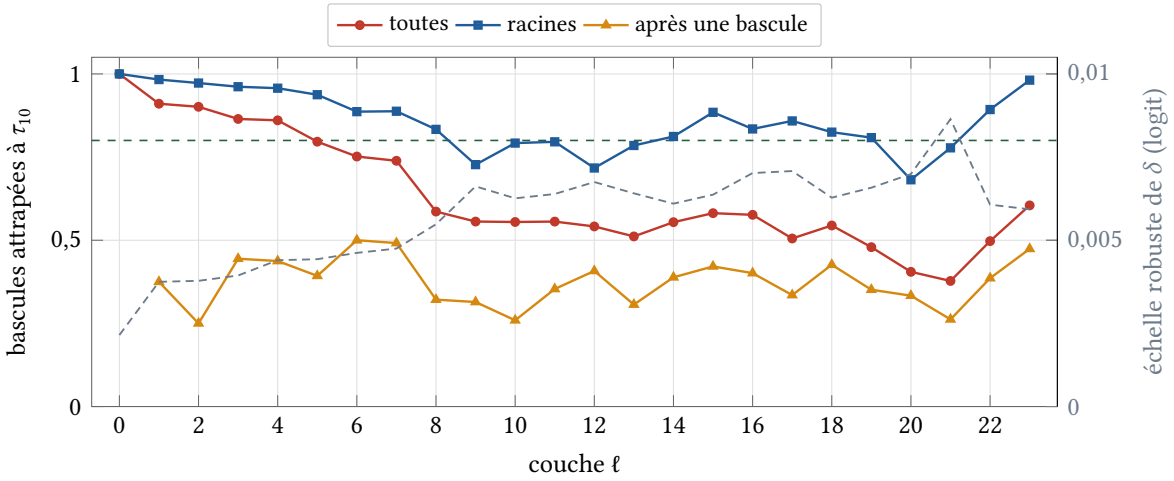


Figure 12.3 : La garde voit les premières couches et perd les dernières, à mesure que la perturbation grandit. [mes.] Part des bascules attrapées au τ_{10} global, par couche : toutes, racines, bascules qui suivent une bascule ; tirets (axe de droite) : l'échelle robuste de la perturbation δ. Trait horizontal : 80 %.

Données : `mesures/pistes/tropicale/figures/fig-par-couche.csv` ; script `outils/pistes/tropicale/figures.py`.

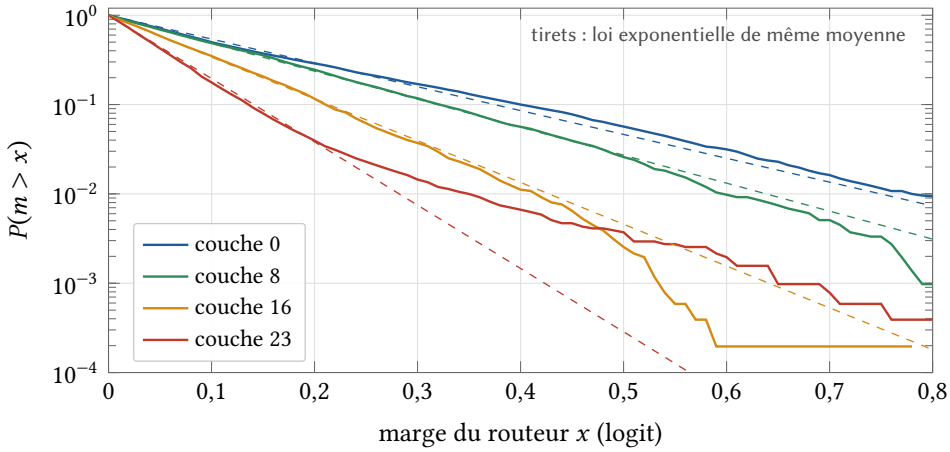


Figure 12.4 : La survie de la marge est une droite en échelle logarithmique : la marge est presque exponentielle, comme le prédit l'espacement des statistiques d'ordre. [mes.] $P(m > x)$ pour quatre couches (0, 8, 16, 23) ; tirets : la loi exponentielle de même moyenne.

Données : `mesures/pistes/tropicale/figures/fig-survie.csv` ; script `outils/pistes/tropicale/figures.py`.

Démonstration. Rangeons les mêmes variables en ordre croissant, $Y_1 \le \dots \le Y_n$ (soit $Y_j = X_{(n+1-j)}$), et posons $Y_0 = 0$, c'est-à-dire $X_{(n+1)} = 0$. Le minimum de n exponentielles indépendantes de taux λ suit la loi exponentielle de taux $n\lambda$, puisque $P(Y_1 > y) = \prod_i P(X_i > y) = e^{-n\lambda y}$. Sachant $Y_1 = y$ et l'indice de la variable qui l'atteint, les $n-1$ autres sont indépendantes et conditionnées par $X_i > y$; par l'absence de mémoire de la loi exponentielle, leurs excès $X_i - y$ sont exponentiels de taux λ, indépendants entre eux, et leur loi ne dépend ni de y ni de l'indice. L'espacement $Y_2 - Y_1$, minimum de ces $n-1$ excès, suit donc la loi exponentielle de taux $(n-1)\lambda$ et il est indépendant de Y_1. En recommençant à partir de Y_2, puis de Y_3, et ainsi de suite, on obtient que les $Y_{j+1} - Y_j$ ($j = 0, \dots, n-1$) sont indépendants, de taux $(n-j)\lambda$. Or $Y_{j+1} - Y_j = X_{(n-j)} - X_{(n-j+1)} = D_{n-j}$: avec $k = n-j$, D_k suit la loi exponentielle de taux $k\lambda$, et les D_k sont indépendants. ■

C'est le théorème de Rényi [126] (notice vérifiée par la piste le 2026-10-22, article non relu) ; la démonstration ci-dessus, par l'absence de mémoire, ne dépend pas de l'article. Pour une loi mère de densité f et de taux de défaillance $\eta(x) = f(x)/(1-F(x))$, la transformation $x \mapsto -\ln(1-F(x))$ rend les variables exponentielles de taux 1 et, au voisinage du haut de l'échantillon, elle est localement linéaire de pente $\eta(x_{(k)})$; d'où $D_k \approx \mathrm{Exp}\big(k\,\eta(x_{(k)})\big)$. La marge $m = D_4$ est donc, au premier ordre, exponentielle : densité finie et non nulle en zéro, $f_m(0) \approx 1/\mathbb{E}[m]$, et $P(m < \tau) \approx \tau/\mathbb{E}[m]$ pour τ petit [calc.]. Les 32 logits ne sont ni indépendants ni de même loi ; la mesure dit que la conclusion tient quand même.

Mesuré à chaque couche [mes.] (`mesures/pistes/tropicale/loi-marge-par-couche.csv`) : l'ajustement gamma donne une forme de 0,91 à 1,09 (1 serait l'exponentielle) ; la distance de Kolmogorov-Smirnov à l'exponentielle de même moyenne vaut 0,006 à 0,042, contre 0,09 à 0,18 à la demi-normale et 0,06 à 0,08 à la log-normale ; la pente de $\log P(m < \tau)$ contre $\log \tau$ sur $[10^{-4}; 10^{-2}]$ vaut 0,90 à 1,20 ; la densité sous 10^{-3} (5,5 à 17,6) suit $1/\mathbb{E}[m]$ (5,2 à 16,3). Les couches profondes ont des marges plus serrées (figure 12.4).

D'une couche à la suivante. Pour une même position, la corrélation de Spearman entre m_ℓ et $m_{\ell+1}$ va de 0,002 à 0,125 ; groupée sur les 23 couples, 0,058, au-dessus du 99,9^{e} centile de 999 permutations (0,023) : réelle et minuscule [mes.]. Une marge serrée à une couche ne dit presque rien de la suivante ;

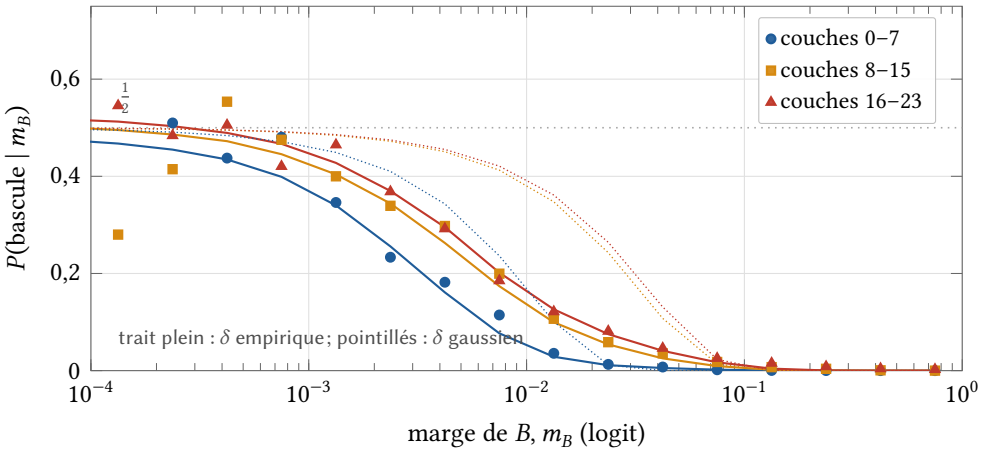


Figure 12.5 : La probabilité de bascule tend vers $\frac{1}{2}$ quand la marge tend vers zéro : la loi est $\Phi(-m/\sigma)$, pas 2Φ. [mes.] [calc.] $P(\text{bascule} \mid m_B)$ mesurée par classes de marge, trois groupes de couches ; trait plein : le modèle à δ empirique $P(\delta < -m)$; pointillés : le gaussien $\Phi(-m/\hat{\sigma})$.

Données : `mesures/pistes/tropicale/figures/fig-loi-bascule.csv` ; script `outils/pistes/tropicale/figures.py`.

chaque couche tire sa marge à neuf. Conséquence directe : comme priorité de préchargement de la couche suivante, la marge fait une aire ROC de 0,567, sous la barre déclarée de 0,60 ; pour le jeton suivant, 0,627 : utile par la barre, mais faiblement (le jeton suivant garde en moyenne 2,0 experts sur 4, 1,89 au décile de marge le plus bas, 2,17 au plus haut, contre 0,5 au hasard).

12.5.2 La loi de bascule

Proposition 12.5 (Loi de bascule). *Soit m_B la marge du calcul qui garde et δ la perturbation, entre A et B, de l'écart entre les experts aux rangs 4 et 5 de B. La cellule bascule dès que l'écart de A passe sous zéro, $m_B + \delta < 0$. Si δ est indépendante de m_B, de loi symétrique G,*

$$P(\textit{bascule} \mid m_B = m) = G(-m), \qquad \lim_{m \to 0} P = G(0) = \frac{1}{2}. \tag{12.3}$$

Démonstration. $P(m + \delta < 0) = P(\delta < -m) = G(-m)$, et $G(0) = \frac{1}{2}$ par symétrie. ∎

Pour une perturbation gaussienne, $\Phi(-m/\sigma)$. La déclaration avait écrit ~~$2\Phi(-m/\sigma)$~~, qui vaudrait 1 en zéro, comme si la bascule était certaine sur la face ; la dérivation donne Φ, et les données tranchent : sous $m = 10^{-4}$, 58 bascules sur 106 cellules (0,55 ± 0,05), et la vraisemblance de Φ bat celle de 2Φ aux 24 couches [mes.] (`loi-bascule-sigma.csv`). C'est une borne inférieure de la probabilité réelle : d'autres couples que (4, 5) peuvent croiser, mais il leur faut une perturbation plus grande que m. Le juge le note : la forme a changé après la lecture ; le théorème qui suit et l'extrapolation reposent sur elle, et restent des modèles. La forme gaussienne est fausse dans la queue : δ a un kurtosis en excès de 3,5 (couche 0) à 134 (couche 7). Le modèle gaussien prédit 7,8 % de bascules (mesuré 4,15 %) ; la forme empirique de δ, mise à l'échelle, 3,6 % [calc.] (figure 12.5).

12.5.3 La propagation : une bascule en appelle d'autres

Sachant une bascule en (ℓ, p), la même position bascule en $\ell + 1$ dans **26,6 %** des cas (4 672 couples), contre **3,4 %** sans bascule en ℓ [mes.] (`propagation.csv` ; le juge : 26,65 et 3,37 %). L'écart grandit avec la marge de la couche suivante (tableau 12.4).

Table 12.4 : La propagation : probabilité de bascule en $\ell+1$, selon la marge en $\ell+1$, après une bascule en ℓ ou sans [mes.]. Entre parenthèses, le nombre de couples.

Marge en $\ell+1$	après une bascule	sans bascule avant	rapport
$< 10^{-3}$	61,8 % (34)	47,5 % (930)	1,3
10^{-3} à 10^{-2}	57,1 % (380)	23,1 % (8 403)	2,5
10^{-2} à $3\cdot10^{-2}$	47,2 % (755)	5,4 % (15 896)	8,7
$3\cdot10^{-2}$ à 0,1	27,0 % (1 716)	1,3 % (38 415)	21
$> 0{,}1$	10,5 % (1 787)	0,14 % (49 214)	75

Données : `mesures/pistes/tropicale/propagation.csv`.

Après une bascule, l'état caché a changé d'experts : la perturbation n'est plus un arrondi, c'est la différence entre deux experts (33 à 84 % de l'énergie de sortie, `05-synthese.md` § 2.2). Le κ local devient grand, et la bascule suivante tombe à une marge large que la garde ne signale pas. C'est la cause principale de la chute en profondeur : les secondaires sont 59 % des bascules (3 006 sur 5 087), de marge médiane 0,0194 contre 0,0037 pour les racines. Le rapport ×7,9 (et ×75 à marge large) est une association ; l'intervention du lot R (routage imposé, 88 % de la KL retirée) va dans le même sens sans la démontrer pour les cascades.

12.5.4 Le nombre de garde : une forme close

Hypothèse 12.6 (Le modèle homogène)**.** La marge M suit une loi exponentielle de taux λ ; la perturbation δ suit $\mathcal{N}(0,\sigma^2)$, indépendante de M ; la cellule bascule si $\delta < -M$; la garde signale $M<\tau$. On note $\kappa=\lambda\sigma=\sigma/\mathbb{E}[M]$, le *nombre de garde*, $t=\tau/\sigma$, Φ et φ la répartition et la densité normales.

Lemme 12.7. *Pour $\kappa>0$ et $u\ge 0$: $e^{-\kappa u}\varphi(u)=e^{\kappa^2/2}\varphi(u+\kappa)$.*

Démonstration. $-\kappa u-u^2/2=-(u+\kappa)^2/2+\kappa^2/2$. ∎

Lemme 12.8. $\displaystyle\int_0^T \lambda e^{-\lambda m}\,\Phi(-m/\sigma)\,\mathrm{d}m=\tfrac12-e^{-\lambda T}\Phi(-T/\sigma)-e^{\kappa^2/2}\big[\Phi(T/\sigma+\kappa)-\Phi(\kappa)\big]$.

Démonstration. Par parties, avec $U=\Phi(-m/\sigma)$, $\mathrm{d}V=\lambda e^{-\lambda m}\,\mathrm{d}m$, $V=-e^{-\lambda m}$, $\mathrm{d}U=-\varphi(m/\sigma)/\sigma\,\mathrm{d}m$:

$$\int_0^T=\big[-e^{-\lambda m}\Phi(-m/\sigma)\big]_0^T-\int_0^T e^{-\lambda m}\,\frac{\varphi(m/\sigma)}{\sigma}\,\mathrm{d}m=\tfrac12-e^{-\lambda T}\Phi(-T/\sigma)-\int_0^{T/\sigma}e^{-\kappa u}\varphi(u)\,\mathrm{d}u,$$

et par le lemme 12.7 la dernière intégrale vaut $e^{\kappa^2/2}\int_0^{T/\sigma}\varphi(u+\kappa)\,\mathrm{d}u=e^{\kappa^2/2}\big[\Phi(T/\sigma+\kappa)-\Phi(\kappa)\big]$. ∎

Théorème 12.9 (Le nombre de garde)**.** *Sous l'hypothèse 12.6, la part des cellules signalées est $c=1-e^{-\lambda\tau}$, le taux de bascule est*

$$P(B)=\tfrac12-e^{\kappa^2/2}\,\Phi(-\kappa),\tag{12.4}$$

et la part des bascules signalées est

$$b=\frac{\tfrac12-e^{-\lambda\tau}\Phi(-t)-e^{\kappa^2/2}\big(\Phi(t+\kappa)-\Phi(\kappa)\big)}{\tfrac12-e^{\kappa^2/2}\,\Phi(-\kappa)},\qquad \lambda\tau=-\ln(1-c),\quad t=\lambda\tau/\kappa.\tag{12.5}$$

La courbe (c,b) ne dépend que de κ.

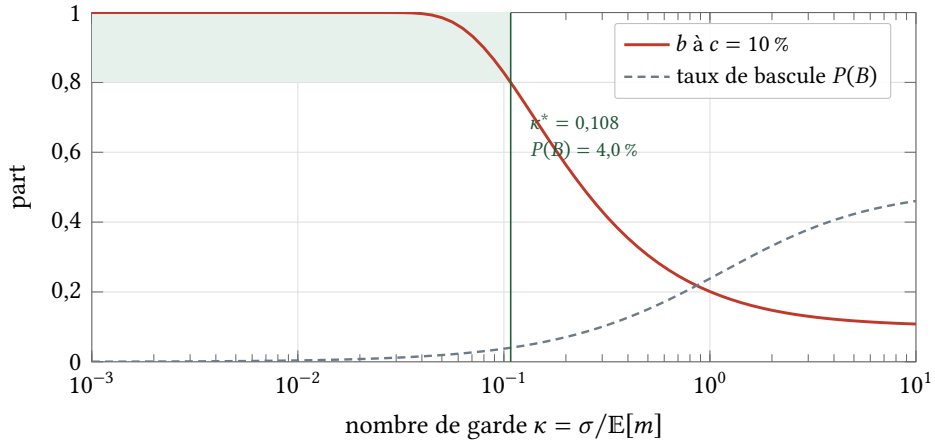


Figure 12.6 : Dans le modèle homogène, une seule grandeur, $\kappa = \sigma/\mathbb{E}[m]$, décide de toute garde à marge : la garde tient 80 / 10 si et seulement si $\kappa \leq 0{,}108$. [calc.] b à $c = 10\,\%$ et taux de bascule $P(B)$ contre κ, par la forme close (12.5) ; zone verte : la zone de construction. Le modèle est homogène (une loi de marge, un σ) ; la garde réelle mêle 24 couches de κ_ℓ de 0,016 à 0,48, et ce théorème n'explique qu'environ un point des 23 qui manquent (texte).

Données : `mesures/pistes/tropicale/theoreme-kappa.csv` ; script `outils/pistes/tropicale/theoreme.py`.

Démonstration. $c = P(M < \tau) = 1 - e^{-\lambda\tau}$. $P(B)$ est le lemme 12.8 quand $T \to \infty$: les termes en $e^{-\lambda T}$ s'annulent, $\Phi(T/\sigma + \kappa) \to 1$, d'où $\frac{1}{2} - e^{\kappa^2/2}(1 - \Phi(\kappa)) = \frac{1}{2} - e^{\kappa^2/2}\Phi(-\kappa)$. $b = P(M < \tau, B)/P(B)$ est le lemme 12.8 en $T = \tau$ divisé par $P(B)$. Enfin $\lambda\tau = -\ln(1 - c)$ et $t = \tau/\sigma = \lambda\tau/\kappa$ ne dépendent que de (c, κ). ∎

Les limites se lisent sur (12.5) : $\kappa \to 0$ donne $b \to 1$ (les bascules se concentrent en $m = 0$) ; $\kappa \to \infty$ donne $b \to c$ (la marge ne sait plus rien), le témoin T3 vu par la théorie. Vérification [calc.] (`mesures/pistes/tropicale/theoreme.json`) : une quadrature numérique (tolérances 10^{-13}) contre la forme close sur 18 couples $\kappa \in \{0{,}01; 0{,}05; 0{,}1; 0{,}3; 1; 3\}$, $c \in \{1; 10; 30\,\%\}$: écart maximal ~~$1{,}3 \cdot 10^{-14}$~~ $3{,}7 \cdot 10^{-15}$; le premier chiffre venait de l'écriture naïve $e^{\kappa^2/2}\Phi(-\kappa)$, qui perd des chiffres quand κ grandit, réécrite depuis par le logarithme de la queue normale. Un exemple : $P(B)$ en $\kappa = 0{,}2$ vaut $\frac{1}{2} - e^{0{,}02} \times 0{,}42074 = 0{,}07076$, 0,0707601917665 par la forme close contre 0,0707601917664 par la quadrature.

Le seuil 80 / 10 se résout en $\kappa^* = 0{,}1079$, où $P(B) = 4{,}03\,\%$ [calc.] (figure 12.6) ; le juge, par une quadrature de Simpson sans la forme close, redonne $b = 79{,}993\,\%$ et $P(B) = 4{,}0294\,\%$ en $\kappa = 0{,}1079$.

Le corollaire, et ce que le juge en a fait. Comme b décroît et $P(B)$ croît avec κ, la piste en avait tiré : ~~« une garde à marge ne peut tenir 80 / 10 que si le calcul rapide fait basculer moins d'environ 4 % des cellules »~~. Le juge (JUGEMENT.md § 4.1) en fait une propriété du seul modèle homogène, par un contre-exemple construit : 2 couches sur 24 à marges serrées (moyenne 10^{-4}, $\sigma = 10^{-3}$) et 22 à marges larges (moyenne 0,1, $\sigma = 0{,}004$), garde à seuil global : $P(B) = 5{,}26\,\%$ et pourtant $b = 85{,}9\,\%$ pour 10 % des cellules [calc.] (`mesures/pistes/tropicale/juge/juge-modele.json`). La phrase ~~« une queue de δ plus lourde ne fait que baisser ce plafond »~~ n'est vraie que dans le seul cas calculé : l'ordre « plus lourde » n'y est pas défini.

Le théorème n'explique pas l'échec. Le livrable plaçait côte à côte « il faut au plus 4,0 % de bascules » et « les deux rampes en font 4,15 % », ce qui se lit comme une explication. Au taux mesuré

Table 12.5 : L'extrapolation : $b(\tau_{10})$ prédit quand la perturbation de l'écart est multipliée par s [calc.] (modèle).

s	toutes, δ gaussien	toutes, δ empirique	racines, δ empirique
0,5	71,4 %	74,5 %	93,8 %
1 (mesuré : 57,1 % ; racines 85,2 %)	49,0 %	62,1 %	85,3 %
1,5	38,4 %	54,9 %	77,2 %
2	32,4 %	49,8 %	69,9 %
3	25,9 %	42,5 %	58,7 %
5	20,2 %	34,2 %	45,1 %
10	15,5 %	25,3 %	30,8 %

Données : `mesures/pistes/tropicale/extrapolation-garde.csv`, `cascades.json`.

(5 087/122 640 = 4,148 %), le modèle homogène donne $\kappa = 0{,}1113$ et $b = 78{,}8$ % ; la mesure dit 57,1 %. Le nombre de garde rend compte d'environ 1 point sur les 23 qui manquent. Le reste vient de l'hétérogénéité des couches (la garde réelle applique un seuil global à 24 couches dont la marge moyenne va de 0,061 à 0,193 et dont κ_ℓ va de 0,016 à 0,48) et des cascades. Couche par couche, la forme close suit la mesure de b aux couches 0 à 9 à 0,1 près et la sous-estime de 0,02 à 0,17 plus profond, où le σ ajusté absorbe les secondaires [mes.] (`theoreme-par-couche.csv`). Le nombre de garde est une belle forme close ; ce n'est pas la raison de l'échec.

12.5.5 L'extrapolation au mode rapide (modèle)

Le mode rapide perturbe sans doute plus que les deux rampes : avec 32 experts sur 768 remplacés, sa KL est déjà 0,036, contre 0,0145 pour les deux rampes, et elle monte avec le nombre d'experts remplacés (0,0636 avec 128). On multiplie la perturbation par s en gardant les marges, $P(\text{bascule} \mid m) = G(-m/s)$, avec deux G : la gaussienne de σ_ℓ ajustée, et la loi empirique de δ par couche (tableau 12.5 et figure 12.7). À $s = 1$, les deux modèles encadrent la mesure (49,0 et 62,1 % contre 57,1 %) ; pour les racines, le modèle empirique tombe juste (1 956 racines prédites contre 2 081 comptées, b 85,3 contre 85,2 %). Les racines passent sous 80 % dès $s \approx 1{,}3$ (interpolation linéaire entre $s = 1$ et 1,5 ; un message de commit disait ~~1,4~~). Ce sont des extrapolations, à lire comme telles.

12.6 Après le verdict : deux explorations

12.6.1 La garde des racines

Cette lecture a été choisie *après* avoir vu la propagation ; elle ne change pas le verdict. Elle n'a pourtant aucun paramètre ajusté sur les bascules : τ_{10} est un quantile des marges seules, et la racine (la première bascule de la position, toutes couches confondues) ne dépend d'aucun seuil. 2 081 positions sur 5 110 (40,7 %) ont au moins une bascule ; la garde signale leur racine dans **85,2 %** des cas (1 773), les secondaires dans 37,7 % (1 132 sur 3 006) ; second chemin et juge égaux à l'unité [mes.] (`cascades.json`, `second-chemin-racines.json`). Par couche, les racines sont attrapées de 68 % (couche 20) à 100 % (couche 0), au-dessus de 80 % à 17 couches sur 24.

Si réparer une racine éteignait ses suites, c'est elle qu'il faudrait attraper. Deux choses manquent, qu'aucun journal ne donne. *La réparation* : dans ces journaux la perturbation vient de la rampe de position, dans l'attention, et refaire une couche en exact n'y changerait rien ; en mode rapide, refaire la couche ℓ retire l'erreur des experts de ℓ, pas celle des couches d'avant ni celle du cache KV des

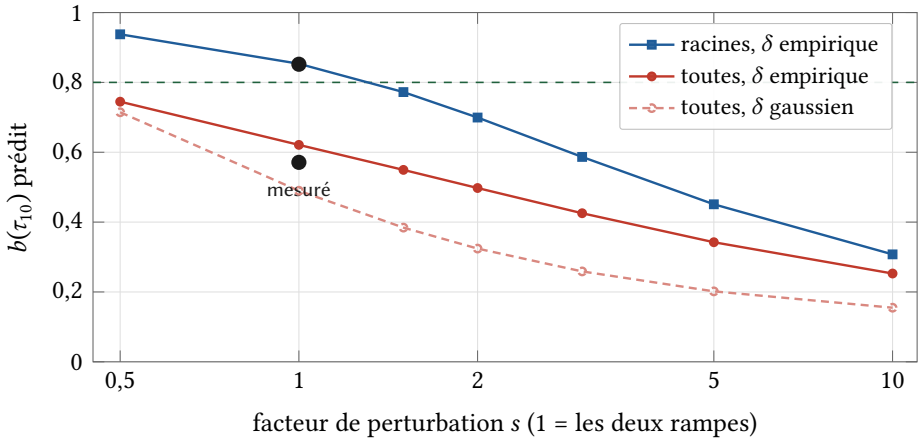


Figure 12.7 : Si le mode rapide perturbe davantage que les deux rampes, la garde attrape encore moins, et la garde des racines passe sous 80 % vers $s \approx 1{,}3$. [calc.] Modèle, non mesure : $b(\tau_{10})$ prédit contre le facteur de perturbation s (racines et toutes, δ empirique ; toutes, δ gaussien) ; points noirs : les deux valeurs mesurées à $s = 1$.

Données : `mesures/pistes/tropicale/figures/fig-extrapolation.csv` ; script `outils/pistes/tropicale/figures.py`.

positions passées. *Le κ du mode rapide.* La définition « première bascule de la position » ignore aussi la propagation par le cache KV : une racine peut être la suite d'une bascule d'une position antérieure.

La mesure qui jugerait (G2, puis G3). Deux journaux au format du lot R, le moteur exact et le moteur en mode rapide, chacun avec sa propre marge (8 octets par cellule, 981 128 octets par passe ; le noyau `routeur_topk` classe déjà, la marge est une soustraction de plus). Seuil à déclarer avant la passe : la garde des racines vaut d'être construite si, au τ_{10} des marges du mode rapide, elle signale au moins 80 % des racines pour au plus 10 % des cellules, et $P(B)$ du mode rapide est mesuré pour situer son κ. La prédiction de la piste, « elle tombe si $P(B)$ dépasse environ 5 % », est celle du modèle homogène, que le contre-exemple du juge peut démentir. Seulement si G2 passe, G3, la réparation : à chaque racine signalée, refaire la couche précédente en exact, et juger la KL appariée contre le mode rapide sans garde.

12.6.2 La mémoire du conflit : une garde à indices multiples

D'où vient l'idée. Le cortex cingulaire antérieur surveille le conflit entre réponses concurrentes et en accumule la trace d'un essai à l'autre [127] ; sous conflit élevé, le cortex médiofrontal relève le seuil de décision [128] (notices vérifiées par la piste le 2026-10-26, articles non relus). La garde fait la moitié de ce mécanisme : elle lit le conflit instantané, la marge. La mesure demande si l'autre moitié, la mémoire du conflit, trie mieux les bascules. C'est une analogie qui propose une mesure ; elle ne prouve rien.

Le protocole (`genome/pistes/tropicale/seuil-declare-indices.md`, commit `50bf472`, inchangé depuis, avant tout journal). Cinq indices tirés de la seule marge de B : $f_1 = \ln m$; f_2 = ln de la plus petite marge des couches plus basses, même position (le conflit déjà vu en profondeur) ; f_3 = ln marge de la position précédente, même couche ; f_4 = part des cellules (couches $\leq \ell$, huit dernières positions) sous 0,01241 (le conflit accumulé dans le temps) ; f_5 = la profondeur. Une régression logistique, dix plis (une fenêtre de test chacun), et une garde qui signale dans chaque fenêtre de test

Table 12.6 : La garde à indices multiples : bascules attrapées pour 10 % des cellules de chaque fenêtre de test [mes.]. IC : 10 000 tirages des dix fenêtres (graine 20261026). Refait à l'unité par le juge.

Modèle	attrapées	gain (points)	IC à 95 %	fenêtres positives
M0, la marge seule	56,99 % (2 899)	—	—	—
M1, f_1 seul	56,99 %	0	—	—
M2, profondeur (f_1, f_5, $f_1 f_5$)	57,26 %	+0,28	−0,74 à +1,17	6/10
M3, tous les indices	**58,60 %** (2 981)	**+1,61**	+0,99 à +2,34	9/10

Données : `mesures/pistes/tropicale/indices/indices-resume.json` ; juge : `mesures/pistes/tropicale/juge/juge-indices.json`.

les cellules de plus fort score. Seuil : utile si le gain sur la marge seule est d'au moins 5 points, avec un intervalle à 95 % qui exclut 0 et un gain positif dans 8 fenêtres sur 10. Prédiction écrite : +1 à +6 points, et 80 % hors de portée.

Deux dérivations. *L'ajustement.* Pour des indices centrés réduits z_i (avec 1 en tête) et des étiquettes $y_i \in \{0, 1\}$, on minimise

$$J(w) = -\sum_i \big[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\big] + \frac{\lambda}{2}\|w_{1:}\|^2, \qquad p_i = \sigma(z_i^\top w), \tag{12.6}$$

avec $\lambda = 1$ et l'ordonnée à l'origine non pénalisée. Le gradient est $g = Z^\top(p - y) + \Lambda w$, la hessienne $H = Z^\top \operatorname{diag}(p(1-p))\, Z + \Lambda$, définie positive pour $\lambda > 0$ hors ordonnée ; le pas de Newton est $w \leftarrow w - H^{-1} g$, arrêté quand $\max|\Delta w| < 10^{-9}$. Le score de la garde est le logit $z_i^\top w$, monotone en p_i : le classement ne dépend pas de la fonction σ. *Le coût fixé.* Dans chaque fenêtre de test ($24 \times 511 = 12\,264$ cellules), la garde signale exactement $K = \lfloor 0{,}10 \times 12\,264 \rfloor = 1\,226$ cellules ; la part attrapée est $b = \sum_w (\text{bascules signalées dans } w)/5\,087$. Chaque modèle coûte la même chose et ne se distingue que par son tri.

Verdict au seuil déclaré : pas utile (tableau 12.6). Le gain est réel (l'intervalle exclut 0, 9 fenêtres sur 10) mais sous les 5 points, et la garde reste très loin de 80 %. Le second chemin recompte les scores écrits en Python pur, égaux à l'unité dans les dix plis, et une autre classe de modèle (probabilités par classes, sans régression) rend 59,05 % (+2,06). Le juge refait l'ajustement lui-même depuis la seule déclaration, et retrouve 2 981 et l'intervalle au millième ; il note que le second chemin de la piste éprouvait le comptage, pas l'ajustement. Robustesse : avec $K = 1\,227$, +1,612 ; sans pénalité, +1,59 [mes.].

D'où vient le gain. Les ablations, exploratoires et lues après le verdict, mettent tout le gain sur f_2, le conflit déjà vu plus bas sur la même position : f_1, $f_5 + f_2$ rend +1,65 (IC +0,77 à +2,60) ; avec f_3 ou f_4 seuls, +0,33 et +0,45, d'intervalles qui contiennent 0 ; M3 sans f_2, +0,43 [mes.] (`indices/complements.json`). L'analogie tient donc à moitié : la mémoire du conflit le long de la *profondeur* aide un peu ; celle le long du *temps* n'apporte ~~rien~~ aucun gain détectable : le juge corrige la formulation, car les intervalles de f_3 et f_4 montent jusqu'à +1,25 et +1,43 point, et une contribution de cet ordre n'est pas exclue.

Les témoins, et trois attendus faux. L'identité M1 = M0 tient pli par pli ; f_2, f_3, f_4 permutés dans chaque couche ramènent le gain à +0,33 (le juge : +0,35). Trois attendus déclarés étaient faux, et la

piste en a dit deux. (1) La cassure de M1 par étiquettes mélangées devait rendre 10 ± 2 %; elle rend ~~50,6 %~~ : un modèle à une seule entrée classe les cellules par $\pm f_1$ quel que soit l'ajustement, seul le signe du coefficient change; la bonne cassure permute les entrées du test (9,97 %). (2) L'oracle « bascule vérifiée en $(\ell - 1, p)$ » devait gagner plus de 15 points; il en gagne 10,99, parce qu'il ne concerne que 3,8 % des cellules et 24,5 % des bascules. (3) Le juge ajoute le troisième : M3 sur étiquettes mélangées, déclaré à 10 ± 2 % et publié à 11,6 %, rend 15,9 % en moyenne sur vingt graines (écart-type 4,8, de 5,3 à 24,5 %), et 3 graines sur 20 seulement tombent dans l'attendu : le 11,6 % était un tirage favorable. Le témoin tombe (58,6 → 16 ± 5 %), mais pas à l'attendu. Aucun de ces attendus ne touche le verdict.

Une phrase à barrer. La piste écrivait que même un oracle ne rapporte qu'environ 11 points à une garde qui juge cellule par cellule : ~~« plafond d'environ 11 points même avec un oracle »~~. Le juge la corrige : l'oracle cellule par cellule du premier verdict (marge nulle aux bascules) attrape 100 % pour 4,2 % des cellules; ce que mesure l'oracle de 10,99 points, c'est que la cascade verticale stricte n'explique qu'un quart des bascules.

12.7 Ce qui pourrait être faux

Ce qui pourrait être faux

- **La perturbation n'est pas celle du mode rapide.** Tout est mesuré sur l'écart entre deux calculs exacts (deux rampes de position, dans l'attention). Le verdict se transporte au mode rapide par le théorème et par l'extrapolation, qui sont des modèles; un mode rapide dont l'erreur se concentrerait aux couches où la garde voit bien (0 à 4) ferait mieux, et le contre-exemple du juge montre qu'une hétérogénéité favorable suffit.
- **Le théorème repose sur une forme choisie après la lecture** (Φ au lieu du 2Φ déclaré) et sur une perturbation gaussienne, fausse dans la queue (kurtosis jusqu'à 134). Il est étiqueté modèle et doit le rester; il n'explique qu'environ un point de l'échec.
- **Les racines sont une lecture choisie après coup** : 85,2 % n'est pas un verdict, et la définition ignore la propagation par le cache KV.
- **La causalité des cascades** n'est pas prouvée par intervention : le rapport ×7,9 est une association.
- **Un seul texte** : dix fenêtres de wikitext; les intervalles disent la variation entre fenêtres d'un même texte, pas entre textes. τ_{10} est le décile du même échantillon, ce qui avantage la garde.
- **La loi de Rényi et les deux notices du cortex** sont citées d'après leurs notices vérifiées, sans relecture des articles; la loi de Rényi est démontrée ici sans l'article (lemme 12.4), mais seulement pour des variables exponentielles indépendantes de même taux, ce que les 32 logits ne sont pas; la mesure tient seule, l'analogie ne prouve rien.
- **La coupe du routeur** passe par l'origine de l'espace des états normalisés, pas par des états réels; ses aires ne sont pas des fréquences, et c'est une figure PNG hors du style commun.
- **La proposition 12.3** corrige une phrase de la note de la piste par un argument de la rédaction (la distance à la frontière d'un polyèdre convexe est le minimum des distances à ses hyperplans); aucun chiffre n'en dépend.
- **Ce que le juge n'a pas refait** : les lois de la marge, la corrélation entre couches, le préchargement, l'extrapolation, les figures, le contrôle croisé K3 d1, et les ablations des indices. Ils ne décident rien, mais ne sont pas éprouvés par un second regard.

Chapitre 13

Les spectres des matrices et la fractale

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, aucune mesure de pente. Stone avait proposé l'image d'une fractale : des poids qui se ressembleraient d'une échelle à l'autre, et qu'on pourrait décrire par peu de paramètres. Les données commitées donnaient seulement, pour l'expert 0 de chaque tenseur, la part d'énergie à sept rangs : un excès modeste sur le hasard pour gate et up, fort pour down (`genome/05-synthese.md` § 3.3). La littérature (Martin et Mahoney) donnait une hypothèse, celle de spectres « à queue lourde » dans les réseaux entraînés, pas la réponse pour gpt-oss ; le pronostic de la synthèse pour la compression était défavorable. **Ce chapitre établit**, par la méthode de Clauset, Shalizi et Newman sur 48 matrices de gpt-oss-20b tirées avant de regarder, que ces matrices *ne suivent pas* une loi de puissance : 0 sur 48 au critère déclaré, 9 sur 48 au critère amendé avant la lecture, quand il en fallait 32. Il pose pourquoi le critère déclaré ne pouvait jamais dire oui (un théorème du juge : la loi de puissance est le bord de la log-normale tronquée), et ce qui est vrai quand même : toutes les matrices ont plus de queue que leur version permutée (48 sur 48), et cette queue, la plus lourde dans down et attn_output de la couche 0, a la forme d'une log-normale. Une phrase de la piste est barrée (« le seul endroit » : onze matrices, pas quatre). **Il reste**, non jugé : le levier de compression (aucun dans les poids, sur l'échantillon), le plafond spectral et sa part au-delà des échelles de rangée (passe inachevée), le test de la règle « $\hat{\alpha} < 2$ » (elle tombe) ; et, non mesurés : les 21 couches non tirées, le 120b, les poids d'origine avant quantification. Sources : `genome/pistes/spectres/spectres.md`, `pour-le-memoire.md`, `seuil.md`, `juge-seuil.md`, `JUGEMENT.md` ; `mesures/pistes/spectres/` (`ajustements.out`, `second-chemin-v2.out`, `temoins.out`, `puissance-critere.out`, `rapport.json`, `fig-*.csv`, `juge/`).

En bref

Une matrice de poids peut se décrire par ses valeurs singulières : combien d'énergie elle met dans chacune de ses directions. Pour une matrice tirée au hasard, cette distribution est connue et bornée, sans queue. Une « fractale », au sens où Stone l'entendait, laisserait une signature précise : une loi de puissance, la même forme à toutes les échelles. La fractale est ici une image, et la question qu'elle pose se mesure. La réponse est non : l'entraînement a bien mis de l'énergie dans quelques directions, partout, mais la forme de cette queue n'est pas une loi de puissance ; là où elle est la plus lourde, elle recopie la loi de quelques rangées géantes, les canaux aberrants, qu'une simple échelle par rangée prend déjà.

13.1 La question et la grandeur

Trois questions : les spectres de gpt-oss suivent-ils une loi de puissance ; avec quels exposants, par rôle et par couche ; et ce spectre ouvre-t-il un levier de compression mesurable, en bits par poids ? Ce chapitre répond aux deux premières, jugées, et rapporte la troisième comme non jugée.

Définition 13.1 (Densité spectrale empirique)**.** Pour $W \in \mathbb{R}^{m\times n}$, $m \le n$ (sinon on transpose), la matrice de corrélation $X = WW^\top/n$ a les valeurs propres $\lambda_i = \sigma_i^2/n$, où les σ_i sont les valeurs singulières de W. La densité spectrale empirique est $\rho_W(\lambda) = \frac{1}{m}\sum_i \delta(\lambda - \lambda_i)$. Une *queue en loi de puissance* est $\rho(\lambda) \propto \lambda^{-\alpha}$ au-delà d'un seuil $\lambda_{\min}$.

L'exposant dépend de la grandeur ajustée : si $\rho(\lambda) \propto \lambda^{-\alpha}$, alors avec $\lambda = \sigma^2$, $\rho(\sigma) = \rho(\lambda)\,|\mathrm{d}\lambda/\mathrm{d}\sigma| \propto \sigma^{-2\alpha}\,\sigma = \sigma^{-(2\alpha-1)}$. On ajuste sur λ, comme Martin et Mahoney [118, 129], qui décrivent dans les réseaux de l'état de l'art une auto-régularisation « à queue lourde » et montrent que des métriques tirées de ces lois jugent la qualité d'un modèle mieux que les normes (résumés lus par la piste le 2026-10-21 ; leurs seuils numériques d'α, dans le corps des articles, ne sont pas cités ici).

L'échantillon, tiré avant de regarder (graine 20261021, `seuil.md` § 1 ; le juge refait le tirage et retrouve les mêmes experts). Couches 0, 12, 23 ; experts [0, 6, 22, 24] (couche 0), [1, 3, 19, 31] (couche 12), [7, 21, 26, 27] (couche 23) ; gate, up et down de chacun (36 matrices $2\,880 \times 2\,880$, MXFP4 déquantifiées en f64 exact), plus attn_q, attn_k, attn_v, attn_output des trois couches (12 matrices Q8_0). En tout 48 matrices et 124 032 valeurs singulières, $36 \times 2\,880 + 3 \times (2\,880 + 512 + 512 + 2\,880)$, sauvées avec celles des 48 matrices permutées et des témoins ; 617 s de calcul sur le processeur [mes.].

13.2 Le témoin du hasard : Marchenko-Pastur

Pour W à entrées indépendantes de variance s^2, quand $m, n \to \infty$ avec $q = m/n \le 1$ fixé, la densité spectrale de $X = WW^\top/n$ tend vers la loi de Marchenko et Pastur [130]

$$\rho_{\mathrm{MP}}(\lambda) = \frac{\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}}{2\pi s^2 q\,\lambda}, \qquad \lambda \in [\lambda_-, \lambda_+], \quad \lambda_\pm = s^2(1 \pm \sqrt{q})^2. \tag{13.1}$$

Pour une matrice carrée ($q = 1$), le support est $[0, 4s^2]$, la densité diverge en $\lambda^{-1/2}$ en 0 et s'annule en racine carrée au bord droit. *Le support est borné* : il n'y a pas de queue, et la plus grande valeur propre ne dépasse λ_+ que de fluctuations d'ordre $n^{-2/3}$. Pour $q = 512/2\,880 = 0{,}178$ (attn_k et attn_v vues par leur côté court), $\lambda_\pm = s^2(1 \pm 0{,}4216)^2$, soit $[0{,}3345\,s^2;\ 2{,}021\,s^2]$ [calc.]. Un ajustement de loi de puissance sur un tel spectre ne doit rien trouver : c'est le témoin T-MP, qui doit tomber.

Le second témoin, *T-perm*, permute les entrées de chaque matrice : il garde la loi marginale des entrées (et leur kurtosis, jusqu'à 2 000 sur down) et détruit toute structure de rang. L'écart entre W et sa permutée est ce que l'entraînement a mis dans les directions.

13.3 L'instrument : Clauset, Shalizi et Newman, pas à pas

La méthode [131] est écrite de zéro par la piste (`outils/pistes/spectres/csn.py`). Les références de journal des articles de CSN, de Marchenko et Pastur et de Vuong, données de mémoire par la piste, ont été vérifiées par la rédaction le 2026-10-29 dans leurs notices Crossref (DOI `10.1137/070710111`, `10.1070/SM1967v001n04ABEH001994`, `10.2307/1912557`) : revue, volume, numéro et première page concordent ; les articles ne sont pas relus.

Table 13.1 : La puissance des deux critères, sur des tirages synthétiques de $n = 2\,880$, avant la lecture des spectres réels [mes.].

20 tirages	critère déclaré	critère amendé « compatible »	$\hat{\alpha}$ médian
Pareto $\alpha = 2{,}5$	0 / 20	**18 / 20**	2,496
Pareto $\alpha = 3{,}5$	0 / 20	**20 / 20**	3,510
exponentielle	0 / 20	0 / 20	—
log-normale $\sigma = 1$	0 / 20	4 / 20	—

Données : `mesures/pistes/spectres/puissance-critere.out` (commit `fdf5929`); `genome/pistes/spectres/spectres.md` § 2.

L'exposant par maximum de vraisemblance. Pour la loi continue $p(x) = \frac{\alpha-1}{x_{\min}}\big(\frac{x}{x_{\min}}\big)^{-\alpha}$, $x \geq x_{\min}$, la log-vraisemblance de n points de la queue est $\ell(\alpha) = n\ln(\alpha-1) - n\ln x_{\min} - \alpha\sum_i \ln(x_i/x_{\min})$. En annulant $\mathrm{d}\ell/\mathrm{d}\alpha = n/(\alpha-1) - \sum_i \ln(x_i/x_{\min})$,

$$\hat{\alpha} = 1 + \frac{n}{\sum_i \ln(x_i/x_{\min})}, \qquad \sigma_{\hat{\alpha}} = \frac{\hat{\alpha}-1}{\sqrt{n}}, \tag{13.2}$$

l'erreur type venant de l'information de Fisher, $\mathrm{d}^2\ell/\mathrm{d}\alpha^2 = -n/(\alpha-1)^2$.

Le seuil par Kolmogorov-Smirnov. Pour chaque candidat $x_{\min}$ (valeurs observées, queue d'au moins 50 points), $\hat{F}(x) = 1 - (x/x_{\min})^{1-\hat{\alpha}}$ et $D = \max_x |S(x) - \hat{F}(x)|$, S la répartition empirique de la queue ; aux points triés $t_{(j)}$, $D = \max\big(\max_j((j+1)/n - \hat{F}(t_{(j)})),\ \max_j(\hat{F}(t_{(j)}) - j/n)\big)$. On retient le $x_{\min}$ de D minimale.

Les concurrentes, tronquées en $x_{\min}$. L'exponentielle, $p(x) = \lambda e^{-\lambda(x-x_{\min})}$, de maximum de vraisemblance $\hat{\lambda} = 1/(\bar{x} - x_{\min})$; la log-normale, $p(x) = \varphi\big((\ln x - \mu)/\sigma\big)/\big(x\sigma\,(1 - \Phi((\ln x_{\min} - \mu)/\sigma))\big)$, sans forme close, maximisée numériquement.

Le rapport de Vuong [132]. Avec $\ell_i = \ln p_{\mathrm{PL}}(x_i) - \ln p_{\mathrm{alt}}(x_i)$ et $R = \sum_i \ell_i$, sous l'hypothèse « les deux lois sont aussi proches de la vraie », $R/(\hat{\sigma}\sqrt{n})$ suit asymptotiquement $\mathcal{N}(0,1)$; $p = \mathrm{erfc}\big(|R|/(\hat{\sigma}\sqrt{2n})\big)$. $R > 0$ avec p petit : la loi de puissance est préférée.

La plausibilité. Des répliques tirent chaque point, avec la probabilité n_{queue}/N, de la loi de puissance ajustée ($x = x_{\min} u^{-1/(\hat{\alpha}-1)}$), sinon parmi les points sous $x_{\min}$; on ré-estime $x_{\min}$ et α, et p_{gof} est la part des répliques dont le D est au moins celui des données (100 répliques).

13.3.1 Le seuil déclaré, et pourquoi il a été amendé avant la lecture

Seuil déclaré (`genome/pistes/spectres/seuil.md`, commit `a171622`, 2026-10-21 23:22:25) : ne dire « loi de puissance » d'une matrice que si la méthode la préfère à l'exponentielle *et* à la log-normale avec $p < 0{,}05$, et des matrices de gpt-oss que si deux tiers de l'échantillon (32 sur 48) le sont. Un essai de l'instrument sur des tirages synthétiques, *avant* tout ajustement de gpt-oss, a montré que ce critère ne pouvait pas dire oui (tableau 13.1).

D'où l'**amendement** (`seuil.md` § 4 bis, commit `bfc1c8c`, 23:25:57, huit minutes avant que les spectres réels n'existent dans le dépôt) : le verdict déclaré est rapporté tel quel, et l'on ajoute le verdict « *compatible loi de puissance* » de CSN (leur « modéré ou mieux ») : $p_{\mathrm{gof}} \geq 0{,}1$, l'exponentielle écartée

($R > 0$, $p < 0{,}05$) et la log-normale non préférée. Il se lit toujours « loi de puissance, ou log-normale de très grand σ, indiscernables ici ». Il n'était retenu que s'il disait « compatible » sur au moins 18 Pareto sur 20 et au plus 2 exponentielles sur 20 : il passe, au bord, avec 18 sur 20 à $\alpha = 2{,}5$. Le juge note deux réserves de forme (`JUGEMENT.md` § 1) : l'amendement cite des p médians de 0,59 et 0,63 contre la log-normale, qui viennent d'un essai sans journal ; le seul journal commité donne 0,61 et 0,69, que le livrable cite.

13.3.2 Le critère déclaré ne peut jamais dire oui : le théorème du juge

La piste expliquait l'absence de puissance par la taille : « sur une étendue observée de quelques unités de y, la courbure qui les sépare est sous le bruit ». Le juge l'a posé, avant calcul, comme un théorème plus fort (`juge-seuil.md`, J1 ; `JUGEMENT.md` § 2).

Théorème 13.2 (La loi de puissance est le bord de la log-normale tronquée)**.** *Sur une queue $\lambda \geq \lambda_{\min}$, posons $y = \ln(\lambda/\lambda_{\min}) \geq 0$. Pour tout échantillon, le rapport de vraisemblance $R = \ell_{\mathrm{PL}}(\hat{\alpha}) - \sup \ell_{\mathrm{LN}}$ de la loi de puissance contre la log-normale tronquée ajustée à son maximum est ≤ 0, à toute taille n. Le critère « $R > 0$ et $p < 0{,}05$ contre la log-normale » est donc impossible.*

Démonstration. En y, la log-normale tronquée de paramètres (μ, σ) a pour densité (le jacobien $1/\lambda$ de la loi en λ s'annule avec $\mathrm{d}\lambda = \lambda\,\mathrm{d}y$)

$$g(y) \propto \exp\!\Big(-\frac{(y + \ln\lambda_{\min} - \mu)^2}{2\sigma^2}\Big) \propto \exp(-c_1 y^2 + b\,y), \qquad c_1 = \frac{1}{2\sigma^2} > 0, \quad b = \frac{\mu - \ln\lambda_{\min}}{\sigma^2},$$

une gaussienne tronquée. La loi de puissance $\rho(\lambda) \propto \lambda^{-\alpha}$ a pour densité en y $(\alpha - 1)\,e^{-(\alpha-1)y}$: c'est le même modèle au bord $c_1 = 0$, $b = -(\alpha - 1)$. La log-vraisemblance $\ell(c_1, b) = \sum_i (-c_1 y_i^2 + b\,y_i) - n \ln Z(c_1, b)$, avec $Z = \int_0^\infty e^{-c_1 y^2 + by}\,\mathrm{d}y = \frac{1}{2}\sqrt{\pi/c_1}\ \mathrm{erfcx}\big(-b/(2\sqrt{c_1})\big)$, est continue jusqu'au bord ($Z \to 1/|b|$ quand $c_1 \to 0^+$, $b < 0$) et concave (famille exponentielle de statistiques $-y^2$, y). Donc $\sup_{c_1 > 0} \ell \geq \ell(0, -(\hat{\alpha} - 1)) = \ell_{\mathrm{PL}}(\hat{\alpha})$, et $R \leq 0$. Les jacobiens, communs aux deux lois, s'annulent dans R. ∎

Vérifié sur les 141 spectres du juge (48 réels, 48 permutés, 45 témoins) : $R_{\text{log-normale}} \leq 2{,}7 \times 10^{-13}$, le zéro de la machine [mes.]. La vraisemblance log-normale atteint le bord ($\sigma = \infty$) sur 11 des 48 matrices réelles et sur la plupart des Pareto vrais. Corollaire : les *treize* valeurs $R_{\text{log-normale}} > 0$ du tableau de la piste (jusqu'à +0,096 sur L12.down.E19) sont des optima numériques inachevés de la paramétrisation (μ, σ), qui part vers $\mu \to -\infty$ sans l'atteindre ; aucun verdict n'en dépend ($p \geq 0{,}34$ partout là), et c'est aussi la vraie raison de l'écart de p contre la log-normale entre les deux chemins de la piste. Le verdict déclaré n'est donc pas « sans puissance à $n = 2\,880$ » : il est vide par construction.

13.3.3 Les témoins, la cassure, le second chemin

La **cassure** échange les entrées de T-Pareto et de T-MP carré : sous le critère amendé, les deux tombent (T-Pareto non compatible, $\hat{\alpha} = 16{,}9$; T-MP compatible, $\hat{\alpha} = 2{,}49$) : l'instrument sait dire oui et non, et c'est la donnée qui décide (tableau 13.2). Sous le critère déclaré, la cassure ne fait tomber que T-Pareto : un verdict qui ne dit jamais oui ne peut pas faire échouer un témoin négatif, seconde preuve, par l'instrument, de son absence de puissance. Une remarque sert à lire la suite : le hasard pur rend $\hat{\alpha} = 9$ à 17, parce que la chute d'un spectre à support borné, vue par le haut, ressemble à une pente très raide. *Un $\hat{\alpha}$ petit veut dire une queue lourde ; un $\hat{\alpha}$ de 10 et plus veut dire « pas de queue ».*

Table 13.2 : Les témoins de l'instrument, sous les deux critères [mes.].

Témoin	attendu	critère déclaré	critère amendé	$\hat{\alpha}$
T-Pareto ($\alpha = 2{,}5$ planté, passé par une SVD)	reconnu, $\alpha \in [2{,}3; 2{,}7]$	échec (prévu)	**tient**	$2{,}492 \pm 0{,}032$
T-MP $2\,880 \times 2\,880$	pas loi de puissance	tient	**tient**	16,9
T-MP, trois autres formes	pas loi de puissance	tient	**tient**	14,9 ; 15,8 ; 8,8
T-exponentielle	pas loi de puissance	tient	**tient**	5,75
T-log-normale ($\sigma = 1$)	rapporté	non départagée	non compatible	3,03

Données : `mesures/pistes/spectres/temoins.out`, `temoins-casse.out` ; `genome/pistes/spectres/spectres.md` § 2.

Table 13.3 : Le verdict par famille, 48 matrices de gpt-oss-20b [mes.]. Refait à l'identique par le juge, avec les mêmes neuf matrices compatibles (accord 48 / 48).

Famille	n	critère déclaré	critère **amendé**	expon. écartée	log-norm. préférée	$p_{\text{gof}} \geq 0{,}1$	permuté compatible
gate	12	0	**4**	4	0	12	0
up	12	0	**0**	0	3	6	0
down	12	0	**3**	7	4	5	0
attention	12	0	**2**	5	2	9	1
tout	**48**	**0**	**9**	16	9	32	1

Données : `mesures/pistes/spectres/ajustements.out`, `rapport.json` ; juge : `mesures/pistes/spectres/juge/juge-csn.csv`, `comparaison-juge-csn.txt`.

Le second chemin (`second-chemin-v2.out`) refait la déquantification depuis la spécification (E2M1 décodé bit à bit, échelle E8M0 $= 2^{e-127}$), les valeurs propres par la matrice de Gram, le balayage de $x_{\min}$ et α autrement : 48 empreintes SHA-256 égales sur 48, valeurs propres de la queue à $3{,}8 \times 10^{-14}$, $\lambda_{\min}$ égaux, α à $2{,}2 \times 10^{-7}$, verdicts égaux [mes.]. Sa première version était fausse : ~~$\hat{\alpha}$ de 1,1 à 2,5 au lieu de 1,6 à 6,9~~ ; sa vraisemblance portait $-\alpha \sum \ln t_i$ au lieu de $-\alpha \sum \ln(t_i/\lambda_{\min})$, et ses empreintes différaient sur le signe des zéros (le décodeur bit à bit rend $-0{,}0$ pour le code 8, la table $+0{,}0$: le codon synonyme du projet, retrouvé ici par accident).

13.4 Le verdict : pas une loi de puissance

Au seuil déclaré : 0 matrice sur 48 au critère déclaré, 9 sur 48 au critère amendé ; il en fallait 32. *On ne peut pas écrire que les matrices de gpt-oss-20b suivent une loi de puissance* : ni sous le critère déclaré, qui ne peut rien dire, ni sous le critère amendé, qui pouvait dire oui et dit non (tableau 13.3). Aucune famille n'atteint ses deux tiers ; gate est la plus proche (4 sur 12). Le juge refait tout par un chemin propre (KS bilatérale exacte, log-normale en paramètres naturels partant des moments, répliques d'une autre graine) : 0 / 48, 9 / 48, les mêmes neuf matrices, $\lambda_{\min}$ et $\hat{\alpha}$ égaux sur 48 / 48. Le nombre 9 dépend un peu de ses bords (quatre compatibles écartent l'exponentielle de justesse, trois non-compatibles la manquent de peu) ; même en faisant basculer tous ces cas dans le sens favorable, on atteint 12, loin de 32. Avec un critère plus lâche encore (l'exponentielle écartée, rien d'autre), 16 sur 48 : le verdict ne dépend pas du choix du critère.

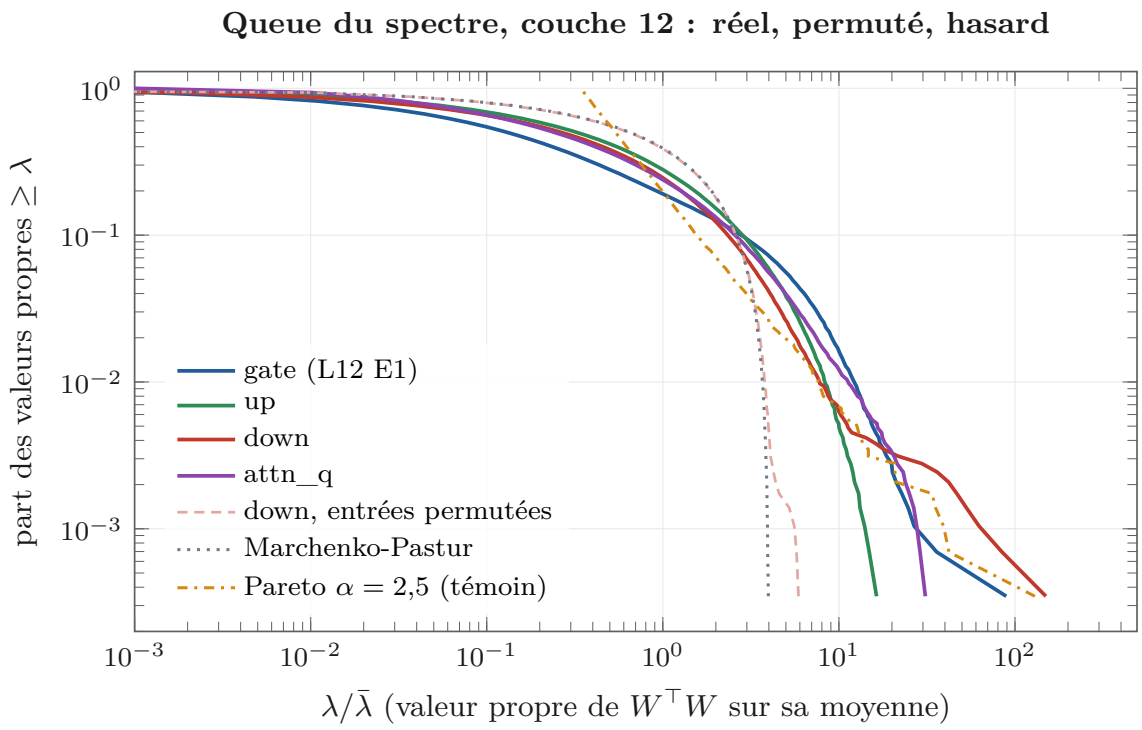


Figure 13.1 : Toutes les familles ont plus de queue que le hasard, mais aucune ne trace la droite d'une loi de puissance. [mes.] Part des valeurs propres $\geq \lambda$ contre $\lambda/\bar{\lambda}$, en log-log, pour une matrice de chaque famille à la couche 12 (gate, up, down de l'expert 1 ; attn_q), la même down aux entrées permutées, la loi de Marchenko-Pastur (chute verticale au bord de son support) et le témoin de Pareto $\alpha = 2{,}5$ (une droite de pente $-(\alpha - 1)$). Page 1 des figures de la piste, jugée. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/pistes/spectres/fig-ccdf-*.csv` ; `genome/pistes/spectres/figures/spectres-figures.tex`, page 1.

Table 13.4 : Les exposants $\hat{\alpha}$ (sur $\lambda = \sigma^2$), **médianes par couche** ; entre crochets, les quatre matrices dans l'ordre du tirage, en gras les compatibles ; dernière ligne, les médianes des matrices permutées [mes.]. Les $\hat{\alpha}$ individuels vont de 2,70 à 5,95 pour gate et de 3,50 à 8,32 pour up.

	couche 0	couche 12	couche 23
gate	3,75 [4,15 ; 3,35 ; 5,95 ; **2,98**]	3,96 [3,67 ; 3,78 ; 4,54 ; 4,13]	3,02 [**3,04** ; 3,00 ; **2,70** ; **3,21**]
up	5,84 [6,24 ; 5,43 ; 6,94 ; 3,96]	4,33 [3,83 ; 4,83 ; 3,50 ; 8,32]	3,70 [3,75 ; 3,64 ; 3,57 ; 4,12]
down	**1,63** [1,65 ; 1,61 ; 1,71 ; 1,61]	3,09 [**2,72** ; 3,18 ; **2,99** ; 3,60]	6,44 [6,06 ; 6,83 ; **2,74** ; 7,25]
attention (q, k, v, o)	6,20 ; 4,23 ; 6,04 ; 1,72	2,87 ; 2,95 ; 5,12 ; **2,89**	**1,90** ; 1,50 ; 1,71 ; 4,05
permuté (gate ; up ; down ; att.)	15,4 ; 17,2 ; 7,9 ; 8,9	12,5 ; 17,3 ; 5,4 ; 7,9	14,9 ; 13,5 ; 16,3 ; 8,7

Données : `mesures/pistes/spectres/rapport.json` (champ `alpha`) ; `genome/pistes/spectres/spectres.md` § 4.1 ; juge : $\hat{\alpha}$ égaux sur les 48 matrices et les 48 permutées.

Ce que sont les 39 non-compatibles (figure 13.1). Le plus souvent une *queue trop courte pour trancher* : la queue retenue par KS ne garde que 50 à 120 valeurs propres pour 18 matrices, quelques directions en pointe au-dessus d'une masse quasi aléatoire, et l'exponentielle n'est écartée que pour 16 matrices sur 48. Pour up, une *queue plus légère* : l'exponentielle ou une log-normale de petit σ (0,17 à 0,85 pour 10 sur 12) est préférée ou équivalente. Pour down et attn_output de la couche 0, une *queue lourde mais courbée* : $\hat{\alpha} \approx 1{,}6$ à 1,7 sur 565 à 973 valeurs propres, mais la log-normale est préférée pour les cinq ($p \leq 0{,}0044$; $\sigma = 1{,}8$ à 3,0), et la loi de puissance n'est même pas plausible pour les quatre down ($p_{\text{gof}} = 0$).

13.4.1 Les exposants, par rôle et par couche

Les erreurs types vont de 0,02 (down, couche 0) à 0,99 (up, couche 12, E31, sur 55 points) [mes.] (tableau 13.4 et figure 13.2). La piste rendait « gate 3,0 à 4,0 ; up 3,7 à 5,8 » : ce sont les médianes par couche, pas des étendues, et le juge demande de l'écrire ainsi.

Ce qu'ils disent. (1) *Toutes les matrices ont plus de queue que le hasard* : le $\hat{\alpha}$ réel est sous celui de la même matrice permutée dans les 48 cas (rapport permuté sur réel de 1,26 à 8,81, médiane 3,49, refait par le juge) ; l'entraînement a mis de l'énergie dans quelques directions (la part des 5 % premières directions est 1,25 à 5,3 fois celle du hasard). C'est le seul résultat « positif » de la piste, et il est large. (2) *up est la plus proche du hasard* ($\hat{\alpha}$ 3,5 à 8,3, aucune compatible) ; gate est intermédiaire et se durcit avec la profondeur (3 compatibles sur 4 à la couche 23). Une lecture, non mesurée : gate et up reçoivent la même entrée, mais seule la sortie de gate passe la non-linéarité qui décide quel canal s'ouvre. (3) *down et attn_output de la couche 0, et q, k, v de la couche 23, ont des queues très lourdes* ($\hat{\alpha} = 1{,}5$ à 1,9 : sous 2, la loi ajustée n'a même pas de moyenne finie) ; ce sont aussi les matrices dont les entrées ont un kurtosis énorme (down de la couche 0 : 60 à 1 988). Les matrices qui écrivent dans le flux résiduel à la couche 0 sont celles qui fabriquent ses canaux aberrants. (4) *Cette queue lourde n'est pas une loi de puissance* : pour down de la couche 0, la log-normale est préférée et la loi de puissance rejetée. (5) La piste écrivait : ~~« La couche 12 de down est le seul endroit où la loi de puissance et la log-normale se confondent vraiment »~~. C'est faux (JUGEMENT.md § 4.1) : la log-normale ajustée *est* la loi de puissance, au bord $c_1 = 0$ du théorème 13.2, sur onze matrices : les quatre down de la couche 12, mais aussi L00.gate.E22, L00.attn_k, L12.gate.E01, L12.attn_output, L23.down.E07, E21 et E27 ; le

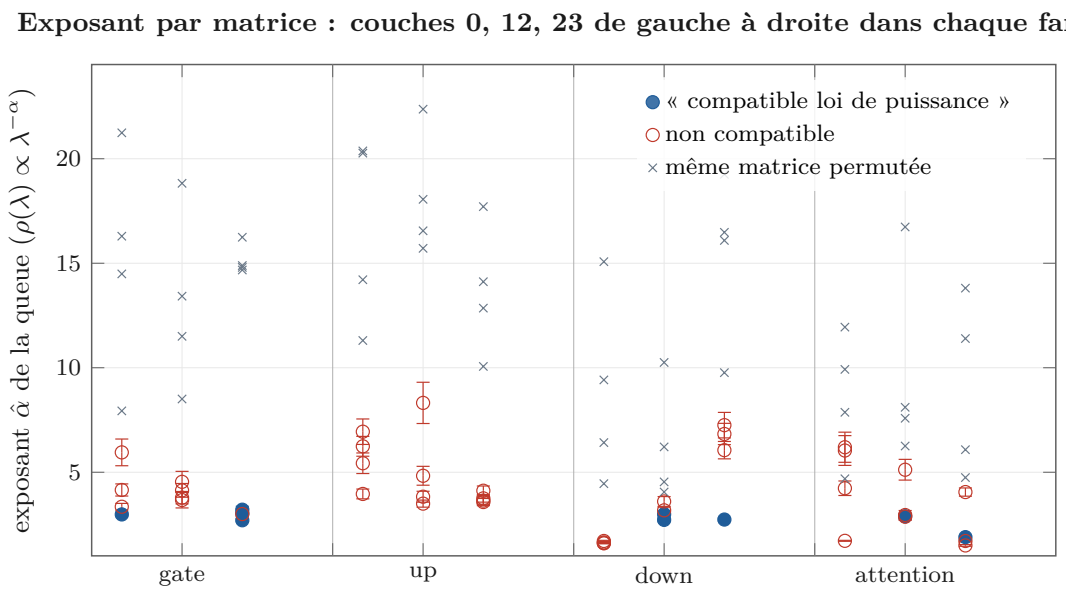


Figure 13.2 : L'exposant réel est sous celui de la même matrice permutée dans les 48 cas ; seules neuf matrices sont compatibles avec une loi de puissance. [mes.] $\hat{\alpha}$ par matrice et son erreur type $(\hat{\alpha}-1)/\sqrt{n_q}$, par famille, couches 0, 12, 23 de gauche à droite dans chaque famille ; disques pleins : compatibles ; cercles : non compatibles ; croix : la même matrice aux entrées permutées. Page 2 des figures de la piste, jugée. (figure hors du style commun du mémoire, incluse telle que sa piste l'a tracée)

Données : `mesures/pistes/spectres/fig-alpha.csv` ; `genome/pistes/spectres/figures/spectres-figures.tex`, page 2.

tableau publié le montrait déjà ($\mu = -69$ à -118 pour ces sept-là).

Rapport à Martin et Mahoney. Ce qui est mesuré ici s'accorde avec la première moitié de leur description (des queues plus lourdes que le hasard dans 48 matrices sur 48, un excès qui varie fortement par rôle et par couche) et ne confirme pas la seconde au sens strict : au test de CSN, la queue n'est une loi de puissance plausible que dans 9 matrices sur 48, et la plus lourde de toutes est une log-normale. Une différence de méthode (ils ajustent souvent la densité de $W^\top W/N$ avec la bibliothèque `powerlaw`) peut déplacer α d'une fraction d'unité ; elle ne change pas la forme.

13.5 Le levier et le plafond : ce qui n'est pas jugé

Le juge de la piste n'a jugé ni le levier, ni le plafond spectral, ni le plafond par rang, ni le test de prédiction, ni la mesure exploratoire de normalisation, ni la passe du plafond par rangée, qui tournait encore (`JUGEMENT.md` § 5). Cette section en pose les dérivations, qui ne dépendent d'aucune mesure, et en rapporte les nombres avec l'étiquette [non jugé] ; leurs figures (pages 3 à 5 du fichier de la piste) ne sont pas placées. Tous ces gains sont *dans le domaine des poids*, et la synthèse de la ronde a montré que l'erreur de poids ne prédit pas la KL : un levier ici serait une condition nécessaire, jamais suffisante.

Le plafond spectral. On lit W comme $\max(m,n)$ vecteurs gaussiens de dimension $p = \min(m,n)$, de covariance de valeurs propres λ_j. À erreur relative D, le remplissage inverse dans la base propre (la transformée de Karhunen-Loève) donne $D_j = \min(\theta, \lambda_j)$, $\sum_j D_j = D \sum_j \lambda_j$, et le débit $R_{\mathrm{KLT}} = \frac{1}{p}\sum_j \max\!\big(0, \frac{1}{2}\log_2(\lambda_j/\theta)\big)$ bits par poids [litt.] (Cover et Thomas [2] ; le passage est cité d'après la note de la piste, non relu ici). Le spectre plat de même énergie coûte $R_{\mathrm{blanc}} = \frac{1}{2}\log_2(1/D)$, soit 2,2535 bits à $D = 0{,}044$. Le plafond $G = R_{\mathrm{blanc}} - R_{\mathrm{KLT}}$ majore le gain de toute méthode qui code dans une base, sous ce modèle : coder coordonnée par coordonnée avec ses propres variances est permis, et ne fait pas mieux que la transformée (à haut débit, c'est l'inégalité de Hadamard, $\det C \le \prod_i c_{ii}$), qui elle-même ne fait pas pire que la base blanche (inégalité des moyennes). À haut débit, si $\theta < \lambda_{\min}$,

$$G = \frac{1}{2}\log_2 \frac{\bar{\lambda}}{\lambda_g}, \tag{13.3}$$

le logarithme du rapport des moyennes arithmétique et géométrique du spectre, une mesure de platitude où les petites valeurs pèsent autant que les grandes. Deux formes closes s'en tirent [calc.] : pour Marchenko-Pastur de rapport 1, $\mathbb{E}\ln\lambda = -1$ et $G = \frac{1}{2}\log_2 e = 0{,}7213$ bit ; pour un Pareto ($\lambda \ge 1$, $\alpha > 2$), $\bar{\lambda} = (\alpha-1)/(\alpha-2)$, $\mathbb{E}\ln\lambda = 1/(\alpha-1)$, et

$$G_{\mathrm{Pareto}}(\alpha) = \frac{1}{2}\log_2 \frac{\alpha-1}{\alpha-2} - \frac{1}{2(\alpha-1)\ln 2}, \tag{13.4}$$

qui vaut $0{,}7925 - 0{,}4809 = 0{,}3116$ à $\alpha = 2{,}5$, *moins* que le hasard carré (0,5899 à $D = 0{,}044$). Une queue lourde en haut ne fait pas, à elle seule, un grand plafond ; c'est pourquoi le premier témoin positif de la piste, ~~« Pareto au-dessus de MP »~~, est tombé, attente fausse remplacée avant toute lecture de gpt-oss par un rang 8 planté (prédit $\approx 0{,}24$, mesuré 0,216), et pourquoi l'excès se lit contre la permutée de la même matrice. Sur l'échantillon, l'excès pondéré vaut 0,424 bit par poids [non jugé], le plus grand dans down et attn_output de la couche 0 et q, k, v de la couche 23 : exactement les matrices de $\hat{\alpha} < 2$.

Ce qu'il en reste une fois la base payée. Offrir au décodeur les r premières directions coûte b bits par coefficient de facteur, $b\,r\,(m+n)/(mn)$ bits par poids, soit $0{,}0111\,r$ à $b = 16$ pour une matrice

carrée [calc.] ; une base complète coûte au moins b bits par poids, pour un plafond qui ne dépasse pas 1,8 bit sur les 48 matrices : elle ne se paie jamais. Les deux tiers du plafond vivent au-delà de la 64e direction, et à $b = 16$ aucune famille n'a de levier au seuil de 0,05 bit (down : 0,042) [non jugé]. Le levier mesuré sur les matrices elles-mêmes, sans perte sur les codes MXFP4 par l'information mutuelle conditionnelle $I(C; B_r \mid E)$ et avec perte par une borne gaussienne par rangée et un quantificateur réel, ne vaut sur l'échantillon que 0,0009, 0,0036 et 0,0007 bit par poids [non jugé]. Le quantificateur réel demande un tramage soustractif : sur une source déjà discrète, un quantificateur uniforme sans tramage mesure la grille et non la structure (le témoin permuté rendait le même gain), alors qu'avec un tramage $u \sim U(-\frac{1}{2}, \frac{1}{2})$ connu du décodeur, l'erreur devient uniforme et indépendante du signal quelle que soit sa loi (résultats de Schuchman et de Zamir et Feder, cités d'après la note de la piste, non relus), d'énergie $N\Delta^2/12$ exactement.

Où est passée la structure. Le plafond mêle deux choses : l'inégalité des variances par rangée, que prend n'importe quel codeur avec une échelle par rangée, et la structure entre rangées, que seule une base prend. La chaîne $R_{\mathrm{KLT}} \le R_{\text{rangée}} \le R_{\text{blanc}}$ les sépare [calc.]. Pour L00.down.E24, $R_{\text{rangée}} = 0{,}755$ bit au lieu de 2,254 : les échelles seules prennent 1,50 des 1,74 bit du plafond. MXFP4 a mieux encore, une échelle par bloc de 32 poids. Une mesure exploratoire, non déclarée et non jugée, le confirme pour la queue : normaliser les rangées de down et attn_output de la couche 0 fait passer leur $\hat{\alpha}$ de 1,6–1,7 à 3,4–7,0, normaliser les colonnes ne le change pas ; leur plus grande valeur singulière n'est que 1,1 à 1,4 fois leur plus grande norme de rangée. La queue la plus lourde est la loi de quelques rangées géantes, recopiée. Enfin, la règle « $\hat{\alpha} < 2$ désigne les matrices où un rang faible se paie », vue avant d'être déclarée, a été testée sur 36 matrices neuves (couches 3, 9 et 21) : 4 sur 6 sous 2, mais 5 sur 30 au-dessus pour 10 % permis : elle tombe [non jugé]. L'idée de la fractale ne donne ici ni une loi, ni un sélecteur.

13.6 Ce qui pourrait être faux

Ce qui pourrait être faux

- **L'échantillon** : 48 matrices sur les 2 400 du modèle (2 %), 3 couches sur 24, 4 experts sur 32. Le verdict tient pour cet échantillon tiré au hasard ; une couche non tirée pourrait différer, et les pentes avec la profondeur sont lues sur trois points.
- **La grandeur** : les spectres sont ceux des poids déquantifiés (MXFP4, Q8_0), pas des poids d'origine en BF16, non téléchargés. La quantification ajoute un bruit à peu près blanc qui relève le bas du spectre et peut raccourcir la queue apparente ; son effet sur α n'est pas mesuré.
- **La garde de 50 points** sur la queue est un choix déclaré, pas une dérivation ; cinq matrices ont leur $\lambda_{\min}$ à la garde ou tout près, et pour elles rien n'est décidable.
- **Le critère amendé** est posé après avoir vu que le critère déclaré n'avait aucune puissance, mais avant toute lecture des spectres réels ; il passe son épreuve de puissance au bord (18 / 20). Le verdict ne dépend pas de ce choix (16 / 48 au plus lâche).
- **Le permuté est une seule permutation par matrice** ; à l'écart mesuré (rapport $\ge 1{,}26$), une autre graine ne renverserait pas « 48 / 48 », mais ce n'est pas mesuré.
- **Les références** : Martin et Mahoney ne sont connus que par leurs résumés ; les notices de CSN, Marchenko-Pastur et Vuong sont vérifiées, pas les articles ; Cover et Thomas, Schuchman, Zamir et Feder, Gray et Markel sont cités d'après la note de la piste, sans vérification.
- **Tout ce qui est étiqueté « non jugé »** (levier, plafonds, test de la règle, normalisation) n'a

pas eu de second regard ; la piste n'est pas close (passe du plafond par rangée inachevée au jugement, résumé du livrable non écrit).
- **Les figures** sont celles de la piste, hors du style commun ; seules les deux pages jugées sont placées.

Chapitre 14

Discussion

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, douze chapitres, chacun fermé sur sa propre section « Ce qui pourrait être faux », et chacun jugé par un juge qui n'avait pas fait le travail. **Ce chapitre établit** ce que le tome dit ensemble : les verdicts, chacun avec son juge et le nombre qui le décide ; les chiffres barrés, rassemblés ; la leçon de la quête ; ce qui pourrait être faux à l'échelle du tome ; et les travaux qui viennent, par ordre de gain attendu, chiffrés quand ils le sont. **Il reste** à les faire.

14.1 Ce que le tome établit, ensemble

Tableau 14.1 rassemble les verdicts. Un verdict est celui du critère déclaré avant la mesure, tel que le juge de l'étape l'a refait ; « ouvert » veut dire qu'aucun des deux seuils, succès ni mort, n'est franchi, et le chapitre dit pourquoi.

Table 14.1 : Les verdicts du Tome III, chacun avec le nombre qui le décide et le chapitre qui le porte.

Mécanisme	Verdict	Le nombre qui le dit	Où
Gagnent (les synonymes)			
codon synonyme du zéro	gagne, sans perte	le signe d'un zéro a une entropie de 0,9999999967 bit et le calcul ne le lit jamais ; 0,127 91 et 0,125 62 bit par poids d'expert ; logits identiques au bit (juge : 0 sur 5 120, deux modèles)	chapitres 3, 9 et 11
contexte de la colonne	gagne, sans perte, à la couche 0	$-0{,}142$ et $-0{,}123$ bit à la couche 0 (gate et up, contre le flux octet) ; sous 0,01 bit ailleurs	chapitre 8
mutation synonyme ($\lambda = 0$, 768 experts)	gagne en KL	0,131 8 contre 0,264 0 (R1) ; $-0{,}156\,5 \pm 0{,}033\,4$ contre la frontière hors de R1 (juge, 4,7 σ)	chapitres 10 et 11
$\lambda = 0$ contre $\lambda = 1$	$\lambda = 0$ retenue	2,09 σ sur R1, 3,6 σ hors de R1	chapitre 10
Égaux			
partitions par κ	égales en KL, moins de bits	appariée $+0{,}006\,6 \pm 0{,}005\,3$; 0,911 fois les bits	chapitre 8
S^* (le brin et ses exceptions)	gagne en sortie d'expert, égal en KL	rapport d'erreur 0,747 ; KL appariée $+0{,}000\,4 \pm 0{,}001\,9$ (dix fenêtres ; R4)	chapitres 4 et 8
Tombent			
graine et coefficients	tombe	3,06 bits à l'erreur de la base	chapitre 4
motifs partagés	tombe	2,90 bits	chapitre 4
recette et complément exact	tombe	4,05 à 4,37 bits contre 3,84	chapitre 4
duplication (le parent)	tombe	0,003 4 bit, dont 0,001 6 au hasard	chapitre 5
hebbien, élagage, partitions par e	tombent	hebbien : KL ×3,06 la base pour une erreur d'expert ×1,02 ; élagage par magnitude : ×1,012 les bits de la base à erreur égale	chapitres 6 et 8

Mécanisme	Verdict	Le nombre qui le dit	Où
mode rapide, gain de génération	ne tient pas	×1,28 (120b) et ×0,94 (20b) contre le v1.2 (juge)	chapitres 10 et 11
garde tropicale	tombe	57,1 % des bascules pour 10 % des cellules ; 80 % coûterait 26,6 % (juge)	chapitre 12
loi de puissance des spectres	ne s'écrit pas	0 matrice sur 48 au critère principal, 9 « compatibles » au critère amendé, il en fallait 32 (juge)	chapitre 13
Ouverts			
P1, mémoire du cache à deux échelles	ouvert	le « tombe » du protocole est décidé par des jetons produits après la fin du message (juge)	chapitre 7
P2, cache exclusif	ouvert	un étage de RAM inclusif plus petit que la VRAM ne sert à rien	chapitre 7
P3, routage conditionné par le cache	à la limite	environ 10 % des défauts retirés pour une KL de 0,031 4 au routeur en ligne ($\beta_1 = 0{,}041$), 0,020 9 et 0,023 9 à l'étage B ($\beta = 0{,}06$ et 0,07) (ce qui est jugé ; réserve R6)	chapitre 7
P4, préchargement à confiance calibrée	ouvert, plutôt tombe	simulateur non validé	chapitre 7
P5, recuit et échange de répliques	ouvert	médiane 5,09 % de gain tenu à l'écart ; le mécanisme annoncé ne tient pas au seuil (juge)	chapitre 7
P6, fidélité décidée par appel	ouvert ; B tient à l'étape 0	un « tient » qui ne pouvait presque pas tomber (juge)	chapitre 7

Ce que cela fait, pour la machine. Des trois synonymes, deux sont construits : le codon du zéro, dans le format, et la mutation synonyme, dans le mode rapide ; le contexte de la colonne, qui ne rend qu'à la couche 0, reste hors du format v1.2 (section 9.4.1). Sans perte, le format v1.2 fait lire 3,3 % d'octets de moins par jeton au 120b, et le J1 y gagne ×1,44 sur ce disque, parce que le motif de ses requêtes de lecture est rapide là où celui du v1.0 ne l'est pas (réserve R1). Avec perte, le mode rapide sert le 20b avec une KL de 0,13 pour 2,67 bits par poids d'expert au lieu de 3,86, mais ce gain d'octets ne devient pas un gain de génération : le routage d'un modèle avec perte manque plus souvent la VRAM, et les octets par pas restent les mêmes. Le gain de vitesse que la quête a trouvé n'est pas un mécanisme du génome : c'est le format, lu sur ce disque.

14.2 Les chiffres barrés

Quand une mesure a contredit un chiffre publié, la phrase a été barrée là où elle était. Tableau 14.2 les rassemble ; c'est la trace que la méthode a fonctionné.

14.3 La leçon de la quête

La synthèse de la ronde l'a dite en une phrase par niveau, et la construction l'a confirmée.

- **Dans le domaine des poids, il n'y a rien à générer.** Le gain se prend sur ce que le calcul ne lit pas (le signe d'un zéro) ou sur ce que le codeur sait déjà (l'échelle, la colonne). La recette, la graine, la parenté supposent une dépendance entre poids que ces poids, une fois l'échelle connue, n'ont pas.
- **Dans le domaine des sorties, la moyenne de l'erreur ne suffit pas.** Ce qui compte est l'endroit où l'erreur tombe : la mutation synonyme la place là où les entrées ne regardent pas, l'hebbien là

Table 14.2 : Les chiffres et les phrases barrés pendant la quête, et ce qui les a remplacés.

Barré	Remplacé par, et pourquoi	Où
~~S* : 33 % de KL de plus~~ (trois fenêtres)	égal à la base sur dix fenêtres, +0,000 4 ± 0,001 9 (R4)	chapitre 8
~~$\lambda = 1$: KL 0,0262~~	la KL 0,0262 était celle de $\lambda = 0$; aucune variante n'était encore sous la frontière en bits et en KL à la fois	chapitre 8
~~768 experts substitués~~ (témoin)	substitués = lus (732 sur 732) : 36 experts ne sont routés par aucune position	chapitre 10
~~20b ×1,24, 120b ×2,00~~ (gains du mode rapide)	×0,94 et ×1,28 contre le v1.2, que l'étage disque lit à la même vitesse (G1, G2)	chapitres 10 et 11
~~cause : le cache de fichiers du système~~	impossible : le moteur lit sans tampon	chapitre 11
~~c'est l'état des fichiers, pas le format~~	le motif des requêtes de lecture du format, propre à ce disque (réserve R1)	chapitre 11
~~avec $\lambda = 0$ la queue baisse aussi, contre $\lambda = 1$~~	non reproduit hors de R1 ; un centile 99,9 de 2 550 positions tient à trois positions (M2)	chapitre 10
~~$\lambda = 0$ déforme dans une direction privilégiée~~	le sens de la perplexité s'inverse hors de R1 (M3)	chapitre 10
~~au plus 4,0 % de cellules qui basculent~~ (garde)	vrai seulement pour une population homogène ; un contre-exemple du juge tient 80/10 avec 5,26 %	chapitre 12
~~seul endroit~~ (spectres)	faux : 11 matrices sur 48	chapitre 13
~~les défauts ne sont pas corrélés aux petits poids~~ (P6)	ils le sont : 35 % de défauts au rang 0, 56 % au rang 3	chapitre 7

où l'énergie est faible ; les deux ont une erreur d'expert basse, seule la première gagne en KL.

– **Dans le domaine du texte, ce qui coûte est la bascule.** Une erreur ne vaut que par sa probabilité de faire franchir une frontière du routeur à une couche suivante ; et une bascule en entraîne d'autres, loin des frontières, que la marge locale ne voit pas (chapitre 12).

Pour la conviction de Stone, la réponse est double, et les deux moitiés sont mesurées. L'ADN comme *recette* ne marche pas sur ces poids : la théorie le prédisait avant la mesure, et la mesure l'a confirmé. L'ADN comme *code redondant*, où un synonyme change la séquence sans changer le phénotype, est exactement ce qui gagne, trois fois. Le vivant avait la bonne idée ; ce n'était pas celle qu'on attendait de lui.

14.4 Ce qui pourrait être faux, à l'échelle du tome

Ce qui pourrait être faux

– **Les échantillons.** La ronde a mesuré sur 12 tenseurs de référence (couches 0, 8, 16, 23 de gpt-oss-20b) et souvent sur 16 ou 32 experts sur 768 ; les spectres sur 48 matrices (2 %) de trois couches ; P5 sur des poids synthétiques. Une propriété vraie de ces échantillons peut ne pas l'être d'un autre.
– **Le texte.** Toutes les KL sont prises sur wiki.test : R1, la calibration et les fenêtres neuves du juge en viennent. Dix fenêtres d'un même corpus, dont les positions sont corrélées (un effet de plan de 32 pour une KL absolue) ; aucun texte de code ni de dialogue. Le mimétisme juge la génération sur deux traces de 256 jetons, dont les juges ont montré qu'elles sont dégénérées.
– **Les protocoles à 32 et à 768 experts** mesurent deux choses différentes (R3) ; une variante jugée sur 32 experts ne se juge pas : $\lambda = 1$ y égale la frontière, et la bat de 42 % à 768.

- **Un seul disque, une seule machine** (réserve R1). Tout chiffre de vitesse du tome vaut pour ce NVMe, derrière Intel RST VMD, sous BitLocker, un jour donné ; le gain du format v1.2 tient au motif de ses requêtes sur ce disque, et la variable exacte du motif n'est pas isolée.
- **Un seul modèle, né en MXFP4.** gpt-oss a des poids d'experts publiés en quatre bits ; le tome ne dit rien d'un modèle publié en BF16, où la frontière et le bruit sont autres.
- **Les analogies.** Le génome, le ribosome, le cortex cingulaire, le verre de spin ont orienté la recherche ; aucun n'a prouvé quoi que ce soit. Un chapitre qui lirait une analogie comme un argument se tromperait, et le tome les dit comme telles.
- **Les chiffres non jugés.** Les pages 3 à 5 des figures des spectres (le levier, le plafond) ne sont pas jugées et ne sont pas placées ; les explorations de la piste tropicale après son verdict (les racines de cascade, la mémoire du conflit) sont présentées comme telles ; le second tirage de P3 n'entre pas dans le tome (R6) ; le proxy de qualité du 120b est validé sur le 20b, pas sur le 120b.
- **Les juges partagent des conventions.** Le juge de la construction a son propre code, mais la même convention de KL (coupe à $\log p > -16$) et le même moteur, construit depuis le même dépôt ; un défaut commun à cette convention ou à ce moteur ne serait pas vu. Le second chemin des logits reste la référence f64 du Tome II.
- **Un juge peut se tromper, et un juge l'a fait** : l'attribution « état du fichier » du juge de la construction est tombée devant le juge du placement. Rien ne garantit qu'une autre lecture ne viendra pas.

14.5 Travaux futurs

Rangés par gain attendu, chacun avec la mesure qui le jugera.

1. Faire du v1.2 le format par défaut, et isoler le motif. Les deux fichiers sont déjà recodés et vérifiés (section 9.6) ; le passage du v1.0 au v1.2 est le seul levier de vitesse mesuré de bout en bout : ×1,30 (20b) à ×1,44 (120b) entre deux fichiers du même modèle sur ce disque, sans perte. Le choix des v1.2 comme fichiers par défaut, et l'effacement des v1.0 (57,7 Go et 11,2 Go), reviennent à Stone. La variable exacte du motif v1.0 lent se cherche à l'étage disque, par des motifs déclarés avant (le juge du placement en propose quatre) ; un levier de lecture ne se mesure de bout en bout qu'une fois trouvé.

2. La queue de la KL, ou les bascules. Elle porte l'essentiel de la moyenne, et aucun mécanisme du tome ne la vise : la mutation synonyme baisse la part lisse (médiane divisée par près de trois) bien plus que la queue (centile 99,9 contre la frontière sur R1 : $-3{,}4\,\%$ pour $\lambda = 1$, $-17{,}2\,\%$ pour $\lambda = 0$) ; la garde tropicale tombe parce que les bascules secondaires arrivent à des marges larges. La suite désignée par les mesures est la cascade : la garde attrape 85,2 % des racines de cascade, lecture exploratoire faite après le verdict, à déclarer comme critère avant toute mesure neuve.

3. La mutation synonyme du 120b. Il lui faut la capture de la référence f64 sur le 120b (36 couches, 128 experts), un codeur sur la carte (le codeur Python demanderait environ 23 h), puis la KL de bout en bout contre une référence f64 du 120b sur les dix fenêtres, qui n'existe pas encore. Prédit, non mesuré : vers 0,07 de KL si le rapport du 20b se transporte.

4. Les usages du mode rapide, refaits avec les bons rapports. La politique mixte des défauts (servir en rapide un expert froid, relire en exact un expert promu) se simule d'abord sur les traces

par le même code que le cache réel, avec $r(R/B)$ et non $r(R/A)$: ×1,28 en génération pour une KL de 0,14 sur le 120b, rien en génération sur le 20b. Le brouillon spéculatif, idée de Stone (« un genre de puissance de raisonnement différent »), rend le texte exact au bit ; mais par le lemme de la synthèse, un brouillon lu du disque n'est rentable que si $\alpha > 1 - \rho + b_d/b_e$, et avec un routage indépendant la réutilisation ρ des experts entre positions voisines est de l'ordre de 0,03 sur le 120b : le seul brouillon résident d'un 120b sur cette machine serait le 20b en mode rapide, dont l'accord avec le 120b n'est pas mesuré (chapitre 8).

5. Un autre corpus. Des fenêtres de code et de dialogue, tenues à l'écart, pour la KL du génome synonyme et pour B2 ; et, pour le mimétisme, des générations du 120b qui s'arrêtent au jeton de fin, la mesure que le juge de P1 demande pour trancher.

6. Ce que la ronde laisse. S^* combiné à la rétroaction par le H complet (la prédiction qui dirait pourquoi S^* ne gagne pas en KL) ; l'expert pont, dont le meilleur paralogue laisse de 33 à 84 % de l'énergie de sortie en erreur ; le second tirage de P3. Aucun n'a de chiffre de gain aujourd'hui.

Conclusion

Le Tome I avait mesuré la machine et démontré que, sur elle, tout se joue en bits lus par poids. Le Tome II avait construit un programme qui sert un modèle quatre fois plus gros que la RAM, exactement. Ce tome a cherché dans le vivant une manière de lire moins de bits, et il en a trouvé trois, toutes de la même famille : un synonyme change le code sans changer ce qui compte. Il a aussi montré, avec la même exigence, ce qui ne marche pas et pourquoi : les poids de gpt-oss, une fois l'échelle connue, sont du bruit ; ce qui coûte au texte est la bascule du routeur ; et le gain de vitesse trouvé en route appartient au format et à ce disque, pas au génome. Chaque verdict a son seuil déclaré avant, son témoin qui sait tomber, et un juge qui n'a pas fait le travail. Ce qui reste est écrit ci-dessus, dans l'ordre où il paierait.

Annexe A

Protocoles : seuils déclarés et témoins

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la méthode des Tomes I et II : chaque mesure a son témoin, et tout témoin neuf doit pouvoir échouer ; un seuil dérive d'une grandeur nommée et se déclare avant la mesure, par un commit daté ; un coût se juge en séries ABBA ; tout chiffre dont on dépend se refait par un second chemin ; une phrase démentie se barre. **Cette annexe établit**, pour le Tome III, la liste des seuils déclarés avec la grandeur dont chacun dérive et son verdict ; la table, engendrée depuis git, de chaque fichier de seuils et de l'heure du commit qui l'a créé ; les témoins et leurs cassures ; les amendements faits avant lecture, et ceux faits après, dits comme tels ; les protocoles des juges. **Il reste** ce que git ne peut pas prouver : qu'aucune lecture exploratoire n'a précédé un seuil (section A.6). Sources : les fichiers de seuils de tableau A.2, les livrables et les jugements des chapitres, `memoire/tome3/JUGEMENT-figures3.md` (§ 4.4), `memoire/outils/tome3_cassures.py`.

A.1 Ce qui est hérité

Les règles ne changent pas d'un tome à l'autre ; elles sont posées au Tome I (annexe des protocoles) et appliquées au programme dans le Tome II (son annexe A). Le Tome III en a une pratique plus dense, parce que la quête du génome a déclaré, du 16 au 27 octobre 2026, trente-cinq fichiers de seuils, de critères et d'amendements (tableau A.2), souvent par des agents qui travaillaient en même temps : la règle « le seuil se déclare avant la passe, et l'heure du commit fait foi » y est la seule qui permette de lire l'ordre des événements. Trois formes y reviennent, que le tome nomme partout de la même façon :

- **succès et mort** : un mécanisme a un seuil de succès et un seuil de mort ; entre les deux, il est « non concluant » ou « ouvert », et le dit (le tri du mimétisme les déclare ainsi pour ses six pistes) ;
- **la règle des différences appariées** : une variante b est meilleure qu'une variante a si $\bar{d} + 2\sigma < 0$, $d = \mathrm{KL}_b - \mathrm{KL}_a$ position par position, σ l'écart-type par fenêtres ; égale si $|\bar{d}| \leq 2\sigma$; pire sinon (règle du lot R, Tome II ; section 11.5.3 pour la raison du σ par fenêtres) ;
- **le gain annoncé** : un rapport de vitesses médian ne s'annonce que s'il dépasse $1 + 2\sigma$, σ le bruit relatif d'un fichier contre lui-même dans la même séance ; le juge de la construction a resserré la règle à σ = max des fichiers de la séance (section 11.6.2).

A.2 Les seuils déclarés, étape par étape

Tableau A.1 donne les critères qui décident d'un verdict du tome, chacun avec la grandeur dont il dérive ; tableau A.2 donne, pour chaque fichier de seuils, de critères ou d'amendement, le commit qui l'a créé. Le verdict est celui du juge de l'étape quand il y en a un.

Table A.1 : Les critères déclarés avant leur mesure qui décident d'un verdict du Tome III, la grandeur dont chacun dérive, et le verdict (celui du juge quand il y en a un).

Étape	Critère déclaré	D'où il dérive	Verdict
La ronde (ch. 4 à 6, 8)			
génome, W	le brin lit moins de bits par poids que la base de B à erreur sur les poids égale, sur chacun des 12 tenseurs	la base de B : 2,644 1 bits à 4,29 % d'erreur relative	tient (12 sur 12, −0,319 bit) ; chapitre 4
génome, O et F	l'erreur de *sortie* d'une couche d'experts sous celle de la base	la sortie est ce que le modèle lit	tombent ; chapitre 4
génome, S*	déclaré avant sa passe : sortie sous la base sur les 8 experts entiers	idem	tient (8 sur 8) ; KL égale à la base sur K10
duplication, D1	bits de l'enfant conditionné au parent sous A_ctx sur chacune des matrices et en global	la frontière sans perte, 3,843 1 bits	ne marche pas (gain net 0,001 9 bit, 21 matrices sur 48) ; chapitre 5
Le mimétisme (ch. 7) : succès et mort, `mesures/mimetisme/tri/00-seuils-declares.md`			
P1	LFU à demi-vie − LRU ≥ +0,02 de succès sur les deux générations ; mort sous +0,005	0,02 sur 0,2621 de défauts en LRU, soit ×1,083 si le disque borne tout	ouvert (juge P1)
P2	part disque −15 % à $R = 336$ et copies ≤ 25 % du jeton ; mort sous 5 %	la moitié du −29,6 % simulé, jugé trop optimiste	ouvert
P3	défauts −10 % pour une KL excédentaire ≤ 0,038 ; mort au-delà de 0,07	la pente du mode rapide, 0,13 de KL pour −34 % d'octets	à la limite (ce qui est jugé ; R6)
P4	gain simulé ≥ +5 % pour ≤ 5 % d'octets de plus ; mort sous +2 %	les trois quarts du ×1,07 calculé sous le plafond ×1,21	ouvert, plutôt tombe
P5	erreur tenue à l'écart −10 % contre Babai sur 6 experts sur 8 ; mort sous 3 %	la moitié du gain de la mutation synonyme (−0,0098)	ouvert
P6	$r(\theta) \leq 0{,}7$ à $\theta = 0{,}20$ pour ≥ 8 % d'octets ; mort au-delà de 0,9	la moitié de la KL proportionnelle du tout-rapide	ouvert ; B tient à l'étape 0
La construction (ch. 9 à 11)			
lot A, A-C1	0 empreinte de logits différente sur 5 120 pas	proposition A1 (exactitude par construction)	tient (lot A, juge)
lot A, A-C2	gain ≥ 0,12 bit par poids d'expert	deux fois la part des −0 mesurée	tient (0,127 91, 0,125 62)
lot A, A-C3	pas de régression : $r \geq 1 - 2\sigma$	bruit de la séance	tient ; gain ×1,44 (R1)
lot B, B2 (i)	synonyme meilleure que la frontière : $\bar{d} + 2\sigma < 0$	règle du lot R	tient (3,8 σ)
lot B, B2 des λ	$\lambda = 0$ retenue si $\bar{d} + 2\sigma < 0$ contre $\lambda = 1$, sinon $\lambda = 1$	déclarée après B0, avant le codage $\lambda = 0$	tient (2,09 σ ; juge 3,6 σ)
lot B, S3-K	KL(f64 rapide ‖ moteur rapide) ≤ 0,011, premier choix ≥ 95 %	3× l'écart C4 du moteur exact (0,003 68)	tient (trois variantes ; juge)
lot B, S3-V	gain annoncé si $B/A > 1 + 2\sigma_A$	bruit des passes A	annoncé, puis barré comme gain du mode (G1, G2)
juge, J-V2 et J-V3	gain du mode si $r(R/B) \geq g\,(1 - 2\sigma)$, $\sigma = \max$	borne basse d'un rapport de médianes	tombent (1,284 contre 1,769 ; 0,942 contre 1,227)
juge, J-K2	B2 refait hors de R1, même règle ; prédiction « égales »	$\Phi(z - 2) \approx 0{,}54$	tient, renforcé ; prédiction démentie
Les pistes (ch. 12 et 13)			

Étape	Critère déclaré	D'où il dérive	Verdict
tropicale	la garde vaut d'être construite si elle attrape ≥ 80 % des bascules pour ≤ 10 % des cellules	cahier de la synthèse ; prédiction écrite : 55 à 65 %	tombe (57,1 % ; juge)
spectres	loi de puissance si la méthode de Clauset, Shalizi et Newman la préfère à la log-normale, sur au moins 2/3 de l'échantillon	32 matrices sur 48	amendé avant lecture (sans puissance) ; 0 sur 48, 9 compatibles (juge)

La table datée. Tableau A.2 est engendrée par `memoire/outils/tome3_annexes.py` depuis `git log` : pour chaque fichier, le commit qui l'a créé et son heure, puis le nombre de commits qui l'ont touché (un amendement daté fait monter ce nombre ; il se lit dans le fichier, avec son heure et la raison). Elle est rangée par heure de création. Le second tirage de P3 (fichiers `07*`), en travail dans une autre session, n'y est pas (réserve R6).

Table A.2 : Les fichiers de seuils, de critères et d'amendements déclarés avant une mesure, avec le commit qui les a créés (heure locale) et le nombre de commits qui les ont touchés (`memoire/outils/tome3_annexes.py`).

Déclaré avant	Créé par le commit	Commits
`mesures/genome/genome/00-seuils-declares.md`	`e777d6d`, 2026-10-16 21:55	11
`mesures/genome/duplication/00-seuils-declares.md`	`40d4b5c`, 2026-10-16 21:58	3
`mesures/genome/construction/synonyme/00-seuils-declares.md`	`78ce4ed`, 2026-10-17 08:19	7
`genome/pistes/tropicale/seuil-declare.md`	`69260f1`, 2026-10-21 23:22	1
`genome/pistes/spectres/seuil.md`	`a171622`, 2026-10-21 23:22	13
`mesures/genome/construction/juge/00-seuils-declares.md`	`1f5fe0f`, 2026-10-22 03:53	1
`mesures/disque/placement/juge/00-seuils-declares.md`	`c34d836`, 2026-10-22 12:44	6
`mesures/pistes/spectres/puissance-critere.json`	`b9e9ea4`, 2026-10-22 13:49	1
`genome/pistes/tropicale/seuil-declare-indices.md`	`50bf472`, 2026-10-26 23:41	1
`genome/mimetisme/terre_cosmos/seuil-declare.md`	`841e914`, 2026-10-26 23:52	3
`mesures/mimetisme/humain/00-seuils-declares.md`	`9055b23`, 2026-10-26 23:53	3
`mesures/mimetisme/physique/00-seuils-declares.md`	`59eb5a2`, 2026-10-26 23:54	4
`mesures/mimetisme/mathematiques/00-seuils-declares.md`	`2017fcf`, 2026-10-26 23:54	3
`genome/mimetisme/vivant/seuil-declare.md`	`b7ace2d`, 2026-10-26 23:54	9
`genome/mimetisme/ingenierie/seuils-declares.md`	`2a4c60e`, 2026-10-26 23:56	3
`mesures/mimetisme/tri/00-seuils-declares.md`	`ec124f5`, 2026-10-27 00:43	1

Déclaré avant	Créé par le commit	Commits
genome/pistes/spectres/juge-seuil.md	a52beeb, 2026-10-27 00:47	1
genome/mimetisme/P2/seuil-declare.md	6d325d8, 2026-10-27 00:47	1
mesures/mimetisme/P5/00-seuil-declare.md	ab92f4e, 2026-10-27 00:49	4
mesures/mimetisme/P4/00-seuils-declares.md	47339c2, 2026-10-27 00:50	3
mesures/mimetisme/P6/00-seuils-declares.md	4d2b915, 2026-10-27 00:50	2
mesures/mimetisme/P3/00-seuils-declares.md	1606f9f, 2026-10-27 00:51	1
genome/mimetisme/P2/seuil-declare-selectif.md	7146816, 2026-10-27 01:08	1
genome/mimetisme/P2/juge-seuil.md	122b81d, 2026-10-27 01:16	1
mesures/mimetisme/P3/01-affinage-declare.md	174b4e2, 2026-10-27 01:20	1
genome/mimetisme/P1/posthoc-declare.md	00093d8, 2026-10-27 01:25	2
mesures/mimetisme/P6/juge/00-seuils-du-juge.md	aa1f5be, 2026-10-27 01:26	1
mesures/mimetisme/P4/juge/00-seuil-du-juge.md	82c49a6, 2026-10-27 01:29	1
mesures/mimetisme/P3/juge/00-criteres-du-juge.md	c23e3db, 2026-10-27 01:32	1
genome/mimetisme/P1/juge/criteres.md	238965b, 2026-10-27 01:34	1
mesures/mimetisme/P5/juge/00-seuil-juge.md	c5d8c30, 2026-10-27 01:55	2
mesures/mimetisme/P3/02-etage-B-declare.md	c4a037a, 2026-10-27 05:48	1
mesures/mimetisme/P3/03-critere-marge-declare.md	e8bf954, 2026-10-27 08:06	1
mesures/mimetisme/P3/05-temoin-b0.3-declare.md	3a3713e, 2026-10-27 10:06	1
mesures/mimetisme/P3/06-routeur-ligne-declare.md	0f38124, 2026-10-27 10:31	1

A.3 Les témoins et leurs cassures

Tableau A.3 rassemble les témoins qui portent un verdict du tome. Chacun a une réponse connue d'avance, et une cassure qui l'a fait tomber : sans elle, le témoin ne prouve rien.

Table A.3 : Les témoins qui portent un verdict du Tome III : la réponse connue d'avance, la cassure, et le résultat [mes.].

Témoin	Réponse connue d'avance	Cassure	Résultat
éclaireur, T1	avec $H = I$, la synonyme rend la frontière (0 symbole différent)	le H réel	1 789 132 symboles différents
lot B, W1	refaire L00-E00 rend le fichier de l'éclaireur octet pour octet	$\delta = 0{,}99$	117 501 symboles différents
lot B, W2	second chemin égal au journal à 10^{-9} relatif	un bit de classe sur une ligne	$7{,}2 \cdot 10^{-3}$
lot B, substitutions	substitués = lus	la passe de l'éclaireur (32 sur 731)	tient ; témoin d'abord mal déclaré, barré

Témoin	Réponse connue d'avance	Cassure	Résultat
lot B, `gemv_base`	égale à la f64 au bit sur un cas exact	le mode exact sur les mêmes octets	0 ligne sur 2 880 ; tombe
lot B, S3-T1	768 experts égaux au `.sym`, aux échelles, aux empreintes	`--casser C X`	l'expert visé, et lui seul
lot B, S3-K	moteur rapide = f64 rapide	f64 *frontière* contre moteur synonyme	0,190 et 0,149 : tombe
juge, J-T1	v1.0 : décodé = GGUF ; changés v1.0 → v1.2 = les −0	mot de flux, canonisation retirée, bit de biais	40 sur 40 ; trois cassures tombent
juge, J-C1	v1.2 évalué deux fois : CSV identiques	biais retourné sur la couche 0	512 sur 512 différentes
juge, KL	sa KL = `juge kl` à 10^{-6}	Q décalé d'une position	$1{,}1 \cdot 10^{-8}$; 1,634
juge, fenêtres	chaque fenêtre fondue égale à sa source	une fenêtre décalée	tombe
tropicale, T2	routage de A imposé à C : 0 bascule	—	0 ; cassure 1
tropicale, T3	marges permutées : la garde au hasard	—	10,0 % ; oracles 100 %
spectres, juge	témoins du juge	sa cassure	tombe sur les deux témoins visés
annexes, `tome3_annexes.py`	tout fichier cité par une ligne « Données » existe	un fichier inexistant ajouté	tombe (code 1)

Les gardes des figures. Les outils de données des figures (`memoire/outils/tome3_*.py`) s'arrêtent à leur première garde ; le juge de la troisième passe des figures en a forcé neuf à faux par son propre moyen, dans une copie, et toutes ont arrêté leur script (`memoire/tome3/JUGEMENT-figures3.md`, § 4.4) : GN2 (III-8.5, un débit du codec réel changé), GN1 (III-10.1, 767 experts décodés au lieu de 768), G7 (III-9.6, la règle de cadre), J5 et J6 (III-8.6 bis), le compagnon (III-8.6 bis), $j = 11$ (la rangée de III-9.1), E1 (III-9.7), la seconde pointe (III-7.4, où c'est la compilation qui s'arrête) et DN7 (III-4.8). `memoire/outils/tome3_cassures.py` rejoue quatre de ces cassures ; il n'a pas sa place dans une construction, puisqu'il force les gardes à faux.

A.4 Les amendements

Un seuil peut changer : avant la lecture, avec sa raison et son heure ; après, jamais sans le dire. Le tome en compte de trois sortes.

Avant la lecture, parce que l'instrument ne pouvait pas répondre. La piste des spectres a amendé son verdict principal le 21 octobre (§ 4 bis de `genome/pistes/spectres/seuil.md`, commit `bfc1c8c`), avant toute lecture des spectres réels : un essai de l'instrument sur 40 tirages de Pareto vrais ($n = 2\,880$, $\alpha = 2{,}5$ et 3,5) les avait tous rendus « non départagés » contre la log-normale. Le critère n'avait aucune puissance ; le juge de la piste a montré depuis qu'il est impossible à satisfaire à toute taille (chapitre 13). Le verdict principal est rapporté tel que déclaré, à côté du verdict amendé.

Avant la mesure, parce qu'une mesure antérieure le demandait. Le lot B a déclaré la règle de départage des deux λ à 12 h 15 le 17 octobre, après B0 et avant le codage $\lambda = 0$ (section 10.5.4), puis les règles de calme de la séance s2 le 21 à 23 h 59, avant toute passe. P1 a déclaré ses analyses post hoc avant de les calculer (`genome/mimetisme/P1/posthoc-declare.md`) ; P2 sa démotion sélective, hypothèse post hoc dite, avant d'ouvrir le 20b (`seuil-declare-selectif.md`) ; P4 un

amendement de lecture du protocole, avant tout lancement ; P5 trois addenda, chacun avant son lancement ; P6 l'amendement A1, avant toute simulation (chapitre 7).

Après une lecture, dit comme tel. Deux témoins du lot B ont été corrigés après la passe qui les avait fait tomber sans cause : le compteur des substitutions (« 768 » devenu « substitués = lus ») et la tolérance de la KL recalculée (10^{-12} absolu devenu 10^{-9} relatif). Ni l'un ni l'autre ne décide d'un verdict ; tous deux sont barrés à leur place, avec l'heure (section 10.5.1). Le critère pondéré par la marge de P3 a été déclaré après la lecture de $N = 146$ et avant toute passe neuve, ce que son fichier dit dans son titre.

A.5 Les protocoles des juges

Chaque étape du tome a eu son juge, qui n'avait rien construit de ce qu'il jugeait. Leur protocole est le même partout :

1. déclarer ses seuils avant toute mesure, avec une prédiction (le juge de la construction : « égales » pour B2, ×1,38 pour $r(R/B)$; les deux démenties, et publiées comme telles) ;
2. refaire chaque chiffre publié par un chemin propre (un autre décodeur, une autre KL, un autre texte, un autre plan d'expérience), sans lire le code jugé avant d'avoir écrit le sien ;
3. par défaut, *un chiffre non reproduit ne tient pas* ; un chiffre reproduit mais mal attribué est barré comme attribution et gardé comme mesure ;
4. lire le livrable phrase à phrase contre ses journaux, et classer ce qui ne tient pas en graves (qui changent ce que le projet ferait ensuite) et mineurs ;
5. porter les barres dans le livrable jugé, sans rien retirer, et écrire sa propre section « Ce qui pourrait être faux ».

Le jugement d'un juge peut lui-même tomber : celui de la construction attribuait l'écart de l'étage disque à « l'état physique des fichiers » ; le juge du placement disque l'a fait tomber le 27 octobre, et le jugement porte la correction à chaque endroit (réserve R1, section 11.6.5).

A.6 Ce qui pourrait être faux

Ce qui pourrait être faux

- **Git date un commit, pas une lecture.** Que le seuil soit commité avant le premier journal ne prouve pas qu'aucune lecture exploratoire ne l'a précédé ; le juge de la piste tropicale le dit de sa propre piste (les 222 lignes de code commitées 74 secondes après le seuil étaient écrites avant lui ; écrire un lecteur n'est pas lire).
- **Les heures écrites dans les notes** ne concordent pas toujours avec git, de quelques minutes ; c'est l'heure du commit qui fait foi.
- **La table des critères est choisie** : elle retient ceux qui décident d'un verdict du tome ; les fichiers de seuils en contiennent d'autres (témoins de second rang, critères de lecture sans verdict), qui se lisent dans les fichiers de tableau A.2.
- **Les verdicts du mimétisme** sont ceux des juges de chaque piste, rapportés par le chapitre 7 ; le second tirage de P3, en travail ailleurs, peut changer le sien.
- **La table datée** ne voit que les fichiers dont le nom dit « seuil », « critère » ou « déclaré » ; un seuil écrit dans un autre fichier (un cahier, un livrable) lui échappe.

Annexe B

Outils

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, l'annexe des outils des Tomes I et II : chaque outil rangé par étude ou par lot, avec la première phrase de sa propre description, et la chaîne qui construit le mémoire. **Cette annexe établit** la même chose pour le Tome III : la chaîne qui le construit et ce qu'elle vérifie ; chaque outil commité de la quête, du mimétisme, des pistes, des juges des figures et du mémoire, rangé par chapitre ; chaque figure au style du tome et les fichiers qu'elle lit ; les licences des dépendances. **Il reste** ~~à mettre l'outil des annexes dans la cible tome3 de construire.sh~~ : rien, pour la chaîne ; l'outil des annexes y est depuis le 2026-10-29 (commit 57db4ce), et il lit le commit plutôt que l'arbre de travail depuis le commit e5ce266 (section B.1).

B.1 La chaîne du Tome III

Le Tome III se construit comme les deux premiers, par le même script et la même configuration de `latexmk` (`memoire/construire.sh`, `memoire/latexmkrc`), depuis `memoire/` :

- `./construire.sh tome3` (ou `tome3-fr`, `tome3-en`) lance d'abord les huit outils de données des figures, un par lot, sous `memoire/outils/` : `tome3_` suivi de `eclaireur`, `genome`, `duplication`, `synthese`, `codon`, `synonyme`, `juge` ou `latents`, puis `.py`. Chacun relit les journaux commités, écrit ses CSV et ses nombres sous `memoire/figures/tome3/donnees/`, et s'arrête à sa première garde ; leur régénération redonne les fichiers commités au bit (`memoire/tome3/JUGEMENT-figures3.md`, § 4.3).
- Il engendre alors les quatre tables des annexes A, B et C par `memoire/outils/tome3_annexes.py` (tableaux A.2, B.1, B.2 et C.1), depuis le commit ~~, il refuse de tourner si un fichier qu'il compte diffère du commit~~ de publication du tome, `4b071d0` (paragraphe « L'outil des annexes », ci-dessous).
- Il compile ensuite, à part, les figures de la piste des spectres : `genome/pistes/spectres/figures/spectres-figures.tex` est un document autonome dont les chemins de données sont relatifs à son dossier, en lecture seule ; il se compile donc dans ce dossier, sa sortie sous `memoire/tome3/pistes/` (ignoré par git), où le chapitre des spectres prend ses pages.
- Il construit enfin les deux PDF par `latexmk` (lualatex, biber) dans `memoire/tome3/fr/` et `memoire/tome3/en/`, et s'arrête en erreur sur toute référence ou citation indéfinie et sur tout fichier de données absent.

Deux outils n'ont pas leur place dans une construction, et n'y sont pas : `memoire/outils/tome3_palette.py`, qui réécrit ses fichiers avec d'autres fins de ligne (égaux au contenu seulement), et `memoire/outils/tome3_cassures.py`, qui force les gardes à faux (annexe A).

L'outil des annexes. `memoire/outils/tome3_annexes.py` engendre les quatre tables des annexes A, B et C depuis le commit ~~HEAD~~ de publication du tome (`git ls-tree`, `git cat-file`, `git log`) : les outils, les figures et leurs données, les dossiers de journaux, les fichiers de seuils datés. Son témoin : tout fichier cité par la ligne « Données » d'une figure (chemin complet, motifs `*` et `{a,b}` permis) existe dans le dépôt ; sa cassure (`--casser`, un fichier inexistant ajouté à la première figure) le fait tomber, code 1 [mes.]. ~~Il n'est pas encore dans la cible tome3 de construire.sh, que ce rédacteur n'avait pas le droit de modifier~~ ; il y est depuis le 2026-10-29 (commit 57db4ce) : la cible `tome3` de `memoire/construire.sh` le lance avant latexmk, et chaque construction du tome réécrit ses tables depuis le dépôt. Corrigé le 2026-10-29 (commit e5ce266, grave G3 de `memoire/tome3/JUGEMENT-final.md`) : il comptait les lignes et les octets dans l'arbre de travail, si bien qu'une modification non commitée d'une autre session y donnait 124 lignes à `outils/mimetisme/humain/e4b_defauts.py`, qui en a 122 au commit, et que les volumes de l'annexe C dépendaient des fins de ligne de la copie (`mesures/genome/construction/codon/` : 2,47 Mo dans une copie, 2,43 Mo au commit). Il compte désormais les lignes sur le contenu des objets git et les volumes sur leur taille, ~~et il refuse de tourner, code 1, si un fichier qu'il compte diffère du commit dans l'arbre de travail~~ [mes.]. *Figé le 2026-10-29* au commit de publication du tome, `4b071d0`, comme les outils des Tomes I et II : lu au commit HEAD, il refusait de tourner dès qu'une autre session laissait un fichier compté non commité ; lu à ce commit, il redonne les quatre tables au bit, et ni l'arbre de travail ni les commits postérieurs n'y entrent plus.

B.2 Les outils commités

Tout nombre [mes.] ou [calc.] de ce tome sort d'un outil commité. Tableau B.1 les liste, rangés par chapitre, chacun avec la première ligne de sa propre description, citée telle qu'écrite (en français), et son nombre de lignes. Les outils de la quête, du mimétisme et des pistes appartiennent à leurs flux ; le mémoire les cite et ne les modifie pas. Ceux du second tirage de P3, en travail dans une autre session, n'y sont pas (réserve R6).

Table B.1 : Les outils commités du Tome III, par chapitre, chacun avec la première ligne de sa propre description (`memoire/outils/tome3_annexes.py`).

Outil	Sa propre description (première ligne, citée)	Lignes
Ch. 2 et 3 : la question, l'éclaireur		
`outils/genome/eclaireur/bib_sources.py`	Écrit le BibTeX et le tableau des sources de l'éclaireur depuis les métadonnées primaires.	128
`outils/genome/eclaireur/borne_rd.py`	La borne distorsion-débit des poids MXFP4 de gpt-oss, dans le domaine des poids.	196
`outils/genome/eclaireur/capture/src/main.rs`	genome-capture : entrées réelles des experts de gpt-oss, et KL d'une base substituée.	237
`outils/genome/eclaireur/capture/src/modele.rs`	FOURCHE de moteur/crates/trc-reference/src/modele.rs (copie du 2026-10-16, commit 6dd7414) pour l'éclaireur du génome des poids (genome/01-eclaireur.md). …	770
`outils/genome/eclaireur/choix_delta.py`	Le choix de l'amortissement δ de la mutation synonyme, par validation croisée SUR LA CALIBRATION SEULE.	74
`outils/genome/eclaireur/diagnostic_calibration.py`	Diagnostic de la mutation synonyme : apprend-elle les entrées, ou seulement l'échantillon ?	67
`outils/genome/eclaireur/frontiere_csv.py`	Place chaque mécanisme mesuré sur le plan débit × erreur (la figure de synthèse du mémoire).	65

Outil	Sa propre description	Lignes
`outils/genome/eclaireur/passes.sh`	Les passes de la mutation synonyme, dans l'ordre déclaré (genome/REPRISE-etat-de-l-art.md). …	23
`outils/genome/eclaireur/synonyme.py`	La mutation synonyme : une base de 2 bits choisie là où la sortie ne la voit pas.	575
`outils/genome/eclaireur/zero-synonyme/src/main.rs`	FOURCHE de outils/arbitre/decoupe (la mesure F1, copiée le 2026-10-16) pour l'éclaireur du génome des poids (genome/01-eclaireur.md § 4) …	844
`outils/genome/eclaireur/zero_compte.py`	Le codon synonyme du zéro, compté : combien de codes −0 et +0, et ce que coûte le signe d'un zéro.	86
Ch. 4 : le génome des poids (la recette)		
`outils/genome/recette-kl/src/main.rs`	genome-recette-kl : les logits des fenêtres d'un .kld avec des experts substitués (base .sym ou recette .f32w) …	98
`outils/genome/recette-kl/src/modele.rs`	Copie (2026-10-16, sujet genome) de la fourche de l'éclaireur (outils/genome/eclaireur/capture/src/modele.rs), avec un seul ajout …	775
`outils/genome/recette/analyse.py`	Second chemin du verdict W : refait, depuis les CSV par rangée de genome codons, les courbes débit-distorsion par allocation lagrangienne et les bits à erreur …	142
`outils/genome/recette/analyse_fidelite.py`	Verdict F (fidélité de sortie) depuis le journal de genome fidelite.	67
`outils/genome/recette/borne_par_tenseur.py`	Ce qu'atteindrait un code gaussien idéal (débit-distorsion de Shannon) sur les rangées de l'échantillon, à l'erreur de la base de chaque tenseur …	33
`outils/genome/recette/chaine.sh`	Chaîne du sujet genome après les passes de poids : sortie des experts sur entrées réelles (et écriture des experts reconstruits), puis KL contre la référence …	30
`outils/genome/recette/erreur_colonnes.py`	Où tombe l'erreur de la recette : par colonne (canal d'entrée) de chaque matrice d'un expert.	53
`outils/genome/recette/erreur_sortie_forme.py`	La forme de l'erreur de sortie : quelle part est un simple changement de gain (parallèle à la sortie exacte), quelle part est orthogonale …	81
`outils/genome/recette/figures_transformations.py`	Données des figures « avant / après » du sujet genome (SOCLE, demande de Stone du 2026-10-17) :	94
`outils/genome/recette/file.sh`	File des passes du sujet genome, lancées l'une après l'autre (chacune prend tous les fils).	12
`outils/genome/recette/graine_theorie.py`	Ce que la recette graine + coefficients peut atteindre au mieux, calculé (sans quantifier les coefficients) …	39
`outils/genome/recette/kl.sh`	Passe K du sujet genome : la base et la recette S* (pondérée, exceptions 0,1 %, 2,64 bits) substituées aux experts 0 et 16 des couches 0, 8, 16, 23 dans la …	20
`outils/genome/recette/kl10.sh`	Passe K du sujet genome : la base et la recette S* (pondérée, exceptions 0,1 %, 2,64 bits) substituées aux experts 0 et 16 des couches 0, 8, 16, 23 dans la …	20
`outils/genome/recette/src/base.rs`	La frontière : la base de l'architecture B (signe + classe de magnitude parmi {0 ; 0,5} {1 ; 1,5 ; 2} {3 …	70
`outils/genome/recette/src/codons.rs`	La lecture par codons glissants : le « ribosome » des poids.	406

Outil	Sa propre description	Lignes
outils/genome/recette/src/donnees.rs	Les poids réels : un expert d'un tenseur MXFP4 de gpt-oss, rangée par rangée, lu par le lecteur GGUF de trc-reference (chemin indépendant du .trc).	95
outils/genome/recette/src/fidelite.rs	La fidélité de sortie en fonction du débit : pour chaque expert, la courbe (bits par poids, erreur relative de la sortie de l'expert sur des entrées réelles) …	336
outils/genome/recette/src/graine.rs	La recette littérale : un bloc de C poids = U(graine) · t, U une matrice C × P tirée d'un générateur pseudo-aléatoire par sa graine de K bits, t P coefficients …	291
outils/genome/recette/src/hadamard.rs	Rotation de Hadamard aléatoire par morceaux, le long des colonnes d'une rangée.	74
outils/genome/recette/src/main.rs	genome : mesures du génome des poids sur gpt-oss (sujet genome). Processeur seul.	42
outils/genome/recette/src/mesure.rs	Les mesures. Chaque sous-commande écrit un journal JSON sur la sortie standard.	832
outils/genome/recette/src/motifs.rs	Les motifs partagés, combinés : un bloc de d poids = somme de M motifs, chacun tiré d'un dictionnaire de 2^b motifs partagé par tous les experts d'un tenseur …	74
outils/genome/recette/src/sortie.rs	L'erreur sur la sortie des experts, pour des entrées réelles.	433
outils/genome/recette/structure.py	Où est le bruit, où est la structure : trois mesures sur les experts MXFP4 réels de gpt-oss-20b.	135
outils/genome/recette/vers_csv.py	Les CSV des figures du mémoire (sujet genome), régénérés depuis les journaux commités.	76
Ch. 5 : la duplication de gènes		
outils/genome/duplication/ecran_parente.py	Écran de parenté : chaque neurone d'un expert a-t-il un « gène parent » ailleurs dans le modèle ?	146
outils/genome/duplication/experts.py	Accès aux experts MXFP4 de gpt-oss pour l'étude de la duplication (génome des poids).	101
outils/genome/duplication/figure_phylogenie.py	Données de la figure « l'arbre phylogénétique des experts » (SOCLE : espaces latents, 2026-10-17).	88
outils/genome/duplication/lancer_echantillon.sh	L'échantillon de D1, D2, D2b, D2c, T, O (seuils : mesures/genome/duplication/00-seuils-declares.md). …	11
outils/genome/duplication/mutation.py	Un expert codé comme mutation de son parent : choix du parent, coût sans perte (D1), coût avec perte contre le même quantificateur sans parent (D2), mutation …	341
outils/genome/duplication/paires_arbre.py	Parenté d'expert à expert à une permutation des neurones près, et l'arbre de parenté le plus économe.	183
outils/genome/duplication/paralogues.py	Les paralogues : un expert absent peut-il être remplacé par un autre expert de sa couche ?	140
outils/genome/duplication/spectre_famille.py	La famille de gènes : les 32 experts d'une couche partagent-ils un sous-espace (un « domaine » commun) ?	111
outils/genome/duplication/synthese.py	Synthèse de l'échantillon de la duplication : CSV pour le mémoire et verdicts contre les seuils déclarés.	143
Ch. 6 : codage prédictif et élagage		
outils/genome/predictif-elagage/contextes.py	Codage prédictif sans perte des codes MXFP4 : ce que chaque prédicteur retire de l'entropie.	176

Outil	Sa propre description	Lignes
outils/genome/predictif-elagage/contextes_temoin.py	Recalcule le témoin « gauche permutée » de contextes.py avec la formule de biais corrigée.	22
outils/genome/predictif-elagage/debit_distorsion.py	Débit-distorsion des codes MXFP4 réels : élagage synaptique et quantification prédictive bornée.	380
outils/genome/predictif-elagage/debit_hebb.py	Débit-distorsion à la sensibilité hebbienne : remplissage inverse sur (e, classe de ligne, classe de colonne).	151
outils/genome/predictif-elagage/debit_sensibilite.py	Débit-distorsion à la sensibilité mesurée : remplissage inverse sur (échelle e, activité de colonne q).	168
outils/genome/predictif-elagage/histo_hebb.py	Sensibilité hebbienne de chaque poids d'un expert : activité présynaptique × influence postsynaptique.	117
outils/genome/predictif-elagage/histo_sensibilite.py	Loi conjointe pondérée par la sensibilité mesurée : l'élagage synaptique dépendant de l'activité.	88
outils/genome/predictif-elagage/histogramme.py	Loi conjointe exacte (échelle e, δ de ligne, code) de chaque tenseur d'experts MXFP4.	59
outils/genome/predictif-elagage/kld_identiques.py	Témoin de la chaîne à poids altérés : les entrées .kld d'un plan sans perte en valeur sont-elles identiques, octet pour octet, à celles de la référence exacte, …	42
outils/genome/predictif-elagage/kld_tranche.py	Tranche un fichier .kld (format de llama-perplexity) : garde les fenêtres [a, b).	45
outils/genome/predictif-elagage/mxfp4.py	Lecture des experts MXFP4 de gpt-oss, matrice par matrice, pour l'étude predictif-elagage.	109
outils/genome/predictif-elagage/plans.py	Plans de reconstruction : pour une famille et une erreur visée, la table T[ctx][code] de chaque tenseur.	164
outils/genome/predictif-elagage/resume_csv.py	Table de synthèse de l'étude (CSV pour le mémoire), tirée des journaux commités, jamais de mémoire.	60
outils/genome/predictif-elagage/sonde/src/main.rs	La sonde de l'étude genome/predictif-elagage, sur la référence f64 de troncons (trc-reference).	254
outils/genome/predictif-elagage/sortie_experts.py	Erreur à la SORTIE des experts, sur les entrées réelles capturées par la référence f64.	186
outils/genome/predictif-elagage/taux_reel/src/bin/taux_plan.rs	Taux réel d'un plan avec perte (élagage, remplissage inverse) : les cellules de chaque poids sont codées par le rANS de l'étude 03, une table par (contexte du …	327
outils/genome/predictif-elagage/taux_reel/src/main.rs	Taux réel des codes MXFP4 de gpt-oss, codés par le rANS de l'étude 03 et décodés exactement.	308
Ch. 7 : le mimétisme, les juges		
outils/mimetisme/P1/juge/juge_p1.py	P1, juge : simulateur indépendant de cache à règle de groupe (LRU, LFU à demi-vie, décroissance inversée).	219
outils/mimetisme/P1/juge/sondes_p1.py	P1, juge : sondes S1 (tête avant la fin de message), S3 (fenêtres froides) et cassure C1ter (post hoc, dite).	92
outils/mimetisme/P2/juge/a_budget_egal.py	P2, juge, contrôle hors seuil (dit comme tel) : l'inclusif à R <= V est dégénéré par construction …	38
outils/mimetisme/P2/juge/depouiller.py	P2, juge : confronte juge-brut.json aux seuils de genome/mimetisme/P2/juge-seuil.md (rend verdict-juge.json). Usage …	80
outils/mimetisme/P2/juge/juge_p2.py	P2, juge : mon chemin pour refaire le chiffre principal (seuils : genome/mimetisme/P2/juge-seuil.md).	240
outils/mimetisme/P3/juge/N238/juge_N238.py	Juge de P3 à N = 238 : mon propre chemin, sans importer p3_*.py, le juge kl ni le lecteur du juge de N = 146.	344

Outil	Sa propre description	Lignes
outils/mimetisme/P3/juge/etageB_juge.py	Juge de l'étage B de P3 : un second lecteur des .kld et du journal de routage, écrit sans reprendre p3_etageB*.py.	277
outils/mimetisme/P3/juge/juge_p3.py	P3, juge indépendant : second chemin de l'étage A (routage biaisé vers les résidents du cache).	254
outils/mimetisme/P3/juge/routeur-ligne/juge_routeur_ligne.py	Juge du routeur biaisé EN LIGNE de P3 : mon propre chemin, sans importer p3_*.py, le juge kl, rejouer, ni le juge de N = 238.	575
outils/mimetisme/P4/juge/juge_generation.py	P4, juge : la jambe « génération » du verdict. Les deux traces de génération du 120b bouclent-elles, et que donne le prédicteur sur leur moitié de test ? Usage …	44
outils/mimetisme/P4/juge/juge_p4.py	P4, juge indépendant : refait précision, couverture, octets et gain du bras P par un second chemin.	319
outils/mimetisme/P5/juge/juge_p5.py	P5, juge indépendant : le gain d'évaluation de la descente cyclique (CD) contre Babai, par un chemin écrit de zéro.	244
outils/mimetisme/P5/juge/juge_pt.py	P5 juge, phase optionnelle (descriptive) : échange de répliques écrit de zéro en numpy, sur les poids de J1.	118
outils/mimetisme/P5/juge/lot_j5_pt.sh	J5 (demi-calibration) puis PT numpy descriptif, sequentiels (2 fils au plus).	16
outils/mimetisme/P5/juge/lot_juge.sh	Lot du juge P5 : J1 (graine 1), J2 (graine 2026), J3 (regimes), temoins T1-T3, lambda 0,2 (graine 1).	16
outils/mimetisme/P5/juge/resume_juge.py	P5 juge : agrège les journaux de juge_p5.py et les compare aux chiffres de P5 (gains-d1.csv, table-regime.csv). …	135
outils/mimetisme/P6/juge/juge_controles.py	Juge P6 : contrôles exploratoires (hors seuils déclarés, étiquetés tels dans JUGEMENT.md).	85
outils/mimetisme/P6/juge/juge_p6.py	Juge P6 : second chemin indépendant du chiffre principal (variante A) et de la variante B.	257
outils/mimetisme/P6/juge/juge_rang.py	Juge P6 : le mélange par couche (T2 et « les défauts ne sont pas corrélés aux petits poids ») garde le rang.	57
Ch. 7 : le mimétisme, les six pistes		
outils/mimetisme/P1/cache_p1.py	P1 : mémoire du cache à deux échelles (LFU à demi-vie), jugée en génération.	526
outils/mimetisme/P1/fenetres_p1.py	P1, analyse post hoc A : fenêtres froides de 256 jetons sur des textes cohérents (lectures du 120b). Déclarée avant calcul …	91
outils/mimetisme/P1/figures_p1.py	P1 : figures du mémoire, tirées des données commitées (hits-*.npz, fenetres-*.csv) et des traces extraites. Usage …	131
outils/mimetisme/P1/noyau-p1/src/main.rs	noyau-p1 : copie de outils/mimetisme/physique/noyau-cache (rôle éclaireur physique, 2026-10-26), adaptée pour P1 (rôle expérimentateur, 2026-10-27). …	229
outils/mimetisme/P1/rust_fenetres_p1.py	P1, second chemin des fenêtres froides : noyau-p1 rejoue LRU et LFU(h) sur des fenêtres de 256 jetons écrites en fichiers, comparées à fenetres-120b.csv …	33
outils/mimetisme/P1/rust_p1.py	P1, second chemin : noyau-p1 (Rust, copie de noyau-cache du rôle physique avec deux changements dits dans son en-tête) rejoue les politiques de cache_p1.py sur …	119
outils/mimetisme/P2/cache_exclusif.py	P2, cache exclusif VRAM vers RAM : simulateur et bancs (mon implémentation, indépendante de cache2.py).	192

Outil	Sa propre description	Lignes
outils/mimetisme/P2/diag_t2_20b.py	P2 : diagnostic du témoin T2 sur le 20b (écart de l'exclusif au LRU de V + R, avec et sans épinglage).	19
outils/mimetisme/P2/explorations.py	P2, explorations POST-HOC (hors seuil déclaré, à lire comme des pistes, pas comme le verdict).	118
outils/mimetisme/P2/figures.py	P2 : figures tirées des CSV commités (aucune valeur saisie à la main). fig1 distances de pile LRU …	80
outils/mimetisme/P2/holdout20b.py	P2, test de la démotion sélective sur le 20b (seuil : genome/mimetisme/P2/seuil-declare-selectif.md). Usage …	62
outils/mimetisme/P2/net_selectif.py	P2 : temps de service des défauts par jeton, 120b, R = 336 : LRU seul, exclusif simple (s=0), sélectif (s=1). …	23
outils/mimetisme/P2/second_chemin.rs	P2, second chemin : cache LRU à deux étages, inclusif et exclusif, en Rust. Indépendant de cache2.py et de cache_exclusif.py …	172
outils/mimetisme/P2/verdict.py	P2 : lit temoins.csv, grille.csv, chaine.csv (et rust.csv s'il existe), vérifie les témoins déclarés dans genome/mimetisme/P2/seuil-declare.md, calcule le …	160
outils/mimetisme/P2/verdict_selectif.py	P2 : verdict de la démotion sélective sur le 20b au seuil déclaré (seuil-declare-selectif.md).	37
outils/mimetisme/P3/p3_affinage.py	P3, affinage post hoc de l'étage A (déclaré dans mesures/mimetisme/P3/01-affinage-declare.md avant ce lancement).	155
outils/mimetisme/P3/p3_cassures_reelles.py	P3 : les cassures des témoins W3 et W5 sur les données réelles (elles n'avaient été passées que sur les logits synthétiques). W3-hasard …	33
outils/mimetisme/P3/p3_etageB.py	P3 étage B : une passe de la référence f64 à routage imposé, sa RAM de pointe, sa KL (chemin 2).	101
outils/mimetisme/P3/p3_etageB_N238.py	P3 étage B à N = 238 : deux passes de la référence f64 à routage imposé (β_1 puis β_2), l'une après l'autre.	163
outils/mimetisme/P3/p3_etageB_N238_analyse.py	P3 étage B à N = 238 : second chemin (numpy depuis le CSV par position de juge kl) et verdict aux deux lectures de 03-critere-marge-declare.md.	92
outils/mimetisme/P3/p3_etageB_N238_lecteur.py	P3 étage B à N = 238 : troisième chemin. Relit les .kld avec le lecteur du juge (etageB_juge.py, écrit sans le code de l'étage B), recalcule KL, premier choix …	61
outils/mimetisme/P3/p3_etageB_chemin2bis.py	P3 étage B, second chemin du chiffre principal : KL moyenne, premier choix identique, KL cumulée par fenêtre et arrêt d'O'Brien-Fleming, recalculés en numpy …	30
outils/mimetisme/P3/p3_fichiers_B.py	P3 : les fichiers d'entrée de l'étage B (ensembles imposés à la référence f64), autour du franchissement de -10 % de défauts.	41
outils/mimetisme/P3/p3_fichiers_N238.py	P3 étage B à N = 238 : choix de beta_1 (premier pas de 0,001 qui retire au moins 10 % des défauts) et fichiers imposés.	34
outils/mimetisme/P3/p3_routage.py	P3 : routage conditionné par le cache, borné par la marge. Étage A (boucle ouverte), outils et témoins.	451
outils/mimetisme/P3/p3_routeur_ligne.py	P3, routeur biaisé en ligne : pilote des passes d'évaluation du moteur (gpt-oss-20b, 10 fenêtres de R1, 238 emplacements).	171
outils/mimetisme/P3/p3_routeur_ligne_analyse.py	P3, routeur biaisé en ligne : analyse des passes d'évaluation du moteur et verdicts aux seuils de 06-routeur-ligne-declare.md.	207

Outil	Sa propre description	Lignes
outils/mimetisme/P3/p3_routeur_ligne_vitesse.py	P3, routeur biaisé en ligne : génération (jetons par seconde), séries ABBA (06-routeur-ligne-declare.md § 7).	120
outils/mimetisme/P3/p3_series.py	P3 : découpe affinage.csv en séries pour pgfplots (une par variante et budget, bêta positifs jusqu'à 1, triés), défauts évités en % du compte à bêta = 0.	16
outils/mimetisme/P4/p4.py	P4 : préchargement et rétention à confiance calibrée (prédiction conforme), qui cède à la demande.	314
outils/mimetisme/P4/p4_figures.py	P4 : figures et CSV du mémoire, tirés de mesures/mimetisme/P4/mesure-*.json (aucune valeur saisie à la main). Usage ...	95
outils/mimetisme/P4/p4_mesure.py	P4 : mesure par session (calibration fendue, bras P et R, simulateur, second chemin, cassure).	187
outils/mimetisme/P4/p4_synthese.py	P4 : synthèse au seuil déclaré (mesures/mimetisme/P4/00-seuils-declares.md § 2) à partir des JSON de p4_mesure.py. Usage ...	88
outils/mimetisme/P4/p4_temoins.py	P4 : témoins du simulateur et de la calibration (déclarés dans mesures/mimetisme/P4/00-seuils-declares.md § 3).	165
outils/mimetisme/P5/agreger_etape1.py	P5 etape 1 : agrege les journaux mesures/mimetisme/P5/etape1/L??-E??-d<delta>.json, applique la regle declaree (mesures/mimetisme/P5/00-seuil-declare.md + ...	89
outils/mimetisme/P5/etape1.py	P5 etape 1 : le H reel des captures, des poids SYNTHETIQUES gaussiens sur la grille MXFP4.	225
outils/mimetisme/P5/etape2.py	P5 etape 2 : poids REELS (gpt-oss-20b), un expert a la fois (13 Mo de lecture : gate, up, down).	151
outils/mimetisme/P5/figures.py	P5 : figures vraies, tirees des donnees commitees (CSV ecrits a cote). Sorties : mesures/mimetisme/P5/figures/.	194
outils/mimetisme/P5/gibbs.py	P5 : l'estimateur de Gibbs. La mecanique statistique de l'apprentissage (Seung, Sompolinsky, Tishby 1992 ...	66
outils/mimetisme/P5/lot_diag.sh	Exploration (addendum 2) : cible diagonale de l'amortissement, Babai et CD seulement. usage : lot_diag.sh CIBLE DAMP	11
outils/mimetisme/P5/lot_etape1.sh	Les 8 instances (couches 0,8,16,23 x experts 0,16) pour un amortissement donne. 2 fils, priorite normale. usage : lot_etape1.sh DAMP	11
outils/mimetisme/P5/lot_regime.sh	Addendum 3 : variation de N. usage : lot_regime.sh REGIME	11
outils/mimetisme/P5/p5core/src/lib.rs	P5 : noyau du recuit et de l'echange de repliques sur le choix des symboles synonymes.	335
outils/mimetisme/P5/p5lib.py	P5 : bibliotheque du recuit et de l'echange de repliques sur les symboles synonymes.	177
outils/mimetisme/P5/resume.py	Affiche les lignes d'un journal etape1 (une ou plusieurs).	13
outils/mimetisme/P5/table_delta.py	P5 : l'erreur d'evaluation de chaque methode a chaque amortissement, rapportee a Babai deploye (delta = 1, lam = 0). ...	31
outils/mimetisme/P5/table_diag.py	Addendum 2 : cible diagonale de l'amortissement. Erreur d'evaluation (Babai, CD) rapportee a Babai iso delta = 1.	32
outils/mimetisme/P5/table_regime.py	Addendum 3 : variation de N. Gain de CD contre Babai du meme regime, sur le test, par instance.	42

Outil	Sa propre description	Lignes
outils/mimetisme/P5/temoins.py	P5 : temoins W1, W2, W4 (sans fichier modele, sans capture). Seuils : mesures/mimetisme/P5/00-seuil-declare.md.	164
outils/mimetisme/P5/temoins_reglage.py	P5 : reglage du PT sur les instances ou la premiere passe (200 balayages, 8 temperatures, 0,03 a 1,5) n'a pas retrouve l'optimum de la force brute. …	32
outils/mimetisme/P6/analyses.py	P6 étape 0 : analyses exploratoires, hors verdict (étiquetées telles dans resultat.md). 1. …	118
outils/mimetisme/P6/etape0.py	P6 étape 0 : la mesure, après les témoins. Seuils : mesures/mimetisme/P6/00-seuils-declares.md. Usage …	287
outils/mimetisme/P6/etape0_B.py	P6 étape 0, variante B : préremplissage par lots, décision rapide/exact par masse agrégée. Seuils : mesures/mimetisme/P6/00-seuils-declares.md § 3. …	112
outils/mimetisme/P6/figures.py	Figures de P6 (étape 0), tirées de mesures/mimetisme/P6/*.json. Usage : python -I -X utf8 figures.py <racine du dépôt>	71
outils/mimetisme/P6/niveaux-rust/src/main.rs	Second chemin de P6, étape 0 : cache d'experts à niveaux (exact / rapide), écrit à part de tiers.py.	126
outils/mimetisme/P6/second_chemin.py	Second chemin du chiffre principal de P6 (étape 0) : le automate Rust à étiquettes contre tiers.py. Usage …	25
outils/mimetisme/P6/second_chemin_B.py	Second chemin de la variante B : boucles en dictionnaires, tri de Python, sans numpy dans le calcul de r. Usage …	33
outils/mimetisme/P6/temoins.py	P6 étape 0 : témoins T0, T1, T4 (et sa cassure), avant toute mesure. Usage : python -I -X utf8 temoins.py <dossier t120> <exe noyau-cache> <dossier_sortie> …	105
outils/mimetisme/P6/tiers.py	P6, étape 0 : cache d'experts à niveaux (exact / rapide) et portillon de poids de mélange.	207
Ch. 7 : le mimétisme, les éclaireurs de domaine		
outils/mimetisme/humain/e1_memoire.py	E1 : politique de cache « mémoire humaine » (Anderson et Schooler 1991, ACT-R) contre LRU, LFU et Belady.	162
outils/mimetisme/humain/e1b_demivie.py	E1b (POST HOC, non déclaré avant) : le LFU du dépôt (demi-vie 8) est-il mal réglé ? Balayage de la demi-vie, pooled et par genre, et meilleur réglage par genre …	25
outils/mimetisme/humain/e2_triage.py	E2 : tri par poids de mélange (triage) des défauts de cache. Proxy de masse, pas de qualité.	132
outils/mimetisme/humain/e345.py	E3 (allocation marginale par couche), E4 (précédent intercouches), E5 (classes ABC). Seuils et témoins déclarés avant l'écriture de cet outil …	289
outils/mimetisme/humain/e4b_defauts.py	E4b (POST HOC, non déclaré avant) : le prédicteur « précédent » de E4 (régime en ligne) couvre-t-il les DÉFAUTS de cache ?	122
outils/mimetisme/humain/e6_sequentiel.py	E6 : arrêt séquentiel de groupe (forme O'Brien-Fleming) rejoué sur les séries de KL par fenêtre du juge. Seuils et témoins déclarés avant …	76
outils/mimetisme/humain/e7_marge.py	E7 (POST HOC, descriptif, sans critère déclaré) : borne haute des défauts qu'un routage conscient du cache pourrait éviter en échangeant le 4e expert contre le …	66
outils/mimetisme/ingenierie/cache2.py	Simulateur de cache d'experts à un ou deux étages, sur les traces de routage du 120b.	322

Outil	Sa propre description	Lignes
outils/mimetisme/mathematiques/m1_m2.py	Éclaireur « mathématiques et théorie de l'information » : mesures M1 et M2.	320
outils/mimetisme/mathematiques/maslov_jouet.py	Calcul jouet (pas une mesure sur le modèle) : lissage de Maslov d'une décision à seuil.	22
outils/mimetisme/mathematiques/plafond_speculatif.py	Calcul (pas une mesure) : gain d'octets du décodage spéculatif à brouillon d'acceptation alpha par jeton, long de gamma jetons, de coût relatif c (octets lus …	23
outils/mimetisme/mathematiques/t2bis.py	Témoin T2bis (amendement du 2026-10-26) : PB avec le prédicteur appris, relevé par l'indicateur vrai du prochain jeton. Seuil déclaré …	19
outils/mimetisme/physique/e1_e2.py	E1 (noyaux de mémoire du cache) et E2 (nombres d'occupation) : seuils dans mesures/mimetisme/physique/00-seuils-declares.md (commit 59eb5a2, avant toute …	188
outils/mimetisme/physique/e3.py	E3 : admission par seuil de score (BYPASS) contre LFU ordinaire à capacité égale de pointe. Seuil …	66
outils/mimetisme/physique/noyau-cache/src/main.rs	Cache global d'experts à score par noyau de mémoire (rôle éclaireur physique, 2026-10-26).	213
outils/mimetisme/terre_cosmos/alloc_lfu.py	Post hoc, hors seuil déclaré : M-A avec une LFU (demi-vie 32 jetons) par couche au lieu d'un LRU. …	65
outils/mimetisme/terre_cosmos/cache_terre.py	Éclaireur Terre et cosmos : deux politiques de cache d'experts inspirées de la Terre, jugées sur les traces de routage commitées (seuils …	376
outils/mimetisme/terre_cosmos/eau_usage.py	Éclaireur Terre et cosmos : eau-filling par l'usage (addendum 2 de seuil-declare.md, commité avant).	172
outils/mimetisme/terre_cosmos/lfu_demivie.py	Post hoc, hors seuil déclaré : la LFU de la base (demi-vie 8 jetons) est-elle réglée au bon endroit ? Taux de succès moyen sur les quatre genres (cache vide au …	31
outils/mimetisme/terre_cosmos/phase_120b_reduit.py	M-C sur le 120b, version réduite (la passe complète coûte 150 s par cas et a été arrêtée après 3 cas sur 16) …	42
outils/mimetisme/terre_cosmos/phase_terre.py	Éclaireur Terre et cosmos : M-C, éviction par temps estimé jusqu'à la prochaine utilisation.	160
outils/mimetisme/terre_cosmos/t2_melange.py	Témoin T2 (cassure) déclaré dans seuil-declare.md : ordre des jetons mélangé, l'avance de M-B sur la LFU doit tomber dans +-0,5 point …	29
outils/mimetisme/vivant/ablation_demivie.py	Contrôle de E-A (non déclaré avant, dit tel) : le gain d'ACT-R contre LFU8 vient-il seulement d'une mémoire plus longue ?	55
outils/mimetisme/vivant/bancs_traces.py	Bancs de l'éclaireur du vivant, sur les traces de routage de gpt-oss (aucun fichier de modèle ouvert).	766
outils/mimetisme/vivant/casser_temoins.py	Cassures des témoins de bancs_traces.py : chaque témoin doit pouvoir tomber.	87
outils/mimetisme/vivant/eviction_episodique.py	E-L : éviction pilotée par la mémoire épisodique (cosinus exact, pas de deux jetons). Seuil : seuil-declare.md (E-L). Usage …	105
outils/mimetisme/vivant/eviction_synergies.py	E-J : éviction pilotée par le prédicteur de synergies (pas de deux jetons). Seuil : seuil-declare.md (E-J). Usage …	146
outils/mimetisme/vivant/memoire_episodique.py	E-K : mémoire épisodique clairsemée (la mouche), cosinus exact. Seuil : seuil-declare.md (E-K). Usage : python -I memoire_episodique.py DOSSIER_TRACES	105

Outil	Sa propre description	Lignes
`outils/mimetisme/vivant/poids_stats.py`	Statistiques complémentaires de E-C : forme de la loi des poids de mélange (descriptif). Usage : python -I poids_stats.py DOSSIER	18
`outils/mimetisme/vivant/predicteur_synergies.py`	E-I : un prédicteur de synergies, appris sur d'autres genres, devine-t-il les experts du jeton suivant ? Seuil et témoins …	201
`outils/mimetisme/vivant/residence_partielle.py`	E-G : résidence partielle (« polymérase en pause »). Seuil et prédiction : seuil-declare.md, ajout du 2026-10-27.	47
`outils/mimetisme/vivant/surprise_mode.py`	E-H : le signal de surprise prédit-il où le mode rapide perd de la KL ? Seuil : seuil-declare.md (ajout E-H).	91
Ch. 8 : la synthèse de la ronde		
`outils/genome/synthese/frontiere.py`	Synthèse de la ronde 1 du génome des poids : la frontière bits par poids contre KL sur le texte.	340
Ch. 9 : le codon (lot A)		
`outils/genome/construction/codon/analyse_codon.py`	Analyse du lot A (codon) : tailles v1.2 contre v1.0, et verdicts du J1 ABBA.	175
`outils/genome/construction/codon/copie_temoin.sh`	Témoin de la disposition sur le disque (lot A, codon) : copie neuve du v1.0 du 120b, SHA-256 des deux fichiers par Get-FileHash (chemin indépendant), puis les …	16
`outils/genome/construction/codon/lancer.ps1`	Lance trc-codeur (cible du lot A, codon), sorties redirigées ; note l'alimentation (Win32_Battery.BatteryStatus …	22
`outils/genome/construction/codon/pilote_codon.py`	Pilote du lot A (codon synonyme du zéro, format .trc v1.2) : critère A-C1 (logits au bit) et jalon J1 refait sur le 120b en v1.2 contre v1.0 (A-C3 vitesse, …	316
`outils/genome/construction/codon/recoder.sh`	Lot A (codon) : recodage v1.2 et vérification C1 des deux modèles, dans l'ordre. 1. témoin d'identité …	40
`outils/genome/construction/codon/recoder_temoin_120b.sh`	Séance s2 du lot A (codon) : le v1.0 du 120b recodé aujourd'hui par trc-codeur (témoin d'identité du codeur modifié sur le 120b, et témoin de disposition pour …	17
`outils/genome/construction/codon/tables_octets.py`	Les fréquences des tables CODES_CTX lues dans les fichiers .trc, v1.0 et v1.2 (figure F2 du mémoire : la carte des 256 octets avant et après le fondu du zéro).	91
`outils/genome/construction/codon/verrou.py`	Le verrou de la carte pour le lot A (codon) : prendre, rendre, état.	46
Ch. 10 : le génome synonyme (lot B)		
`outils/genome/synonyme/apparier.py`	Différence appariée de deux passes KL (lot B, génome synonyme ; règle déclarée dans mesures/genome/construction/synonyme/00-seuils-declares.md).	117
`outils/genome/synonyme/coder_768.sh`	Lot B, pas S1 : la mutation synonyme ($\lambda = 1$, $\delta^* = 1$) et la base frontière des 768 experts de gpt-oss-20b, par l'outil de l'éclaireur tel quel …	46
`outils/genome/synonyme/donnees_figures.py`	Données des figures du lot B (génome synonyme) pour le mémoire : la transformation du synonyme vue dans la base propre de H (avant : frontière ; après …	97
`outils/genome/synonyme/evaluer_moteur.sh`	Lot B, S3-K et S3-KL : la KL du moteur (carte, f32) sur les 10 fenêtres de R1, sous le verrou de la carte et sur secteur …	22

Outil	Sa propre description	Lignes
outils/genome/synonyme/kl_passe.sh	Lot B, pas S2 : la KL de bout en bout d'un dossier de .sym substitués (768 experts), sur les 10 fenêtres de R1, par la référence f64 (genome-capture kl), puis …	20
outils/genome/synonyme/mode_rapide_l0.sh	Lot B, après B2 (λ = 0 meilleure à 768 experts, 2026-10-21) : le compagnon rapide de la variante λ = 0, sa vérification S3-T1 (et sa cassure), la KL du moteur …	37
outils/genome/synonyme/pilote_vitesse.py	Lot B, pas S3-V : la vitesse du mode rapide contre le mode exact sur gpt-oss-20b, séance ABBA.	183
outils/genome/synonyme/proxy_120b.sh	Lot B, 120b (frontière seulement ; seuils : 00-seuils-declares.md § 120b, déclarés avant) : T2 …	51
outils/genome/synonyme/rapide/src/main.rs	trc-rapide : le fichier compagnon du mode rapide (lot B, génome synonyme ; genome/05-synthese.md § 4.8, genome/07-genome-synonyme.md).	498
outils/genome/synonyme/repere_llama.sh	Lot B, critère B2 (ii) : le repère de l'outil de référence. La requantification llama.cpp b11515 des seuls tenseurs d'experts en Q2_0 (2,25 bits par poids, …	18
outils/genome/synonyme/survie_kl.py	Données de la figure F-S2 du lot B : la fonction de survie de la KL par position, P(KL > t), sur une grille logarithmique de t, pour plusieurs passes juge kl …	43
outils/genome/synonyme/tableau_variantes.py	Lot B : la table des variantes à 768 experts (figure F-S8), tirée des journaux commités.	33
outils/genome/synonyme/verifier_sym.py	Second chemin du lot B (génome synonyme) : relire les fichiers .sym et recalculer, sans le code qui les a écrits, ce que l'éclaireur a publié …	225
Ch. 11 : le juge de la construction		
outils/genome/construction/juge/construire.bat	Juge de la construction : construit depuis main (arbre du depot) le moteur, juge et genome-capture dans la cible du juge, par la recette du SOCLE …	14
outils/genome/construction/juge/decodeur_juge.py	Le décodeur indépendant du juge de la construction : le format .trc (v1.0 et v1.2), lu et décodé depuis la seule description de decisions/ARCHITECTURE.md §§ …	705
outils/genome/construction/juge/donnees_juge.py	Les CSV du juge pour le mémoire (Tome III), tirés des journaux commités du juge.	85
outils/genome/construction/juge/evaluer_juge.sh	Juge de la construction : les passes troncons evaluer –flux sur les fenêtres neuves (J-C1 et J-K, chemin du moteur), sous le verrou de la carte et sur …	27
outils/genome/construction/juge/fenetres_juge.py	Les fenêtres neuves du juge : dix fenêtres de 512 jetons de wiki.test que les lots n'ont pas vues.	170
outils/genome/construction/juge/kl_juge.py	La KL du juge, calculée par son propre code depuis les fichiers .kld (format de llama-perplexity).	179
outils/genome/construction/juge/logits_juge.py	Les logits au bit (J-C1) : comparaison des empreintes de logits de deux passes troncons evaluer, et la copie à biais retournés de la cassure.	105
outils/genome/construction/juge/reference_juge.sh	Juge de la construction : la référence f64 des fenêtres neuves, puis la référence f64 substituée (le chemin des lots, genome-capture kl –sub) pour λ = 0, la …	38
outils/genome/construction/juge/routage_juge.py	Les experts routés par les fenêtres neuves que R1 n'a jamais routés (J-K, « ce que je publie de plus »).	52

Outil	Sa propre description	Lignes
`outils/genome/construction/juge/seance_juge.py`	La séance de vitesse du juge : trois fichiers par modèle, ordre équilibré, machine calme relevée.	269
`outils/genome/construction/juge/verrou_juge.py`	Le verrou de la carte pour le juge de la construction : prendre, rafraîchir, rendre, état.	59
Ch. 12 : la géométrie tropicale		
`outils/pistes/tropicale/cascades.py`	Exploration, après le verdict (ne décide rien, seuil-declare.md § 5) : les cascades de bascules.	114
`outils/pistes/tropicale/figures.py`	Données des figures de la garde tropicale (Tome III), tirées des journaux commités, et la coupe réelle du routeur (régions top-4 et bande de garde) rendue par …	228
`outils/pistes/tropicale/garde.py`	La courbe de la garde tropicale : (part des cellules sous τ, part des bascules sous τ).	123
`outils/pistes/tropicale/indices.py`	La garde à indices multiples (mémoire du conflit) : protocole de genome/pistes/tropicale/seuil-declare-indices.md.	236
`outils/pistes/tropicale/indices_complements.py`	Compléments de la garde à indices multiples, APRÈS le verdict (étiquetés exploratoires).	99
`outils/pistes/tropicale/juge/juge.py`	Juge indépendant de la garde tropicale : le chiffre principal refait par un autre chemin.	263
`outils/pistes/tropicale/juge/juge_classes.py`	Juge : l'autre classe de modèle (probabilités par classes) refaite, et sa sensibilité aux ex aequo.	94
`outils/pistes/tropicale/juge/juge_indices.py`	Juge indépendant de la garde à indices multiples (la mémoire du conflit).	274
`outils/pistes/tropicale/juge/juge_modele.py`	Juge de la garde tropicale : deux contrôles du théorème du nombre de garde, par quadrature seule.	104
`outils/pistes/tropicale/juge/juge_w4.py`	Juge : le témoin W4 (M3 entraîné sur étiquettes mélangées) sur vingt graines.	49
`outils/pistes/tropicale/lois.py`	Ce que 05 § 3.1 dit non mesuré, lu sur les mêmes journaux (seuil-declare.md § 5) :	273
`outils/pistes/tropicale/routage.py`	Lecture des journaux de routage du lot R, et la cohabitation avec le flux qui mesure des vitesses.	114
`outils/pistes/tropicale/second_chemin.py`	Second chemin du point principal de la garde tropicale, indépendant du chemin numpy.	61
`outils/pistes/tropicale/second_chemin_indices.py`	Second chemin de la garde à indices multiples (seuil-declare-indices.md § 6).	137
`outils/pistes/tropicale/second_chemin_racines.py`	Second chemin, en Python pur (lecture de bascules.py, lot R), du chiffre exploratoire des racines …	52
`outils/pistes/tropicale/temoins.py`	Témoins T1, T2, T3 de la garde tropicale (genome/pistes/tropicale/seuil-declare.md § 4).	114
`outils/pistes/tropicale/theoreme.py`	Le nombre de garde $\kappa = \sigma$ / E[m] : forme close de la courbe de la garde sous deux hypothèses (marge exponentielle de taux λ, bascule si la perturbation …	94
`outils/pistes/tropicale/verifier_chiffres.py`	Relit les chiffres principaux de garde-tropicale.md contre les fichiers commités qui les portent. Chaque ligne …	54
Ch. 13 : les spectres		
`outils/pistes/spectres/ajuster.py`	Piste spectres, pas 3 : ajustement CSN des 48 spectres réels et de leurs témoins T-perm.	72
`outils/pistes/spectres/alpha_levier.py`	Piste spectres : l'exposant $\hat{\alpha}$ prédit-il le levier de rang payé ? (exploratoire, NON déclaré avant)	56

Outil	Sa propre description	Lignes
outils/pistes/spectres/apres-chaine.sh	Piste spectres : lance le test de prédiction (seuil.md § 7 octies) une fois la chaîne levier3 normalise finie (« fin normalise » dans chaine.log), pour ne …	14
outils/pistes/spectres/chaine.sh	Piste spectres : enchaîne les calculs, sans jamais démarrer pendant une séance de vitesse. …	58
outils/pistes/spectres/commun.py	Piste spectres : ce que partagent les outils (priorité, garde du verrou, lecture des matrices).	152
outils/pistes/spectres/csn.py	Loi de puissance par la méthode de Clauset, Shalizi et Newman (arXiv 0706.1062), écrite de zéro.	144
outils/pistes/spectres/fig_plafond.py	Piste spectres : CSV des figures du plafond (P1 : plafond par rang ; P2 : $\hat{\alpha}$ contre levier payé).	66
outils/pistes/spectres/fig_rangee.py	Piste spectres : CSV de la figure P3 (d'où vient le plafond : échelles de rangée, artefact MP, structure).	56
outils/pistes/spectres/juge/comparer.py	Juge de la piste « spectres » : comparaison du chemin du juge au tableau publié (ajustements.out).	103
outils/pistes/spectres/juge/juge_csn.py	Juge de la piste « spectres » : le chiffre principal refait par un chemin propre.	344
outils/pistes/spectres/juge/juge_npz.py	Juge de la piste « spectres » : les spectres du npz viennent-ils bien du modèle ?	91
outils/pistes/spectres/levier.py	Piste spectres, pas 4 : le spectre ouvre-t-il un levier de compression ? (seuil.md § 7)	287
outils/pistes/spectres/levier_l3.py	Piste spectres : L3 v3, le quantificateur scalaire à tramage soustractif (seuil.md § 7 ter).	94
outils/pistes/spectres/normalise.py	Piste spectres : d'où vient la queue ? (mesure exploratoire, NON déclarée avant ; dite comme telle)	62
outils/pistes/spectres/plafond.py	Piste spectres : le plafond spectral (seuil.md § 7 quinquies), sur les valeurs singulières sauvées.	256
outils/pistes/spectres/plafond_rang.py	Piste spectres : le plafond par rang, offert ou payé (seuil.md § 7 septies).	140
outils/pistes/spectres/plafond_rangee.py	Piste spectres : le plafond au-delà des échelles de rangée (seuil.md § 7 nonies et § 7 decies).	167
outils/pistes/spectres/prediction.py	Piste spectres : l'exposant $\hat{\alpha}$ prédit-il le levier de rang payé ? (seuil.md § 7 octies)	133
outils/pistes/spectres/puissance_critere.py	Piste spectres : la puissance du critère déclaré, mesurée sur des lois de puissance vraies.	56
outils/pistes/spectres/rangee_graine.py	Piste spectres : stabilité de la référence « permuté par rangée » d'une graine à l'autre (seuil.md § 7 undecies).	64
outils/pistes/spectres/rapport.py	Piste spectres : les chiffres du livrable, lus dans les journaux commités (rien de mémoire).	253
outils/pistes/spectres/second_chemin.py	Piste spectres : le second chemin du chiffre principal (seuil.md § 6).	186
outils/pistes/spectres/spectres_svd.py	Piste spectres, pas 2 : les valeurs singulières complètes de l'échantillon et de ses témoins.	86
outils/pistes/spectres/temoins.py	Piste spectres : les témoins de l'instrument (seuil.md § 5), et leur cassure.	60
Les figures : espaces latents et juges des figures		
outils/genome/tome3/juge-construction/lectures.py	Juge des figures du lot « construction » : les lectures (ce qui serait faux ou trop beau).	198

Outil	Sa propre description	Lignes
outils/genome/tome3/juge-construction/points.py	Juge des figures du lot « construction » : un point tiré au hasard par série tracée, contrôlé à la source.	296
outils/genome/tome3/juge-construction/recalcul.py	Juge des figures du lot « construction » du Tome III (ch. 8 à 10) : les nombres des légendes.	447
outils/genome/tome3/juge-construction/regenerer.py	Juge des figures du lot « construction » : régénérer les données des figures et les comparer au bit.	85
outils/genome/tome3/juge-design/palette.py	Juge de design du Tome III : distances entre les couleurs du préambule et des lots, en vision normale, deutéranope, protanope (Machado, Oliveira, Fernandes …	71
outils/genome/tome3/juge-design/planche.py	Planche : rend des pages d'un PDF, découpe la zone utile (fractions), les colle en grille. Usage …	42
outils/genome/tome3/juge-design/zoom.py	Rend une page d'un PDF à une résolution donnée et en découpe une zone (fractions de page). Usage …	34
outils/genome/tome3/juge-final/listing.py	Juge final : le listing du fondu (chapitre 08) contre les sources Rust commitées, par mon propre extrait (git show main:..., lignes 67 à 77 et 317 à 327), …	31
outils/genome/tome3/juge-final/parite_ef.py	Juge final : dans un fichier bilingue (\EF{anglais}{français}), compare les nombres et les renvois de chaque paire. Usage …	34
outils/genome/tome3/juge-final/parite_nombres.py	Juge final du Tome III : compare, fichier par fichier, les nombres du français et de l'anglais. Chemin propre au juge (aucun outil des traducteurs repris) …	199
outils/genome/tome3/juge-final/refaire_neufs.py	Juge final du Tome III : refait, par mon propre chemin (lecture directe des JSON et CSV commités), les chiffres neufs ou déplacés depuis …	211
outils/genome/tome3/juge-final/resumes.py	Juge final : les deux blocs du résumé (Résumé, Abstract) sont-ils les mêmes, au caractère près, dans fr/resume.tex et en/resume.tex ? Usage …	25
outils/genome/tome3/juge-final/typo_en.py	Juge final : typographie anglaise du Tome III (sources anglaises), relevé mécanique, hors commentaires. Relève : espace avant « …	28
outils/genome/tome3/juge-latents/juge_commun.py	Juge du lot « espaces latents » du Tome III : lectures communes, écrites de zéro.	150
outils/genome/tome3/juge-latents/juge_etats.py	Juge du lot « espaces latents » : les nombres de III-7.6 (l'espace latent des 6 132 états de la couche 23), refaits de zéro depuis la capture h-L23.f32, le …	227
outils/genome/tome3/juge-latents/juge_etats_marges.py	Juge du lot « espaces latents », III-7.6 : ce que dit la taille des points.	76
outils/genome/tome3/juge-latents/juge_rangee_nuage.py	Juge du lot « espaces latents » : les nombres de III-9.1 (c) (la rangée) et de III-9.7 (le nuage), refaits de zéro depuis le GGUF, les .sym du lot B, les …	226
outils/genome/tome3/juge-latents/juge_regen.py	Juge du lot « espaces latents » : compare au bit les fichiers régénérés par les scripts du lot (dans une racine de brouillon, jamais dans le dépôt) aux …	109
outils/genome/tome3/juge-redaction/assembler.py	Juge de la rédaction du Tome III (français) : assemble le tome à juger dans un dossier d'essai.	83
outils/genome/tome3/juge-redaction/barres.py	Juge de la rédaction du Tome III (français) : les chiffres barrés dans les sources reviennent-ils, non barrés, dans le texte ?	112

Outil	Sa propre description	Lignes
outils/genome/tome3/juge-redaction/calculs.py	Juge de la rédaction du Tome III (français) : refait les calculs neufs des rédacteurs (étiquetés \calc, qu'aucune source n'avait posés), par un second chemin, …	67
outils/genome/tome3/juge-redaction/construire.sh	Juge de la rédaction du Tome III (français) : construit le tome assemblé par assembler.py, comme memoire/construire.sh tome3-fr le construit (même latexmkrc, …	39
outils/genome/tome3/juge-redaction/enonces.py	Juge de la rédaction du Tome III (français) : chaque énoncé (théorème, proposition, lemme, corollaire) est-il suivi de sa démonstration ? Rend, par chapitre, …	35
outils/genome/tome3/juge-redaction/journal.py	Juge de la rédaction du Tome III (français) : lit le journal de la dernière passe de lualatex du tome assemblé et rend, par fichier source, les boîtes trop …	45
outils/genome/tome3/juge-redaction/paralogues.py	Juge de la rédaction du Tome III (français) : refait la correction neuve du chapitre 4 (rédacteur A), « substituer n'est jamais meilleur que retirer l'expert » …	36
outils/genome/tome3/juge-redaction/refaire.py	Juge de la rédaction du Tome III (français) : refait, par un chemin à moi, chaque nombre tiré par tirage.py (graine 20261029, journaux/tirage.csv), depuis le …	247
outils/genome/tome3/juge-redaction/statique.py	Juge de la rédaction du Tome III (français) : contrôles statiques du tome assemblé, sans compiler.	183
outils/genome/tome3/juge-redaction/tirage.py	Juge de la rédaction du Tome III (français) : tire au hasard les chiffres du texte à refaire.	98
outils/genome/tome3/juge-redaction/typographie.py	Juge de la rédaction du Tome III (français) : les décimaux à point qui s'imprimeraient tels quels.	62
outils/genome/tome3/juge-ronde/constats.py	Juge du lot « ronde » : les nombres des constats du jugement qui ne sont pas des clés du lot.	150
outils/genome/tome3/juge-ronde/echantillon.py	Juge du lot « ronde » : un point tiré au hasard dans chaque série tracée, contrôlé à la source.	251
outils/genome/tome3/juge-ronde/refaire.py	Juge du lot « ronde » (Tome III, figures des ch. 1 à 4 et 7) : chaque nombre refait par un autre chemin.	589
outils/genome/tome3/juge-ronde/regenerer.py	Juge du lot « ronde » : régénère les CSV du lot par ses propres scripts et les compare au bit.	83
outils/genome/tome3/juge-ronde/zeros.py	Juge du lot « ronde » : les nombres imprimés qui perdent un zéro final (fixed sans zerofill).	26
outils/genome/tome3/juge2-construction/cadres.py	Juge des chiffres du second flux, lot « construction » : chaque cadre de chaque figure contre ses données.	191
outils/genome/tome3/juge2-construction/points.py	Juge des chiffres du second flux, lot « construction » : un point tiré au hasard dans chaque série NEUVE ou CHANGÉE des CSV du lot, recalculé à la source …	193
outils/genome/tome3/juge2-construction/refaire.py	Juge des chiffres du second flux, lot « construction » : refait chaque nombre NEUF ou DÉPLACÉ des trois fichiers *-nombres.tex (codon, synonyme, juge) depuis …	400
outils/genome/tome3/juge2-construction/regenerer.py	Juge des chiffres du second flux, lot « construction » : régénère au bit les fichiers de données du lot.	159
outils/genome/tome3/juge2-design/cadres.py	Juge de design du second flux (Tome III) : deux critères déclarés (JUGEMENT-figures2-design.md § 1.3), mesurés sur les fichiers commités, par un code à moi. …	67

Outil	Sa propre description	Lignes
outils/genome/tome3/juge2-design/couleurs.py	Juge de design du second flux (Tome III) : les couleurs RENDUES d'une page de pilote, et leurs distances.	180
outils/genome/tome3/juge2-design/lot.py	Juge de design du second flux (Tome III) : couleurs rendues de chaque page de figure des pilotes (français), par couleurs.py, et les paires d'entités NOMMÉES …	34
outils/genome/tome3/juge2-design/tetes.py	Juge de design du second flux (Tome III) : relevé des légendes dans les fichiers *-figures.tex. …	47
outils/genome/tome3/juge2-design/vues.py	Juge de design du second flux (Tome III) : une zone d'une page en trois vues côte à côte, couleur, gris (luminance relative) et deutéranope (Machado, Oliveira …	48
outils/genome/tome3/juge2-latents/juge2_barrees.py	Juge des chiffres du lot « espaces latents », second flux : une phrase barrée revient-elle dans une légende ?	91
outils/genome/tome3/juge2-latents/juge2_compiler.py	Juge des chiffres du lot « espaces latents », second flux : compiler le pilote du lot, en français et en anglais, sur les fichiers du commit jugé, et relever …	91
outils/genome/tome3/juge2-latents/juge2_etats.py	Juge des chiffres du lot « espaces latents », second flux : III-7.6 (petits multiples, marge z(1) − z(2), bords des groupes, noms des experts), refait à la …	275
outils/genome/tome3/juge2-latents/juge2_lecteurs.py	Juge des chiffres du lot « espaces latents », second flux : les lectures à la source, écrites de zéro.	141
outils/genome/tome3/juge2-latents/juge2_nuage_rangee.py	Juge des chiffres du lot « espaces latents », second flux : III-9.7 (fonction d'appui, rose, cône, barres) et III-9.1 (d) (la rangée, sa bande, sa phrase de …	350
outils/genome/tome3/juge2-latents/juge2_regen.py	Juge des chiffres du lot « espaces latents », second flux : la régénération au bit des données de figure.	144
outils/genome/tome3/juge2-ronde/barrees.py	Juge des chiffres du lot « ronde », second flux : une phrase barrée revient-elle dans une légende ?	70
outils/genome/tome3/juge2-ronde/cles.py	Juge des chiffres du lot « ronde », second flux : l'inventaire des nombres neufs ou déplacés.	151
outils/genome/tome3/juge2-ronde/langues.py	Juge des chiffres du lot « ronde », second flux : les nombres des deux langues, page par page.	61
outils/genome/tome3/juge2-ronde/refaire2.py	Juge des chiffres du lot « ronde », second flux : chaque nombre neuf ou déplacé, refait par un autre chemin.	426
outils/genome/tome3/juge2-ronde/regenerer2.py	Juge des chiffres du lot « ronde », second flux : les données du lot se régénèrent-elles au bit ?	114
outils/genome/tome3/juge2-ronde/series2.py	Juge des chiffres du lot « ronde », second flux : les séries neuves, point par point.	274
outils/genome/tome3/juge3/comparer_pdf.py	Juge de la troisième passe : compare deux PDF page à page, par les mots et leurs boîtes (PyMuPDF) et par l'image (rendu en gris à 50 dpi par PyMuPDF, le même …	70
outils/genome/tome3/juge3/compiler_pilotes.py	Juge de la troisième passe : copie les quatre pilotes du Tome III (memoire/essai/pilote-tome3-*.tex, non commités) dans memoire/essai/juge3/pilotes/ et les …	59
outils/genome/tome3/juge3/construire_tomes.sh	Juge de la troisième passe : construit les Tomes I et II tels que commités (préambule réparé), sous MiKTeX tel qu'installé, sans toucher les PDF commités. …	25
outils/genome/tome3/juge3/croisements.py	Juge de la troisième passe : sur une page de PDF, relève les traits dessinés (segments droits des chemins vectoriels, par PyMuPDF get_drawings) qui traversent …	72

Outil	Sa propre description	Lignes
outils/genome/tome3/juge3/croisements_vite.py	Juge de la troisième passe : version rapide de croisements.py pour toutes les pages d'un PDF. …	69
outils/genome/tome3/juge3/extraire_correctifs.py	Juge de la troisième passe : extrait du préambule commité (memoire/commun/preambule.tex, tel que dans git HEAD) les lignes des deux correctifs, mot pour mot, …	36
outils/genome/tome3/juge3/figure3.py	Juge de la troisième passe : la zone de la figure d'une page de pilote (du haut du texte à la ligne « Figure n.m »), rendue par zoom3.py en couleur, gris et …	21
outils/genome/tome3/juge3/gd1_bleu.py	Juge de la troisième passe : GD1, le bleu au rendu. Pour chaque page de figure donnée, rend la zone de la figure (du haut de la page au début de la légende, « …	74
outils/genome/tome3/juge3/minimal_variantes.py	Juge de la troisième passe : compile les variantes du document minimal (deux passes chacune) et relève, pour chacune, les erreurs du journal (lignes « ! »), …	31
outils/genome/tome3/juge3/paires3.py	Juge de la troisième passe : refait, par une colorimétrie écrite ici (sRGB → CIELAB D65 ; gris = luminance Rec. 709 affichée en gris …	73
outils/genome/tome3/juge3/refaire3.py	Juge de la troisième passe : refait, par mon code, chaque nombre neuf ou déplacé des retouches (7a40c54, 6137230, 1482dbe), depuis les fichiers commités sous …	290
outils/genome/tome3/juge3/regenerer3.py	Juge de la troisième passe : régénère les données des figures du Tome III par les scripts commités, dans une copie tirée par « git -c core.autocrlf=false …	125
outils/genome/tome3/juge3/sources_latents.py	Juge de la troisième passe : refait à la source, par un troisième chemin, les nombres neufs de la tête de III-9.7 (rapport ‖ΔW_s x‖ / ‖ΔW_f x‖ sur les 1 329 …	149
outils/genome/tome3/juge3/tetes3.py	Juge de la troisième passe : DN1 au rendu. Pour chaque légende des pilotes (bloc « Figure 1.n »), prend la tête (du début jusqu'à l'étiquette de provenance « …	37
outils/genome/tome3/juge3/zoom3.py	Juge de la troisième passe : rend une zone d'une page (en points PDF) à fort agrandissement par PyMuPDF (clip), en couleur, et, sur demande, en gris et en …	43
outils/genome/tome3/latents_commun.py	Lectures communes du lot « espaces latents » du Tome III (figures III-9.1 (c) et III-9.7).	98
outils/genome/tome3/latents_etats.py	L'espace latent des états de la couche 23 de gpt-oss-20b (Tome III, figure III-7.6).	365
outils/genome/tome3/latents_nuage.py	Le nuage des entrées réelles d'un expert sur les trois premiers axes propres de H, et les ellipses de l'erreur de sortie de la frontière et du synonyme (Tome …	152
outils/genome/tome3/latents_rangee.py	Une rangée de 32 poids, avant et après la mutation synonyme (Tome III, figure III-9.1 (c)).	122
Le mémoire du Tome III		
memoire/outils/tome3_annexes.py	Produit les tables des annexes du Tome III depuis le commit HEAD du dépôt (git ls-tree, git cat-file, git log).	386
memoire/outils/tome3_cassures.py	Cassures des gardes neuves de la troisième passe des figures du Tome III (SOCLE : « tout témoin neuf doit pouvoir échouer »). …	101

Outil	Sa propre description	Lignes
memoire/outils/tome3_codon.py	Données des figures du chapitre 8 du Tome III (lot A, le codon synonyme du zéro), tirées des fichiers commités (aucun chiffre recopié à la main).	456
memoire/outils/tome3_duplication.py	Données des figures du chapitre 4 du Tome III (la duplication de gènes), tirées des fichiers commités de mesures/genome/duplication/ (aucun chiffre recopié à …	320
memoire/outils/tome3_eclaireur.py	Données des figures des chapitres 1 et 2 du Tome III (la question du génome, l'éclaireur), tirées des journaux commités de mesures/genome/etat-de-l-art/ (aucun …	141
memoire/outils/tome3_genome.py	Données des figures du chapitre 3 du Tome III (le génome des poids : le brin lu par codons), tirées des fichiers commités de mesures/genome/genome/ (aucun …	178
memoire/outils/tome3_juge.py	Données des figures du juge de la construction pour le Tome III (III-8.8, III-10.1, III-10.3), tirées des fichiers commités sous …	246
memoire/outils/tome3_latents.py	Données des figures du lot « espaces latents » du Tome III (III-7.6, III-9.1 (d), III-9.7), tirées des fichiers commités de mesures/genome/tome3/latents/ …	573
memoire/outils/tome3_palette.py	Le dictionnaire des couleurs du Tome III, chiffré : pour chaque paire que les figures demandent de distinguer, ΔE CIE76 (CIELAB, D65) en vision normale, …	321
memoire/outils/tome3_synonyme.py	Données des figures du chapitre 9 du Tome III (lot B, le génome synonyme), tirées des fichiers commités (aucun chiffre recopié à la main).	385
memoire/outils/tome3_synthese.py	Données des figures du chapitre 7 du Tome III (la synthèse de la ronde : frontière commune, erreur d'expert contre KL, deux parts de la KL, colonnes géantes), …	370
Total	332 outils	49 577

B.3 Chaque figure et les fichiers qu'elle lit

Une figure belle reste une figure vraie : elle se tire des données commitées, par un script commité. Tableau B.2 donne, pour chaque figure au style du Tome III, les fichiers que sa ligne « Données » cite ; ce sont ceux que lit l'outil du lot nommé à la fin de la ligne. Les figures hors du style commun (le mimétisme, les deux pistes) citent leurs données dans leur légende, au chapitre qui les place.

Table B.2 : Chaque figure au style du Tome III (son \Source) et les fichiers de données que cite sa ligne « Données » (memoire/outils/tome3_annexes.py ; un nom sans dossier se lit dans le dossier du précédent).

Figure (fichier, nom)	Données
codon, CodonTheoreme	mesures/genome/construction/codon/tailles-par-tenseur.csv, memoire/outils/tome3_codon.py
codon, CodonOctets	mesures/genome/construction/codon/tables-octets.csv, memoire/outils/tome3_codon.py

Figure (fichier, nom)	Données
codon, CodonRoue	mesures/genome/predictif-elagage/histo-toutes-gptoss20b.npz, mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json, mesures/genome/construction/codon/tailles.json, memoire/outils/tome3_codon.py
codon, CodonAccumulateur	mesures/genome/construction/codon/essais-espace-2026-10-17-a.log, genome/06-codon-v1-2.md, decisions/ARCHITECTURE.md
codon, CodonColonne	mesures/genome/construction/synthese/colonne-par-tenseur.csv, mesures/genome/construction/codon/tailles.json, mesures/genome/construction/synthese/k10/frontiere.csv, memoire/outils/tome3_codon.py
codon, CodonJalon	mesures/genome/construction/codon/j1-s2-passes.csv, j1-s2-verdicts.json, memoire/outils/tome3_codon.py
codon, CodonJalonBis	mesures/disque/placement/juge/seance/Jb3-*, J5-*, J6-*, J1-cartes.json, genome/JUGEMENT-placement.md, mesures/genome/construction/codon/disque-s2.json, mesures/genome/construction/juge/figures/vitesse-passes.csv, disque-passes.csv, genome/JUGEMENT-construction.md, memoire/outils/tome3_codon.py
codon, CodonFormat	decisions/ARCHITECTURE.md, genome/06-codon-v1-2.md
duplication, DupParente	mesures/genome/duplication/csv/scores-parente.csv, paires-temoin-planter-0.5.json, arbre-c23.json, memoire/outils/tome3_duplication.py
duplication, DupArbre	mesures/genome/duplication/csv/phylogenie-c23-noeuds.csv, phylogenie-c23-aretes.csv, phylogenie-c23.json, outils/genome/duplication/figure_phylogenie.py, arbre-c23.json, memoire/outils/tome3_duplication.py
eclaireur, EclBloc	—
eclaireur, EclBorne	mesures/genome/etat-de-l-art/journaux/borne-rd.json, zero-temoin-sans-drapeau.json, zero-canonique.json, mesures/genome/etat-de-l-art/passes/frontiere-lam0-damp0.01-e0_8_16_24.json, memoire/outils/tome3_eclaireur.py
eclaireur, EclZeros	mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json, zero-compte-120b.json, memoire/outils/tome3_eclaireur.py
genome, GenGauss	mesures/genome/genome/gauss-balayage.jsonl, gauss-T1.json, memoire/outils/tome3_genome.py
genome, GenCourbes	mesures/genome/genome/csv/courbes-tourne-l12.csv, courbes-brut-l12.csv, verdict-par-tenseur.csv, mesures/genome/genome/codons-tourne-l12.json, memoire/outils/tome3_genome.py
juge, JconsCodon	mesures/genome/construction/juge/J-T1-echantillon.csv, J-T0-fichiers.json, mesures/genome/construction/codon/tailles.json, memoire/outils/tome3_juge.py

Figure (fichier, nom)	Données
juge, JconsExperts	mesures/genome/construction/juge/figures/routage-grille.csv, J-T0-fichiers.json, mesures/genome/construction/synonyme/kl/kl-frontiere-768-positions.csv, mesures/genome/construction/codon/tailles.json, memoire/outils/tome3_juge.py
juge, JconsFenetres	mesures/genome/construction/juge/figures/kl-par-fenetre.csv, mesures/moteur/O/journaux/o3/c4-20b-comparer.csv, mesures/genome/construction/synonyme/moteur/kl-moteur-frontiere-positions.csv, memoire/outils/tome3_juge.py
latents, LatentsEtats	mesures/genome/tome3/latents/etats-c23.csv, etats-c23.json, outils/genome/tome3/latents_etats.py, h-L23.f32, routage-L23.bin, outils/genome/tome3/juge-latents/juge-etats-marges.json, memoire/outils/tome3_latents.py, latents-temoins.json
latents, LatentsRangee	mesures/genome/tome3/latents/rangee-L08-E00-gate.csv, rangee-L08-E00-gate.json, outils/genome/tome3/latents_rangee.py, mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv, memoire/outils/tome3_latents.py
latents, LatentsNuage	mesures/genome/tome3/latents/nuage-L08-E00-gate-points.csv, nuage-L08-E00-gate-ellipses.csv, nuage-L08-E00-gate.json, outils/genome/tome3/latents_nuage.py, outils/genome/tome3/juge2-latents/journaux/juge2-nuage-rangee.json, memoire/outils/tome3_latents.py, latents-temoins.json
synonyme, SynoBasePropre	mesures/genome/construction/synonyme/figures/spectre-L08-E00-gate.csv, memoire/outils/tome3_synonyme.py
synonyme, SynoQueue	mesures/genome/construction/synonyme/kl/kl-*-768-positions.csv, repere/kl-q2_0-positions.csv, kl/kl-*-768-juge.json, mesures/genome/construction/juge/kl/kl-neuf-*-positions.csv, memoire/outils/tome3_synonyme.py
synonyme, SynoForet	mesures/genome/construction/synonyme/kl/apparie-*-par-fenetre.csv, mesures/genome/construction/juge/figures/apparie-par-fenetre.csv, mesures/genome/construction/synonyme/moteur/kl-moteur-*-positions.csv, memoire/outils/tome3_synonyme.py
synonyme, SynoVariantes	mesures/genome/construction/synonyme/figures/variantes-768.csv, kl/kl-*-768-positions.csv, repere/kl-q2_0-positions.csv, mesures/moteur/O/journaux/o3/c4-20b-comparer.json, mesures/genome/construction/juge/J-T0-fichiers.json, memoire/outils/tome3_synonyme.py
synthese, SynthFrontiere	mesures/genome/construction/synthese/k10/frontiere.csv, outils/genome/synthese/frontiere.py, mesures/genome/etat-de-l-art/passes/synonyme-lam1-damp1-e0_8_16_24.json, memoire/outils/tome3_synthese.py

Figure (fichier, nom)	Données
synthese, SynthErreurKL	mesures/genome/construction/synthese/k10/erreur-contre-kl.csv, frontiere.py, mesures/genome/genome/kl/comparer10-base-contre-recette.json, memoire/outils/tome3_synthese.py
synthese, SynthDeuxParts	mesures/genome/etat-de-l-art/journaux/kl-frontiere-positions.csv, kl-synonyme-d1-positions.csv, kl-frontiere-juge.json, kl-synonyme-d1-juge.json, memoire/outils/tome3_synthese.py
synthese, SynthColonnes	mesures/genome/genome/csv/colonnes-energie-erreur.csv, outils/genome/recette/figures_transformations.py, memoire/outils/tome3_synthese.py

B.4 Les licences

Le SOCLE demande d'écrire la licence de chaque dépendance et de chaque dépôt étudié. Aucun dépôt de code extérieur n'a été lu par la quête du génome : l'état de l'art n'a lu que des articles et leurs résumés (genome/00-etat-de-l-art.md), et aucune licence de code n'y est en jeu. Les dépendances sont celles du tableau B.3, chacune lue dans le paquet installé sur la machine.

Table B.3 : Les licences des dépendances des outils et du mémoire du Tome III, lues dans les paquets installés le 29 octobre 2026.

Dépendance	Licence	Où elle est lue
numpy 2.5.4	BSD-3-Clause (et 0BSD, MIT, Zlib, CC0-1.0 pour des parties)	License-Expression de numpy-2.5.4.dist-info/METADATA
scipy 1.18.1	BSD à trois clauses	scipy-1.18.1.dist-info/LICENSE.txt
rayon 1.12.0, half 2.7.1, anyhow 1.0.104 (Rust)	MIT ou Apache-2.0	champ license de leur Cargo.toml (registre local de Cargo)
llama.cpp b11515	MIT	Tome I, annexe des outils (sources officielles du même commit)
pgfplots	GNU GPL, version 3 ou ultérieure	en-tête de pgfplots.code.tex (MiKTeX)
PGF/TikZ	LPPL et/ou GNU GPL	en-tête de tikz.sty (MiKTeX)
biblatex	LPPL 1.3c	en-tête de biblatex.sty (MiKTeX)

Ce qui pourrait être faux

Ce qui pourrait être faux

- **La première ligne d'une description** est celle que l'outil donne de lui-même ; elle peut avoir vieilli. L'outil des annexes la cite, il ne la juge pas.
- **Le témoin des figures** vérifie que les fichiers cités existent, pas qu'ils sont ceux que l'outil lit vraiment ; c'est la régénération au bit des données (juge de la troisième passe des figures) qui le vérifie.
- **Les licences** sont lues dans les paquets installés ; une autre version peut en avoir une autre.

Annexe C

Journaux

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la règle des deux premiers tomes : ce qui produit un chiffre publié est commité, outils et journaux. **Cette annexe établit** où sont les journaux du Tome III, dossier par dossier ; les séances qui portent un chiffre de vitesse, avec leurs conditions ; et les passes coupées, jetées ou publiées comme non concluantes, avec leur raison, parce qu'aucune ne disparaît en silence. **Il reste** les fichiers lourds hors du dépôt (références f64, `.kld`), régénérables par les outils commités.

C.1 Les dossiers de journaux

Tableau C.1 donne chaque dossier de journaux, de CSV et de notes commité pour ce tome, son nombre de fichiers et son volume (les outils sont à l'annexe B). Elle est engendrée par `memoire/outils/tome3_annexes.py` depuis le commit : le volume d'un dossier est la somme des tailles des objets git de ses fichiers (`git ls-tree -l`), indépendante des fins de ligne de la copie (corrigé le 2026-10-29, annexe B). Les dossiers des pistes (`mesures/pistes/`) et du mimétisme appartiennent à leurs flux ; ceux du juge du placement disque (`mesures/disque/placement/juge/`) sont là pour la réserve R1. Ceux du second tirage de P3, en travail ailleurs, n'y sont pas (R6).

Table C.1 : Les dossiers de journaux commités du Tome III (journaux, CSV, notes ; outils exclus) (`memoire/outils/tome3_annexes.py`).

Dossier	Fichiers	Volume (Mo)
`mesures/disque/placement/juge/`	316	5,91
`mesures/genome/construction/codon/`	178	2,43
`mesures/genome/construction/juge/`	190	4,47
`mesures/genome/construction/synonyme/`	234	4,90
`mesures/genome/construction/synthese/`	10	0,02
`mesures/genome/duplication/`	35	0,95
`mesures/genome/duplication/csv/`	9	0,33
`mesures/genome/etat-de-l-art/`	6	0,03
`mesures/genome/etat-de-l-art/journaux/`	54	0,58
`mesures/genome/etat-de-l-art/passes/`	23	0,11
`mesures/genome/genome/`	39	0,15
`mesures/genome/genome/csv/`	12	10,36
`mesures/genome/genome/kl/`	21	0,26
`mesures/genome/predictif-elagage/`	41	6,15

Dossier	Fichiers	Volume (Mo)
mesures/genome/predictif-elagage/kl/	33	0,05
mesures/genome/predictif-elagage/plans/	37	7,67
mesures/genome/tome3/latents/	7	0,67
mesures/mimetisme/P1/	22	1,75
mesures/mimetisme/P1/juge/	7	0,04
mesures/mimetisme/P2/	13	0,11
mesures/mimetisme/P2/juge/	4	0,03
mesures/mimetisme/P3/	18	0,09
mesures/mimetisme/P3/etage-B/	56	2,47
mesures/mimetisme/P3/juge/	26	2,54
mesures/mimetisme/P3/routeur-ligne/	85	13,19
mesures/mimetisme/P3/series/	4	0,00
mesures/mimetisme/P4/	14	0,38
mesures/mimetisme/P4/figures/	3	0,16
mesures/mimetisme/P4/juge/	6	0,01
mesures/mimetisme/P5/	8	0,63
mesures/mimetisme/P5/etape1-diag/	65	0,05
mesures/mimetisme/P5/etape1-regime/	49	0,04
mesures/mimetisme/P5/etape1/	70	0,15
mesures/mimetisme/P5/figures/	10	0,32
mesures/mimetisme/P5/gibbs/	1	0,00
mesures/mimetisme/P5/juge/	162	0,09
mesures/mimetisme/P6/	11	0,20
mesures/mimetisme/P6/juge/	8	0,03
mesures/mimetisme/P6/temoins/	8	0,01
mesures/mimetisme/humain/	32	0,12
mesures/mimetisme/ingenierie/	9	0,01
mesures/mimetisme/mathematiques/	8	0,03
mesures/mimetisme/physique/	1	0,01
mesures/mimetisme/physique/e1/	17	0,22
mesures/mimetisme/physique/e2/	6	0,00
mesures/mimetisme/physique/e3/	24	0,07
mesures/mimetisme/terre_cosmos/	21	0,10
mesures/mimetisme/tri/	1	0,01
mesures/mimetisme/vivant/	48	0,18
mesures/pistes/spectres/	81	2,52
mesures/pistes/spectres/juge/	9	0,13
mesures/pistes/tropicale/	32	0,17
mesures/pistes/tropicale/figures/	9	0,35
mesures/pistes/tropicale/indices/	4	3,47
mesures/pistes/tropicale/juge/	10	0,03
outils/genome/tome3/juge-construction/	8	0,06
outils/genome/tome3/juge-design/	2	0,00
outils/genome/tome3/juge-final/	14	0,20
outils/genome/tome3/juge-latents/	5	0,02

Dossier	Fichiers	Volume (Mo)
`outils/genome/tome3/juge-redaction/`	18	0,18
`outils/genome/tome3/juge-ronde/`	18	0,11
`outils/genome/tome3/juge2-construction/`	12	0,05
`outils/genome/tome3/juge2-design/`	37	2,68
`outils/genome/tome3/juge2-latents/`	9	0,05
`outils/genome/tome3/juge2-ronde/`	23	0,09
`outils/genome/tome3/juge3/`	47	0,32
Total	2 400	78,56

Ce qui n'est pas dans le dépôt. Les fichiers lourds : les références f64 et les `.kld` des passes de KL (environ 13 Go pour le seul juge de la construction, sous `~/dev/essais/troncons/`), les fichiers `.sym` des 768 experts et les compagnons rapides. Chacun se régénère par un outil commité, et son empreinte est dans un journal commité quand un chiffre en dépend (par exemple les `.kld` de sortie de J-C1, section 11.4).

C.2 Les séances qui portent un chiffre de vitesse

Un chiffre de vitesse ne vaut que par sa séance : la machine, l'alimentation, la charge des autres flux, le verrou de la carte. Tableau C.2 donne celles du tome.

Table C.2 : Les séances qui portent un chiffre de vitesse du Tome III, leurs conditions et ce qu'elles décident [mes.].

Séance	Quand	Conditions	Ce qu'elle décide
lot B, s1 (20b)	17 oct., départ à 11 h 45	A B B A A B B A ; sous la charge de la passe KL B0, du calcul du débit et de l'écriture du compagnon du 120b ($\sigma_A = 17{,}8\,\%$)	publiée, non concluante
lot B, s2 (20b)	22 oct., 1 h 11 – 1 h 25	A B B A A B B A ; deux relevés vides au départ, aucune passe sous charge, secteur, verrou ($\sigma_A = 2{,}3\,\%$)	×1,24 annoncé, barré comme gain du mode (G2)
lot B, 120b-V	22 oct., 2 h 34 – 3 h 38	mêmes règles ($\sigma_A = 6{,}6\,\%$)	×2,00 annoncé, barré comme gain du mode (G1)
juge, séance à trois fichiers (120b puis 20b)	22 oct., moteur de 6 h 17 à 7 h 11	A B R R B A A B R R B A, étage disque d'abord ; aucune des 36 passes sous charge ; secteur à chaque passe ; verrou tenu	$r(R/B) = 1{,}284$ et 0,942 ; J1 redonné (1,442)

Données : `mesures/genome/construction/synonyme/vitesse/pilote-s1.json`, `pilote-s2.json`, `pilote-s2-120b.json` ; `mesures/genome/construction/juge/vitesse/seance-\{120b,20b\}.\{json,log\}`. La séance du lot A est au chapitre du codon ; celles du placement disque, au chapitre « L'étage disque » prévu pour le Tome II.

C.3 Les passes coupées, jetées, ou non concluantes

Aucune passe ne disparaît en silence : celles qui n'ont rien décidé sont publiées comme telles, avec leur raison.

- **La séance s1 du lot B** : publiée, non concluante, parce qu'elle a tourné sous la charge d'autres passes ; refaite machine calme (s2), et c'est ce qui a fait écrire la recette « une séance de vitesse ne part que sur une machine calme relevée » (section 10.7.1).
- **La première passe B de la séance s2** (5,46 jetons/s, juste après l'écriture du compagnon) : gardée dans la médiane, comme la règle déclarée le voulait ; le juge montre qu'avec le bruit des passes B elle aurait empêché l'annonce (constat M4).
- **La passe f64 substituée $\lambda = 0$ du juge** : la boucle bash qui la lançait a été arrêtée, mais le programme natif a fini seul ; sa sortie redirigée était complète et a été gardée sous le nom `sub-10.json`. La passe f64 substituée $\lambda = 1$ n'a pas été lancée (le temps, et le flux du placement qui attendait le calme), et c'est dit à chaque verdict qui en dépend (section 11.5.2).
- **Les copies cassées** (le biais retourné de J-C1, les cassures des figures) sont effacées après usage ; leurs journaux restent.
- **Le second tirage de P3**, en travail dans une autre session, a des tentatives coupées par des causes extérieures (secteur, mémoire de la machine, disque), sans aucune mesure utilisable et sans aucun seuil changé ; ses journaux sont commités par sa session, et le tome n'en tire aucun chiffre (réserve R6).

Ce qui pourrait être faux

> **Ce qui pourrait être faux**
>
> - **Le volume d'un dossier** est celui des fichiers commités, en fins de ligne LF (une copie en CRLF est plus grosse) ; il ne dit rien des fichiers lourds régénérables hors du dépôt.
> - **Le rangement par dossier** suit la profondeur des chemins (comme au Tome II) ; un dossier qui mêle deux étapes y apparaît comme une seule ligne.
> - « **Aucune passe sous charge** » est ce que dit le relevé déclaré (plus de 5 % d'un cœur ou plus de 8 Mo lus en 3 s) ; une charge plus faible, ou entre deux relevés, lui échappe.

Bibliographie

[1] C. E. Shannon. « A Mathematical Theory of Communication ». In : *Bell System Technical Journal* 27.3 (1948), p. 379-423. doi : 10.1002/j.1538-7305.1948.tb01338.x. (Visité le 2026-10-15).

[2] T. M. Cover et J. A. Thomas. *Elements of Information Theory*. 2e éd. Chapitre 10 : théorème 10.3.2 (source gaussienne) et théorème du débit-distorsion. Hoboken, NJ : Wiley-Interscience, 2006. isbn : 978-0-471-24195-9. doi : 10.1002/047174882X. (Visité le 2026-10-15).

[3] F. H. C. Crick, L. Barnett, S. Brenner et R. J. Watts-Tobin. « General Nature of the Genetic Code for Proteins ». In : *Nature* 192.4809 (1961). Lecture du brin par triplets à partir d'un point fixe, sans virgule : le cadre de lecture, p. 1227-1232. doi : 10.1038/1921227a0. (Visité le 2026-10-16).

[4] F. H. C. Crick. « The Origin of the Genetic Code ». In : *Journal of Molecular Biology* 38.3 (1968). Le code génétique, table dégénérée (64 codons pour 20 acides aminés et l'arrêt), p. 367-379. doi : 10.1016/0022-2836(68)90392-6. (Visité le 2026-10-16).

[5] S. J. Freeland et L. D. Hurst. « The Genetic Code Is One in a Million ». In : *Journal of Molecular Evolution* 47.3 (1998-09). Métadonnées Crossref ; résumé de l'éditeur lu., p. 238-248. doi : 10.1007/pl00006381. url : https://doi.org/10.1007/pl00006381 (visité le 2026-10-16).

[6] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025-08-08. arXiv : 2508.10925. url : https://arxiv.org/abs/2508.10925 (visité le 2026-10-16).

[7] B. D. Rouhani, R. Zhao, A. More et al. *Microscaling Data Formats for Deep Learning*. Version v3 du 2023-10-19. Résumé lu (API arXiv). 2023-10-16. arXiv : 2310.10537. url : https://arxiv.org/abs/2310.10537 (visité le 2026-10-16).

[8] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Définit MXFP4 : éléments E2M1, échelle partagée E8M0 par bloc de 32. 2023. url : https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf (visité le 2026-10-15).

[9] C. E. Shannon. « Coding Theorems for a Discrete Source with a Fidelity Criterion ». In : *IRE National Convention Record*. T. 7. 4. Fonction distorsion-débit ; borne de la source gaussienne. 1959, p. 142-163.

[10] R. E. Blahut. « Computation of channel capacity and rate-distortion functions ». In : *IEEE Transactions on Information Theory* 18.4 (1972-07). Métadonnées Crossref., p. 460-473. doi : 10.1109/TIT.1972.1054855. url : https://doi.org/10.1109/TIT.1972.1054855 (visité le 2026-10-16).

[11] S. Kullback et R. A. Leibler. « On Information and Sufficiency ». In : *The Annals of Mathematical Statistics* 22.1 (1951-03). La divergence d'information entre deux lois, positive et nulle seulement pour deux lois égales ; c'est la mesure du juge., p. 79-86. doi : 10.1214/aoms/1177729694. url : https://doi.org/10.1214/aoms/1177729694 (visité le 2026-10-16).

[12] H. Gish et J. N. Pierce. « Asymptotically efficient quantizing ». In : *IEEE Transactions on Information Theory* 14.5 (1968), p. 676-683. doi : 10.1109/TIT.1968.1054193.

[13] M. W. Marcellin et T. R. Fischer. « Trellis Coded Quantization of Memoryless and Gauss-Markov Sources ». In : *IEEE Transactions on Communications* 38.1 (1990), p. 82-93. doi : 10.1109/26.46532. (Visité le 2026-10-15).

[14] J. Lin, J. Tang, H. Tang et al. *AWQ : Activation-aware Weight Quantization for LLM Compression and Acceleration*. Version v6 du 2026-04-25. Résumé lu (API arXiv). 2023-06-01. arXiv : 2306.00978. url : https://arxiv.org/abs/2306.00978 (visité le 2026-10-16).

[15] R. Shafipour, D. Harrison, M. Horton et al. *SeedLM : Compressing LLM Weights into Seeds of Pseudo-Random Generators*. v1 du 2024-10-14, v2 du 2024-10-16 (la plus récente à la consultation). 2024-10-14. arXiv : 2410.10714. url : https://arxiv.org/abs/2410.10714 (visité le 2026-10-16).

[16] P. Nooralinejad, A. Abbasi, S. A. Koohpayegani et al. *PRANC : Pseudo RAndom Networks for Compacting deep models*. Version v2 du 2023-08-28. Résumé lu (API arXiv). 2022-06-16. arXiv : 2206.08464. url : https://arxiv.org/abs/2206.08464 (visité le 2026-10-16).

[17] S. A. Koohpayegani, K. Navaneet, P. Nooralinejad, S. Kolouri et H. Pirsiavash. *NOLA : Compressing LoRA using Linear Combination of Random Basis*. Version v2 du 2024-04-30. Résumé lu (API arXiv). 2023-10-04. arXiv : 2310.02556. url : https://arxiv.org/abs/2310.02556 (visité le 2026-10-16).

[18] C. Li, H. Farkhoor, R. Liu et J. Yosinski. *Measuring the Intrinsic Dimension of Objective Landscapes*. Version v1 du 2018-04-24. Résumé lu (API arXiv). 2018-04-24. arXiv : 1804.08838. url : https://arxiv.org/abs/1804.08838 (visité le 2026-10-16).

[19] D. Ha, A. Dai et Q. V. Le. *HyperNetworks*. v4 du 2016-12-01 (la plus récente à la consultation). 2016-09-27. arXiv : 1609.09106. url : https://arxiv.org/abs/1609.09106 (visité le 2026-10-16).

[20] K. O. Stanley, D. B. D'Ambrosio et J. Gauci. « A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks ». In : *Artificial Life* 15.2 (2009-04). Métadonnées Crossref ; résumé de l'éditeur lu., p. 185-212. doi : 10.1162/artl.2009.15.2.15202. url : https://doi.org/10.1162/artl.2009.15.2.15202 (visité le 2026-10-16).

[21] A. Mordvintsev, E. Randazzo, E. Niklasson et M. Levin. « Growing Neural Cellular Automata ». In : *Distill* 5.2 (2020-02-11). Métadonnées Crossref ; résumé de l'éditeur lu. doi : 10.23915/distill.00023. url : https://doi.org/10.23915/distill.00023 (visité le 2026-10-16).

[22] E. Najarro, S. Sudhakaran, C. Glanois et S. Risi. *HyperNCA : Growing Developmental Networks with Neural Cellular Automata*. Version v1 du 2022-04-25. Résumé lu (API arXiv). 2022-04-25. arXiv : 2204.11674. url : https://arxiv.org/abs/2204.11674 (visité le 2026-10-16).

[23] P. Li, Z. Zhang, P. Yadav et al. *Merge, Then Compress : Demystify Efficient SMoE with Hints from Its Routing Policy*. Version v2 du 2024-03-14. Résumé lu (API arXiv). 2023-10-02. arXiv : 2310.01334. URL : https://arxiv.org/abs/2310.01334 (visité le 2026-10-16).

[24] S. He, R.-Z. Fan, L. Ding, L. Shen, T. Zhou et D. Tao. *Merging Experts into One : Improving Computational Efficiency of Mixture of Experts*. Version v3 du 2023-11-21. Résumé lu (API arXiv). 2023-10-15. arXiv : 2310.09832. URL : https://arxiv.org/abs/2310.09832 (visité le 2026-10-16).

[25] I.-C. Chen, H.-S. Liu, W.-F. Sun, C.-H. Chao, Y.-C. Hsu et C.-Y. Lee. *Retraining-Free Merging of Sparse MoE via Hierarchical Clustering*. Version v4 du 2025-10-26. Résumé lu (API arXiv). 2024-10-11. arXiv : 2410.08589. URL : https://arxiv.org/abs/2410.08589 (visité le 2026-10-16).

[26] L. Li, Z. Qiyuan, J. Wang et al. *Sub-MoE : Efficient Mixture-of-Expert LLMs Compression via Subspace Expert Merging*. Version v1 du 2025-06-29. Résumé lu (API arXiv). 2025-06-29. arXiv : 2506.23266. URL : https://arxiv.org/abs/2506.23266 (visité le 2026-10-16).

[27] H. Gu, W. Li, L. Li et al. *Delta Decompression for MoE-based LLMs Compression*. Version v1 du 2025-02-24. Résumé lu (API arXiv). 2025-02-24. arXiv : 2502.17298. URL : https://arxiv.org/abs/2502.17298 (visité le 2026-10-16).

[28] X. Chen, M. Ha, Z. Lan, J. Zhang et J. Li. *MoBE : Mixture-of-Basis-Experts for Compressing MoE-based LLMs*. Version v1 du 2025-08-07. Résumé lu (API arXiv). 2025-08-07. arXiv : 2508.05257. URL : https://arxiv.org/abs/2508.05257 (visité le 2026-10-16).

[29] D. Li, L. Li, Q. Zhu et al. *OMP-MoE : Efficient Expert Pruning for Mixture-of-Experts LLMs via Orthogonal Matching Pursuit*. Version v2 du 2026-09-29. Résumé lu (API arXiv). 2026-09-09. arXiv : 2609.31631. URL : https://arxiv.org/abs/2609.31631 (visité le 2026-10-16).

[30] J. Liu, G. Xiao, K. Li et al. *BitDelta : Your Fine-Tune May Only Be Worth One Bit*. Version v3 du 2024-10-13. Résumé lu (API arXiv). 2024-02-15. arXiv : 2402.10193. URL : https://arxiv.org/abs/2402.10193 (visité le 2026-10-16).

[31] B. Ping, S. Wang, H. Wang et al. *Delta-CoMe : Training-Free Delta-Compression with Mixed-Precision for Large Language Models*. Version v3 du 2024-11-26. Résumé lu (API arXiv). 2024-06-13. arXiv : 2406.08903. URL : https://arxiv.org/abs/2406.08903 (visité le 2026-10-16).

[32] R. P. N. Rao et D. H. Ballard. « Predictive coding in the visual cortex : a functional interpretation of some extra-classical receptive-field effects ». In : *Nature Neuroscience* 2.1 (1999). Métadonnées relues par l'API Crossref, p. 79-87. DOI : 10.1038/4580. (Visité le 2026-10-16).

[33] S. Wiedemann, H. Kirchhoffer, S. Matlage et al. *DeepCABAC : Context-adaptive binary arithmetic coding for deep neural network compression*. Version v1 du 2019-05-15. Résumé lu (API arXiv). 2019-05-15. arXiv : 1905.08318. URL : https://arxiv.org/abs/1905.08318 (visité le 2026-10-16).

[34] M. Hershcovitch, A. Wood, L. Choshen et al. *ZipNN : Lossless Compression for AI Models*. v2 du 2025-06-04. Code : github.com/zipnn/zipnn, MIT, commit 6009704 du 2026-04-11. 2024. arXiv : 2411.05239. URL : https://arxiv.org/abs/2411.05239 (visité le 2026-10-15).

[35] T. Zhang, M. Hariri, S. Zhong et al. *70% Size, 100% Accuracy : Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11)*. NeurIPS 2025 ; v3 du 2026-01-01. Code : github.com/LeanModels/DFloat11, Apache-2.0, commit 4577338 du 2025-11-24. 2025-04-15. arXiv : 2504.11651. URL : https://arxiv.org/abs/2504.11651 (visité le 2026-10-15).

[36] P. Yubeaton, T. Mahmoud, S. Naga et al. *Huff-LLM : End-to-End Lossless Compression for Efficient LLM Inference*. Aucun code publié trouvé. 2025-02-02. arXiv : 2502.00922. URL : https://arxiv.org/abs/2502.00922 (visité le 2026-10-15).

[37] R. Xie, A. Ul Haq, Y. Fang et al. *TRACE : Unlocking Effective CXL Bandwidth via Lossless Compression and Precision Scaling*. v3 du 2026-01-30. Résumé seul lu : plans de bits par canal codés sans perte, lecture proportionnelle à la précision, dans un contrôleur CXL matériel ; BF16 réduit de 25,2 % sans perte (éditeur, non reproduit). 2025-09-03. arXiv : 2509.03377. (Visité le 2026-10-15).

[38] P. R. Huttenlocher. « Synaptic density in human frontal cortex : Developmental changes and effects of aging ». In : *Brain Research* 163.2 (1979-03). Métadonnées relues par l'API Crossref, p. 195-205. DOI : 10.1016/0006-8993(79)90349-4. (Visité le 2026-10-16).

[39] Y. LeCun, J. S. Denker et S. A. Solla. « Optimal Brain Damage ». In : *Advances in Neural Information Processing Systems 2 (NIPS 1989)*. 1989. URL : https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html (visité le 2026-10-16).

[40] B. Hassibi et D. G. Stork. « Second order derivatives for network pruning : Optimal Brain Surgeon ». In : *Advances in Neural Information Processing Systems 5 (NIPS 1992)*. Page des actes vérifiée (titre) le 2026-10-16. 1992. URL : https://proceedings.neurips.cc/paper/1992/hash/303ed4c69846ab36c2904d3ba8573050-Abstract.html (visité le 2026-10-16).

[41] S. Han, H. Mao et W. J. Dally. *Deep Compression : Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. v5 du 2016-02-15 (la plus récente à la consultation). 2015-10-01. arXiv : 1510.00149. URL : https://arxiv.org/abs/1510.00149 (visité le 2026-10-16).

[42] X. Lu, Q. Liu, Y. Xu et al. *Not All Experts are Equal : Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models*. Version v2 du 2024-05-30. Résumé lu (API arXiv). 2024-02-22. arXiv : 2402.14800. URL : https://arxiv.org/abs/2402.14800 (visité le 2026-10-16).

[43] L. O. Martin, L. Bandarkar et N. Peng. *ESTS at WMT26 : Routing-Informed Expert Pruning for Model Compression*. Version v1 du 2026-09-11. Résumé lu (API arXiv). 2026-09-11. arXiv : 2609.12310. URL : https://arxiv.org/abs/2609.12310 (visité le 2026-10-16).

[44] B. A. Olshausen et D. J. Field. « Emergence of simple-cell receptive field properties by learning a sparse code for natural images ». In : *Nature* 381.6583 (1996-06). Métadonnées Crossref ; résumé de l'éditeur lu., p. 607-609. DOI : 10.1038/381607a0. URL : https://doi.org/10.1038/381607a0 (visité le 2026-10-16).

[45] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko et D. Alistarh. *Extreme Compression of Large Language Models via Additive Quantization*. Première version 2024-01-11, dernière 2024-09-11 (2401.06118v4). ICML, 2024. 2024-01-11. arXiv : 2401.06118. URL : https://arxiv.org/abs/2401.06118 (visité le 2026-10-15).

[46] A. Tseng, J. Chee, Q. Sun, V. Kuleshov et C. D. Sa. *QuIP# : Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks*. Première version 2024-02-06, dernière 2024-06-04 (2402.04396v2). ICML 2024. 2024-02-06. arXiv : 2402.04396. URL : https://arxiv.org/abs/2402.04396 (visité le 2026-10-15).

[47] A. Tseng, Q. Sun, D. Hou et C. De Sa. *QTIP : Quantization with Trellises and Incoherence Processing*. Version 4 du 18 juin 2025 (NeurIPS 2024), sections 2.3 à 3.2 et annexe A.1.1 lues ; erreur 0,069 d'un treillis de 256 dimensions à 2 bits sur une gaussienne, tableau 1 ; 0,0733 (tableau 2, treillis à queue mordue). 2024. arXiv : 2406.11235. URL : https://arxiv.org/abs/2406.11235v4 (visité le 2026-10-16).

[48] Y. Domb, H. Sackstein et T. Solberg. *HyperQuant : A Rate-Distortion-Optimal Quantization Pipeline for Large Language and Diffusion Models*. Version v1 du 2026-06-22. Résumé lu (API arXiv). 2026-06-22. arXiv : 2606.23406. URL : https://arxiv.org/abs/2606.23406 (visité le 2026-10-16).

[49] M. Zhussip, D. Shopkhoev, A. Ali et S. Lefkimmiatis. *Share Your Attention : Transformer Weight Sharing via Matrix-based Dictionary Learning*. Version v2 du 2026-02-19. Résumé lu (API arXiv). 2025-08-06. arXiv : 2508.04581. URL : https://arxiv.org/abs/2508.04581 (visité le 2026-10-16).

[50] E. Frantar, S. P. Singh et D. Alistarh. *Optimal Brain Compression : A Framework for Accurate Post-Training Quantization and Pruning*. Version v2 du 2023-01-08. Résumé lu (API arXiv). 2022-08-24. arXiv : 2208.11580. URL : https://arxiv.org/abs/2208.11580 (visité le 2026-10-16).

[51] E. Frantar, S. Ashkboos, T. Hoefler et D. Alistarh. *GPTQ : Accurate Post-Training Quantization for Generative Pre-trained Transformers*. Version v2 du 2023-03-22. Résumé lu (API arXiv). 2022-10-31. arXiv : 2210.17323. URL : https://arxiv.org/abs/2210.17323 (visité le 2026-10-16).

[52] E. Frantar et D. Alistarh. *QMoE : Practical Sub-1-Bit Compression of Trillion-Parameter Models*. Version v1 du 2023-10-25. Résumé lu (API arXiv). 2023-10-25. arXiv : 2310.16795. URL : https://arxiv.org/abs/2310.16795 (visité le 2026-10-16).

[53] X. Hu, Z. Chen, D. Yang et al. *MoEQuant : Enhancing Quantization for Mixture-of-Experts Large Language Models via Expert-Balanced Sampling and Affinity Guidance*. Version v1 du 2025-05-02. Résumé lu (API arXiv). 2025-05-02. arXiv : 2505.03804. URL : https://arxiv.org/abs/2505.03804 (visité le 2026-10-16).

[54] Z. Fu, T. Zhao, N. Ding et al. *EAQuant : Enhancing Post-Training Quantization for MoE Models via Expert-Aware Optimization*. Version v3 du 2026-02-01. Résumé lu (API arXiv). 2025-06-16. arXiv : 2506.13329. URL : https://arxiv.org/abs/2506.13329 (visité le 2026-10-16).

[55] Y. C. Yoon. « LIF and Simplified SRM Neurons Encode Signals Into Spikes via a Form of Asynchronous Pulse Sigma–Delta Modulation ». In : *IEEE Transactions on Neural Networks and Learning Systems* 28.5 (2017-05). Métadonnées Crossref ; résumé de l'éditeur lu., p. 1192-1205. DOI : 10.1109/tnnls.2016.2526029. URL : https://doi.org/10.1109/tnnls.2016.2526029 (visité le 2026-10-16).

[56] P. Nair, P. Datta, J. Dean, P. Jain et A. Kusupati. *Matryoshka Quantization*. Version v3 du 2025-03-03. Résumé lu (API arXiv). 2025-02-10. arXiv : 2502.06786. URL : https://arxiv.org/abs/2502.06786 (visité le 2026-10-16).

[57] Y. Park, J. Hyun, S. Cho, B. Sim et J. W. Lee. *Any-Precision LLM : Low-Cost Deployment of Multiple, Different-Sized LLMs*. Version v4 du 2024-06-21. Résumé lu (API arXiv). 2024-02-16. arXiv : 2402.10517. URL : https://arxiv.org/abs/2402.10517 (visité le 2026-10-16).

[58] M. Kleinegger, E. Crnčević et D. Alistarh. *MatGPTQ : Efficient and Accurate Inference over Nested Quantized Models*. v2 du 2026-09-30. Résumé seul lu ; code annoncé IST-DASLab/MatGPTQ, licence non vérifiée. 2026-02-03. arXiv : 2602.03537. (Visité le 2026-10-15).

[59] A. Tseng, T. Yu et Y. Park. *Training LLMs with MXFP4*. Version v3 du 2025-08-26. Résumé lu (API arXiv). 2025-02-27. arXiv : 2502.20586. URL : https://arxiv.org/abs/2502.20586 (visité le 2026-10-16).

[60] T.-L. Li, J. Huang, L.-C. Wang et J. R. Gotei. *Scale-QLoRA : Code-Invariant Adapter Merging for Native 4-bit Microscaling LLMs*. Version v1 du 2026-09-03. Résumé lu (API arXiv). 2026-09-03. arXiv : 2609.04526. URL : https://arxiv.org/abs/2609.04526 (visité le 2026-10-16).

[61] M. Grella. *Tied Trit-Planes : Constraining PTQTP to a Uniform Nine-Level Quantizer, with a Persistent Folded Format for Disk-Streamed Mixture-of-Experts Serving*. Première version 2026-08-09, dernière 2026-08-09 (2608.08910v1). 2026-08-09. arXiv : 2608.08910. URL : https://arxiv.org/abs/2608.08910 (visité le 2026-10-15).

[62] S. Ma, H. Wang, L. Ma et al. *The Era of 1-bit LLMs : All Large Language Models are in 1.58 Bits*. Première version 2024-02-27, dernière 2024-02-27 (2402.17764v1). Work in progress. 2024-02-27. arXiv : 2402.17764. URL : https://arxiv.org/abs/2402.17764 (visité le 2026-10-15).

[63] L. M. Adleman. « Molecular Computation of Solutions to Combinatorial Problems ». In : *Science* 266.5187 (1994-11-11). Métadonnées Crossref ; résumé de l'éditeur lu., p. 1021-1024. DOI : 10.1126/science.7973651. URL : https://doi.org/10.1126/science.7973651 (visité le 2026-10-16).

[64] G. M. Church, Y. Gao et S. Kosuri. « Next-Generation Digital Information Storage in DNA ». In : *Science* 337.6102 (2012-09-28). Métadonnées Crossref ; résumé de l'éditeur lu., p. 1628-1628. DOI : 10.1126/science.1226355. URL : https://doi.org/10.1126/science.1226355 (visité le 2026-10-16).

[65] N. Goldman, P. Bertone, S. Chen et al. « Towards practical, high-capacity, low-maintenance information storage in synthesized DNA ». In : *Nature* 494.7435 (2013-01-23). Métadonnées Crossref ; résumé de l'éditeur lu., p. 77-80. DOI : 10.1038/nature11875. URL : https://doi.org/10.1038/nature11875 (visité le 2026-10-16).

[66] Y. Erlich et D. Zielinski. « DNA Fountain enables a robust and efficient storage architecture ». In : *Science* 355.6328 (2017-03-03). Métadonnées Crossref ; résumé de l'éditeur lu., p. 950-954. DOI : 10.1126/science.aaj2038. URL : https://doi.org/10.1126/science.aaj2038 (visité le 2026-10-16).

[67] P. M. Sharp et W.-H. Li. « The codon adaptation index-a measure of directional synonymous codon usage bias, and its potential applications ». In : *Nucleic Acids Research* 15.3 (1987). Métadonnées Crossref ; résumé de l'éditeur lu., p. 1281-1295. DOI : 10.1093/nar/15.3.1281. URL : https://doi.org/10.1093/nar/15.3.1281 (visité le 2026-10-16).

[68] O. Sieberling, D. Kuznedelev, E. Kurtic et D. Alistarh. *EvoPress : Accurate Dynamic Model Compression via Evolutionary Search*. Version v2 du 2025-07-01. Résumé lu (API arXiv). 2024-10-18. arXiv : 2410.14649. URL : https://arxiv.org/abs/2410.14649 (visité le 2026-10-16).

[69] T. Plate. « Holographic reduced representations ». In : *IEEE Transactions on Neural Networks* 6.3 (1995-05). Métadonnées Crossref ; résumé de l'éditeur lu., p. 623-641. DOI : 10.1109/72.377968. URL : https://doi.org/10.1109/72.377968 (visité le 2026-10-16).

[70] T. J. Teyler et P. DiScenna. « The hippocampal memory indexing theory. » In : *Behavioral Neuroscience* 100.2 (1986). Métadonnées Crossref ; résumé de l'éditeur lu., p. 147-154. DOI : 10.1037/0735-7044.100.2.147. URL : https://doi.org/10.1037/0735-7044.100.2.147 (visité le 2026-10-16).

[71] B. Peng, J. Quesnelle, H. Fan et E. Shippole. « YaRN : Efficient Context Window Extension of Large Language Models ». In : *arXiv* (2023-08-31). L'interpolation « NTK par parties » et le facteur de température $\sqrt{1/t} = 0{,}1 \ln s + 1$; cité par l'implémentation d'OpenAI. arXiv : 2309.00071. URL : https://arxiv.org/abs/2309.00071 (visité le 2026-10-15).

[72] *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE. 2019. DOI : 10.1109/IEEESTD.2019.8766229. (Visité le 2026-10-15).

[73] E. B. Wilson. « Probable Inference, the Law of Succession, and Statistical Inference ». In : *Journal of the American Statistical Association* 22.158 (1927). Intervalle de score utilisé pour le taux de premier choix identique (C4)., p. 209-212. DOI : 10.1080/01621459.1927.10502953.

[74] M. Z. Mao et R. M. Gray. « Stationary and Trellis Encoding for IID Sources and Simulation ». In : *2010 Data Compression Conference*. Le treillis « à décalage » et le codeur à permutation aléatoire (RPTC). 2010, p. 3-12. DOI : 10.1109/DCC.2010.8. (Visité le 2026-10-16).

[75] T. R. Fischer, M. W. Marcellin et M. Wang. « Trellis-Coded Vector Quantization ». In : *IEEE Transactions on Information Theory* 37.6 (1991), p. 1551-1566. DOI : 10.1109/18.104316. (Visité le 2026-10-16).

[76] A. J. Viterbi. « Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm ». In : *IEEE Transactions on Information Theory* 13.2 (1967), p. 260-269. DOI : 10.1109/TIT.1967.1054010. (Visité le 2026-10-15).

[77] J. Max. « Quantizing for Minimum Distortion ». In : *IRE Transactions on Information Theory* 6.1 (1960), p. 7-12. DOI : 10.1109/TIT.1960.1057548. (Visité le 2026-10-15).

[78] S. Ohno. *Evolution by Gene Duplication*. Métadonnées relues par l'API Crossref. Berlin, Heidelberg : Springer, 1970. DOI : 10.1007/978-3-642-86659-3. (Visité le 2026-10-16).

[79] S. K. Ainsworth, J. Hayase et S. Srinivasa. *Git Re-Basin : Merging Models modulo Permutation Symmetries*. v3 du 2023-03-01 ; ICLR 2023. 2022-09-11. arXiv : 2209.04836. URL : https://arxiv.org/abs/2209.04836 (visité le 2026-10-16).

[80] R. Entezari, H. Sedghi, O. Saukh et B. Neyshabur. *The Role of Permutation Invariance in Linear Mode Connectivity of Neural Networks*. v2 du 2022-07-05 ; ICLR 2022. 2021-10-12. arXiv : 2110.06296. URL : https://arxiv.org/abs/2110.06296 (visité le 2026-10-16).

[81] H. W. Kuhn. « The Hungarian method for the assignment problem ». In : *Naval Research Logistics Quarterly* 2.1–2 (1955). mise en œuvre utilisée : scipy.optimize.linear_sum_assignment (Crouse 2016), scipy 1.18.1, licence BSD-3, p. 83-97. DOI : 10.1002/nav.3800020109. URL : https://doi.org/10.1002/nav.3800020109 (visité le 2026-10-16).

[82] D. F. Crouse. « On implementing 2D rectangular assignment algorithms ». In : *IEEE Transactions on Aerospace and Electronic Systems* 52.4 (2016), p. 1679-1696. doi : 10.1109/TAES.2016.140952. url : https://doi.org/10.1109/TAES.2016.140952 (visité le 2026-10-16).

[83] G. A. Miller. « Note on the bias of information estimates ». In : *Information Theory in Psychology : Problems and Methods* (1955). chapitre d'ouvrage collectif (éd. H. Quastler), p. 95-100.

[84] J. B. Kruskal. « On the shortest spanning subtree of a graph and the traveling salesman problem ». In : *Proceedings of the American Mathematical Society* 7.1 (1956), p. 48-50. doi : 10.1090/S0002-9939-1956-0078686-7. url : https://doi.org/10.1090/S0002-9939-1956-0078686-7 (visité le 2026-10-16).

[85] Z. Gu, L. M. Steinmetz, X. Gu, C. Scharfe, R. W. Davis et W.-H. Li. « Role of duplicate genes in genetic robustness against null mutations ». In : *Nature* 421 (2003), p. 63-66. doi : 10.1038/nature01198. url : https://doi.org/10.1038/nature01198 (visité le 2026-10-16).

[86] A. Skliar, T. van Rozendaal, R. Lepert, T. Boinovski et al. *Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference*. v2 du 2025-06-24. 2024-11-27. arXiv : 2412.00099. url : https://arxiv.org/abs/2412.00099 (visité le 2026-10-16).

[87] S. Laughlin. « A Simple Coding Procedure Enhances a Neuron's Information Capacity ». In : *Zeitschrift für Naturforschung C* 36.9-10 (1981-10). Codage efficace dans l'œil de la mouche : la précision d'un neurone se répartit selon la loi de ce qu'il code., p. 910-912. doi : 10.1515/znc-1981-9-1040. url : https://doi.org/10.1515/znc-1981-9-1040 (visité le 2026-10-16).

[88] F. H. C. Crick. « Codon–anticodon pairing : The wobble hypothesis ». In : *Journal of Molecular Biology* 19.2 (1966-08). Les codons synonymes : plusieurs mots du code pour un même sens ; l'analogue des deux codes ±0 du E2M1., p. 548-555. doi : 10.1016/S0022-2836(66)80022-0. url : https://doi.org/10.1016/S0022-2836(66)80022-0 (visité le 2026-10-16).

[89] P. A. Chou, T. Lookabaugh et R. M. Gray. « Entropy-constrained vector quantization ». In : *IEEE Transactions on Acoustics, Speech, and Signal Processing* 37.1 (1989-01). Le lagrangien D + λR et la condition d'optimalité des centres ; l'ECSQ en est le cas scalaire., p. 31-42. doi : 10.1109/29.17498. url : https://doi.org/10.1109/29.17498 (visité le 2026-10-16).

[90] N. Farvardin et J. W. Modestino. « Optimum quantizer performance for a class of non-Gaussian memoryless sources ». In : *IEEE Transactions on Information Theory* 30.3 (1984-05). Quantification scalaire contrainte par l'entropie (ECSQ) : à débit élevé, des cellules uniformes et une zone morte., p. 485-497. doi : 10.1109/TIT.1984.1056920. url : https://doi.org/10.1109/TIT.1984.1056920 (visité le 2026-10-16).

[91] D. Muresan et M. Effros. « Quantization as histogram segmentation : globally optimal scalar quantizer design in network systems ». In : *Proceedings DCC 2002, Data Compression Conference*. La quantification comme segmentation d'un histogramme en intervalles, résolue exactement par programmation dynamique : notre geste sur les 16 codes MXFP4. 2002, p. 302-311. doi : 10.1109/dcc.2002.999968. url : https://doi.org/10.1109/dcc.2002.999968 (visité le 2026-10-16).

[92] X. Wu et K. Zhang. « Quantizer monotonicities and globally optimal scalar quantizer design ». In : *IEEE Transactions on Information Theory* 39.3 (1993-05). Optimum global d'un quantificateur scalaire par programmation dynamique sur une source discrète., p. 1049-1053. DOI : 10.1109/18.256513. URL : https://doi.org/10.1109/18.256513 (visité le 2026-10-16).

[93] M. Sun, Z. Liu, A. Bair et J. Z. Kolter. *A Simple and Effective Pruning Approach for Large Language Models*. Version 3. v1 du 2023-06-20, v3 du 2024-05-06. Élagage par |W_ij| · ‖X_j‖ : la sensibilité diagonale par colonne d'entrée. 2023. arXiv : 2306.11695. URL : https://arxiv.org/abs/2306.11695v3 (visité le 2026-10-16).

[94] E. Frantar et D. Alistarh. *SparseGPT : Massive Language Models Can Be Accurately Pruned in One-Shot*. Version 3. v1 du 2023-01-02, v3 du 2023-03-22. Élagage avec compensation par la hessienne des entrées (OBS en ligne). 2023. arXiv : 2301.00774. URL : https://arxiv.org/abs/2301.00774v3 (visité le 2026-10-16).

[95] P. C. O'Brien et T. R. Fleming. « A Multiple Testing Procedure for Clinical Trials ». In : *Biometrics* 35.3 (1979-09). Notice Crossref relue le 2026-10-29 (elle ne donne que la première page) ; article non relu., p. 549. DOI : 10.2307/2530245. URL : https://doi.org/10.2307/2530245 (visité le 2026-10-29).

[96] J. R. Anderson et L. J. Schooler. « Reflections of the Environment in Memory ». In : *Psychological Science* 2.6 (1991). [mém.] non relu pour ce travail, p. 396-408.

[97] K. Hasselmann. « Stochastic climate models. Part I. Theory ». In : *Tellus* 28.6 (1976). [mém.] non relu pour ce travail, p. 473-485.

[98] D. Lee, S. H. Noh, S. L. Min et al. « LRFU : A Spectrum of Policies that Subsumes the Least Recently Used and Least Frequently Used Policies ». In : *IEEE Transactions on Computers* 50.12 (2001). [mém.] non relu pour ce travail, p. 1352-1361.

[99] L. A. Belady. « A Study of Replacement Algorithms for a Virtual-Storage Computer ». In : *IBM Systems Journal* 5.2 (1966), p. 78-101. DOI : 10.1147/sj.52.0078. (Visité le 2026-10-15).

[100] T. M. Wong et J. Wilkes. « My Cache or Yours ? Making Storage More Exclusive ». In : *Proceedings of the 2002 USENIX Annual Technical Conference*. Page de l'éditeur relue le 2026-10-29 (titre, auteurs) ; lue par la piste P2 le 2026-10-27. 2002. URL : https://www.usenix.org/conference/2002-usenix-annual-technical-conference/my-cache-or-yours-making-storage-more-exclusive (visité le 2026-10-29).

[101] R. L. Mattson, J. Gecsei, D. R. Slutz et I. L. Traiger. « Evaluation techniques for storage hierarchies ». In : *IBM Systems Journal* 9.2 (1970). Les algorithmes à pile (le contenu à la capacité C est inclus dans celui à C' > C) : LRU et MIN en sont ; c'est la propriété qui rend l'étage RAM inclusif de trc-cache égal à un cache unique de sa taille (lemme 2 du lot O)., p. 78-117. DOI : 10.1147/sj.92.0078. URL : https://doi.org/10.1147/sj.92.0078 (visité le 2026-10-15).

[102] A. Skliar, T. van Rozendaal, R. Lepert et al. « Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference ». In : *Transactions on Machine Learning Research* (2025). arXiv:2412.00099, version 2 du 24 juin 2025. URL : https://arxiv.org/abs/2412.00099 (visité le 2026-10-27).

[103] A. N. Angelopoulos et S. Bates. *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification*. Version v6 du 2022-12-07 (API arXiv, 2026-10-29) ; résumé lu par la piste P4 le 2026-10-27. 2021-07-15. arXiv : 2107.07511. URL : https://arxiv.org/abs/2107.07511 (visité le 2026-10-29).

[104] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan et L. Lei. *Learn then Test : Calibrating Predictive Algorithms to Achieve Risk Control*. Version v5 du 2022-09-29 (API arXiv, 2026-10-29) ; résumé lu par la piste P4 le 2026-10-27. 2021-10-03. arXiv : 2110.01052. URL : https://arxiv.org/abs/2110.01052 (visité le 2026-10-29).

[105] S. Shalunov, G. Hazel, J. Iyengar et M. Kuehlewind. *Low Extra Delay Background Transport (LEDBAT)*. RFC 6817. Page de la RFC atteinte le 2026-10-29 ; résumé lu par la piste P4 le 2026-10-27. Internet Engineering Task Force, 2012-12. URL : https://www.rfc-editor.org/rfc/rfc6817.html (visité le 2026-10-29).

[106] C. J. Clopper et E. S. Pearson. « The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial ». In : *Biometrika* 26.4 (1934). Notice Crossref relue le 2026-10-29 ; article non relu., p. 404-413. DOI : 10.1093/biomet/26.4.404. URL : https://doi.org/10.1093/biomet/26.4.404 (visité le 2026-10-29).

[107] S. Kirkpatrick, C. D. Gelatt et M. P. Vecchi. « Optimization by Simulated Annealing ». In : *Science* 220.4598 (1983-05-13). Notice Crossref relue le 2026-10-29 ; article non relu., p. 671-680. DOI : 10.1126/science.220.4598.671. URL : https://doi.org/10.1126/science.220.4598.671 (visité le 2026-10-29).

[108] K. Hukushima et K. Nemoto. « Exchange Monte Carlo Method and Application to Spin Glass Simulations ». In : *Journal of the Physical Society of Japan* 65.6 (1996-06-15). Notice Crossref relue le 2026-10-29 ; article non relu., p. 1604-1608. DOI : 10.1143/JPSJ.65.1604. URL : https://doi.org/10.1143/JPSJ.65.1604 (visité le 2026-10-29).

[109] L. Babai. « On Lovász' lattice reduction and the nearest lattice point problem ». In : *Combinatorica* 6.1 (1986). Notice Crossref relue le 2026-10-29 ; article non relu., p. 1-13. DOI : 10.1007/BF02579403. URL : https://doi.org/10.1007/BF02579403 (visité le 2026-10-29).

[110] J. Chen, Y. Shabanzadeh, E. Crnčević, T. Hoefler et D. Alistarh. *The Geometry of LLM Quantization : GPTQ as Babai's Nearest Plane Algorithm*. Version v4 du 2026-05-13 (API arXiv, 2026-10-29), ICLR 2026 ; page lue par la piste P5 le 2026-10-27. 2025-07-24. arXiv : 2507.18553. URL : https://arxiv.org/abs/2507.18553 (visité le 2026-10-29).

[111] B. Aubin, W. Perkins et L. Zdeborová. *Storage capacity in symmetric binary perceptrons*. Version v2 du 2019-03-31 (API arXiv, 2026-10-29) ; J. Phys. A 52, 294003 (2019), référence donnée par la piste P5, non relue ; résumé lu par la piste le 2026-10-27. 2019-01-02. arXiv : 1901.00314. URL : https://arxiv.org/abs/1901.00314 (visité le 2026-10-29).

[112] U. Fincke et M. Pohst. « Improved methods for calculating vectors of short length in a lattice, including a complexity analysis ». In : *Mathematics of Computation* 44.170 (1985). Notice Crossref relue le 2026-10-29 ; article non relu., p. 463-471. DOI : 10.1090/S0025-5718-1985-0777278-8. URL : https://doi.org/10.1090/S0025-5718-1985-0777278-8 (visité le 2026-10-29).

[113] P. Tang et al. *HOBBIT : A Mixed Precision Expert Offloading System for Fast MoE Inference*. dernière version 2024-11-06. 2024-11-03. arXiv : 2411.01433. URL : https://arxiv.org/abs/2411.01433 (visité le 2026-10-15).

[114] A. Haidar, S. Tomov, J. Dongarra et N. J. Higham. « Harnessing GPU Tensor Cores for Fast FP16 Arithmetic to Speed up Mixed-Precision Iterative Refinement Solvers ». In : *SC18 : International Conference for High Performance Computing, Networking, Storage and Analysis*. Notice Crossref relue le 2026-10-29 ; article non relu. 2018-11, p. 603-613. doi : 10.1109/SC.2018.00050. url : https://doi.org/10.1109/SC.2018.00050 (visité le 2026-10-29).

[115] L. Zhang, G. Naitzat et L.-H. Lim. *Tropical Geometry of Deep Neural Networks*. v1 du 2018-05-18 ; résumé lu le 2026-10-17 (05-synthese § 3.1). 2018. arXiv : 1805.07091. url : https://arxiv.org/abs/1805.07091 (visité le 2026-10-17).

[116] ggml-org. *GGUF file format specification (docs/gguf.md)*. Spécification publique lue par l'étude 06. url : https://github.com/ggml-org/ggml/blob/master/docs/gguf.md (visité le 2026-10-15).

[117] Y. Leviathan, M. Kalman et Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023 ; dernière version 2023-05-18. 2022-11-30. arXiv : 2211.17192. url : https://arxiv.org/abs/2211.17192 (visité le 2026-10-15).

[118] C. H. Martin et M. W. Mahoney. *Implicit Self-Regularization in Deep Neural Networks : Evidence from Random Matrix Theory and Implications for Learning*. v1 du 2018-10-02, seule version. 2018. arXiv : 1810.01075 [cs.LG]. url : https://arxiv.org/abs/1810.01075 (visité le 2026-10-21).

[119] J. Duda. *Asymmetric numeral systems : entropy coding combining speed of Huffman coding with compression rate of arithmetic coding*. 2013. arXiv : 1311.2540. url : https://arxiv.org/abs/1311.2540 (visité le 2026-10-15).

[120] F. Giesen. *Interleaved entropy coders*. 2014. arXiv : 1402.3392. url : https://arxiv.org/abs/1402.3392 (visité le 2026-10-15).

[121] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication 180-4. SHA-256, dont le préfixe de 64 bits est l'empreinte64 du format ; la garde de production (M1). 2015-08. doi : 10.6028/NIST.FIPS.180-4. url : https://doi.org/10.6028/NIST.FIPS.180-4 (visité le 2026-10-16).

[122] Microsoft. *FSCTL_GET_RETRIEVAL_POINTERS IOCTL (winioctl.h)*. Microsoft Learn. 2018-12-05. url : https://learn.microsoft.com/en-us/windows/win32/api/winioctl/ni-winioctl-fsctl_get_retrieval_pointers (visité le 2026-10-22).

[123] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Rapp. tech. MXFP4 : blocs de 32 éléments E2M1 et une échelle E8M0 ; non reconsulté par le lot F le 2026-10-15. 2023. url : https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

[124] The ggml authors. *llama.cpp, version b11515*. Version b11515, commit 3d65c90d0. MIT. Lus : `tools/perplexity/perplexity.cpp` (format du fichier `--kl-divergence-base`, codage des log-probabilités sur 16 bits, formule de la KL, quantiles), `src/models/openai-moe.cpp`, `src/llama-graph.cpp` (routage SOFTMAX_WEIGHT), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims` : bornes de la rampe arrondies par `floorf` et `ceilf`), `ggml/src/ggml-cpu/ops.cpp` (`rope_yarn`, `swiglu_oai`), `src/llama-context.cpp` (facteur d'attention de YaRN), `include/llama.h` (API publique). Relus le

2026-10-15 (reprise, par raw.githubusercontent.com à l'étiquette b11515) : `ggml/src/ggml-cpu/ggml-cpu.c` (`type_traits_cpu` : MXFP4 et Q8_0 ont `vec_dot_type` = Q8_0, l'entrée d'un produit quantifié est donc ramenée en Q8_0), `ggml/src/ggml-cpu/arch/x86/quants.c` (`quantize_row_q8_0` AVX2 : d = amax/127 en f16, id = 127/amax, arrondi au plus proche), `ggml/src/ggml-cpu/ops.cpp` (attention éclair du processeur : noyau pavé quand le lot de requêtes est grand, puits appliqués au premier morceau), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims`, revérifié). Rien n'est copié. 2026. URL : https://github.com/ggml-org/llama.cpp (visité le 2026-10-15).

[125] S. Merity, C. Xiong, J. Bradbury et R. Socher. *Pointer Sentinel Mixture Models*. Le corpus WikiText-2 (CC BY-SA 3.0) ; `wiki.test.raw`, empreinte dans les journaux de l'étude 06. 2016-09-26. arXiv : 1609.07843. URL : https://arxiv.org/abs/1609.07843 (visité le 2026-10-15).

[126] A. Rényi. « On the theory of order statistics ». In : *Acta Mathematica Academiae Scientiarum Hungaricae* 4 (1953). Notice bibliographique vérifiée ; article non relu dans ce rôle, p. 191-231. DOI : 10.1007/BF02127580. URL : https://link.springer.com/doi/10.1007/BF02127580 (visité le 2026-10-22).

[127] M. M. Botvinick, T. S. Braver, D. M. Barch, C. S. Carter et J. D. Cohen. « Conflict monitoring and cognitive control ». In : *Psychological Review* 108.3 (2001). Notice vérifiée ; l'ordre des auteurs varie selon les fiches (celle de Princeton : Botvinick, Carter, Braver, Barch, Cohen), à confirmer sur la page de l'éditeur ; article non relu, p. 624-652. URL : https://collaborate.princeton.edu/en/publications/conflict-monitoring-and-cognitive-control-2/ (visité le 2026-10-26).

[128] J. F. Cavanagh, T. V. Wiecki, M. X. Cohen et al. « Subthalamic nucleus stimulation reverses mediofrontal influence over decision threshold ». In : *Nature Neuroscience* 14.11 (2011). Notice vérifiée ; manuscrit d'auteur PMC3394226 ; article non relu dans ce rôle, p. 1462-1467. DOI : 10.1038/nn.2925. URL : https://pmc.ncbi.nlm.nih.gov/articles/PMC3394226 (visité le 2026-10-26).

[129] C. H. Martin, T. (Peng et M. W. Mahoney. *Predicting trends in the quality of state-of-the-art neural networks without access to training or testing data*. v2 du 2021-06-02 ; Nature Communications. 2021. DOI : 10.1038/s41467-021-24025-8. arXiv : 2002.06716 [cs.LG]. URL : https://arxiv.org/abs/2002.06716 (visité le 2026-10-21).

[130] V. A. Marchenko et L. A. Pastur. « Distribution of eigenvalues for some sets of random matrices ». In : *Mathematics of the USSR-Sbornik* 1.4 (1967), p. 457-483. DOI : 10.1070/SM1967v001n04ABEH001994.

[131] A. Clauset, C. R. Shalizi et M. E. J. Newman. « Power-Law Distributions in Empirical Data ». In : *SIAM Review* 51.4 (2009), p. 661-703. DOI : 10.1137/070710111. arXiv : 0706.1062. URL : https://arxiv.org/abs/0706.1062.

[132] Q. H. Vuong. « Likelihood Ratio Tests for Model Selection and Non-Nested Hypotheses ». In : *Econometrica* 57.2 (1989), p. 307-333. DOI : 10.2307/1912557.